

=== ARQUIVO: .lconfig.py ===

```
001 | from __future__ import annotations
002 |
003 | import os
004 | import sys
005 | from pathlib import Path
006 |
007 | from dotenv import load_dotenv
008 |
009 | load_dotenv()
010 |
011 | if getattr(sys, 'frozen', False):
012 |     DATA_DIR: Path = Path(os.getenv("DATA_DIR", str(Path(os.environ.get('LOCALAPPDATA', Path.home()) / 'AppD
013 |     BUNDLE_DIR: Path = Path(getattr(sys, '_MEIPASS', Path(sys.executable).parent))
014 | else:
015 |     DATA_DIR: Path = Path(__file__).parent
016 |     BUNDLE_DIR: Path = Path(__file__).parent
017 |
018 | BASE_DIR: Path = BUNDLE_DIR
019 | DB_PATH: Path = Path(os.getenv("DB_PATH", str(DATA_DIR / "app.db")))
020 |
021 | BACKUP_DIR: Path = DATA_DIR / "backups"
022 | BACKUP_DIR.mkdir(parents=True, exist_ok=True)
023 |
024 | ANEXOS_DIR: Path = DATA_DIR / "anexos"
025 | ANEXOS_DIR.mkdir(parents=True, exist_ok=True)
026 |
027 | SECRET_KEY: str = os.getenv("SECRET_KEY", "change-me-in-production")
028 | DEBUG: bool = os.getenv("DEBUG", "false").lower() == "true"
```

=== ARQUIVO: .ldb.py ===

```
001 | from __future__ import annotations
002 |
003 | import logging
004 | from collections.abc import Iterator
005 | from contextlib import contextmanager
006 | from typing import Any
007 |
008 | from sqlalchemy import create_engine, event
009 | from sqlalchemy.engine import Engine
010 | from sqlalchemy.orm import Session, scoped_session, sessionmaker
011 |
012 | from config import DB_PATH
013 |
014 | log: logging.Logger = logging.getLogger(__name__)
015 |
016 | ENGINE: Engine = create_engine(
017 |     f"sqlite:/// {DB_PATH}",
018 |     echo=False,
019 |     connect_args={"check_same_thread": False},
020 | )
021 |
022 |
023 | @event.listens_for(ENGINE, "connect")
024 | def _set_sqlite_pragma(dbapi_connection: Any, connection_record: Any) -> None:
025 |     cursor = dbapi_connection.cursor()
026 |     cursor.execute("PRAGMA foreign_keys = ON")
027 |     cursor.execute("PRAGMA journal_mode = WAL")
```

```

028 |     cursor.close()
029 |
030 | SessionFactory: sessionmaker = sessionmaker(bind=ENGINE)
031 | ScopedSession: scoped_session = scoped_session(SessionFactory)
032 |
033 | def get_session() -> Session:
034 |     return ScopedSession()
035 |
036 | def close_session(session: Session | None) -> None:
037 |     if session:
038 |         try:
039 |             session.close()
040 |         except Exception as exc:
041 |             log.warning("Erro ao fechar sessão: %s", exc)
042 |
043 |
044 | @contextmanager
045 | def transacao(session: Session) -> Iterator[None]:
046 |     """Context manager for atomic transactions.
047 |
048 |     Service calls inside this block use flush() instead of commit(),
049 |     so the entire block is committed atomically on success,
050 |     or fully rolled back on any error.
051 |     """
052 |     session.info["_em_transacao"] = True
053 |     try:
054 |         yield
055 |         session.commit()
056 |     except Exception:
057 |         session.rollback()
058 |         raise
059 |     finally:
060 |         session.info.pop("_em_transacao", None)
061 |
062 |
063 | def safe_commit(session: Session) -> None:
064 |     """Commits or flushes depending on whether we're inside a transacao() block."""
065 |     try:
066 |         if session.info.get("_em_transacao"):
067 |             session.flush()
068 |         else:
069 |             session.commit()
070 |     except Exception:
071 |         session.rollback()
072 |         raise

```

=== ARQUIVO: .\main.py ===

```

001 | from __future__ import annotations
002 |
003 | import logging
004 | import sqlite3
005 | import sys
006 | import traceback
007 | import types
008 |
009 | from PySide6.QtCore import QEvent, QObject
010 | from PySide6.QtWidgets import (QApplication, QComboBox, QDateEdit,
011 |                                QDateTimeEdit, QMessageBox, QTimeEdit,
012 |                                QStyleFactory)
013 |

```

```

014 | from alembic.command import upgrade as alembic_upgrade
015 | from alembic.config import Config as AlembicConfig
016 | from app.utils.logger import setup_logging
017 | from config import BUNDLE_DIR, DB_PATH
018 | from db import ENGINE, SessionFactory, close_session, get_session
019 |
020 | setup_logging()
021 | log: logging.Logger = logging.getLogger(__name__)
022 | log.info("DB: %s", DB_PATH)
023 |
024 | _alembic_cfg = AlembicConfig(BUNDLE_DIR / "alembic.ini")
025 | _alembic_cfg.set_main_option("sqlalchemy.url", f"sqlite:///{{DB_PATH}}")
026 |
027 | try:
028 |     conn = sqlite3.connect(str(DB_PATH))
029 |     cursor = conn.execute(
030 |         "SELECT name FROM sqlite_master WHERE type='table' AND name='alembic_version'"
031 |     )
032 |     precisa_migrar = cursor.fetchone() is None
033 |     conn.close()
034 |     if precisa_migrar:
035 |         log.info("DB sem alembic_version - aplicando migrations completas")
036 | except Exception:
037 |     log.exception("Falha ao verificar alembic_version")
038 |
039 | alembic_upgrade(_alembic_cfg, "head")
040 |
041 | from app.utils.security import hash_password
042 |
043 |
044 | def _criar_admin_padrao() -> None:
045 |     try:
046 |         from app.models.user import User
047 |         session = SessionFactory()
048 |         existe = session.query(User).first()
049 |         if existe:
050 |             session.close()
051 |             return
052 |         admin = User(
053 |             nome="Administrador",
054 |             email="admin@controleimpressoras.local",
055 |             username="admin",
056 |             senha_hash=hash_password("admin123"),
057 |             perfil="admin",
058 |             ativo=True,
059 |         )
060 |         session.add(admin)
061 |         session.commit()
062 |         log.info("Usuário admin padrão criado (admin / admin123)")
063 |         session.close()
064 |     except Exception:
065 |         log.exception("Falha ao criar admin padrão")
066 |
067 |
068 | _criar_admin_padrao()
069 |
070 | def _excepthook(tipo: type, valor: BaseException, tb: types.TracebackType | None) -> None:
071 |     msg = "".join(traceback.format_exception(tipo, valor, tb))
072 |     log.critical("Exceção não tratada:\n%s", msg)
073 |     try:
074 |         QMessageBox.critical(None, "Erro Inesperado",
075 |             f"Ocorreu um erro inesperado:\n\n{valor}\n\n"
076 |             "Verifique o log para mais detalhes.")

```

```

077 |     except Exception:
078 |         pass
079 |
080 | sys.excepthook = _excepthook
081 |
082 | class _NowheelFilter(QObject):
083 |     _TYPES = (QComboBox, QDateTimeEdit, QTimeEdit)
084 |
085 |     def eventFilter(self, obj: QObject, event: QEvent) -> bool:
086 |         if isinstance(obj, self._TYPES) and event.type() == QEvent.Wheel:
087 |             return True
088 |         return super().eventFilter(obj, event)
089 |
090 | def main() -> None:
091 |     app = QApplication(sys.argv)
092 |     app.installEventFilter(_NowheelFilter(app))
093 |     app.setStyle(QStyleFactory.create("Fusion"))
094 |     app.setApplicationName("Controle de Impressoras Pro")
095 |
096 |     import os
097 |     _theme_path = os.path.join(str(BUNDLE_DIR), "themes", "dark_premium.qss")
098 |     if os.path.exists(_theme_path):
099 |         with open(_theme_path, "r", encoding="utf-8") as _f:
100 |             app.setStyleSheet(_f.read())
101 |
102 |     from app.views.login_dialog import LoginDialog
103 |     login = LoginDialog()
104 |     if login.exec() != LoginDialog.Accepted:
105 |         sys.exit(0)
106 |
107 |     user = login.authenticated_user
108 |
109 |     session = get_session()
110 |     try:
111 |         from app.views.main_window import MainWindow
112 |         window = MainWindow(session, user)
113 |         window.showMaximized()
114 |         exit_code = app.exec()
115 |     finally:
116 |         close_session(session)
117 |
118 |     sys.exit(exit_code)
119 |
120 | if __name__ == "__main__":
121 |     main()

```

=== ARQUIVO: .lalembiclenv.py ===

```

001 | import sys
002 | from logging.config import fileConfig
003 | from pathlib import Path
004 |
005 | from alembic import context
006 | from sqlalchemy import pool
007 |
008 | sys.path.insert(0, str(Path(__file__).parent.parent))
009 |
010 | config = context.config
011 |
012 | if config.config_file_name is not None:
013 |     fileConfig(config.config_file_name)

```

```

014 |
015 | from app.models.base import Base
016 | from db import ENGINE
017 |
018 | target_metadata = Base.metadata
019 |
020 |
021 | def run_migrations_offline() -> None:
022 |     url = config.get_main_option("sqlalchemy.url")
023 |     context.configure(
024 |         url=url,
025 |         target_metadata=target_metadata,
026 |         literal_binds=True,
027 |         dialect_opts={"paramstyle": "named"},
028 |         render_as_batch=True,
029 |     )
030 |     with context.begin_transaction():
031 |         context.run_migrations()
032 |
033 |
034 | def run_migrations_online() -> None:
035 |     with ENGINE.connect() as connection:
036 |         context.configure(
037 |             connection=connection,
038 |             target_metadata=target_metadata,
039 |             render_as_batch=True,
040 |         )
041 |     with context.begin_transaction():
042 |         context.run_migrations()
043 |
044 |
045 | if context.is_offline_mode():
046 |     run_migrations_offline()
047 | else:
048 |     run_migrations_online()

```

=== **ARQUIVO: .alembic\versions\01c2af25a419_add_inicio_fim_atendimento.py**
===

```

001 | """add_inicio_fim_atendimento
002 |
003 | Revision ID: 01c2af25a419
004 | Revises: 370e4ecbc5dc
005 | Create Date: 2026-06-05 18:08:33.665588
006 |
007 | """
008 | from typing import Sequence, Union
009 |
010 | from alembic import op
011 | import sqlalchemy as sa
012 |
013 |
014 | revision: str = '01c2af25a419'
015 | down_revision: Union[str, Sequence[str], None] = '370e4ecbc5dc'
016 | branch_labels: Union[str, Sequence[str], None] = None
017 | depends_on: Union[str, Sequence[str], None] = None
018 |
019 |
020 | def upgrade() -> None:
021 |     with op.batch_alter_table('activities', schema=None) as batch_op:

```

```

022 |         batch_op.add_column(sa.Column('inicio_atendimento', sa.DateTime(), nullable=True))
023 |         batch_op.add_column(sa.Column('fim_atendimento', sa.DateTime(), nullable=True))
024 |
025 |
026 | def downgrade() -> None:
027 |     with op.batch_alter_table('activities', schema=None) as batch_op:
028 |         batch_op.drop_column('inicio_atendimento')
029 |         batch_op.drop_column('fim_atendimento')

```

=== ARQUIVO:

`.alembic\versions\0df5fd127c31_add_notificado_fields_to_alert.py` ===

```

001 | """add_notificado_fields_to_alert
002 |
003 | Revision ID: 0df5fd127c31
004 | Revises: 43f9d2337797
005 | Create Date: 2026-06-02 16:47:18.668008
006 |
007 | """
008 | from typing import Sequence, Union
009 |
010 | from alembic import op
011 | import sqlalchemy as sa
012 |
013 |
014 | revision: str = '0df5fd127c31'
015 | down_revision: Union[str, None] = '43f9d2337797'
016 | branch_labels: Union[str, Sequence[str], None] = None
017 | depends_on: Union[str, Sequence[str], None] = None
018 |
019 |
020 | def upgrade() -> None:
021 |     with op.batch_alter_table('alerts') as batch_op:
022 |         batch_op.add_column(sa.Column('notificado', sa.Boolean(), nullable=False, server_default=sa.text('0'))
023 |         batch_op.add_column(sa.Column('notificado_em', sa.DateTime(), nullable=True))
024 |
025 |
026 | def downgrade() -> None:
027 |     with op.batch_alter_table('alerts') as batch_op:
028 |         batch_op.drop_column('notificado_em')
029 |         batch_op.drop_column('notificado')

```

=== ARQUIVO:

`.alembic\versions\1a2b3c4d5e6f_add_ultima_revisao_to_printers.py` ===

```

001 | """add ultima_revisao to printers
002 |
003 | Revision ID: 1a2b3c4d5e6f
004 | Revises: 01c2af25a419
005 | Create Date: 2026-06-09 18:15:00.000000
006 |
007 | """
008 | from typing import Sequence, Union
009 |
010 | from alembic import op
011 | import sqlalchemy as sa
012 |

```

```

013 |
014 | revision: str = '1a2b3c4d5e6f'
015 | down_revision: Union[str, Sequence[str], None] = '01c2af25a419'
016 | branch_labels: Union[str, Sequence[str], None] = None
017 | depends_on: Union[str, Sequence[str], None] = None
018 |
019 |
020 | def upgrade() -> None:
021 |     with op.batch_alter_table('printers', schema=None) as batch_op:
022 |         batch_op.add_column(sa.Column('ultima_revisao', sa.DateTime(), nullable=True))
023 |
024 |
025 | def downgrade() -> None:
026 |     with op.batch_alter_table('printers', schema=None) as batch_op:
027 |         batch_op.drop_column('ultima_revisao')

```

=== ARQUIVO: .\alembic\versions\2b3c4d5e6f7a_add_urgencia_to_printers.py ===

```

001 | """add urgencia_prox_manutencao to printers
002 |
003 | Revision ID: 2b3c4d5e6f7a
004 | Revises: 1a2b3c4d5e6f
005 | Create Date: 2026-06-09 18:30:00.000000
006 |
007 | """
008 | from typing import Sequence, Union
009 |
010 | from alembic import op
011 | import sqlalchemy as sa
012 |
013 |
014 | revision: str = '2b3c4d5e6f7a'
015 | down_revision: Union[str, Sequence[str], None] = '1a2b3c4d5e6f'
016 | branch_labels: Union[str, Sequence[str], None] = None
017 | depends_on: Union[str, Sequence[str], None] = None
018 |
019 |
020 | def upgrade() -> None:
021 |     with op.batch_alter_table('printers', schema=None) as batch_op:
022 |         batch_op.add_column(sa.Column('urgencia_prox_manutencao', sa.String(20), nullable=False, server_def
023 |
024 |
025 | def downgrade() -> None:
026 |     with op.batch_alter_table('printers', schema=None) as batch_op:
027 |         batch_op.drop_column('urgencia_prox_manutencao')

```

=== ARQUIVO: .\alembic\versions\370e4ecbc5dc_add_diagnostico_solucao_to_activities.py ===

```

001 | """add_diagnostico_solucao_to_activities
002 |
003 | Revision ID: 370e4ecbc5dc
004 | Revises: e2ea9a0e5059
005 | Create Date: 2026-06-05 18:04:38.775529
006 |
007 | """

```

```

008 | from typing import Sequence, Union
009 |
010 | from alembic import op
011 | import sqlalchemy as sa
012 |
013 |
014 | revision: str = '370e4ecbc5dc'
015 | down_revision: Union[str, Sequence[str], None] = 'e2ea9a0e5059'
016 | branch_labels: Union[str, Sequence[str], None] = None
017 | depends_on: Union[str, Sequence[str], None] = None
018 |
019 |
020 | def upgrade() -> None:
021 |     with op.batch_alter_table('activities', schema=None) as batch_op:
022 |         batch_op.add_column(sa.Column('diagnostico_tecnico', sa.Text(), nullable=False, server_default=""))
023 |         batch_op.add_column(sa.Column('solucao_aplicada', sa.Text(), nullable=False, server_default=""))
024 |
025 |
026 | def downgrade() -> None:
027 |     with op.batch_alter_table('activities', schema=None) as batch_op:
028 |         batch_op.drop_column('diagnostico_tecnico')
029 |         batch_op.drop_column('solucao_aplicada')

```

=== ARQUIVO: [.alembic\versions\3c4d5e6f7a8b_add_urgencia_to_activities.py](#)

===

```

001 | """add urgencia to activities
002 |
003 | Revision ID: 3c4d5e6f7a8b
004 | Revises: 2b3c4d5e6f7a
005 | Create Date: 2026-06-09 19:00:00.000000
006 |
007 | """
008 | from typing import Sequence, Union
009 |
010 | from alembic import op
011 | import sqlalchemy as sa
012 |
013 |
014 | revision: str = '3c4d5e6f7a8b'
015 | down_revision: Union[str, Sequence[str], None] = '2b3c4d5e6f7a'
016 | branch_labels: Union[str, Sequence[str], None] = None
017 | depends_on: Union[str, Sequence[str], None] = None
018 |
019 |
020 | def upgrade() -> None:
021 |     with op.batch_alter_table('activities', schema=None) as batch_op:
022 |         batch_op.add_column(sa.Column('urgencia', sa.String(20), nullable=False, server_default='Normal'))
023 |
024 |
025 | def downgrade() -> None:
026 |     with op.batch_alter_table('activities', schema=None) as batch_op:
027 |         batch_op.drop_column('urgencia')

```

=== ARQUIVO:

[.alembic\versions\3df1b9c20c20_add_procedimentos_to_activities.py](#) ===


```

001 | """add_procedimentos_to_activities
002 |
003 | Revision ID: 3df1b9c20c20
004 | Revises: 4113ad29b4c6
005 | Create Date: 2026-06-05 17:15:07.351949
006 |
007 | """
008 | from typing import Sequence, Union
009 |
010 | from alembic import op
011 | import sqlalchemy as sa
012 |
013 |
014 | # revision identifiers, used by Alembic.
015 | revision: str = '3df1b9c20c20'
016 | down_revision: Union[str, Sequence[str], None] = '4113ad29b4c6'
017 | branch_labels: Union[str, Sequence[str], None] = None
018 | depends_on: Union[str, Sequence[str], None] = None
019 |
020 |
021 | def upgrade() -> None:
022 |     with op.batch_alter_table('activities', schema=None) as batch_op:
023 |         batch_op.add_column(sa.Column('procedimentos', sa.Text(), nullable=False, server_default=''))
024 |
025 |
026 | def downgrade() -> None:
027 |     with op.batch_alter_table('activities', schema=None) as batch_op:
028 |         batch_op.drop_column('procedimentos')

```

=== ARQUIVO:

[.alembic\versions\4113ad29b4c6_add_responsavel_to_activities.py](#) ===

```

001 | """add_responsavel_to_activities
002 |
003 | Revision ID: 4113ad29b4c6
004 | Revises: b2c3d4e5f6a7
005 | Create Date: 2026-06-05 16:20:56.050336
006 |
007 | """
008 | from typing import Sequence, Union
009 |
010 | from alembic import op
011 | import sqlalchemy as sa
012 |
013 |
014 | # revision identifiers, used by Alembic.
015 | revision: str = '4113ad29b4c6'
016 | down_revision: Union[str, Sequence[str], None] = 'b2c3d4e5f6a7'
017 | branch_labels: Union[str, Sequence[str], None] = None
018 | depends_on: Union[str, Sequence[str], None] = None
019 |
020 |
021 | def upgrade() -> None:
022 |     with op.batch_alter_table('activities', schema=None) as batch_op:
023 |         batch_op.add_column(sa.Column('responsavel', sa.String(length=120), nullable=False, server_default=''))
024 |
025 |
026 | def downgrade() -> None:
027 |     with op.batch_alter_table('activities', schema=None) as batch_op:
028 |         batch_op.drop_column('responsavel')

```

=== ARQUIVO: .lalembic\versions\43f9d2337797_add_deleted_at_soft_delete.py ===

```
001 | """add deleted_at columns for soft delete
002 |
003 | Revision ID: 43f9d2337797
004 | Revises: 860330a5ed5a
005 | Create Date: 2026-06-02 16:02:04.880542
006 |
007 | """
008 | from typing import Sequence, Union
009 |
010 | from alembic import op
011 | import sqlalchemy as sa
012 |
013 |
014 | revision: str = '43f9d2337797'
015 | down_revision: Union[str, Sequence[str], None] = '860330a5ed5a'
016 | branch_labels: Union[str, Sequence[str], None] = None
017 | depends_on: Union[str, Sequence[str], None] = None
018 |
019 |
020 | TABELAS = ["printers", "activities", "companies", "parts", "technicians", "alerts", "transfers", "users"]
021 |
022 |
023 | def upgrade() -> None:
024 |     for tabela in TABELAS:
025 |         with op.batch_alter_table(tabela, schema=None) as batch_op:
026 |             batch_op.add_column(sa.Column("deleted_at", sa.DateTime(), nullable=True))
027 |
028 |
029 | def downgrade() -> None:
030 |     for tabela in TABELAS:
031 |         with op.batch_alter_table(tabela, schema=None) as batch_op:
032 |             batch_op.drop_column("deleted_at")
```

=== ARQUIVO: .lalembic\versions\7899a6741eeb_add_part_movements_reservations_requisitions.py ===

```
001 | """add part_movements, part_reservations, purchase_requisitions
002 |
003 | Revision ID: 7899a6741eeb
004 | Revises: f6a7b8c9d0e1
005 | Create Date: 2026-06-15 14:00:00.000000
006 |
007 | """
008 | from typing import Sequence, Union
009 |
010 | from alembic import op
011 | import sqlalchemy as sa
012 |
013 |
014 | revision: str = '7899a6741eeb'
015 | down_revision: Union[str, Sequence[str], None] = 'f6a7b8c9d0e1'
016 | branch_labels: Union[str, Sequence[str], None] = None
017 | depends_on: Union[str, Sequence[str], None] = None
```

```

018 |
019 |
020 | def upgrade() -> None:
021 |     op.create_table('part_movements',
022 |         sa.Column('id', sa.Integer(), autoincrement=True, nullable=False),
023 |         sa.Column('part_id', sa.Integer(), sa.ForeignKey('parts.id'), nullable=False, index=True),
024 |         sa.Column('activity_id', sa.Integer(), sa.ForeignKey('activities.id'), nullable=True, index=True),
025 |         sa.Column('tipo', sa.String(length=30), nullable=False),
026 |         sa.Column('quantidade', sa.Integer(), nullable=False),
027 |         sa.Column('saldo_anterior', sa.Integer(), nullable=False, server_default='0'),
028 |         sa.Column('saldo_posterior', sa.Integer(), nullable=False, server_default='0'),
029 |         sa.Column('observacao', sa.Text(), nullable=False, server_default=''),
030 |         sa.Column('created_at', sa.DateTime(), nullable=True),
031 |         sa.PrimaryKeyConstraint('id'),
032 |     )
033 |
034 |     op.create_table('part_reservations',
035 |         sa.Column('id', sa.Integer(), autoincrement=True, nullable=False),
036 |         sa.Column('part_id', sa.Integer(), sa.ForeignKey('parts.id'), nullable=False, index=True),
037 |         sa.Column('activity_id', sa.Integer(), sa.ForeignKey('activities.id'), nullable=False, index=True),
038 |         sa.Column('quantidade', sa.Integer(), nullable=False, server_default='1'),
039 |         sa.Column('status', sa.String(length=20), nullable=False, server_default='reservada'),
040 |         sa.Column('created_at', sa.DateTime(), nullable=True),
041 |         sa.Column('updated_at', sa.DateTime(), nullable=True),
042 |         sa.PrimaryKeyConstraint('id'),
043 |     )
044 |
045 |     op.create_table('purchase_requisitions',
046 |         sa.Column('id', sa.Integer(), autoincrement=True, nullable=False),
047 |         sa.Column('part_id', sa.Integer(), sa.ForeignKey('parts.id'), nullable=False, index=True),
048 |         sa.Column('quantidade_sugerida', sa.Integer(), nullable=False, server_default='1'),
049 |         sa.Column('status', sa.String(length=20), nullable=False, server_default='pendente'),
050 |         sa.Column('observacao', sa.Text(), nullable=False, server_default=''),
051 |         sa.Column('created_at', sa.DateTime(), nullable=True),
052 |         sa.Column('updated_at', sa.DateTime(), nullable=True),
053 |         sa.PrimaryKeyConstraint('id'),
054 |     )
055 |
056 |
057 | def downgrade() -> None:
058 |     op.drop_table('purchase_requisitions')
059 |     op.drop_table('part_reservations')
060 |     op.drop_table('part_movements')

```

=== ARQUIVO: .\alembic\versions\860330a5ed5a_sync_models.py ===

```

001 | """sync models with DB
002 |
003 | Revision ID: 860330a5ed5a
004 | Revises: d46315348779
005 | Create Date: 2026-06-01 11:24:43.623550
006 |
007 | """
008 | from typing import Sequence, Union
009 |
010 | from alembic import op
011 | import sqlalchemy as sa
012 |
013 |
014 | revision: str = '860330a5ed5a'
015 | down_revision: Union[str, Sequence[str], None] = 'd46315348779'

```



```

079 | with op.batch_alter_table('printers', schema=None) as batch_op:
080 |     batch_op.add_column(sa.Column('ultima_revisao_data', sa.DATETIME(), nullable=True))
081 |     batch_op.add_column(sa.Column('tecnico_revisao', sa.TEXT(), nullable=True))
082 |     batch_op.drop_constraint('uq_printers_patrimonio', type_='unique')
083 |     batch_op.alter_column('patrimonio',
084 |         existing_type=sa.String(length=80),
085 |         type_=sa.VARCHAR(length=50),
086 |         existing_nullable=False)
087 |
088 | with op.batch_alter_table('users', schema=None) as batch_op:
089 |     batch_op.drop_constraint('uq_users_email', type_='unique')
090 |     batch_op.add_column(sa.Column('empresa_id', sa.INTEGER(), nullable=True))
091 |
092 | op.drop_index('ix_alerts_printer_id', table_name='alerts')
093 | op.drop_index('ix_user_username', table_name='users')
094 | op.create_index('ix_users_username', 'users', ['username'], unique=False)
095 | op.drop_index('ix_transfer_tipo', table_name='transfers')
096 | op.create_index('ix_transfers_tipo', 'transfers', ['tipo'], unique=False)
097 | op.drop_index('ix_transfer_numero_os', table_name='transfers')
098 | op.create_index('ix_transfers_numero_os', 'transfers', ['numero_os'], unique=False)
099 | op.drop_index('ix_company_cnpj', table_name='companies')
100 | op.create_index('ix_companies_cnpj', 'companies', ['cnpj'], unique=False)
101 | op.drop_index('ix_part_codigo', table_name='parts')
102 | op.create_index('ix_parts_codigo', 'parts', ['codigo'], unique=False)
103 |
104 | op.execute("PRAGMA foreign_keys=ON")

```

=== ARQUIVO:

.\alembic\versions\8a7b6c5d4e3f_add_activity_id_to_printer_locations.py ===

```

001 | """Add activity_id to printer_locations
002 |
003 | Revision ID: 8a7b6c5d4e3f
004 | Revises: 7899a6741eeb
005 | Create Date: 2026-07-01 12:00:00.000000
006 |
007 | """
008 | from typing import Sequence, Union
009 |
010 | from alembic import op
011 | import sqlalchemy as sa
012 |
013 |
014 | revision: str = '8a7b6c5d4e3f'
015 | down_revision: Union[str, None] = '7899a6741eeb'
016 | branch_labels: Union[str, Sequence[str], None] = None
017 | depends_on: Union[str, Sequence[str], None] = None
018 |
019 |
020 | def upgrade() -> None:
021 |     with op.batch_alter_table('printer_locations', schema=None) as batch_op:
022 |         batch_op.add_column(sa.Column('activity_id', sa.Integer(), nullable=True))
023 |         batch_op.create_foreign_key('fk_printer_locations_activity', 'activities', ['activity_id'], ['id'])
024 |
025 |
026 | def downgrade() -> None:
027 |     with op.batch_alter_table('printer_locations', schema=None) as batch_op:
028 |         batch_op.drop_constraint('fk_printer_locations_activity', type_='foreignkey')
029 |         batch_op.drop_column('activity_id')

```

=== ARQUIVO:

.alembic\versions\9a3b1c2d4e5f_add_part_id_data_agendada_to_alerts.py ===

```
001 | """add_part_id_data_agendada_to_alerts
002 |
003 | Revision ID: 9a3b1c2d4e5f
004 | Revises: 0df5fd127c31
005 | Create Date: 2026-06-03 11:30:00.000000
006 |
007 | """
008 | from typing import Sequence, Union
009 |
010 | from alembic import op
011 | import sqlalchemy as sa
012 |
013 |
014 | revision: str = '9a3b1c2d4e5f'
015 | down_revision: Union[str, None] = '0df5fd127c31'
016 | branch_labels: Union[str, Sequence[str], None] = None
017 | depends_on: Union[str, Sequence[str], None] = None
018 |
019 |
020 | def upgrade() -> None:
021 |     with op.batch_alter_table('alerts') as batch_op:
022 |         batch_op.alter_column('printer_id', existing_type=sa.String(36), nullable=True)
023 |         batch_op.add_column(sa.Column('part_id', sa.Integer(), nullable=True))
024 |         batch_op.add_column(sa.Column('data_agendada', sa.DateTime(), nullable=True))
025 |         batch_op.create_index('ix_alert_part_id', ['part_id'])
026 |         batch_op.create_foreign_key('fk_alert_part_id', 'parts', ['part_id'], ['id'])
027 |
028 |
029 | def downgrade() -> None:
030 |     with op.batch_alter_table('alerts') as batch_op:
031 |         batch_op.drop_constraint('fk_alert_part_id', type_='foreignkey')
032 |         batch_op.drop_index('ix_alert_part_id')
033 |         batch_op.drop_column('data_agendada')
034 |         batch_op.drop_column('part_id')
035 |         batch_op.alter_column('printer_id', existing_type=sa.String(36), existing_nullable=True, nullable=False)
```

=== ARQUIVO: .alembic\versions\1b2c3d4e5f6_add_foto_path_to_printers.py

===

```
001 | """add_foto_path_to_printers
002 |
003 | Revision ID: 1b2c3d4e5f6
004 | Revises: 9a3b1c2d4e5f
005 | Create Date: 2026-06-04 10:30:00.000000
006 |
007 | """
008 | from typing import Sequence, Union
009 |
010 | from alembic import op
011 | import sqlalchemy as sa
012 |
013 |
014 | revision: str = '1b2c3d4e5f6'
015 | down_revision: Union[str, None] = '9a3b1c2d4e5f'
016 | branch_labels: Union[str, Sequence[str], None] = None
017 | depends_on: Union[str, Sequence[str], None] = None
```

```

018 |
019 |
020 | def upgrade() -> None:
021 |     with op.batch_alter_table('printers') as batch_op:
022 |         batch_op.add_column(sa.Column('foto_path', sa.String(255), nullable=True))
023 |
024 |
025 | def downgrade() -> None:
026 |     with op.batch_alter_table('printers') as batch_op:
027 |         batch_op.drop_column('foto_path')

```

=== ARQUIVO: .lalembic\versions\la44fc2747c28_add_performance_indexes.py ===

```

001 | """add performance indexes
002 |
003 | Revision ID: a44fc2747c28
004 | Revises: 8a7b6c5d4e3f
005 | Create Date: 2026-07-10 12:00:00.000000
006 |
007 | """
008 | from typing import Sequence, Union
009 |
010 | from alembic import op
011 | import sqlalchemy as sa
012 |
013 |
014 | revision: str = 'a44fc2747c28'
015 | down_revision: Union[str, Sequence[str], None] = '8a7b6c5d4e3f'
016 | branch_labels: Union[str, Sequence[str], None] = None
017 | depends_on: Union[str, Sequence[str], None] = None
018 |
019 |
020 | def upgrade() -> None:
021 |     # --- printers ---
022 |     with op.batch_alter_table("printers", schema=None) as batch_op:
023 |         batch_op.create_index("ix_printers_status", ["status"])
024 |         batch_op.create_index("ix_printers_deleted_at", ["deleted_at"])
025 |
026 |     # --- activities ---
027 |     with op.batch_alter_table("activities", schema=None) as batch_op:
028 |         batch_op.create_index("ix_activities_deleted_at", ["deleted_at"])
029 |         batch_op.create_index("ix_activities_event_at", ["event_at"])
030 |         batch_op.create_index("ix_activities_tecnico_id", ["tecnico_id"])
031 |         batch_op.create_index("ix_activities_tecnico_id_event_at", ["tecnico_id", "event_at"])
032 |         batch_op.create_index("ix_activities_printer_id_event_at", ["printer_id", "event_at"])
033 |         batch_op.create_index("ix_activities_status_event_at", ["status_atividade", "event_at"])
034 |
035 |     # --- alerts ---
036 |     with op.batch_alter_table("alerts", schema=None) as batch_op:
037 |         batch_op.create_index("ix_alerts_deleted_at", ["deleted_at"])
038 |         batch_op.create_index("ix_alerts_resolvido", ["resolvido"])
039 |         batch_op.create_index("ix_alerts_resolvido_created_at", ["resolvido", "created_at"])
040 |         batch_op.create_index("ix_alerts_tipo_resolvido", ["tipo", "resolvido"])
041 |
042 |     # --- companies ---
043 |     with op.batch_alter_table("companies", schema=None) as batch_op:
044 |         batch_op.create_index("ix_companies_deleted_at", ["deleted_at"])
045 |
046 |     # --- parts ---

```

```

047 | with op.batch_alter_table("parts", schema=None) as batch_op:
048 |     batch_op.create_index("ix_parts_deleted_at", ["deleted_at"])
049 |     batch_op.create_index("ix_parts_nome", ["nome"])
050 |
051 | # --- technicians ---
052 | with op.batch_alter_table("technicians", schema=None) as batch_op:
053 |     batch_op.create_index("ix_technicians_deleted_at", ["deleted_at"])
054 |
055 | # --- users ---
056 | with op.batch_alter_table("users", schema=None) as batch_op:
057 |     batch_op.create_index("ix_users_deleted_at", ["deleted_at"])
058 |
059 | # --- FK columns sem indice ---
060 | with op.batch_alter_table("login_history", schema=None) as batch_op:
061 |     batch_op.create_index("ix_login_history_user_id", ["user_id"])
062 |
063 | with op.batch_alter_table("technician_specialties", schema=None) as batch_op:
064 |     batch_op.create_index("ix_technician_specialties_technician_id", ["technician_id"])
065 |
066 | with op.batch_alter_table("printer_locations", schema=None) as batch_op:
067 |     batch_op.create_index("ix_printer_locations_activity_id", ["activity_id"])
068 |
069 | with op.batch_alter_table("attachments", schema=None) as batch_op:
070 |     batch_op.create_index("ix_attachments_uploader_id", ["uploader_id"])
071 |
072 | with op.batch_alter_table("audit_log", schema=None) as batch_op:
073 |     batch_op.create_index("ix_audit_log_user_id", ["user_id"])
074 |     batch_op.create_index("ix_audit_log_created_at", ["created_at"])
075 |     batch_op.create_index("ix_audit_log_tabela_alvo", ["tabela_alvo"])
076 |
077 |
078 | def downgrade() -> None:
079 |     with op.batch_alter_table("printers", schema=None) as batch_op:
080 |         batch_op.drop_index("ix_printers_status")
081 |         batch_op.drop_index("ix_printers_deleted_at")
082 |
083 |     with op.batch_alter_table("activities", schema=None) as batch_op:
084 |         batch_op.drop_index("ix_activities_deleted_at")
085 |         batch_op.drop_index("ix_activities_event_at")
086 |         batch_op.drop_index("ix_activities_tecnico_id")
087 |         batch_op.drop_index("ix_activities_tecnico_id_event_at")
088 |         batch_op.drop_index("ix_activities_printer_id_event_at")
089 |         batch_op.drop_index("ix_activities_status_event_at")
090 |
091 |     with op.batch_alter_table("alerts", schema=None) as batch_op:
092 |         batch_op.drop_index("ix_alerts_deleted_at")
093 |         batch_op.drop_index("ix_alerts_resolvido")
094 |         batch_op.drop_index("ix_alerts_resolvido_created_at")
095 |         batch_op.drop_index("ix_alerts_tipo_resolvido")
096 |
097 |     with op.batch_alter_table("companies", schema=None) as batch_op:
098 |         batch_op.drop_index("ix_companies_deleted_at")
099 |
100 |     with op.batch_alter_table("parts", schema=None) as batch_op:
101 |         batch_op.drop_index("ix_parts_deleted_at")
102 |         batch_op.drop_index("ix_parts_nome")
103 |
104 |     with op.batch_alter_table("technicians", schema=None) as batch_op:
105 |         batch_op.drop_index("ix_technicians_deleted_at")
106 |
107 |     with op.batch_alter_table("users", schema=None) as batch_op:
108 |         batch_op.drop_index("ix_users_deleted_at")
109 |

```



```

041 |         nullable=True,
042 |         existing_server_default=sa.text('(CURRENT_TIMESTAMP)'))
043 |     batch_op.alter_column('ativo',
044 |         existing_type=sa.BOOLEAN(),
045 |         nullable=True,
046 |         existing_server_default=sa.text('1'))
047 |     batch_op.alter_column('perfil',
048 |         existing_type=sa.VARCHAR(length=20),
049 |         nullable=True,
050 |         existing_server_default=sa.text("'visualizador'"))
051 |     batch_op.alter_column('username',
052 |         existing_type=sa.VARCHAR(length=50),
053 |         nullable=True)
054 |     batch_op.alter_column('id',
055 |         existing_type=sa.INTEGER(),
056 |         nullable=True,
057 |         autoincrement=True)
058 |
059 | with op.batch_alter_table('transfers', schema=None) as batch_op:
060 |     batch_op.drop_constraint('fk_transfers_to_company', type_='foreignkey')
061 |     batch_op.drop_constraint('fk_transfers_printer', type_='foreignkey')
062 |     batch_op.drop_constraint('fk_transfers_from_company', type_='foreignkey')
063 |     batch_op.alter_column('created_at',
064 |         existing_type=sa.DATETIME(),
065 |         nullable=True,
066 |         existing_server_default=sa.text('(CURRENT_TIMESTAMP)'))
067 |     batch_op.alter_column('observacao',
068 |         existing_type=sa.TEXT(),
069 |         nullable=True,
070 |         existing_server_default=sa.text("(')'))
071 |     batch_op.alter_column('responsavel_recebimento',
072 |         existing_type=sa.VARCHAR(length=120),
073 |         nullable=True)
074 |     batch_op.alter_column('responsavel_entrega',
075 |         existing_type=sa.VARCHAR(length=120),
076 |         nullable=True)
077 |     batch_op.alter_column('tipo',
078 |         existing_type=sa.VARCHAR(length=20),
079 |         nullable=True)
080 |     batch_op.alter_column('numero_os',
081 |         existing_type=sa.VARCHAR(length=50),
082 |         nullable=True)
083 |     batch_op.alter_column('printer_id',
084 |         existing_type=sa.VARCHAR(length=36),
085 |         nullable=True)
086 |     batch_op.alter_column('id',
087 |         existing_type=sa.INTEGER(),
088 |         nullable=True,
089 |         autoincrement=True)
090 |
091 | with op.batch_alter_table('printers', schema=None) as batch_op:
092 |     batch_op.drop_constraint('fk_printers_company', type_='foreignkey')
093 |     batch_op.alter_column('tecnico',
094 |         existing_type=sa.VARCHAR(length=120),
095 |         nullable=True,
096 |         existing_server_default=sa.text("(')'))
097 |     batch_op.alter_column('mac_address',
098 |         existing_type=sa.VARCHAR(length=17),
099 |         nullable=True,
100 |         existing_server_default=sa.text("(')'))
101 |     batch_op.alter_column('ip_rede',
102 |         existing_type=sa.VARCHAR(length=45),
103 |         nullable=True,

```

```

104 |         existing_server_default=sa.text("('')"))
105 |     batch_op.alter_column('tipo',
106 |         existing_type=sa.VARCHAR(length=20),
107 |         nullable=True,
108 |         existing_server_default=sa.text("('')"))
109 |     batch_op.alter_column('marca',
110 |         existing_type=sa.VARCHAR(length=80),
111 |         nullable=True,
112 |         existing_server_default=sa.text("('')"))
113 |
114 |     with op.batch_alter_table('parts', schema=None) as batch_op:
115 |         batch_op.alter_column('modelo_compativel',
116 |             existing_type=sa.VARCHAR(length=200),
117 |             nullable=True,
118 |             existing_server_default=sa.text("('')"))
119 |
120 |     with op.batch_alter_table('companies', schema=None) as batch_op:
121 |         batch_op.alter_column('created_at',
122 |             existing_type=sa.DATETIME(),
123 |             nullable=True,
124 |             existing_server_default=sa.text('(CURRENT_TIMESTAMP)'))
125 |         batch_op.alter_column('tipo',
126 |             existing_type=sa.VARCHAR(length=20),
127 |             nullable=True,
128 |             existing_server_default=sa.text("'cliente'"))
129 |         batch_op.alter_column('email',
130 |             existing_type=sa.VARCHAR(length=120),
131 |             nullable=True)
132 |         batch_op.alter_column('telefone',
133 |             existing_type=sa.VARCHAR(length=20),
134 |             nullable=True)
135 |         batch_op.alter_column('uf',
136 |             existing_type=sa.VARCHAR(length=2),
137 |             nullable=True)
138 |         batch_op.alter_column('cidade',
139 |             existing_type=sa.VARCHAR(length=100),
140 |             nullable=True)
141 |         batch_op.alter_column('endereco',
142 |             existing_type=sa.VARCHAR(length=255),
143 |             nullable=True)
144 |         batch_op.alter_column('cnpj',
145 |             existing_type=sa.VARCHAR(length=18),
146 |             nullable=True)
147 |         batch_op.alter_column('id',
148 |             existing_type=sa.INTEGER(),
149 |             nullable=True,
150 |             autoincrement=True)
151 |
152 |     with op.batch_alter_table('attachments', schema=None) as batch_op:
153 |         batch_op.drop_constraint('fk_attachments_uploader', type_='foreignkey')
154 |         batch_op.alter_column('created_at',
155 |             existing_type=sa.DATETIME(),
156 |             nullable=True,
157 |             existing_server_default=sa.text('(CURRENT_TIMESTAMP)'))
158 |         batch_op.alter_column('categoria',
159 |             existing_type=sa.VARCHAR(length=50),
160 |             nullable=True)
161 |         batch_op.alter_column('size_bytes',
162 |             existing_type=sa.INTEGER(),
163 |             nullable=True)
164 |         batch_op.alter_column('mime_type',
165 |             existing_type=sa.VARCHAR(length=100),
166 |             nullable=True)

```

```

167 |         batch_op.alter_column('file_path',
168 |                               existing_type=sa.VARCHAR(length=500),
169 |                               nullable=True)
170 |         batch_op.alter_column('original_name',
171 |                               existing_type=sa.VARCHAR(length=255),
172 |                               nullable=True)
173 |         batch_op.alter_column('filename',
174 |                               existing_type=sa.VARCHAR(length=255),
175 |                               nullable=True)
176 |         batch_op.alter_column('entity_id',
177 |                               existing_type=sa.VARCHAR(length=36),
178 |                               nullable=True)
179 |         batch_op.alter_column('entity_type',
180 |                               existing_type=sa.VARCHAR(length=50),
181 |                               nullable=True)
182 |         batch_op.alter_column('id',
183 |                               existing_type=sa.INTEGER(),
184 |                               nullable=True,
185 |                               autoincrement=True)
186 |
187 |     with op.batch_alter_table('alerts', schema=None) as batch_op:
188 |         batch_op.drop_constraint('fk_alerts_printer', type_='foreignkey')
189 |         batch_op.drop_constraint('fk_alerts_resolved_by', type_='foreignkey')
190 |         batch_op.drop_index(batch_op.f('ix_alerts_part_id'))
191 |         batch_op.create_index(batch_op.f('ix_alert_part_id'), ['part_id'], unique=False)
192 |         batch_op.alter_column('created_at',
193 |                               existing_type=sa.DATETIME(),
194 |                               nullable=True,
195 |                               existing_server_default=sa.text('(CURRENT_TIMESTAMP)'))
196 |         batch_op.alter_column('resolvido',
197 |                               existing_type=sa.BOOLEAN(),
198 |                               nullable=True,
199 |                               existing_server_default=sa.text('0'))
200 |         batch_op.alter_column('titulo',
201 |                               existing_type=sa.VARCHAR(length=200),
202 |                               nullable=True)
203 |         batch_op.alter_column('tipo',
204 |                               existing_type=sa.VARCHAR(length=20),
205 |                               nullable=True)
206 |         batch_op.alter_column('id',
207 |                               existing_type=sa.INTEGER(),
208 |                               nullable=True,
209 |                               autoincrement=True)
210 |
211 |     with op.batch_alter_table('activities', schema=None) as batch_op:
212 |         batch_op.drop_constraint('fk_activities_to_company', type_='foreignkey')
213 |         batch_op.drop_constraint('fk_activities_technician', type_='foreignkey')
214 |         batch_op.drop_constraint('fk_activities_from_company', type_='foreignkey')
215 |         batch_op.drop_constraint('fk_activities_os_vinculada', type_='foreignkey')
216 |         batch_op.drop_constraint('fk_activities_printer', type_='foreignkey')
217 |         batch_op.drop_index(batch_op.f('ix_activities_os_vinculada_id'))
218 |         batch_op.alter_column('custo_servico',
219 |                               existing_type=sa.FLOAT(),
220 |                               nullable=True,
221 |                               existing_server_default=sa.text('(0.0)'))
222 |         batch_op.alter_column('numero_recibo',
223 |                               existing_type=sa.VARCHAR(length=50),
224 |                               nullable=True,
225 |                               existing_server_default=sa.text("('')"))
226 |         batch_op.drop_column('os_vinculada_id')
227 |
228 |     # ### end Alembic commands ###

```

=== ARQUIVO:

.alembic\versions\aa953b7eee9b_add_deleted_at_to_part_movements.py ===

```
001 | """add_deleted_at_to_part_movements
002 |
003 | Revision ID: aa953b7eee9b
004 | Revises: b5f6a7c8d9e0
005 | Create Date: 2026-07-15 17:12:43.424357
006 |
007 | """
008 | from typing import Sequence, Union
009 |
010 | from alembic import op
011 | import sqlalchemy as sa
012 |
013 |
014 | # revision identifiers, used by Alembic.
015 | revision: str = 'aa953b7eee9b'
016 | down_revision: Union[str, Sequence[str], None] = 'b5f6a7c8d9e0'
017 | branch_labels: Union[str, Sequence[str], None] = None
018 | depends_on: Union[str, Sequence[str], None] = None
019 |
020 |
021 | def upgrade() -> None:
022 |     with op.batch_alter_table('part_movements', schema=None) as batch_op:
023 |         batch_op.add_column(sa.Column('deleted_at', sa.DateTime(), nullable=True))
024 |
025 | def downgrade() -> None:
026 |     with op.batch_alter_table('part_movements', schema=None) as batch_op:
027 |         batch_op.drop_column('deleted_at')
```

=== ARQUIVO:

.alembic\versions\b1c2d3e4f5a6_create_maintenance_schedules.py ===

```
001 | """create maintenance_schedules table
002 |
003 | Revision ID: b1c2d3e4f5a6
004 | Revises: a605d5c373df
005 | Create Date: 2026-06-12 12:00:00.000000
006 |
007 | """
008 | from typing import Sequence, Union
009 |
010 | from alembic import op
011 | import sqlalchemy as sa
012 |
013 |
014 | revision: str = 'b1c2d3e4f5a6'
015 | down_revision: Union[str, Sequence[str], None] = 'a605d5c373df'
016 | branch_labels: Union[str, Sequence[str], None] = None
017 | depends_on: Union[str, Sequence[str], None] = None
018 |
019 |
020 | def upgrade() -> None:
021 |     op.create_table('maintenance_schedules',
022 |         sa.Column('id', sa.Integer(), autoincrement=True, nullable=False),
023 |         sa.Column('printer_id', sa.String(length=36), nullable=False),
024 |         sa.Column('tipo', sa.String(length=20), nullable=False),
025 |         sa.Column('intervalo_dias', sa.Integer(), nullable=True),
```

```

026 |         sa.Column('intervalo_paginas', sa.Integer(), nullable=True),
027 |         sa.Column('contador_inicial', sa.Integer(), nullable=True),
028 |         sa.Column('contador_atual', sa.Integer(), nullable=True),
029 |         sa.Column('ultima_geracao', sa.DateTime(), nullable=True),
030 |         sa.Column('proxima_geracao', sa.DateTime(), nullable=True),
031 |         sa.Column('proxima_meta_paginas', sa.Integer(), nullable=True),
032 |         sa.Column('dias_aviso', sa.Integer(), nullable=False, server_default=sa.text('7')),
033 |         sa.Column('observacao', sa.Text(), nullable=False, server_default=''),
034 |         sa.Column('ativo', sa.Boolean(), nullable=False, server_default=sa.text('1')),
035 |         sa.Column('created_at', sa.DateTime(), nullable=False),
036 |         sa.Column('updated_at', sa.DateTime(), nullable=False),
037 |         sa.ForeignKeyConstraint(['printer_id'], ['printers.id'], ),
038 |         sa.PrimaryKeyConstraint('id'),
039 |     )
040 |     with op.batch_alter_table('maintenance_schedules', schema=None) as batch_op:
041 |         batch_op.create_index('ix_maintenance_schedules_printer_id', ['printer_id'])
042 |
043 |
044 | def downgrade() -> None:
045 |     with op.batch_alter_table('maintenance_schedules', schema=None) as batch_op:
046 |         batch_op.drop_index('ix_maintenance_schedules_printer_id')
047 |     op.drop_table('maintenance_schedules')

```

=== ARQUIVO: .lalembic\versions\b2c3d4e5f6a7_add_printer_locations.py ===

```

001 | """add_printer_locations
002 |
003 | Revision ID: b2c3d4e5f6a7
004 | Revises: a1b2c3d4e5f6
005 | Create Date: 2026-06-04 11:00:00.000000
006 |
007 | """
008 | from typing import Sequence, Union
009 |
010 | from alembic import op
011 | import sqlalchemy as sa
012 |
013 |
014 | revision: str = 'b2c3d4e5f6a7'
015 | down_revision: Union[str, None] = 'a1b2c3d4e5f6'
016 | branch_labels: Union[str, Sequence[str], None] = None
017 | depends_on: Union[str, Sequence[str], None] = None
018 |
019 |
020 | def upgrade() -> None:
021 |     op.create_table('printer_locations',
022 |         sa.Column('id', sa.Integer(), autoincrement=True, nullable=False),
023 |         sa.Column('printer_id', sa.String(36), nullable=False),
024 |         sa.Column('local', sa.String(120), nullable=False),
025 |         sa.Column('data', sa.DateTime(), nullable=True),
026 |         sa.Column('observacao', sa.Text(), nullable=False),
027 |         sa.Column('created_at', sa.DateTime(), nullable=False),
028 |         sa.ForeignKeyConstraint(['printer_id'], ['printers.id'], ),
029 |         sa.PrimaryKeyConstraint('id')
030 |     )
031 |     op.create_index('ix_printer_locations_printer_id', 'printer_locations', ['printer_id'])
032 |
033 |
034 | def downgrade() -> None:
035 |     op.drop_index('ix_printer_locations_printer_id', table_name='printer_locations')
036 |     op.drop_table('printer_locations')

```

=== ARQUIVO: .lalembic\versions\b5f6a7c8d9e0_create_printer_health.py ===

```
001 | """create printer_health table
002 |
003 | Revision ID: b5f6a7c8d9e0
004 | Revises: a44fc2747c28
005 | Create Date: 2026-07-10 18:00:00.000000
006 | """
007 | from __future__ import annotations
008 |
009 | from typing import Sequence
010 |
011 | import sqlalchemy as sa
012 | from alembic import op
013 |
014 | revision: str = "b5f6a7c8d9e0"
015 | down_revision: str | None = "a44fc2747c28"
016 | branch_labels: str | Sequence[str] | None = None
017 | depends_on: str | Sequence[str] | None = None
018 |
019 |
020 | def upgrade() -> None:
021 |     op.create_table(
022 |         "printer_health",
023 |         sa.Column("id", sa.Integer(), autoincrement=True, nullable=False),
024 |         sa.Column("printer_id", sa.String(36), nullable=False),
025 |         sa.Column("timestamp", sa.DateTime(), nullable=False),
026 |         sa.Column("is_online", sa.Integer(), nullable=False, server_default=sa.text("0")),
027 |         sa.Column("ping_ms", sa.Float(), nullable=True),
028 |         sa.Column("toner_level", sa.Integer(), nullable=True),
029 |         sa.Column("toner_max", sa.Integer(), nullable=True),
030 |         sa.Column("drum_life", sa.Integer(), nullable=True),
031 |         sa.Column("drum_max", sa.Integer(), nullable=True),
032 |         sa.Column("page_count", sa.Integer(), nullable=True),
033 |         sa.Column("error_code", sa.String(50), nullable=True),
034 |         sa.Column("error_message", sa.String(255), nullable=True),
035 |         sa.Column("raw_data", sa.Text(), nullable=True),
036 |         sa.Column("created_at", sa.DateTime(), nullable=False),
037 |         sa.PrimaryKeyConstraint("id"),
038 |         sa.ForeignKeyConstraint(["printer_id"], ["printers.id"], ondelete="CASCADE"),
039 |     )
040 |     op.create_index(op.f("ix_printer_health_printer_id"), "printer_health", ["printer_id"])
041 |     op.create_index(op.f("ix_printer_health_timestamp"), "printer_health", ["timestamp"])
042 |     op.create_index(
043 |         "ix_printer_health_printer_timestamp",
044 |         "printer_health",
045 |         ["printer_id", "timestamp"],
046 |     )
047 |
048 |
049 | def downgrade() -> None:
050 |     op.drop_index("ix_printer_health_printer_timestamp", table_name="printer_health")
051 |     op.drop_index(op.f("ix_printer_health_timestamp"), table_name="printer_health")
052 |     op.drop_index(op.f("ix_printer_health_printer_id"), table_name="printer_health")
053 |     op.drop_table("printer_health")
```

=== ARQUIVO:

.alembic\versions\c3d4e5f6a7b8_add_capacidade_diaria_to_technicians.py ===

```
001 | """add capacidade_diaria to technicians
002 |
003 | Revision ID: c3d4e5f6a7b8
004 | Revises: b1c2d3e4f5a6
005 | Create Date: 2026-06-15 09:00:00.000000
006 |
007 | """
008 | from typing import Sequence, Union
009 |
010 | from alembic import op
011 | import sqlalchemy as sa
012 |
013 |
014 | revision: str = 'c3d4e5f6a7b8'
015 | down_revision: Union[str, Sequence[str], None] = 'b1c2d3e4f5a6'
016 | branch_labels: Union[str, Sequence[str], None] = None
017 | depends_on: Union[str, Sequence[str], None] = None
018 |
019 |
020 | def upgrade() -> None:
021 |     with op.batch_alter_table('technicians', schema=None) as batch_op:
022 |         batch_op.add_column(sa.Column('capacidade_diaria', sa.Integer(), nullable=False, server_default=sa.
023 |
024 |
025 | def downgrade() -> None:
026 |     with op.batch_alter_table('technicians', schema=None) as batch_op:
027 |         batch_op.drop_column('capacidade_diaria')
```

=== ARQUIVO:

.alembic\versions\d46315348779_add_indexes_and_ip_rede_size.py ===

```
001 | """add indexes and increase ip_rede
002 |
003 | Revision ID: d46315348779
004 | Revises: ded4cb6c701a
005 | Create Date: 2026-06-01 12:00:00.000000
006 |
007 | """
008 | from typing import Sequence, Union
009 |
010 | from alembic import op
011 | import sqlalchemy as sa
012 |
013 |
014 | revision: str = 'd46315348779'
015 | down_revision: Union[str, Sequence[str], None] = 'ded4cb6c701a'
016 | branch_labels: Union[str, Sequence[str], None] = None
017 | depends_on: Union[str, Sequence[str], None] = None
018 |
019 |
020 | def upgrade() -> None:
021 |     # increase ip_rede from 15 to 45
022 |     with op.batch_alter_table('printers', schema=None) as batch_op:
023 |         batch_op.alter_column('ip_rede',
024 |             existing_type=sa.VARCHAR(length=15),
025 |             type_=sa.String(length=45),
```



```

026 |         existing_nullable=False,
027 |         existing_server_default=sa.text("('')"))
028 |
029 |     # fix status_atividade default
030 |     with op.batch_alter_table('activities', schema=None) as batch_op:
031 |         batch_op.alter_column('status_atividade',
032 |             existing_type=sa.VARCHAR(length=30),
033 |             existing_server_default=sa.text("'concluida'"),
034 |             nullable=False)
035 |
036 |     # indexes
037 |     op.create_index(op.f('ix_parts_codigo'), 'parts', ['codigo'], unique=False)
038 |     op.create_index(op.f('ix_companies_cnpj'), 'companies', ['cnpj'], unique=False)
039 |     op.create_index(op.f('ix_activities_kind'), 'activities', ['kind'], unique=False)
040 |     op.create_index(op.f('ix_activities_numero_recibo'), 'activities', ['numero_recibo'], unique=False)
041 |     op.create_index(op.f('ix_activities_status_atividade'), 'activities', ['status_atividade'], unique=False)
042 |     op.create_index(op.f('ix_transfers_numero_os'), 'transfers', ['numero_os'], unique=False)
043 |     op.create_index(op.f('ix_transfers_tipo'), 'transfers', ['tipo'], unique=False)
044 |     op.create_index(op.f('ix_users_username'), 'users', ['username'], unique=False)
045 |
046 |
047 | def downgrade() -> None:
048 |     op.drop_index(op.f('ix_users_username'), table_name='users')
049 |     op.drop_index(op.f('ix_transfers_tipo'), table_name='transfers')
050 |     op.drop_index(op.f('ix_transfers_numero_os'), table_name='transfers')
051 |     op.drop_index(op.f('ix_activities_status_atividade'), table_name='activities')
052 |     op.drop_index(op.f('ix_activities_numero_recibo'), table_name='activities')
053 |     op.drop_index(op.f('ix_activities_kind'), table_name='activities')
054 |     op.drop_index(op.f('ix_companies_cnpj'), table_name='companies')
055 |     op.drop_index(op.f('ix_parts_codigo'), table_name='parts')
056 |
057 |     with op.batch_alter_table('printers', schema=None) as batch_op:
058 |         batch_op.alter_column('ip_rede',
059 |             existing_type=sa.String(length=45),
060 |             type_=sa.VARCHAR(length=15),
061 |             existing_nullable=False,
062 |             existing_server_default=sa.text("('')"))

```

=== ARQUIVO: .alembic\versions\d4e5f6a7b8c9_add_user_id_to_technicians.py

```

001 | """add user_id to technicians for user association
002 |
003 | Revision ID: d4e5f6a7b8c9
004 | Revises: c3d4e5f6a7b8
005 | Create Date: 2026-06-15 10:00:00.000000
006 |
007 | """
008 | from typing import Sequence, Union
009 |
010 | from alembic import op
011 | import sqlalchemy as sa
012 |
013 |
014 | revision: str = 'd4e5f6a7b8c9'
015 | down_revision: Union[str, Sequence[str], None] = 'c3d4e5f6a7b8'
016 | branch_labels: Union[str, Sequence[str], None] = None
017 | depends_on: Union[str, Sequence[str], None] = None
018 |
019 |

```

```

020 | def upgrade() -> None:
021 |     with op.batch_alter_table('technicians', schema=None) as batch_op:
022 |         batch_op.add_column(sa.Column('user_id', sa.Integer(), nullable=True))
023 |         batch_op.create_foreign_key('fk_technicians_user_id', 'users', ['user_id'], ['id'])
024 |
025 |
026 | def downgrade() -> None:
027 |     with op.batch_alter_table('technicians', schema=None) as batch_op:
028 |         batch_op.drop_constraint('fk_technicians_user_id', type_='foreignkey')
029 |         batch_op.drop_column('user_id')

```

=== ARQUIVO:

[.alembic\versions\ded4cb6c701a_add_estoque_minimo_to_part.py](#) ===

```

001 | """add_estoque_minimo_to_part
002 |
003 | Revision ID: ded4cb6c701a
004 | Revises: initial
005 | Create Date: 2026-05-18 14:57:26.514563
006 |
007 | """
008 | from typing import Sequence, Union
009 |
010 | from alembic import op
011 | import sqlalchemy as sa
012 |
013 |
014 | revision: str = 'ded4cb6c701a'
015 | down_revision: Union[str, Sequence[str], None] = 'initial'
016 | branch_labels: Union[str, Sequence[str], None] = None
017 | depends_on: Union[str, Sequence[str], None] = None
018 |
019 |
020 | def upgrade() -> None:
021 |     with op.batch_alter_table('parts', schema=None) as batch_op:
022 |         batch_op.add_column(sa.Column('estoque_minimo', sa.Integer(), nullable=False, server_default=sa.text('0')))
023 |
024 |
025 | def downgrade() -> None:
026 |     with op.batch_alter_table('parts', schema=None) as batch_op:
027 |         batch_op.drop_column('estoque_minimo')

```

=== ARQUIVO:

[.alembic\versions\le2ea9a0e5059_add_sintoma_relatoado_to_activities.py](#) ===

```

001 | """add_sintoma_relatoado_to_activities
002 |
003 | Revision ID: e2ea9a0e5059
004 | Revises: 3df1b9c20c20
005 | Create Date: 2026-06-05 17:42:45.220479
006 |
007 | """
008 | from typing import Sequence, Union
009 |
010 | from alembic import op
011 | import sqlalchemy as sa
012 |

```

```

013 |
014 | # revision identifiers, used by Alembic.
015 | revision: str = 'e2ea9a0e5059'
016 | down_revision: Union[str, Sequence[str], None] = '3df1b9c20c20'
017 | branch_labels: Union[str, Sequence[str], None] = None
018 | depends_on: Union[str, Sequence[str], None] = None
019 |
020 |
021 | def upgrade() -> None:
022 |     with op.batch_alter_table('activities', schema=None) as batch_op:
023 |         batch_op.add_column(sa.Column('sintoma_relato', sa.Text(), nullable=False, server_default=''))
024 |
025 |
026 | def downgrade() -> None:
027 |     with op.batch_alter_table('activities', schema=None) as batch_op:
028 |         batch_op.drop_column('sintoma_relato')

```

=== ARQUIVO:

[.alembic\versions\le5f6a7b8c9d0_create_technician_specialties.py](#) ===

```

001 | """create technician_specialties table
002 |
003 | Revision ID: e5f6a7b8c9d0
004 | Revises: d4e5f6a7b8c9
005 | Create Date: 2026-06-15 11:00:00.000000
006 |
007 | """
008 | from typing import Sequence, Union
009 |
010 | from alembic import op
011 | import sqlalchemy as sa
012 |
013 |
014 | revision: str = 'e5f6a7b8c9d0'
015 | down_revision: Union[str, Sequence[str], None] = 'd4e5f6a7b8c9'
016 | branch_labels: Union[str, Sequence[str], None] = None
017 | depends_on: Union[str, Sequence[str], None] = None
018 |
019 |
020 | def upgrade() -> None:
021 |     op.create_table('technician_specialties',
022 |         sa.Column('technician_id', sa.Integer(), nullable=False),
023 |         sa.Column('printer_id', sa.String(length=36), nullable=False),
024 |         sa.ForeignKeyConstraint(['technician_id'], ['technicians.id'], ),
025 |         sa.ForeignKeyConstraint(['printer_id'], ['printers.id'], ),
026 |         sa.PrimaryKeyConstraint('technician_id', 'printer_id'),
027 |     )
028 |
029 |
030 | def downgrade() -> None:
031 |     op.drop_table('technician_specialties')

```

=== ARQUIVO:

[.alembic\versions\lf1a2b3c4d5e6_add_marca_categoria_fornecedor_to_parts.py](#)

===

```

001 | """add_marca_categoria_fornecedor_to_parts

```

```

002 |
003 | Revision ID: f1a2b3c4d5e6
004 | Revises: aa953b7eee9b
005 | Create Date: 2026-07-23 10:00:00.000000
006 |
007 | """
008 | from typing import Sequence, Union
009 |
010 | from alembic import op
011 | import sqlalchemy as sa
012 |
013 |
014 | revision: str = 'f1a2b3c4d5e6'
015 | down_revision: Union[str, Sequence[str], None] = 'aa953b7eee9b'
016 | branch_labels: Union[str, Sequence[str], None] = None
017 | depends_on: Union[str, Sequence[str], None] = None
018 |
019 |
020 | def upgrade() -> None:
021 |     op.execute("PRAGMA foreign_keys = OFF")
022 |     with op.batch_alter_table('parts', schema=None) as batch_op:
023 |         batch_op.add_column(sa.Column('marca', sa.String(100), server_default=''))
024 |         batch_op.add_column(sa.Column('categoria', sa.String(100), server_default=''))
025 |         batch_op.add_column(sa.Column('fornecedor', sa.String(150), server_default=''))
026 |     op.execute("PRAGMA foreign_keys = ON")
027 |
028 |
029 | def downgrade() -> None:
030 |     op.execute("PRAGMA foreign_keys = OFF")
031 |     with op.batch_alter_table('parts', schema=None) as batch_op:
032 |         batch_op.drop_column('fornecedor')
033 |         batch_op.drop_column('categoria')
034 |         batch_op.drop_column('marca')
035 |     op.execute("PRAGMA foreign_keys = ON")

```

=== ARQUIVO:

[.alembic\versions\f6a7b8c9d0e1_refactor_technician_specialties.py](#) ===

```

001 | """refactor technician_specialties to use modelo instead of printer_id
002 |
003 | Revision ID: f6a7b8c9d0e1
004 | Revises: e5f6a7b8c9d0
005 | Create Date: 2026-06-15 12:00:00.000000
006 |
007 | """
008 | from typing import Sequence, Union
009 |
010 | from alembic import op
011 | import sqlalchemy as sa
012 |
013 |
014 | revision: str = 'f6a7b8c9d0e1'
015 | down_revision: Union[str, Sequence[str], None] = 'e5f6a7b8c9d0'
016 | branch_labels: Union[str, Sequence[str], None] = None
017 | depends_on: Union[str, Sequence[str], None] = None
018 |
019 |
020 | def upgrade() -> None:
021 |     op.drop_table('technician_specialties')
022 |     op.create_table('technician_specialties',

```

```

023 |         sa.Column('id', sa.Integer(), autoincrement=True, nullable=False),
024 |         sa.Column('technician_id', sa.Integer(), sa.ForeignKey('technicians.id'), nullable=False),
025 |         sa.Column('modelo', sa.String(length=120), nullable=False),
026 |         sa.PrimaryKeyConstraint('id'),
027 |     )
028 |
029 |
030 | def downgrade() -> None:
031 |     op.drop_table('technician_specialties')
032 |     op.create_table('technician_specialties',
033 |         sa.Column('technician_id', sa.Integer(), nullable=False),
034 |         sa.Column('printer_id', sa.String(length=36), nullable=False),
035 |         sa.ForeignKeyConstraint(['technician_id'], ['technicians.id'], ),
036 |         sa.ForeignKeyConstraint(['printer_id'], ['printers.id'], ),
037 |         sa.PrimaryKeyConstraint('technician_id', 'printer_id'),
038 |     )

```

=== ARQUIVO: .alembic\versions\initial_schema.py ===

```

001 | """initial_schema
002 |
003 | Revision ID: initial
004 | Revises:
005 | Create Date: 2026-05-18
006 | """
007 |
008 | from typing import Sequence, Union
009 | from alembic import op
010 | import sqlalchemy as sa
011 |
012 | revision: str = "initial"
013 | down_revision: Union[str, Sequence[str], None] = None
014 | branch_labels: Union[str, Sequence[str], None] = None
015 | depends_on: Union[str, Sequence[str], None] = None
016 |
017 |
018 | def upgrade() -> None:
019 |     pass
020 |
021 |
022 | def downgrade() -> None:
023 |     pass

```

=== ARQUIVO: .lapp__init__.py ===

```

001 | from __future__ import annotations
002 |
003 | __version__: str = "2.0.0"
004 | __author__: str = "Otavio Oliveira"

```

=== ARQUIVO: .lapp\models\activity.py ===

```

001 | from __future__ import annotations
002 |
003 | from datetime import datetime
004 | from typing import TYPE_CHECKING, Optional

```

```

005 |
006 | from sqlalchemy import DateTime, Float, ForeignKey, Index, Integer, String, Text
007 | from sqlalchemy.orm import Mapped, mapped_column, relationship
008 |
009 | from app.models.base import Base, SoftDeleteMixin, utcnow
010 |
011 | if TYPE_CHECKING:
012 |     from app.models.company import Company
013 |     from app.models.part_movement import PartMovement
014 |     from app.models.part_reservation import PartReservation
015 |     from app.models.printer import Printer
016 |     from app.models.printer_location import PrinterLocation
017 |     from app.models.technician import Technician
018 |
019 |
020 | class Activity(Base, SoftDeleteMixin):
021 |     __tablename__ = "activities"
022 |     __table_args__ = (
023 |         Index("ix_activity_kind", "kind"),
024 |         Index("ix_activity_numero_recibo", "numero_recibo"),
025 |         Index("ix_activity_status", "status_atividade"),
026 |     )
027 |     id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
028 |     printer_id: Mapped[str] = mapped_column(String(36), ForeignKey("printers.id"), index=True)
029 |     kind: Mapped[str] = mapped_column(String(30))
030 |
031 |     from_company_id: Mapped[Optional[int]] = mapped_column(Integer, ForeignKey("companies.id"), nullable=True)
032 |     to_company_id: Mapped[Optional[int]] = mapped_column(Integer, ForeignKey("companies.id"), nullable=True)
033 |     numero_recibo: Mapped[str] = mapped_column(String(50), default="")
034 |     responsavel: Mapped[str] = mapped_column(String(120), default="")
035 |     tecnico_id: Mapped[Optional[int]] = mapped_column(Integer, ForeignKey("technicians.id"), nullable=True)
036 |     custo_servico: Mapped[float] = mapped_column(Float, default=0.0)
037 |     status_atividade: Mapped[str] = mapped_column(String(30), default="Aberta")
038 |
039 |     created_at: Mapped[datetime] = mapped_column(DateTime, default=utcnow)
040 |     event_at: Mapped[datetime] = mapped_column(DateTime, default=utcnow)
041 |     inicio_atendimento: Mapped[Optional[datetime]] = mapped_column(DateTime, nullable=True)
042 |     fim_atendimento: Mapped[Optional[datetime]] = mapped_column(DateTime, nullable=True)
043 |     notes: Mapped[str] = mapped_column(Text, default="")
044 |     parts_used: Mapped[str] = mapped_column(Text, default="")
045 |     from_location: Mapped[str] = mapped_column(String(120), default="")
046 |     to_location: Mapped[str] = mapped_column(String(120), default="")
047 |     procedimentos: Mapped[str] = mapped_column(Text, default="")
048 |     urgencia: Mapped[str] = mapped_column(String(20), default="Normal")
049 |     sintoma_relato: Mapped[str] = mapped_column(Text, default="")
050 |     diagnostico_tecnico: Mapped[str] = mapped_column(Text, default="")
051 |     solucao_aplicada: Mapped[str] = mapped_column(Text, default="")
052 |     os_vinculada_id: Mapped[Optional[int]] = mapped_column(Integer, ForeignKey("activities.id"), nullable=True)
053 |
054 |     printer: Mapped[Printer] = relationship(back_populates="activities")
055 |     part_movements: Mapped[list[PartMovement]] = relationship(back_populates="activity", cascade="all, delete-orphan")
056 |     part_reservations: Mapped[list[PartReservation]] = relationship(back_populates="activity", cascade="all, delete-orphan")
057 |     from_company: Mapped[Optional[Company]] = relationship(back_populates="activities_from", foreign_keys=[from_company_id])
058 |     to_company: Mapped[Optional[Company]] = relationship(back_populates="activities_to", foreign_keys=[to_company_id])
059 |     technician: Mapped[Optional[Technician]] = relationship(back_populates="activities")
060 |     os_vinculada: Mapped[Optional[Activity]] = relationship("Activity", remote_side="Activity.id", back_populates="os_filhas")
061 |     os_filhas: Mapped[list[Activity]] = relationship("Activity", back_populates="os_vinculada")
062 |     location_records: Mapped[list[PrinterLocation]] = relationship(back_populates="activity", cascade="all, delete-orphan")
063 |
064 |

```

=== ARQUIVO: .lapp\models\alert.py ===

```
001 | ■from __future__ import annotations
002 |
003 | from datetime import datetime
004 | from typing import TYPE_CHECKING, Optional
005 |
006 | from sqlalchemy import Boolean, DateTime, ForeignKey, Integer, String, Text
007 | from sqlalchemy.orm import Mapped, mapped_column, relationship
008 |
009 | from app.models.base import Base, SoftDeleteMixin, utcnow
010 |
011 | if TYPE_CHECKING:
012 |     from app.models.part import Part
013 |     from app.models.printer import Printer
014 |     from app.models.user import User
015 |
016 |
017 | class Alert(Base, SoftDeleteMixin):
018 |     __tablename__ = "alerts"
019 |     id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
020 |     printer_id: Mapped[Optional[str]] = mapped_column(String(36), ForeignKey("printers.id"), nullable=True,
021 |     part_id: Mapped[Optional[int]] = mapped_column(Integer, ForeignKey("parts.id"), nullable=True, index=True)
022 |     tipo: Mapped[str] = mapped_column(String(20), default="revisao")
023 |     titulo: Mapped[str] = mapped_column(String(200), default="")
024 |     descricao: Mapped[Optional[str]] = mapped_column(Text, default="")
025 |     data_alerta: Mapped[Optional[datetime]] = mapped_column(DateTime, nullable=True)
026 |     data_agendada: Mapped[Optional[datetime]] = mapped_column(DateTime, nullable=True)
027 |     resolvido: Mapped[bool] = mapped_column(Boolean, default=False)
028 |     resolvido_em: Mapped[Optional[datetime]] = mapped_column(DateTime, nullable=True)
029 |     resolvido_por: Mapped[Optional[int]] = mapped_column(Integer, ForeignKey("users.id"), nullable=True)
030 |     notificado: Mapped[bool] = mapped_column(Boolean, default=False)
031 |     notificado_em: Mapped[Optional[datetime]] = mapped_column(DateTime, nullable=True)
032 |     created_at: Mapped[datetime] = mapped_column(DateTime, default=utcnow)
033 |
034 |     printer: Mapped[Optional[Printer]] = relationship(back_populates="alerts")
035 |     part: Mapped[Optional[Part]] = relationship(back_populates="alerts")
036 |     resolved_by: Mapped[Optional[User]] = relationship(back_populates="alerts_resolved")
037 |
038 |
```

=== ARQUIVO: .lapp\models\attachment.py ===

```
001 | ■from __future__ import annotations
002 |
003 | from datetime import datetime
004 | from typing import TYPE_CHECKING, Optional
005 |
006 | from sqlalchemy import DateTime, ForeignKey, Integer, String
007 | from sqlalchemy.orm import Mapped, mapped_column, relationship
008 |
009 | from app.models.base import Base, utcnow
010 |
011 | if TYPE_CHECKING:
012 |     from app.models.user import User
013 |
014 |
015 | class Attachment(Base):
016 |     __tablename__ = "attachments"
017 |     id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
```

```

018 |     entity_type: Mapped[str] = mapped_column(String(50), default="")
019 |     entity_id: Mapped[str] = mapped_column(String(36), default="")
020 |     filename: Mapped[str] = mapped_column(String(255), default="")
021 |     original_name: Mapped[str] = mapped_column(String(255), default="")
022 |     file_path: Mapped[str] = mapped_column(String(500), default="")
023 |     mime_type: Mapped[str] = mapped_column(String(100), default="")
024 |     size_bytes: Mapped[int] = mapped_column(Integer, default=0)
025 |     categoria: Mapped[str] = mapped_column(String(50), default="recibo")
026 |     created_at: Mapped[datetime] = mapped_column(DateTime, default=utcnow)
027 |     uploader_id: Mapped[Optional[int]] = mapped_column(Integer, ForeignKey("users.id"), nullable=True)
028 |
029 |     uploader: Mapped[Optional[User]] = relationship(back_populates="uploads")
030 |
031 |

```

=== ARQUIVO: .\app\models\audit_log.py ===

```

001 | from __future__ import annotations
002 |
003 | from datetime import datetime
004 | from typing import TYPE_CHECKING, Optional
005 |
006 | from sqlalchemy import DateTime, ForeignKey, Integer, String, Text
007 | from sqlalchemy.orm import Mapped, mapped_column, relationship
008 |
009 | from app.models.base import Base, utcnow
010 |
011 | if TYPE_CHECKING:
012 |     from app.models.user import User
013 |
014 |
015 | class AuditLog(Base):
016 |     __tablename__ = "audit_log"
017 |     id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
018 |     user_id: Mapped[Optional[int]] = mapped_column(Integer, ForeignKey("users.id"), nullable=True)
019 |     acao: Mapped[str] = mapped_column(String(50))
020 |     tabela_alvo: Mapped[str] = mapped_column(String(50), default="")
021 |     registro_id: Mapped[str] = mapped_column(String(50), default="")
022 |     ip_address: Mapped[str] = mapped_column(String(45), default="")
023 |     dados_antes: Mapped[Optional[str]] = mapped_column(Text, nullable=True)
024 |     dados_depois: Mapped[Optional[str]] = mapped_column(Text, nullable=True)
025 |     created_at: Mapped[datetime] = mapped_column(DateTime, default=utcnow)
026 |
027 |     user: Mapped[Optional[User]] = relationship(back_populates="audit_logs")
028 |
029 |

```

=== ARQUIVO: .\app\models\base.py ===

```

001 | from datetime import datetime, timezone
002 | from typing import Optional
003 |
004 | from sqlalchemy import DateTime
005 | from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column
006 |
007 |
008 | def utcnow() -> datetime:
009 |     return datetime.now(timezone.utc).replace(tzinfo=None)
010 |

```



```

011 |
012 | class SoftDeleteMixin:
013 |     deleted_at: Mapped[Optional[datetime]] = mapped_column(DateTime, default=None, nullable=True)
014 |
015 |
016 | class Base(DeclarativeBase):
017 |     pass

```

=== ARQUIVO: .\app\models\company.py ===

```

001 | ■from __future__ import annotations
002 |
003 | from datetime import datetime
004 | from typing import TYPE_CHECKING, List, Optional
005 |
006 | from sqlalchemy import DateTime, Index, Integer, String, Text
007 | from sqlalchemy.orm import Mapped, mapped_column, relationship
008 |
009 | from app.models.base import Base, SoftDeleteMixin, utcnow
010 |
011 | if TYPE_CHECKING:
012 |     from app.models.activity import Activity
013 |     from app.models.printer import Printer
014 |
015 |
016 | class Company(Base, SoftDeleteMixin):
017 |     __tablename__ = "companies"
018 |     __table_args__ = (Index("ix_company_cnpj", "cnpj"),)
019 |     id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
020 |     nome: Mapped[str] = mapped_column(String(150), nullable=False)
021 |     cnpj: Mapped[str] = mapped_column(String(18), default="")
022 |     endereco: Mapped[str] = mapped_column(String(255), default="")
023 |     cidade: Mapped[str] = mapped_column(String(100), default="")
024 |     uf: Mapped[str] = mapped_column(String(2), default="")
025 |     telefone: Mapped[str] = mapped_column(String(20), default="")
026 |     email: Mapped[str] = mapped_column(String(120), default="")
027 |     tipo: Mapped[str] = mapped_column(String(20), default="cliente")
028 |     observacao: Mapped[Optional[str]] = mapped_column(Text, default="")
029 |     created_at: Mapped[datetime] = mapped_column(DateTime, default=utcnow)
030 |
031 |     printers: Mapped[List[Printer]] = relationship(back_populates="company")
032 |     activities_from: Mapped[List[Activity]] = relationship(back_populates="from_company", foreign_keys="Activit
033 |     activities_to: Mapped[List[Activity]] = relationship(back_populates="to_company", foreign_keys="Activit
034 |
035 |
036 |

```

=== ARQUIVO: .\app\models\login_history.py ===

```

001 | from __future__ import annotations
002 |
003 | from datetime import datetime
004 | from typing import TYPE_CHECKING, Optional
005 |
006 | from sqlalchemy import DateTime, ForeignKey, Integer, String
007 | from sqlalchemy.orm import Mapped, mapped_column, relationship
008 |
009 | from app.models.base import Base, utcnow
010 |

```

```

011 | if TYPE_CHECKING:
012 |     from app.models.user import User
013 |
014 |
015 | class LoginHistory(Base):
016 |     __tablename__ = "login_history"
017 |     id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
018 |     user_id: Mapped[int] = mapped_column(Integer, ForeignKey("users.id"), nullable=False)
019 |     login_at: Mapped[datetime] = mapped_column(DateTime, default=utcnow)
020 |     logout_at: Mapped[Optional[datetime]] = mapped_column(DateTime, nullable=True)
021 |     ip_address: Mapped[str] = mapped_column(String(45), default="")
022 |
023 |     user: Mapped[User] = relationship(back_populates="login_history")

```

=== ARQUIVO: .\app\models\maintenance_schedule.py ===

```

001 | from __future__ import annotations
002 |
003 | from datetime import datetime
004 | from typing import TYPE_CHECKING, Optional
005 |
006 | from sqlalchemy import Boolean, DateTime, ForeignKey, Integer, String, Text
007 | from sqlalchemy.orm import Mapped, mapped_column, relationship
008 |
009 | from app.models.base import Base, utcnow
010 |
011 | if TYPE_CHECKING:
012 |     from app.models.printer import Printer
013 |
014 |
015 | class MaintenanceSchedule(Base):
016 |     __tablename__ = "maintenance_schedules"
017 |
018 |     id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
019 |     printer_id: Mapped[str] = mapped_column(String(36), ForeignKey("printers.id"), nullable=False, index=True)
020 |     tipo: Mapped[str] = mapped_column(String(20), nullable=False)
021 |     intervalo_dias: Mapped[Optional[int]] = mapped_column(Integer, nullable=True)
022 |     intervalo_paginas: Mapped[Optional[int]] = mapped_column(Integer, nullable=True)
023 |     contador_inicial: Mapped[Optional[int]] = mapped_column(Integer, nullable=True)
024 |     contador_atual: Mapped[Optional[int]] = mapped_column(Integer, nullable=True)
025 |     ultima_geracao: Mapped[Optional[datetime]] = mapped_column(DateTime, nullable=True)
026 |     proxima_geracao: Mapped[Optional[datetime]] = mapped_column(DateTime, nullable=True)
027 |     proxima_meta_paginas: Mapped[Optional[int]] = mapped_column(Integer, nullable=True)
028 |     dias_aviso: Mapped[int] = mapped_column(Integer, default=7)
029 |     observacao: Mapped[str] = mapped_column(Text, default="")
030 |     ativo: Mapped[bool] = mapped_column(Boolean, default=True)
031 |     created_at: Mapped[datetime] = mapped_column(DateTime, default=utcnow)
032 |     updated_at: Mapped[datetime] = mapped_column(DateTime, default=utcnow, onupdate=utcnow)
033 |
034 |     printer: Mapped[Printer] = relationship("Printer", back_populates="maintenance_schedules")
035 |
036 |     def __repr__(self) -> str:
037 |         return f"<MaintenanceSchedule id={self.id} printer={self.printer_id} tipo={self.tipo}>"

```

=== ARQUIVO: .\app\models\part.py ===

```

001 | from __future__ import annotations
002 |
003 | from datetime import datetime

```

```

004 | from typing import TYPE_CHECKING, Optional
005 |
006 | from sqlalchemy import DateTime, Float, Index, Integer, String, Text
007 | from sqlalchemy.orm import Mapped, mapped_column, relationship
008 |
009 | from app.models.base import Base, SoftDeleteMixin, utcnow
010 |
011 | if TYPE_CHECKING:
012 |     from app.models.alert import Alert
013 |     from app.models.part_movement import PartMovement
014 |     from app.models.part_reservation import PartReservation
015 |     from app.models.purchase_requisition import PurchaseRequisition
016 |
017 |
018 | class Part(Base, SoftDeleteMixin):
019 |     __tablename__ = "parts"
020 |     __table_args__ = (Index("ix_part_codigo", "codigo"),)
021 |     id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
022 |     codigo: Mapped[str] = mapped_column(String(50), default="")
023 |     nome: Mapped[str] = mapped_column(String(150), nullable=False)
024 |     descricao: Mapped[str] = mapped_column(Text, default="")
025 |     quantidade_estoque: Mapped[int] = mapped_column(Integer, default=0)
026 |     estoque_minimo: Mapped[int] = mapped_column(Integer, default=1)
027 |     preco_unitario: Mapped[float] = mapped_column(Float, default=0.0)
028 |     modelo_compativel: Mapped[str] = mapped_column(String(200), default="")
029 |     marca: Mapped[str] = mapped_column(String(100), default="")
030 |     categoria: Mapped[str] = mapped_column(String(100), default="")
031 |     fornecedor: Mapped[str] = mapped_column(String(150), default="")
032 |     created_at: Mapped[datetime] = mapped_column(DateTime, default=utcnow)
033 |
034 |     alerts: Mapped[list[Alert]] = relationship(back_populates="part")
035 |     movements: Mapped[list[PartMovement]] = relationship(back_populates="part", cascade="all, delete-orphan")
036 |     reservations: Mapped[list[PartReservation]] = relationship(back_populates="part", cascade="all, delete-orphan")
037 |     purchase_requisitions: Mapped[list[PurchaseRequisition]] = relationship(back_populates="part", cascade="all, delete-orphan")
038 |
039 |

```

=== ARQUIVO: .lapp\models\part_movement.py ===

```

001 | from __future__ import annotations
002 |
003 | from datetime import datetime
004 | from typing import TYPE_CHECKING, Optional
005 |
006 | from sqlalchemy import DateTime, ForeignKey, Integer, String, Text
007 | from sqlalchemy.orm import Mapped, mapped_column, relationship
008 |
009 | from app.models.base import Base, SoftDeleteMixin, utcnow
010 |
011 | if TYPE_CHECKING:
012 |     from app.models.activity import Activity
013 |     from app.models.part import Part
014 |
015 |
016 | class PartMovement(Base, SoftDeleteMixin):
017 |     __tablename__ = "part_movements"
018 |     id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
019 |     part_id: Mapped[int] = mapped_column(Integer, ForeignKey("parts.id"), nullable=False, index=True)
020 |     activity_id: Mapped[Optional[int]] = mapped_column(Integer, ForeignKey("activities.id"), nullable=True, index=True)
021 |     tipo: Mapped[str] = mapped_column(String(30), nullable=False)
022 |     quantidade: Mapped[int] = mapped_column(Integer, nullable=False)

```

```

023 |     saldo_anterior: Mapped[int] = mapped_column(Integer, default=0)
024 |     saldo_posterior: Mapped[int] = mapped_column(Integer, default=0)
025 |     observacao: Mapped[str] = mapped_column(Text, default="")
026 |     created_at: Mapped[datetime] = mapped_column(DateTime, default=utcnow)
027 |
028 |     part: Mapped[Part] = relationship(back_populates="movements")
029 |     activity: Mapped[Optional[Activity]] = relationship(back_populates="part_movements")

```

=== ARQUIVO: .lapp\models\part_reservation.py ===

```

001 | from __future__ import annotations
002 |
003 | from datetime import datetime
004 | from typing import TYPE_CHECKING, Optional
005 |
006 | from sqlalchemy import DateTime, ForeignKey, Integer, String
007 | from sqlalchemy.orm import Mapped, mapped_column, relationship
008 |
009 | from app.models.base import Base, utcnow
010 |
011 | if TYPE_CHECKING:
012 |     from app.models.activity import Activity
013 |     from app.models.part import Part
014 |
015 |
016 | class PartReservation(Base):
017 |     __tablename__ = "part_reservations"
018 |     id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
019 |     part_id: Mapped[int] = mapped_column(Integer, ForeignKey("parts.id"), nullable=False, index=True)
020 |     activity_id: Mapped[int] = mapped_column(Integer, ForeignKey("activities.id"), nullable=False, index=True)
021 |     quantidade: Mapped[int] = mapped_column(Integer, nullable=False, default=1)
022 |     status: Mapped[str] = mapped_column(String(20), nullable=False, default="reservada")
023 |     created_at: Mapped[datetime] = mapped_column(DateTime, default=utcnow)
024 |     updated_at: Mapped[Optional[datetime]] = mapped_column(DateTime, nullable=True, onupdate=utcnow)
025 |
026 |     part: Mapped[Part] = relationship(back_populates="reservations")
027 |     activity: Mapped[Activity] = relationship(back_populates="part_reservations")

```

=== ARQUIVO: .lapp\models\printer.py ===

```

001 | ■from __future__ import annotations
002 |
003 | import uuid
004 | from datetime import datetime
005 | from typing import TYPE_CHECKING, List, Optional
006 |
007 | from sqlalchemy import DateTime, ForeignKey, Integer, String, Text
008 | from sqlalchemy.orm import Mapped, mapped_column, relationship
009 |
010 | from app.models.base import Base, SoftDeleteMixin, utcnow
011 |
012 | if TYPE_CHECKING:
013 |     from app.models.activity import Activity
014 |     from app.models.alert import Alert
015 |     from app.models.company import Company
016 |     from app.models.maintenance_schedule import MaintenanceSchedule
017 |     from app.models.printer_location import PrinterLocation
018 |
019 |

```

```

020 |
021 | class Printer(Base, SoftDeleteMixin):
022 |     __tablename__ = "printers"
023 |     id: Mapped[str] = mapped_column(String(36), primary_key=True, default=lambda: str(uuid.uuid4()))
024 |     patrimonio: Mapped[str] = mapped_column(String(80), unique=True, nullable=False)
025 |     modelo: Mapped[str] = mapped_column(String(80), default="")
026 |     serial: Mapped[str] = mapped_column(String(80), default="")
027 |     status: Mapped[str] = mapped_column(String(30), default="Operacional")
028 |     local_atual: Mapped[str] = mapped_column(String(120), default="")
029 |
030 |     marca: Mapped[str] = mapped_column(String(80), default="")
031 |     tipo: Mapped[str] = mapped_column(String(20), default="")
032 |     ip_rede: Mapped[str] = mapped_column(String(45), default="")
033 |     mac_address: Mapped[str] = mapped_column(String(17), default="")
034 |     empresa_id: Mapped[Optional[int]] = mapped_column(Integer, ForeignKey("companies.id"), nullable=True)
035 |     ultima_revisao: Mapped[Optional[datetime]] = mapped_column(DateTime, nullable=True)
036 |     proxima_revisao: Mapped[Optional[datetime]] = mapped_column(DateTime, nullable=True)
037 |     urgencia_prox_manutencao: Mapped[str] = mapped_column(String(20), default="Normal")
038 |     tecnico: Mapped[str] = mapped_column(String(120), default="")
039 |
040 |     foto_path: Mapped[Optional[str]] = mapped_column(String(255), nullable=True)
041 |     pecas_faltantes: Mapped[Optional[str]] = mapped_column(Text, nullable=True)
042 |     observacao: Mapped[str] = mapped_column(Text, default="")
043 |     created_at: Mapped[datetime] = mapped_column(DateTime, default=utcnow)
044 |     updated_at: Mapped[datetime] = mapped_column(DateTime, default=utcnow, onupdate=utcnow)
045 |
046 |     company: Mapped[Optional[Company]] = relationship(back_populates="printers")
047 |     activities: Mapped[List[Activity]] = relationship(back_populates="printer")
048 |     alerts: Mapped[List[Alert]] = relationship(back_populates="printer")
049 |     location_history: Mapped[List[PrinterLocation]] = relationship(back_populates="printer", order_by="PrinterLocation.timestamp")
050 |     maintenance_schedules: Mapped[List[MaintenanceSchedule]] = relationship(back_populates="printer")
051 |
052 |

```

=== ARQUIVO: .\app\models\printer_health.py ===

```

001 | from __future__ import annotations
002 |
003 | from datetime import datetime
004 | from typing import Optional
005 |
006 | from sqlalchemy import Column, DateTime, Float, ForeignKey, Integer, String, Text
007 |
008 | from app.models.base import Base
009 |
010 |
011 | class PrinterHealth(Base):
012 |     __tablename__ = "printer_health"
013 |
014 |     id: int = Column(Integer, primary_key=True, autoincrement=True)
015 |     printer_id: str = Column(String(36), ForeignKey("printers.id", ondelete="CASCADE"), nullable=False, index=True)
016 |     timestamp: datetime = Column(DateTime, default=datetime.utcnow, nullable=False, index=True)
017 |
018 |     is_online: bool = Column(Integer, default=False, nullable=False)
019 |
020 |     ping_ms: Optional[float] = Column(Float, nullable=True)
021 |
022 |     toner_level: Optional[int] = Column(Integer, nullable=True)
023 |     toner_max: Optional[int] = Column(Integer, nullable=True)
024 |     drum_life: Optional[int] = Column(Integer, nullable=True)
025 |     drum_max: Optional[int] = Column(Integer, nullable=True)

```

```

026 |     page_count: Optional[int] = Column(Integer, nullable=True)
027 |
028 |     error_code: Optional[str] = Column(String(50), nullable=True)
029 |     error_message: Optional[str] = Column(String(255), nullable=True)
030 |     raw_data: Optional[str] = Column(Text, nullable=True)
031 |
032 |     created_at: datetime = Column(DateTime, default=datetime.utcnow, nullable=False)

```

=== ARQUIVO: .lapp\models\printer_location.py ===

```

001 | from __future__ import annotations
002 |
003 | from datetime import datetime
004 | from typing import TYPE_CHECKING, Optional
005 |
006 | from sqlalchemy import DateTime, ForeignKey, Integer, String, Text
007 | from sqlalchemy.orm import Mapped, mapped_column, relationship
008 |
009 | from app.models.base import Base, utcnow
010 |
011 | if TYPE_CHECKING:
012 |     from app.models.activity import Activity
013 |     from app.models.printer import Printer
014 |
015 |
016 | class PrinterLocation(Base):
017 |     __tablename__ = "printer_locations"
018 |     id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
019 |     printer_id: Mapped[str] = mapped_column(String(36), ForeignKey("printers.id"), nullable=False, index=True)
020 |     local: Mapped[str] = mapped_column(String(120), default="")
021 |     data: Mapped[Optional[datetime]] = mapped_column(DateTime, nullable=True)
022 |     observacao: Mapped[str] = mapped_column(Text, default="")
023 |     activity_id: Mapped[Optional[int]] = mapped_column(Integer, ForeignKey("activities.id"), nullable=True)
024 |     created_at: Mapped[datetime] = mapped_column(DateTime, default=utcnow)
025 |
026 |     printer: Mapped[Printer] = relationship(back_populates="location_history")
027 |     activity: Mapped[Optional[Activity]] = relationship(back_populates="location_records")

```

=== ARQUIVO: .lapp\models\purchase_requisition.py ===

```

001 | from __future__ import annotations
002 |
003 | from datetime import datetime
004 | from typing import TYPE_CHECKING, Optional
005 |
006 | from sqlalchemy import DateTime, ForeignKey, Integer, String, Text
007 | from sqlalchemy.orm import Mapped, mapped_column, relationship
008 |
009 | from app.models.base import Base, utcnow
010 |
011 | if TYPE_CHECKING:
012 |     from app.models.part import Part
013 |
014 |
015 | class PurchaseRequisition(Base):
016 |     __tablename__ = "purchase_requisitions"
017 |     id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
018 |     part_id: Mapped[int] = mapped_column(Integer, ForeignKey("parts.id"), nullable=False, index=True)
019 |     quantidade_sugerida: Mapped[int] = mapped_column(Integer, nullable=False, default=1)

```

```

020 |     status: Mapped[str] = mapped_column(String(20), nullable=False, default="pendente")
021 |     observacao: Mapped[str] = mapped_column(Text, default="")
022 |     created_at: Mapped[datetime] = mapped_column(DateTime, default=utcnow)
023 |     updated_at: Mapped[Optional[datetime]] = mapped_column(DateTime, nullable=True, onupdate=utcnow)
024 |
025 |     part: Mapped[Part] = relationship(back_populates="purchase_requisitions")

```

=== ARQUIVO: .lapp\models\technician.py ===

```

001 | from __future__ import annotations
002 |
003 | from datetime import datetime
004 | from typing import TYPE_CHECKING, List
005 |
006 | from sqlalchemy import Boolean, DateTime, ForeignKey, Integer, String
007 | from sqlalchemy.orm import Mapped, mapped_column, relationship
008 |
009 | from app.models.base import Base, SoftDeleteMixin, utcnow
010 |
011 | if TYPE_CHECKING:
012 |     from app.models.activity import Activity
013 |     from app.models.technician_specialty import TechnicianSpecialty
014 |     from app.models.user import User
015 |
016 |
017 | class Technician(Base, SoftDeleteMixin):
018 |     __tablename__ = "technicians"
019 |     id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
020 |     nome_completo: Mapped[str] = mapped_column(String(150), nullable=False)
021 |     nome_exibicao: Mapped[str] = mapped_column(String(80), default="")
022 |     telefone: Mapped[str] = mapped_column(String(20), default="")
023 |     email: Mapped[str] = mapped_column(String(120), default="")
024 |     ativo: Mapped[bool] = mapped_column(Boolean, default=True)
025 |     capacidade_diaria: Mapped[int] = mapped_column(Integer, default=5)
026 |     user_id: Mapped[int | None] = mapped_column(Integer, ForeignKey("users.id"), nullable=True)
027 |     created_at: Mapped[datetime] = mapped_column(DateTime, default=utcnow)
028 |
029 |     activities: Mapped[List[Activity]] = relationship(back_populates="technician")
030 |     user: Mapped[User | None] = relationship()
031 |     specialties: Mapped[List[TechnicianSpecialty]] = relationship(back_populates="technician", cascade="all")
032 |
033 |

```

=== ARQUIVO: .lapp\models\technician_specialty.py ===

```

001 | from __future__ import annotations
002 |
003 | from typing import TYPE_CHECKING
004 |
005 | from sqlalchemy import ForeignKey, Integer, String
006 | from sqlalchemy.orm import Mapped, mapped_column, relationship
007 |
008 | from app.models.base import Base
009 |
010 | if TYPE_CHECKING:
011 |     from app.models.technician import Technician
012 |
013 |
014 | class TechnicianSpecialty(Base):

```



```

015 |     __tablename__ = "technician_specialties"
016 |     id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
017 |     technician_id: Mapped[int] = mapped_column(Integer, ForeignKey("technicians.id"), nullable=False)
018 |     modelo: Mapped[str] = mapped_column(String(120), nullable=False)
019 |
020 |     technician: Mapped[Technician] = relationship(back_populates="specialties")

```

=== ARQUIVO: .\app\models\user.py ===

```

001 | from __future__ import annotations
002 |
003 | from datetime import datetime
004 | from typing import TYPE_CHECKING, List, Optional
005 |
006 | from sqlalchemy import Boolean, DateTime, Index, Integer, String
007 | from sqlalchemy.orm import Mapped, mapped_column, relationship
008 |
009 | from app.models.base import Base, SoftDeleteMixin, utcnow
010 |
011 | if TYPE_CHECKING:
012 |     from app.models.alert import Alert
013 |     from app.models.attachment import Attachment
014 |     from app.models.audit_log import AuditLog
015 |     from app.models.login_history import LoginHistory
016 |
017 |
018 | class User(Base, SoftDeleteMixin):
019 |     __tablename__ = "users"
020 |     __table_args__ = (Index("ix_user_username", "username"),)
021 |     id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
022 |     nome: Mapped[str] = mapped_column(String(120), nullable=False)
023 |     email: Mapped[str] = mapped_column(String(120), unique=True, nullable=False)
024 |     username: Mapped[str] = mapped_column(String(50), default="")
025 |     senha_hash: Mapped[str] = mapped_column(String(255), nullable=False)
026 |     perfil: Mapped[str] = mapped_column(String(20), default="visualizador")
027 |     ativo: Mapped[bool] = mapped_column(Boolean, default=True)
028 |     created_at: Mapped[datetime] = mapped_column(DateTime, default=utcnow)
029 |     ultimo_login: Mapped[Optional[datetime]] = mapped_column(DateTime, nullable=True)
030 |
031 |     alerts_resolved: Mapped[List[Alert]] = relationship(back_populates="resolved_by")
032 |     audit_logs: Mapped[List[AuditLog]] = relationship(back_populates="user")
033 |     uploads: Mapped[List[Attachment]] = relationship(back_populates="uploader")
034 |     login_history: Mapped[List[LoginHistory]] = relationship(back_populates="user", order_by="LoginHistory")
035 |
036 |

```

=== ARQUIVO: .\app\models__init__.py ===

```

001 | from app.models.activity import Activity
002 | from app.models.alert import Alert
003 | from app.models.attachment import Attachment
004 | from app.models.audit_log import AuditLog
005 | from app.models.base import Base
006 | from app.models.company import Company
007 | from app.models.login_history import LoginHistory
008 | from app.models.maintenance_schedule import MaintenanceSchedule
009 | from app.models.part import Part
010 | from app.models.part_movement import PartMovement
011 | from app.models.part_reservation import PartReservation

```



```

012 | from app.models.printer import Printer
013 | from app.models.purchase_requisition import PurchaseRequisition
014 | from app.models.printer_location import PrinterLocation
015 | from app.models.printer_health import PrinterHealth
016 | from app.models.technician import Technician
017 | from app.models.technician_specialty import TechnicianSpecialty
018 | from app.models.user import User
019 |
020 | __all__ = [
021 |     "Activity",
022 |     "Alert",
023 |     "Attachment",
024 |     "AuditLog",
025 |     "Base",
026 |     "Company",
027 |     "LoginHistory",
028 |     "MaintenanceSchedule",
029 |     "Part",
030 |     "PartMovement",
031 |     "PartReservation",
032 |     "Printer",
033 |     "PurchaseRequisition",
034 |     "PrinterLocation",
035 |     "PrinterHealth",
036 |     "Technician",
037 |     "TechnicianSpecialty",
038 |     "User",
039 | ]

```

=== ARQUIVO: .lapl\services\activity_service.py ===

```

001 | from __future__ import annotations
002 |
003 | import logging
004 |
005 | from db import safe_commit
006 |
007 | log = logging.getLogger(__name__)
008 | import calendar
009 | from datetime import datetime
010 | from typing import TYPE_CHECKING, Any, Optional
011 |
012 | from app.models.base import utcnow
013 | from sqlalchemy import func
014 | from sqlalchemy.orm import Session, selectinload
015 |
016 | from app.models import Activity, Printer
017 | from app.utils.cache import cached, invalidate
018 | from app.utils.sanitize import sanitizar
019 |
020 | if TYPE_CHECKING:
021 |     from app.services.audit_service import AuditService
022 |
023 |
024 | class ActivityService:
025 |     def __init__(self, session: Session, audit_service: Optional[AuditService] = None, user_id: Optional[int] = None):
026 |         self.session: Session = session
027 |         self.audit_service: Optional[AuditService] = audit_service
028 |         self.user_id: Optional[int] = user_id
029 |
030 |     def buscar_por_id(self, activity_id: int) -> Optional[Activity]:

```

```

031 |         return self.session.query(Activity).options(
032 |             selectinload(Activity.printer)
033 |         ).filter(Activity.deleted_at == None, Activity.id == activity_id).first()
034 |
035 | def listar(self, filtro_tipo: Optional[str] = None, limite: int = 200, offset: Optional[int] = None) ->
036 |     query = self.session.query(Activity).filter(Activity.deleted_at == None).options(
037 |         selectinload(Activity.printer)
038 |     ).order_by(Activity.event_at.desc())
039 |     if filtro_tipo and filtro_tipo != "TODAS":
040 |         query = query.filter(Activity.kind == filtro_tipo)
041 |     if limite is not None:
042 |         query = query.limit(limite)
043 |     if offset is not None:
044 |         query = query.offset(offset)
045 |     return query.all()
046 |
047 | def listar_por_status(self, status: str, limite: int = 200, offset: Optional[int] = None) -> list[Activ
048 |     query = self.session.query(Activity).filter(Activity.deleted_at == None).options(
049 |         selectinload(Activity.printer)
050 |     ).order_by(Activity.event_at.desc())
051 |     query = query.filter(Activity.status_atividade.in_([status, status.lower(), status.capitalize()]))
052 |     if limite is not None:
053 |         query = query.limit(limite)
054 |     if offset is not None:
055 |         query = query.offset(offset)
056 |     return query.all()
057 |
058 | def listar_movimentacoes(self, limite: int = 100, offset: Optional[int] = None) -> list[Activity]:
059 |     query = self.session.query(Activity).filter(Activity.deleted_at == None).filter(
060 |         Activity.kind == "MOVIMENTACAO"
061 |     ).order_by(Activity.event_at.desc())
062 |     if limite is not None:
063 |         query = query.limit(limite)
064 |     if offset is not None:
065 |         query = query.offset(offset)
066 |     return query.all()
067 |
068 | def listar_movimentacoes_por_tipo(self, tipo: str, limite: int = 100, offset: Optional[int] = None) ->
069 |     query = self.session.query(Activity).filter(
070 |         Activity.deleted_at == None, Activity.kind == "MOVIMENTACAO"
071 |     )
072 |     if tipo == "saida":
073 |         query = query.filter(Activity.from_location != "", Activity.to_location == "")
074 |     elif tipo == "entrada":
075 |         query = query.filter(Activity.to_location != "", Activity.from_location == "")
076 |     elif tipo == "pendente":
077 |         query = query.filter(Activity.status_atividade == "Aberta")
078 |     query = query.order_by(Activity.event_at.desc())
079 |     if limite is not None:
080 |         query = query.limit(limite)
081 |     if offset is not None:
082 |         query = query.offset(offset)
083 |     return query.all()
084 |
085 | def contar_por_tipo(self, tipo: str) -> int:
086 |     query = self.session.query(Activity).filter(
087 |         Activity.deleted_at == None, Activity.kind == "MOVIMENTACAO"
088 |     )
089 |     if tipo == "saida":
090 |         query = query.filter(Activity.from_location != "", Activity.to_location == "")
091 |     elif tipo == "entrada":
092 |         query = query.filter(Activity.to_location != "", Activity.from_location == "")
093 |     elif tipo == "pendente":

```

```

094 |         query = query.filter(Activity.status_atividade == "Aberta")
095 |     return query.count()
096 |
097 | def listar_por_impressora(self, printer_id: str, limite: Optional[int] = None, offset: Optional[int] =
098 |     query = self.session.query(Activity).filter(Activity.deleted_at == None).filter(
099 |         Activity.printer_id == printer_id
100 |     ).order_by(Activity.event_at.desc())
101 |     if limite is not None:
102 |         query = query.limit(limite)
103 |     if offset is not None:
104 |         query = query.offset(offset)
105 |     return query.all()
106 |
107 | def buscar_ultima_manutencao(self, printer_id: str) -> Optional[Activity]:
108 |     return self.session.query(Activity).filter(Activity.deleted_at == None).filter(
109 |         Activity.printer_id == printer_id,
110 |         Activity.kind == "MANUTENCAO"
111 |     ).order_by(Activity.event_at.desc()).first()
112 |
113 | def buscar_por_descricao(self, printer_id: str, descricao: str) -> Optional[Activity]:
114 |     return self.session.query(Activity).filter(Activity.deleted_at == None).filter(
115 |         Activity.printer_id == printer_id,
116 |         Activity.notes == descricao
117 |     ).order_by(Activity.event_at.desc()).first()
118 |
119 | def criar(self, printer_id: str, kind: str, notes: str = "",
120 |         parts_used: str = "", from_location: str = "", to_location: str = "",
121 |         numero_recibo: str = "", status_atividade: str = "Aberta",
122 |         event_at: Optional[datetime] = None, tecnico_id: Optional[int] = None,
123 |         procedimentos: str = "", sintoma_relato: str = "",
124 |         diagnostico_tecnico: str = "", solucao_aplicada: str = "",
125 |         inicio_atendimento: Optional[datetime] = None,
126 |         fim_atendimento: Optional[datetime] = None,
127 |         urgencia: str = "Normal",
128 |         os_vinculada_id: Optional[int] = None) -> Activity:
129 |     atividade = Activity(
130 |         printer_id=printer_id,
131 |         kind=sanitizar(kind, "Activity", "kind"),
132 |         event_at=event_at or utcnow(),
133 |         notes=sanitizar(notes),
134 |         parts_used=sanitizar(parts_used),
135 |         from_location=sanitizar(from_location, "Activity", "from_location"),
136 |         to_location=sanitizar(to_location, "Activity", "to_location"),
137 |         numero_recibo=sanitizar(numero_recibo, "Activity", "numero_recibo"),
138 |         status_atividade=sanitizar(status_atividade, "Activity", "status_atividade"),
139 |         tecnico_id=tecnico_id,
140 |         procedimentos=procedimentos,
141 |         urgencia=sanitizar(urgencia, "Activity", "urgencia") if isinstance(urgencia, str) else urgencia,
142 |         sintoma_relato=sanitizar(sintoma_relato),
143 |         diagnostico_tecnico=sanitizar(diagnostico_tecnico),
144 |         solucao_aplicada=sanitizar(solucao_aplicada),
145 |         inicio_atendimento=inicio_atendimento,
146 |         fim_atendimento=fim_atendimento,
147 |         os_vinculada_id=os_vinculada_id,
148 |     )
149 |     self.session.add(atividade)
150 |     safe_commit(self.session)
151 |     if self.audit_service:
152 |         self.audit_service.log(self.user_id, "criar", tabela_alvo="activities", registro_id=atividade.id)
153 |     invalidate("activity")
154 |     invalidate("dashboard")
155 |     return atividade
156 |

```

```

157 | def listar_vinculadas(self, activity_id: int, limite: int = 20) -> list[Activity]:
158 |     """Retorna OSs que estão vinculadas à activity_id (filhas + pai)."""
159 |     activity = self.buscar_por_id(activity_id)
160 |     if not activity:
161 |         return []
162 |     resultado: list[Activity] = []
163 |     if activity.os_vinculada_id:
164 |         pai = self.buscar_por_id(activity.os_vinculada_id)
165 |         if pai:
166 |             resultado.append(pai)
167 |     filhas = (
168 |         self.session.query(Activity)
169 |         .filter(Activity.os_vinculada_id == activity_id, Activity.deleted_at == None)
170 |         .limit(limite)
171 |         .all()
172 |     )
173 |     resultado.extend(filhas)
174 |     return resultado
175 |
176 | def listar_candidatas_vinculo(self, printer_id: str, excluir_id: Optional[int] = None, limite: int = 30) -> list[Activity]:
177 |     """Retorna OSs recentes da mesma impressora para vínculo."""
178 |     q = self.session.query(Activity).filter(
179 |         Activity.printer_id == printer_id, Activity.deleted_at == None
180 |     )
181 |     if excluir_id:
182 |         q = q.filter(Activity.id != excluir_id)
183 |     q = q.order_by(Activity.event_at.desc()).limit(limite)
184 |     return q.all()
185 |
186 | def atualizar(self, atividade: Activity, **kwargs: Any) -> None:
187 |     if self.audit_service:
188 |         dados_antes = {chave: getattr(atividade, chave, None) for chave in kwargs}
189 |         for chave, valor in kwargs.items():
190 |             if hasattr(atividade, chave):
191 |                 if isinstance(valor, str):
192 |                     valor = sanitizar(valor, "Activity", chave)
193 |                 setattr(atividade, chave, valor)
194 |         safe_commit(self.session)
195 |     if self.audit_service:
196 |         self.audit_service.log(self.user_id, "atualizar", tabela_alvo="activities", registro_id=atividade.id)
197 |         invalidate("activity")
198 |         invalidate("dashboard")
199 |
200 | def excluir(self, atividade: Activity) -> None:
201 |     if self.audit_service:
202 |         self.audit_service.log(self.user_id, "excluir", tabela_alvo="activities", registro_id=atividade.id)
203 |         atividade.deleted_at = utcnow()
204 |         safe_commit(self.session)
205 |         invalidate("activity")
206 |         invalidate("dashboard")
207 |
208 | @cached(ttl=15, namespace="activity")
209 | def contar_total(self) -> int:
210 |     return self.session.query(Activity).filter(Activity.deleted_at == None).count()
211 |
212 | @cached(ttl=15, namespace="activity")
213 | def contar_por_status(self, status: str) -> int:
214 |     return self.session.query(Activity).filter(
215 |         Activity.deleted_at == None,
216 |         Activity.status_atividade.in_([status, status.lower(), status.capitalize()])
217 |     ).count()
218 |
219 | @cached(ttl=60, namespace="activity")

```

```

220 | def contar_por_mes(self, ano: int, mes: int, kind: Optional[str] = None) -> int:
221 |     inicio = datetime(ano, mes, 1)
222 |     fim = datetime(ano + 1, 1, 1) if mes == 12 else datetime(ano, mes + 1, 1)
223 |     query = self.session.query(Activity).filter(
224 |         Activity.deleted_at == None,
225 |         Activity.event_at >= inicio, Activity.event_at < fim
226 |     )
227 |     if kind:
228 |         query = query.filter(Activity.kind == kind)
229 |     return query.count()
230 |
231 | @cached(ttl=60, namespace="activity")
232 | def contar_ultimos_6_meses(self) -> dict[str, list]:
233 |     hoje = datetime.now()
234 |     resultado: dict[str, list] = {"manut": [], "mov": [], "labels": []}
235 |
236 |     mes_start = hoje.month - 5
237 |     ano_start = hoje.year
238 |     while mes_start <= 0:
239 |         mes_start += 12
240 |         ano_start -= 1
241 |     inicio = datetime(ano_start, mes_start, 1)
242 |
243 |     rows = (
244 |         self.session.query(
245 |             func.strftime("%Y-%m", Activity.event_at),
246 |             Activity.kind,
247 |             func.count(Activity.id),
248 |         )
249 |         .filter(
250 |             Activity.deleted_at == None,
251 |             Activity.event_at >= inicio,
252 |             Activity.kind.in_(["MANUTENCAO", "MOVIMENTACAO"]),
253 |         )
254 |         .group_by(func.strftime("%Y-%m", Activity.event_at), Activity.kind)
255 |         .order_by(func.strftime("%Y-%m", Activity.event_at))
256 |         .all()
257 |     )
258 |
259 |     data_map: dict[str, dict[str, int]] = {}
260 |     for month_str, kind, count in rows:
261 |         if month_str not in data_map:
262 |             data_map[month_str] = {"manut": 0, "mov": 0}
263 |         if kind == "MANUTENCAO":
264 |             data_map[month_str]["manut"] = count
265 |         elif kind == "MOVIMENTACAO":
266 |             data_map[month_str]["mov"] = count
267 |
268 |     for i in range(5, -1, -1):
269 |         mes_n = hoje.month - i
270 |         ano_n = hoje.year
271 |         while mes_n <= 0:
272 |             mes_n += 12
273 |             ano_n -= 1
274 |         resultado["labels"].append(calendar.month_abbr[mes_n])
275 |         key = f"{ano_n:04d}-{mes_n:02d}"
276 |         month_data = data_map.get(key, {"manut": 0, "mov": 0})
277 |         resultado["manut"].append(month_data["manut"])
278 |         resultado["mov"].append(month_data["mov"])
279 |
280 |     return resultado
281 |
282 | def buscar_por_filtro_busca(self, texto: str, limite: int = 200, offset: Optional[int] = None) -> list[

```

```

283 |         query = self.session.query(Activity).filter(Activity.deleted_at == None).order_by(Activity.event_at)
284 |         printers = self.session.query(Printer).filter(
285 |             Printer.patrimonio.like(f"%{texto}%")
286 |         ).all()
287 |         printer_ids = [p.id for p in printers]
288 |         if printer_ids:
289 |             query = query.filter(Activity.printer_id.in_(printer_ids))
290 |         else:
291 |             query = query.filter(Activity.printer_id == "NENHUM")
292 |         if limite is not None:
293 |             query = query.limit(limite)
294 |         if offset is not None:
295 |             query = query.offset(offset)
296 |         return query.all()
297 |
298 | def buscar_movimentacoes_por_filtro(self, texto: str, limite: int = 100, offset: Optional[int] = None)
299 |     filtro = f"%{texto}%"
300 |     query = self.session.query(Activity).filter(Activity.kind == "MOVIMENTACAO")
301 |     printers = self.session.query(Printer).filter(
302 |         Printer.patrimonio.like(filtro)
303 |     ).all()
304 |     printer_ids = [p.id for p in printers]
305 |     if printer_ids:
306 |         query = query.filter(
307 |             Activity.printer_id.in_(printer_ids) |
308 |             Activity.from_location.like(filtro) |
309 |             Activity.to_location.like(filtro) |
310 |             Activity.parts_used.like(filtro) |
311 |             Activity.notes.like(filtro) |
312 |             Activity.numero_recibo.like(filtro)
313 |         )
314 |     else:
315 |         query = query.filter(
316 |             Activity.from_location.like(filtro) |
317 |             Activity.to_location.like(filtro) |
318 |             Activity.parts_used.like(filtro) |
319 |             Activity.notes.like(filtro) |
320 |             Activity.numero_recibo.like(filtro)
321 |         )
322 |     query = query.order_by(Activity.event_at.desc())
323 |     if limite is not None:
324 |         query = query.limit(limite)
325 |     if offset is not None:
326 |         query = query.offset(offset)
327 |     return query.all()
328 |
329 | def buscar_por_origem_destino(self, origem: str, destino: str) -> Optional[Activity]:
330 |     return self.session.query(Activity).filter(
331 |         Activity.kind == "MOVIMENTACAO",
332 |         Activity.from_location == origem,
333 |         Activity.to_location == destino
334 |     ).order_by(Activity.event_at.desc()).first()
335 |
336 | def listar_por_tecnico(self, tecnico_id: int, limite: int = 200) -> list[Activity]:
337 |     return self.session.query(Activity).filter(Activity.deleted_at == None).filter(
338 |         Activity.tecnico_id == tecnico_id
339 |     ).order_by(Activity.event_at.desc()).limit(limite).all()
340 |
341 | def listar_por_tecnico_e_status(self, tecnico_id: int, status: str, limite: int = 200) -> list[Activity]:
342 |     return self.session.query(Activity).filter(
343 |         Activity.tecnico_id == tecnico_id,
344 |         Activity.status_atividade.in_([status, status.lower(), status.capitalize()])
345 |     ).order_by(Activity.event_at.desc()).limit(limite).all()

```

```

346 |
347 | def listar_por_tecnico_e_tipo(self, tecnico_id: int, kind: str, limite: int = 200) -> list[Activity]:
348 |     return self.session.query(Activity).filter(
349 |         Activity.tecnico_id == tecnico_id,
350 |         Activity.kind == kind
351 |     ).order_by(Activity.event_at.desc()).limit(limite).all()
352 |
353 | def contar_por_tecnico(self, tecnico_id: int) -> int:
354 |     return self.session.query(Activity).filter(
355 |         Activity.deleted_at == None,
356 |         Activity.tecnico_id == tecnico_id
357 |     ).count()
358 |
359 | def contar_por_tecnico_por_status(self, tecnico_id: int, status: str) -> int:
360 |     return self.session.query(Activity).filter(
361 |         Activity.deleted_at == None,
362 |         Activity.tecnico_id == tecnico_id,
363 |         Activity.status_atividade.in_([status, status.lower(), status.capitalize()])
364 |     ).count()
365 |
366 | def listar_por_tecnico_e_data(self, tecnico_id: int, data: datetime, limite: int = 200) -> list[Activity]:
367 |     inicio = datetime(data.year, data.month, data.day, 0, 0, 0)
368 |     fim = datetime(data.year, data.month, data.day, 23, 59, 59)
369 |     return self.session.query(Activity).filter(
370 |         Activity.deleted_at == None,
371 |         Activity.tecnico_id == tecnico_id,
372 |         Activity.event_at >= inicio,
373 |         Activity.event_at <= fim,
374 |     ).order_by(Activity.from_location, Activity.event_at).limit(limite).all()
375 |
376 | def contar_por_tecnico_por_data(self, tecnico_id: int, data: datetime) -> int:
377 |     inicio = datetime(data.year, data.month, data.day, 0, 0, 0)
378 |     fim = datetime(data.year, data.month, data.day, 23, 59, 59)
379 |     return self.session.query(Activity).filter(
380 |         Activity.deleted_at == None,
381 |         Activity.tecnico_id == tecnico_id,
382 |         Activity.event_at >= inicio,
383 |         Activity.event_at <= fim,
384 |     ).count()
385 |
386 | def listar_concluidas_por_tecnico_no_periodo(
387 |     self, tecnico_id: int, inicio: datetime, fim: datetime, limite: int = 500
388 | ) -> list[Activity]:
389 |     return self.session.query(Activity).filter(
390 |         Activity.deleted_at == None,
391 |         Activity.tecnico_id == tecnico_id,
392 |         Activity.kind == "MANUTENCAO",
393 |         Activity.status_atividade.in_(["Concluido", "Concluído", "Verificada"]),
394 |         Activity.fim_atendimento >= inicio,
395 |         Activity.fim_atendimento <= fim,
396 |     ).order_by(Activity.fim_atendimento.desc()).limit(limite).all()
397 |
398 | def listar_pecas_por_tecnico(self, tecnico_id: int, limite: int = 20) -> list[tuple[str, int]]:
399 |     from collections import Counter
400 |     pecas = Counter()
401 |     atividades = self.session.query(Activity).filter(
402 |         Activity.deleted_at == None,
403 |         Activity.tecnico_id == tecnico_id,
404 |         Activity.parts_used != "",
405 |     ).all()
406 |     for a in atividades:
407 |         if a.parts_used:
408 |             for p in a.parts_used.split(","):

```

```

409 |         p = p.strip()
410 |         if p:
411 |             pecas[p] += 1
412 |     return pecas.most_common(limite)
413 |
414 | def listar_tecnicos_por_pecas(self, nome_pecas: str, limite: int = 50) -> list[Activity]:
415 |     filtro = f"%{nome_pecas}%"
416 |     resultados = self.session.query(Activity).options(
417 |         selectinload(Activity.technician)
418 |     ).filter(
419 |         Activity.deleted_at == None,
420 |         Activity.parts_used.like(filtro),
421 |     ).order_by(Activity.event_at.desc()).limit(limite * 5).all()
422 |     nome_pecas_lower = nome_pecas.strip().lower()
423 |     return [
424 |         a for a in resultados
425 |         if a.technician and any(p.strip().lower() == nome_pecas_lower for p in (a.parts_used or "").split(", "))
426 |    ][:limite]
427 |
428 | def listar_os_abertas_por_tecnico(self, tecnico_id: int, limite: int = 100) -> list[Activity]:
429 |     excluidos = ["Concluido", "Concluído", "Verificada"]
430 |     return self.session.query(Activity).options(
431 |         selectinload(Activity.printer)
432 |     ).filter(
433 |         Activity.deleted_at == None,
434 |         Activity.tecnico_id == tecnico_id,
435 |         ~Activity.status_atividade.in_(excluidos),
436 |     ).order_by(Activity.event_at.desc()).limit(limite).all()
437 |
438 | def contar_por_impressora_e_kind(self, printer_id: str, kind: str) -> int:
439 |     return self.session.query(Activity).filter(
440 |         Activity.deleted_at == None,
441 |         Activity.printer_id == printer_id,
442 |         Activity.kind == kind
443 |     ).count()
444 |
445 | def listar_por_pecas(self, nome_pecas: str, limite: int = 100) -> list[Activity]:
446 |     filtro = f"%{nome_pecas}%"
447 |     resultados = self.session.query(Activity).options(
448 |         selectinload(Activity.printer)
449 |     ).filter(
450 |         Activity.deleted_at == None, Activity.parts_used.like(filtro)
451 |     ).order_by(Activity.event_at.desc()).limit(limite * 5).all()
452 |     nome_pecas_lower = nome_pecas.strip().lower()
453 |     filtrados = [
454 |         a for a in resultados
455 |         if any(p.strip().lower() == nome_pecas_lower for p in (a.parts_used or "").split(", "))
456 |     ]
457 |     return filtrados[:limite]
458 |
459 | def listar_por_pecas_com_relacionadas(self, nome_pecas: str, limite: int = 100) -> tuple[list[Activity],
460 |     diretas = self.listar_por_pecas(nome_pecas, limite=limite)
461 |     printer_ids = list({a.printer_id for a in diretas if a.printer_id})
462 |     if not printer_ids:
463 |         return diretas, []
464 |     diretas_ids = {a.id for a in diretas}
465 |     query = self.session.query(Activity).options(
466 |         selectinload(Activity.printer)
467 |     ).filter(Activity.deleted_at == None, Activity.printer_id.in_(printer_ids))
468 |     if diretas_ids:
469 |         query = query.filter(~Activity.id.in_(diretas_ids))
470 |     relacionadas = query.order_by(Activity.event_at.desc()).limit(limite).all()
471 |     return diretas, relacionadas

```



```

472 |
473 | def top_pecas_trocadas(self, limite: int = 8) -> list[tuple[str, int]]:
474 |     from collections import Counter
475 |     pecas_count = Counter()
476 |     atividades = self.session.query(Activity).filter(
477 |         Activity.deleted_at == None, Activity.parts_used != ""
478 |     ).all()
479 |     for a in atividades:
480 |         if a.parts_used:
481 |             for peca in a.parts_used.split(','):
482 |                 peca = peca.strip()
483 |                 if peca:
484 |                     pecas_count[peca] += 1
485 |     return pecas_count.most_common(limite)
486 |
487 | def listar_excluidos(self, limite: int = 50) -> list[Activity]:
488 |     return self.session.query(Activity).filter(
489 |         Activity.deleted_at != None
490 |     ).order_by(Activity.deleted_at.desc()).limit(limite).all()
491 |
492 | def restaurar(self, obj: Activity) -> None:
493 |     obj.deleted_at = None
494 |     safe_commit(self.session)
495 |     if self.audit_service:
496 |         self.audit_service.log(self.user_id, "restaurar", tabela_alvo="activities", registro_id=obj.id)
497 |     invalidate("activity")
498 |     invalidate("dashboard")
499 |

```

=== ARQUIVO: .\appls\services\alert_service.py ===

```

001 | from __future__ import annotations
002 |
003 | import logging
004 |
005 | from db import safe_commit
006 |
007 | log = logging.getLogger(__name__)
008 | from app.models.base import utcnow
009 | from datetime import datetime, timedelta
010 | from typing import TYPE_CHECKING, Any, Optional
011 |
012 | from sqlalchemy.orm import Session, selectinload
013 |
014 | from app.models import Alert, Printer
015 | from app.utils.cache import cached, invalidate
016 | from app.utils.sanitize import sanitizar
017 |
018 | if TYPE_CHECKING:
019 |     from app.models.part import Part
020 |     from app.models.user import User
021 |     from app.services.audit_service import AuditService
022 |     from app.services.notification_service import NotificadorService
023 |
024 |
025 | class AlertService:
026 |     def __init__(self, session: Session, audit_service: Optional[AuditService] = None, user_id: Optional[int] = None):
027 |         self.session: Session = session
028 |         self.audit_service: Optional[AuditService] = audit_service
029 |         self.user_id: Optional[int] = user_id
030 |         self.notificador: Optional[NotificadorService] = notificador

```

```

031 |
032 | def listar_todos(self, apenas_pendentes: bool = False, apenas_resolvidos: bool = False, filtro_busca: Optional[str] = None, limite: Optional[int] = None, offset: Optional[int] = None) -> List[Alert]:
033 |     query = self.session.query(Alert).filter(Alert.deleted_at == None).options(
034 |         selectinload(Alert.printer), selectinload(Alert.part)
035 |     ).order_by(Alert.created_at.desc())
036 |     if apenas_pendentes:
037 |         query = query.filter(Alert.resolvido == False)
038 |     if apenas_resolvidos:
039 |         query = query.filter(Alert.resolvido == True)
040 |     if filtro_busca:
041 |         f = f"%{filtro_busca}%"
042 |         query = query.filter(
043 |             Alert.titulo.like(f) | Alert.tipo.like(f) | Alert.descricao.like(f)
044 |         )
045 |     if limite is not None:
046 |         query = query.limit(limite)
047 |     if offset is not None:
048 |         query = query.offset(offset)
049 |     return query.all()
050 |
051 | def listar_por_impressora(self, printer_id: Optional[str], limite: Optional[int] = None, offset: Optional[int] = None) -> List[Alert]:
052 |     query = self.session.query(Alert).filter(Alert.deleted_at == None).filter(
053 |         Alert.printer_id == printer_id
054 |     ).order_by(Alert.created_at.desc())
055 |     if limite is not None:
056 |         query = query.limit(limite)
057 |     if offset is not None:
058 |         query = query.offset(offset)
059 |     return query.all()
060 |
061 | def buscar_por_id(self, alert_id: int) -> Optional[Alert]:
062 |     return self.session.query(Alert).filter(Alert.deleted_at == None, Alert.id == alert_id).first()
063 |
064 | def criar(self, tipo: str, titulo: str, descricao: str = "", printer_id: Optional[str] = None, part_id: Optional[int] = None) -> Alert:
065 |     alerta = Alert(
066 |         printer_id=printer_id,
067 |         part_id=part_id,
068 |         tipo=sanitizar(tipo, "Alert", "tipo"),
069 |         titulo=sanitizar(titulo, "Alert", "titulo"),
070 |         descricao=sanitizar(descricao),
071 |         data_alerta=data_alerta or datetime.now(),
072 |         data_agendada=data_agendada,
073 |     )
074 |     self.session.add(alerta)
075 |     safe_commit(self.session)
076 |     if self.audit_service:
077 |         self.audit_service.log(self.user_id, "criar", tabela_alvo="alerts", registro_id=alerta.id, dado=alerta.__dict__)
078 |         invalidate("alert")
079 |         invalidate("dashboard")
080 |
081 |     if not data_agendada and self.notificador:
082 |         try:
083 |             self.notificador.notificar_alerta(alerta)
084 |             alerta.notificado = True
085 |             alerta.notificado_em = datetime.now()
086 |             safe_commit(self.session)
087 |         except Exception as e:
088 |             log.warning("Erro ao notificar alerta %s: %s", alerta.id, e)
089 |
090 |     return alerta
091 |
092 | def resolver(self, alerta: Alert, user_id: Optional[int] = None) -> None:
093 |     if self.audit_service:

```

```

094 |         self.audit_service.log(user_id or self.user_id, "resolver", tabela_alvo="alerts", registro_id=a
095 |         alerta.resolvido = True
096 |         alerta.resolvido_em = datetime.now()
097 |         alerta.resolvido_por = user_id
098 |         safe_commit(self.session)
099 |         invalidate("alert")
100 |
101 | def excluir(self, alerta: Alert) -> None:
102 |     if self.audit_service:
103 |         self.audit_service.log(self.user_id, "excluir", tabela_alvo="alerts", registro_id=alerta.id, da
104 |         alerta.deleted_at = utcnow()
105 |         safe_commit(self.session)
106 |         invalidate("alert")
107 |
108 | def contar_todos(self, apenas_pendentes: bool = False, apenas_resolvidos: bool = False, filtro_busca: O
109 |     query = self.session.query(Alert).filter(Alert.deleted_at == None)
110 |     if apenas_pendentes:
111 |         query = query.filter(Alert.resolvido == False)
112 |     if apenas_resolvidos:
113 |         query = query.filter(Alert.resolvido == True)
114 |     if filtro_busca:
115 |         f = f"%{filtro_busca}%"
116 |         query = query.filter(
117 |             Alert.titulo.like(f) | Alert.tipo.like(f) | Alert.descricao.like(f)
118 |         )
119 |     return query.count()
120 |
121 | @cached(ttl=15, namespace="alert")
122 | def contar_pendentes(self) -> int:
123 |     return self.session.query(Alert).filter(
124 |         Alert.deleted_at == None, Alert.resolvido == False
125 |     ).count()
126 |
127 | def gerar_alertas_revisao(self, dias_limite: int = 30) -> int:
128 |     hoje = datetime.now()
129 |     limite = hoje + timedelta(days=dias_limite)
130 |     printers = self.session.query(Printer).filter(
131 |         Printer.proxima_revisao <= limite
132 |     ).all()
133 |     criados = 0
134 |     for p in printers:
135 |         if not p.proxima_revisao:
136 |             continue
137 |         existente = self.session.query(Alert).filter(
138 |             Alert.printer_id == p.id,
139 |             Alert.tipo == "revisao",
140 |             Alert.resolvido == False
141 |         ).first()
142 |         if existente:
143 |             continue
144 |         self.criar(
145 |             printer_id=p.id,
146 |             tipo="revisao",
147 |             titulo=f"Revisão próxima: {p.patrimonio}",
148 |             descricao=f"Impressora {p.modelo} ({p.patrimonio}) precisa de revisão até {p.proxima_revisa
149 |             data_alerta=p.proxima_revisao,
150 |             data_agendada=p.proxima_revisao,
151 |         )
152 |         criados += 1
153 |     return criados
154 |
155 | def listar_excluidos(self, limite: int = 50) -> list[Alert]:
156 |     return self.session.query(Alert).filter(

```

```

157 |         Alert.deleted_at != None
158 |     ).order_by(Alert.deleted_at.desc()).limit(limite).all()
159 |
160 | def restaurar(self, obj: Alert) -> None:
161 |     obj.deleted_at = None
162 |     safe_commit(self.session)
163 |     if self.audit_service:
164 |         self.audit_service.log(self.user_id, "restaurar", tabela_alvo="alerts", registro_id=obj.id)
165 |     invalidate("alert")
166 |
167 | def verificar_estoque_baixo(self) -> int:
168 |     from app.models import Part
169 |     from sqlalchemy import or_
170 |     partes = self.session.query(Part).filter(
171 |         or_(
172 |             Part.quantidade_estoque <= 0,
173 |             Part.quantidade_estoque <= Part.estoque_minimo,
174 |         )
175 |     ).all()
176 |     criados = 0
177 |     for peca in partes:
178 |         existente = self.session.query(Alert).filter(
179 |             Alert.tipo == "estoque",
180 |             Alert.resolvido == False,
181 |             Alert.titulo.like(f"%{peca.nome}%"),
182 |         ).first()
183 |         if existente:
184 |             continue
185 |         self.criar(
186 |             part_id=peca.id,
187 |             tipo="estoque",
188 |             titulo=f"Estoque baixo: {peca.nome}",
189 |             descricao=f"Peça {peca.nome} (cód. {peca.codigo}) tem apenas {peca.quantidade_estoque} unid
190 |         )
191 |         criados += 1
192 |     return criados
193 |
194 | def verificar_agendados(self) -> list[Alert]:
195 |     hoje = datetime.now().replace(hour=0, minute=0, second=0, microsecond=0)
196 |     amanha = hoje + timedelta(days=1)
197 |     alerts = self.session.query(Alert).filter(
198 |         Alert.deleted_at == None,
199 |         Alert.resolvido == False,
200 |         Alert.notificado == False,
201 |         Alert.data_agendada != None,
202 |         Alert.data_agendada <= amanha,
203 |     ).options(selectinload(Alert.printer), selectinload(Alert.part)).all()
204 |
205 |     notificados = []
206 |     for a in alerts:
207 |         if self.notificador:
208 |             try:
209 |                 self.notificador.notificar_alerta(a)
210 |                 a.notificado = True
211 |                 a.notificado_em = datetime.now()
212 |                 safe_commit(self.session)
213 |                 notificados.append(a)
214 |             except Exception as e:
215 |                 log.warning("Erro ao notificar alerta agendado %s: %s", a.id, e)
216 |     return notificados
217 |

```

=== ARQUIVO: .\app\services\audit_service.py ===

```
001 | ■from __future__ import annotations
002 |
003 | import json
004 | import logging
005 |
006 | from typing import Any, Optional
007 |
008 | from db import safe_commit
009 | from sqlalchemy.orm import Session
010 |
011 | from app.models import AuditLog
012 |
013 | log = logging.getLogger(__name__)
014 |
015 |
016 | class AuditService:
017 |     def __init__(self, session: Session) -> None:
018 |         self.session: Session = session
019 |
020 |     def log(self, user_id: Optional[int], acao: str, tabela_alvo: str = "", registro_id: str = "",
021 |            ip_address: str = "", dados_antes: Any = None, dados_depois: Any = None) -> AuditLog:
022 |         def serializar(obj):
023 |             if obj is None:
024 |                 return None
025 |             if isinstance(obj, dict):
026 |                 return {k: str(v) if hasattr(v, 'isoformat') else v for k, v in obj.items()}
027 |             if hasattr(obj, '__dict__'):
028 |                 return {k: str(v) if hasattr(v, 'isoformat') else v
029 |                         for k, v in obj.__dict__.items() if not k.startswith('_')}
030 |             return str(obj)
031 |
032 |         registro = AuditLog(
033 |             user_id=user_id,
034 |             acao=acao,
035 |             tabela_alvo=tabela_alvo,
036 |             registro_id=str(registro_id) if registro_id else "",
037 |             ip_address=ip_address,
038 |             dados_antes=json.dumps(serializar(dados_antes), ensure_ascii=False, default=str) if dados_antes
039 |             dados_depois=json.dumps(serializar(dados_depois), ensure_ascii=False, default=str) if dados_dep
040 |         )
041 |         self.session.add(registro)
042 |         safe_commit(self.session)
043 |         return registro
044 |
045 |     def listar(self, limite: int = 100) -> list[AuditLog]:
046 |         return self.session.query(AuditLog).order_by(
047 |             AuditLog.created_at.desc()
048 |         ).limit(limite).all()
049 |
050 |     def listar_por_tabela(self, tabela: str, limite: int = 50) -> list[AuditLog]:
051 |         return self.session.query(AuditLog).filter(
052 |             AuditLog.tabela_alvo == tabela
053 |         ).order_by(AuditLog.created_at.desc()).limit(limite).all()
054 |
055 |     def listar_por_usuario(self, user_id: int, limite: int = 50) -> list[AuditLog]:
056 |         return self.session.query(AuditLog).filter(
057 |             AuditLog.user_id == user_id
058 |         ).order_by(AuditLog.created_at.desc()).limit(limite).all()
059 |
```

=== ARQUIVO: .lapp\services\company_service.py ===

```
001 | ■from __future__ import annotations
002 |
003 | import logging
004 | from datetime import datetime
005 | from typing import TYPE_CHECKING, Any, Optional
006 |
007 | from app.models.base import utcnow
008 | from db import safe_commit
009 | from sqlalchemy.orm import Session
010 |
011 | from app.models import Company
012 | from app.utils.cache import cached, invalidate
013 | from app.utils.sanitizar import sanitizar
014 |
015 | if TYPE_CHECKING:
016 |     from app.services.audit_service import AuditService
017 |
018 | log = logging.getLogger(__name__)
019 |
020 |
021 | class CompanyService:
022 |     def __init__(self, session: Session, audit_service: Optional[AuditService] = None, user_id: Optional[int] = None):
023 |         self.session: Session = session
024 |         self.audit_service: Optional[AuditService] = audit_service
025 |         self.user_id: Optional[int] = user_id
026 |
027 |     @cached(ttl=60, namespace="company")
028 |     def listar_todas(self, limite: Optional[int] = None, offset: Optional[int] = None) -> list[Company]:
029 |         query = self.session.query(Company).filter(Company.deleted_at == None).order_by(Company.nome)
030 |         if limite is not None:
031 |             query = query.limit(limite)
032 |         if offset is not None:
033 |             query = query.offset(offset)
034 |         return query.all()
035 |
036 |     def buscar_por_nome(self, nome: str) -> Optional[Company]:
037 |         return self.session.query(Company).filter(Company.deleted_at == None, Company.nome == nome).first()
038 |
039 |     def criar(self, nome: str, cnpj: str = "", telefone: str = "", email: str = "", tipo: str = "Cliente",
040 |               empresa = Company(
041 |                   nome=sanitizar(nome, "Company", "nome"),
042 |                   cnpj=sanitizar(cnpj, "Company", "cnpj"),
043 |                   telefone=sanitizar(telefone, "Company", "telefone"),
044 |                   email=sanitizar(email, "Company", "email"),
045 |                   tipo=sanitizar(tipo, "Company", "tipo"),
046 |                   endereco=sanitizar(endereco, "Company", "endereco"),
047 |                   cidade=sanitizar(cidade, "Company", "cidade"),
048 |                   uf=sanitizar(uf, "Company", "uf"),
049 |                   observacao=sanitizar(observacao, "Company", "observacao"),
050 |               )
051 |         self.session.add(empresa)
052 |         safe_commit(self.session)
053 |         if self.audit_service:
054 |             self.audit_service.log(self.user_id, "criar", tabela_alvo="companies", registro_id=empresa.id,
055 |                                   invalidate("company"))
056 |         return empresa
057 |
058 |     def atualizar(self, empresa: Company, **kwargs: Any) -> None:
059 |         if self.audit_service:
060 |             dados_antes = {chave: getattr(empresa, chave, None) for chave in kwargs}
061 |             for chave, valor in kwargs.items():
```

```

062 |         if hasattr(empresa, chave):
063 |             if isinstance(valor, str):
064 |                 valor = sanitizar(valor, "Company", chave)
065 |                 setattr(empresa, chave, valor)
066 |         safe_commit(self.session)
067 |         if self.audit_service:
068 |             self.audit_service.log(self.user_id, "atualizar", tabela_alvo="companies", registro_id=empresa.id)
069 |         invalidate("company")
070 |
071 |     def excluir(self, empresa: Company) -> None:
072 |         if self.audit_service:
073 |             self.audit_service.log(self.user_id, "excluir", tabela_alvo="companies", registro_id=empresa.id)
074 |             empresa.deleted_at = utcnow()
075 |             safe_commit(self.session)
076 |             invalidate("company")
077 |
078 |     @cached(ttl=60, namespace="company")
079 |     def listar_nomes(self) -> list[str]:
080 |         return [emp.nome for emp in self.listar_todas()]
081 |
082 |     @cached(ttl=60, namespace="company")
083 |     def contar_todas(self) -> int:
084 |         return self.session.query(Company).filter(Company.deleted_at == None).count()
085 |
086 |     def listar_excluidos(self, limite: int = 50) -> list[Company]:
087 |         return self.session.query(Company).filter(
088 |             Company.deleted_at != None
089 |         ).order_by(Company.deleted_at.desc()).limit(limite).all()
090 |
091 |     def restaurar(self, obj: Company) -> None:
092 |         obj.deleted_at = None
093 |         safe_commit(self.session)
094 |         if self.audit_service:
095 |             self.audit_service.log(self.user_id, "restaurar", tabela_alvo="companies", registro_id=obj.id)
096 |         invalidate("company")
097 |

```

=== ARQUIVO: .lapp\services\dashboard_service.py ===

```

001 | from __future__ import annotations
002 |
003 | from datetime import datetime, timedelta
004 | from typing import Any, Optional
005 |
006 | from sqlalchemy import func
007 | from sqlalchemy.orm import Session
008 |
009 | from app.models import Activity, Alert, Printer
010 | from app.utils.cache import cached
011 | from app.utils.constants import STATUS_MANUTENCAO, STATUS_OPERACIONAL
012 |
013 |
014 | class DashboardService:
015 |     def __init__(self, session: Session) -> None:
016 |         self.session: Session = session
017 |
018 |     @cached(ttl=30, namespace="dashboard")
019 |     def resumo(self) -> dict[str, int]:
020 |         status_counts_raw = self.session.query(Printer.status, func.count(Printer.id)).group_by(Printer.status).all()
021 |         status_counts = dict(status_counts_raw)
022 |         total = sum(status_counts.values())

```

```

023 |         em_manutencao = sum(status_counts.get(s, 0) for s in STATUS_MANUTENCAO)
024 |         operacionais = sum(status_counts.get(s, 0) for s in STATUS_OPERACIONAL)
025 |         total_atividades = self.session.query(Activity).filter(Activity.deleted_at == None).count()
026 |         return {
027 |             "total_impressoras": total,
028 |             "em_manutencao": em_manutencao,
029 |             "operacionais": operacionais,
030 |             "total_atividades": total_atividades,
031 |         }
032 |
033 |     @cached(ttl=30, namespace="dashboard")
034 |     def dados_grafico_pizza(self) -> tuple[list[str], list[int]]:
035 |         rows = self.session.query(Printer.status, func.count(Printer.id)).group_by(Printer.status).all()
036 |         labels = [r[0] or "Sem status" for r in rows]
037 |         valores = [r[1] for r in rows]
038 |         return labels, valores
039 |
040 |     @cached(ttl=60, namespace="dashboard")
041 |     def dados_grafico_atividades(self, meses: int = 6) -> tuple[list[str], list[int]]:
042 |         hoje = datetime.now()
043 |         inicio = hoje.replace(day=1) - timedelta(days=meses * 30)
044 |         rows = (
045 |             self.session.query(
046 |                 func.strftime("%Y-%m", Activity.event_at),
047 |                 func.count(Activity.id),
048 |             )
049 |             .filter(Activity.event_at >= inicio)
050 |             .group_by(func.strftime("%Y-%m", Activity.event_at))
051 |             .order_by(func.strftime("%Y-%m", Activity.event_at))
052 |             .all()
053 |         )
054 |         meses_labels = []
055 |         valores = []
056 |         for r in rows:
057 |             meses_labels.append(r[0])
058 |             valores.append(r[1])
059 |         return meses_labels, valores
060 |
061 |     @cached(ttl=60, namespace="dashboard")
062 |     def dados_grafico_alertas(self, dias: int = 30) -> tuple[list[str], list[int]]:
063 |         inicio = datetime.now() - timedelta(days=dias)
064 |         rows = (
065 |             self.session.query(
066 |                 func.strftime("%Y-%m-%d", Alert.created_at),
067 |                 func.count(Alert.id),
068 |             )
069 |             .filter(Alert.created_at >= inicio)
070 |             .group_by(func.strftime("%Y-%m-%d", Alert.created_at))
071 |             .order_by(func.strftime("%Y-%m-%d", Alert.created_at))
072 |             .all()
073 |         )
074 |         dias_labels = []
075 |         criados = []
076 |         for r in rows:
077 |             dias_labels.append(r[0])
078 |             criados.append(r[1])
079 |         return dias_labels, criados

```

=== ARQUIVO: .\app\services\login_history_service.py ===

```

001 | from __future__ import annotations

```



```

002 |
003 | from datetime import datetime
004 | from typing import Any, Optional
005 |
006 | from db import safe_commit
007 | from sqlalchemy.orm import Session
008 |
009 | from app.models import LoginHistory
010 | from app.models.base import utcnow
011 |
012 |
013 | class LoginHistoryService:
014 |     def __init__(self, session: Session) -> None:
015 |         self.session: Session = session
016 |
017 |     def registrar_login(self, user_id: int, ip_address: str = "") -> LoginHistory:
018 |         sessao = LoginHistory(user_id=user_id, ip_address=ip_address)
019 |         self.session.add(sessao)
020 |         safe_commit(self.session)
021 |         return sessao
022 |
023 |     def registrar_logout(self, user_id: int) -> Optional[LoginHistory]:
024 |         sessao = self.session.query(LoginHistory).filter(
025 |             LoginHistory.user_id == user_id,
026 |             LoginHistory.logout_at.is_(None)
027 |         ).order_by(LoginHistory.login_at.desc()).first()
028 |         if sessao:
029 |             sessao.logout_at = utcnow()
030 |             safe_commit(self.session)
031 |             return sessao
032 |
033 |     def listar_por_usuario(self, user_id: int, limite: int = 50) -> list[LoginHistory]:
034 |         return self.session.query(LoginHistory).filter(
035 |             LoginHistory.user_id == user_id
036 |         ).order_by(LoginHistory.login_at.desc()).limit(limite).all()
037 |
038 |     def listar_todos(self, limite: int = 100) -> list[LoginHistory]:
039 |         return self.session.query(LoginHistory).order_by(
040 |             LoginHistory.login_at.desc()
041 |         ).limit(limite).all()
042 |
043 |     def ultimo_login(self, user_id: int) -> Optional[LoginHistory]:
044 |         return self.session.query(LoginHistory).filter(
045 |             LoginHistory.user_id == user_id
046 |         ).order_by(LoginHistory.login_at.desc()).first()
047 |
048 |     def tempo_medio_sessao(self, user_id: int) -> Any:
049 |         from sqlalchemy import func
050 |         result = self.session.query(
051 |             func.avg(LoginHistory.logout_at - LoginHistory.login_at)
052 |         ).filter(
053 |             LoginHistory.user_id == user_id,
054 |             LoginHistory.logout_at.isnot(None)
055 |         ).scalar()
056 |         return result

```

=== ARQUIVO: .\app\services\maintenance_scheduler.py ===

```

001 | from __future__ import annotations
002 |
003 | import logging

```

[illegible]

```

067 |         return (
068 |             self.session.query(MaintenanceSchedule)
069 |             .options(selectinload(MaintenanceSchedule.printer))
070 |             .filter(MaintenanceSchedule.id == schedule_id)
071 |             .first()
072 |         )
073 |
074 |     def agendar_periodo(
075 |         self,
076 |         printer_id: str,
077 |         intervalo_dias: int,
078 |         dias_aviso: int = 7,
079 |         observacao: str = "",
080 |     ) -> MaintenanceSchedule:
081 |         schedule = MaintenanceSchedule(
082 |             printer_id=printer_id,
083 |             tipo="periodo",
084 |             intervalo_dias=intervalo_dias,
085 |             dias_aviso=dias_aviso,
086 |             observacao=sanitizar(observacao),
087 |             ativo=True,
088 |         )
089 |         self._recalcular_proxima(schedule)
090 |         self.session.add(schedule)
091 |         safe_commit(self.session)
092 |         return schedule
093 |
094 |     def agendar_paginas(
095 |         self,
096 |         printer_id: str,
097 |         intervalo_paginas: int,
098 |         contador_inicial: int = 0,
099 |         dias_aviso: int = 7,
100 |         observacao: str = "",
101 |     ) -> MaintenanceSchedule:
102 |         schedule = MaintenanceSchedule(
103 |             printer_id=printer_id,
104 |             tipo="paginas",
105 |             intervalo_paginas=intervalo_paginas,
106 |             contador_inicial=contador_inicial,
107 |             contador_atual=contador_inicial,
108 |             proxima_meta_paginas=contador_inicial + intervalo_paginas,
109 |             dias_aviso=dias_aviso,
110 |             observacao=sanitizar(observacao),
111 |             ativo=True,
112 |         )
113 |         self._recalcular_proxima(schedule)
114 |         self.session.add(schedule)
115 |         safe_commit(self.session)
116 |         return schedule
117 |
118 |     def atualizar(
119 |         self,
120 |         schedule_id: int,
121 |         **kwargs: Any,
122 |     ) -> Optional[MaintenanceSchedule]:
123 |         schedule = self.buscar_por_id(schedule_id)
124 |         if not schedule:
125 |             return None
126 |         for chave, valor in kwargs.items():
127 |             if hasattr(schedule, chave):
128 |                 setattr(schedule, chave, valor)
129 |         if "intervalo_dias" in kwargs or "dias_aviso" in kwargs:

```

[illegible]

[illegible]

```

256 | def _criar_alerta_aviso(
257 |     self, schedule: MaintenanceSchedule, paginas_restantes: Optional[int] = None
258 | ) -> None:
259 |     if not self.alert_service:
260 |         return
261 |     printer = schedule.printer
262 |     if not printer:
263 |         return
264 |
265 |     if paginas_restantes is not None:
266 |         titulo = f"Manutenção por páginas: {printer.patrimonio}"
267 |         desc = (
268 |             f"Impressora {printer.modelo} ({printer.patrimonio}) - "
269 |             f"{paginas_restantes} páginas restantes para a manutenção."
270 |         )
271 |     else:
272 |         titulo = f"Manutenção periódica próxima: {printer.patrimonio}"
273 |         desc = (
274 |             f"Impressora {printer.modelo} ({printer.patrimonio}) - "
275 |             f"revisão programada para "
276 |             f"{schedule.proxima_geracao.strftime('%d/%m/%Y')}."
277 |         )
278 |
279 |     from app.models import Alert
280 |
281 |     ja_existe = (
282 |         self.session.query(Alert)
283 |         .filter(
284 |             Alert.printer_id == printer.id,
285 |             Alert.tipo == "revisao",
286 |             Alert.resolvido == False,
287 |             Alert.titulo.like(f"%{titulo}%"),
288 |         )
289 |         .first()
290 |     )
291 |     if ja_existe:
292 |         return
293 |
294 |     self.alert_service.criar(
295 |         printer_id=printer.id,
296 |         tipo="revisao",
297 |         titulo=titulo,
298 |         descricao=desc,
299 |         data_agendada=schedule.proxima_geracao,
300 |     )
301 |
302 | def _resolver_alertas_pendentes(self, schedule: MaintenanceSchedule) -> int:
303 |     if not self.alert_service:
304 |         return 0
305 |     from app.models import Alert
306 |
307 |     alerts = (
308 |         self.session.query(Alert)
309 |         .filter(
310 |             Alert.printer_id == schedule.printer_id,
311 |             Alert.tipo == "revisao",
312 |             Alert.resolvido == False,
313 |         )
314 |         .all()
315 |     )
316 |     for a in alerts:
317 |         self.alert_service.resolver(a, user_id=self.user_id)
318 |     return len(alerts)

```

[illegible]

=== ARQUIVO: .lappl\services\notification_service.py ===

```

001 | from __future__ import annotations
002 |
003 | import json
004 | import logging
005 | import smtplib
006 | import ssl
007 | from dataclasses import dataclass, field
008 | from datetime import datetime
009 | from email.mime.text import MIMEText
010 | from pathlib import Path
011 | from threading import Thread
012 | from typing import Any
013 |
014 | from config import DATA_DIR
015 |
016 | log = logging.getLogger(__name__)
017 |
018 | NOTIFICACAO_CONFIG_PATH = DATA_DIR / "notification_config.json"
019 |
020 |
021 | @dataclass
022 | class NotificacaoConfig:
023 |     desktop_ativado: bool = True
024 |     email_ativado: bool = False
025 |     smtp_host: str = ""
026 |     smtp_port: int = 587
027 |     smtp_usuario: str = ""
028 |     smtp_senha: str = ""
029 |     email_remetente: str = ""
030 |     email_destinatario: str = ""
031 |
032 |     @classmethod
033 |     def carregar(cls) -> "NotificacaoConfig":
034 |         if NOTIFICACAO_CONFIG_PATH.exists():
035 |             try:
036 |                 dados = json.loads(NOTIFICACAO_CONFIG_PATH.read_text("utf-8"))
037 |                 return cls(**dados)
038 |             except Exception as e:
039 |                 log.warning("Erro ao carregar config de notificação: %s", e)
040 |         return cls()
041 |
042 |     def salvar(self) -> None:
043 |         dados = {
044 |             "desktop_ativado": self.desktop_ativado,
045 |             "email_ativado": self.email_ativado,
046 |             "smtp_host": self.smtp_host,
047 |             "smtp_port": self.smtp_port,
048 |             "smtp_usuario": self.smtp_usuario,
049 |             "smtp_senha": self.smtp_senha,
050 |             "email_remetente": self.email_remetente,
051 |             "email_destinatario": self.email_destinatario,
052 |         }
053 |         NOTIFICACAO_CONFIG_PATH.write_text(json.dumps(dados, indent=2, ensure_ascii=False), "utf-8")
054 |
055 |     def testar_email(self) -> tuple[bool, str]:
056 |         if not all([self.smtp_host, self.smtp_usuario, self.smtp_senha, self.email_remetente, self.email_de
057 |             return False, "Preencha todos os campos de e-mail."
058 |         try:
059 |             contexto = ssl.create_default_context()
060 |             with smtplib.SMTP(self.smtp_host, self.smtp_port) as server:
061 |                 server.ehlo()
062 |                 server.starttls(context=contexto)
063 |                 server.ehlo()

```



```

064 |         server.login(self.smtp_usuario, self.smtp_senha)
065 |         msg = MIMEText("Teste de notificação do Controle de Impressoras.\n\nSe voc\u00ea recebeu es
066 |         msg["Subject"] = "Teste de Notificação - Controle de Impressoras"
067 |         msg["From"] = self.email_remetente
068 |         msg["To"] = self.email_destinatario
069 |         server.sendmail(self.email_remetente, [self.email_destinatario], msg.as_string())
070 |         return True, "E-mail de teste enviado com sucesso!"
071 |     except smtplib.SMTPAuthenticationError:
072 |         return False, "Falha de autenticação SMTP. Verifique usuário/senha."
073 |     except smtplib.SMTPException as e:
074 |         return False, f"Erro SMTP: {e}"
075 |     except Exception as e:
076 |         return False, f"Erro: {e}"
077 |
078 |
079 | class NotificadorService:
080 |     def __init__(self, tray_icon: Any = None) -> None:
081 |         self._config: NotificacaoConfig = NotificacaoConfig.carregar()
082 |         self._tray_icon: Any = tray_icon
083 |
084 |     @property
085 |     def config(self) -> NotificacaoConfig:
086 |         return self._config
087 |
088 |     def recarregar_config(self) -> None:
089 |         self._config = NotificacaoConfig.carregar()
090 |
091 |     def salvar_config(self) -> None:
092 |         self._config.salvar()
093 |
094 |     def notificar_alerta(self, alerta: Any) -> None:
095 |         if not alerta:
096 |             return
097 |
098 |         if self._config.desktop_ativado:
099 |             self._notificar_desktop(alerta)
100 |
101 |         if self._config.email_ativado:
102 |             Thread(target=self._notificar_email, args=(alerta,), daemon=True).start()
103 |
104 |     def _notificar_desktop(self, alerta: Any) -> None:
105 |         if not self._tray_icon or not self._tray_icon.isVisible():
106 |             return
107 |         try:
108 |             titulo = f"\u26a0 Alerta: {alerta.titulo or 'Sem título'}"
109 |             if hasattr(alerta, "printer") and alerta.printer:
110 |                 msg = f"Impressora: {alerta.printer.patrimonio} - {alerta.printer.modelo}"
111 |             else:
112 |                 msg = alerta.descricao or ""
113 |             self._tray_icon.showMessage(titulo, msg, msecs=8000)
114 |         except Exception as e:
115 |             log.warning("Erro ao notificar desktop: %s", e)
116 |
117 |     def _notificar_email(self, alerta: Any) -> None:
118 |         cfg = self._config
119 |         if not all([cfg.smtp_host, cfg.smtp_usuario, cfg.smtp_senha, cfg.email_remetente, cfg.email_destina
120 |             return
121 |         try:
122 |             printer_info = ""
123 |             if hasattr(alerta, "printer") and alerta.printer:
124 |                 printer_info = f"Impressora: {alerta.printer.patrimonio} - {alerta.printer.modelo} ({alerta
125 |                 corpo = f""
126 | Alerta Crítico - Controle de Impressoras

```

```

127 |
128 | Título: {alerta.titulo}
129 | Tipo: {alerta.tipo}
130 | Descrição: {alerta.descricao or "N/A"}
131 | Data: {alerta.data_alerta or datetime.now()}
132 | {printer_info}
133 |
134 | Este é um alerta automático do sistema Controle de Impressoras Pro.
135 | """
136 |         msg = MIMEText(corpo.strip())
137 |         msg["Subject"] = f"\u26a0 Alerta: {alerta.titulo}"
138 |         msg["From"] = cfg.email_remetente
139 |         msg["To"] = cfg.email_destinatario
140 |
141 |         contexto = ssl.create_default_context()
142 |         with smtplib.SMTP(cfg.smtp_host, cfg.smtp_port) as server:
143 |             server.ehlo()
144 |             server.starttls(context=contexto)
145 |             server.ehlo()
146 |             server.login(cfg.smtp_usuario, cfg.smtp_senha)
147 |             server.sendmail(cfg.email_remetente, [cfg.email_destinatario], msg.as_string())
148 |         log.info("E-mail de notificação enviado para %s", cfg.email_destinatario)
149 |     except Exception as e:
150 |         log.warning("Erro ao enviar e-mail de notificação: %s", e)

```

=== ARQUIVO: .lapp\services\part_service.py ===

```

001 | ■from __future__ import annotations
002 |
003 | import logging
004 | from datetime import datetime
005 | from typing import TYPE_CHECKING, Any, Optional
006 |
007 | from app.models.base import utcnow
008 | from db import safe_commit
009 | from sqlalchemy.orm import Session
010 |
011 | from app.models import Part
012 | from app.utils.cache import cached, invalidate
013 | from app.utils.sanitize import sanitizar
014 |
015 | if TYPE_CHECKING:
016 |     from app.models.activity import Activity
017 |     from app.models.part_movement import PartMovement
018 |     from app.models.part_reservation import PartReservation
019 |     from app.models.purchase_requisition import PurchaseRequisition
020 |     from app.services.audit_service import AuditService
021 |
022 | log = logging.getLogger(__name__)
023 |
024 |
025 | class PartService:
026 |     def __init__(self, session: Session, audit_service: Optional[AuditService] = None, user_id: Optional[int] = None):
027 |         self.session: Session = session
028 |         self.audit_service: Optional[AuditService] = audit_service
029 |         self.user_id: Optional[int] = user_id
030 |
031 |     @cached(ttl=15, namespace="part")
032 |     def listar_todas(self, filtro: Optional[str] = None, limite: Optional[int] = None, offset: Optional[int] = None):
033 |         query = self.session.query(Part).filter(Part.deleted_at == None)
034 |         if filtro:

```

```

035 |         f = f"%{filtro}%"
036 |         query = query.filter(
037 |             Part.nome.like(f) | Part.codigo.like(f) | Part.modelo_compativel.like(f)
038 |             | Part.marca.like(f) | Part.categoria.like(f) | Part.fornecedor.like(f)
039 |         )
040 |         query = query.order_by(Part.nome)
041 |         if limite is not None:
042 |             query = query.limit(limite)
043 |         if offset is not None:
044 |             query = query.offset(offset)
045 |         return query.all()
046 |
047 | def buscar_por_codigo(self, codigo: str) -> Optional[Part]:
048 |     return self.session.query(Part).filter(Part.codigo == codigo).first()
049 |
050 | def buscar_por_nome(self, nome: str) -> Optional[Part]:
051 |     return self.session.query(Part).filter(Part.nome == nome, Part.deleted_at == None).first()
052 |
053 | @cached(ttl=15, namespace="part")
054 | def buscar_por_id(self, part_id: int) -> Optional[Part]:
055 |     return self.session.query(Part).filter(Part.deleted_at == None, Part.id == part_id).first()
056 |
057 | def criar(self, codigo: str, nome: str, descricao: str = "", modelo_compativel: str = "", quantidade: int = 0):
058 |     from app.utils.importer import normalizar_modelos
059 |     peca = Part(
060 |         codigo=sanitizar(codigo, "Part", "codigo"),
061 |         nome=sanitizar(nome, "Part", "nome"),
062 |         descricao=sanitizar(descricao),
063 |         modelo_compativel=sanitizar(normalizar_modelos(modelo_compativel), "Part", "modelo_compativel"),
064 |         quantidade_estoque=quantidade,
065 |         estoque_minimo=estoque_minimo,
066 |         preco_unitario=0.0,
067 |         marca=sanitizar(marca, "Part", "marca"),
068 |         categoria=sanitizar(categoria, "Part", "categoria"),
069 |         fornecedor=sanitizar(fornecedor, "Part", "fornecedor"),
070 |     )
071 |     self.session.add(peca)
072 |     safe_commit(self.session)
073 |     if self.audit_service:
074 |         self.audit_service.log(self.user_id, "criar", tabela_alvo="parts", registro_id=peca.id, dados={"codigo": codigo, "nome": nome, "descricao": descricao, "modelo_compativel": modelo_compativel, "quantidade": quantidade})
075 |     invalidate("part")
076 |     if quantidade > 0:
077 |         self._log_movimento(peca, "entrada", quantidade, observacao="Estoque inicial")
078 |     return peca
079 |
080 | def atualizar(self, peca: Part, **kwargs: Any) -> None:
081 |     from app.utils.importer import normalizar_modelos
082 |     quantidade_antiga = peca.quantidade_estoque
083 |     if self.audit_service:
084 |         dados_antes = {chave: getattr(peca, chave, None) for chave in kwargs}
085 |         if "modelo_compativel" in kwargs and isinstance(kwargs["modelo_compativel"], str):
086 |             kwargs["modelo_compativel"] = normalizar_modelos(kwargs["modelo_compativel"])
087 |         for chave, valor in kwargs.items():
088 |             if hasattr(peca, chave):
089 |                 if isinstance(valor, str):
090 |                     valor = sanitizar(valor, "Part", chave)
091 |                 setattr(peca, chave, valor)
092 |     safe_commit(self.session)
093 |     if self.audit_service:
094 |         self.audit_service.log(self.user_id, "atualizar", tabela_alvo="parts", registro_id=peca.id, dados={"codigo": peca.codigo, "nome": peca.nome, "descricao": peca.descricao, "modelo_compativel": peca.modelo_compativel, "quantidade": peca.quantidade_estoque})
095 |     invalidate("part")
096 |     if "quantidade_estoque" in kwargs:
097 |         diff = peca.quantidade_estoque - quantidade_antiga

```

[illegible]

```

161 |         saldo_anterior=saldo_anterior,
162 |         saldo_posterior=saldo_posterior,
163 |         observacao=observacao,
164 |     )
165 |     self.session.add(mov)
166 |     safe_commit(self.session)
167 |     return mov
168 |
169 | def receber_estoque(self, part: Part, quantidade: int, observacao: str = "") -> PartMovement:
170 |     part.quantidade_estoque += quantidade
171 |     safe_commit(self.session)
172 |     return self._log_movimento(part, "entrada", quantidade, observacao=observacao)
173 |
174 | def retirar_estoque_por_nome(self, nomes: str, activity_id: Optional[int] = None) -> None:
175 |     for nome_pecas in nomes.split(","):
176 |         nome_pecas = nome_pecas.strip()
177 |         if not nome_pecas:
178 |             continue
179 |         part = self.buscar_por_nome(nome_pecas)
180 |         if part and part.quantidade_estoque > 0:
181 |             self.retirar_estoque(part, activity_id=activity_id)
182 |
183 | def retirar_estoque(self, part: Part, quantidade: int = 1, activity_id: Optional[int] = None, observacao: str = "") -> PartMovement:
184 |     if part.quantidade_estoque < quantidade:
185 |         quantidade = part.quantidade_estoque
186 |     part.quantidade_estoque -= quantidade
187 |     safe_commit(self.session)
188 |     mov = self._log_movimento(part, "saida", quantidade, activity_id=activity_id, observacao=observacao)
189 |     if part.quantidade_estoque < part.estoque_minimo:
190 |         self._criar_requisicao_auto(part)
191 |     return mov
192 |
193 | def movimentacoes(self, part_id: int, limite: int = 100) -> list[PartMovement]:
194 |     from app.models.part_movement import PartMovement
195 |     return self.session.query(PartMovement).filter(
196 |         PartMovement.part_id == part_id,
197 |         PartMovement.deleted_at == None,
198 |     ).order_by(PartMovement.created_at.desc()).limit(limite).all()
199 |
200 | def listar_movimentacoes_por_os(self, activity_id: int) -> list[PartMovement]:
201 |     from app.models.part_movement import PartMovement
202 |     return self.session.query(PartMovement).filter(
203 |         PartMovement.activity_id == activity_id,
204 |         PartMovement.deleted_at == None,
205 |     ).order_by(PartMovement.created_at.asc()).all()
206 |
207 | def movimentacoes_geral(self, limite: int = 200) -> list[PartMovement]:
208 |     from app.models.part_movement import PartMovement
209 |     return self.session.query(PartMovement).filter(
210 |         PartMovement.deleted_at == None,
211 |     ).order_by(PartMovement.created_at.desc()).limit(limite).all()
212 |
213 | def estornar_movimento(self, movimento_id: int) -> None:
214 |     from app.models.part_movement import PartMovement
215 |     mov = self.session.query(PartMovement).filter(
216 |         PartMovement.id == movimento_id,
217 |         PartMovement.deleted_at == None,
218 |     ).first()
219 |     if not mov:
220 |         log.warning("estornar_movimento #s: não encontrado ou já estornado", movimento_id)
221 |         return
222 |     part = self.session.query(Part).filter(Part.id == mov.part_id).first()
223 |     if part is None:

```

[illegible]

[illegible]

```

350 |     def cancelar_requisicao(self, req_id: int) -> None:
351 |         from app.models.purchase_requisition import PurchaseRequisition
352 |         req = self.session.query(PurchaseRequisition).filter(PurchaseRequisition.id == req_id).first()
353 |         if req and req.status != "recebida":
354 |             req.status = "cancelada"
355 |             req.updated_at = datetime.utcnow()
356 |             safe_commit(self.session)

```

=== ARQUIVO: .\app\services\printer_location_service.py ===

```

001 | from __future__ import annotations
002 |
003 | import logging
004 | from datetime import datetime
005 | from typing import TYPE_CHECKING, Any, Optional
006 |
007 | from sqlalchemy.orm import Session
008 |
009 | from app.models import Printer, PrinterLocation
010 | from app.utils.sanitize import sanitizar
011 | from db import safe_commit
012 |
013 | if TYPE_CHECKING:
014 |     from app.services.audit_service import AuditService
015 |
016 | log = logging.getLogger(__name__)
017 |
018 |
019 | class PrinterLocationService:
020 |     def __init__(self, session: Session, audit_service: Optional[AuditService] = None, user_id: Optional[int] = None):
021 |         self.session: Session = session
022 |         self.audit_service: Optional[AuditService] = audit_service
023 |         self.user_id: Optional[int] = user_id
024 |
025 |     def listar_por_impressora(self, printer_id: str, limite: int = 200) -> list[PrinterLocation]:
026 |         return self.session.query(PrinterLocation).filter(
027 |             PrinterLocation.printer_id == printer_id
028 |         ).order_by(PrinterLocation.data.desc().nullslast(), PrinterLocation.created_at.desc()).limit(limite)
029 |
030 |     def adicionar(self, printer_id: str, local: str, data: Optional[datetime] = None, observacao: str = ""):
031 |         reg = PrinterLocation(
032 |             printer_id=printer_id,
033 |             local=sanitizar(local, "PrinterLocation", "local"),
034 |             data=data,
035 |             observacao=sanitizar(observacao, "PrinterLocation", "observacao")
036 |         )
037 |         self.session.add(reg)
038 |         safe_commit(self.session)
039 |         if self.audit_service:
040 |             self.audit_service.log(self.user_id, "criar", tabela_alvo="printer_locations", registro_id=str(reg.id))
041 |         return reg
042 |
043 |     def excluir(self, reg: PrinterLocation) -> None:
044 |         if self.audit_service:
045 |             self.audit_service.log(self.user_id, "excluir", tabela_alvo="printer_locations", registro_id=str(reg.id))
046 |         self.session.delete(reg)
047 |         safe_commit(self.session)
048 |
049 |     def importar(self, printer_id: str, registros: list[dict[str, Any]]) -> int:
050 |         importados = 0
051 |         for r in registros:

```



```

052 |         local = r.get("local") or r.get("Local") or ""
053 |         data = r.get("data") or r.get("Data") or None
054 |         obs = r.get("observacao") or r.get("Observacao") or r.get("Observação") or ""
055 |         if data:
056 |             try:
057 |                 data = datetime.strptime(str(data).strip(), "%d/%m/%Y")
058 |             except ValueError:
059 |                 try:
060 |                     data = datetime.strptime(str(data).strip(), "%Y-%m-%d")
061 |                 except ValueError:
062 |                     data = None
063 |         self.adicionar(printer_id, local, data, obs)
064 |         importados += 1
065 |     return importados

```

=== ARQUIVO: .\app\services\printer_service.py ===

```

001 | from __future__ import annotations
002 |
003 | import logging
004 | from datetime import datetime
005 | from typing import TYPE_CHECKING, Any, Optional
006 |
007 | from app.models.base import utcnow
008 | from sqlalchemy import func
009 | from sqlalchemy.orm import Session
010 |
011 | from app.models import Activity, Printer
012 | from app.utils.sanitize import sanitizar
013 | from app.utils.cache import cached, invalidate
014 | from db import safe_commit
015 |
016 | if TYPE_CHECKING:
017 |     from app.services.audit_service import AuditService
018 |
019 | log = logging.getLogger(__name__)
020 |
021 |
022 | class PrinterService:
023 |     def __init__(self, session: Session, audit_service: Optional[AuditService] = None, user_id: Optional[int] = None):
024 |         self.session: Session = session
025 |         self.audit_service: Optional[AuditService] = audit_service
026 |         self.user_id: Optional[int] = user_id
027 |
028 |     @cached(ttl=30, namespace="printer")
029 |     def listar_todos(self, filtro: Optional[str] = None, limite: Optional[int] = None, offset: Optional[int] = None):
030 |         query = self.session.query(Printer).filter(Printer.deleted_at == None)
031 |         if filtro:
032 |             f = f"%{filtro}%"
033 |             query = query.filter(
034 |                 Printer.patrimonio.like(f) | Printer.modelo.like(f) |
035 |                 Printer.serial.like(f) | Printer.local_atual.like(f) |
036 |                 Printer.marca.like(f)
037 |             )
038 |         query = query.order_by(Printer.patrimonio)
039 |         if limite is not None:
040 |             query = query.limit(limite)
041 |         if offset is not None:
042 |             query = query.offset(offset)
043 |         return query.all()
044 |

```

```

045 | def buscar_por_patrimonio(self, patrimonio: str) -> Optional[Printer]:
046 |     return self.session.query(Printer).filter(Printer.deleted_at == None, Printer.patrimonio == patrimonio)
047 |
048 | @cached(ttl=15, namespace="printer")
049 | def buscar_por_id(self, printer_id: str) -> Optional[Printer]:
050 |     return self.session.query(Printer).filter(Printer.deleted_at == None, Printer.id == printer_id).first()
051 |
052 | @cached(ttl=15, namespace="printer")
053 | def buscar_por_ids(self, ids: list[str]) -> dict[str, Printer]:
054 |     if not ids:
055 |         return {}
056 |     return {
057 |         p.id: p
058 |         for p in self.session.query(Printer).filter(Printer.id.in_(ids)).all()
059 |     }
060 |
061 | @cached(ttl=15, namespace="printer")
062 | def mapa_patrimonio(self, ids: list[str]) -> dict[str, str]:
063 |     if not ids:
064 |         return {}
065 |     return {
066 |         p.id: p.patrimonio
067 |         for p in self.session.query(Printer.id, Printer.patrimonio)
068 |         .filter(Printer.deleted_at == None, Printer.id.in_(ids)).all()
069 |     }
070 |
071 | def buscar_por_status(self, status_list: list[str]) -> list[Printer]:
072 |     return self.session.query(Printer).filter(
073 |         Printer.status.in_(status_list)
074 |     ).order_by(Printer.patrimonio).all()
075 |
076 | def verificar_patrimonio_existe(self, patrimonio: str) -> Optional[Printer]:
077 |     return self.session.query(Printer).filter(Printer.patrimonio == patrimonio).first()
078 |
079 | def criar(self, patrimonio: str, modelo: str = "", marca: str = "", serial: str = "", tipo: str = "",
080 |           local_atual: str = "", status: str = "Operacional", ip_rede: str = "",
081 |           mac_address: str = "", tecnico: str = "", observacao: str = "",
082 |           pecas_faltantes: str = "", foto_path: Optional[str] = None,
083 |           ultima_revisao: Any = None,
084 |           proxima_revisao: Any = None,
085 |           urgencia_prox_manutencao: str = "Normal") -> Printer:
086 |     printer = Printer(
087 |         patrimonio=sanitizar(patrimonio, "Printer", "patrimonio"),
088 |         modelo=sanitizar(modelo, "Printer", "modelo"),
089 |         marca=sanitizar(marca, "Printer", "marca"),
090 |         serial=sanitizar(serial, "Printer", "serial"),
091 |         tipo=sanitizar(tipo, "Printer", "tipo"),
092 |         local_atual=sanitizar(local_atual, "Printer", "local_atual"),
093 |         status=sanitizar(status, "Printer", "status"),
094 |         ip_rede=sanitizar(ip_rede, "Printer", "ip_rede"),
095 |         mac_address=sanitizar(mac_address, "Printer", "mac_address"),
096 |         tecnico=sanitizar(tecnico, "Printer", "tecnico"),
097 |         observacao=sanitizar(observacao, "Printer", "observacao"),
098 |         pecas_faltantes=sanitizar(pecas_faltantes, "Printer", "pecas_faltantes"),
099 |         foto_path=foto_path,
100 |         ultima_revisao=ultima_revisao,
101 |         proxima_revisao=proxima_revisao,
102 |         urgencia_prox_manutencao=sanitizar(urgencia_prox_manutencao, "Printer", "urgencia_prox_manutencao")
103 |     )
104 |     self.session.add(printer)
105 |     safe_commit(self.session)
106 |     if self.audit_service:
107 |         self.audit_service.log(self.user_id, "criar", tabela_alvo="printers", registro_id=printer.id, d

```

```

108 |         invalidate("printer")
109 |         invalidate("dashboard")
110 |         return printer
111 |
112 |     def atualizar(self, printer: Printer, **kwargs: Any) -> None:
113 |         if self.audit_service:
114 |             dados_antes = {chave: getattr(printer, chave, None) for chave in kwargs}
115 |             for chave, valor in kwargs.items():
116 |                 if hasattr(printer, chave):
117 |                     if isinstance(valor, str):
118 |                         valor = sanitizar(valor, "Printer", chave)
119 |                     setattr(printer, chave, valor)
120 |             printer.updated_at = func.now()
121 |             safe_commit(self.session)
122 |             if self.audit_service:
123 |                 self.audit_service.log(self.user_id, "atualizar", tabela_alvo="printers", registro_id=printer.id)
124 |             invalidate("printer")
125 |             invalidate("dashboard")
126 |
127 |     def excluir(self, printer: Printer) -> None:
128 |         if self.audit_service:
129 |             self.audit_service.log(self.user_id, "excluir", tabela_alvo="printers", registro_id=printer.id)
130 |             printer.deleted_at = utcnow()
131 |             safe_commit(self.session)
132 |             invalidate("printer")
133 |             invalidate("dashboard")
134 |
135 |     @cached(ttl=30, namespace="printer")
136 |     def contar_todos(self, filtro: Optional[str] = None) -> int:
137 |         query = self.session.query(Printer).filter(Printer.deleted_at == None)
138 |         if filtro:
139 |             f = f"%{filtro}%"
140 |             query = query.filter(
141 |                 Printer.patrimonio.like(f) | Printer.modelo.like(f) |
142 |                 Printer.serial.like(f) | Printer.local_atual.like(f) |
143 |                 Printer.marca.like(f)
144 |             )
145 |         return query.count()
146 |
147 |     def contar_atividades(self, printer_ids: list[str]) -> dict[str, int]:
148 |         if not printer_ids:
149 |             return {}
150 |         return {
151 |             pid: cnt
152 |             for pid, cnt in (
153 |                 self.session.query(Activity.printer_id, func.count(Activity.id))
154 |                 .filter(Activity.printer_id.in_(printer_ids))
155 |                 .group_by(Activity.printer_id)
156 |                 .all()
157 |             )
158 |         }
159 |
160 |     @cached(ttl=30, namespace="printer")
161 |     def total_por_status(self) -> dict[str | None, int]:
162 |         dados = (
163 |             self.session.query(Printer.status, func.count(Printer.id))
164 |             .group_by(Printer.status).all()
165 |         )
166 |         return {s: c for s, c in dados}
167 |
168 |     @cached(ttl=60, namespace="printer")
169 |     def modelos_distintos(self) -> list[str]:
170 |         return [

```

```

171 |         m[0] for m in self.session.query(Printer.modelo).filter(Printer.deleted_at == None)
172 |         .filter(Printer.modelo != "").distinct().order_by(Printer.modelo).all()
173 |         if m[0]
174 |     ]
175 |
176 | @cached(ttl=60, namespace="printer")
177 | def locais_distintos(self) -> list[str]:
178 |     return [
179 |         l[0] for l in self.session.query(Printer.local_atual).filter(Printer.deleted_at == None)
180 |         .filter(Printer.local_atual != "").distinct().order_by(Printer.local_atual).all()
181 |     ]
182 |
183 | def agrupar_por_status(self) -> list[tuple[str | None, int]]:
184 |     dados = self.session.query(Printer.status, func.count(Printer.status)).group_by(Printer.status).all()
185 |     return [(s[0], s[1]) for s in dados]
186 |
187 | def top_modelos(self, limite: int = 5) -> list[tuple[str | None, int]]:
188 |     dados = (
189 |         self.session.query(Printer.modelo, func.count(Printer.modelo))
190 |         .group_by(Printer.modelo)
191 |         .order_by(func.count(Printer.modelo).desc())
192 |         .limit(limite).all()
193 |     )
194 |     return [(m[0], m[1]) for m in dados]
195 |
196 | @cached(ttl=30, namespace="printer")
197 | def contar_por_local(self, nome_local: str) -> int:
198 |     return self.session.query(Printer).filter(
199 |         Printer.local_atual.like(f"%{nome_local}%")
200 |     ).count()
201 |
202 | def contar_por_locais(self, nomes: list[str]) -> dict[str, int]:
203 |     if not nomes:
204 |         return {}
205 |     from sqlalchemy import func
206 |     results: dict[str, int] = {}
207 |     rows = (
208 |         self.session.query(Printer.local_atual, func.count(Printer.id))
209 |         .filter(Printer.deleted_at == None)
210 |         .filter(Printer.local_atual.in_(nomes))
211 |         .group_by(Printer.local_atual)
212 |         .all()
213 |     )
214 |     for local, cnt in rows:
215 |         results[local] = cnt
216 |     for nome in nomes:
217 |         if nome not in results:
218 |             results[nome] = 0
219 |     return results
220 |
221 | @cached(ttl=30, namespace="printer")
222 | def listar_patrimonios(self, status: Optional[str] = None) -> list[str]:
223 |     query = self.session.query(Printer.patrimonio)
224 |     if status and status != "Todos":
225 |         query = query.filter(Printer.status == status)
226 |     return [p[0] for p in query.order_by(Printer.patrimonio).all()]
227 |
228 | def listar_patrimonios_por_busca(self, texto: str, status: Optional[str] = None) -> list[str]:
229 |     query = self.session.query(Printer.patrimonio)
230 |     if status and status != "Todos":
231 |         query = query.filter(Printer.status == status)
232 |     if texto.strip():
233 |         query = query.filter(Printer.patrimonio.like(f"%{texto.strip()}%"))

```

```

234 |         return [p[0] for p in query.order_by(Printer.patrimonio).all()]
235 |
236 |     def listar_por_local(self, local: str) -> list[Printer]:
237 |         return self.session.query(Printer).filter(
238 |             Printer.local_atual == local
239 |         ).order_by(Printer.patrimonio).all()
240 |
241 |     def atualizar_local_por_nome_antigo(self, nome_antigo: str, nome_novo: str) -> None:
242 |         impressoraS = self.session.query(Printer).filter(
243 |             Printer.local_atual == nome_antigo
244 |         ).all()
245 |         for p in impressoraS:
246 |             p.local_atual = nome_novo
247 |         safe_commit(self.session)
248 |         invalidate("printer")
249 |
250 |     def listar_por_filtros(self, status: Optional[str] = None, patrimonio: Optional[str] = None, modelo: Optional[str] = None, local: Optional[str] = None) -> list[Printer]:
251 |         query = self.session.query(Printer).filter(Printer.deleted_at == None)
252 |         if status and status != "Todos":
253 |             query = query.filter(Printer.status == status)
254 |         if patrimonio:
255 |             query = query.filter(Printer.patrimonio.like(f"%{patrimonio}%"))
256 |         if modelo and modelo != "Todos":
257 |             query = query.filter(Printer.modelo == modelo)
258 |         if local and local != "Todos":
259 |             query = query.filter(Printer.local_atual.like(f"%{local}%"))
260 |         return query.order_by(Printer.patrimonio).all()
261 |
262 |     def listar_excluidos(self, limite: int = 50) -> list[Printer]:
263 |         return self.session.query(Printer).filter(
264 |             Printer.deleted_at != None
265 |         ).order_by(Printer.deleted_at.desc()).limit(limite).all()
266 |
267 |     def restaurar(self, obj: Printer) -> None:
268 |         obj.deleted_at = None
269 |         safe_commit(self.session)
270 |         if self.audit_service:
271 |             self.audit_service.log(self.user_id, "restaurar", tabela_alvo="printers", registro_id=obj.id)
272 |         invalidate("printer")
273 |         invalidate("dashboard")

```

=== ARQUIVO: .\app\services\relatorio_service.py ===

```

001 | from __future__ import annotations
002 |
003 | from datetime import datetime
004 |
005 | import openpyxl
006 | from openpyxl.styles import Alignment, Border, Font, PatternFill, Side
007 | from reportlab.lib import colors
008 | from reportlab.lib.pagesizes import A4
009 | from reportlab.lib.styles import ParagraphStyle, getSampleStyleSheet
010 | from reportlab.lib.units import cm
011 | from reportlab.platypus import Paragraph, SimpleDocTemplate, Spacer, Table, TableStyle
012 |
013 | from app.models.activity import Activity
014 | from app.models.printer import Printer
015 |
016 |
017 | class RelatorioService:
018 |

```

```

019 | @staticmethod
020 | def exportar_impressoras_pdf(printers: list[Printer], filepath: str) -> str:
021 |     """Gera PDF com lista de impressoras"""
022 |     doc = SimpleDocTemplate(filepath, pagesize=A4, leftMargin=1.5*cm, rightMargin=1.5*cm, topMargin=2*cm)
023 |     styles = getSampleStyleSheet()
024 |     elements = []
025 |
026 |     # Título
027 |     title_style = ParagraphStyle('Title2', parent=styles['Title'], fontSize=18, textColor=colors.HexColor('#45475a'))
028 |     elements.append(Paragraph("Relatório de Impressoras", title_style))
029 |     elements.append(Paragraph(f"Gerado em: {datetime.now().strftime('%d/%m/%Y %H:%M')}", styles['Normal']))
030 |     elements.append(Spacer(1, 15))
031 |
032 |     # Resumo
033 |     total = len(printers)
034 |     operacionais = len([p for p in printers if p.status in ['Operacional', 'Em uso']])
035 |     manutencao = len([p for p in printers if p.status in ['Em manutenção', 'Manutenção', 'Aguardando peça']])
036 |
037 |     resumo_data = [
038 |         ["Total Impressoras", str(total)],
039 |         ["Operacionais", str(operacionais)],
040 |         ["Em Manutenção/Paradas", str(manutencao)],
041 |     ]
042 |     t_resumo = Table(resumo_data, colWidths=[200, 100])
043 |     t_resumo.setStyle(TableStyle([
044 |         ('BACKGROUND', (0, 0), (-1, -1), colors.HexColor('#45475a')),
045 |         ('TEXTCOLOR', (0, 0), (-1, -1), colors.white),
046 |         ('FONTSIZE', (0, 0), (-1, -1), 11),
047 |         ('PADDING', (0, 0), (-1, -1), 8),
048 |         ('BOX', (0, 0), (-1, -1), 1, colors.HexColor('#c8a6f7')),
049 |         ('INNERGRID', (0, 0), (-1, -1), 0.5, colors.HexColor('#c8a6f7')),
050 |     ]))
051 |     elements.append(t_resumo)
052 |     elements.append(Spacer(1, 20))
053 |
054 |     # Tabela principal
055 |     header = ['Patrimônio', 'Modelo', 'Marca', 'Status', 'Local Atual', 'Última Revisão']
056 |     data = [header]
057 |
058 |     for p in printers:
059 |         data.append([
060 |             p.patrimonio,
061 |             p.modelo,
062 |             p.marca or '-',
063 |             p.status,
064 |             p.local_atual or '-',
065 |             p.proxima_revisao.strftime("%d/%m/%Y") if p.proxima_revisao else '-'
066 |         ])
067 |
068 |     t = Table(data, colWidths=[55, 115, 70, 75, 120, 70])
069 |     t_style = TableStyle([
070 |         ('BACKGROUND', (0, 0), (-1, 0), colors.HexColor('#c8a6f7')),
071 |         ('TEXTCOLOR', (0, 0), (-1, 0), colors.HexColor('#1e1e2e')),
072 |         ('FONTSIZE', (0, 0), (-1, -1), 8),
073 |         ('FONTNAME', (0, 0), (-1, 0), 'Helvetica-Bold'),
074 |         ('PADDING', (0, 0), (-1, -1), 5),
075 |         ('BACKGROUND', (0, 1), (-1, -1), colors.HexColor('#313244')),
076 |         ('TEXTCOLOR', (0, 1), (-1, -1), colors.white),
077 |         ('BOX', (0, 0), (-1, -1), 1, colors.HexColor('#c8a6f7')),
078 |         ('INNERGRID', (0, 0), (-1, -1), 0.3, colors.HexColor('#585b70')),
079 |         ('ROWBACKGROUNDS', (0, 1), (-1, -1), [colors.HexColor('#313244'), colors.HexColor('#45475a')]),
080 |     ])
081 |     t.setStyle(t_style)

```

```

082 |         elements.append(t)
083 |
084 |         # Rodapé
085 |         elements.append(Spacer(1, 20))
086 |         elements.append(Paragraph(f"Total de impressoras: {total} | Operacionais: {operacionais} | Em manu
087 |
088 |         doc.build(elements)
089 |         return filepath
090 |
091 |     @staticmethod
092 |     def exportar_impressoras_excel(printers: list[Printer], filepath: str) -> str:
093 |         """Gera planilha Excel com lista de impressoras"""
094 |         wb = openpyxl.Workbook()
095 |         ws = wb.active
096 |         ws.title = "Impressoras"
097 |
098 |         # Estilos
099 |         header_font = Font(name='Arial', size=12, bold=True, color='FFFFFF')
100 |         header_fill = PatternFill(start_color='cba6f7', end_color='cba6f7', fill_type='solid')
101 |         row_fill1 = PatternFill(start_color='313244', end_color='313244', fill_type='solid')
102 |         row_fill2 = PatternFill(start_color='45475a', end_color='45475a', fill_type='solid')
103 |         white_font = Font(name='Arial', size=10, color='FFFFFF')
104 |         border = Border(
105 |             left=Side(style='thin', color='cba6f7'),
106 |             right=Side(style='thin', color='cba6f7'),
107 |             top=Side(style='thin', color='cba6f7'),
108 |             bottom=Side(style='thin', color='cba6f7')
109 |         )
110 |
111 |         # Titulo
112 |         ws.merge_cells('A1:G1')
113 |         ws['A1'] = f'RELATÓRIO DE IMPRESSORAS - {datetime.now().strftime("%d/%m/%Y %H:%M")}'
114 |         ws['A1'].font = Font(name='Arial', size=14, bold=True, color='cba6f7')
115 |         ws['A1'].alignment = Alignment(horizontal='center')
116 |
117 |         # Resumo
118 |         total = len(printers)
119 |         op = len([p for p in printers if p.status in ['Operacional', 'Em uso']])
120 |         man = len([p for p in printers if p.status in ['Em manutenção', 'Manutenção', 'Aguardando peça', 'P
121 |
122 |         ws['A3'] = f'Total: {total} | Operacionais: {op} | Em Manutenção: {man}'
123 |         ws['A3'].font = Font(name='Arial', size=11, bold=True, color='89b4fa')
124 |
125 |         # Cabeçalho
126 |         headers = ['Patrimônio', 'Modelo', 'Serial', 'Marca', 'Status', 'Local Atual', 'Observação']
127 |         for col, h in enumerate(headers, 1):
128 |             cell = ws.cell(row=5, column=col, value=h)
129 |             cell.font = header_font
130 |             cell.fill = header_fill
131 |             cell.alignment = Alignment(horizontal='center')
132 |             cell.border = border
133 |
134 |         # Dados
135 |         for i, p in enumerate(printers, 6):
136 |             dados = [p.patrimonio, p.modelo, p.serial or '-', p.marca or '-', p.status, p.local_atual or '-
137 |             for col, val in enumerate(dados, 1):
138 |                 cell = ws.cell(row=i, column=col, value=val)
139 |                 cell.font = white_font
140 |                 cell.fill = row_fill1 if i % 2 == 0 else row_fill2
141 |                 cell.border = border
142 |                 cell.alignment = Alignment(vertical='center')
143 |
144 |         # Ajusta largura

```

```

145 |         col_widths = [12, 22, 18, 15, 18, 25, 30]
146 |         for col, w in enumerate(col_widths, 1):
147 |             ws.column_dimensions[openpyxl.utils.get_column_letter(col)].width = w
148 |
149 |         wb.save(filepath)
150 |         return filepath
151 |
152 | @staticmethod
153 | def exportar_atividades_pdf(atividades: list[Activity], filepath: str) -> str:
154 |     """Gera PDF com histórico de atividades"""
155 |     doc = SimpleDocTemplate(filepath, pagesize=A4, leftMargin=1*cm, rightMargin=1*cm, topMargin=2*cm, b
156 |     styles = getSampleStyleSheet()
157 |     elements = []
158 |
159 |     title_style = ParagraphStyle('T', parent=styles['Title'], fontSize=18, textColor=colors.HexColor('#
160 |     elements.append(Paragraph("Relatório de Atividades / OS", title_style))
161 |     elements.append(Paragraph(f"Gerado em: {datetime.now().strftime('%d/%m/%Y %H:%M')} | Total: {len(at
162 |     elements.append(Spacer(1, 15))
163 |
164 |     header = ['Data', 'Patrimônio', 'Tipo', 'Descrição', 'Peças', 'Origem', 'Destino', 'Status']
165 |     data = [header]
166 |
167 |     for a in atividades:
168 |         data.append([
169 |             a.event_at.strftime("%d/%m/%Y %H:%M") if a.event_at else '-',
170 |             a.printer.patrimonio if a.printer else '-',
171 |             'Manutenção' if a.kind == 'MANUTENCAO' else 'Movimentação',
172 |             a.notes or '-',
173 |             a.parts_used or '-',
174 |             a.from_location or '-',
175 |             a.to_location or '-',
176 |             a.status_atividade or 'Concluida'
177 |         ])
178 |
179 |     t = Table(data, colWidths=[60, 40, 55, 120, 60, 55, 55, 50])
180 |     t.setStyle(TableStyle([
181 |         ('BACKGROUND', (0, 0), (-1, 0), colors.HexColor('#cba6f7')),
182 |         ('TEXTCOLOR', (0, 0), (-1, 0), colors.HexColor('#1e1e2e')),
183 |         ('FONTSIZE', (0, 0), (-1, -1), 7),
184 |         ('FONTNAME', (0, 0), (-1, 0), 'Helvetica-Bold'),
185 |         ('PADDING', (0, 0), (-1, -1), 4),
186 |         ('BACKGROUND', (0, 1), (-1, -1), colors.HexColor('#313244')),
187 |         ('TEXTCOLOR', (0, 1), (-1, -1), colors.white),
188 |         ('BOX', (0, 0), (-1, -1), 1, colors.HexColor('#cba6f7')),
189 |         ('INNERGRID', (0, 0), (-1, -1), 0.3, colors.HexColor('#585b70')),
190 |         ('ROWBACKGROUNDS', (0, 1), (-1, -1), [colors.HexColor('#313244'), colors.HexColor('#45475a')]),
191 |     ]))
192 |     elements.append(t)
193 |
194 |     doc.build(elements)
195 |     return filepath
196 |
197 | @staticmethod
198 | def exportar_atividades_excel(atividades: list[Activity], filepath: str) -> str:
199 |     """Gera Excel com atividades"""
200 |     wb = openpyxl.Workbook()
201 |     ws = wb.active
202 |     ws.title = "Atividades"
203 |
204 |     header_font = Font(name='Arial', size=11, bold=True, color='FFFFFF')
205 |     header_fill = PatternFill(start_color='cba6f7', end_color='cba6f7', fill_type='solid')
206 |     white_font = Font(name='Arial', size=9, color='FFFFFF')
207 |     fill1 = PatternFill(start_color='313244', end_color='313244', fill_type='solid')

```



```

208 |         fill2 = PatternFill(start_color='45475a', end_color='45475a', fill_type='solid')
209 |         border = Border(left=Side(style='thin', color='cba6f7'), right=Side(style='thin', color='cba6f7'),
210 |                         top=Side(style='thin', color='cba6f7'), bottom=Side(style='thin', color='cba6f7'))
211 |
212 |         ws.merge_cells('A1:H1')
213 |         ws['A1'] = f'RELATÓRIO DE ATIVIDADES - {datetime.now().strftime("%d/%m/%Y %H:%M")}'
214 |         ws['A1'].font = Font(name='Arial', size=14, bold=True, color='cba6f7')
215 |         ws['A1'].alignment = Alignment(horizontal='center')
216 |
217 |         ws['A3'] = f'Total de atividades: {len(atividades)}'
218 |         ws['A3'].font = Font(name='Arial', size=11, bold=True, color='89b4fa')
219 |
220 |         headers = ['Data/Hora', 'Tipo', 'Descrição', 'Peças', 'Origem', 'Destino', 'Recibo', 'Status']
221 |         for col, h in enumerate(headers, 1):
222 |             cell = ws.cell(row=5, column=col, value=h)
223 |             cell.font = header_font
224 |             cell.fill = header_fill
225 |             cell.alignment = Alignment(horizontal='center')
226 |             cell.border = border
227 |
228 |         for i, a in enumerate(atividades, 6):
229 |             dados = [
230 |                 a.event_at.strftime("%d/%m/%Y %H:%M") if a.event_at else '-',
231 |                 'Manutenção' if a.kind == 'MANUTENCAO' else 'Movimentação',
232 |                 a.notes or '-',
233 |                 a.parts_used or '-',
234 |                 a.from_location or '-',
235 |                 a.to_location or '-',
236 |                 a.numero_recibo or '-',
237 |                 a.status_atividade or 'Concluída'
238 |             ]
239 |             for col, val in enumerate(dados, 1):
240 |                 cell = ws.cell(row=i, column=col, value=val)
241 |                 cell.font = white_font
242 |                 cell.fill = fill1 if i % 2 == 0 else fill2
243 |                 cell.border = border
244 |
245 |         col_widths = [18, 15, 40, 20, 18, 18, 15, 15]
246 |         for col, w in enumerate(col_widths, 1):
247 |             ws.column_dimensions[openpyxl.utils.get_column_letter(col)].width = w
248 |
249 |         wb.save(filepath)
250 |         return filepath

```

=== ARQUIVO: .\app\services\report_service.py ===

```

001 | from __future__ import annotations
002 |
003 | import csv
004 | from datetime import datetime
005 |
006 | from reportlab.lib import colors
007 | from reportlab.lib.pagesizes import A4
008 | from reportlab.lib.styles import getSampleStyleSheet
009 | from reportlab.platypus import Paragraph, SimpleDocTemplate, Spacer, Table, TableStyle
010 |
011 | from app.models.activity import Activity
012 | from app.models.printer import Printer
013 |
014 |
015 | class ReportService:

```

```

016 | @staticmethod
017 | def export_history_csv(path: str, printer: Printer, activities: list[Activity]) -> None:
018 |     with open(path, "w", newline="", encoding="utf-8") as f:
019 |         w = csv.writer(f, delimiter=";")
020 |         w.writerow(["Patrimônio", "Modelo", "Serial", "Data/Hora", "Tipo", "Descrição", "Peças", "De",
021 |                     for a in activities:
022 |                         w.writerow([
023 |                             printer.patrimonio, printer.modelo, printer.serial,
024 |                             a.event_at.strftime("%d/%m/%Y %H:%M"),
025 |                             "Manutenção" if a.kind == "MANUTENCAO" else "Movimentação",
026 |                             a.notes or "",
027 |                             a.parts_used or "",
028 |                             a.from_location or "",
029 |                             a.to_location or ""
030 |                         ])
031 |
032 | @staticmethod
033 | def export_pdf(path: str, printer: Printer, activities: list[Activity], maintenance_count: int) -> None:
034 |     styles = getSampleStyleSheet()
035 |     doc = SimpleDocTemplate(path, pagesize=A4, leftMargin=28, rightMargin=28, topMargin=28, bottomMargin=28)
036 |     elems = []
037 |
038 |     elems.append(Paragraph("Relatório de Impressora", styles["Title"]))
039 |     elems.append(Spacer(1, 12))
040 |
041 |     header = [
042 |         ["Patrimônio", printer.patrimonio or "-"],
043 |         ["Modelo", printer.modelo or "-"],
044 |         ["Serial", printer.serial or "-"],
045 |         ["Status", printer.status or "-"],
046 |         ["Local atual", printer.local_atual or "-"],
047 |         ["Manutenções (contador)", str(maintenance_count)],
048 |     ]
049 |     t_header = Table(header, colWidths=[130, 360])
050 |     t_header.setStyle(TableStyle([
051 |         ("BACKGROUND", (0, 0), (0, -1), colors.whitesmoke),
052 |         ("BOX", (0, 0), (-1, -1), 0.8, colors.grey),
053 |         ("INNERGRID", (0, 0), (-1, -1), 0.4, colors.lightgrey),
054 |         ("FONTSIZE", (0, 0), (-1, -1), 9),
055 |         ("PADDING", (0, 0), (-1, -1), 6),
056 |     ]))
057 |     elems.append(t_header)
058 |     elems.append(Spacer(1, 14))
059 |
060 |     if (printer.observacao or "").strip():
061 |         elems.append(Paragraph("Observação geral:", styles["Heading3"]))
062 |         elems.append(Paragraph((printer.observacao or "").replace("\n", "<br/>"), styles["BodyText"]))
063 |         elems.append(Spacer(1, 10))
064 |
065 |     elems.append(Paragraph("Histórico de atividades:", styles["Heading3"]))
066 |     elems.append(Spacer(1, 6))
067 |
068 |     data = [
069 |         ["Data/Hora", "Tipo", "Descrição", "Peças", "De", "Para"]
070 |         for a in activities:
071 |             data.append([
072 |                 a.event_at.strftime("%d/%m/%Y %H:%M"),
073 |                 "Manutenção" if a.kind == "MANUTENCAO" else "Movimentação",
074 |                 (a.notes or "")[:1200],
075 |                 (a.parts_used or "")[:400],
076 |                 a.from_location or "",
077 |                 a.to_location or "",
078 |             ])

```

```

079 |         t = Table(data, colWidths=[75, 70, 170, 90, 55, 55])
080 |         t.setStyle(TableStyle([
081 |             ("BACKGROUND", (0, 0), (-1, 0), colors.lightgrey),
082 |             ("BOX", (0, 0), (-1, -1), 0.8, colors.grey),
083 |             ("INNERGRID", (0, 0), (-1, -1), 0.4, colors.lightgrey),
084 |             ("FONTSIZE", (0, 0), (-1, -1), 8),
085 |             ("VALIGN", (0, 0), (-1, -1), "TOP"),
086 |             ("PADDING", (0, 0), (-1, -1), 5),
087 |         ]))
088 |         elems.append(t)
089 |
090 |         elems.append(Spacer(1, 10))
091 |         elems.append(Paragraph(f"Gerado em: {datetime.now().strftime('%d/%m/%Y %H:%M')}", styles["Normal"]))
092 |
093 |         doc.build(elems)
094 |

```

=== ARQUIVO: .lapl\services\snmp_service.py ===

```

001 | from __future__ import annotations
002 |
003 | import logging
004 | from datetime import datetime
005 | from typing import Optional
006 |
007 | from sqlalchemy.orm import Session
008 |
009 | from app.models.printer_health import PrinterHealth
010 | from app.models.printer import Printer
011 | from app.utils.monitoring_config import MonitoringConfig
012 | from app.utils.network import ping, snmp_get_printer_info
013 |
014 | log = logging.getLogger(__name__)
015 |
016 |
017 | class SnmpMonitorService:
018 |     def __init__(self, session: Session, alert_service=None, notificador=None):
019 |         self.session = session
020 |         self.alert_service = alert_service
021 |         self.notificador = notificador
022 |         self.config = MonitoringConfig.carregar()
023 |
024 |     def verificar_impressora(self, printer: Printer) -> PrinterHealth:
025 |         host = self._resolve_host(printer)
026 |         ping_ms = ping(host, timeout_ms=self.config.ping_timeout_ms, retries=self.config.ping_retries)
027 |
028 |         health = PrinterHealth(
029 |             printer_id=printer.id,
030 |             timestamp=datetime.utcnow(),
031 |             is_online=ping_ms is not None,
032 |             ping_ms=ping_ms,
033 |         )
034 |
035 |         if ping_ms is not None and self._snmp_disponivel():
036 |             info = self._consultar_snmp(host)
037 |             health.toner_level = info.get("toner_level")
038 |             health.toner_max = info.get("toner_max")
039 |             health.drum_life = info.get("drum_life")
040 |             health.drum_max = info.get("drum_max")
041 |             health.page_count = info.get("page_count")
042 |             health.error_code = info.get("error_code")

```

```

043 |         health.error_message = info.get("error_message")
044 |
045 |     self.session.add(health)
046 |     self.session.commit()
047 |     return health
048 |
049 | def verificar_todas(self) -> list[PrinterHealth]:
050 |     impressoras = self.session.query(Printer).all()
051 |     resultados = []
052 |     for p in impressoras:
053 |         try:
054 |             h = self.verificar_impressora(p)
055 |             resultados.append(h)
056 |         except Exception as e:
057 |             log.exception("Erro ao verificar %s: %s", p.ip or p.hostname, e)
058 |     if self.config.auto_alert_offline or self.config.auto_alert_toner:
059 |         self.gerar_alertas()
060 |     return resultados
061 |
062 | def gerar_alertas(self) -> None:
063 |     if not self.alert_service:
064 |         return
065 |     if self.config.auto_alert_offline:
066 |         self._alertar_offline()
067 |     if self.config.auto_alert_toner:
068 |         self._alertar_toner()
069 |
070 | def _alertar_offline(self) -> None:
071 |     from app.models.alert import Alert
072 |     for printer, _health in self.impressoras_offline():
073 |         ja_tem = self.session.query(Alert).filter(
074 |             Alert.printer_id == printer.id,
075 |             Alert.tipo == "offline",
076 |             Alert.resolvido == False,
077 |             Alert.deleted_at == None,
078 |         ).first()
079 |         if ja_tem:
080 |             continue
081 |         self.alert_service.criar(
082 |             printer_id=printer.id,
083 |             tipo="offline",
084 |             titulo=f"Offline: {printer.patrimonio}",
085 |             descricao=f"Impressora {printer.modelo} ({printer.patrimonio}) não responde ao ping.",
086 |         )
087 |         log.info("Alerta offline criado para %s", printer.patrimonio)
088 |
089 | def _alertar_toner(self) -> None:
090 |     from app.models.alert import Alert
091 |     for printer, _health in self.impressoras_toner_baixo():
092 |         ja_tem = self.session.query(Alert).filter(
093 |             Alert.printer_id == printer.id,
094 |             Alert.tipo == "toner",
095 |             Alert.resolvido == False,
096 |             Alert.deleted_at == None,
097 |         ).first()
098 |         if ja_tem:
099 |             continue
100 |         self.alert_service.criar(
101 |             printer_id=printer.id,
102 |             tipo="toner",
103 |             titulo=f"Toner baixo: {printer.patrimonio}",
104 |             descricao=f"Impressora {printer.modelo} ({printer.patrimonio}) está com nível de toner abaixo",
105 |         )

```

```

106 |         log.info("Alerta toner criado para %s", printer.patrimonio)
107 |
108 | def _resolve_host(self, printer: Printer) -> str:
109 |     return printer.ip or printer.hostname or "0.0.0.0"
110 |
111 | def _snmp_disponivel(self) -> bool:
112 |     try:
113 |         import pysnmp # noqa: F401
114 |         return True
115 |     except ImportError:
116 |         return False
117 |
118 | def _consultar_snmp(self, host: str) -> dict:
119 |     if self._snmp_disponivel():
120 |         return snmp_get_printer_info(host, community=self.config.snmp_community)
121 |     return {}
122 |
123 | def ultima_leitura(self, printer_id: str) -> Optional[PrinterHealth]:
124 |     return (
125 |         self.session.query(PrinterHealth)
126 |         .filter(PrinterHealth.printer_id == printer_id)
127 |         .order_by(PrinterHealth.timestamp.desc())
128 |         .first()
129 |     )
130 |
131 | def status_atual(self, printer_id: str) -> Optional[PrinterHealth]:
132 |     threshold = self.config.offline_threshold
133 |     leituras = (
134 |         self.session.query(PrinterHealth)
135 |         .filter(PrinterHealth.printer_id == printer_id)
136 |         .order_by(PrinterHealth.timestamp.desc())
137 |         .limit(threshold)
138 |         .all()
139 |     )
140 |     if not leituras:
141 |         return None
142 |     online_count = sum(1 for l in leituras if l.is_online)
143 |     if online_count == 0:
144 |         if leituras:
145 |             ultima = leituras[0]
146 |             ultima.is_online = False
147 |             return ultima
148 |     return leituras[0] if leituras else None
149 |
150 | def impressoras_offline(self) -> list[tuple[Printer, Optional[PrinterHealth]]]:
151 |     impressoras = self.session.query(Printer).all()
152 |     offline = []
153 |     for p in impressoras:
154 |         status = self.status_atual(p.id)
155 |         if status and not status.is_online:
156 |             offline.append((p, status))
157 |     return offline
158 |
159 | def impressoras_toner_baixo(self) -> list[tuple[Printer, PrinterHealth]]:
160 |     impressoras = self.session.query(Printer).all()
161 |     baixo = []
162 |     for p in impressoras:
163 |         status = self.ultima_leitura(p.id)
164 |         if status and status.toner_level is not None and status.toner_max:
165 |             pct = (status.toner_level / status.toner_max) * 100
166 |             if pct < self.config.toner_warn_percent:
167 |                 baixo.append((p, status))
168 |     return baixo

```

```

169 |
170 |     def historico(self, printer_id: str, limit: int = 50) -> list[PrinterHealth]:
171 |         return (
172 |             self.session.query(PrinterHealth)
173 |             .filter(PrinterHealth.printer_id == printer_id)
174 |             .order_by(PrinterHealth.timestamp.desc())
175 |             .limit(limit)
176 |             .all()
177 |         )

```

=== ARQUIVO: .\app\services\technician_service.py ===

```

001 | from __future__ import annotations
002 |
003 | import logging
004 | from datetime import datetime
005 | from typing import TYPE_CHECKING, Any, Optional
006 |
007 | from app.models.base import utcnow
008 | from db import safe_commit
009 | from sqlalchemy.orm import Session
010 |
011 | from app.models import Technician
012 | from app.utils.cache import cached, invalidate
013 | from app.utils.sanitize import sanitizar
014 |
015 | if TYPE_CHECKING:
016 |     from app.services.audit_service import AuditService
017 |
018 | log = logging.getLogger(__name__)
019 |
020 |
021 | class TechnicianService:
022 |     def __init__(self, session: Session, audit_service: Optional[AuditService] = None, user_id: Optional[int] = None):
023 |         self.session: Session = session
024 |         self.audit_service: Optional[AuditService] = audit_service
025 |         self.user_id: Optional[int] = user_id
026 |
027 |     @cached(ttl=60, namespace="technician")
028 |     def listar_todos(self, limite: Optional[int] = None, offset: Optional[int] = None) -> list[Technician]:
029 |         query = self.session.query(Technician).filter(Technician.deleted_at == None).order_by(Technician.nome_exibicao)
030 |         if limite is not None:
031 |             query = query.limit(limite)
032 |         if offset is not None:
033 |             query = query.offset(offset)
034 |         return query.all()
035 |
036 |     @cached(ttl=60, namespace="technician")
037 |     def listar_ativos(self, limite: Optional[int] = None, offset: Optional[int] = None) -> list[Technician]:
038 |         query = self.session.query(Technician).filter(Technician.deleted_at == None).filter(
039 |             Technician.ativo == True
040 |         ).order_by(Technician.nome_exibicao)
041 |         if limite is not None:
042 |             query = query.limit(limite)
043 |         if offset is not None:
044 |             query = query.offset(offset)
045 |         return query.all()
046 |
047 |     @cached(ttl=30, namespace="technician")
048 |     def buscar_por_id(self, tecnico_id: int) -> Optional[Technician]:
049 |         return self.session.query(Technician).filter(

```

```

050 |         Technician.deleted_at == None, Technician.id == tecnico_id
051 |     ).first()
052 |
053 | def buscar_por_nome(self, nome: str) -> Optional[Technician]:
054 |     return self.session.query(Technician).filter(
055 |         Technician.deleted_at == None, Technician.nome_completo == nome
056 |     ).first()
057 |
058 | def criar(self, nome_completo: str, nome_exibicao: str = "", telefone: str = "", email: str = "") -> Technician:
059 |     t = Technician(
060 |         nome_completo=sanitizar(nome_completo, "Technician", "nome_completo"),
061 |         nome_exibicao=sanitizar(nome_exibicao or nome_completo.split()[0], "Technician", "nome_exibicao"),
062 |         telefone=sanitizar(telefone, "Technician", "telefone"),
063 |         email=sanitizar(email, "Technician", "email"),
064 |     )
065 |     self.session.add(t)
066 |     self.session.commit()
067 |     if self.audit_service:
068 |         self.audit_service.log(self.user_id, "criar", tabela_alvo="technicians", registro_id=t.id, dados={"nome_completo": nome_completo, "nome_exibicao": nome_exibicao, "telefone": telefone, "email": email})
069 |     invalidate("technician")
070 |     return t
071 |
072 | def atualizar(self, tecnico: Technician, **kwargs: Any) -> None:
073 |     if self.audit_service:
074 |         dados_antes = {chave: getattr(tecnico, chave, None) for chave in kwargs}
075 |         for chave, valor in kwargs.items():
076 |             if hasattr(tecnico, chave):
077 |                 if isinstance(valor, str):
078 |                     valor = sanitizar(valor, "Technician", chave)
079 |                 setattr(tecnico, chave, valor)
080 |         self.session.commit()
081 |     if self.audit_service:
082 |         self.audit_service.log(self.user_id, "atualizar", tabela_alvo="technicians", registro_id=tecnico.id, dados=dados_antes)
083 |     invalidate("technician")
084 |
085 | def excluir(self, tecnico: Technician) -> None:
086 |     if self.audit_service:
087 |         self.audit_service.log(self.user_id, "excluir", tabela_alvo="technicians", registro_id=tecnico.id, dados={"nome_completo": tecnico.nome_completo, "nome_exibicao": tecnico.nome_exibicao, "telefone": tecnico.telefone, "email": tecnico.email})
088 |     tecnico.deleted_at = utcnow()
089 |     self.session.commit()
090 |     invalidate("technician")
091 |
092 | def associar_usuario(self, tecnico_id: int, user_id: int | None) -> None:
093 |     tecnico = self.buscar_por_id(tecnico_id)
094 |     if not tecnico:
095 |         return
096 |     if self.audit_service:
097 |         dados_antes = {"user_id": tecnico.user_id}
098 |         tecnico.user_id = user_id
099 |         self.session.commit()
100 |     if self.audit_service:
101 |         self.audit_service.log(self.user_id, "associar_usuario", tabela_alvo="technicians", registro_id=tecnico.id, dados=dados_antes)
102 |     invalidate("technician")
103 |
104 | def listar_especialidades(self, tecnico_id: int) -> list[str]:
105 |     from app.models.technician_specialty import TechnicianSpecialty
106 |     registros = self.session.query(TechnicianSpecialty).filter(
107 |         TechnicianSpecialty.technician_id == tecnico_id
108 |     ).all()
109 |     return [r.modelo for r in registros]
110 |
111 | def adicionar_especialidades(self, tecnico_id: int, modelos: list[str]) -> None:
112 |     from app.models.technician_specialty import TechnicianSpecialty

```

```

113 |         existentes = set(self.listar_especialidades(tecnico_id))
114 |         for modelo in modelos:
115 |             if modelo and modelo not in existentes:
116 |                 self.session.add(TechnicianSpecialty(technician_id=tecnico_id, modelo=modelo))
117 |                 existentes.add(modelo)
118 |         safe_commit(self.session)
119 |
120 |     def remover_especialidades(self, tecnico_id: int, modelos: list[str]) -> None:
121 |         from app.models.technician_specialty import TechnicianSpecialty
122 |         self.session.query(TechnicianSpecialty).filter(
123 |             TechnicianSpecialty.technician_id == tecnico_id,
124 |             TechnicianSpecialty.modelo.in_(modelos),
125 |         ).delete(synchronize_session=False)
126 |         safe_commit(self.session)
127 |
128 |     def limpar_especialidades(self, tecnico_id: int) -> None:
129 |         from app.models.technician_specialty import TechnicianSpecialty
130 |         self.session.query(TechnicianSpecialty).filter(
131 |             TechnicianSpecialty.technician_id == tecnico_id
132 |         ).delete(synchronize_session=False)
133 |         safe_commit(self.session)
134 |
135 |     def nomes_exibicao(self) -> list[str]:
136 |         return [t.nome_exibicao for t in self.listar_ativos()]
137 |
138 |     @cached(ttl=60, namespace="technician")
139 |     def contar_todos(self) -> int:
140 |         return self.session.query(Technician).filter(Technician.deleted_at == None).count()
141 |
142 |     def listar_excluidos(self, limite: int = 50) -> list[Technician]:
143 |         return self.session.query(Technician).filter(
144 |             Technician.deleted_at != None
145 |         ).order_by(Technician.deleted_at.desc()).limit(limite).all()
146 |
147 |     def restaurar(self, obj: Technician) -> None:
148 |         obj.deleted_at = None
149 |         safe_commit(self.session)
150 |         if self.audit_service:
151 |             self.audit_service.log(self.user_id, "restaurar", tabela_alvo="technicians", registro_id=obj.id)
152 |         invalidate("technician")
153 |

```

=== ARQUIVO: .lapplservices\user_service.py ===

```

001 | from __future__ import annotations
002 |
003 | import logging
004 | from datetime import datetime
005 | from typing import TYPE_CHECKING, Any, Optional
006 |
007 | from db import safe_commit
008 | from sqlalchemy.orm import Session
009 |
010 | from app.models import User
011 | from app.utils.sanitize import sanitizar
012 | from app.utils.security import hash_password
013 |
014 | if TYPE_CHECKING:
015 |     from app.services.audit_service import AuditService
016 |
017 | log = logging.getLogger(__name__)

```



```

018 |
019 |
020 | class UserService:
021 |     def __init__(self, session: Session, audit_service: Optional[AuditService] = None, user_id: Optional[int] = None):
022 |         self.session: Session = session
023 |         self.audit_service: Optional[AuditService] = audit_service
024 |         self.user_id: Optional[int] = user_id
025 |
026 |     def listar_todos(self, limite: Optional[int] = None, offset: Optional[int] = None) -> list[User]:
027 |         query = self.session.query(User).filter(User.deleted_at == None).order_by(User.nome)
028 |         if limite is not None:
029 |             query = query.limit(limite)
030 |         if offset is not None:
031 |             query = query.offset(offset)
032 |         return query.all()
033 |
034 |     def buscar_por_id(self, user_id: int) -> Optional[User]:
035 |         return self.session.query(User).filter(User.deleted_at == None, User.id == user_id).first()
036 |
037 |     def buscar_por_username_ou_email(self, valor: str) -> Optional[User]:
038 |         return self.session.query(User).filter(
039 |             User.deleted_at == None, (User.email == valor) | (User.username == valor)
040 |         ).first()
041 |
042 |     def verificar_existente(self, email: str, username: str) -> Optional[User]:
043 |         return self.session.query(User).filter(
044 |             User.deleted_at == None, (User.email == email) | (User.username == username)
045 |         ).first()
046 |
047 |     def criar(self, nome: str, username: str, email: str, senha: str, perfil: str = "visualizador") -> User:
048 |         u = User(
049 |             nome=sanitizar(nome, "User", "nome"),
050 |             username=sanitizar(username, "User", "username"),
051 |             email=sanitizar(email, "User", "email"),
052 |             senha_hash=hash_password(senha),
053 |             perfil=sanitizar(perfil, "User", "perfil"),
054 |             ativo=True
055 |         )
056 |         self.session.add(u)
057 |         self.session.commit()
058 |         if self.audit_service:
059 |             self.audit_service.log(self.user_id, "criar", tabela_alvo="users", registro_id=u.id, dados_depois={})
060 |         return u
061 |
062 |     def atualizar(self, user: User, **kwargs: Any) -> None:
063 |         if self.audit_service:
064 |             dados_antes = {chave: getattr(user, chave, None) for chave in kwargs}
065 |             senha = kwargs.pop("senha", None)
066 |             if senha:
067 |                 user.senha_hash = hash_password(senha)
068 |             for chave, valor in kwargs.items():
069 |                 if hasattr(user, chave):
070 |                     if isinstance(valor, str):
071 |                         valor = sanitizar(valor, "User", chave)
072 |                     setattr(user, chave, valor)
073 |             self.session.commit()
074 |             if self.audit_service:
075 |                 self.audit_service.log(self.user_id, "atualizar", tabela_alvo="users", registro_id=user.id, dados_depois=dados_antes)
076 |
077 |     def contar_todos(self) -> int:
078 |         return self.session.query(User).filter(User.deleted_at == None).count()
079 |
080 |     def listar_excluidos(self, limite: int = 50) -> list[User]:

```

```

001 |         return self.session.query(User).filter(
002 |             User.deleted_at != None
003 |         ).order_by(User.deleted_at.desc()).limit(limite).all()
004 |
005 |     def restaurar(self, obj: User) -> None:
006 |         obj.deleted_at = None
007 |         safe_commit(self.session)
008 |         if self.audit_service:
009 |             self.audit_service.log(self.user_id, "restaurar", tabela_alvo="users", registro_id=obj.id)
010 |

```

=== ARQUIVO: .\appl\services__init__.py ===

```

001 | from .activity_service import ActivityService
002 | from .alert_service import AlertService
003 | from .audit_service import AuditService
004 | from .company_service import CompanyService
005 | from .dashboard_service import DashboardService
006 | from .login_history_service import LoginHistoryService
007 | from .maintenance_scheduler import MaintenanceScheduler
008 | from .part_service import PartService
009 | from .printer_location_service import PrinterLocationService
010 | from .printer_service import PrinterService
011 | from .snmp_service import SnmpMonitorService
012 | from .technician_service import TechnicianService
013 | from .user_service import UserService

```

=== ARQUIVO: .\appl\utils\cache.py ===

```

001 | from __future__ import annotations
002 |
003 | import logging
004 | import time
005 | from functools import wraps
006 | from typing import Any, Callable
007 |
008 | log = logging.getLogger(__name__)
009 |
010 | _MISSING: Any = object()
011 |
012 |
013 | class TTLCache:
014 |     def __init__(self) -> None:
015 |         self._store: dict[str, tuple[float, Any, float]] = {}
016 |
017 |     def get(self, key: str) -> Any:
018 |         entry = self._store.get(key)
019 |         if entry is not None:
020 |             ts, value, ttl = entry
021 |             if time.monotonic() - ts < ttl:
022 |                 return value
023 |             del self._store[key]
024 |         return _MISSING
025 |
026 |     def set(self, key: str, value: Any, ttl: float) -> None:
027 |         self._store[key] = (time.monotonic(), value, ttl)
028 |
029 |     def invalidate_prefix(self, prefix: str) -> None:
030 |         pfx = prefix + ":"

```

```

031 |         to_remove = [k for k in self._store if k.startswith(pfx)]
032 |         for k in to_remove:
033 |             del self._store[k]
034 |
035 |     def clear(self) -> None:
036 |         self._store.clear()
037 |
038 |
039 | _global_cache = TTLCache()
040 |
041 |
042 | def invalidate(namespace: str) -> None:
043 |     _global_cache.invalidate_prefix(namespace)
044 |
045 |
046 | def invalidate_all() -> None:
047 |     _global_cache.clear()
048 |
049 |
050 | def cached(ttl: int = 30, namespace: str = "") -> Callable[..., Any]:
051 |     def decorator(fn: Callable[..., Any]) -> Any:
052 |         ns = namespace or fn.__name__
053 |
054 |         @wraps(fn)
055 |         def wrapper(*args: Any, **kwargs: Any) -> Any:
056 |             key_parts: list[str] = [ns, fn.__name__]
057 |             for a in args[1:]:
058 |                 key_parts.append(repr(a))
059 |             for k in sorted(kwargs):
060 |                 key_parts.append(f"{k}={kwargs[k]!r}")
061 |             cache_key = ":".join(key_parts)
062 |             result = _global_cache.get(cache_key)
063 |             if result is not _MISSING:
064 |                 return result
065 |             result = fn(*args, **kwargs)
066 |             _global_cache.set(cache_key, result, ttl=ttl)
067 |             return result
068 |
069 |         def _invalidate() -> None:
070 |             invalidate(ns)
071 |
072 |         wrapper.invalidate = _invalidate
073 |         return wrapper
074 |
075 |     return decorator

```

=== ARQUIVO: .\apltutils\constants.py ===

```

001 | STATUS_MANUTENCAO = ["Em manutenção", "Manutenção", "Aguardando peça", "Parada"]
002 | STATUS_OPERACIONAL = ["Operacional", "Em uso"]
003 | STATUS_ATIVIDADE OPCOES = [
004 |     "Aberta",
005 |     "Aguardando Peça",
006 |     "Técnico Designado",
007 |     "Em Deslocamento",
008 |     "Em Manutenção",
009 |     "Em Atendimento",
010 |     "Aguardando Aprovação",
011 |     "Concluído",
012 |     "Verificada",
013 | ]

```

=== ARQUIVO: .lapp\utils\effects.py ===

```
001 | from PySide6.QtGui import QColor
002 | from PySide6.QtWidgets import QGraphicsDropShadowEffect, QWidget
003 |
004 |
005 | def _hex_to_qcolor(hex_cor: str, alpha: int = 60) -> QColor:
006 |     h = hex_cor.lstrip("#")
007 |     if len(h) == 6:
008 |         r, g, b = int(h[0:2], 16), int(h[2:4], 16), int(h[4:6], 16)
009 |         return QColor(r, g, b, alpha)
010 |     return QColor(0, 0, 0, alpha)
011 |
012 |
013 | def sombra_card(widget: QWidget, blur: int = 20, offset: int = 4) -> QGraphicsDropShadowEffect:
014 |     efeito = QGraphicsDropShadowEffect(widget)
015 |     efeito.setBlurRadius(blur)
016 |     efeito.setOffset(0, offset)
017 |     efeito.setColor(_hex_to_qcolor("#000000", 60))
018 |     widget.setGraphicsEffect(efeito)
019 |     return efeito
020 |
021 |
022 | def sombra_glow(widget: QWidget, cor: str = "#a78bfa", blur: int = 30) -> QGraphicsDropShadowEffect:
023 |     efeito = QGraphicsDropShadowEffect(widget)
024 |     efeito.setBlurRadius(blur)
025 |     efeito.setOffset(0, 0)
026 |     efeito.setColor(_hex_to_qcolor(cor, 40))
027 |     widget.setGraphicsEffect(efeito)
028 |     return efeito
```

=== ARQUIVO: .lapp\utils\helpers.py ===

```
001 | from datetime import datetime
002 | from typing import Optional
003 |
004 |
005 | def limpar_local(texto: Optional[str]) -> str:
006 |     if not texto:
007 |         return ""
008 |     for prefixo in ["■ ", "■ "]:
009 |         if texto.startswith(prefixo):
010 |             return texto[len(prefixo):].strip()
011 |     return texto.strip()
012 |
013 |
014 | def formatar_data(data: Optional[datetime], formato: str = "%d/%m/%Y") -> str:
015 |     return data.strftime(formato) if data else "-"
016 |
017 |
018 | def formatar_data_hora(data: Optional[datetime], formato: str = "%d/%m/%Y %H:%M") -> str:
019 |     return data.strftime(formato) if data else "-"
020 |
021 |
022 | def parse_data(texto: Optional[str]) -> Optional[datetime]:
023 |     if not texto:
024 |         return None
025 |     for fmt in ["%d/%m/%Y %H:%M", "%d/%m/%Y"]:
```

```

026 |         try:
027 |             return datetime.strptime(texto.strip(), fmt)
028 |         except ValueError:
029 |             continue
030 |     return None
031 |
032 |
033 | def encurtar(texto: Optional[str], max_len: int = 40) -> str:
034 |     if not texto:
035 |         return "-"
036 |     if len(texto) > max_len:
037 |         return texto[:max_len-3] + "..."
038 |     return texto

```

=== ARQUIVO: .app\utils\limporter.py ===

```

001 | import csv
002 | import io
003 | import os
004 | import re
005 | from dataclasses import dataclass, field
006 | from typing import Any, Optional
007 |
008 | from app.utils.sanitize import sanitizar
009 |
010 |
011 | @dataclass
012 | class ImportResult:
013 |     total: int = 0
014 |     importados: int = 0
015 |     erros: list[tuple[int, str]] = field(default_factory=list)
016 |
017 |
018 | def detectar_encoding(caminho: str) -> str:
019 |     with open(caminho, "rb") as f:
020 |         raw = f.read(4096)
021 |         try:
022 |             import chardet
023 |             return chardet.detect(raw).get("encoding", "utf-8") or "utf-8"
024 |         except ImportError:
025 |             if raw.startswith(b"\xff\xfe"):
026 |                 return "utf-16-le"
027 |             if raw.startswith(b"\xfe\xff"):
028 |                 return "utf-16-be"
029 |             return "utf-8"
030 |
031 |
032 | def _limpar_cabecalho(nome: str) -> str:
033 |     nome = nome.strip().lower()
034 |     nome = re.sub(r"^[a-z0-9\u00e0-\u00ff]", " ", nome)
035 |     nome = re.sub(r"\s+", "_", nome).strip("_")
036 |     return nome
037 |
038 |
039 | def normalizar_modelos(valor: str) -> str:
040 |     """Normaliza string de modelos: separa por /, ; ou , e deduplica.
041 |     Ex: 'HP P1102, M1132; HP 2035' → 'HP P1102/M1132/HP 2035'
042 |     """
043 |     if not valor:
044 |         return ""
045 |     partes = re.split(r"\s*[;,/]s*", valor.strip())

```

```

046 |     vistos: set[str] = set()
047 |     resultado: list[str] = []
048 |     for p in partes:
049 |         p = p.strip()
050 |         if not p:
051 |             continue
052 |         chave = p.upper()
053 |         if chave not in vistos:
054 |             vistos.add(chave)
055 |             resultado.append(p)
056 |     return "/".join(resultado)
057 |
058 |
059 | def merge_modelos(existente: str, novos: str) -> str:
060 |     """Mescla string de modelos existente com novos, sem duplicar.
061 |     Ex: merge_modelos('HP P1102/M1132', 'M1132/HP 2035') → 'HP P1102/M1132/HP 2035'
062 |     """
063 |     todos = f"{existente}/{novos}" if existente and novos else (existente or novos)
064 |     return normalizar_modelos(todos)
065 |
066 |
067 | COLUNAS_PADRAO: dict[str, dict[str, str]] = {
068 |     "printers": {
069 |         "patrimonio": "patrimonio",
070 |         "patrimônio": "patrimonio",
071 |         "n_patrimonio": "patrimonio",
072 |         "n_patrimônio": "patrimonio",
073 |         "num_patrimonio": "patrimonio",
074 |         "modelo": "modelo",
075 |         "marca": "marca",
076 |         "serial": "serial",
077 |         "numero_serial": "serial",
078 |         "número_serial": "serial",
079 |         "n_serial": "serial",
080 |         "no_serial": "serial",
081 |         "status": "status",
082 |         "local": "local_atual",
083 |         "local_atual": "local_atual",
084 |         "ip": "ip_rede",
085 |         "ip_rede": "ip_rede",
086 |         "tecnico": "tecnico",
087 |         "técnico": "tecnico",
088 |         "tipo": "tipo",
089 |         "observacao": "observacao",
090 |         "observação": "observacao",
091 |         "obs": "observacao",
092 |         "mac": "mac_address",
093 |         "mac_address": "mac_address",
094 |     },
095 |     "parts": {
096 |         "codigo": "codigo",
097 |         "código": "codigo",
098 |         "cod": "codigo",
099 |         "nome": "nome",
100 |         "descricao": "descricao",
101 |         "descrição": "descricao",
102 |         "desc": "descricao",
103 |         "quantidade": "quantidade_estoque",
104 |         "qtd": "quantidade_estoque",
105 |         "qty": "quantidade_estoque",
106 |         "estoque": "quantidade_estoque",
107 |         "estoque_minimo": "estoque_minimo",
108 |         "est_min": "estoque_minimo",

```

```

109 |         "minimo": "estoque_minimo",
110 |         "mínimo": "estoque_minimo",
111 |         "preco": "preco_unitario",
112 |         "preço": "preco_unitario",
113 |         "preco_unitario": "preco_unitario",
114 |         "modelo_compativel": "modelo_compativel",
115 |         "modelo": "modelo_compativel",
116 |         "marca": "marca",
117 |         "fabricante": "marca",
118 |         "brand": "marca",
119 |         "categoria": "categoria",
120 |         "cat": "categoria",
121 |         "category": "categoria",
122 |         "fornecedor": "fornecedor",
123 |         "supplier": "fornecedor",
124 |     },
125 |     "companies": {
126 |         "nome": "nome",
127 |         "empresa": "nome",
128 |         "razao_social": "nome",
129 |         "razão_social": "nome",
130 |         "cnpj": "cnpj",
131 |         "cpf": "cnpj",
132 |         "endereco": "endereco",
133 |         "endereço": "endereco",
134 |         "cidade": "cidade",
135 |         "uf": "uf",
136 |         "estado": "uf",
137 |         "telefone": "telefone",
138 |         "tel": "telefone",
139 |         "fone": "telefone",
140 |         "email": "email",
141 |         "e_mail": "email",
142 |         "tipo": "tipo",
143 |         "observacao": "observacao",
144 |         "observação": "observacao",
145 |     },
146 | }
147 |
148 | CAMPOS_OBRIGATORIOS: dict[str, list[str]] = {
149 |     "printers": ["patrimonio"],
150 |     "parts": ["nome"],
151 |     "companies": ["nome"],
152 | }
153 |
154 | CAMPOS_EDITAVEIS: dict[str, list[str]] = {
155 |     "printers": ["patrimonio", "modelo", "marca", "serial", "status", "local_atual",
156 |                 "ip_rede", "mac_address", "tecnico", "tipo", "observacao"],
157 |     "parts": ["codigo", "nome", "descricao", "quantidade_estoque", "estoque_minimo",
158 |              "preco_unitario", "modelo_compativel", "marca", "categoria", "fornecedor"],
159 |     "companies": ["nome", "cnpj", "endereco", "cidade", "uf", "telefone", "email", "tipo", "observacao"],
160 | }
161 |
162 |
163 | def _valor_convertido(valor: str, campo: str) -> Any:
164 |     if valor is None:
165 |         return None
166 |     valor = valor.strip()
167 |     if not valor:
168 |         return None
169 |     if campo in ("quantidade_estoque", "estoque_minimo"):
170 |         try:
171 |             return int(float(valor))

```

```

172 |         except ValueError:
173 |             return 0
174 |     if campo == "preco_unitario":
175 |         valor = valor.replace(",", ".")
176 |         try:
177 |             return float(valor)
178 |         except ValueError:
179 |             return 0.0
180 |     return valor
181 |
182 |
183 | def parse_csv(caminho: str, encoding: str = "utf-8", delimiter: str = ";") -> tuple[list[str], list[list[str]]]:
184 |     with open(caminho, encoding=encoding, errors="replace") as f:
185 |         reader = csv.reader(f, delimiter=delimiter)
186 |         linhas = list(reader)
187 |     if not linhas:
188 |         return [], []
189 |     cabecalho = [c.lstrip("\ufeff") for c in linhas[0]]
190 |     dados = linhas[1:]
191 |     return cabecalho, dados
192 |
193 |
194 | def parse_xlsx(caminho: str, sheet_name: Optional[str] = None) -> tuple[list[str], list[list[str]]]:
195 |     from openpyxl import load_workbook
196 |     wb = load_workbook(caminho, read_only=True, data_only=True)
197 |     if sheet_name:
198 |         ws = wb[sheet_name]
199 |     else:
200 |         ws = wb.active
201 |     linhas = list(ws.iter_rows(values_only=True))
202 |     if not linhas:
203 |         return [], []
204 |     cabecalho = [str(c or "") for c in linhas[0]]
205 |     dados = []
206 |     for row in linhas[1:]:
207 |         dados.append([str(c or "") for c in row])
208 |     return cabecalho, dados
209 |
210 |
211 | def detectar_mapeamento(cabecalho: list[str], entity: str) -> dict[int, Optional[str]]:
212 |     mapa: dict[int, Optional[str]] = {}
213 |     colunas = COLUNAS_PADRAO.get(entity, {})
214 |     chaves_ordenadas = sorted(colunas.keys(), key=len, reverse=True)
215 |     usados: set[str] = set()
216 |     for i, nome in enumerate(cabecalho):
217 |         chave = _limpar_cabecalho(nome)
218 |         campo = colunas.get(chave)
219 |         if campo:
220 |             mapa[i] = campo
221 |             usados.add(campo)
222 |         else:
223 |             melhor_campo: Optional[str] = None
224 |             for k in chaves_ordenadas:
225 |                 if k in chave and colunas[k] not in usados:
226 |                     melhor_campo = colunas[k]
227 |                     break
228 |             mapa[i] = melhor_campo
229 |             if melhor_campo:
230 |                 usados.add(melhor_campo)
231 |     return mapa
232 |
233 |
234 | def mapeamento_para_lista(mapa: dict[int, Optional[str]], entity: str) -> list[tuple[int, str, Optional[str]]]:

```



```

235 |     result = []
236 |     for idx in sorted(mapa.keys()):
237 |         result.append((idx, f"Coluna {idx + 1}", mapa.get(idx)))
238 |     return result
239 |
240 |
241 | class Importador:
242 |     def __init__(self, entity: str, service, session):
243 |         self.entity = entity
244 |         self.service = service
245 |         self.session = session
246 |
247 |     def importar_linhas(self, dados: list[list[str]], mapa_colunas: list[Optional[str]],
248 |                        cabecalho: list[str]) -> ImportResult:
249 |         result = ImportResult()
250 |         result.total = len(dados)
251 |         obrigatorios = CAMPOS_OBRIGATORIOS.get(self.entity, [])
252 |         editaveis = CAMPOS_EDITAVEIS.get(self.entity, [])
253 |
254 |         for idx_linha, linha in enumerate(dados):
255 |             num_linha = idx_linha + 2
256 |             dados_linha: dict[str, Any] = {}
257 |             pulada = False
258 |             for col_idx, campo in enumerate(mapa_colunas):
259 |                 if campo and col_idx < len(linha):
260 |                     valor = _valor_convertido(linha[col_idx], campo)
261 |                     if valor is not None:
262 |                         dados_linha[campo] = valor
263 |
264 |             for req in obrigatorios:
265 |                 if req not in dados_linha or dados_linha[req] in (None, ""):
266 |                     result.erros.append((num_linha, f"Campo obrigatório '{req}' vazio"))
267 |                     pulada = True
268 |                     break
269 |             if pulada:
270 |                 continue
271 |
272 |             # Remove campos não editáveis
273 |             dados_filtrados = {k: v for k, v in dados_linha.items() if k in editaveis}
274 |
275 |             # Sanitiza strings
276 |             for k, v in list(dados_filtrados.items()):
277 |                 if isinstance(v, str):
278 |                     dados_filtrados[k] = sanitizar(v, self.entity.capitalize(), k)
279 |
280 |             try:
281 |                 self._importar_um(dados_filtrados)
282 |                 result.importados += 1
283 |             except Exception as e:
284 |                 result.erros.append((num_linha, str(e)))
285 |
286 |         return result
287 |
288 |     def _importar_um(self, dados: dict[str, Any]):
289 |         if self.entity == "printers":
290 |             patrimonio = dados.get("patrimonio")
291 |             if not patrimonio:
292 |                 raise ValueError("patrimonio é obrigatório")
293 |             existente = self.service.buscar_por_patrimonio(patrimonio)
294 |             if existente:
295 |                 self.service.atualizar(existente, **dados)
296 |             else:
297 |                 self.service.criar(**dados)

```

```

298 |         elif self.entity == "parts":
299 |             nome = dados.get("nome", "")
300 |             descricao = dados.get("descricao", "")
301 |             if not nome and descricao:
302 |                 dados["nome"] = descricao
303 |                 nome = descricao
304 |             if not nome:
305 |                 raise ValueError("nome é obrigatório")
306 |             codigo = dados.pop("codigo", None) or ""
307 |             modelo_raw = dados.pop("modelo_compativel", "") or ""
308 |             modelo_norm = normalizar_modelos(modelo_raw)
309 |
310 |             # Dedup: se veio código na planilha, usa como chave; senão, cada linha é um novo registro
311 |             if codigo:
312 |                 existente = self.service.buscar_por_codigo(codigo)
313 |                 if existente:
314 |                     self.service.atualizar(existente, **dados)
315 |                     return
316 |             # Fallback: se já existe peça com mesmo nome + mesmo modelo, atualiza (evita duplicatas)
317 |             if modelo_norm:
318 |                 existente = self.service.buscar_por_nome(nome)
319 |                 if existente and existente.modelo_compativel:
320 |                     if modelo_norm.upper() == existente.modelo_compativel.upper():
321 |                         self.service.atualizar(existente, **dados)
322 |                         return
323 |             codigo_final = codigo or self.service.gerar_codigo()
324 |             self.service.criar(
325 |                 codigo=codigo_final,
326 |                 nome=dados.pop("nome", nome),
327 |                 descricao=dados.pop("descricao", ""),
328 |                 modelo_compativel=modelo_norm,
329 |                 quantidade=dados.pop("quantidade_estoque", 0),
330 |                 estoque_minimo=dados.pop("estoque_minimo", 1),
331 |                 marca=dados.pop("marca", ""),
332 |                 categoria=dados.pop("categoria", ""),
333 |                 fornecedor=dados.pop("fornecedor", ""),
334 |             )
335 |         elif self.entity == "companies":
336 |             nome = dados.get("nome", "")
337 |             if not nome:
338 |                 raise ValueError("nome é obrigatório")
339 |             existente = self.service.buscar_por_nome(nome)
340 |             if existente:
341 |                 self.service.atualizar(existente, **dados)
342 |             else:
343 |                 self.service.criar(
344 |                     nome=nome,
345 |                     cnpj=dados.pop("cnpj", ""),
346 |                     telefone=dados.pop("telefone", ""),
347 |                     email=dados.pop("email", ""),
348 |                     tipo=dados.pop("tipo", "Cliente"),
349 |                 )
350 |             else:
351 |                 raise ValueError(f"Entidade desconhecida: {self.entity}")
352 |
353 |     def preview_data(self, cabecalho: list[str], dados: list[list[str]],
354 |                      mapa_colunas: list[Optional[str]]) -> tuple[list[str], list[list[str]]]:
355 |         colunas_exibidas = []
356 |         indices_exibidos = []
357 |         for i, campo in enumerate(mapa_colunas):
358 |             if campo:
359 |                 colunas_exibidas.append(campo)
360 |                 indices_exibidos.append(i)

```

```

361 |
362 |         if not colunas_exibidas:
363 |             return [], []
364 |
365 |         rows = []
366 |         for linha in dados[:20]:
367 |             row = []
368 |             for idx in indices_exibidos:
369 |                 val = linha[idx] if idx < len(linha) else ""
370 |                 row.append(val)
371 |             rows.append(row)
372 |
373 |         return colunas_exibidas, rows

```

=== ARQUIVO: .lapp\utils\logger.py ===

```

001 | import logging
002 | import logging.handlers
003 | import sys
004 | from pathlib import Path
005 |
006 | from config import DATA_DIR
007 |
008 | LOG_DIR = DATA_DIR / "logs"
009 | LOG_FILE = LOG_DIR / "app.log"
010 |
011 |
012 | def setup_logging() -> None:
013 |     LOG_DIR.mkdir(parents=True, exist_ok=True)
014 |     fmt = logging.Formatter(
015 |         "%(asctime)s [%(levelname)s] %(name)s: %(message)s",
016 |         datefmt="%Y-%m-%d %H:%M:%S",
017 |     )
018 |
019 |     file_handler = logging.handlers.RotatingFileHandler(
020 |         LOG_FILE, maxBytes=5 * 1024 * 1024, backupCount=3, encoding="utf-8"
021 |     )
022 |     file_handler.setLevel(logging.DEBUG)
023 |     file_handler.setFormatter(fmt)
024 |
025 |     console_handler = logging.StreamHandler(sys.stdout)
026 |     console_handler.setLevel(logging.INFO)
027 |     console_handler.setFormatter(fmt)
028 |
029 |     root = logging.getLogger()
030 |     root.setLevel(logging.DEBUG)
031 |     root.addHandler(file_handler)
032 |     root.addHandler(console_handler)

```

=== ARQUIVO: .lapp\utils\monitoring_config.py ===

```

001 | from __future__ import annotations
002 |
003 | import json
004 | import logging
005 | from dataclasses import dataclass
006 | from pathlib import Path
007 |
008 | from config import DATA_DIR

```

```

009 |
010 | log = logging.getLogger(__name__)
011 |
012 | MONITORING_CONFIG_PATH: Path = DATA_DIR / "monitoring_config.json"
013 |
014 |
015 | @dataclass
016 | class MonitoringConfig:
017 |     enabled: bool = False
018 |     interval_seconds: int = 300
019 |     ping_timeout_ms: int = 2000
020 |     ping_retries: int = 2
021 |     snmp_community: str = "public"
022 |     snmp_port: int = 161
023 |     snmp_timeout_s: int = 3
024 |     offline_threshold: int = 3
025 |     toner_warn_percent: int = 15
026 |     auto_alert_offline: bool = True
027 |     auto_alert_toner: bool = True
028 |
029 | @classmethod
030 | def carregar(cls) -> MonitoringConfig:
031 |     if MONITORING_CONFIG_PATH.exists():
032 |         try:
033 |             dados = json.loads(MONITORING_CONFIG_PATH.read_text("utf-8"))
034 |             return cls(**dados)
035 |         except Exception as e:
036 |             log.warning("Erro ao carregar config de monitoramento: %s", e)
037 |     return cls()
038 |
039 | def salvar(self) -> None:
040 |     MONITORING_CONFIG_PATH.write_text(
041 |         json.dumps(self._asdict(), indent=2, ensure_ascii=False),
042 |         encoding="utf-8",
043 |     )
044 |
045 | def _asdict(self) -> dict:
046 |     return {
047 |         "enabled": self.enabled,
048 |         "interval_seconds": self.interval_seconds,
049 |         "ping_timeout_ms": self.ping_timeout_ms,
050 |         "ping_retries": self.ping_retries,
051 |         "snmp_community": self.snmp_community,
052 |         "snmp_port": self.snmp_port,
053 |         "snmp_timeout_s": self.snmp_timeout_s,
054 |         "offline_threshold": self.offline_threshold,
055 |         "toner_warn_percent": self.toner_warn_percent,
056 |         "auto_alert_offline": self.auto_alert_offline,
057 |         "auto_alert_toner": self.auto_alert_toner,
058 |     }

```

=== ARQUIVO: .\apputils\network.py ===

```

001 | from __future__ import annotations
002 |
003 | import logging
004 | import subprocess
005 | import sys
006 | import time
007 | from typing import Optional
008 |

```

```

009 | log = logging.getLogger(__name__)
010 |
011 |
012 | def ping(host: str, timeout_ms: int = 2000, retries: int = 1) -> Optional[float]:
013 |     if not host or host == "0.0.0.0":
014 |         return None
015 |     for _ in range(retries):
016 |         try:
017 |             if sys.platform == "win32":
018 |                 args = ["ping", "-n", "1", "-w", str(timeout_ms), host]
019 |             else:
020 |                 args = ["ping", "-c", "1", "-W", str(timeout_ms // 1000), host]
021 |             start = time.monotonic()
022 |             proc = subprocess.run(
023 |                 args,
024 |                 capture_output=True,
025 |                 text=True,
026 |                 timeout=max(timeout_ms // 1000 + 1, 3),
027 |             )
028 |             elapsed = (time.monotonic() - start) * 1000
029 |             if proc.returncode == 0:
030 |                 return round(elapsed, 1)
031 |         except (subprocess.TimeoutExpired, FileNotFoundError, PermissionError) as e:
032 |             log.debug("ping %s falhou: %s", host, e)
033 |     return None
034 |
035 |
036 | def snmp_get(host: str, oid: str, community: str = "public", port: int = 161, timeout: int = 3) -> Optional[
037 |     try:
038 |         from pysnmp.hlapi import CommunityData, ContextData, ObjectIdentity, ObjectType, SnmpEngine, UdpTra
039 |         error_indication, error_status, error_index, var_binds = next(
040 |             getCmd(
041 |                 SnmpEngine(),
042 |                 CommunityData(community),
043 |                 UdpTransportTarget((host, port), timeout=timeout),
044 |                 ContextData(),
045 |                 ObjectIdentity(ObjectIdentity(oid)),
046 |             )
047 |         )
048 |         if error_indication:
049 |             log.debug("SNMP error for %s/%s: %s", host, oid, error_indication)
050 |             return None
051 |         if error_status:
052 |             log.debug("SNMP error status for %s/%s: %s", host, oid, error_status)
053 |             return None
054 |         if var_binds:
055 |             return str(var_binds[0][1])
056 |     except ImportError:
057 |         log.warning("pysnmp nao instalado – instale com: pip install pysnmp")
058 |         return None
059 |     except Exception as e:
060 |         log.debug("SNMP exception for %s/%s: %s", host, oid, e)
061 |     return None
062 |
063 |
064 | def snmp_walk(host: str, oid_prefix: str, community: str = "public", port: int = 161, timeout: int = 3) ->
065 |     try:
066 |         from pysnmp.hlapi import CommunityData, ContextData, ObjectIdentity, ObjectType, SnmpEngine, UdpTra
067 |         results = []
068 |         for error_indication, error_status, error_index, var_binds in nextCmd(
069 |             SnmpEngine(),
070 |             CommunityData(community),
071 |             UdpTransportTarget((host, port), timeout=timeout),

```

```

072 |         ContextData(),
073 |         ObjectType(ObjectIdentity(oid_prefix)),
074 |         lexicographicMode=False,
075 |     ):
076 |         if error_indication:
077 |             break
078 |         if error_status:
079 |             break
080 |         for name, val in var_binds:
081 |             results.append({"oid": str(name), "value": str(val)})
082 |     return results
083 | except ImportError:
084 |     log.warning("pysnmp nao instalado")
085 |     return []
086 | except Exception as e:
087 |     log.debug("SNMP walk exception for %s/%s: %s", host, oid_prefix, e)
088 | return []
089 |
090 |
091 | def snmp_get_printer_info(host: str, community: str = "public") -> dict:
092 |     OIDS = {
093 |         "toner_level": ".1.3.6.1.2.1.43.11.1.1.9.1.1",
094 |         "toner_max": ".1.3.6.1.2.1.43.11.1.1.8.1.1",
095 |         "page_count": ".1.3.6.1.2.1.43.10.2.1.4.1.1",
096 |         "drum_life": ".1.3.6.1.2.1.43.12.1.1.4.1.1",
097 |         "drum_max": ".1.3.6.1.2.1.43.12.1.1.5.1.1",
098 |         "error_code": ".1.3.6.1.2.1.25.3.5.1.2.1",
099 |         "error_message": ".1.3.6.1.2.1.43.18.1.1.8.1.1",
100 |         "device_status": ".1.3.6.1.2.1.25.3.5.1.1.1",
101 |     }
102 |     result = {}
103 |     for key, oid in OIDS.items():
104 |         val = snmp_get(host, oid, community=community)
105 |         if val is not None:
106 |             result[key] = val
107 |     return result

```

=== ARQUIVO: .lapp\utils\sanitize.py ===

```

001 | import logging
002 | from typing import Any, Optional
003 |
004 | log = logging.getLogger(__name__)
005 |
006 | COLUMN_LENGTHS: dict[str, dict[str, int]] = {
007 |     "Printer": {"patrimonio": 80, "modelo": 80, "serial": 80, "status": 30, "local_atual": 120, "marca": 80},
008 |     "Activity": {"kind": 30, "numero_recibo": 50, "status_atividade": 30, "from_location": 120, "to_location": 120},
009 |     "Company": {"nome": 150, "cnpj": 18, "endereco": 255, "cidade": 100, "uf": 2, "telefone": 20, "email": 100},
010 |     "User": {"nome": 120, "email": 120, "username": 50, "perfil": 20},
011 |     "Technician": {"nome_completo": 150, "nome_exibicao": 80, "telefone": 20, "email": 120},
012 |     "Part": {"codigo": 50, "nome": 150, "modelo_compativel": 200},
013 |     "Alert": {"tipo": 20, "titulo": 200},
014 |     "PrinterLocation": {"local": 120, "observacao": 255},
015 |     "Transfer": {"numero_os": 50, "tipo": 20, "responsavel_entrega": 120, "responsavel_recebimento": 120},
016 |     "Attachment": {"entity_type": 50, "filename": 255, "original_name": 255, "mime_type": 100, "categoria": 100},
017 | }
018 |
019 |
020 | def truncar(valor: str, max_len: int) -> str:
021 |     if not valor:
022 |         return ""

```

```

023 |     return valor.strip()[:max_len]
024 |
025 |
026 | def sanitizar(valor: Any, model_class: Optional[str] = None, field: Optional[str] = None, max_len: Optional[int] = None):
027 |     if valor is None:
028 |         return None
029 |     if not isinstance(valor, str):
030 |         return valor
031 |     original = valor
032 |     valor = valor.strip()
033 |     if model_class and field:
034 |         col_len = COLUMN_LENGTHS.get(model_class, {}).get(field)
035 |         if col_len and len(valor) > col_len:
036 |             log.warning("Truncando %s.%s de %d para %d caracteres", model_class, field, len(valor), col_len)
037 |             valor = valor[:col_len]
038 |     if max_len and len(valor) > max_len:
039 |         log.warning("Truncando campo de %d para %d caracteres (max_len=%d)", len(original), max_len, max_len)
040 |         valor = valor[:max_len]
041 |     return valor

```

=== ARQUIVO: .\app\utils\security.py ===

```

001 | import bcrypt
002 |
003 |
004 | def hash_password(password: str) -> str:
005 |     return bcrypt.hashpw(password.encode("utf-8")[:72], bcrypt.gensalt()).decode("utf-8")
006 |
007 |
008 | def verify_password(plain_password: str, hashed_password: str) -> bool:
009 |     return bcrypt.checkpw(plain_password.encode("utf-8")[:72], hashed_password.encode("utf-8"))
010 |

```

=== ARQUIVO: .\app\utils\ui_helpers.py ===

```

001 | from __future__ import annotations
002 |
003 | import logging
004 | from collections.abc import Callable, Iterator
005 | from contextlib import contextmanager
006 | from typing import Any
007 |
008 | from PySide6.QtCore import QObject, QRunnable, QThreadPool, Qt, Signal
009 | from PySide6.QtWidgets import QPushButton
010 | from sqlalchemy.exc import SQLAlchemyError
011 |
012 | from app.views.styles.theme import (
013 |     ESTILO_BOTAO_AVISO,
014 |     ESTILO_BOTAO_ERRO,
015 |     ESTILO_BOTAO_FECHAR,
016 |     ESTILO_BOTAO_PRIMARIO,
017 |     ESTILO_BOTAO_SECUNDARIO,
018 |     ESTILO_BOTAO_SUCESSO,
019 | )
020 | from app.views.widgets.toast import ToastManager
021 |
022 | log = logging.getLogger(__name__)
023 |
024 |

```

```

025 | def criar_botao(texto: str, estilo: str = ESTILO_BOTAO_PRIMARIO, callback: Callable[[], object] | None = No
026 |     btn = QPushButton(texto)
027 |     btn.setCursor(Qt.PointingHandCursor)
028 |     btn.setStyleSheet(estilo)
029 |     if callback:
030 |         btn.clicked.connect(callback)
031 |     return btn
032 |
033 |
034 | def criar_botao_sucesso(texto: str, callback: Callable[[], object] | None = None) -> QPushButton:
035 |     return criar_botao(texto, ESTILO_BOTAO_SUCESSO, callback)
036 |
037 |
038 | def criar_botao_erro(texto: str, callback: Callable[[], object] | None = None) -> QPushButton:
039 |     return criar_botao(texto, ESTILO_BOTAO_ERRO, callback)
040 |
041 |
042 | def criar_botao_fechar(texto: str = "Cancelar", callback: Callable[[], object] | None = None) -> QPushButton:
043 |     return criar_botao(texto, ESTILO_BOTAO_FECHAR, callback)
044 |
045 |
046 | def criar_botao_aviso(texto: str, callback: Callable[[], object] | None = None) -> QPushButton:
047 |     return criar_botao(texto, ESTILO_BOTAO_AVISO, callback)
048 |
049 |
050 | def criar_botao_secundario(texto: str, callback: Callable[[], object] | None = None) -> QPushButton:
051 |     return criar_botao(texto, ESTILO_BOTAO_SECUNDARIO, callback)
052 |
053 |
054 | @contextmanager
055 | def tratar_erro(operacao: str) -> Iterator[None]:
056 |     try:
057 |         yield
058 |     except SQLAlchemyError:
059 |         log.exception("Erro em %s", operacao)
060 |         ToastManager.erro(f"Erro ao {operacao}. Verifique os dados e tente novamente.")
061 |     except Exception:
062 |         log.exception("Erro inesperado em %s", operacao)
063 |         ToastManager.erro(f"Erro inesperado ao {operacao}.")
064 |
065 |
066 | class _WorkerSignals(QObject):
067 |     finished = Signal(object)
068 |     error = Signal(str)
069 |
070 |
071 | class ExportWorker(QRunnable):
072 |     def __init__(self, fn: Callable[..., Any], *args: Any) -> None:
073 |         super().__init__()
074 |         self.fn: Callable[..., Any] = fn
075 |         self.args: tuple[Any, ...] = args
076 |         self.signals: _WorkerSignals = _WorkerSignals()
077 |
078 |     def run(self) -> None:
079 |         try:
080 |             self.fn(*self.args)
081 |             self.signals.finished.emit(self.args[-1])
082 |         except Exception as e:
083 |             self.signals.error.emit(str(e))
084 |
085 |
086 | def exportar_em_thread(fn: Callable[..., Any], *args: Any, on_finish: Callable[[object], object] | None = N
087 |     worker = ExportWorker(fn, *args)

```



```

088 |     if on_finish:
089 |         worker.signals.finished.connect(on_finish)
090 |     if on_error:
091 |         worker.signals.error.connect(on_error)
092 |     else:
093 |         def _erro_padrao(msg: str) -> None:
094 |             log.exception("Erro na exportação: %s", msg)
095 |             worker.signals.error.connect(_erro_padrao)
096 |     QThreadPool.globalInstance().start(worker)

```

=== ARQUIVO: .\apptools\validacao.py ===

```

001 | import re
002 | from collections.abc import Callable
003 | from typing import Optional
004 |
005 | from PySide6.QtWidgets import QLabel, QLineEdit
006 |
007 | from app.views.styles.theme import ESTILO_INPUT
008 |
009 |
010 | _PADRAO_EMAIL = re.compile(r"^[^@\s]+@[^\s]+\.[^@\s]+$")
011 |
012 | Regra = Callable[[str], Optional[str]]
013 |
014 |
015 | class ValidadorCampo:
016 |     def __init__(self, line_edit: QLineEdit, regra: Regra, label_erro: Optional[QLabel] = None) -> None:
017 |         self._line: QLineEdit = line_edit
018 |         self._regra: Regra = regra
019 |         self._label_erro: Optional[QLabel] = label_erro
020 |         self._base: str = ESTILO_INPUT
021 |         line_edit.textChanged.connect(lambda: self._validar())
022 |         self._validar()
023 |
024 |     def _validar(self) -> None:
025 |         texto: str = self._line.text()
026 |         erro: Optional[str] = self._regra(texto)
027 |         if erro:
028 |             self._line.setStyleSheet(self._base + " border-color: #ef4444;")
029 |             if self._label_erro:
030 |                 self._label_erro.setText(erro)
031 |                 self._label_erro.show()
032 |         elif texto:
033 |             self._line.setStyleSheet(self._base + " border-color: #10b981;")
034 |             if self._label_erro:
035 |                 self._label_erro.hide()
036 |         else:
037 |             self._line.setStyleSheet(self._base)
038 |             if self._label_erro:
039 |                 self._label_erro.hide()
040 |
041 |     @property
042 |     def valido(self) -> bool:
043 |         return self._regra(self._line.text()) is None
044 |
045 |
046 | def obrigatorio(texto: str) -> Optional[str]:
047 |     if not texto.strip():
048 |         return "Campo obrigatório"
049 |     return None

```

```

050 |
051 |
052 | def email(texto: str) -> Optional[str]:
053 |     if not texto.strip():
054 |         return "Campo obrigatório"
055 |     if not _PADRAO_EMAIL.match(texto.strip()):
056 |         return "E-mail inválido"
057 |     return None
058 |
059 |
060 | def email_opcional(texto: str) -> Optional[str]:
061 |     if not texto.strip():
062 |         return None
063 |     if not _PADRAO_EMAIL.match(texto.strip()):
064 |         return "E-mail inválido"
065 |     return None
066 |
067 |
068 | def minimo(n: int) -> Regra:
069 |     def _regra(texto: str) -> Optional[str]:
070 |         if texto.strip() and len(texto.strip()) < n:
071 |             return f"Mínimo de {n} caracteres"
072 |         return None
073 |     return _regra
074 |
075 |
076 | def alfanumerico(texto: str) -> Optional[str]:
077 |     if not texto.strip():
078 |         return "Campo obrigatório"
079 |     if not texto.strip().isalnum():
080 |         return "Apenas letras e números"
081 |     return None
082 |

```

=== ARQUIVO: .\apptutils__init__.py ===

```

001 | from .helpers import formatar_data, formatar_data_hora, limpar_local
002 | from .sanitize import sanitizar, truncar
003 | from .security import hash_password, verify_password

```

=== ARQUIVO: .\app\views\login_dialog.py ===

```

001 | from __future__ import annotations
002 |
003 | from app.models.base import utcnow
004 | from datetime import datetime as dt, timedelta
005 | from pathlib import Path
006 | from typing import Any
007 |
008 | from PySide6.QtCore import QEasingCurve, QPoint, QPropertyAnimation, Qt, QTimer
009 | from PySide6.QtGui import QKeySequence, QMouseEvent, QShortcut
010 | from PySide6.QtWidgets import QDialog, QFrame, QLabel, QLineEdit, QPushButton, QSizePolicy, QVBoxLayout
011 |
012 | from app.models import LoginHistory, User
013 | from app.utils.effects import sombra_glow
014 | from app.utils.security import verify_password
015 | from db import close_session, get_session
016 |
017 | MAX_TENTATIVAS: int = 5

```

```

018 | BLOQUEIO_MINUTOS: int = 5
019 | _attempts: dict[str, list[dt]] = {}
020 |
021 |
022 | def _limpar_attempts_velhos() -> None:
023 |     agora: dt = utcnow()
024 |     limite: dt = agora - timedelta(minutes=BLOQUEIO_MINUTOS)
025 |     for usuario in list(_attempts.keys()):
026 |         _attempts[usuario] = [t for t in _attempts[usuario] if t > limite]
027 |         if not _attempts[usuario]:
028 |             del _attempts[usuario]
029 |
030 |
031 | def _esta_bloqueado(username: str) -> bool:
032 |     _limpar_attempts_velhos()
033 |     return len(_attempts.get(username, [])) >= MAX_TENTATIVAS
034 |
035 |
036 | def _registrar_tentativa(username: str) -> None:
037 |     _attempts.setdefault(username, []).append(utcnow())
038 |
039 |
040 | def _resetar_tentativas(username: str) -> None:
041 |     _attempts.pop(username, None)
042 |
043 |
044 | class LoginDialog(QDialog):
045 |     authenticated_user: dict[str, Any] | None
046 |     drag_pos: QPoint | None
047 |     animation: QPropertyAnimation
048 |     user_input: QLineEdit
049 |     senha_input: QLineEdit
050 |     error_label: QLabel
051 |
052 |     def __init__(self) -> None:
053 |         super().__init__()
054 |         self.authenticated_user = None
055 |         self.drag_pos = None
056 |         self.init_ui()
057 |
058 |     def mousePressEvent(self, event: QMouseEvent):
059 |         if event.button() == Qt.LeftButton:
060 |             self.drag_pos = event.globalPosition().toPoint() - self.frameGeometry().topLeft()
061 |
062 |     def mouseMoveEvent(self, event: QMouseEvent):
063 |         if self.drag_pos is not None and event.buttons() == Qt.LeftButton:
064 |             self.move(event.globalPosition().toPoint() - self.drag_pos)
065 |
066 |     def mouseReleaseEvent(self, event: QMouseEvent):
067 |         self.drag_pos = None
068 |
069 |     def init_ui(self) -> None:
070 |         self.setWindowTitle("Controle de Impressoras Pro")
071 |         self.setFixedSize(400, 500)
072 |         self.setWindowFlags(Qt.FramelessWindowHint)
073 |         self.setAttribute(Qt.WA_TranslucentBackground)
074 |
075 |         self.setObjectName("loginDialog")
076 |         self.setStyleSheet("""
077 |             #loginDialog {
078 |                 background: qlineargradient(x1:0, y1:0, x2:1, y2:1,
079 |                     stop:0 #0a0a0f, stop:1 #14141f);
080 |                 border-radius: 20px;

```

```

081 |     }
082 |     """)
083 |
084 |     main_layout = QVBoxLayout(self)
085 |     main_layout.setContentsMargins(16, 16, 16, 16)
086 |     main_layout.setSpacing(0)
087 |
088 |     # CARD INTERNO
089 |     card = QFrame()
090 |     card.setStyleSheet("""
091 |         #loginCard {
092 |             background: qlineargradient(x1:0, y1:0, x2:1, y2:1,
093 |                 stop:0 #14141f, stop:1 #1a1a28);
094 |             border-radius: 16px;
095 |             border: 1px solid #2a2a3e;
096 |         }
097 |     """)
098 |     card.setObjectName("loginCard")
099 |     card.setSizePolicy(QSizePolicy.Expanding, QSizePolicy.Expanding)
100 |
101 |     card_layout = QVBoxLayout(card)
102 |     card_layout.setContentsMargins(28, 26, 28, 22)
103 |     card_layout.setSpacing(7)
104 |     card_layout.setAlignment(Qt.AlignCenter)
105 |
106 |     # Logo
107 |     from PySide6.QtGui import QPixmap
108 |     from config import BUNDLE_DIR
109 |     logo_path = BUNDLE_DIR / "logo.png"
110 |     if not logo_path.exists():
111 |         logo_path = Path(__file__).parent.parent.parent / "1.PNG"
112 |     if not logo_path.exists():
113 |         logo_path = Path(__file__).parent.parent.parent / "1.png"
114 |     if logo_path.exists():
115 |         logo_lbl = QLabel()
116 |         pix = QPixmap(str(logo_path)).scaled(64, 64, Qt.KeepAspectRatio, Qt.SmoothTransformation)
117 |         logo_lbl.setPixmap(pix)
118 |         logo_lbl.setAlignment(Qt.AlignCenter)
119 |         logo_lbl.setStyleSheet("background: transparent; border: none;")
120 |         card_layout.addWidget(logo_lbl)
121 |     else:
122 |         icon = QLabel("■■")
123 |         icon.setAlignment(Qt.AlignCenter)
124 |         icon.setStyleSheet("font-size: 44px; background: transparent; border: none;")
125 |         card_layout.addWidget(icon)
126 |
127 |     # TÍTULO
128 |     title = QLabel("Impressoras\nBrasil Toner")
129 |     title.setAlignment(Qt.AlignCenter)
130 |     title.setStyleSheet(
131 |         "color: #e8e8f0; font-size: 20px; font-weight: 700;"
132 |         " background: transparent; letter-spacing: -0.5px; line-height: 1.3;"
133 |     )
134 |     card_layout.addWidget(title)
135 |
136 |     sub = QLabel("Faça login para continuar")
137 |     sub.setAlignment(Qt.AlignCenter)
138 |     sub.setStyleSheet("color: #94949f; font-size: 11px; background: transparent; margin-top: -4px;")
139 |     card_layout.addWidget(sub)
140 |
141 |     card_layout.addSpacing(18)
142 |
143 |     # USUÁRIO

```

```

144 |         lbl_user = QLabel("Usuário")
145 |         lbl_user.setStyleSheet("color: #717182; font-size: 10px; font-weight: 600; background: transparent;
146 |         card_layout.addWidget(lbl_user)
147 |
148 |         self.user_input = QLineEdit()
149 |         self.user_input.setPlaceholderText("Digite seu usuário")
150 |         self.user_input.setMinimumHeight(44)
151 |         self.user_input.setStyleSheet(self._input_style())
152 |         card_layout.addWidget(self.user_input)
153 |
154 |         card_layout.addSpacing(6)
155 |
156 |         # SENHA
157 |         lbl_senha = QLabel("Senha")
158 |         lbl_senha.setStyleSheet("color: #717182; font-size: 10px; font-weight: 600; background: transparent;
159 |         card_layout.addWidget(lbl_senha)
160 |
161 |         self.senha_input = QLineEdit()
162 |         self.senha_input.setPlaceholderText("Digite sua senha")
163 |         self.senha_input.setEchoMode(QLineEdit.Password)
164 |         self.senha_input.setMinimumHeight(44)
165 |         self.senha_input.setStyleSheet(self._input_style())
166 |         self.senha_input.setFocus()
167 |         card_layout.addWidget(self.senha_input)
168 |
169 |         # ERRO
170 |         self.error_label = QLabel("")
171 |         self.error_label.setAlignment(Qt.AlignCenter)
172 |         self.error_label.setStyleSheet("color: #ef4444; font-size: 11px; background: transparent; min-height: 20px;
173 |         card_layout.addWidget(self.error_label)
174 |
175 |         # BOTÃO ENTRAR
176 |         btn = QPushButton("ENTRAR")
177 |         btn.setMinimumHeight(44)
178 |         btn.setCursor(Qt.PointingHandCursor)
179 |         btn.setStyleSheet("""
180 |             QPushButton {
181 |                 background: #6366f1;
182 |                 color: #ffffff;
183 |                 border: none;
184 |                 border-radius: 10px;
185 |                 font-size: 13px;
186 |                 font-weight: 700;
187 |                 letter-spacing: 1px;
188 |                 margin-top: 4px;
189 |             }
190 |             QPushButton:hover { background: #4f46e5; }
191 |             QPushButton:pressed { background: #4338ca; }
192 |         """)
193 |         btn.clicked.connect(self.fazer_login)
194 |         QShortcut(QKeySequence("Return"), self).activated.connect(self.fazer_login)
195 |         QShortcut(QKeySequence("Enter"), self).activated.connect(self.fazer_login)
196 |         card_layout.addWidget(btn)
197 |
198 |         # FECHAR
199 |         close_btn = QPushButton("Sair")
200 |         close_btn.setCursor(Qt.PointingHandCursor)
201 |         close_btn.setStyleSheet("""
202 |             QPushButton {
203 |                 background: transparent;
204 |                 color: #717182;
205 |                 border: none;
206 |                 font-size: 12px;

```

```

207 |         padding: 4px;
208 |         font-weight: 500;
209 |     }
210 |     QPushButton:hover { color: #ef4444; }
211 | """
212 | close_btn.clicked.connect(self.reject)
213 | card_layout.addWidget(close_btn)
214 |
215 | # VERSÃO
216 | from app import __version__
217 | ver = QLabel(f"v{__version__} - Desenvolvido por Otávio Oliveira")
218 | ver.setAlignment(Qt.AlignCenter)
219 | ver.setStyleSheet("color: #3a3a50; font-size: 9px; background: transparent;")
220 | card_layout.addWidget(ver)
221 |
222 | main_layout.addWidget(card)
223 |
224 | sombra_glow(card, cor="#6366f1", blur=60)
225 | self.animar_entrada()
226 |
227 | def _input_style(self) -> str:
228 |     return """
229 |         QLineEdit {
230 |             background-color: #1e1e2e;
231 |             color: #e8e8f0;
232 |             border: 1px solid #2a2a3e;
233 |             border-radius: 10px;
234 |             padding: 10px 14px;
235 |             font-size: 13px;
236 |         }
237 |         QLineEdit:hover { border-color: #3a3a50; }
238 |         QLineEdit:focus { border-color: #6366f1; background-color: #1e1e2e; }
239 |     """
240 |
241 | def animar_entrada(self) -> None:
242 |     self.animation = QPropertyAnimation(self, b"pos")
243 |     self.animation.setDuration(400)
244 |     self.animation.setEasingCurve(QEasingCurve.OutBack)
245 |
246 |     screen = self.screen().geometry()
247 |     final_x: int = (screen.width() - self.width()) // 2
248 |     final_y: int = (screen.height() - self.height()) // 2
249 |
250 |     self.animation.setStartValue(QPoint(final_x, final_y - 60))
251 |     self.animation.setEndValue(QPoint(final_x, final_y))
252 |     self.animation.start()
253 |
254 | def fazer_login(self) -> None:
255 |     username: str = self.user_input.text().strip()
256 |     senha: str = self.senha_input.text()
257 |
258 |     if not username or not senha:
259 |         self.mostrar_erro("Preencha todos os campos!")
260 |         return
261 |
262 |     if _esta_bloqueado(username):
263 |         self.mostrar_erro(f"Muitas tentativas! Aguarde {BLOQUEIO_MINUTOS} min.")
264 |         return
265 |
266 |     session = get_session()
267 |     try:
268 |         user: User | None = session.query(User).filter(User.username == username).first()
269 |

```

```

270 |         if not user:
271 |             _registrar_tentativa(username)
272 |             self.mostrar_erro("Usuário não encontrado!")
273 |             return
274 |
275 |         if not user.ativo:
276 |             _registrar_tentativa(username)
277 |             self.mostrar_erro("Usuário desativado!")
278 |             return
279 |
280 |         if not verify_password(senha, user.senha_hash):
281 |             _registrar_tentativa(username)
282 |             restam: int = MAX_TENTATIVAS - len(_attempts.get(username, []))
283 |             self.mostrar_erro(f"Senha incorreta! ({restam} tentativa(s) restante(s))")
284 |             return
285 |
286 |         _resetar_tentativas(username)
287 |         user.ultimo_login = utcnow()
288 |         login_history = LoginHistory(user_id=user.id, login_at=utcnow())
289 |         session.add(login_history)
290 |         session.commit()
291 |
292 |         self.authenticated_user = {
293 |             "id": user.id,
294 |             "nome": user.nome,
295 |             "username": user.username,
296 |             "perfil": user.perfil,
297 |             "login_history_id": login_history.id,
298 |         }
299 |
300 |         self.accept()
301 |
302 |     except Exception as e:
303 |         session.rollback()
304 |         self.mostrar_erro(f"Erro: {str(e)}")
305 |     finally:
306 |         close_session(session)
307 |
308 | def mostrar_erro(self, mensagem: str) -> None:
309 |     self.error_label.setText(mensagem)
310 |     QTimer.singleShot(3000, lambda: self.error_label.setText(""))

```

=== ARQUIVO: .lapp\views\main_window.py ===

```

001 | from __future__ import annotations
002 |
003 | import logging
004 | from typing import Any
005 |
006 | from PySide6.QtCore import Qt, QTimer
007 | from PySide6.QtGui import QIcon, QPixmap
008 | from PySide6.QtWidgets import (
009 |     QFrame,
010 |     QHBoxLayout,
011 |     QLabel,
012 |     QMainWindow,
013 |     QMessageBox,
014 |     QPushButton,
015 |     QScrollArea,
016 |     QStackedWidget,
017 |     QSystemTrayIcon,

```

```

018 |     QVBoxLayout,
019 |     QWidget,
020 | )
021 | from datetime import datetime as dt
022 |
023 | from config import BASE_DIR
024 | from app.services.notification_service import NotificadorService
025 | from app.utils.ui_helpers import exportar_em_thread
026 | from app.services import (
027 |     ActivityService,
028 |     AlertService,
029 |     AuditService,
030 |     CompanyService,
031 |     DashboardService,
032 |     LoginHistoryService,
033 |     MaintenanceScheduler,
034 |     PartService,
035 |     PrinterLocationService,
036 |     PrinterService,
037 |     SnmpMonitorService,
038 |     TechnicianService,
039 |     UserService,
040 | )
041 | from app.views.widgets.toast import ToastManager
042 | from app.views.pages import (
043 |     AlertasPage,
044 |     CalendarPage,
045 |     ClientsPage,
046 |     ConfigPage,
047 |     DashboardPage,
048 |     MonitoringPage,
049 |     OSPage,
050 |     PartsPage,
051 |     PrintersPage,
052 |     PurchaseRequisitionsPage,
053 |     ReportsPage,
054 |     TechnicianAgendaPage,
055 |     TechnicianHistoryPage,
056 |     TechniciansPage,
057 |     TransfersPage,
058 | )
059 |
060 | log = logging.getLogger(__name__)
061 |
062 |
063 | class MainWindow(QMainWindow):
064 |     session: Any
065 |     user: dict[str, Any]
066 |     audit_service: AuditService
067 |     printer_service: PrinterService
068 |     activity_service: ActivityService
069 |     company_service: CompanyService
070 |     part_service: PartService
071 |     technician_service: TechnicianService
072 |     user_service: UserService
073 |     dashboard_service: DashboardService
074 |     alert_service: AlertService
075 |     maintenance_scheduler: MaintenanceScheduler
076 |     login_history_service: LoginHistoryService
077 |     snmp_monitor_service: SnmpMonitorService # type: ignore[name-defined]
078 |     printer_location_service: PrinterLocationService
079 |     tray_icon: QSystemTrayIcon
080 |     notificador: NotificadorService

```


[illegible]

[illegible]

[illegible]

```
right_layout.setSpacing(0)

# Topbar
self._adicionar_topbar(right_layout)

# ■■ Pages ████████████████████████████████████████████████████████
self.content_area = QStackedWidget()
self.content_area.setStyleSheet("background-color: #0a0a0f; border: none;")

self.pagina_dashboard = DashboardPage(
    self.session, self.printer_service,
    self.activity_service, self.dashboard_service
)
self.pagina_impressoras = PrintersPage(
    self.session, self.printer_service,
    self.company_service, self.technician_service,
    self.activity_service, self.part_service,
    self.printer_location_service,
    scheduler=self.maintenance_scheduler,
    monitor_service=self.snmp_monitor_service,
)
self.pagina_os = OSPage(
    self.session, self.printer_service,
    self.activity_service, self.company_service,
    self.technician_service,
    alert_service=self.alert_service,
)
self.pagina_clientes = ClientsPage(
    self.session, self.company_service, self.printer_service
)
self.pagina_pecas = PartsPage(
    self.session, self.part_service, self.printer_service, self.activity_service
)
self.pagina_transferencias = TransfersPage(
    self.session, self.printer_service,
    self.activity_service, self.company_service,
)
self.pagina_tecnicos = TechniciansPage(
    self.session, self.technician_service, printer_service=self.printer_service
)
self.pagina_historico = TechnicianHistoryPage(
    self.session, self.technician_service,
    self.activity_service, self.user_service,
    self.login_history_service, self.printer_service
)
self.pagina_relatorios = ReportsPage(
    self.session, self.printer_service, self.activity_service, self.company_service
)
self.pagina_alertas = AlertasPage(
    self.session, self.alert_service, self.printer_service, part_service=self.part_service
)
self.pagina_calendario = CalendarPage(
    scheduler=self.maintenance_scheduler,
    activity_service=self.activity_service,
    technician_service=self.technician_service,
    alert_service=self.alert_service,
    printer_service=self.printer_service,
)
self.pagina_agenda = TechnicianAgendaPage(
    self.session, self.technician_service,
    self.activity_service, self.printer_service,
)
self.pagina_requisicoes = PurchaseRequisitionsPage(
```

[illegible]

```

396 |         tema_frame_layout.setContentsMargins(14, 6, 14, 6)
397 |         tema_frame_layout.setSpacing(8)
398 |
399 |         self.btn_tema = QPushButton()
400 |         self.btn_tema.setCursor(Qt.PointingHandCursor)
401 |         self.btn_tema.setFixedHeight(30)
402 |         self._atualizar_texto_tema()
403 |         self.btn_tema.clicked.connect(self._alternar_tema)
404 |         tema_frame_layout.addWidget(self.btn_tema)
405 |         layout.addWidget(tema_frame)
406 |
407 |     def _atualizar_texto_tema(self) -> None:
408 |         from app.views.styles.theme_manager import TemaManager
409 |         is_dark = TemaManager.atual() == "dark"
410 |         icone = "■" if is_dark else "■■"
411 |         label = " Escuro" if is_dark else " Claro"
412 |         self.btn_tema.setText(f"{icone}{label}")
413 |         base = (
414 |             "QPushButton { background: transparent; color: #717182; border: 1px solid #2a2a3e;"
415 |             " border-radius: 6px; font-size: 11px; font-weight: 500; padding: 4px 12px;"
416 |             " text-align: left; }"
417 |             " QPushButton:hover { background: #1e1e2e; color: #e8e8f0; border-color: #6366f1; }"
418 |         )
419 |         self.btn_tema.setStyleSheet(base)
420 |
421 |     def _alternar_tema(self) -> None:
422 |         from app.views.styles.theme_manager import TemaManager
423 |         TemaManager.limpar_estilos(self)
424 |         TemaManager.alternar()
425 |         self._atualizar_texto_tema()
426 |         self._restyle_sidebar()
427 |         paginas = [self.content_area.widget(i) for i in range(self.content_area.count())]
428 |         TemaManager.reestilizar_paginas([w for w in paginas if w])
429 |
430 |     def _restyle_sidebar(self) -> None:
431 |         from app.views.styles.theme_manager import TemaManager
432 |         is_dark = TemaManager.atual() == "dark"
433 |         if is_dark:
434 |             bg = "#0f0f16"
435 |             border = "#1e1e2e"
436 |             sep = "#1e1e2e"
437 |             nav_hover = "#1e1e2e"
438 |             nav_active = "rgba(99, 102, 241, 0.12)"
439 |             nav_text = "#717182"
440 |             nav_text_hover = "#e8e8f0"
441 |             nav_text_active = "#6366f1"
442 |             user_bg = "transparent"
443 |             user_text = "#e8e8f0"
444 |             user_role = "#717182"
445 |             btn_border = "#2a2a3e"
446 |             btn_text = "#717182"
447 |         else:
448 |             bg = "ffffff"
449 |             border = "#e2e8f0"
450 |             sep = "#e2e8f0"
451 |             nav_hover = "#f1f5f9"
452 |             nav_active = "#eef2ff"
453 |             nav_text = "#64748b"
454 |             nav_text_hover = "#334155"
455 |             nav_text_active = "#7c3aed"
456 |             user_bg = "#f8fafc"
457 |             user_text = "#0f172a"
458 |             user_role = "#64748b"

```

[illegible]

```

522 |         " background: transparent;"
523 |     )
524 |     layout.addWidget(lbl)
525 |
526 | def _adicionar_logo(self, layout: QVBoxLayout) -> None:
527 |     logo_frame = QFrame()
528 |     logo_frame.setStyleSheet("background: transparent; border: none;")
529 |     logo_frame_layout = QHBoxLayout(logo_frame)
530 |     logo_frame_layout.setContentsMargins(10, 20, 10, 14)
531 |     logo_frame_layout.setSpacing(10)
532 |
533 |     logo_icon = QLabel("■")
534 |     logo_icon.setFixedSize(36, 36)
535 |     logo_icon.setAlignment(Qt.AlignCenter)
536 |     logo_icon.setStyleSheet(
537 |         "background: qlineargradient(x1:0,y1:0,x2:1,y2:1,stop:0 #4f46e5,stop:1 #7c3aed);"
538 |         " color: white; border-radius: 8px; font-size: 18px; font-weight: 700;"
539 |     )
540 |     logo_frame_layout.addWidget(logo_icon)
541 |
542 |     logo_text_frame = QVBoxLayout()
543 |     logo_text_frame.setSpacing(0)
544 |     logo_text1 = QLabel("Controle de")
545 |     logo_text1.setStyleSheet(
546 |         "color: #e8e8f0; font-size: 11px; font-weight: 700;"
547 |         " background: transparent; letter-spacing: 0.3px;"
548 |     )
549 |     logo_text2 = QLabel("Impressoras Pro")
550 |     logo_text2.setStyleSheet(
551 |         "color: #e8e8f0; font-size: 11px; font-weight: 700;"
552 |         " background: transparent; letter-spacing: 0.3px;"
553 |     )
554 |     logo_text_frame.addWidget(logo_text1)
555 |     logo_text_frame.addWidget(logo_text2)
556 |     logo_frame_layout.addLayout(logo_text_frame)
557 |     logo_frame_layout.addStretch()
558 |     layout.addWidget(logo_frame)
559 |
560 |     sep = QFrame()
561 |     sep.setFrameShape(QFrame.HLine)
562 |     sep.setStyleSheet("background-color: #1e1e2e; max-height: 1px; margin: 0 12px;")
563 |     layout.addWidget(sep)
564 |
565 | def _adicionar_card_usuario(self, layout: QVBoxLayout) -> None:
566 |     user_frame = QFrame()
567 |     user_frame.setStyleSheet(
568 |         "QFrame { background: transparent; border: none; }"
569 |     )
570 |     user_frame_layout = QVBoxLayout(user_frame)
571 |     user_frame_layout.setContentsMargins(12, 10, 12, 12)
572 |     user_frame_layout.setSpacing(8)
573 |
574 |     info_layout = QHBoxLayout()
575 |     info_layout.setSpacing(10)
576 |
577 |     avatar = QLabel(self.user["nome"][:2].upper())
578 |     avatar.setFixedSize(36, 36)
579 |     avatar.setAlignment(Qt.AlignCenter)
580 |     avatar.setStyleSheet(
581 |         "background: qlineargradient(x1:0,y1:0,x2:1,y2:1,stop:0 #8b5cf6,stop:1 #ec4899);"
582 |         " color: white; border-radius: 18px; font-size: 11px; font-weight: 700;"
583 |     )
584 |     info_layout.addWidget(avatar)

```



```

585 |
586 |     user_info = QVBoxLayout()
587 |     user_info.setSpacing(1)
588 |     user_name = QLabel(self.user["nome"])
589 |     user_name.setStyleSheet("color: #e8e8f0; font-size: 12px; font-weight: 500; background: transparent")
590 |     user_info.addWidget(user_name)
591 |     user_role = QLabel(self.user["perfil"].capitalize())
592 |     user_role.setStyleSheet("color: #717182; font-size: 10px; background: transparent; border: none;")
593 |     user_info.addWidget(user_role)
594 |     info_layout.addLayout(user_info)
595 |     info_layout.addStretch()
596 |     user_frame_layout.addLayout(info_layout)
597 |
598 |     btn_sair = QPushButton("Sair")
599 |     btn_sair.setCursor(Qt.PointingHandCursor)
600 |     btn_sair.setMinimumHeight(32)
601 |     btn_sair.setStyleSheet(
602 |         "QPushButton { background: transparent; color: #717182; border: 1px solid #2a2a3e;"
603 |         " border-radius: 6px; font-size: 11px; font-weight: 500; }"
604 |         " QPushButton:hover { background: #ef444415; color: #ef4444; border-color: #ef444433; }"
605 |     )
606 |     btn_sair.clicked.connect(self.close)
607 |     user_frame_layout.addWidget(btn_sair)
608 |
609 |     layout.addWidget(user_frame)
610 |
611 | def _criar_botao_menu(self, icone: str, texto: str, indice: int) -> QFrame:
612 |     container = QFrame()
613 |     container.setObjectName("menuContainer")
614 |     container.setStyleSheet("QFrame#menuContainer { background: transparent; border: none; }")
615 |     container_layout = QHBoxLayout(container)
616 |     container_layout.setContentsMargins(0, 0, 0, 0)
617 |     container_layout.setSpacing(0)
618 |
619 |     indicador = QFrame()
620 |     indicador.setFixedWidth(2)
621 |     indicador.setObjectName("activeIndicator")
622 |     indicador.setStyleSheet("QFrame#activeIndicator { background: transparent; border-radius: 0 2px 2px 0px; }")
623 |     indicador.setAttribute(Qt.WA_TransparentForMouseEvents)
624 |     container_layout.addWidget(indicador)
625 |
626 |     btn = QPushButton(f"{icone} {texto}")
627 |     btn.setCheckable(True)
628 |     btn.setCursor(Qt.PointingHandCursor)
629 |     btn.setMinimumHeight(36)
630 |     btn.setObjectName("menuBtn")
631 |     btn.setStyleSheet(
632 |         "QPushButton#menuBtn { background: transparent; color: #94949f; border: none;"
633 |         " text-align: left; font-size: 13px; padding: 6px 12px; border-radius: 0px; font-weight: 500; }"
634 |         " QPushButton#menuBtn:hover { background: #1e1e2e; color: #e8e8f0; }"
635 |         " QPushButton#menuBtn:checked { background: rgba(99, 102, 241, 0.2); color: #6366f1; font-weight: 500; }"
636 |     )
637 |     container_layout.addWidget(btn)
638 |     btn.clicked.connect(lambda: self._trocar_pagina(indice))
639 |
640 |     self._menu_containers.append(container)
641 |     self._menu_indicators.append(indicador)
642 |     self.menu_buttons.append(btn)
643 |     self._menu_indices.append(indice)
644 |     return container
645 |
646 | def _criar_botao_menu_com_imagem(self, caminho_icone: str, texto: str, indice: int) -> QFrame:
647 |     container = QFrame()

```

[illegible]

[illegible]

```

774 |
775 |     from PySide6.QtWidgets import QFileDialog
776 |
777 |     from app.services.relatorio_service import RelatorioService
778 |
779 |     default_name = f"{tipo}_{dt.now().strftime('%Y%m%d')}.{formato}"
780 |     filepath, _ = QFileDialog.getSaveFileName(
781 |         self, "Salvar Relatório", default_name,
782 |         f"'PDF' if formato == 'pdf' else 'Excel' (*.{formato})"
783 |     )
784 |     if not filepath:
785 |         return
786 |
787 |     try:
788 |         if tipo == "impressoras":
789 |             data = self.printer_service.listar_todos()
790 |         else:
791 |             data = self.activity_service.listar(limite=500)
792 |     except Exception as e:
793 |         QMessageBox.critical(self, "Erro", f"Erro ao buscar dados: {str(e)}")
794 |         return
795 |
796 |     export_fn = None
797 |     if tipo == "impressoras":
798 |         export_fn = RelatorioService.exportar_impressoras_pdf if formato == "pdf" else RelatorioService.
799 |     else:
800 |         export_fn = RelatorioService.exportar_atividades_pdf if formato == "pdf" else RelatorioService.
801 |
802 |     def _ao_finalizar(fp: str) -> None:
803 |         try:
804 |             os.startfile(fp)
805 |         except Exception:
806 |             log.exception("Erro ao abrir arquivo exportado")
807 |             QMessageBox.information(self, "Sucesso", f"Relatório salvo em:\n{fp}")
808 |
809 |     exportar_em_thread(export_fn, data, filepath, on_finish=_ao_finalizar)

```

=== ARQUIVO: .lapp\views\relatorio_dialog.py ===

```

001 | from __future__ import annotations
002 |
003 | import os
004 | from datetime import datetime as dt
005 | from typing import Any
006 |
007 | from PySide6.QtCore import Qt
008 | from PySide6.QtWidgets import (
009 |     QCheckBox,
010 |     QComboBox,
011 |     QCompleter,
012 |     QDialog,
013 |     QFrame,
014 |     QGroupBox,
015 |     QHBoxLayout,
016 |     QLabel,
017 |     QLineEdit,
018 |     QPushButton,
019 |     QRadioButton,
020 |     QScrollArea,
021 |     QVBoxLayout,
022 |     QWidget,

```

```

023 | )
024 |
025 | from app.utils.ui_helpers import tratar_erro
026 | from app.views.styles.theme import configurar_combo, COR, group_box, input_label, campo_rotulo, campo_reado
027 |
028 | from config import BUNDLE_DIR
029 |
030 | BG: str = COR["fundo"]
031 | CARD: str = COR["fundo_card"]
032 | BORDA: str = COR["borda"]
033 | TEXTO: str = COR["texto"]
034 | TEXTO_SEC: str = COR["texto_sec"]
035 | AZUL: str = COR["azul"]
036 | VERDE: str = COR["sucesso"]
037 | AMARELO: str = COR["aviso"]
038 | ROXO: str = COR["roxo"]
039 | VERMELHO: str = COR["erro"]
040 | ROXO_BTN: str = '#a78bfa'
041 |
042 |
043 | class RelatorioDialog(QDialog):
044 |     session: Any
045 |     printer_service: Any
046 |     company_service: Any
047 |     activity_service: Any
048 |     rb_impressoras: QRadioButton
049 |     rb_atividades: QRadioButton
050 |     status_combo: QComboBox
051 |     pat_input: QLineEdit
052 |     modelo_combo: QComboBox
053 |     local_combo: QComboBox
054 |     rb_pdf: QRadioButton
055 |     rb_excel: QRadioButton
056 |     cb_detalhado: QCheckBox
057 |     cb_abrir: QCheckBox
058 |
059 |     def __init__(self, session: Any, printer_service: Any, company_service: Any, activity_service: Any, par
060 |         super().__init__(parent)
061 |         self.session = session
062 |         self.printer_service = printer_service
063 |         self.company_service = company_service
064 |         self.activity_service = activity_service
065 |         self.setWindowTitle("■ Gerar Relatório")
066 |         self.setMinimumSize(480, 580)
067 |         self.setMaximumSize(480, 750)
068 |         self.setStyleSheet(f"""
069 |             QDialog {{ background-color: {BG}; }}
070 |             QLabel {{ color: {TEXTO}; font-size: 12px; background: transparent; }}
071 |             QGroupBox {{
072 |                 color: {AZUL}; font-weight: bold;
073 |                 border: 1px solid {BORDA}; border-radius: 10px;
074 |                 margin-top: 12px; padding: 18px 12px 12px 12px;
075 |                 font-size: 14px; background-color: {BG};
076 |             }}
077 |             QGroupBox::title {{
078 |                 subcontrol-origin: margin; left: 12px; padding: 0 8px;
079 |                 background-color: {BG}; color: {AZUL};
080 |             }}
081 |             QRadioButton, QCheckBox {{
082 |                 color: {TEXTO}; font-size: 13px; spacing: 8px;
083 |                 padding: 4px 2px; background: transparent;
084 |             }}
085 |             QRadioButton::indicator, QCheckBox::indicator {{

```

```

086 |         width: 16px; height: 16px;
087 |         border: 2px solid {BORDA};
088 |         background-color: {CARD};
089 |     }}
090 |     QRadioButton::indicator {{ border-radius: 8px; }}
091 |     QCheckBox::indicator {{ border-radius: 3px; }}
092 |     QRadioButton::indicator:checked, QCheckBox::indicator:checked {{
093 |         background-color: {ROXO_BTN}; border-color: {ROXO_BTN};
094 |     }}
095 |     QComboBox {{
096 |         background-color: {CARD}; color: {TEXT0};
097 |         border: 1px solid {BORDA}; border-radius: 6px;
098 |         padding: 7px 12px; font-size: 13px; min-height: 20px;
099 |     }}
100 |     QComboBox:hover {{ border-color: {AZUL}; }}
101 |     QComboBox QAbstractItemView {{
102 |         background-color: {CARD}; color: {TEXT0};
103 |         selection-background-color: {ROXO_BTN};
104 |         border: 1px solid {BORDA}; outline: none;
105 |     }}
106 |     QComboBox QAbstractItemView::item {{ padding: 6px 12px; }}
107 |     QComboBox QAbstractItemView::item:hover {{ background-color: {ROXO_BTN}; }}
108 |     QLineEdit {{
109 |         background-color: {CARD}; color: {TEXT0};
110 |         border: 1px solid {BORDA}; border-radius: 6px;
111 |         padding: 7px 12px; font-size: 13px;
112 |     }}
113 |     QLineEdit:hover {{ border-color: {AZUL}; }}
114 |     QLineEdit:focus {{ border-color: {ROXO_BTN}; }}
115 |     QPushButton {{ border: none; border-radius: 6px; padding: 10px 22px; font-size: 14px; font-weight: bold; }}
116 |     QScrollArea {{ border: none; background-color: {BG}; }}
117 |     QScrollBar:vertical {{ background: {CARD}; width: 8px; border-radius: 4px; }}
118 |     QScrollBar::handle:vertical {{ background: {BORDA}; border-radius: 4px; min-height: 25px; }}
119 |     QScrollBar::add-line:vertical, QScrollBar::sub-line:vertical {{ height: 0px; }}
120 |     """)
121 |     self.init_ui()
122 |
123 | def init_ui(self) -> None:
124 |     scroll = QScrollArea()
125 |     scroll.setWidgetResizable(True)
126 |     scroll.setStyleSheet(f"QScrollArea {{ background-color: {BG}; border: none; }}")
127 |
128 |     container = QWidget()
129 |     container.setStyleSheet(f"background-color: {BG};")
130 |     layout = QVBoxLayout(container)
131 |     layout.setSpacing(8)
132 |     layout.setContentsMargins(20, 20, 20, 20)
133 |
134 |     titulo = QLabel("■ Configurações do Relatório")
135 |     titulo.setStyleSheet(f"color: {ROXO}; font-size: 18px; font-weight: bold; background: transparent;")
136 |     titulo.setAlignment(Qt.AlignCenter)
137 |     layout.addWidget(titulo)
138 |
139 |     sep = QFrame()
140 |     sep.setFrameShape(QFrame.HLine)
141 |     sep.setStyleSheet(f"background-color: {BORDA}; max-height: 1px;")
142 |     layout.addWidget(sep)
143 |     layout.addSpacing(5)
144 |
145 |     grupo_tipo = QGroupBox("Tipo de Relatório")
146 |     tipo_layout = QVBoxLayout(grupo_tipo)
147 |     tipo_layout.setSpacing(6)
148 |     self.rb_impressoras = QRadioButton("■■ Impressoras")

```

```

149 |         self.rb_atividades = QRadioButton("■ Atividades / Ordens de Serviço")
150 |         self.rb_impressoras.setChecked(True)
151 |         tipo_layout.addWidget(self.rb_impressoras)
152 |         tipo_layout.addWidget(self.rb_atividades)
153 |         layout.addWidget(grupo_tipo)
154 |
155 |         grupo_filtro = QGroupBox("■ Filtros")
156 |         filtro_layout = QVBoxLayout(grupo_filtro)
157 |         filtro_layout.setSpacing(8)
158 |
159 |         filtro_layout.addWidget(QLabel("Status da Impressora:"))
160 |         self.status_combo = QComboBox()
161 |         configurar_combo(self.status_combo)
162 |         cmp = self.status_combo.completer()
163 |         if cmp:
164 |             cmp.setFilterMode(Qt.MatchFlag.MatchContains)
165 |             cmp.setCaseSensitivity(Qt.CaseSensitivity.CaseInsensitive)
166 |         self.status_combo.addItem("Todos")
167 |         self.status_combo.currentTextChanged.connect(self._atualizar_patrimonios)
168 |         filtro_layout.addWidget(self.status_combo)
169 |
170 |         filtro_layout.addWidget(QLabel("Patrimônio Específico:"))
171 |         self.pat_input = QLineEdit()
172 |         self.pat_input.setPlaceholderText("Deixe em branco ou digite para buscar...")
173 |         self.pat_input.textChanged.connect(self._ao_selecionar_patrimonio)
174 |         filtro_layout.addWidget(self.pat_input)
175 |
176 |         filtro_layout.addWidget(QLabel("Modelo:"))
177 |         self.modelo_combo = QComboBox()
178 |         configurar_combo(self.modelo_combo)
179 |         cmp = self.modelo_combo.completer()
180 |         if cmp:
181 |             cmp.setFilterMode(Qt.MatchFlag.MatchContains)
182 |             cmp.setCaseSensitivity(Qt.CaseSensitivity.CaseInsensitive)
183 |         self.modelo_combo.addItem("Todos")
184 |         from app.models import Printer
185 |         modelos = self.printer_service.modelos_distintos()
186 |         for m in modelos:
187 |             if m:
188 |                 self.modelo_combo.addItem(m)
189 |         filtro_layout.addWidget(self.modelo_combo)
190 |
191 |         filtro_layout.addWidget(QLabel("Local / Empresa:"))
192 |         self.local_combo = QComboBox()
193 |         configurar_combo(self.local_combo)
194 |         cmp = self.local_combo.completer()
195 |         if cmp:
196 |             cmp.setFilterMode(Qt.MatchFlag.MatchContains)
197 |             cmp.setCaseSensitivity(Qt.CaseSensitivity.CaseInsensitive)
198 |         self.local_combo.addItem("Todos")
199 |         from app.models import Company
200 |         empresas = self.company_service.listar_todas()
201 |         for emp in empresas:
202 |             self.local_combo.addItem(emp.nome)
203 |         filtro_layout.addWidget(self.local_combo)
204 |
205 |         layout.addWidget(grupo_filtro)
206 |
207 |         grupo_saida = QGroupBox("■ Formato de Saída")
208 |         saida_layout = QVBoxLayout(grupo_saida)
209 |         saida_layout.setSpacing(6)
210 |         self.rb_pdf = QRadioButton("■ PDF - Relatório formatado")
211 |         self.rb_excel = QRadioButton("■ Excel - Planilha editável")

```

```

212 |         self.rb_pdf.setChecked(True)
213 |         saida_layout.addWidget(self.rb_pdf)
214 |         saida_layout.addWidget(self.rb_excel)
215 |         layout.addWidget(grupo_saida)
216 |
217 |         grupo_extra = QGroupBox("■ Opções Adicionais")
218 |         extra_layout = QVBoxLayout(grupo_extra)
219 |         extra_layout.setSpacing(6)
220 |         self.cb_detalhado = QCheckBox("■ Incluir histórico completo (apenas PDF)")
221 |         self.cb_detalhado.setChecked(True)
222 |         extra_layout.addWidget(self.cb_detalhado)
223 |         self.cb_abrir = QCheckBox("■ Abrir arquivo ao gerar")
224 |         self.cb_abrir.setChecked(True)
225 |         extra_layout.addWidget(self.cb_abrir)
226 |         layout.addWidget(grupo_extra)
227 |         layout.addSpacing(10)
228 |
229 |         btn_layout = QHBoxLayout()
230 |         btn_layout.setSpacing(15)
231 |         btn_layout.addStretch()
232 |         btn_gerar = QPushButton("Gerar Relatório")
233 |         btn_gerar.setCursor(Qt.PointingHandCursor)
234 |         btn_gerar.setMinimumWidth(180)
235 |         btn_gerar.setMinimumHeight(40)
236 |         btn_gerar.setToolTip("Gerar o relatório com os filtros selecionados")
237 |         btn_gerar.setStyleSheet(f"QPushButton {{ background-color: {VERDE}; color: #ffffff; border: none; b
238 |         btn_gerar.clicked.connect(self.gerar_relatorio)
239 |         btn_layout.addWidget(btn_gerar)
240 |         btn_cancelar = QPushButton("Cancelar")
241 |         btn_cancelar.setToolTip("Fechar sem gerar relatório")
242 |         btn_cancelar.setCursor(Qt.PointingHandCursor)
243 |         btn_cancelar.setStyleSheet(f"QPushButton {{ background-color: #2a2a3e; color: {TEXT0}; padding: 10p
244 |         btn_cancelar.clicked.connect(self.reject)
245 |         btn_layout.addWidget(btn_cancelar)
246 |         btn_layout.addStretch()
247 |         layout.addLayout(btn_layout)
248 |         layout.addSpacing(10)
249 |
250 |         scroll.setWidget(container)
251 |         main_layout = QVBoxLayout(self)
252 |         main_layout.setContentsMargins(0, 0, 0, 0)
253 |         main_layout.addWidget(scroll)
254 |         self._atualizar_patrimonios()
255 |
256 |     def _atualizar_patrimonios(self) -> None:
257 |         status: str = self.status_combo.currentText()
258 |         patrimonios: list[str] = self.printer_service.listar_patrimonios(status)
259 |         completer: QCompleter = QCompleter(patrimonios)
260 |         completer.setCaseSensitivity(Qt.CaseInsensitive)
261 |         completer.setFilterMode(Qt.MatchContains)
262 |         self.pat_input.setCompleter(completer)
263 |
264 |     def _ao_selecionar_patrimonio(self, texto: str) -> None:
265 |         status: str = self.status_combo.currentText()
266 |         patrimonios: list[str] = self.printer_service.listar_patrimonios_por_busca(texto, status)
267 |         completer: QCompleter = QCompleter(patrimonios)
268 |         completer.setCaseSensitivity(Qt.CaseInsensitive)
269 |         completer.setFilterMode(Qt.MatchContains)
270 |         self.pat_input.setCompleter(completer)
271 |
272 |         if texto.strip():
273 |             printer = self.printer_service.buscar_por_patrimonio(texto.strip())
274 |             if printer:

```



```

275 |         if printer.modelo:
276 |             idx = self.modelo_combo.findText(printer.modelo)
277 |             if idx >= 0: self.modelo_combo.setCurrentIndex(idx)
278 |         if printer.local_atual:
279 |             idx = self.local_combo.findText(printer.local_atual)
280 |             if idx >= 0: self.local_combo.setCurrentIndex(idx)
281 |         if printer.status:
282 |             idx = self.status_combo.findText(printer.status)
283 |             if idx >= 0: self.status_combo.setCurrentIndex(idx)
284 |
285 |     def gerar_relatorio(self) -> None:
286 |         from PySide6.QtWidgets import QFileDialog, QMessageBox
287 |
288 |         from app.services.relatorio_service import RelatorioService
289 |
290 |         status_filtro = self.status_combo.currentText()
291 |         patrimonio = self.pat_input.text().strip()
292 |         modelo = self.modelo_combo.currentText()
293 |         local = self.local_combo.currentText()
294 |         formato = 'pdf' if self.rb_pdf.isChecked() else 'xlsx'
295 |         detalhado = self.cb_detalhado.isChecked()
296 |         abrir = self.cb_abrir.isChecked()
297 |         tipo_rel = 'impressoras' if self.rb_impressoras.isChecked() else 'atividades'
298 |
299 |         sufixo = f"_{patrimonio}" if patrimonio else ""
300 |         nome_padrao = f"relatorio_{tipo_rel}{sufixo}_{dt.now().strftime('%Y%m%d_%H%M%S')}.{formato}"
301 |
302 |         filepath, _ = QFileDialog.getSaveFileName(self, "Salvar Relatório", nome_padrao, f"{'PDF' if format
303 |         if not filepath: return
304 |
305 |         try:
306 |             if tipo_rel == 'impressoras':
307 |                 printers = self.printer_service.listar_por_filtros(status_filtro, patrimonio, modelo, local
308 |                 if not printers: QMessageBox.warning(self, "Aviso", "Nenhuma impressora encontrada!"); retu
309 |                 if formato == 'pdf' and detalhado: self._gerar_pdf_detalhado(printers, filepath)
310 |                 elif formato == 'pdf': RelatorioService.exportar_impressoras_pdf(printers, filepath)
311 |                 else: RelatorioService.exportar_impressoras_excel(printers, filepath)
312 |             else:
313 |                 if patrimonio:
314 |                     printer_obj = self.printer_service.buscar_por_patrimonio(patrimonio)
315 |                     atividades = self.activity_service.listar_por_impressora(printer_obj.id, limite=500) if
316 |                 else:
317 |                     atividades = self.activity_service.listar(limite=500)
318 |                 if not atividades: QMessageBox.warning(self, "Aviso", "Nenhuma atividade encontrada!"); ret
319 |                 if formato == 'pdf': RelatorioService.exportar_atividades_pdf(atividades, filepath)
320 |                 else: RelatorioService.exportar_atividades_excel(atividades, filepath)
321 |             if abrir:
322 |                 try:
323 |                     os.startfile(filepath)
324 |                 except Exception as e:
325 |                     QMessageBox.warning(self, "Aviso", f"Não foi possível abrir o arquivo:\n{e}")
326 |                     QMessageBox.information(self, "Sucesso", f"Relatório gerado com sucesso!\n\n{filepath}")
327 |                     self.accept()
328 |             except Exception as e:
329 |                 QMessageBox.critical(self, "Erro", f"Erro ao gerar relatório:\n\n{str(e)}")
330 |
331 |     def _gerar_pdf_detalhado(self, printers: list[Any], filepath: str) -> None:
332 |         from reportlab.lib import colors
333 |         from reportlab.lib.pagesizes import A4
334 |         from reportlab.lib.styles import getSampleStyleSheet
335 |         from reportlab.lib.units import cm
336 |         from reportlab.platypus import (
337 |             Image,

```

```

338 |         PageBreak,
339 |         Paragraph,
340 |         SimpleDocTemplate,
341 |         Spacer,
342 |         Table,
343 |         TableStyle,
344 |     )
345 |
346 |     logo_path = str(BUNDLE_DIR / "logo.png")
347 |
348 |     doc = SimpleDocTemplate(filepath, pagesize=A4, leftMargin=28, rightMargin=28, topMargin=28, bottomMargin=28)
349 |     styles = getSampleStyleSheet()
350 |     cor_primaria = colors.HexColor("#1A3C6C")
351 |     cor_fundo_zebra = colors.HexColor("#F8F9FA")
352 |
353 |     def draw_footer(canvas, doc):
354 |         canvas.saveState()
355 |         largura, altura = A4
356 |
357 |         y_assinatura = 2.5 * cm
358 |         centro = largura / 2
359 |         linha_largura = 9 * cm
360 |         x1 = centro - linha_largura / 2
361 |         x2 = centro + linha_largura / 2
362 |
363 |         canvas.setStrokeColor(colors.black)
364 |         canvas.setLineWidth(1)
365 |         canvas.line(x1, y_assinatura + 18, x2, y_assinatura + 18)
366 |
367 |         canvas.setFont("Helvetica", 10)
368 |         canvas.drawCentredString(centro, y_assinatura + 4, "Técnico Responsável")
369 |
370 |         canvas.setFont("Helvetica", 8)
371 |         canvas.drawCentredString(centro, y_assinatura - 12, f"Data: {dt.now().strftime('%d/%m/%Y %H:%M')}")
372 |
373 |         y_divisoria = 1.2 * cm
374 |         canvas.setStrokeColor(colors.lightgrey)
375 |         canvas.setLineWidth(0.5)
376 |         canvas.line(doc.leftMargin, y_divisoria, largura - doc.rightMargin, y_divisoria)
377 |
378 |         canvas.setFont("Helvetica", 8)
379 |         canvas.setFillColor(colors.gray)
380 |
381 |         canvas.drawString(doc.leftMargin, y_divisoria - 12, "Brasil Toner - Recife PE")
382 |
383 |         num_pagina = canvas.getPageNumber()
384 |         canvas.drawRightString(largura - doc.rightMargin, y_divisoria - 12, f"Página {num_pagina}")
385 |         canvas.restoreState()
386 |
387 |     elements = []
388 |
389 |     for idx, p in enumerate(printers):
390 |         if os.path.exists(logo_path):
391 |             img = Image(logo_path, width=4*cm)
392 |             img.hAlign = 'LEFT'
393 |             elements.append(img)
394 |             elements.append(Spacer(1, 12))
395 |         else:
396 |             elements.append(Spacer(1, 40))
397 |
398 |     style_title = styles["Title"]
399 |     style_title.textColor = cor_primaria
400 |     style_title.fontSize = 18

```

```

401 |     elements.append(Paragraph(f"Relatório de Impressora - Patrimônio: {p.patrimonio}", style_title))
402 |     elements.append(Spacer(1, 20))
403 |
404 |     maint_count = self.activity_service.contar_por_impressora_e_kind(p.id, "MANUTENCAO")
405 |
406 |     header_data = [
407 |         ["Patrimônio", p.patrimonio or "-"],
408 |         ["Modelo", p.modelo or "-"],
409 |         ["Serial", p.serial or "-"],
410 |         ["Status", p.status or "-"],
411 |         ["Peças Faltantes", p.pecas_faltantes or "-"],
412 |         ["Local atual", p.local_atual or "-"],
413 |         ["Contador de Manutenções", str(maint_count)],
414 |         ["Última Revisão", p.proxima_revisao.strftime("%d/%m/%Y") if p.proxima_revisao else "-"],
415 |         ["Técnico Revisor", p.tecnico or "-"]
416 |     ]
417 |
418 |     t_header = Table(header_data, colWidths=[140, 380])
419 |     t_header.setStyle(TableStyle([
420 |         ('BACKGROUND', (0, 0), (0, -1), cor_fundo_zebra),
421 |         ('TEXTCOLOR', (0, 0), (0, -1), cor_primaria),
422 |         ('FONTNAME', (0, 0), (0, -1), 'Helvetica-Bold'),
423 |         ('GRID', (0, 0), (-1, -1), 0.5, colors.lightgrey),
424 |         ('FONTSIZE', (0, 0), (-1, -1), 10),
425 |         ('PADDING', (0, 0), (-1, -1), 6),
426 |     ]))
427 |     elements.append(t_header)
428 |     elements.append(Spacer(1, 20))
429 |
430 |     if (p.observacao or "").strip():
431 |         elements.append(Paragraph("<b>Observação geral:</b>", styles["Normal"]))
432 |         elements.append(Paragraph((p.observacao or "").replace("\n", "<br/>"), styles["BodyText"]))
433 |         elements.append(Spacer(1, 15))
434 |
435 |     elements.append(Paragraph("<b>Histórico de atividades:</b>", styles["Normal"]))
436 |     elements.append(Spacer(1, 8))
437 |
438 |     atividades = self.activity_service.listar_por_impressora(p.id, limite=15)
439 |
440 |     data_hist = [
441 |         ["Data/Hora", "Tipo", "Descrição", "Peças", "De", "Para"]
442 |     ]
443 |     for a in atividades:
444 |         desc = Paragraph((a.notes or "")[:1200], styles["BodyText"])
445 |         pecas = Paragraph((a.parts_used or "")[:400], styles["BodyText"])
446 |         data_hist.append([
447 |             a.event_at.strftime("%d/%m/%Y %H:%M") if a.event_at else "-",
448 |             "Manutenção" if a.kind == "MANUTENCAO" else "Movimentação",
449 |             desc, pecas,
450 |             a.from_location or "-", a.to_location or "-",
451 |         ])
452 |
453 |     t_hist = Table(data_hist, colWidths=[75, 75, 170, 90, 55, 55], repeatRows=1)
454 |     t_hist.setStyle(TableStyle([
455 |         ('BACKGROUND', (0, 0), (-1, 0), cor_primaria),
456 |         ('TEXTCOLOR', (0, 0), (-1, 0), colors.whitesmoke),
457 |         ('FONTNAME', (0, 0), (-1, 0), 'Helvetica-Bold'),
458 |         ('ALIGN', (0, 0), (-1, 0), 'CENTER'),
459 |         ('FONTSIZE', (0, 0), (-1, 0), 9),
460 |         ('ROWBACKGROUNDS', (0, 1), (-1, -1), [colors.white, cor_fundo_zebra]),
461 |         ('GRID', (0, 0), (-1, -1), 0.5, colors.lightgrey),
462 |         ('VALIGN', (0, 0), (-1, -1), "TOP"),
463 |         ('FONTSIZE', (0, 1), (-1, -1), 8),
464 |         ('PADDING', (0, 0), (-1, -1), 5),
465 |     ]))

```

```

464 |             elements.append(t_hist)
465 |
466 |             if idx < len(printers) - 1:
467 |                 elements.append(PageBreak())
468 |
469 |         doc.build(elements, onFirstPage=draw_footer, onLaterPages=draw_footer)

```

=== ARQUIVO: .lapp\views__init__.py ===

```

001 | from __future__ import annotations
002 |
003 | from typing import TYPE_CHECKING
004 |
005 | if TYPE_CHECKING:
006 |     from .login_dialog import LoginDialog
007 |     from .main_window import MainWindow
008 |     from .relatorio_dialog import RelatorioDialog

```

=== ARQUIVO: .lapp\views\dialogs__init__.py ===

=== ARQUIVO: .lapp\views\pages\alertas_page.py ===

```

001 | from __future__ import annotations
002 |
003 | from datetime import datetime as dt
004 | from typing import Any
005 |
006 | from PySide6.QtCore import Qt
007 | from PySide6.QtGui import QColor
008 | from PySide6.QtWidgets import (
009 |     QComboBox,
010 |     QDialog,
011 |     QDialogButtonBox,
012 |     QFormLayout,
013 |     QFrame,
014 |     QGridLayout,
015 |     QHBoxLayout,
016 |     QLabel,
017 |     QLineEdit,
018 |     QMessageBox,
019 |     QPushButton,
020 |     QTableWidgetItem,
021 |     QTextEdit,
022 |     QVBoxLayout,
023 |     QWidget,
024 | )
025 |
026 | from app.utils.helpers import formatar_data_hora
027 | from app.utils.ui_helpers import tratar_erro
028 | from app.utils.validacao import ValidadorCampo, obrigatorio
029 | from app.views.styles.theme import (
030 |     ESTILO_BOTAO_AVISO,
031 |     ESTILO_BOTAO_ERRO,
032 |     ESTILO_BOTAO_FECHAR,
033 |     ESTILO_BOTAO_PRIMARIO,

```

```

034 |     ESTILO_BOTAO_SECUNDARIO,
035 |     configurar_combo,
036 |     ESTILO_DIALOG,
037 |     ESTILO_INPUT,
038 |     ESTILO_SUBTITULO,
039 |     ESTILO_TITULO_PAGINA,
040 |     group_box,
041 |     input_label,
042 |     campo_rotulo,
043 |     campo_readonly,
044 | )
045 | from app.views.widgets.confirm_dialog import ConfirmacaoDigitalDialog
046 | from app.views.widgets.pagination import PaginacaoWidget
047 | from app.views.widgets.search_bar import SearchBar
048 | from app.views.widgets.table_widget import TabelaPadrao
049 | from app.views.widgets.toast import ToastManager
050 |
051 |
052 | CORES_TIPO: dict[str, str] = {
053 |     "revisao": "#60a5fa",
054 |     "critico": "#f87171",
055 |     "aviso": "#fbbf24",
056 |     "info": "#a78bfa",
057 |     "estoque": "#fb923c",
058 | }
059 |
060 | STATUS_ALERTA: dict[bool, tuple[str, str]] = {
061 |     True: ("Resolvido", "#34d399"),
062 |     False: ("Pendente", "#fbbf24"),
063 | }
064 |
065 | TIPO_LABELS: dict[str, str] = {
066 |     "revisao": "Revisão",
067 |     "critico": "Crítico",
068 |     "aviso": "Aviso",
069 |     "info": "Informação",
070 |     "estoque": "Estoque",
071 | }
072 |
073 | TIPO_OPCOES: list[str] = ["revisao", "critico", "aviso", "info", "estoque"]
074 |
075 |
076 | class AlertasPage(QWidget):
077 |     session: Any
078 |     alert_service: Any
079 |     printer_service: Any
080 |     part_service: Any | None
081 |     _filtro_status: str | None
082 |     _filtro_busca: str
083 |     btn_todos: QPushButton
084 |     btn_pendentes: QPushButton
085 |     btn_resolvidos: QPushButton
086 |     lbl_contador: QLabel
087 |     search: SearchBar
088 |     tabela: TabelaPadrao
089 |     _alertas_visiveis: list[Any]
090 |     _paginacao: PaginacaoWidget
091 |
092 |     def __init__(self, session: Any, alert_service: Any, printer_service: Any, part_service: Any | None = None):
093 |         super().__init__()
094 |         self.session = session
095 |         self.alert_service = alert_service
096 |         self.printer_service = printer_service

```

```

097 |         self.part_service = part_service
098 |         self._filtro_status = None
099 |         self._filtro_busca = ""
100 |
101 |         layout = QVBoxLayout(self)
102 |         layout.setContentsMargins(24, 24, 24, 24)
103 |         layout.setSpacing(16)
104 |
105 |         header = QHBoxLayout()
106 |         header.setSpacing(10)
107 |
108 |         titulo = QLabel("Alertas")
109 |         titulo.setStyleSheet(ESTILO_TITULO_PAGINA)
110 |         sub = QLabel("Monitore revisões, avisos e problemas das impressoras")
111 |         sub.setStyleSheet(ESTILO_SUBTITULO)
112 |
113 |         titulo_col = QVBoxLayout()
114 |         titulo_col.setSpacing(2)
115 |         titulo_col.addWidget(titulo)
116 |         titulo_col.addWidget(sub)
117 |         header.addLayout(titulo_col)
118 |         header.addStretch()
119 |
120 |         btn_estoque = QPushButton("■ Verif. Estoque")
121 |         btn_estoque.setCursor(Qt.PointingHandCursor)
122 |         btn_estoque.setStyleSheet(ESTILO_BOTAO_SECUNDARIO)
123 |         btn_estoque.setToolTip("Verificar alertas de estoque baixo")
124 |         btn_estoque.clicked.connect(self._gerar_alertas_estoque)
125 |         header.addWidget(btn_estoque)
126 |
127 |         btn_novo = QPushButton("■ Novo Alerta")
128 |         btn_novo.setCursor(Qt.PointingHandCursor)
129 |         btn_novo.setStyleSheet(ESTILO_BOTAO_PRIMARIO)
130 |         btn_novo.setToolTip("Criar novo alerta")
131 |         btn_novo.clicked.connect(self._novo)
132 |         header.addWidget(btn_novo)
133 |
134 |         btn_atualizar = QPushButton("■ Atualizar")
135 |         btn_atualizar.setCursor(Qt.PointingHandCursor)
136 |         btn_atualizar.setStyleSheet(ESTILO_BOTAO_SECUNDARIO)
137 |         btn_atualizar.setToolTip("Recarregar lista de alertas")
138 |         btn_atualizar.clicked.connect(self.recarregar)
139 |         header.addWidget(btn_atualizar)
140 |
141 |         layout.addLayout(header)
142 |
143 |         filtros = QHBoxLayout()
144 |         filtros.setSpacing(8)
145 |
146 |         self.btn_todos = QPushButton("■ Todos")
147 |         self.btn_todos.setCursor(Qt.PointingHandCursor)
148 |         self.btn_todos.setStyleSheet(self._estilo_filtro(True))
149 |         self.btn_todos.setToolTip("Mostrar todos os alertas")
150 |         self.btn_todos.clicked.connect(lambda: self._alternar_filtro(None))
151 |
152 |         self.btn_pendentes = QPushButton("■ Pendentes")
153 |         self.btn_pendentes.setCursor(Qt.PointingHandCursor)
154 |         self.btn_pendentes.setStyleSheet(self._estilo_filtro(False))
155 |         self.btn_pendentes.setToolTip("Filtrar alertas pendentes")
156 |         self.btn_pendentes.clicked.connect(lambda: self._alternar_filtro("pendentes"))
157 |
158 |         self.btn_resolvidos = QPushButton("■ Resolvidos")
159 |         self.btn_resolvidos.setCursor(Qt.PointingHandCursor)

```

```

160 |         self.btn_resolvidos.setStyleSheet(self._estilo_filtro(False))
161 |         self.btn_resolvidos.setToolTip("Filtrar alertas resolvidos")
162 |         self.btn_resolvidos.clicked.connect(lambda: self._alternar_filtro("resolvidos"))
163 |
164 |         filtros.addWidget(self.btn_todos)
165 |         filtros.addWidget(self.btn_pendentes)
166 |         filtros.addWidget(self.btn_resolvidos)
167 |         filtros.addStretch()
168 |
169 |         self.lbl_contador = QLabel("")
170 |         self.lbl_contador.setStyleSheet("color: #475569; font-size: 12px; background: transparent;")
171 |
172 |         layout.addLayout(filtros)
173 |
174 |         self.search = SearchBar(placeholder="Buscar por título, impressora ou tipo...")
175 |         self.search.textChanged().connect(lambda texto: self._buscar(texto))
176 |         layout.addWidget(self.search)
177 |
178 |         colunas = ["Referência", "Tipo", "Título", "Descrição", "Data", "Status"]
179 |         self.tabela = TabelaPadrao(colunas)
180 |         self.tabela.cellDoubleClicked.connect(self._detalhes)
181 |         layout.addWidget(self.tabela)
182 |
183 |         self._alertas_visiveis = []
184 |         self._paginacao = PaginacaoWidget()
185 |         self._paginacao.pagina_alterada.connect(lambda p: self._carregar())
186 |         layout.addWidget(self._paginacao)
187 |
188 |         self._carregar()
189 |
190 |     def _estilo_filtro(self, ativo: bool) -> str:
191 |         if ativo:
192 |             return (
193 |                 "QPushButton { background-color: #a78bfa; color: #ffffff; border: none;"
194 |                 " border-radius: 6px; padding: 7px 16px; font-size: 12px; font-weight: 600; }"
195 |             )
196 |         return (
197 |             "QPushButton { background-color: transparent; color: #475569;"
198 |             " border: 1px solid #2e2e4a; border-radius: 6px; padding: 7px 16px;"
199 |             " font-size: 12px; font-weight: 500; }"
200 |             " QPushButton:hover { border-color: #4a4a6a; color: #94a3b8; }"
201 |         )
202 |
203 |     def _atualizar_botoes_filtro(self) -> None:
204 |         self.btn_todos.setStyleSheet(self._estilo_filtro(self._filtro_status is None))
205 |         self.btn_pendentes.setStyleSheet(self._estilo_filtro(self._filtro_status == "pendentes"))
206 |         self.btn_resolvidos.setStyleSheet(self._estilo_filtro(self._filtro_status == "resolvidos"))
207 |
208 |     def _alternar_filtro(self, status: str | None) -> None:
209 |         self._filtro_status = status
210 |         self._atualizar_botoes_filtro()
211 |         self._carregar()
212 |
213 |     def _buscar(self, texto: str) -> None:
214 |         self._filtro_busca = texto
215 |         self._carregar()
216 |
217 |     def recarregar(self) -> None:
218 |         self._filtro_busca = ""
219 |         self._carregar()
220 |
221 |     def _carregar(self):
222 |         filtro = self._filtro_busca or None

```

```

223 |         apenas_pendentes = self._filtro_status == "pendentes"
224 |         apenas_resolvidos = self._filtro_status == "resolvidos"
225 |         alertas = self.alert_service.listar_todos(
226 |             apenas_pendentes=apenas_pendentes,
227 |             apenas_resolvidos=apenas_resolvidos,
228 |             filtro_busca=filtro,
229 |             limite=self._paginacao.limit, offset=self._paginacao.offset,
230 |         )
231 |         total = self.alert_service.contar_todos(
232 |             apenas_pendentes=apenas_pendentes,
233 |             apenas_resolvidos=apenas_resolvidos,
234 |             filtro_busca=filtro,
235 |         )
236 |         self._paginacao.configurar(total, pagina_atual=self._paginacao.pagina,
237 |                                     itens_por_pagina=self._paginacao.limit)
238 |         self._alertas_visiveis = alertas
239 |
240 |         total_pendentes = self.alert_service.contar_pendentes()
241 |         self.lbl_contador.setText(f"{total_pendentes} pendente(s)")
242 |
243 |         self.tabela.setRowCount(len(alertas))
244 |
245 |         for i, a in enumerate(alertas):
246 |             if a.printer_id:
247 |                 printer = a.printer or self.printer_service.buscar_por_id(a.printer_id)
248 |                 ref = printer.patrimonio if printer else a.printer_id
249 |             elif a.part_id:
250 |                 ref = a.part.nome if a.part else f"Peça #{a.part_id}"
251 |             else:
252 |                 ref = "-"
253 |
254 |             self.tabela.setItem(i, 0, self.tabela.item_colorido(ref, "#e2e8f0"))
255 |             cor_tipo = CORES_TIPO.get(a.tipo, "#94949f")
256 |             label_tipo = TIPO_LABELS.get(a.tipo, a.tipo)
257 |             self.tabela.definir_badge(i, 1, label_tipo, cor_tipo)
258 |             self.tabela.setItem(i, 2, QTableWidgetItem(a.titulo or ""))
259 |             self.tabela.setItem(i, 3, QTableWidgetItem((a.descricao or "")[:60] + ("..." if len(a.descricao) > 60 else "")))
260 |             self.tabela.setItem(i, 4, QTableWidgetItem(formatar_data_hora(a.data_alerta or a.created_at)))
261 |
262 |             label_status, cor_status = STATUS_ALERTA.get(a.resolvido, ("Desconhecido", "#4b5563"))
263 |             self.tabela.definir_badge(i, 5, label_status, cor_status)
264 |
265 |         self.tabela.redimensionar()
266 |
267 |     def _gerar_alertas_estoque(self) -> None:
268 |         try:
269 |             criados = self.alert_service.verificar_estoque_baixo()
270 |             ToastManager.atualizar_status(self.alert_service.contar_pendentes())
271 |             if criados:
272 |                 ToastManager.info(f"{criados} alerta(s) de estoque baixo gerado(s)")
273 |             else:
274 |                 ToastManager.sucesso("Nenhuma peça com estoque crítico")
275 |         except Exception as e:
276 |             ToastManager.erro(f"Erro ao verificar estoque: {str(e)}")
277 |         self.recarregar()
278 |
279 |     def _novo(self) -> None:
280 |         dialog = QDialog(self)
281 |         dialog.setWindowTitle("Novo Alerta")
282 |         dialog.setMinimumSize(520, 500)
283 |         dialog.setStyleSheet(ESTILO_DIALOG)
284 |         layout = QVBoxLayout(dialog)
285 |         layout.setSpacing(14)

```



```

286 |         layout.setContentsMargins(20, 20, 20, 20)
287 |
288 |         alerta_box, alerta_layout = group_box("Alerta")
289 |         form_alerta = QFormLayout()
290 |         form_alerta.setSpacing(8)
291 |
292 |         printer_combo = QComboBox()
293 |         configurar_combo(printer_combo)
294 |         cmp = printer_combo.completer()
295 |         if cmp:
296 |             cmp.setFilterMode(Qt.MatchFlag.MatchContains)
297 |             cmp.setCaseSensitivity(Qt.CaseSensitivity.CaseInsensitive)
298 |         impressoras = self.printer_service.listar_todos()
299 |         printer_combo.addItem("(nenhuma impressora)", None)
300 |         for p in impressoras:
301 |             printer_combo.addItem(f"{p.patrimonio} - {p.modelo}", p.id)
302 |         form_alerta.addRow("Impressora:", printer_combo)
303 |
304 |         part_combo = QComboBox()
305 |         configurar_combo(part_combo)
306 |         cmp = part_combo.completer()
307 |         if cmp:
308 |             cmp.setFilterMode(Qt.MatchFlag.MatchContains)
309 |             cmp.setCaseSensitivity(Qt.CaseSensitivity.CaseInsensitive)
310 |         part_combo.addItem("(nenhuma peça)", None)
311 |         if self.part_service:
312 |             partes = self.part_service.listar_todas()
313 |             for p in partes:
314 |                 part_combo.addItem(f"{p.nome} ({p.codigo})", p.id)
315 |         form_alerta.addRow("Peça:", part_combo)
316 |
317 |         tipo_combo = QComboBox()
318 |         configurar_combo(tipo_combo)
319 |         cmp = tipo_combo.completer()
320 |         if cmp:
321 |             cmp.setFilterMode(Qt.MatchFlag.MatchContains)
322 |             cmp.setCaseSensitivity(Qt.CaseSensitivity.CaseInsensitive)
323 |         for t in TIPO_OPCOES:
324 |             tipo_combo.addItem(TIPO_LABELS.get(t, t), t)
325 |         form_alerta.addRow("Tipo *:", tipo_combo)
326 |
327 |         titulo_input = QLineEdit()
328 |         titulo_input.setPlaceholderText("Título do alerta")
329 |         titulo_input.setStyleSheet(ESTILO_INPUT)
330 |         form_alerta.addRow("Título *:", titulo_input)
331 |
332 |         erro_titulo = QLabel()
333 |         erro_titulo.setStyleSheet("color: #ef4444; font-size: 10px; background: transparent;")
334 |         erro_titulo.hide()
335 |         form_alerta.addRow("", erro_titulo)
336 |         ValidadorCampo(titulo_input, obrigatorio, erro_titulo)
337 |
338 |         alerta_layout.addLayout(form_alerta)
339 |         layout.addWidget(alerta_box)
340 |
341 |         detalhes_box, detalhes_layout = group_box("Detalhes")
342 |         form_detalhes = QFormLayout()
343 |         form_detalhes.setSpacing(8)
344 |
345 |         desc_input = QTextEdit()
346 |         desc_input.setPlaceholderText("Descrição detalhada...")
347 |         desc_input.setMaximumHeight(80)
348 |         desc_input.setStyleSheet(ESTILO_INPUT)

```

```

349 |         form_detalhes.addRow("Descrição:", desc_input)
350 |
351 |         from PySide6.QtWidgets import QDateEdit
352 |         data_agendada = QDateEdit()
353 |         data_agendada.setCalendarPopup(True)
354 |         data_agendada.setDate(data_agendada.date().addDays(1))
355 |         data_agendada.setStyleSheet(ESTILO_INPUT)
356 |         data_agendada.setSpecialValueText("Sem data")
357 |         form_detalhes.addRow("Agendar para:", data_agendada)
358 |
359 |         detalhes_layout.addLayout(form_detalhes)
360 |         layout.addWidget(detalhes_box)
361 |
362 |         layout.addStretch()
363 |         botoes = QDialogButtonBox(QDialogButtonBox.Ok | QDialogButtonBox.Cancel)
364 |         botoes.setStyleSheet(
365 |             "QPushButton { border: none; border-radius: 6px; padding: 8px 16px; font-weight: bold; }"
366 |             " QPushButton[text='OK'] { background-color: #a78bfa; color: white; }"
367 |             " QPushButton[text='Cancel'] { background-color: #1f2937; color: #94a3b8; }"
368 |         )
369 |         btn_ok = botoes.button(QDialogButtonBox.Ok)
370 |         if btn_ok:
371 |             btn_ok.setToolTip("Salvar novo alerta")
372 |         btn_cancel = botoes.button(QDialogButtonBox.Cancel)
373 |         if btn_cancel:
374 |             btn_cancel.setToolTip("Cancelar")
375 |         botoes.accepted.connect(lambda: self._salvar_novo(dialog, printer_combo, part_combo, tipo_combo, ti
376 |         botoes.rejected.connect(dialog.reject)
377 |         layout.addWidget(botoes)
378 |         dialog.exec()
379 |
380 |     def _salvar_novo(self, dialog: QDialog, printer_combo: QComboBox, part_combo: QComboBox, tipo_combo: QC
381 |         titulo = titulo_input.text().strip()
382 |         if not titulo:
383 |             QMessageBox.warning(dialog, "Aviso", "Preencha o título!")
384 |             return
385 |
386 |         printer_id = printer_combo.currentData()
387 |         part_id = part_combo.currentData()
388 |         data_ag = data_agendada.date().toPython() if data_agendada.date() != data_agendada.minimumDate() el
389 |         from datetime import datetime, time
390 |         data_agendada_dt = datetime.combine(data_ag, time(0, 0)) if data_ag else None
391 |
392 |         with tratar_erro("criar alerta"):
393 |             self.alert_service.criar(
394 |                 printer_id=printer_id,
395 |                 part_id=part_id,
396 |                 tipo=tipo_combo.currentData(),
397 |                 titulo=titulo,
398 |                 descricao=desc_input.toPlainText().strip(),
399 |                 data_agendada=data_agendada_dt,
400 |             )
401 |             ToastManager.atualizar_status(self.alert_service.contar_pendentes())
402 |             ToastManager.sucesso(f"Alerta criado: {titulo}")
403 |             agendados = self.alert_service.verificar_agendados()
404 |             for a in agendados:
405 |                 ref = a.printer.patrimonio if a.printer else (a.part.nome if a.part else "")
406 |                 ToastManager.aviso(f"■ {a.titulo}{' - ' + ref if ref else ''}")
407 |             self.recarregar()
408 |             dialog.accept()
409 |
410 |     def _detalhes(self, row: int) -> None:
411 |         if row < 0 or row >= len(self._alertas_visiveis):

```

```

412 |         return
413 |     alerta = self._alertas_visiveis[row]
414 |     printer = alerta.printer if alerta.printer_id else None
415 |     pat = printer.patrimonio if printer else (alerta.printer_id or "-")
416 |
417 |     dialog = QDialog(self)
418 |     dialog.setWindowTitle(f"Alerta - {alerta.titulo}")
419 |     dialog.setMinimumSize(600, 500)
420 |     dialog.setStyleSheet(ESTILO_DIALOG)
421 |
422 |     layout = QVBoxLayout(dialog)
423 |     layout.setContentsMargins(24, 24, 24, 24)
424 |     layout.setSpacing(14)
425 |
426 |     header_row = QHBoxLayout()
427 |     header_row.setSpacing(10)
428 |     header_row.addWidget(QLabel(f"<b style='color:#a78bfa;font-size:18px'>{alerta.titulo}</b>"), stretch=1)
429 |
430 |     cor_tipo = CORES_TIPO.get(alerta.tipo, "#94949f")
431 |     label_tipo = TIPO_LABELS.get(alerta.tipo, alerta.tipo)
432 |     badge_tipo = QLabel(label_tipo)
433 |     badge_tipo.setStyleSheet(
434 |         f"background-color: {cor_tipo}22; color: {cor_tipo};"
435 |         " padding: 4px 12px; border-radius: 10px; font-size: 11px; font-weight: 700;"
436 |     )
437 |     header_row.addWidget(badge_tipo)
438 |
439 |     label_status, cor_status = STATUS_ALERTA.get(alerta.resolvido, ("-", "#4b5563"))
440 |     badge_status = QLabel(label_status)
441 |     badge_status.setStyleSheet(
442 |         f"background-color: {cor_status}22; color: {cor_status};"
443 |         " padding: 4px 12px; border-radius: 10px; font-size: 11px; font-weight: 700;"
444 |     )
445 |     header_row.addWidget(badge_status)
446 |
447 |     layout.addLayout(header_row)
448 |
449 |     linha = QFrame()
450 |     linha.setFrameShape(QFrame.HLine)
451 |     linha.setStyleSheet("border: none; border-top: 1px solid #2a2a3e;")
452 |     layout.addWidget(linha)
453 |
454 |     info_box, info_layout = group_box("Informações")
455 |     info_grid = QGridLayout()
456 |     info_grid.setSpacing(6)
457 |     info_grid.setHorizontalSpacing(12)
458 |
459 |     campos_info = [
460 |         ("■■ Impressora", str(pat)),
461 |     ]
462 |     if alerta.part_id:
463 |         nome_pecas = alerta.part.nome if alerta.part else f"ID {alerta.part_id}"
464 |         campos_info.append(("■■ Peça", nome_pecas))
465 |     campos_info.append(("■■ Tipo", label_tipo))
466 |     campos_info.append(("■■ Data", formatar_data_hora(alerta.data_alerta or alerta.created_at)))
467 |     if alerta.data_agendada:
468 |         campos_info.append(("■■ Agendado", formatar_data_hora(alerta.data_agendada)))
469 |     if alerta.resolvido:
470 |         campos_info.append(("■■ Resolvido em", formatar_data_hora(alerta.resolvido_em)))
471 |
472 |     for i, (rotulo, valor) in enumerate(campos_info):
473 |         col = i % 2
474 |         row_g = (i // 2) * 2

```

```

475 |         info_grid.addWidget(campo_rotulo(rotulo), row_g, col)
476 |         val_lbl = QLabel(str(valor))
477 |         val_lbl.setWordWrap(True)
478 |         val_lbl.setStyleSheet("color: #c8c8e0; font-size: 11px; font-weight: 500; background: transparent; border: 1px solid #c8c8e0; padding: 2px;")
479 |         info_grid.addWidget(val_lbl, row_g + 1, col)
480 |
481 |     info_grid.setColumnStretch(0, 1)
482 |     info_grid.setColumnStretch(1, 1)
483 |     info_layout.addLayout(info_grid)
484 |     layout.addWidget(info_box)
485 |
486 |     desc_box, desc_layout = group_box("Descrição")
487 |     desc_label = campo_readonly(alerta.descricao or "-")
488 |     desc_layout.addWidget(desc_label)
489 |     layout.addWidget(desc_box)
490 |
491 |     botoes = QHBoxLayout()
492 |     botoes.setSpacing(10)
493 |
494 |     if not alerta.resolvido:
495 |         btn_resolver = QPushButton("■ Resolver Alerta")
496 |         btn_resolver.setCursor(Qt.PointingHandCursor)
497 |         btn_resolver.setMinimumHeight(38)
498 |         btn_resolver.setStyleSheet(ESTILO_BOTAO_PRIMARIO)
499 |         btn_resolver.setToolTip("Marcar este alerta como resolvido")
500 |         btn_resolver.clicked.connect(lambda: self._resolver(alerta, dialog))
501 |         botoes.addWidget(btn_resolver)
502 |
503 |         btn_editar = QPushButton("■ Editar")
504 |         btn_editar.setCursor(Qt.PointingHandCursor)
505 |         btn_editar.setMinimumHeight(38)
506 |         btn_editar.setStyleSheet(ESTILO_BOTAO_SECUNDARIO)
507 |         btn_editar.setToolTip("Editar este alerta")
508 |         btn_editar.clicked.connect(lambda: self._editar(alerta, dialog))
509 |         botoes.addWidget(btn_editar)
510 |
511 |         btn_excluir = QPushButton("■ Excluir")
512 |         btn_excluir.setCursor(Qt.PointingHandCursor)
513 |         btn_excluir.setMinimumHeight(38)
514 |         btn_excluir.setStyleSheet(ESTILO_BOTAO_ERRO)
515 |         btn_excluir.setToolTip("Excluir este alerta (pode ser desfeito pela Lixeira)")
516 |         btn_excluir.clicked.connect(lambda: self._excluir(alerta, dialog))
517 |         botoes.addWidget(btn_excluir)
518 |
519 |     botoes.addStretch()
520 |
521 |     btn_fechar = QPushButton("■ Fechar")
522 |     btn_fechar.setCursor(Qt.PointingHandCursor)
523 |     btn_fechar.setMinimumHeight(38)
524 |     btn_fechar.setStyleSheet(ESTILO_BOTAO_FECHAR)
525 |     btn_fechar.setToolTip("Fechar janela")
526 |     btn_fechar.clicked.connect(dialog.accept)
527 |     botoes.addWidget(btn_fechar)
528 |
529 |     layout.addLayout(botoes)
530 |     dialog.exec()
531 |
532 | def _editar(self, alerta: Any, parent_dialog: QDialog) -> None:
533 |     dialog = QDialog(parent_dialog)
534 |     dialog.setWindowTitle("Editar Alerta")
535 |     dialog.setMinimumSize(520, 540)
536 |     dialog.setStyleSheet(ESTILO_DIALOG)
537 |     layout = QVBoxLayout(dialog)

```

```

538 |         layout.setSpacing(14)
539 |         layout.setContentsMargins(20, 20, 20, 20)
540 |
541 |         alerta_box, alerta_layout = group_box("Alerta")
542 |         form_alerta = QFormLayout()
543 |         form_alerta.setSpacing(8)
544 |
545 |         printer_combo = QComboBox()
546 |         configurar_combo(printer_combo)
547 |         cmp = printer_combo.completer()
548 |         if cmp:
549 |             cmp.setFilterMode(Qt.MatchFlag.MatchContains)
550 |             cmp.setCaseSensitivity(Qt.CaseSensitivity.CaseInsensitive)
551 |         impressoras = self.printer_service.listar_todos()
552 |         printer_combo.addItem("(nenhuma impressora)", None)
553 |         idx_selecionado = 0
554 |         for i, p in enumerate(impressoras):
555 |             printer_combo.addItem(f"{p.patrimonio} - {p.modelo}", p.id)
556 |             if p.id == alerta.printer_id:
557 |                 idx_selecionado = i + 1
558 |         printer_combo.setCurrentIndex(idx_selecionado)
559 |         form_alerta.addRow("Impressora:", printer_combo)
560 |
561 |         part_combo = QComboBox()
562 |         configurar_combo(part_combo)
563 |         cmp = part_combo.completer()
564 |         if cmp:
565 |             cmp.setFilterMode(Qt.MatchFlag.MatchContains)
566 |             cmp.setCaseSensitivity(Qt.CaseSensitivity.CaseInsensitive)
567 |         part_combo.addItem("(nenhuma peça)", None)
568 |         idx_part = 0
569 |         if self.part_service:
570 |             partes = self.part_service.listar_todas()
571 |             for i, p in enumerate(partes):
572 |                 part_combo.addItem(f"{p.nome} ({p.codigo})", p.id)
573 |                 if p.id == alerta.part_id:
574 |                     idx_part = i + 1
575 |         part_combo.setCurrentIndex(idx_part)
576 |         form_alerta.addRow("Peça:", part_combo)
577 |
578 |         tipo_combo = QComboBox()
579 |         configurar_combo(tipo_combo)
580 |         cmp = tipo_combo.completer()
581 |         if cmp:
582 |             cmp.setFilterMode(Qt.MatchFlag.MatchContains)
583 |             cmp.setCaseSensitivity(Qt.CaseSensitivity.CaseInsensitive)
584 |         for t in TIPO_OPCOES:
585 |             tipo_combo.addItem(TIPO_LABELS.get(t, t), t)
586 |         tipo_combo.setCurrentIndex(TIPO_OPCOES.index(alerta.tipo) if alerta.tipo in TIPO_OPCOES else 0)
587 |         form_alerta.addRow("Tipo *:", tipo_combo)
588 |
589 |         titulo_input = QLineEdit()
590 |         titulo_input.setText(alerta.titulo or "")
591 |         titulo_input.setStyleSheet(ESTILO_INPUT)
592 |         form_alerta.addRow("Título *:", titulo_input)
593 |
594 |         erro_titulo = QLabel()
595 |         erro_titulo.setStyleSheet("color: #ef4444; font-size: 10px; background: transparent;")
596 |         erro_titulo.hide()
597 |         form_alerta.addRow("", erro_titulo)
598 |         ValidadorCampo(titulo_input, obrigatorio, erro_titulo)
599 |
600 |         alerta_layout.addLayout(form_alerta)

```

```

601 |         layout.addWidget(alerta_box)
602 |
603 |         detalhes_box, detalhes_layout = group_box("Detalhes")
604 |         form_detalhes = QFormLayout()
605 |         form_detalhes.setSpacing(8)
606 |
607 |         desc_input = QTextEdit()
608 |         desc_input.setPlainText(alerta.descricao or "")
609 |         desc_input.setMaximumHeight(80)
610 |         desc_input.setStyleSheet(ESTILO_INPUT)
611 |         form_detalhes.addRow("Descrição:", desc_input)
612 |
613 |         from PySide6.QtWidgets import QCheckBox, QDateEdit
614 |         chk_resolvido = QCheckBox("Alerta resolvido")
615 |         chk_resolvido.setChecked(alerta.resolvido)
616 |         chk_resolvido.setStyleSheet(
617 |             "QCheckBox { color: #e2e8f0; font-size: 12px; spacing: 8px; }"
618 |             " QCheckBox::indicator { width: 18px; height: 18px; border-radius: 4px;"
619 |             " border: 2px solid #475569; background: transparent; }"
620 |             " QCheckBox::indicator:checked { background-color: #a78bfa; border-color: #a78bfa; }"
621 |         )
622 |         form_detalhes.addRow("Status:", chk_resolvido)
623 |
624 |         data_agendada = QDateEdit()
625 |         data_agendada.setCalendarPopup(True)
626 |         data_agendada.setStyleSheet(ESTILO_INPUT)
627 |         data_agendada.setSpecialValueText("Sem data")
628 |         if alerta.data_agendada:
629 |             from datetime import datetime
630 |             data_agendada.setDate(data_agendada.date().fromPython(alerta.data_agendada.date()))
631 |         else:
632 |             data_agendada.setDate(data_agendada.minimumDate())
633 |         form_detalhes.addRow("Agendar para:", data_agendada)
634 |
635 |         detalhes_layout.addLayout(form_detalhes)
636 |         layout.addWidget(detalhes_box)
637 |
638 |         layout.addStretch()
639 |         botoes = QDialogButtonBox(QDialogButtonBox.Ok | QDialogButtonBox.Cancel)
640 |         botoes.setStyleSheet(
641 |             "QPushButton { border: none; border-radius: 6px; padding: 8px 16px; font-weight: bold; }"
642 |             " QPushButton[text='OK'] { background-color: #a78bfa; color: white; }"
643 |             " QPushButton[text='Cancel'] { background-color: #1f2937; color: #94a3b8; }"
644 |         )
645 |         btn_ok = botoes.button(QDialogButtonBox.Ok)
646 |         if btn_ok:
647 |             btn_ok.setToolTip("Salvar alterações do alerta")
648 |         btn_cancel = botoes.button(QDialogButtonBox.Cancel)
649 |         if btn_cancel:
650 |             btn_cancel.setToolTip("Cancelar")
651 |         botoes.accepted.connect(lambda: self._salvar_edicao(dialog, alerta, printer_combo, part_combo, tipo))
652 |         botoes.rejected.connect(dialog.reject)
653 |         layout.addWidget(botoes)
654 |         dialog.exec()
655 |
656 |     def _salvar_edicao(self, dialog: QDialog, alerta: Any, printer_combo: QComboBox, part_combo: QComboBox,
657 |                       titulo = titulo_input.text().strip())
658 |         if not titulo:
659 |             QMessageBox.warning(dialog, "Aviso", "Preencha o título!")
660 |             return
661 |
662 |         alerta.printer_id = printer_combo.currentData()
663 |         alerta.part_id = part_combo.currentData()

```

```

664 |         alerta.tipo = tipo_combo.currentData()
665 |         alerta.titulo = titulo
666 |         alerta.descricao = desc_input.toPlainText().strip()
667 |
668 |         data_ag = data_agendada.date().toPython() if data_agendada.date() != data_agendada.minimumDate() else None
669 |         from datetime import datetime, time
670 |         alerta.data_agendada = datetime.combine(data_ag, time(0, 0)) if data_ag else None
671 |
672 |         if chk_resolvido.isChecked() and not alerta.resolvido:
673 |             alerta.resolvido = True
674 |             alerta.resolvido_em = dt.now()
675 |         elif not chk_resolvido.isChecked() and alerta.resolvido:
676 |             alerta.resolvido = False
677 |             alerta.resolvido_em = None
678 |         from db import safe_commit
679 |         safe_commit(self.session)
680 |         agendados = self.alert_service.verificar_agendados()
681 |         for a in agendados:
682 |             ref = a.printer.patrimonio if a.printer else (a.part.nome if a.part else "")
683 |             ToastManager.avisar(f"■ {a.titulo}{' - ' + ref if ref else ''}", persistente=True)
684 |         ToastManager.atualizar_status(self.alert_service.contar_pendentes())
685 |         ToastManager.info(f"Alerta atualizado: {titulo}")
686 |         self.recarregar()
687 |         dialog.accept()
688 |         parent_dialog.accept()
689 |
690 |     def _excluir(self, alerta: Any, dialog: QDialog) -> None:
691 |         if ConfirmacaoDigitarDialog.confirmar(
692 |             "Excluir Alerta",
693 |             f"Deseja realmente excluir o alerta \"{alerta.titulo}\"?",
694 |             dialog,
695 |         ):
696 |             try:
697 |                 self.alert_service.excluir(alerta)
698 |                 ToastManager.atualizar_status(self.alert_service.contar_pendentes())
699 |                 ToastManager.mostrar(
700 |                     f"Alerta '{alerta.titulo}' excluído.",
701 |                     "aviso", duracao=8000,
702 |                     acao=("Desfazer", lambda o=alerta, svc=self.alert_service, pag=self: (svc.restaurar(o),
703 |                     ))
704 |                     self.recarregar()
705 |                     dialog.accept()
706 |             except Exception as e:
707 |                 QMessageBox.critical(dialog, "Erro", f"Erro ao excluir alerta:\n{e}")
708 |
709 |     def _resolver(self, alerta: Any, dialog: QDialog) -> None:
710 |         resp = QMessageBox.question(
711 |             dialog, "Resolver Alerta",
712 |             "Marcar este alerta como resolvido?",
713 |             QMessageBox.Yes | QMessageBox.No
714 |         )
715 |         if resp == QMessageBox.Yes:
716 |             with tratar_erro("resolver alerta"):
717 |                 self.alert_service.resolver(alerta)
718 |                 ToastManager.atualizar_status(self.alert_service.contar_pendentes())
719 |                 ToastManager.info(f"Alerta resolvido: {alerta.titulo}")
720 |                 self.recarregar()
721 |                 dialog.accept()

```

=== ARQUIVO: .lapp\views\pages\calendar_page.py ===

```

001 | from __future__ import annotations
002 |
003 | import logging
004 | from datetime import datetime
005 | from typing import Any
006 |
007 | from PySide6.QtCore import QDate, Qt, Signal
008 | from PySide6.QtGui import QColor, QMouseEvent, QPixmap
009 | from PySide6.QtWidgets import (
010 |     QFrame,
011 |     QGridLayout,
012 |     QHBoxLayout,
013 |     QHeaderView,
014 |     QLabel,
015 |     QPushButton,
016 |     QTableWidgetItem,
017 |     QTableWidgetItem,
018 |     QVBoxLayout,
019 |     QWidget,
020 |     QSizePolicy,
021 | )
022 |
023 | from config import BASE_DIR
024 | from app.models import Activity, Alert, Printer
025 | from app.services.maintenance_scheduler import MaintenanceScheduler
026 | from app.views.widgets.table_widget import tornar_interativa
027 |
028 | log = logging.getLogger(__name__)
029 |
030 | _DIAS_SEMANA = ["seg", "ter", "qua", "qui", "sex", "sáb", "dom"]
031 | _MESES_PT = [
032 |     "janeiro", "fevereiro", "março", "abril", "maio", "junho",
033 |     "julho", "agosto", "setembro", "outubro", "novembro", "dezembro",
034 | ]
035 |
036 | _STATUS_GRUPOS: dict[str, tuple[str, str, str]] = {
037 |     "Aberta": ("Em Aberto", "#60a5fa", "■"),
038 |     "Aguardando Peça": ("Em Aberto", "#60a5fa", "■"),
039 |     "Aguardando Aprovação": ("Em Aberto", "#60a5fa", "■"),
040 |     "Técnico Designado": ("Em Progresso", "#f59e0b", "■"),
041 |     "Em Deslocamento": ("Em Progresso", "#f59e0b", "■"),
042 |     "Em Manutenção": ("Em Progresso", "#f59e0b", "■"),
043 |     "Em Atendimento": ("Em Progresso", "#f59e0b", "■"),
044 |     "Concluído": ("Concluído", "#34d399", "■"),
045 |     "Verificada": ("Concluído", "#34d399", "■"),
046 | }
047 | _STATUS_AGENDADO = ("Em Aberto", "#60a5fa", "■")
048 | _STATUS_ATRASADO = ("Atrasada", "#f38ba8", "■")
049 |
050 |
051 | class _DayCell(QFrame):
052 |     def __init__(self, dia: int, mes_atual: bool, hoje: bool, selecionado: bool,
053 |                  cores_dot: list[str], parent: QWidget | None = None) -> None:
054 |         super().__init__(parent)
055 |         self._callback = None
056 |         self.setFixedSize(64, 44)
057 |         self.setCursor(Qt.PointingHandCursor)
058 |
059 |         cor_texto = "#3a3a4e" if not mes_atual else "#e2e8f0"
060 |         if selecionado:
061 |             cor_texto = "ffffff"
062 |
063 |         self.setObjectName("dayCell")

```



```

064 |
065 |         if selecionado:
066 |             bg = "background: rgba(99,102,241,0.35);"
067 |             borda = "1px solid #6366f1;"
068 |         elif hoje:
069 |             bg = "background: transparent;"
070 |             borda = "1.5px solid #6366f1;"
071 |         else:
072 |             bg = "background: transparent;"
073 |             borda = "1px solid transparent;"
074 |
075 |         self.setStyleSheet(
076 |             f"QFrame#dayCell {{ {bg} border: {borda} border-radius: 8px; }}"
077 |             f" QFrame#dayCell:hover {{ background: rgba(99,102,241,0.15); }}"
078 |         )
079 |
080 |         layout = QVBoxLayout(self)
081 |         layout.setContentsMargins(2, 4, 2, 2)
082 |         layout.setSpacing(2)
083 |         layout.setAlignment(Qt.AlignHCenter | Qt.AlignTop)
084 |
085 |         lbl_dia = QLabel(str(dia))
086 |         lbl_dia.setAlignment(Qt.AlignCenter)
087 |         peso = "700" if (hoje or selecionado) else "600"
088 |         lbl_dia.setStyleSheet(
089 |             f"color: {cor_texto}; font-size: 13px; font-weight: {peso}; background: transparent; border: no
090 |         )
091 |         layout.addWidget(lbl_dia)
092 |
093 |         if cores_dot:
094 |             dots_row = QHBoxLayout()
095 |             dots_row.setSpacing(3)
096 |             dots_row.setAlignment(Qt.AlignCenter)
097 |             for cor in cores_dot[:3]:
098 |                 dot = QLabel()
099 |                 dot.setFixedSize(5, 5)
100 |                 dot.setStyleSheet(f"background-color: {cor}; border-radius: 2px; border: none;")
101 |                 dots_row.addWidget(dot)
102 |             layout.addLayout(dots_row)
103 |
104 |     def set_click(self, callback) -> None:
105 |         self._callback = callback
106 |
107 |     def mousePressEvent(self, event: QMouseEvent) -> None:
108 |         if self._callback:
109 |             self._callback()
110 |         super().mousePressEvent(event)
111 |
112 |
113 | class CalendarPage(QWidget):
114 |     signal_trocar_pagina = Signal(str, object)
115 |     abrir_atividade = Signal(int)
116 |     abrir_impressora = Signal(str)
117 |
118 |     def __init__(
119 |         self,
120 |         scheduler: MaintenanceScheduler,
121 |         activity_service: Any = None,
122 |         technician_service: Any = None,
123 |         alert_service: Any = None,
124 |         printer_service: Any = None,
125 |         parent: QWidget | None = None,
126 |     ) -> None:

```

```

127 |         super().__init__(parent)
128 |         self.scheduler = scheduler
129 |         self.activity_service = activity_service
130 |         self.technician_service = technician_service
131 |         self.alert_service = alert_service
132 |         self.printer_service = printer_service
133 |
134 |         hoje = datetime.now()
135 |         self._mes_atual = QDate(hoje.year, hoje.month, 1)
136 |         self._data_selecionada = QDate(hoje.year, hoje.month, hoje.day)
137 |         self._eventos_por_data: dict[str, list[dict[str, Any]]] = {}
138 |         self._eventos_da_tabela: list[dict[str, Any]] = []
139 |
140 |         self._inicializar_ui()
141 |         self._recarregar_tudo()
142 |
143 |     # UI
144 |     def _inicializar_ui(self) -> None:
145 |         layout = QVBoxLayout(self)
146 |         layout.setContentsMargins(20, 20, 20, 20)
147 |         layout.setSpacing(12)
148 |
149 |         header = QHBoxLayout()
150 |         header.setContentsMargins(0, 0, 0, 10)
151 |         header.setSpacing(10)
152 |
153 |         # Ícone do Flaticon carregado via arquivo
154 |         icone_label = QLabel()
155 |         pixmap = QPixmap(str(BASE_DIR / "anexos" / "calendar_icon.png"))
156 |         if not pixmap.isNull():
157 |             pixmap_redimensionado = pixmap.scaled(28, 28, Qt.KeepAspectRatio, Qt.SmoothTransformation)
158 |             icone_label.setPixmap(pixmap_redimensionado)
159 |         icone_label.setStyleSheet("background: transparent; border: none;")
160 |
161 |         titulo = QLabel("Calendário de Manutenções")
162 |         titulo.setStyleSheet(
163 |             "font-size: 20px; font-weight: 700; color: #e8e8f0; background: transparent; border: none;"
164 |         )
165 |
166 |         header.addWidget(icone_label)
167 |         header.addWidget(titulo)
168 |         header.addStretch()
169 |
170 |         btn_hoje = QPushButton("Hoje")
171 |         btn_hoje.setCursor(Qt.PointingHandCursor)
172 |         btn_hoje.setStyleSheet(
173 |             "QPushButton { background: transparent; color: #e8e8f0; border: 1px solid #2a2a3e;"
174 |             " border-radius: 8px; padding: 6px 16px; font-size: 12px; font-weight: 600; }"
175 |             " QPushButton:hover { background: #1e1e2e; border-color: #6366f1; }"
176 |         )
177 |         btn_hoje.clicked.connect(self._ir_para_hoje)
178 |         header.addWidget(btn_hoje)
179 |         layout.addLayout(header)
180 |
181 |         # ■■ Card Calendário ■■
182 |         cal_card = QFrame()
183 |         cal_card.setObjectName("calCard")
184 |         cal_card.setStyleSheet(
185 |             "QFrame#calCard { background-color: #14141f; border: 1px solid #2a2a3e; border-radius: 14px; }"
186 |         )
187 |         cal_layout = QVBoxLayout(cal_card)
188 |         cal_layout.setContentsMargins(16, 12, 16, 14)
189 |         cal_layout.setSpacing(8)

```

```

190 |
191 |     nav_row = QHBoxLayout()
192 |     btn_prev = QPushButton("<")
193 |     btn_prev.setFixedSize(28, 28)
194 |     btn_prev.setCursor(Qt.PointingHandCursor)
195 |     btn_prev.setStyleSheet(self._estilo_seta())
196 |     btn_prev.clicked.connect(lambda: self._mudar_mes(-1))
197 |
198 |     self._lbl_mes = QLabel()
199 |     self._lbl_mes.setAlignment(Qt.AlignCenter)
200 |     self._lbl_mes.setStyleSheet(
201 |         "color: #e8e8f0; font-size: 15px; font-weight: 700; background: transparent; border: none;"
202 |     )
203 |
204 |     btn_next = QPushButton(">")
205 |     btn_next.setFixedSize(28, 28)
206 |     btn_next.setCursor(Qt.PointingHandCursor)
207 |     btn_next.setStyleSheet(self._estilo_seta())
208 |     btn_next.clicked.connect(lambda: self._mudar_mes(1))
209 |
210 |     nav_row.addStretch()
211 |     nav_row.addWidget(btn_prev)
212 |     nav_row.addWidget(self._lbl_mes)
213 |     nav_row.addWidget(btn_next)
214 |     nav_row.addStretch()
215 |     cal_layout.addLayout(nav_row)
216 |
217 |     semana_row = QHBoxLayout()
218 |     semana_row.setSpacing(0)
219 |     for i, dia in enumerate(_DIAS_SEMANA):
220 |         lbl = QLabel(dia)
221 |         lbl.setAlignment(Qt.AlignCenter)
222 |         cor = "#f38ba8" if i >= 5 else "#717182"
223 |         lbl.setStyleSheet(
224 |             f"color: {cor}; font-size: 10px; font-weight: 700; background: transparent; border: none;"
225 |         )
226 |         lbl.setFixedWidth(64)
227 |         semana_row.addWidget(lbl)
228 |     cal_layout.addLayout(semana_row)
229 |
230 |     self._grid = QGridLayout()
231 |     self._grid.setSpacing(3)
232 |     cal_layout.addLayout(self._grid)
233 |
234 |     layout.addWidget(cal_card)
235 |
236 |     # ■■ Área de Detalhes ■■
237 |     detalhes_card = QFrame()
238 |     detalhes_card.setObjectName("detalhesCard")
239 |     detalhes_card.setStyleSheet(
240 |         "QFrame#detalhesCard { background-color: #14141f; border: 1px solid #2a2a3e; border-radius: 12px;"
241 |     )
242 |     detalhes_layout = QVBoxLayout(detalhes_card)
243 |     detalhes_layout.setContentsMargins(16, 12, 16, 12)
244 |     detalhes_layout.setSpacing(8)
245 |
246 |     self._detalhes_label = QLabel("Selecione uma data para ver os eventos")
247 |     self._detalhes_label.setStyleSheet(
248 |         "font-size: 14px; font-weight: 600; color: #a6adc8; background: transparent; border: none;"
249 |     )
250 |     detalhes_layout.addWidget(self._detalhes_label)
251 |
252 |     self._tabela_eventos = QTableWidgetItem()

```

[illegible]

```

316 |         fim = datetime(ultimo_dia_grid.year(), ultimo_dia_grid.month(), ultimo_dia_grid.day(), 23, 59, 59)
317 |
318 |         self._eventos_por_data = {}
319 |
320 |         # ■■ Agendamentos avulsos (Printer.proxima_revisao) ■■
321 |         if self.printer_service:
322 |             try:
323 |                 printers = (
324 |                     self.printer_service.session.query(Printer)
325 |                     .filter(
326 |                         Printer.deleted_at.is_(None),
327 |                         Printer.proxima_revisao >= inicio,
328 |                         Printer.proxima_revisao <= fim,
329 |                     )
330 |                     .all()
331 |                 )
332 |                 for p in printers:
333 |                     if not p.proxima_revisao:
334 |                         continue
335 |                     chave = p.proxima_revisao.strftime("%Y-%m-%d")
336 |                     atrasado = p.proxima_revisao < datetime.now()
337 |                     status = _STATUS_ATRASADO if atrasado else _STATUS_AGENDADO
338 |                     self._eventos_por_data.setdefault(chave, []).append({
339 |                         "impressora": p.patrimonio,
340 |                         "tipo": "Agendamento",
341 |                         "previsao": p.proxima_revisao,
342 |                         "status_label": status[0],
343 |                         "status_cor": status[1],
344 |                         "status_icone": status[2],
345 |                         "observacao": p.urgencia_prox_manutencao or "-",
346 |                         "activity_id": None,
347 |                         "alert_id": None,
348 |                         "printer_patrimonio": p.patrimonio,
349 |                     })
350 |             except Exception:
351 |                 log.warning("Erro ao buscar agendamentos avulsos para calendário", exc_info=True)
352 |
353 |         # ■■ Agendamentos recorrentes ■■
354 |         for ev in self.scheduler.listar_eventos(inicio, fim):
355 |             data = ev["data"]
356 |             if not data:
357 |                 continue
358 |             atrasado = data < datetime.now()
359 |             status = _STATUS_ATRASADO if atrasado else _STATUS_AGENDADO
360 |             chave = data.strftime("%Y-%m-%d")
361 |             self._eventos_por_data.setdefault(chave, []).append({
362 |                 "impressora": ev["printer_patrimonio"],
363 |                 "tipo": "Preventiva" if ev["tipo"] == "periodo" else "Preventiva (páginas)",
364 |                 "previsao": data,
365 |                 "status_label": status[0],
366 |                 "status_cor": status[1],
367 |                 "status_icone": status[2],
368 |                 "observacao": ev.get("observacao") or "-",
369 |                 "activity_id": None,
370 |                 "alert_id": None,
371 |                 "printer_patrimonio": ev["printer_patrimonio"],
372 |             })
373 |
374 |         # ■■ OS de manutenção reais ■■
375 |         if self.activity_service:
376 |             try:
377 |                 atividades = (
378 |                     self.activity_service.session.query(Activity)

```

```

379 |         .filter(
380 |             Activity.deleted_at.is_(None),
381 |             Activity.kind == "MANUTENCAO",
382 |             Activity.event_at >= inicio,
383 |             Activity.event_at <= fim,
384 |         )
385 |         .all()
386 |     )
387 |     for a in atividades:
388 |         if not a.event_at:
389 |             continue
390 |         chave = a.event_at.strftime("%Y-%m-%d")
391 |         status = _STATUS_GRUPOS.get(a.status_atividade or "Aberta", _STATUS_AGENDADO)
392 |         pat = a.printer.patrimonio if a.printer else (a.printer_id or "-")
393 |
394 |         self._eventos_por_data.setdefault(chave, []).append({
395 |             "impressora": pat,
396 |             "tipo": "Corretiva",
397 |             "previsao": a.event_at,
398 |             "status_label": status[0],
399 |             "status_cor": status[1],
400 |             "status_icone": status[2],
401 |             "observacao": a.notes or "-",
402 |             "activity_id": a.id,
403 |             "alert_id": None,
404 |             "printer_patrimonio": pat,
405 |         })
406 |     except Exception:
407 |         log.warning("Erro ao buscar atividades para calendário", exc_info=True)
408 |
409 |     # ■■ Alertas agendados ■■
410 |     if self.alert_service:
411 |         try:
412 |             alertas = (
413 |                 self.alert_service.session.query(Alert)
414 |                 .filter(
415 |                     Alert.deleted_at.is_(None),
416 |                     Alert.resolvido == False,
417 |                     Alert.data_agendada >= inicio,
418 |                     Alert.data_agendada <= fim,
419 |                 )
420 |                 .all()
421 |             )
422 |             for al in alertas:
423 |                 if not al.data_agendada:
424 |                     continue
425 |                 chave = al.data_agendada.strftime("%Y-%m-%d")
426 |                 atrasado = al.data_agendada < datetime.now()
427 |                 status = _STATUS_ATRASADO if atrasado else _STATUS_AGENDADO
428 |                 pat = al.printer.patrimonio if al.printer else "-"
429 |
430 |                 self._eventos_por_data.setdefault(chave, []).append({
431 |                     "impressora": pat,
432 |                     "tipo": al.titulo or "Revisão",
433 |                     "previsao": al.data_agendada,
434 |                     "status_label": status[0],
435 |                     "status_cor": status[1],
436 |                     "status_icone": status[2],
437 |                     "observacao": al.descricao or "-",
438 |                     "activity_id": None,
439 |                     "alert_id": al.id,
440 |                     "printer_patrimonio": pat,
441 |                 })

```

```

442 |         except Exception:
443 |             log.warning("Erro ao buscar alertas para calendário", exc_info=True)
444 |
445 | def _montar_grid(self) -> None:
446 |     while self._grid.count():
447 |         item = self._grid.takeAt(0)
448 |         if item.widget():
449 |             item.widget().deleteLater()
450 |
451 |     self._lbl_mes.setText(
452 |         f"{_MESES_PT[self._mes_atual.month() - 1]} {self._mes_atual.year()}"
453 |     )
454 |
455 |     primeiro_dia_grid = self._mes_atual.addDays(-(self._mes_atual.dayOfWeek() - 1))
456 |     qhoje = QDate.currentDate()
457 |
458 |     cores_ordem = {"#f38ba8": 0, "#f59e0b": 1, "#60a5fa": 2, "#34d399": 3}
459 |
460 |     for i in range(42):
461 |         data = primeiro_dia_grid.addDays(i)
462 |         chave = data.toString("yyyy-MM-dd")
463 |         eventos = self._eventos_por_data.get(chave, [])
464 |         cores = sorted(
465 |             {ev["status_cor"] for ev in eventos},
466 |             key=lambda c: cores_ordem.get(c, 9),
467 |         )
468 |
469 |         cell = _DayCell(
470 |             dia=data.day(),
471 |             mes_atual=(data.month() == self._mes_atual.month()),
472 |             hoje=(data == qhoje),
473 |             selecionado=(data == self._data_selecionada),
474 |             cores_dot=cores,
475 |         )
476 |         cell.set_click(lambda d=data: self._selecionar_data(d))
477 |         self._grid.addWidget(cell, i // 7, i % 7)
478 |
479 | def _selecionar_data(self, data: QDate) -> None:
480 |     self._data_selecionada = data
481 |     self._montar_grid()
482 |     self._mostrar_eventos_do_dia(data)
483 |
484 | def _mostrar_eventos_do_dia(self, data: QDate) -> None:
485 |     chave = data.toString("yyyy-MM-dd")
486 |     eventos = self._eventos_por_data.get(chave, [])
487 |
488 |     if not eventos:
489 |         self._detalhes_label.setText(f"Nenhum evento em {data.toString('dd/MM/yyyy')}")
490 |         self._tabela_eventos.setRowCount(0)
491 |         self._eventos_da_tabela = []
492 |         return
493 |
494 |     self._detalhes_label.setText(
495 |         f"Manutenções para {data.toString('dd/MM/yyyy')} – {len(eventos)} evento(s)"
496 |     )
497 |     eventos = sorted(eventos, key=lambda e: e["previsao"])
498 |     self._eventos_da_tabela = eventos
499 |     self._tabela_eventos.setRowCount(len(eventos))
500 |
501 |     for i, ev in enumerate(eventos):
502 |         self._tabela_eventos.setItem(i, 0, self._item(ev["impressora"]))
503 |         self._tabela_eventos.setItem(i, 1, self._item(ev["tipo"]))
504 |         self._tabela_eventos.setItem(i, 2, self._item(ev["previsao"].strftime("%H:%M")))

```

```

505 |
506 |         status_item = QTableWidgetItem(f"{ev['status_icone']} {ev['status_label']}")
507 |         status_item.setTextAlignment(Qt.AlignCenter)
508 |         status_item.setForeground(QColor(ev["status_cor"]))
509 |         self._tabela_eventos.setItem(i, 3, status_item)
510 |
511 |         obs_item = QTableWidgetItem(ev["observacao"])
512 |         self._tabela_eventos.setItem(i, 4, obs_item)
513 |
514 |         # Removido o resizeColumnsToContents antigo que forçava encolhimento,
515 |         # mantendo o Stretch ativo para preencher a largura de ponta a ponta.
516 |
517 |     def _ao_duplo_clique(self) -> None:
518 |         row = self._tabela_eventos.currentRow()
519 |         if row < 0 or row >= len(self._eventos_da_tabela):
520 |             return
521 |         ev = self._eventos_da_tabela[row]
522 |         try:
523 |             if ev.get("activity_id"):
524 |                 self.abrir_atividade.emit(ev["activity_id"])
525 |             elif ev.get("printer_patrimonio") and ev["printer_patrimonio"] != "-":
526 |                 self.abrir_impressora.emit(ev["printer_patrimonio"])
527 |         except Exception:
528 |             log.warning("Erro ao navegar a partir do calendário", exc_info=True)
529 |
530 |     def _item(self, texto: str) -> QTableWidgetItem:
531 |         item = QTableWidgetItem(str(texto))
532 |         item.setTextAlignment(Qt.AlignCenter)
533 |         return item

```

=== ARQUIVO: .lapp\views\pages\clients_page.py ===

```

001 | from __future__ import annotations
002 |
003 | from typing import Any
004 |
005 | from PySide6.QtCore import Qt, Signal
006 | from PySide6.QtGui import QColor
007 | from PySide6.QtWidgets import (
008 |     QAbstractItemView,
009 |     QComboBox,
010 |     QDialog,
011 |     QDialogButtonBox,
012 |     QFormLayout,
013 |     QHBoxLayout,
014 |     QHeaderView,
015 |     QLabel,
016 |     QLineEdit,
017 |     QMessageBox,
018 |     QPushButton,
019 |     QScrollArea,
020 |     QTableWidgetItem,
021 |     QTableWidgetItem,
022 |     QTabWidget,
023 |     QTextEdit,
024 |     QVBoxLayout,
025 |     QWidget,
026 | )
027 |
028 | from app.utils.ui_helpers import tratar_erro
029 | from app.utils.validacao import ValidadorCampo, email_opcional, obrigatorio

```



```

030 | from app.views.styles.theme import (
031 |     COR,
032 |     ESTILO_BOTAO_AVISO,
033 |     ESTILO_BOTAO_ERRO,
034 |     ESTILO_BOTAO_FECHAR,
035 |     ESTILO_BOTAO_PRIMARIO,
036 |     ESTILO_BOTAO_SECUNDARIO,
037 |     ESTILO_BOTAO_SUCESSO,
038 |     ESTILO_SUBTITULO,
039 |     configurar_combo,
040 |     configurar_combo_colorido,
041 |     ESTILO_COMBO,
042 |     ESTILO_DIALOG,
043 |     ESTILO_INPUT,
044 |     ESTILO_LABEL_CAMPO,
045 |     ESTILO_TABELA_SIMPLES,
046 |     ESTILO_TITULO_PAGINA,
047 |     TIPO_CLIENTE_CORES,
048 |     group_box,
049 |     input_label,
050 |     campo_rotulo,
051 |     campo_readonly,
052 | )
053 | from app.views.widgets import ToastManager
054 | from app.views.widgets.confirm_dialog import ConfirmacaoDigitarDialog
055 | from app.views.widgets.import_dialog import ImportDialog
056 | from app.views.widgets.pagination import PaginacaoWidget
057 | from app.views.widgets.table_widget import TabelaPadrao, tornar_interativa
058 |
059 |
060 | class ClientsPage(QWidget):
061 |     abrir_impressora: Signal = Signal(str)
062 |
063 |     session: Any
064 |     company_service: Any
065 |     printer_service: Any
066 |     _empresas_visiveis: list[Any]
067 |     tabela: TabelaPadrao
068 |     _paginacao: PaginacaoWidget
069 |
070 |     def __init__(self, session: Any, company_service: Any, printer_service: Any, parent: QWidget | None = None):
071 |         super().__init__(parent)
072 |         self.session = session
073 |         self.company_service = company_service
074 |         self.printer_service = printer_service
075 |         self._init_ui()
076 |
077 |     def _init_ui(self) -> None:
078 |         layout = QVBoxLayout(self)
079 |         layout.setContentsMargins(24, 24, 24, 24)
080 |         layout.setSpacing(16)
081 |
082 |         header = QHBoxLayout()
083 |         titulo = QLabel("\u0001f465 Clientes / Empresas")
084 |         titulo.setStyleSheet(ESTILO_TITULO_PAGINA)
085 |         header.addWidget(titulo)
086 |         header.addStretch()
087 |
088 |         btn_nova = QPushButton("\u2795 Nova Empresa")
089 |         btn_nova.setToolTip("Cadastrar nova empresa")
090 |         btn_nova.setCursor(Qt.PointingHandCursor)
091 |         btn_nova.setStyleSheet(ESTILO_BOTAO_PRIMARIO)
092 |         btn_nova.clicked.connect(self._nova)

```

```

093 |         header.addWidget(btn_nova)
094 |
095 |         btn_importar = QPushButton(" Importar")
096 |         btn_importar.setToolTip("Importar empresas de arquivo CSV ou XLSX")
097 |         btn_importar.setCursor(Qt.PointingHandCursor)
098 |         btn_importar.setStyleSheet(ESTILO_BOTAO_SECUNDARIO)
099 |         btn_importar.clicked.connect(lambda: self._importar())
100 |         header.addWidget(btn_importar)
101 |
102 |         layout.addLayout(header)
103 |
104 |         subtitulo = QLabel("Gerencie as empresas e clientes")
105 |         subtitulo.setStyleSheet(ESTILO_SUBTITULO)
106 |         layout.addWidget(subtitulo)
107 |
108 |         self._empresas_visiveis = []
109 |         self.tabela = TabelaPadrao(["Nome", "CNPJ", "Cidade/UF", "Telefone", "Email", "Impressoras"])
110 |         self.tabela.cellDoubleClicked.connect(self._detalhes)
111 |         layout.addWidget(self.tabela)
112 |
113 |         self._paginacao = PaginacaoWidget()
114 |         self._paginacao.pagina_alterada.connect(lambda p: self._carregar())
115 |         layout.addWidget(self._paginacao)
116 |
117 |         self._carregar()
118 |
119 |     def recarregar(self) -> None:
120 |         self._carregar()
121 |
122 |     def _carregar(self) -> None:
123 |         empresas = self.company_service.listar_todas(
124 |             limite=self._paginacao.limit, offset=self._paginacao.offset
125 |         )
126 |         total = self.company_service.contar_todas()
127 |         self._paginacao.configurar(total, pagina_atual=self._paginacao.pagina,
128 |             itens_por_pagina=self._paginacao.limit)
129 |         self._empresas_visiveis = empresas
130 |         self.tabela.setRowCount(len(empresas))
131 |         nomes_locais = [emp.nome for emp in empresas]
132 |         contagem = self.printer_service.contar_por_locais(nomes_locais)
133 |
134 |         for i, emp in enumerate(empresas):
135 |             self.tabela.setItem(i, 0, QTableWidgetItem(emp.nome))
136 |             self.tabela.setItem(i, 1, QTableWidgetItem(emp.cnpj or "-"))
137 |             cidade_uf = f"{emp.cidade or ''}/{emp.uf or ''}".strip("/")
138 |             self.tabela.setItem(i, 2, QTableWidgetItem(cidade_uf if cidade_uf else "-"))
139 |             self.tabela.setItem(i, 3, QTableWidgetItem(emp.telefone or "-"))
140 |             self.tabela.setItem(i, 4, QTableWidgetItem(emp.email or "-"))
141 |             self.tabela.setItem(i, 5, QTableWidgetItem(str(contagem.get(emp.nome, 0))))
142 |         self.tabela.redimensionar()
143 |
144 |     def _detalhes(self, row: int) -> None:
145 |         nome_empresa = self.tabela.item(row, 0).text()
146 |         empresa = self.company_service.buscar_por_nome(nome_empresa)
147 |         if not empresa:
148 |             return
149 |
150 |         dialog = QDialog(self)
151 |         dialog.setWindowTitle(empresa.nome)
152 |         dialog.setMinimumSize(650, 500)
153 |         dialog.setStyleSheet(ESTILO_DIALOG + ESTILO_TABELA_SIMPLES)
154 |
155 |         tabs = QTabWidget()

```

```

156 |
157 | # ■■ Tab Dados (read-only) ■■
158 | tab_dados = QWidget()
159 | tab_dados_layout = QVBoxLayout(tab_dados)
160 | tab_dados_layout.setContentsMargins(0, 0, 0, 0)
161 |
162 | scroll = QScrollArea()
163 | scroll.setWidgetResizable(True)
164 | scroll.setHorizontalScrollBarPolicy(Qt.ScrollBarAlwaysOff)
165 | scroll.setStyleSheet("QScrollArea { border: none; background: transparent; }")
166 |
167 | container = QWidget()
168 | container.setStyleSheet("background: transparent;")
169 | content = QVBoxLayout(container)
170 | content.setContentsMargins(20, 16, 20, 16)
171 | content.setSpacing(16)
172 |
173 | # Identificação
174 | id_box, id_layout = group_box("Identificação")
175 | id_form = QFormLayout()
176 | id_form.setSpacing(8)
177 | id_form.addRow(campo_rotulo("Nome"), campo_readonly(empresa.nome))
178 | id_form.addRow(campo_rotulo("CNPJ"), campo_readonly(empresa.cnpj or "-"))
179 | id_form.addRow(campo_rotulo("Tipo"), campo_readonly(empresa.tipo or "-"))
180 | id_layout.addLayout(id_form)
181 | content.addWidget(id_box)
182 |
183 | # Contato
184 | cont_box, cont_layout = group_box("Contato")
185 | cont_form = QFormLayout()
186 | cont_form.setSpacing(8)
187 | cont_form.addRow(campo_rotulo("Telefone"), campo_readonly(empresa.telefone or "-"))
188 | cont_form.addRow(campo_rotulo("Email"), campo_readonly(empresa.email or "-"))
189 | cont_layout.addLayout(cont_form)
190 | content.addWidget(cont_box)
191 |
192 | # Endereço
193 | end_box, end_layout = group_box("Endereço")
194 | end_form = QFormLayout()
195 | end_form.setSpacing(8)
196 | end_form.addRow(campo_rotulo("Endereço"), campo_readonly(empresa.endereco or "-"))
197 | end_form.addRow(campo_rotulo("Cidade"), campo_readonly(empresa.cidade or "-"))
198 | end_form.addRow(campo_rotulo("UF"), campo_readonly(empresa.uf or "-"))
199 | end_layout.addLayout(end_form)
200 | content.addWidget(end_box)
201 |
202 | # Observações
203 | obs_box, obs_layout = group_box("Observações")
204 | obs_layout.addWidget(campo_readonly(empresa.observacao or "-"))
205 | content.addWidget(obs_box)
206 |
207 | content.addStretch()
208 | scroll.setWidget(container)
209 | tab_dados_layout.addWidget(scroll)
210 |
211 | # Buttons at bottom of Dados tab
212 | btn_layout = QHBoxLayout()
213 | btn_layout.setContentsMargins(20, 8, 20, 12)
214 | btn_layout.setSpacing(10)
215 |
216 | btn_editar = QPushButton("■ Editar")
217 | btn_editar.setCursor(Qt.PointingHandCursor)
218 | btn_editar.setStyleSheet(ESTILO_BOTA0_AVISO)

```

```

219 |         btn_editar.clicked.connect(lambda: (dialog.accept(), self._editar(row)))
220 |         btn_layout.addWidget(btn_editar)
221 |
222 |         btn_excluir = QPushButton("■ Excluir")
223 |         btn_excluir.setCursor(Qt.PointingHandCursor)
224 |         btn_excluir.setStyleSheet(ESTILO_BOTAO_ERRO)
225 |         btn_excluir.clicked.connect(lambda: self._excluir(dialog, empresa))
226 |         btn_layout.addWidget(btn_excluir)
227 |
228 |         btn_fechar = QPushButton("Fechar")
229 |         btn_fechar.setCursor(Qt.PointingHandCursor)
230 |         btn_fechar.setStyleSheet(ESTILO_BOTAO_FECHAR)
231 |         btn_fechar.clicked.connect(dialog.accept)
232 |         btn_layout.addStretch()
233 |         btn_layout.addWidget(btn_fechar)
234 |
235 |         tab_dados_layout.addLayout(btn_layout)
236 |
237 |         tabs.addTab(tab_dados, "\U0001f4cb Dados")
238 |
239 |         # ■ Tab Impressoras (read-only, same as _editar) ■
240 |         tab_impressoras = QWidget()
241 |         imp_layout = QVBoxLayout(tab_impressoras)
242 |
243 |         impressoras = self.printer_service.listar_por_local(empresa.nome)
244 |
245 |         imp_label = QLabel(f"\U0001f5a8\ufe0f Impressoras em '{empresa.nome}' ({len(impressoras)})")
246 |         imp_label.setStyleSheet(ESTILO_LABEL_CAMPO.replace("font-size: 12px;", "font-size: 14px;"))
247 |         imp_layout.addWidget(imp_label)
248 |
249 |         if impressoras:
250 |             imp_tabela = QTableWidget()
251 |             imp_tabela.setColumnCount(5)
252 |             imp_tabela.setHorizontalHeaderLabels(["Patrimônio", "Modelo", "Serial", "Status", "Última Revisão"])
253 |             imp_tabela.setRowCount(len(impressoras))
254 |             imp_tabela.setStyleSheet(ESTILO_TABELA_SIMPLES)
255 |             imp_tabela.setSelectionBehavior(QAbstractItemView.SelectRows)
256 |             imp_tabela.setEditTriggers(QAbstractItemView.NoEditTriggers)
257 |             imp_tabela.verticalHeader().setVisible(False)
258 |
259 |             for i, p in enumerate(impressoras):
260 |                 imp_tabela.setItem(i, 0, QTableWidgetItem(p.patrimonio))
261 |                 imp_tabela.setItem(i, 1, QTableWidgetItem(p.modelo))
262 |                 imp_tabela.setItem(i, 2, QTableWidgetItem(p.serial or "-"))
263 |
264 |                 status_item = QTableWidgetItem(p.status)
265 |                 if p.status in ["Operacional", "Em uso"]:
266 |                     status_item.setForeground(QColor(COR["sucesso"]))
267 |                 elif p.status in ["Em manutenção", "Manutenção", "Aguardando peça"]:
268 |                     status_item.setForeground(QColor(COR["aviso"]))
269 |                 elif p.status in ["Parada", "Sucata"]:
270 |                     status_item.setForeground(QColor(COR["erro"]))
271 |                 imp_tabela.setItem(i, 3, status_item)
272 |
273 |                 rev_text = p.proxima_revisao.strftime("%d/%m/%Y") if p.proxima_revisao else "-"
274 |                 imp_tabela.setItem(i, 4, QTableWidgetItem(rev_text))
275 |
276 |             imp_tabela.verticalHeader().setDefaultSectionSize(44)
277 |             tornar_interativa(imp_tabela)
278 |
279 |             imp_tabela.cellDoubleClicked.connect(lambda r, c: self._abrir_impressora_por_patrimonio(
280 |                 imp_tabela.item(r, 0).text(), dialog
281 |             ))

```

```

282 |
283 |         imp_layout.addWidget(imp_tabela)
284 |     else:
285 |         sem = QLabel("Nenhuma impressora vinculada a esta empresa.")
286 |         sem.setStyleSheet("color: #94949f; font-size: 13px; padding: 30px;")
287 |         sem.setAlignment(Qt.AlignCenter)
288 |         imp_layout.addWidget(sem)
289 |
290 |     tabs.addTab(tab_impressoras, "\U0001f5a8\ufe0f Impressoras")
291 |
292 |     main_layout = QVBoxLayout(dialog)
293 |     main_layout.addWidget(tabs)
294 |     dialog.exec()
295 |
296 | def _nova(self) -> None:
297 |     dialog = QDialog(self)
298 |     dialog.setWindowTitle("Nova Empresa")
299 |     dialog.setMinimumSize(600, 500)
300 |     dialog.setStyleSheet(ESTILO_DIALOG + ESTILO_INPUT + ESTILO_COMBO)
301 |
302 |     layout = QVBoxLayout(dialog)
303 |     layout.setContentsMargins(0, 0, 0, 0)
304 |
305 |     scroll = QScrollArea()
306 |     scroll.setWidgetResizable(True)
307 |     scroll.setHorizontalScrollBarPolicy(Qt.ScrollBarAlwaysOff)
308 |     scroll.setStyleSheet("QScrollArea { border: none; background: transparent; }")
309 |
310 |     container = QWidget()
311 |     container.setStyleSheet("background: transparent;")
312 |     content = QVBoxLayout(container)
313 |     content.setContentsMargins(16, 8, 16, 8)
314 |     content.setSpacing(16)
315 |
316 |     id_box, id_layout = group_box("Identificação")
317 |     id_form = QFormLayout()
318 |     id_form.setSpacing(12)
319 |
320 |     nome_input = QLineEdit()
321 |     nome_input.setPlaceholderText("Nome da empresa")
322 |     nome_input.setMaxLength(150)
323 |     id_form.addRow("Nome:", nome_input)
324 |
325 |     erro_nome = QLabel()
326 |     erro_nome.setStyleSheet("color: #ef4444; font-size: 10px; background: transparent;")
327 |     erro_nome.hide()
328 |     id_form.addRow("", erro_nome)
329 |     ValidadorCampo(nome_input, obrigatorio, erro_nome)
330 |
331 |     cnpj_input = QLineEdit()
332 |     cnpj_input.setPlaceholderText("00.000.000/0000-00")
333 |     cnpj_input.setMaxLength(18)
334 |     id_form.addRow("CNPJ:", cnpj_input)
335 |
336 |     tipo_input = QComboBox()
337 |     configurar_combo(tipo_input)
338 |     cmp = tipo_input.completer()
339 |     if cmp:
340 |         cmp.setFilterMode(Qt.MatchFlag.MatchContains)
341 |         cmp.setCaseSensitivity(Qt.CaseSensitivity.CaseInsensitive)
342 |         tipo_input.addItem("Cliente", "Filial", "Parceiro"])
343 |         configurar_combo_colorido(tipo_input, TIPO_CLIENTE_CORES)
344 |         id_form.addRow("Tipo:", tipo_input)

```

```

345 |
346 |     id_layout.addLayout(id_form)
347 |     content.addWidget(id_box)
348 |
349 |     cont_box, cont_layout = group_box("Contato")
350 |     cont_form = QFormLayout()
351 |     cont_form.setSpacing(12)
352 |
353 |     telefone_input = QLineEdit()
354 |     telefone_input.setPlaceholderText("(00) 00000-0000")
355 |     telefone_input.setMaxLength(20)
356 |     cont_form.addRow("Telefone:", telefone_input)
357 |
358 |     email_input = QLineEdit()
359 |     email_input.setPlaceholderText("email@empresa.com")
360 |     email_input.setMaxLength(120)
361 |     cont_form.addRow("Email:", email_input)
362 |
363 |     erro_email = QLabel()
364 |     erro_email.setStyleSheet("color: #ef4444; font-size: 10px; background: transparent;")
365 |     erro_email.hide()
366 |     cont_form.addRow("", erro_email)
367 |     ValidadorCampo(email_input, email_opcional, erro_email)
368 |
369 |     cont_layout.addLayout(cont_form)
370 |     content.addWidget(cont_box)
371 |
372 |     end_box, end_layout = group_box("Endereço")
373 |     end_form = QFormLayout()
374 |     end_form.setSpacing(12)
375 |
376 |     endereco_input = QLineEdit()
377 |     endereco_input.setPlaceholderText("Rua/Av, número")
378 |     endereco_input.setMaxLength(255)
379 |     end_form.addRow("Endereço:", endereco_input)
380 |
381 |     cidade_input = QLineEdit()
382 |     cidade_input.setPlaceholderText("Cidade")
383 |     cidade_input.setMaxLength(100)
384 |     end_form.addRow("Cidade:", cidade_input)
385 |
386 |     uf_input = QLineEdit()
387 |     uf_input.setPlaceholderText("UF")
388 |     uf_input.setMaxLength(2)
389 |     end_form.addRow("UF:", uf_input)
390 |
391 |     end_layout.addLayout(end_form)
392 |     content.addWidget(end_box)
393 |
394 |     obs_box, obs_lay = group_box("Observações")
395 |     obs_input = QTextEdit()
396 |     obs_input.setPlaceholderText("Observações sobre a empresa...")
397 |     obs_input.setStyleSheet(ESTILO_INPUT)
398 |     obs_input.setMaximumHeight(100)
399 |     obs_lay.addWidget(input_label("Observações"))
400 |     obs_lay.addWidget(obs_input)
401 |     content.addWidget(obs_box)
402 |
403 |     content.addStretch()
404 |     scroll.setWidget(container)
405 |     layout.addWidget(scroll)
406 |
407 |     btn_layout = QHBoxLayout()

```

```

408 |         btn_layout.setSpacing(10)
409 |
410 |         btn_salvar = QPushButton("■ Salvar")
411 |         btn_salvar.setToolTip("Salvar nova empresa")
412 |         btn_salvar.setCursor(Qt.PointingHandCursor)
413 |         btn_salvar.setStyleSheet(ESTILO_BOTAO_SUCESSO)
414 |         btn_salvar.clicked.connect(lambda: self._salvar_nova(dialog, nome_input, cnpj_input, telefone_input))
415 |
416 |         btn_cancelar = QPushButton("Cancelar")
417 |         btn_cancelar.setToolTip("Descartar alterações e fechar")
418 |         btn_cancelar.setCursor(Qt.PointingHandCursor)
419 |         btn_cancelar.setStyleSheet(ESTILO_BOTAO_FECHAR)
420 |         btn_cancelar.clicked.connect(dialog.reject)
421 |
422 |         btn_layout.addStretch()
423 |         btn_layout.addWidget(btn_salvar)
424 |         btn_layout.addWidget(btn_cancelar)
425 |         layout.addLayout(btn_layout)
426 |
427 |         dialog.exec()
428 |
429 |     def _salvar_nova(self, dialog: QDialog, nome: QLineEdit, cnpj: QLineEdit, telefone: QLineEdit, email: QLineEdit):
430 |         nome_text = nome.text().strip()
431 |         if not nome_text:
432 |             return
433 |         with tratar_erro("criar empresa"):
434 |             self.company_service.criar(
435 |                 nome=nome_text,
436 |                 cnpj=cnpj.text().strip(),
437 |                 telefone=telefone.text().strip(),
438 |                 email=email.text().strip(),
439 |                 tipo=tipo.currentText(),
440 |                 endereco=endereco.text().strip(),
441 |                 cidade=cidade.text().strip(),
442 |                 uf=uf.text().strip().upper(),
443 |                 observacao=obs.toPlainText().strip()
444 |             )
445 |             self.recarregar()
446 |             dialog.accept()
447 |
448 |     def _editar(self, row: int) -> None:
449 |         nome_empresa = self.tabela.item(row, 0).text()
450 |         empresa = self.company_service.buscar_por_nome(nome_empresa)
451 |
452 |         if not empresa:
453 |             return
454 |
455 |         dialog = QDialog(self)
456 |         dialog.setWindowTitle(f"✎ Editar Empresa: {empresa.nome}")
457 |         dialog.setMinimumSize(600, 500)
458 |         dialog.setStyleSheet(ESTILO_DIALOG + ESTILO_INPUT + ESTILO_COMBO + ESTILO_TABELA_SIMPLES)
459 |
460 |         tabs = QTabWidget()
461 |
462 |         tab_dados = QWidget()
463 |         outer_layout = QVBoxLayout(tab_dados)
464 |         outer_layout.setContentsMargins(0, 0, 0, 0)
465 |
466 |         scroll = QScrollArea()
467 |         scroll.setWidgetResizable(True)
468 |         scroll.setHorizontalScrollBarPolicy(Qt.ScrollBarAlwaysOff)
469 |         scroll.setStyleSheet("QScrollArea { border: none; background: transparent; }")
470 |

```

```

471 |         container = QWidget()
472 |         container.setStyleSheet("background: transparent;")
473 |         content = QVBoxLayout(container)
474 |         content.setContentsMargins(16, 8, 16, 8)
475 |         content.setSpacing(16)
476 |
477 |         id_box, id_layout = group_box("Identificação")
478 |         id_form = QFormLayout()
479 |         id_form.setSpacing(12)
480 |
481 |         nome_input = QLineEdit(empresa.nome or "")
482 |         nome_input.setMaxLength(150)
483 |         id_form.addRow("Nome:", nome_input)
484 |
485 |         erro_nome = QLabel()
486 |         erro_nome.setStyleSheet("color: #ef4444; font-size: 10px; background: transparent;")
487 |         erro_nome.hide()
488 |         id_form.addRow("", erro_nome)
489 |         ValidadorCampo(nome_input, obrigatorio, erro_nome)
490 |
491 |         cnpj_input = QLineEdit(empresa.cnpj or "")
492 |         cnpj_input.setPlaceholderText("00.000.000/0000-00")
493 |         cnpj_input.setMaxLength(18)
494 |         id_form.addRow("CNPJ:", cnpj_input)
495 |
496 |         tipo_combo = QComboBox()
497 |         configurar_combo(tipo_combo)
498 |         cmp = tipo_combo.completer()
499 |         if cmp:
500 |             cmp.setFilterMode(Qt.MatchFlag.MatchContains)
501 |             cmp.setCaseSensitivity(Qt.CaseSensitivity.CaseInsensitive)
502 |             tipo_combo.addItem("Cliente", "Filial", "Parceiro")
503 |             configurar_combo_colorido(tipo_combo, TIPO_CLIENTE_CORES)
504 |             tipo_combo.setCurrentText(empresa.tipo or "Cliente")
505 |             id_form.addRow("Tipo:", tipo_combo)
506 |
507 |         id_layout.addLayout(id_form)
508 |         content.addWidget(id_box)
509 |
510 |         cont_box, cont_layout = group_box("Contato")
511 |         cont_form = QFormLayout()
512 |         cont_form.setSpacing(12)
513 |
514 |         tel_input = QLineEdit(empresa.telefone or "")
515 |         tel_input.setPlaceholderText("(00) 00000-0000")
516 |         tel_input.setMaxLength(20)
517 |         cont_form.addRow("Telefone:", tel_input)
518 |
519 |         email_input = QLineEdit(empresa.email or "")
520 |         email_input.setPlaceholderText("email@empresa.com")
521 |         email_input.setMaxLength(120)
522 |         cont_form.addRow("Email:", email_input)
523 |
524 |         erro_email = QLabel()
525 |         erro_email.setStyleSheet("color: #ef4444; font-size: 10px; background: transparent;")
526 |         erro_email.hide()
527 |         cont_form.addRow("", erro_email)
528 |         ValidadorCampo(email_input, email_opcional, erro_email)
529 |
530 |         cont_layout.addLayout(cont_form)
531 |         content.addWidget(cont_box)
532 |
533 |         end_box, end_layout = group_box("Endereço")

```



```

534 |         end_form = QFormLayout()
535 |         end_form.setSpacing(12)
536 |
537 |         end_input = QLineEdit(empresa.endereco or "")
538 |         end_input.setPlaceholderText("Rua/Av, n\u00f3famer0")
539 |         end_input.setMaxLength(255)
540 |         end_form.addRow("Endere\u00e7o:", end_input)
541 |
542 |         cidade_input = QLineEdit(empresa.cidade or "")
543 |         cidade_input.setPlaceholderText("Cidade")
544 |         cidade_input.setMaxLength(100)
545 |         end_form.addRow("Cidade:", cidade_input)
546 |
547 |         uf_input = QLineEdit(empresa.uf or "")
548 |         uf_input.setPlaceholderText("UF")
549 |         uf_input.setMaxLength(2)
550 |         end_form.addRow("UF:", uf_input)
551 |
552 |         end_layout.addLayout(end_form)
553 |         content.addWidget(end_box)
554 |
555 |         obs_box, obs_layout = group_box("Observa\u00e7\u00f5es")
556 |         obs_input = QTextEdit()
557 |         obs_input.setMaximumHeight(100)
558 |         obs_input.setStyleSheet(ESTILO_INPUT)
559 |         obs_input.setPlainText(empresa.observacao or "")
560 |         obs_layout.addWidget(input_label("Observa\u00e7\u00f5es"))
561 |         obs_layout.addWidget(obs_input)
562 |         content.addWidget(obs_box)
563 |
564 |         btn_form_layout = QHBoxLayout()
565 |         btn_form_layout.setSpacing(10)
566 |
567 |         btn_salvar = QPushButton("\u0001f4be Salvar")
568 |         btn_salvar.setToolTip("Salvar altera\u00e7\u00f5es da empresa")
569 |         btn_salvar.setCursor(Qt.PointingHandCursor)
570 |         btn_salvar.setStyleSheet(ESTILO_BOTAO_SUCESSO)
571 |         btn_salvar.clicked.connect(lambda: self._salvar_edicao(
572 |             dialog, empresa, nome_input, cnpj_input, tipo_combo,
573 |             tel_input, email_input, end_input, cidade_input, uf_input, obs_input
574 |         ))
575 |         btn_form_layout.addWidget(btn_salvar)
576 |
577 |         btn_excluir = QPushButton("\u0001f5d1\u00f0 Excluir")
578 |         btn_excluir.setToolTip("Excluir esta empresa (pode ser desfeito pela Lixeira)")
579 |         btn_excluir.setCursor(Qt.PointingHandCursor)
580 |         btn_excluir.setStyleSheet(ESTILO_BOTAO_ERRO)
581 |         btn_excluir.clicked.connect(lambda: self._excluir(dialog, empresa))
582 |         btn_form_layout.addWidget(btn_excluir)
583 |
584 |         btn_cancelar = QPushButton("Cancelar")
585 |         btn_cancelar.setToolTip("Descartar altera\u00e7\u00f5es e fechar")
586 |         btn_cancelar.setCursor(Qt.PointingHandCursor)
587 |         btn_cancelar.setStyleSheet(ESTILO_BOTAO_FECHAR)
588 |         btn_cancelar.clicked.connect(dialog.reject)
589 |         btn_form_layout.addWidget(btn_cancelar)
590 |
591 |         content.addLayout(btn_form_layout)
592 |         content.addStretch()
593 |         scroll.setWidget(container)
594 |         outer_layout.addWidget(scroll)
595 |
596 |         tabs.addTab(tab_dados, "\u0001f4cb Dados")

```

```

597 |
598 |     tab_impressoras = QWidget()
599 |     imp_layout = QVBoxLayout(tab_impressoras)
600 |
601 |     impressoras = self.printer_service.listar_por_local(empresa.nome)
602 |
603 |     imp_label = QLabel(f"\U0001f5a8\u0000 Impressoras em '{empresa.nome}' ({len(impressoras)})")
604 |     imp_label.setStyleSheet(ESTILO_LABEL_CAMPO.replace("font-size: 12px;", "font-size: 14px;"))
605 |     imp_layout.addWidget(imp_label)
606 |
607 |     if impressoras:
608 |         imp_tabela = QTableWidgetItem()
609 |         imp_tabela.setColumnCount(5)
610 |         imp_tabela.setHorizontalHeaderLabels(["Patrim\u00f4nio", "Modelo", "Serial", "Status", "\u00da"])
611 |         imp_tabela.setRowCount(len(impressoras))
612 |         imp_tabela.setStyleSheet(ESTILO_TABELA_SIMPLES)
613 |         imp_tabela.setSelectionBehavior(QAbstractItemView.SelectRows)
614 |         imp_tabela.setEditTriggers(QAbstractItemView.NoEditTriggers)
615 |         imp_tabela.verticalHeader().setVisible(False)
616 |
617 |         for i, p in enumerate(impressoras):
618 |             imp_tabela.setItem(i, 0, QTableWidgetItem(p.patrimonio))
619 |             imp_tabela.setItem(i, 1, QTableWidgetItem(p.modelo))
620 |             imp_tabela.setItem(i, 2, QTableWidgetItem(p.serial or "-"))
621 |
622 |             status_item = QTableWidgetItem(p.status)
623 |             if p.status in ["Operacional", "Em uso"]:
624 |                 status_item.setForeground(QColor(COR["sucesso"]))
625 |             elif p.status in ["Em manuten\u00e7\u00e3o", "Manuten\u00e7\u00e3o", "Aguardando pe\u00e7a"]:
626 |                 status_item.setForeground(QColor(COR["aviso"]))
627 |             elif p.status in ["Parada", "Sucata"]:
628 |                 status_item.setForeground(QColor(COR["erro"]))
629 |             imp_tabela.setItem(i, 3, status_item)
630 |
631 |             rev_text = p.proxima_revisao.strftime("%d/%m/%Y") if p.proxima_revisao else "-"
632 |             imp_tabela.setItem(i, 4, QTableWidgetItem(rev_text))
633 |
634 |         imp_tabela.verticalHeader().setDefaultSectionSize(44)
635 |         tornar_interativa(imp_tabela)
636 |
637 |         imp_tabela.cellDoubleClicked.connect(lambda r, c: self._abrir_impressora_por_patrimonio(
638 |             imp_tabela.item(r, 0).text(), dialog
639 |         ))
640 |
641 |         imp_layout.addWidget(imp_tabela)
642 |     else:
643 |         sem = QLabel("Nenhuma impressora vinculada a esta empresa.")
644 |         sem.setStyleSheet("color: #94949f; font-size: 13px; padding: 30px;")
645 |         sem.setAlignment(Qt.AlignCenter)
646 |         imp_layout.addWidget(sem)
647 |
648 |     tabs.addTab(tab_impressoras, "\U0001f5a8\u0000 Impressoras")
649 |
650 |     main_layout = QVBoxLayout(dialog)
651 |     main_layout.addWidget(tabs)
652 |     dialog.exec()
653 |
654 | def _salvar_edicao(self, dialog: QDialog, empresa: Any, nome: QLineEdit, cnpj: QLineEdit, tipo: QComboBox)
655 |     nome_antigo = empresa.nome
656 |     nome_novo = nome.text().strip()
657 |
658 |     empresa.cnpj = cnpj.text().strip()
659 |     empresa.tipo = tipo.currentText()

```

```

660 |         empresa.telefone = telefone.text().strip()
661 |         empresa.email = email.text().strip()
662 |         empresa.endereco = endereco.text().strip()
663 |         empresa.cidade = cidade.text().strip()
664 |         empresa.uf = uf.text().strip().upper()
665 |         empresa.observacao = obs.toPlainText().strip()
666 |
667 |         with tratar_erro("salvar edição da empresa"):
668 |             if nome_antigo != nome_novo and nome_novo:
669 |                 self.printer_service.atualizar_local_por_nome_antigo(nome_antigo, nome_novo)
670 |             empresa.nome = nome_novo
671 |             self.session.commit()
672 |             self.recarregar()
673 |             dialog.accept()
674 |
675 |     def _excluir(self, dialog: QDialog, empresa: Any) -> None:
676 |         if ConfirmacaoDigitarDialog.confirmar(
677 |             "Confirmar Exclusão",
678 |             f"Deseja realmente excluir a empresa '{empresa.nome}'?\n\n"
679 |             "As impressoras associadas NÃO serão excluídas, apenas ficarão sem empresa vinculada.",
680 |             dialog,
681 |         ):
682 |             with tratar_erro("excluir empresa"):
683 |                 self.company_service.excluir(empresa)
684 |                 ToastManager.mostrar(
685 |                     f"Empresa '{empresa.nome}' excluída.",
686 |                     "aviso", duracao=8000,
687 |                     acao=("Desfazer", lambda o=empresa, svc=self.company_service, pag=self: (svc.restaurar(
688 |                         o
689 |                     ), self.recarregar())
690 |                 )
691 |
692 |     def _importar(self) -> None:
693 |         dialog: ImportDialog = ImportDialog(self, "companies", "Empresas", self.company_service, self.session)
694 |         if dialog.exec() == ImportDialog.Accepted:
695 |             self.recarregar()
696 |
697 |     def _abrir_impressora_por_patrimonio(self, patrimonio: str, parent_dialog: QDialog) -> None:
698 |         parent_dialog.accept()
699 |         self.abrir_impressora.emit(patrimonio)

```

=== ARQUIVO: .laplviews\pages\config_page.py ===

```

001 | from __future__ import annotations
002 |
003 | import json
004 | import shutil
005 | from datetime import datetime
006 | from typing import Any
007 |
008 | from PySide6.QtCore import Qt
009 | from PySide6.QtWidgets import (
010 |     QCheckBox,
011 |     QComboBox,
012 |     QDialog,
013 |     QDialogButtonBox,
014 |     QFormLayout,
015 |     QFrame,
016 |     QHBoxLayout,
017 |     QLabel,
018 |     QLineEdit,

```

```

019 |     QMessageBox,
020 |     QPushButton,
021 |     QSpinBox,
022 |     QTableWidget,
023 |     QTableWidgetItem,
024 |     QTabWidget,
025 |     QVBoxLayout,
026 |     QWidget,
027 | )
028 |
029 | from app.utils.helpers import formatar_data_hora
030 | from app.utils.ui_helpers import tratar_erro
031 | from app.utils.validacao import ValidadorCampo, obrigatorio, email, minimo
032 | from app.views.styles.theme import (
033 |     COR,
034 |     ESTILO_BOTAO_AVISO,
035 |     ESTILO_BOTAO_FECHAR,
036 |     ESTILO_BOTAO_PRIMARIO,
037 |     ESTILO_BOTAO_SECUNDARIO,
038 |     ESTILO_BOTAO_SUCESSO,
039 |     ESTILO_BOTAO_ERRO,
040 |     configurar_combo,
041 |     configurar_combo_colorido,
042 |     ESTILO_COMBO,
043 |     ESTILO_DIALOG,
044 |     ESTILO_INPUT,
045 |     ESTILO_INPUT_READONLY,
046 |     ESTILO_SUBTITULO,
047 |     ESTILO_TITULO_PAGINA,
048 |     PERFIL_CORES,
049 |     SIM_NAO_CORES,
050 |     group_box,
051 |     input_label,
052 |     campo_rotulo,
053 |     campo_readonly,
054 | )
055 | from app.views.widgets import ToastManager
056 | from app.views.widgets.confirm_dialog import ConfirmacaoDigitalDialog
057 | from app.views.widgets.table_widget import TabelaPadrao
058 | from config import DB_PATH
059 |
060 |
061 | class ConfigPage(QWidget):
062 |     session: Any
063 |     user_service: Any
064 |     notificador: Any | None
065 |     audit_service: Any | None
066 |     printer_service: Any | None
067 |     part_service: Any | None
068 |     company_service: Any | None
069 |     activity_service: Any | None
070 |     technician_service: Any | None
071 |     alert_service: Any | None
072 |     user: dict[str, Any]
073 |     abas: QTabWidget
074 |     tabela: TabelaPadrao
075 |     btn_novo: QPushButton
076 |     btn_editar: QPushButton
077 |     btn_backup: QPushButton
078 |     label_backup: QLabel
079 |     btn_lixeira: QPushButton
080 |     _notif_config: Any
081 |     _chk_desktop: QCheckBox

```

```

082 |     _chk_email: QCheckBox
083 |     _input_smtp_host: QLineEdit
084 |     _input_smtp_port: QSpinBox
085 |     _input_smtp_user: QLineEdit
086 |     _input_smtp_senha: QLineEdit
087 |     _input_email_de: QLineEdit
088 |     _input_email_para: QLineEdit
089 |     _lbl_status_notif: QLabel
090 |     _combo_filtro_tabela: QComboBox
091 |     _tabela_auditoria: TabelaPadrao
092 |     _logs_auditoria: list[Any]
093 |
094 |     def __init__(self, session: Any, user_service: Any, user: dict[str, Any], notificador: Any | None = None):
095 |         super().__init__(parent)
096 |         self.session = session
097 |         self.user_service = user_service
098 |         self.notificador = notificador
099 |         self.audit_service = audit_service
100 |         self.printer_service = printer_service
101 |         self.part_service = part_service
102 |         self.company_service = company_service
103 |         self.activity_service = activity_service
104 |         self.technician_service = technician_service
105 |         self.alert_service = alert_service
106 |         self.user = user
107 |         self._setup_ui()
108 |         self.recarregar()
109 |
110 |     def _setup_ui(self) -> None:
111 |         layout = QVBoxLayout(self)
112 |         layout.setContentsMargins(24, 24, 24, 24)
113 |         layout.setSpacing(16)
114 |
115 |         header_layout = QHBoxLayout()
116 |         header_text_layout = QVBoxLayout()
117 |         titulo = QLabel("Configurações")
118 |         titulo.setStyleSheet(ESTILO_TITULO_PAGINA)
119 |         header_text_layout.addWidget(titulo)
120 |         subtitulo = QLabel("Gerencie usuários, backup e preferências")
121 |         subtitulo.setStyleSheet(ESTILO_SUBTITULO)
122 |         header_text_layout.addWidget(subtitulo)
123 |         header_layout.addLayout(header_text_layout)
124 |         header_layout.addStretch()
125 |         layout.addLayout(header_layout)
126 |
127 |         self.abas = QTabWidget()
128 |         self.abas.setStyleSheet("""
129 |             QTabWidget::pane { margin-top: -1px; }
130 |             QTabBar::tab { color: #94949f; font-weight: 600; }
131 |             QTabBar::tab:selected { color: #a78bfa; border-bottom-color: #a78bfa; }
132 |             QTabBar::tab:hover { color: #e8e8f0; }
133 |         """)
134 |         self.abas.addTab(self._criar_aba_usuarios(), "■ Usuários")
135 |         self.abas.addTab(self._criar_aba_backup(), "■ Backup")
136 |         self.abas.addTab(self._criar_aba_lixeira(), "■■ Lixeira")
137 |         self.abas.addTab(self._criar_aba_auditoria(), "■ Auditoria")
138 |         self.abas.addTab(self._criar_aba_notificacoes(), "■ Notificações")
139 |         layout.addWidget(self.abas)
140 |
141 |     def _criar_aba_usuarios(self) -> QWidget:
142 |         tab = QWidget()
143 |         layout = QVBoxLayout(tab)
144 |         layout.setContentsMargins(0, 16, 0, 0)

```

```

145 |         layout.setSpacing(12)
146 |
147 |         self.tabela = TabelaPadrao(["Nome", "Usuário", "Email", "Perfil", "Ativo"])
148 |         layout.addWidget(self.tabela)
149 |
150 |         botoes = QHBoxLayout()
151 |         botoes.setSpacing(10)
152 |         self.btn_novo = QPushButton("■ Novo Usuário")
153 |         self.btn_novo.setToolTip("Cadastrar novo usuário")
154 |         self.btn_novo.setStyleSheet(ESTILO_BOTAO_SUCESSO)
155 |         self.btn_novo.clicked.connect(self._novo_usuario)
156 |         botoes.addWidget(self.btn_novo)
157 |         self.btn_editar = QPushButton("■ Editar")
158 |         self.btn_editar.setToolTip("Editar usuário selecionado")
159 |         self.btn_editar.setStyleSheet(ESTILO_BOTAO_AVISO)
160 |         self.btn_editar.clicked.connect(self._editar_usuario)
161 |         botoes.addWidget(self.btn_editar)
162 |         botoes.addStretch()
163 |         layout.addLayout(botoes)
164 |         return tab
165 |
166 |     def _criar_aba_backup(self) -> QWidget:
167 |         tab = QWidget()
168 |         layout = QVBoxLayout(tab)
169 |         layout.setContentsMargins(0, 16, 0, 0)
170 |         layout.setSpacing(12)
171 |
172 |         desc = QLabel("Faça backup do banco de dados SQLite para evitar perda de dados.")
173 |         desc.setStyleSheet("color: #94a3b8; font-size: 12px; background: transparent;")
174 |         desc.setWordWrap(True)
175 |         layout.addWidget(desc)
176 |
177 |         self.btn_backup = QPushButton("■ Fazer Backup Agora")
178 |         self.btn_backup.setMinimumHeight(48)
179 |         self.btn_backup.setCursor(Qt.PointingHandCursor)
180 |         self.btn_backup.setToolTip("Criar backup completo do banco de dados")
181 |         self.btn_backup.setStyleSheet("""
182 |             QPushButton {
183 |                 background: qlineargradient(x1:0, y1:0, x2:1, y2:1,
184 |                     stop:0 #a78bfa, stop:1 #7c3aed);
185 |                 color: #ffffff;
186 |                 border: none; border-radius: 10px;
187 |                 padding: 14px 24px; font-size: 14px; font-weight: 700;
188 |             }
189 |             QPushButton:hover {
190 |                 background: qlineargradient(x1:0, y1:0, x2:1, y2:1,
191 |                     stop:0 #c4b5fd, stop:1 #a78bfa);
192 |             }
193 |         """)
194 |         self.btn_backup.clicked.connect(self._fazer_backup)
195 |         layout.addWidget(self.btn_backup)
196 |
197 |         self.label_backup = QLabel("")
198 |         self.label_backup.setStyleSheet("color: #475569; font-size: 12px; background: transparent; padding: 10px 0;")
199 |         layout.addWidget(self.label_backup)
200 |
201 |         layout.addStretch()
202 |         return tab
203 |
204 |     def _criar_aba_lixeira(self) -> QWidget:
205 |         tab = QWidget()
206 |         layout = QVBoxLayout(tab)
207 |         layout.setContentsMargins(0, 16, 0, 0)

```

```

208 |         layout.setSpacing(12)
209 |
210 |         desc = QLabel("Restaura registros que foram excluídos (soft delete).")
211 |         desc.setStyleSheet("color: #94a3b8; font-size: 12px; background: transparent;")
212 |         desc.setWordWrap(True)
213 |         layout.addWidget(desc)
214 |
215 |         self.btn_lixeira = QPushButton("■ Ver Registros Excluídos")
216 |         self.btn_lixeira.setMinimumHeight(44)
217 |         self.btn_lixeira.setCursor(Qt.PointingHandCursor)
218 |         self.btn_lixeira.setToolTip("Abrir lixeira para restaurar registros excluídos")
219 |         self.btn_lixeira.setStyleSheet("""
220 |             QPushButton {
221 |                 background: qlineargradient(x1:0, y1:0, x2:1, y2:1,
222 |                     stop:0 #ef4444, stop:1 #dc2626);
223 |                 color: #ffffff;
224 |                 border: none; border-radius: 10px;
225 |                 padding: 12px 20px; font-size: 13px; font-weight: 700;
226 |             }
227 |             QPushButton:hover {
228 |                 background: qlineargradient(x1:0, y1:0, x2:1, y2:1,
229 |                     stop:0 #f87171, stop:1 #ef4444);
230 |             }
231 |         """)
232 |         self.btn_lixeira.clicked.connect(self._abrir_lixeira)
233 |         layout.addWidget(self.btn_lixeira)
234 |
235 |         layout.addStretch()
236 |         return tab
237 |
238 |     def _criar_aba_notificacoes(self) -> QWidget:
239 |         tab = QWidget()
240 |         layout = QVBoxLayout(tab)
241 |         layout.setContentsMargins(0, 16, 0, 0)
242 |         layout.setSpacing(12)
243 |
244 |         desc = QLabel("Configure notificações para alertas críticos do sistema.")
245 |         desc.setStyleSheet("color: #94a3b8; font-size: 12px; background: transparent;")
246 |         desc.setWordWrap(True)
247 |         layout.addWidget(desc)
248 |
249 |         self._criar_notificacao_ui(layout)
250 |
251 |         layout.addStretch()
252 |         return tab
253 |
254 |     def _criar_notificacao_ui(self, parent_layout: QVBoxLayout) -> None:
255 |         from app.services.notification_service import NotificacaoConfig
256 |
257 |         self._notif_config = NotificacaoConfig.carregar()
258 |
259 |         self._chk_desktop = QCheckBox("Notificação na área de trabalho (sistema)")
260 |         self._chk_desktop.setChecked(self._notif_config.desktop_ativado)
261 |         self._chk_desktop.setStyleSheet(
262 |             "QCheckBox { color: #e2e8f0; font-size: 12px; spacing: 8px; }"
263 |             " QCheckBox::indicator { width: 18px; height: 18px; border-radius: 4px;"
264 |             " border: 2px solid #475569; background: transparent; }"
265 |             " QCheckBox::indicator:checked { background-color: #a78bfa; border-color: #a78bfa; }"
266 |         )
267 |         parent_layout.addWidget(self._chk_desktop)
268 |
269 |         self._chk_email = QCheckBox("Notificação por e-mail")
270 |         self._chk_email.setChecked(self._notif_config.email_ativado)

```

```

271 |         self._chk_email.setStyleSheet(
272 |             "QCheckBox { color: #e2e8f0; font-size: 12px; spacing: 8px; margin-top: 4px; }"
273 |             " QCheckBox::indicator { width: 18px; height: 18px; border-radius: 4px;"
274 |             " border: 2px solid #475569; background: transparent; }"
275 |             " QCheckBox::indicator:checked { background-color: #a78bfa; border-color: #a78bfa; }"
276 |         )
277 |         parent_layout.addWidget(self._chk_email)
278 |
279 |         form_notif = QFormLayout()
280 |         form_notif.setSpacing(8)
281 |         form_notif.setContentsMargins(16, 8, 16, 0)
282 |
283 |         self._input_smtp_host = QLineEdit()
284 |         self._input_smtp_host.setStyleSheet(ESTILO_INPUT)
285 |         self._input_smtp_host.setText(self._notif_config.smtp_host)
286 |         self._input_smtp_host.setPlaceholderText("smtp.gmail.com")
287 |         form_notif.addRow("SMTP Servidor:", self._input_smtp_host)
288 |
289 |         erro_host = QLabel()
290 |         erro_host.setStyleSheet("color: #ef4444; font-size: 10px; background: transparent;")
291 |         erro_host.hide()
292 |         form_notif.addRow("", erro_host)
293 |         ValidadorCampo(self._input_smtp_host, obrigatorio, erro_host)
294 |
295 |         self._input_smtp_port = QSpinBox()
296 |         self._input_smtp_port.setRange(1, 65535)
297 |         self._input_smtp_port.setValue(self._notif_config.smtp_port)
298 |         self._input_smtp_port.setStyleSheet(ESTILO_INPUT)
299 |         form_notif.addRow("SMTP Porta:", self._input_smtp_port)
300 |
301 |         self._input_smtp_user = QLineEdit()
302 |         self._input_smtp_user.setStyleSheet(ESTILO_INPUT)
303 |         self._input_smtp_user.setText(self._notif_config.smtp_usuario)
304 |         self._input_smtp_user.setPlaceholderText("seuemail@gmail.com")
305 |         form_notif.addRow("SMTP Usuário:", self._input_smtp_user)
306 |
307 |         self._input_smtp_senha = QLineEdit()
308 |         self._input_smtp_senha.setStyleSheet(ESTILO_INPUT)
309 |         self._input_smtp_senha.setText(self._notif_config.smtp_senha)
310 |         self._input_smtp_senha.setEchoMode(QLineEdit.Password)
311 |         self._input_smtp_senha.setPlaceholderText("senha ou app password")
312 |         form_notif.addRow("SMTP Senha:", self._input_smtp_senha)
313 |
314 |         self._input_email_de = QLineEdit()
315 |         self._input_email_de.setStyleSheet(ESTILO_INPUT)
316 |         self._input_email_de.setText(self._notif_config.email_remetente)
317 |         self._input_email_de.setPlaceholderText("remetente@email.com")
318 |         form_notif.addRow("E-mail remetente:", self._input_email_de)
319 |
320 |         erro_de = QLabel()
321 |         erro_de.setStyleSheet("color: #ef4444; font-size: 10px; background: transparent;")
322 |         erro_de.hide()
323 |         form_notif.addRow("", erro_de)
324 |         ValidadorCampo(self._input_email_de, email, erro_de)
325 |
326 |         self._input_email_para = QLineEdit()
327 |         self._input_email_para.setStyleSheet(ESTILO_INPUT)
328 |         self._input_email_para.setText(self._notif_config.email_destinatario)
329 |         self._input_email_para.setPlaceholderText("destinatario@email.com")
330 |         form_notif.addRow("E-mail destinatário:", self._input_email_para)
331 |
332 |         erro_para = QLabel()
333 |         erro_para.setStyleSheet("color: #ef4444; font-size: 10px; background: transparent;")

```



```

334 |         erro_para.hide()
335 |         form_notif.addRow("", erro_para)
336 |         ValidadorCampo(self._input_email_para, email, erro_para)
337 |
338 |         parent_layout.addLayout(form_notif)
339 |
340 |         botoes_notif = QHBoxLayout()
341 |         botoes_notif.setSpacing(10)
342 |
343 |         btn_salvar_notif = QPushButton("■ Salvar Configuração")
344 |         btn_salvar_notif.setCursor(Qt.PointingHandCursor)
345 |         btn_salvar_notif.setToolTip("Salvar configurações de notificação")
346 |         btn_salvar_notif.setStyleSheet(ESTILO_BOTAO_SUCESSO)
347 |         btn_salvar_notif.clicked.connect(self._salvar_notificacao)
348 |         botoes_notif.addWidget(btn_salvar_notif)
349 |
350 |         btn_testar = QPushButton("■ Testar E-mail")
351 |         btn_testar.setCursor(Qt.PointingHandCursor)
352 |         btn_testar.setToolTip("Enviar e-mail de teste")
353 |         btn_testar.setStyleSheet(ESTILO_BOTAO_AVISO)
354 |         btn_testar.clicked.connect(self._testar_email)
355 |         botoes_notif.addWidget(btn_testar)
356 |
357 |         botoes_notif.addStretch()
358 |         parent_layout.addLayout(botoes_notif)
359 |
360 |         self._lbl_status_notif = QLabel("")
361 |         self._lbl_status_notif.setStyleSheet("color: #475569; font-size: 12px; background: transparent; padding: 5px;")
362 |         parent_layout.addWidget(self._lbl_status_notif)
363 |
364 |     def _salvar_notificacao(self) -> None:
365 |         cfg = self._notif_config
366 |         cfg.desktop_ativado = self._chk_desktop.isChecked()
367 |         cfg.email_ativado = self._chk_email.isChecked()
368 |         cfg.smtp_host = self._input_smtp_host.text().strip()
369 |         cfg.smtp_port = self._input_smtp_port.value()
370 |         cfg.smtp_usuario = self._input_smtp_user.text().strip()
371 |         cfg.smtp_senha = self._input_smtp_senha.text()
372 |         cfg.email_remetente = self._input_email_de.text().strip()
373 |         cfg.email_destinatario = self._input_email_para.text().strip()
374 |         cfg.salvar()
375 |         if hasattr(self, "_notificador"):
376 |             self._notificador.recarregar_config()
377 |         self._lbl_status_notif.setStyleSheet("color: #34d399; font-size: 12px; background: transparent;")
378 |         self._lbl_status_notif.setText("Configuração salva com sucesso!")
379 |
380 |     def _testar_email(self) -> None:
381 |         from app.services.notification_service import NotificacaoConfig
382 |         cfg = NotificacaoConfig()
383 |         cfg.desktop_ativado = self._chk_desktop.isChecked()
384 |         cfg.email_ativado = True
385 |         cfg.smtp_host = self._input_smtp_host.text().strip()
386 |         cfg.smtp_port = self._input_smtp_port.value()
387 |         cfg.smtp_usuario = self._input_smtp_user.text().strip()
388 |         cfg.smtp_senha = self._input_smtp_senha.text()
389 |         cfg.email_remetente = self._input_email_de.text().strip()
390 |         cfg.email_destinatario = self._input_email_para.text().strip()
391 |
392 |         self._lbl_status_notif.setStyleSheet("color: #fbbf24; font-size: 12px; background: transparent;")
393 |         self._lbl_status_notif.setText("Enviando e-mail de teste...")
394 |         ok, msg = cfg.testar_email()
395 |         if ok:
396 |             self._lbl_status_notif.setStyleSheet("color: #34d399; font-size: 12px; background: transparent;")

```

```

397 |         else:
398 |             self._lbl_status_notif.setStyleSheet("color: #f87171; font-size: 12px; background: transparent;
399 | self._lbl_status_notif.setText(msg)
400 |
401 | def recarregar(self) -> None:
402 |     usuarios = self.user_service.listar_todos()
403 |     self.tabela.limpar()
404 |     self.tabela.setRowCount(len(usuarios))
405 |     for i, u in enumerate(usuarios):
406 |         self.tabela.setItem(i, 0, QTableWidgetItem(u.nome))
407 |         self.tabela.setItem(i, 1, QTableWidgetItem(u.username))
408 |         self.tabela.setItem(i, 2, QTableWidgetItem(u.email))
409 |
410 |         cor_perfil = "#a78bfa" if u.perfil == "admin" else "#60a5fa"
411 |         self.tabela.definir_badge(i, 3, u.perfil.capitalize(), cor_perfil)
412 |
413 |         cor_ativo = "#34d399" if u.ativo else "#475569"
414 |         self.tabela.definir_badge(i, 4, "Sim" if u.ativo else "Não", cor_ativo)
415 |
416 |     self.tabela.redimensionar()
417 |
418 | def _novo_usuario(self) -> None:
419 |     dialog = _UserDialog(self, self.user_service, modo="novo")
420 |     if dialog.exec() == QDialog.Accepted:
421 |         self.recarregar()
422 |
423 | def _editar_usuario(self) -> None:
424 |     linha = self.tabela.currentRow()
425 |     if linha < 0:
426 |         QMessageBox.warning(self, "Aviso", "Selecione um usuário para editar.")
427 |         return
428 |     usuarios = self.user_service.listar_todos()
429 |     if linha >= len(usuarios):
430 |         return
431 |     usuario = usuarios[linha]
432 |     dialog = _UserDialog(self, self.user_service, modo="editar", usuario=usuario, current_user_perfil=s
433 |     if dialog.exec() == QDialog.Accepted:
434 |         self.recarregar()
435 |
436 | def _abrir_lixeira(self) -> None:
437 |     servicos = [
438 |         ("Impressoras", self.printer_service, ["Patrimônio", "Modelo", "Marca", "Serial", "Status", "Lo
439 |             lambda r: [r.patrimonio or "-", r.modelo or "-", r.marca or "-", r.serial or "-", r.status or
440 |         ("Peças", self.part_service, ["Nome", "Código", "Modelo Compatível", "Qtd Estoque", "Estoque Mí
441 |             lambda r: [r.nome or "-", r.codigo or "-", r.modelo_compativel or "-", str(r.quantidade_estoqu
442 |         ("Clientes", self.company_service, ["Nome", "CNPJ", "Telefone", "Email", "Tipo", "Excluído em"]
443 |             lambda r: [r.nome or "-", r.cnpj or "-", r.telefone or "-", r.email or "-", r.tipo or "-", for
444 |         ("OS", self.activity_service, ["Tipo", "Impressora", "Data", "Status", "Recibo", "Excluído em"]
445 |             lambda r: [r.kind or "-", r.printer.patrimonio if r.printer else "-", formatar_data_hora(r.eve
446 |         ("Técnicos", self.technician_service, ["Nome", "Exibição", "Telefone", "Email", "Excluído em"],
447 |             lambda r: [r.nome_completo or "-", r.nome_exibicao or "-", r.telefone or "-", r.email or "-",
448 |         ("Alertas", self.alert_service, ["Título", "Tipo", "Descrição", "Data", "Excluído em"],
449 |             lambda r: [r.titulo or "-", r.tipo or "-", (r.descricao or "")[:80] + ("..." if len(r.descrica
450 |     ]
451 |     dialog = QDialog(self)
452 |     dialog.setWindowTitle("■■■ Lixeira – Registros Excluídos")
453 |     dialog.setMinimumSize(850, 500)
454 |     dialog.setStyleSheet(ESTILO_DIALOG)
455 |     layout = QVBoxLayout(dialog)
456 |     layout.setSpacing(12)
457 |
458 |     abas = QTabWidget()
459 |     abas.setStyleSheet("")

```

```

460 |         QTabBar::tab { color: #94949f; font-weight: 600; padding: 8px 18px; }
461 |         QTabBar::tab:selected { color: #ef4444; border-bottom-color: #ef4444; }
462 |         QTabBar::tab:hover { color: #e8e8f0; }
463 |     """)
464 |
465 |     for nome_tab, svc, colunas, extrair in servicos:
466 |         if not svc:
467 |             continue
468 |         registros = svc.listar_excluidos() if hasattr(svc, "listar_excluidos") else []
469 |         tab = self._criar_tab_lixeira(registros, colunas, extrair, svc, dialog)
470 |         abas.addTab(tab, f"{nome_tab} ({len(registros)})")
471 |
472 |     layout.addWidget(abas)
473 |     btn_fechar = QPushButton("Fechar")
474 |     btn_fechar.setToolTip("Fechar janela")
475 |     btn_fechar.setStyleSheet(ESTILO_BOTAO_FECHAR)
476 |     btn_fechar.clicked.connect(dialog.accept)
477 |     layout.addWidget(btn_fechar, alignment=Qt.AlignCenter)
478 |     dialog.exec()
479 |
480 | def _criar_tab_lixeira(self, registros: list[Any], colunas: list[str], extrair: Any, svc: Any, parent_d
481 |     tab = QWidget()
482 |     layout = QVBoxLayout(tab)
483 |     layout.setSpacing(8)
484 |     if not registros:
485 |         layout.addWidget(QLabel("Nenhum registro excluído."))
486 |         return tab
487 |
488 |     tabela = TabelaPadrao(colunas)
489 |     tabela.setRowCount(len(registros))
490 |     for i, r in enumerate(registros):
491 |         valores = extrair(r)
492 |         for j, val in enumerate(valores):
493 |             tabela.setItem(i, j, QTableWidgetItem(val))
494 |     tabela.redimensionar()
495 |     layout.addWidget(tabela)
496 |
497 |     btn_restaurar = QPushButton("■ Restaurar Selecionado")
498 |     btn_restaurar.setToolTip("Restaurar registro selecionado da lixeira")
499 |     btn_restaurar.setStyleSheet(ESTILO_BOTAO_PRIMARIO)
500 |     btn_restaurar.clicked.connect(lambda: self._restaurar_selecionado(tabela, registros, svc, parent_di
501 |     layout.addWidget(btn_restaurar, alignment=Qt.AlignCenter)
502 |     return tab
503 |
504 | def _restaurar_selecionado(self, tabela: TabelaPadrao, registros: list[Any], svc: Any, dialog: QDialog)
505 |     row = tabela.currentRow()
506 |     if row < 0 or row >= len(registros):
507 |         QMessageBox.warning(self, "Aviso", "Selecione um registro para restaurar.")
508 |         return
509 |     obj = registros[row]
510 |     if ConfirmacaoDigitarDialog.confirmar(
511 |         "Confirmar",
512 |         f"Restaurar '{getattr(obj, 'nome', getattr(obj, 'patrimonio', obj.id))}'?\n\n"
513 |         "O registro voltará a aparecer nas listagens normais.",
514 |         self,
515 |     ):
516 |         with tratar_erro("restaurar registro"):
517 |             svc.restaurar(obj)
518 |             QMessageBox.information(self, "Restaurado", "Registro restaurado com sucesso.")
519 |             dialog.accept()
520 |             self._abrir_lixeira()
521 |
522 | def _criar_aba_auditoria(self) -> QWidget:

```

```

523 |         tab = QWidget()
524 |         layout = QVBoxLayout(tab)
525 |         layout.setContentsMargins(0, 16, 0, 0)
526 |         layout.setSpacing(12)
527 |
528 |         desc = QLabel("Histórico de alterações feitas no sistema.")
529 |         desc.setStyleSheet("color: #94a3b8; font-size: 12px; background: transparent;")
530 |         desc.setWordWrap(True)
531 |         layout.addWidget(desc)
532 |
533 |         filtros = QHBoxLayout()
534 |         filtros.setSpacing(8)
535 |
536 |         lbl_tabela = QLabel("Tabela:")
537 |         lbl_tabela.setStyleSheet("color: #c8c8d8; font-size: 12px; background: transparent;")
538 |         filtros.addWidget(lbl_tabela)
539 |
540 |         self._combo_filtro_tabela = QComboBox()
541 |         self._combo_filtro_tabela.addItem("Todas", "printers", "activities", "companies", "parts", "techn
542 |         self._combo_filtro_tabela.setStyleSheet(ESTILO_COMBO)
543 |         self._combo_filtro_tabela.currentIndexChanged.connect(self._recarregar_auditoria)
544 |         filtros.addWidget(self._combo_filtro_tabela)
545 |
546 |         filtros.addStretch()
547 |
548 |         btn_recargar = QPushButton("■ Recargar")
549 |         btn_recargar.setToolTip("Recargar registros de auditoria")
550 |         btn_recargar.setStyleSheet(ESTILO_BOTAO_SECUNDARIO)
551 |         btn_recargar.clicked.connect(self._recarregar_auditoria)
552 |         filtros.addWidget(btn_recargar)
553 |
554 |         layout.addLayout(filtros)
555 |
556 |         self._tabela_auditoria = TabelaPadrao(["Data/Hora", "Usuário", "Ação", "Tabela", "Registro"])
557 |         self._tabela_auditoria.setEditTriggers(QTableWidget.NoEditTriggers)
558 |         self._tabela_auditoria.setSelectionBehavior(QTableWidget.SelectRows)
559 |         self._tabela_auditoria.setAlternatingRowColors(True)
560 |         self._tabela_auditoria.verticalHeader().setVisible(False)
561 |         layout.addWidget(self._tabela_auditoria, stretch=1)
562 |
563 |         botoes = QHBoxLayout()
564 |         botoes.setSpacing(10)
565 |
566 |         btn_detalhes = QPushButton("■ Ver Detalhes")
567 |         btn_detalhes.setToolTip("Ver detalhes do registro de auditoria selecionado")
568 |         btn_detalhes.setStyleSheet(ESTILO_BOTAO_AVISO)
569 |         btn_detalhes.clicked.connect(self._detalhes_auditoria)
570 |         botoes.addWidget(btn_detalhes)
571 |
572 |         botoes.addStretch()
573 |         layout.addLayout(botoes)
574 |
575 |         self._logs_auditoria: list = []
576 |         self._recarregar_auditoria()
577 |         return tab
578 |
579 |     def _recarregar_auditoria(self) -> None:
580 |         if not self.audit_service:
581 |             return
582 |         filtro_tabela = self._combo_filtro_tabela.currentText()
583 |         if filtro_tabela == "Todas":
584 |             self._logs_auditoria = self.audit_service.listar(limite=200)
585 |         else:

```

```

586 |         self._logs_auditoria = self.audit_service.listar_por_tabela(filtro_tabela, limite=200)
587 |
588 |     self._tabela_auditoria.limpar()
589 |     self._tabela_auditoria.setRowCount(len(self._logs_auditoria))
590 |     for i, log in enumerate(self._logs_auditoria):
591 |         self._tabela_auditoria.setItem(i, 0, QTableWidgetItem(formatar_data_hora(log.created_at)))
592 |         nome_user = log.user.nome if log.user else "-"
593 |         self._tabela_auditoria.setItem(i, 1, QTableWidgetItem(nome_user))
594 |         self._tabela_auditoria.setItem(i, 2, QTableWidgetItem(log.acao))
595 |         self._tabela_auditoria.setItem(i, 3, QTableWidgetItem(log.tabela_alvo))
596 |         self._tabela_auditoria.setItem(i, 4, QTableWidgetItem(log.registro_id))
597 |     self._tabela_auditoria.redimensionar()
598 |
599 |     def _detalhes_auditoria(self) -> None:
600 |         row = self._tabela_auditoria.currentRow()
601 |         if row < 0 or row >= len(self._logs_auditoria):
602 |             QMessageBox.warning(self, "Aviso", "Selecione um registro para ver detalhes.")
603 |             return
604 |         log = self._logs_auditoria[row]
605 |
606 |         dialog = QDialog(self)
607 |         dialog.setWindowTitle(f"Detalhes - #{log.id}")
608 |         dialog.setMinimumSize(600, 400)
609 |         dialog.setStyleSheet(ESTILO_DIALOG)
610 |
611 |         layout = QVBoxLayout(dialog)
612 |         layout.setSpacing(12)
613 |         layout.setContentsMargins(20, 20, 20, 20)
614 |
615 |         info = QLabel(
616 |             f"<b style='color:#a78bfa'>#{log.id}</b> - "
617 |             f"<b style='color:#e8e8f0'>{log.acao}</b> "
618 |             f"em <b style='color:#60a5fa'>{log.tabela_alvo}</b> "
619 |             f"(registro {log.registro_id})<br>"
620 |             f"<span style='color:#94949f'>Por: {log.user.nome if log.user else '-'} • {formatar_data_hora(log.created_at)}</span>"
621 |         )
622 |         info.setWordWrap(True)
623 |         info.setStyleSheet("background: transparent; padding: 8px 0;")
624 |         layout.addWidget(info)
625 |
626 |         abas = QTabWidget()
627 |         abas.setStyleSheet("""
628 |             QTabWidget::pane { border: 1px solid #2a2a3e; background: #16162a; border-radius: 6px; }
629 |             QTabBar::tab { color: #94949f; font-weight: 600; padding: 8px 16px; }
630 |             QTabBar::tab:selected { color: #a78bfa; border-bottom-color: #a78bfa; }
631 |         """)
632 |
633 |         for titulo_aba, dado, cor in [("■ Antes", log.dados_antes, "#f87171"),
634 |                                         ("■ Depois", log.dados_depois, "#34d399")]:
635 |             tab_aba = QWidget()
636 |             lay = QVBoxLayout(tab_aba)
637 |             lay.setContentsMargins(12, 12, 12, 12)
638 |
639 |             txt = QLabel()
640 |             txt.setWordWrap(True)
641 |             txt.setTextInteractionFlags(Qt.TextSelectableByMouse)
642 |             if dado:
643 |                 try:
644 |                     parsed = json.loads(dado)
645 |                     formatted = json.dumps(parsed, indent=2, ensure_ascii=False)
646 |                     txt.setText(f"<pre style='color:{cor}; font-size:12px; font-family:Courier New;'>{formatted}</pre>")
647 |                 except json.JSONDecodeError:
648 |                     txt.setText(f"<pre style='color:{cor};'>{dado}</pre>")

```

```

649 |         else:
650 |             txt.setText("<span style='color:#475569;'>Sem dados</span>")
651 |             lay.addWidget(txt)
652 |             abas.addTab(tab_abas, titulo_abas)
653 |
654 |         layout.addWidget(abas)
655 |
656 |         btn_fechar = QPushButton("Fechar")
657 |         btn_fechar.setToolTip("Fechar janela")
658 |         btn_fechar.setStyleSheet(ESTILO_BOTAO_FECHAR)
659 |         btn_fechar.clicked.connect(dialog.accept)
660 |         layout.addWidget(btn_fechar, alignment=Qt.AlignCenter)
661 |
662 |         dialog.exec()
663 |
664 | def _fazer_backup(self) -> None:
665 |     try:
666 |         from config import ANEXOS_DIR
667 |         backup_dir = DB_PATH.parent / "backups"
668 |         backup_dir.mkdir(parents=True, exist_ok=True)
669 |         timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
670 |         nome_arquivo = f"backup_{timestamp}.db"
671 |         destino = backup_dir / nome_arquivo
672 |         shutil.copy2(str(DB_PATH), str(destino))
673 |         backup_anexos = backup_dir / f"anexos_{timestamp}"
674 |         if ANEXOS_DIR.exists():
675 |             shutil.copypath(str(ANEXOS_DIR), str(backup_anexos))
676 |         self.label_backup.setStyleSheet("color: #34d399; font-size: 12px; background: transparent; padding: 5px;")
677 |         self.label_backup.setText(f"Backup criado: {nome_arquivo}")
678 |     except Exception as e:
679 |         self.label_backup.setStyleSheet("color: #f87171; font-size: 12px; background: transparent; padding: 5px;")
680 |         self.label_backup.setText(f"Erro: {e}")
681 |
682 |
683 | class _UserDialog(QDialog):
684 |     user_service: Any
685 |     modo: str
686 |     usuario: Any | None
687 |     input_nome: QLineEdit
688 |     input_username: QLineEdit
689 |     input_email: QLineEdit
690 |     input_senha: QLineEdit
691 |     combo_perfil: QComboBox
692 |     combo_ativo: QComboBox | None
693 |
694 |     def __init__(self, parent: QWidget | None, user_service: Any, modo: str = "novo", usuario: Any | None = None):
695 |         super().__init__(parent)
696 |         self.user_service = user_service
697 |         self.modo = modo
698 |         self.usuario = usuario
699 |         self._current_user_perfil = current_user_perfil
700 |         self._setup_ui()
701 |
702 |     def _setup_ui(self) -> None:
703 |         titulo = "Novo Usuário" if self.modo == "novo" else "Editar Usuário"
704 |         self.setWindowTitle(titulo)
705 |         self.setMinimumWidth(420)
706 |         self.setStyleSheet(ESTILO_DIALOG)
707 |
708 |         layout = QVBoxLayout(self)
709 |         layout.setSpacing(15)
710 |         layout.setContentsMargins(25, 25, 25, 25)
711 |

```

```

712 |         # ■■ SEÇÃO: Identificação ■■
713 |         id_box, id_layout = group_box("Identificação")
714 |         id_form = QFormLayout()
715 |         id_form.setSpacing(10)
716 |         id_form.setLabelAlignment(Qt.AlignRight)
717 |
718 |         self.input_nome = QLineEdit()
719 |         self.input_nome.setStyleSheet(ESTILO_INPUT)
720 |         self.input_nome.setMaxLength(120)
721 |         self.input_nome.setPlaceholderText("Nome completo")
722 |         if self.usuario:
723 |             self.input_nome.setText(self.usuario.nome)
724 |         id_form.addRow("Nome:", self.input_nome)
725 |
726 |         erro_nome = QLabel()
727 |         erro_nome.setStyleSheet("color: #ef4444; font-size: 10px; background: transparent;")
728 |         erro_nome.hide()
729 |         id_form.addRow("", erro_nome)
730 |         ValidadorCampo(self.input_nome, obrigatorio, erro_nome)
731 |
732 |         self.input_username = QLineEdit()
733 |         self.input_username.setStyleSheet(ESTILO_INPUT)
734 |         self.input_username.setMaxLength(50)
735 |         self.input_username.setPlaceholderText("Nome de usuário")
736 |         if self.usuario:
737 |             self.input_username.setText(self.usuario.username)
738 |         if self.moda == "editar" and self._current_user_perfil != "admin":
739 |             self.input_username.setReadOnly(True)
740 |             self.input_username.setStyleSheet(ESTILO_INPUT_READONLY)
741 |         id_form.addRow("Usuário:", self.input_username)
742 |
743 |         erro_user = QLabel()
744 |         erro_user.setStyleSheet("color: #ef4444; font-size: 10px; background: transparent;")
745 |         erro_user.hide()
746 |         id_form.addRow("", erro_user)
747 |         if self.moda == "novo":
748 |             ValidadorCampo(self.input_username, minimo(3), erro_user)
749 |
750 |         self.input_email = QLineEdit()
751 |         self.input_email.setStyleSheet(ESTILO_INPUT)
752 |         self.input_email.setMaxLength(120)
753 |         self.input_email.setPlaceholderText("email@exemplo.com")
754 |         if self.usuario:
755 |             self.input_email.setText(self.usuario.email)
756 |         id_form.addRow("Email:", self.input_email)
757 |
758 |         erro_email = QLabel()
759 |         erro_email.setStyleSheet("color: #ef4444; font-size: 10px; background: transparent;")
760 |         erro_email.hide()
761 |         id_form.addRow("", erro_email)
762 |         ValidadorCampo(self.input_email, email, erro_email)
763 |
764 |         id_layout.addLayout(id_form)
765 |         layout.addWidget(id_box)
766 |
767 |         # ■■ SEÇÃO: Segurança ■■
768 |         seg_box, seg_layout = group_box("Segurança")
769 |         seg_form = QFormLayout()
770 |         seg_form.setSpacing(10)
771 |         seg_form.setLabelAlignment(Qt.AlignRight)
772 |
773 |         self.input_senha = QLineEdit()
774 |         self.input_senha.setStyleSheet(ESTILO_INPUT)

```

```

775 |         self.input_senha.setPlaceholderText("Digite a senha" if self.modos == "novo" else "Deixe em branco p
776 |         self.input_senha.setEchoMode(QLineEdit.Password)
777 |         seg_form.addRow("Senha:", self.input_senha)
778 |
779 |         self.combo_perfil = QComboBox()
780 |         configurar_combo(self.combo_perfil)
781 |         cmp = self.combo_perfil.completer()
782 |         if cmp:
783 |             cmp.setFilterMode(Qt.MatchFlag.MatchContains)
784 |             cmp.setCaseSensitivity(Qt.CaseSensitivity.CaseInsensitive)
785 |         self.combo_perfil.addItem("admin", "tecnico", "visualizador"])
786 |         configurar_combo_colorido(self.combo_perfil, PERFIL_CORES)
787 |         if self.usuario:
788 |             idx = self.combo_perfil.findText(self.usuario.perfil)
789 |             if idx >= 0:
790 |                 self.combo_perfil.setCurrentIndex(idx)
791 |         seg_form.addRow("Perfil:", self.combo_perfil)
792 |
793 |         if self.modos == "editar":
794 |             self.combo_ativo = QComboBox()
795 |             configurar_combo(self.combo_ativo)
796 |             cmp = self.combo_ativo.completer()
797 |             if cmp:
798 |                 cmp.setFilterMode(Qt.MatchFlag.MatchContains)
799 |                 cmp.setCaseSensitivity(Qt.CaseSensitivity.CaseInsensitive)
800 |             self.combo_ativo.addItem("Sim", "Nao"])
801 |             configurar_combo_colorido(self.combo_ativo, SIM_NAO_CORES)
802 |             if self.usuario and not self.usuario.ativo:
803 |                 self.combo_ativo.setCurrentIndex(1)
804 |             seg_form.addRow("Ativo:", self.combo_ativo)
805 |
806 |         seg_layout.addLayout(seg_form)
807 |         layout.addWidget(seg_box)
808 |
809 |         botoes = QDialogButtonBox(QDialogButtonBox.Ok | QDialogButtonBox.Cancel)
810 |         botoes.accepted.connect(self._validar)
811 |         botoes.rejected.connect(self.reject)
812 |         btn_ok = botoes.button(QDialogButtonBox.Ok)
813 |         if btn_ok:
814 |             btn_ok.setToolTip("Salvar usuário")
815 |         btn_cancel = botoes.button(QDialogButtonBox.Cancel)
816 |         if btn_cancel:
817 |             btn_cancel.setToolTip("Cancelar alterações")
818 |         layout.addWidget(botoes)
819 |
820 |     def _validar(self) -> None:
821 |         nome = self.input_nome.text().strip()
822 |         username = self.input_username.text().strip()
823 |         email = self.input_email.text().strip()
824 |         senha = self.input_senha.text()
825 |         perfil = self.combo_perfil.currentText()
826 |
827 |         if not nome:
828 |             QMessageBox.warning(self, "Validação", "O campo Nome é obrigatório.")
829 |             return
830 |         if not username:
831 |             QMessageBox.warning(self, "Validação", "O campo Usuário é obrigatório.")
832 |             return
833 |         if not email:
834 |             QMessageBox.warning(self, "Validação", "O campo Email é obrigatório.")
835 |             return
836 |         if self.modos == "novo" and not senha:
837 |             QMessageBox.warning(self, "Validação", "O campo Senha é obrigatório.")

```



```

838 |         return
839 |
840 |     with tratar_erro("salvar usuário"):
841 |         if self.modo == "novo":
842 |             if self.user_service.verificar_existente(email, username):
843 |                 QMessageBox.warning(self, "Validação", "Usuário já existe.")
844 |                 return
845 |             self.user_service.criar(nome, username, email, senha, perfil)
846 |         else:
847 |             kwargs = {"nome": nome, "email": email, "perfil": perfil}
848 |             if senha:
849 |                 kwargs["senha"] = senha
850 |             if hasattr(self, "combo_ativo"):
851 |                 kwargs["ativo"] = self.combo_ativo.currentText() == "Sim"
852 |             self.user_service.atualizar(self.usuario, **kwargs)
853 |         self.accept()

```

=== ARQUIVO: .\app\views\pages\dashboard_page.py ===

```

001 | from __future__ import annotations
002 |
003 | from datetime import datetime
004 | from typing import Any
005 |
006 | from PySide6.QtCore import Qt, Signal
007 | from PySide6.QtWidgets import (
008 |     QAbstractItemView,
009 |     QDialog,
010 |     QHBoxLayout,
011 |     QHeaderView,
012 |     QLabel,
013 |     QPushButton,
014 |     QScrollArea,
015 |     QTableWidgetItem,
016 |     QTableWidgetItem,
017 |     QVBoxLayout,
018 |     QWidget,
019 | )
020 |
021 | from app.utils.helpers import formatar_data
022 | from app.views.styles.theme import (
023 |     ESTILO_SUBTITULO,
024 |     ESTILO_TITULO_PAGINA,
025 |     STATUS_CORES,
026 |     STATUS_MANUTENCAO,
027 |     STATUS_OPERACIONAL,
028 |     _cor_rgba,
029 | )
030 | from app.views.widgets.card_widget import CardWidget
031 | from app.views.widgets.table_widget import tornar_interativa
032 | from app.views.widgets.chart_widget import BarChart, LineChart, PizzaChart
033 |
034 |
035 | class DashboardPage(QWidget):
036 |     signal_trocar_pagina: Signal = Signal(int)
037 |
038 |     session: Any
039 |     printer_service: Any
040 |     activity_service: Any
041 |     dashboard_service: Any
042 |     lbl_atualizacao: QLabel

```

```

043 |     card_total: CardWidget
044 |     card_manut: CardWidget
045 |     card_op: CardWidget
046 |     card_os: CardWidget
047 |     chart_pizza: PizzaChart
048 |     chart_barras: BarChart
049 |     chart_linha: LineChart
050 |     atv_container: QVBoxLayout
051 |
052 |     def __init__(self, session: Any, printer_service: Any, activity_service: Any, dashboard_service: Any, p
053 |         super().__init__(parent)
054 |         self.session = session
055 |         self.printer_service = printer_service
056 |         self.activity_service = activity_service
057 |         self.dashboard_service = dashboard_service
058 |
059 |         outer = QVBoxLayout(self)
060 |         outer.setContentsMargins(0, 0, 0, 0)
061 |
062 |         scroll = QScrollArea()
063 |         scroll.setWidgetResizable(True)
064 |         scroll.setHorizontalScrollBarPolicy(Qt.ScrollBarAlwaysOff)
065 |         scroll.setStyleSheet("QScrollArea { border: none; background: transparent; }")
066 |
067 |         container = QWidget()
068 |         container.setStyleSheet("background: transparent;")
069 |         layout = QVBoxLayout(container)
070 |         layout.setContentsMargins(24, 24, 24, 24)
071 |         layout.setSpacing(24)
072 |
073 |         header = QVBoxLayout()
074 |         header.setSpacing(4)
075 |         titulo = QLabel("Dashboard")
076 |         titulo.setStyleSheet(ESTILO_TITULO_PAGINA)
077 |         header.addWidget(titulo)
078 |         subtitulo = QLabel("Visão geral do sistema de controle de impressoras")
079 |         subtitulo.setStyleSheet(ESTILO_SUBTITULO)
080 |         header.addWidget(subtitulo)
081 |         self.lbl_atualizacao = QLabel("")
082 |         self.lbl_atualizacao.setStyleSheet("color: #475569; font-size: 11px; background: transparent;")
083 |         header.addWidget(self.lbl_atualizacao)
084 |         layout.addWidget(header)
085 |
086 |         cards = QHBoxLayout()
087 |         cards.setSpacing(16)
088 |
089 |         self.card_total = CardWidget("■■", "Total Impressoras", "0", estilo="azul",
090 |             callback=lambda: self.signal_trocar_pagina.emit(1))
091 |         self.card_manut = CardWidget("■■", "Em Manutenção", "0", estilo="amarelo",
092 |             callback=lambda: self.ao_clicar_status("manutencao"))
093 |         self.card_op = CardWidget("■", "Operacionais", "0", estilo="verde",
094 |             callback=lambda: self.ao_clicar_status("operacional"))
095 |         self.card_os = CardWidget("■", "Atividades", "0", estilo="roxo",
096 |             callback=lambda: self.signal_trocar_pagina.emit(2))
097 |
098 |         cards.addWidget(self.card_total)
099 |         cards.addWidget(self.card_manut)
100 |         cards.addWidget(self.card_op)
101 |         cards.addWidget(self.card_os)
102 |         layout.addWidget(cards)
103 |
104 |         charts_row = QHBoxLayout()
105 |         charts_row.setSpacing(16)

```

```

106 |
107 | chart_pizza_card = QWidget()
108 | chart_pizza_card.setStyleSheet("background: rgba(20,20,31,0.5); border: 1px solid rgba(42,42,62,0.5); border-radius: 10px;")
109 | cp_layout = QVBoxLayout(chart_pizza_card)
110 | cp_layout.setContentsMargins(0, 0, 0, 0)
111 | cp_layout.setSpacing(0)
112 | cp_header = QLabel("Status das Impressoras")
113 | cp_header.setStyleSheet(
114 |     "font-size: 14px; font-weight: 600; color: #e8e8f0; "
115 |     "padding: 16px 20px; border-bottom: 1px solid rgba(42,42,62,0.5); "
116 |     "background: transparent;"
117 | )
118 | cp_layout.addWidget(cp_header)
119 | self.chart_pizza = PizzaChart("", width=4, height=3)
120 | self.chart_pizza.setStyleSheet("background: transparent; border: none;")
121 | cp_layout.addWidget(self.chart_pizza)
122 |
123 | chart_bar_card = QWidget()
124 | chart_bar_card.setStyleSheet("background: rgba(20,20,31,0.5); border: 1px solid rgba(42,42,62,0.5); border-radius: 10px;")
125 | cb_layout = QVBoxLayout(chart_bar_card)
126 | cb_layout.setContentsMargins(0, 0, 0, 0)
127 | cb_layout.setSpacing(0)
128 | cb_header = QLabel("Atividades por Mês")
129 | cb_header.setStyleSheet(
130 |     "font-size: 14px; font-weight: 600; color: #e8e8f0; "
131 |     "padding: 16px 20px; border-bottom: 1px solid rgba(42,42,62,0.5); "
132 |     "background: transparent;"
133 | )
134 | cb_layout.addWidget(cb_header)
135 | self.chart_barras = BarChart("", width=4, height=3)
136 | self.chart_barras.setStyleSheet("background: transparent; border: none;")
137 | cb_layout.addWidget(self.chart_barras)
138 |
139 | charts_row.addWidget(chart_pizza_card, 1)
140 | charts_row.addWidget(chart_bar_card, 1)
141 | layout.addLayout(charts_row)
142 |
143 | linha_card = QWidget()
144 | linha_card.setStyleSheet("background: rgba(20,20,31,0.5); border: 1px solid rgba(42,42,62,0.5); border-radius: 10px;")
145 | cl_layout = QVBoxLayout(linha_card)
146 | cl_layout.setContentsMargins(0, 0, 0, 0)
147 | cl_layout.setSpacing(0)
148 | cl_header = QLabel("Alertas (últimos 30 dias)")
149 | cl_header.setStyleSheet(
150 |     "font-size: 14px; font-weight: 600; color: #e8e8f0; "
151 |     "padding: 16px 20px; border-bottom: 1px solid rgba(42,42,62,0.5); "
152 |     "background: transparent;"
153 | )
154 | cl_layout.addWidget(cl_header)
155 | self.chart_linha = LineChart("", width=8.2, height=2.5)
156 | self.chart_linha.setStyleSheet("background: transparent; border: none;")
157 | cl_layout.addWidget(self.chart_linha)
158 | layout.addWidget(linha_card)
159 |
160 | atv_card = QWidget()
161 | atv_card.setStyleSheet("background: rgba(20,20,31,0.5); border: 1px solid rgba(42,42,62,0.5); border-radius: 10px;")
162 | atv_layout = QVBoxLayout(atv_card)
163 | atv_layout.setContentsMargins(0, 0, 0, 0)
164 | atv_layout.setSpacing(0)
165 | atv_header = QLabel("Atividades Recentes")
166 | atv_header.setStyleSheet(
167 |     "font-size: 14px; font-weight: 600; color: #e8e8f0; "
168 |     "padding: 16px 20px; border-bottom: 1px solid rgba(42,42,62,0.5); "

```

```

169 |         "background: transparent;"
170 |     )
171 |     atv_layout.addWidget(atv_header)
172 |
173 |     self.atv_container = QVBoxLayout()
174 |     self.atv_container.setContentsMargins(20, 12, 20, 12)
175 |     self.atv_container.setSpacing(0)
176 |     atv_layout.addLayout(self.atv_container)
177 |     layout.addWidget(atv_card)
178 |
179 |     layout.addStretch(1)
180 |
181 |     scroll.setWidget(container)
182 |     outer.addWidget(scroll)
183 |
184 | def recarregar(self) -> None:
185 |     dados: dict[str, Any] = self.dashboard_service.resumo()
186 |     self.card_total.atualizar_valor(dados.get("total_impressoras", 0))
187 |     self.card_manut.atualizar_valor(dados.get("em_manutencao", 0))
188 |     self.card_op.atualizar_valor(dados.get("operacionais", 0))
189 |     self.card_os.atualizar_valor(dados.get("total_atividades", 0))
190 |     self.lbl_atualizacao.setText(f"Atualizado: {datetime.now():%H:%M:%S}")
191 |
192 |     try:
193 |         labels_pizza, valores_pizza = self.dashboard_service.dados_grafico_pizza()
194 |         self.chart_pizza.atualizar(labels_pizza, valores_pizza)
195 |
196 |         labels_barras, valores_barras = self.dashboard_service.dados_grafico_atividades()
197 |         self.chart_barras.atualizar(labels_barras, valores_barras)
198 |
199 |         labels_linha, valores_linha = self.dashboard_service.dados_grafico_alertas()
200 |         self.chart_linha.atualizar(labels_linha, valores_linha)
201 |     except Exception:
202 |         import traceback; traceback.print_exc()
203 |
204 |     self._preencher_atividades()
205 |
206 | def _preencher_atividades(self) -> None:
207 |     while self.atv_container.count():
208 |         item = self.atv_container.takeAt(0)
209 |         if item.widget():
210 |             item.widget().deleteLater()
211 |
212 |     try:
213 |         recentes = self.activity_service.listar(limite=5)
214 |     except Exception:
215 |         recentes = []
216 |
217 |     if not recentes:
218 |         lbl = QLabel("Nenhuma atividade recente")
219 |         lbl.setStyleSheet("color: #717182; font-size: 12px; padding: 16px 0; background: transparent;")
220 |         self.atv_container.addWidget(lbl)
221 |         return
222 |
223 |     for i, atv in enumerate(recentes):
224 |         row = QWidget()
225 |         row.setStyleSheet("background: transparent;")
226 |         row_layout = QHBoxLayout(row)
227 |         row_layout.setContentsMargins(0, 10, 0, 10)
228 |         row_layout.setSpacing(12)
229 |
230 |         if i < len(recentes) - 1:
231 |             row.setStyleSheet(f"background: transparent; border-bottom: 1px solid {_cor_rgba('#2a2a3e',

```

```

232 |
233 |     cor_dot = "#6366f1"
234 |     if atv.kind == "MANUTENCAO":
235 |         cor_dot = "#f97316"
236 |     elif atv.kind == "MOVIMENTACAO":
237 |         cor_dot = "#3b82f6"
238 |
239 |     dot = QLabel()
240 |     dot.setFixedSize(8, 8)
241 |     dot.setStyleSheet(
242 |         f"background-color: {cor_dot}; border-radius: 4px; border: none; "
243 |         f"margin-top: 4px;"
244 |     )
245 |     row_layout.addWidget(dot, alignment=Qt.AlignTop)
246 |
247 |     texto_layout = QVBoxLayout()
248 |     texto_layout.setSpacing(2)
249 |
250 |     printer_info = ""
251 |     if atv.printer_id:
252 |         try:
253 |             p = self.printer_service.buscar_por_id(atv.printer_id)
254 |             if p:
255 |                 printer_info = f" - {p.patrimonio}"
256 |         except Exception:
257 |             log.exception("Erro ao buscar impressora para activity card")
258 |
259 |     action = QLabel(f"{atv.notes or atv.kind or 'Atividade'}{printer_info}")
260 |     action.setStyleSheet(
261 |         "font-size: 13px; font-weight: 500; color: #e8e8f0; "
262 |         "background: transparent; border: none;"
263 |     )
264 |     action.setWordWrap(True)
265 |     texto_layout.addWidget(action)
266 |
267 |     data_str = formatar_data(atv.event_at) if atv.event_at else ""
268 |     if data_str:
269 |         data_lbl = QLabel(data_str)
270 |         data_lbl.setStyleSheet(
271 |             "font-size: 11px; color: #717182; background: transparent; border: none;"
272 |         )
273 |         texto_layout.addWidget(data_lbl)
274 |
275 |     row_layout.addLayout(texto_layout)
276 |     row_layout.addStretch()
277 |
278 |     self.atv_container.addWidget(row)
279 |
280 | def ao_clicar_status(self, tipo: str) -> None:
281 |     if tipo == "manutencao":
282 |         titulo: str = "■ Impressoras em Manutenção"
283 |         status_list: list[str] = STATUS_MANUTENCAO
284 |     elif tipo == "operacional":
285 |         titulo = "■ Impressoras Operacionais"
286 |         status_list = STATUS_OPERACIONAL
287 |     else:
288 |         return
289 |
290 |     impressoras = self.printer_service.buscar_por_status(status_list)
291 |
292 |     dialog = QDialog(self)
293 |     dialog.setWindowTitle(titulo)
294 |     dialog.setMinimumSize(800, 520)

```

```

295 |         dialog.setStyleSheet("""
296 |             QDialog {
297 |                 background-color: #14141f;
298 |                 border: 1px solid #2a2a3e;
299 |                 border-radius: 16px;
300 |             }
301 |             QLabel {
302 |                 color: #e8e8f0; font-size: 18px; font-weight: 700;
303 |                 background: transparent; padding: 16px 20px 0 20px;
304 |             }
305 |             QTableWidget {
306 |                 background-color: rgba(20,20,31,0.5); color: #e8e8f0;
307 |                 border: 1px solid rgba(42,42,62,0.5); border-radius: 10px;
308 |                 gridline-color: transparent; font-size: 13px;
309 |             }
310 |             QTableWidget::item { padding: 8px; border-bottom: 1px solid rgba(42,42,62,0.3); }
311 |             QTableWidget::item:hover { background-color: rgba(30,30,46,0.5); }
312 |             QHeaderView::section {
313 |                 background-color: transparent; color: #717182;
314 |                 font-weight: 700; padding: 10px; border: none;
315 |                 border-bottom: 1px solid rgba(42,42,62,0.5);
316 |             }
317 |             QPushButton {
318 |                 background-color: transparent; color: #717182;
319 |                 border: 1px solid #2a2a3e; border-radius: 8px;
320 |                 padding: 10px 30px; font-size: 13px; font-weight: 600;
321 |             }
322 |             QPushButton:hover {
323 |                 background-color: #1e1e2e; color: #e8e8f0;
324 |                 border-color: #6366f1;
325 |             }
326 |         """)
327 |
328 |         layout = QVBoxLayout(dialog)
329 |         layout.setContentsMargins(20, 10, 20, 20)
330 |         layout.setSpacing(12)
331 |
332 |         lbl_titulo = QLabel(f"{titulo} · {len(impressoras)} encontrada(s)")
333 |         layout.addWidget(lbl_titulo)
334 |
335 |         tabela = QTableWidget()
336 |         tabela.setColumnCount(5)
337 |         tabela.setHorizontalHeaderLabels(["Patrimônio", "Modelo", "Status", "Local Atual", "Última Revisão"])
338 |         tabela.setRowCount(len(impressoras))
339 |
340 |         from app.views.widgets.badge_widget import BadgeWidget
341 |         for i, p in enumerate(impressoras):
342 |             tabela.setItem(i, 0, QTableWidgetItem(p.patrimonio))
343 |             tabela.setItem(i, 1, QTableWidgetItem(p.modelo))
344 |
345 |             cor_status = STATUS_CORES.get(p.status, "#94949f")
346 |             badge = BadgeWidget(p.status, cor_status)
347 |             tabela.setCellWidget(i, 2, badge)
348 |
349 |             tabela.setItem(i, 3, QTableWidgetItem(p.local_atual or "-"))
350 |
351 |             ultima = self.activity_service.buscar_ultima_manutencao(p.id)
352 |             rev = formatar_data(ultima.event_at) if ultima and ultima.event_at else "Nunca"
353 |             tabela.setItem(i, 4, QTableWidgetItem(rev))
354 |
355 |         tabela.setSelectionBehavior(QAbstractItemView.SelectRows)
356 |         tabela.setEditTriggers(QAbstractItemView.NoEditTriggers)
357 |         tabela.verticalHeader().setVisible(False)

```

```

358 |         tabela.verticalHeader().setDefaultSectionSize(44)
359 |         tornar_interativa(tabela)
360 |
361 |         def ao_duplo_clique(row, col):
362 |             self.signal_trocar_pagina.emit(1)
363 |             dialog.accept()
364 |
365 |         tabela.cellDoubleClicked.connect(ao_duplo_clique)
366 |         layout.addWidget(tabela)
367 |
368 |         btn_fechar = QPushButton("Fechar")
369 |         btn_fechar.setCursor(Qt.PointingHandCursor)
370 |         btn_fechar.clicked.connect(dialog.accept)
371 |         layout.addWidget(btn_fechar, alignment=Qt.AlignCenter)
372 |
373 |         dialog.exec()

```

=== ARQUIVO: .\app\views\pages\monitoring_page.py ===

```

001 | from __future__ import annotations
002 |
003 | import logging
004 | from datetime import datetime as dt
005 | from typing import Any
006 |
007 | from PySide6.QtCore import Qt, QTimer
008 | from PySide6.QtGui import QColor, QPixmap
009 | from PySide6.QtWidgets import (
010 |     QComboBox,
011 |     QFrame,
012 |     QHBoxLayout,
013 |     QHeaderView,
014 |     QLabel,
015 |     QMessageBox,
016 |     QPushButton,
017 |     QSpinBox,
018 |     QTableWidgetItem,
019 |     QVBoxLayout,
020 |     QWidget,
021 | )
022 | from sqlalchemy.orm import Session
023 |
024 | from config import BASE_DIR
025 | from app.services.snmp_service import SnmpMonitorService
026 | from app.views.styles.theme import (
027 |     ESTILO_BOTAO_PRIMARIO,
028 |     ESTILO_BOTAO_SECUNDARIO,
029 |     ESTILO_BOTAO_SUCESSO,
030 |     ESTILO_INPUT,
031 |     ESTILO_LABEL_CAMPO,
032 |     ESTILO_SUBTITULO,
033 |     ESTILO_TITULO_PAGINA,
034 |     configurar_combo,
035 | )
036 | from app.views.widgets.table_widget import TabelaPadrao
037 |
038 | log = logging.getLogger(__name__)
039 |
040 |
041 | class MonitoringPage(QWidget):
042 |     def __init__(self, session: Session, monitor_service: SnmpMonitorService | None = None, parent: QWidget

```

```

043 |         super().__init__(parent)
044 |         self.session = session
045 |         self.monitor_service = monitor_service or SnmpMonitorService(session)
046 |         self.config = self.monitor_service.config
047 |
048 |         self._timer: QTimer | None = None
049 |         self._em_execucao = False
050 |
051 |         layout = QVBoxLayout(self)
052 |         layout.setContentsMargins(24, 24, 24, 24)
053 |         layout.setSpacing(16)
054 |
055 |         # Header
056 |         header = QHBoxLayout()
057 |         header.setSpacing(10)
058 |
059 |         icone_label = QLabel()
060 |         pixmap = QPixmap(str(BASE_DIR / "anexos" / "monitoramento_icon.png"))
061 |         if not pixmap.isNull():
062 |             pixmap_redimensionado = pixmap.scaled(28, 28, Qt.KeepAspectRatio, Qt.SmoothTransformation)
063 |             icone_label.setPixmap(pixmap_redimensionado)
064 |         icone_label.setStyleSheet("background: transparent; border: none;")
065 |
066 |         titulo_col = QVBoxLayout()
067 |         titulo_col.setSpacing(2)
068 |         titulo = QLabel("Monitoramento de Rede")
069 |         titulo.setStyleSheet(ESTILO_TITULO_PAGINA)
070 |         titulo_col.addWidget(titulo)
071 |         sub = QLabel("Monitore status, toner e páginas das impressoras via rede")
072 |         sub.setStyleSheet(ESTILO_SUBTITULO)
073 |         titulo_col.addWidget(sub)
074 |
075 |         header.addWidget(icone_label)
076 |         header.addLayout(titulo_col)
077 |         header.addStretch()
078 |
079 |         self.btn_alternar = QPushButton("■ Iniciar")
080 |         self.btn_alternar.setCursor(Qt.PointingHandCursor)
081 |         self.btn_alternar.setStyleSheet(ESTILO_BOTAO_PRIMARIO)
082 |         self.btn_alternar.clicked.connect(self._alternar)
083 |         header.addWidget(self.btn_alternar)
084 |
085 |         self.btn_verificar_agora = QPushButton("■ Verificar Agora")
086 |         self.btn_verificar_agora.setCursor(Qt.PointingHandCursor)
087 |         self.btn_verificar_agora.setStyleSheet(ESTILO_BOTAO_SECUNDARIO)
088 |         self.btn_verificar_agora.clicked.connect(self._verificar_agora)
089 |         header.addWidget(self.btn_verificar_agora)
090 |         layout.addLayout(header)
091 |
092 |         # Stats bar
093 |         stats_bar = QHBoxLayout()
094 |         stats_bar.setSpacing(20)
095 |
096 |         self._stats_online = self._stat_card("■ Online", "0")
097 |         self._stats_offline = self._stat_card("■ Offline", "0")
098 |         self._stats_toner = self._stat_card("■ Toner Baixo", "0")
099 |         self._stats_total = self._stat_card("■ Total", "0")
100 |
101 |         stats_bar.addWidget(self._stats_online)
102 |         stats_bar.addWidget(self._stats_offline)
103 |         stats_bar.addWidget(self._stats_toner)
104 |         stats_bar.addWidget(self._stats_total)
105 |         layout.addLayout(stats_bar)

```


[illegible]

[illegible]

```

232 | self.config.salvar()
233 | if self._timer:
234 |     self._timer.stop()
235 | self.btn_alternar.setText("■ Iniciar")
236 | self.btn_alternar.setStyleSheet(ESTILO_BOTAO_PRIMARIO)
237 | log.info("Monitoramento pausado")
238 |
239 | def _verificar_agora(self) -> None:
240 |     if self._em_execucao:
241 |         return
242 |     self._em_execucao = True
243 |     self.btn_verificar_agora.setEnabled(False)
244 |     self.btn_verificar_agora.setText("■ Verificando...")
245 |     try:
246 |         self.monitor_service.verificar_todas()
247 |         self._atualizar_tabela()
248 |     except Exception as e:
249 |         log.exception("Erro na verificacao de rede")
250 |     finally:
251 |         self._em_execucao = False
252 |         self.btn_verificar_agora.setEnabled(True)
253 |         self.btn_verificar_agora.setText("■ Verificar Agora")
254 |
255 | # ■■ Tabela ████████████████████████████████████████████████████████████
256 |
257 | def _atualizar_tabela(self) -> None:
258 |     from app.models.printer import Printer
259 |     impressoras = self.session.query(Printer).all()
260 |     online = offline = toner_baixo = 0
261 |
262 |     self.tabela.setRowCount(len(impressoras))
263 |     for i, p in enumerate(impressoras):
264 |         status = self.monitor_service.ultima_leitura(p.id)
265 |
266 |         self.tabela.setItem(i, 0, QTableWidgetItem(p.patrimonio or "-"))
267 |         self.tabela.setItem(i, 1, QTableWidgetItem(p.modelo or "-"))
268 |         host = p.ip_rede or "-"
269 |         self.tabela.setItem(i, 2, QTableWidgetItem(host))
270 |
271 |         if status:
272 |             if status.is_online:
273 |                 online += 1
274 |                 status_text = "■ Online"
275 |                 status_cor = "#a6e3a1"
276 |             else:
277 |                 offline += 1
278 |                 status_text = "■ Offline"
279 |                 status_cor = "#f38ba8"
280 |
281 |             if status.toner_level is not None and status.toner_max:
282 |                 pct = (status.toner_level / status.toner_max) * 100
283 |                 if pct < self.config.toner_warn_percent:
284 |                     toner_baixo += 1
285 |
286 |             toner_txt = f"{status.toner_level or '-'}/{status.toner_max or '-'}" if status.toner_level
287 |
288 |             ping_txt = f"{status.ping_ms:.0f}ms" if status.ping_ms is not None else "-"
289 |             ultima_txt = status.timestamp.strftime("%d/%m %H:%M") if status.timestamp else "-"
290 |         else:
291 |             status_text = "■ Desconhecido"
292 |             status_cor = "#717182"
293 |             toner_txt = "-"
294 |             ping_txt = "-"

```

```

295 |         ultima_txt = "-"
296 |
297 |         status_item = QTableWidgetItem(status_text)
298 |         status_item.setForeground(QColor(status_cor))
299 |         self.tabela.setItem(i, 3, status_item)
300 |         self.tabela.setItem(i, 4, QTableWidgetItem(ping_txt))
301 |         self.tabela.setItem(i, 5, QTableWidgetItem(toner_txt))
302 |         self.tabela.setItem(i, 6, QTableWidgetItem(ultima_txt))
303 |
304 |     self.tabela.redimensionar()
305 |
306 |     self._stats_online.findChildren(QLabel)[1].setText(str(online))
307 |     self._stats_offline.findChildren(QLabel)[1].setText(str(offline))
308 |     self._stats_toner.findChildren(QLabel)[1].setText(str(toner_baixo))
309 |     self._stats_total.findChildren(QLabel)[1].setText(str(len(impressoras)))
310 |
311 |     def recarregar(self) -> None:
312 |         self._atualizar_tabela()

```

=== ARQUIVO: .lapp\views\pages\os_page.py ===

```

001 | from __future__ import annotations
002 |
003 | import json
004 | import logging
005 | import os
006 | import shutil
007 | from datetime import datetime as dt
008 | from collections.abc import Callable
009 | from pathlib import Path
010 | from typing import Any
011 |
012 | log = logging.getLogger(__name__)
013 |
014 | from PySide6.QtCore import QDate, QDateTime, Qt
015 | from PySide6.QtGui import QPixmap
016 | from PySide6.QtWidgets import (
017 |     QAbstractItemView,
018 |     QCheckBox,
019 |     QComboBox,
020 |     QDateEdit,
021 |     QDateTimeEdit,
022 |     QDialog,
023 |     QDialogButtonBox,
024 |     QFileDialog,
025 |     QFormLayout,
026 |     QGridLayout,
027 |     QHBoxLayout,
028 |     QHeaderView,
029 |     QLabel,
030 |     QLineEdit,
031 |     QMessageBox,
032 |     QPushButton,
033 |     QScrollArea,
034 |     QTableWidgetItem,
035 |     QTableWidgetItem,
036 |     QTabWidget,
037 |     QStackedWidget,
038 |     QTextEdit,
039 |     QVBoxLayout,
040 |     QWidget,

```

```

041 | )
042 |
043 | from app.models import Attachment, Part
044 | from app.services.part_service import PartService
045 | from app.utils.ui_helpers import tratar_erro
046 | from app.utils.helpers import formatar_data_hora
047 | from db import transacao
048 | from app.utils.constants import STATUS_ATIVIDADE_OPCOES
049 | from app.views.styles.theme import (
050 |     ATIVIDADE_CORES,
051 |     ESTILO_BOTAO_AVISO,
052 |     ESTILO_BOTAO_ERRO,
053 |     ESTILO_BOTAO_FECHAR,
054 |     ESTILO_BOTAO_PRIMARIO,
055 |     ESTILO_BOTAO_SECUNDARIO,
056 |     ESTILO_BOTAO_SUCESSO,
057 |     ESTILO_COMBO,
058 |     ESTILO_DIALOG,
059 |     ESTILO_INPUT,
060 |     ESTILO_SUBTITULO,
061 |     ESTILO_TABELA_SIMPLES,
062 |     ESTILO_TITULO_PAGINA,
063 |     URGENCIAS,
064 |     URGENCIA_CORES,
065 |     campo_readonly,
066 |     campo_rotulo,
067 |     configurar_combo,
068 |     configurar_combo_colorido,
069 |     group_box,
070 |     input_label,
071 | )
072 | from app.views.widgets import ToastManager
073 | from app.views.widgets.card_widget import CardMiniClicavel
074 | from app.views.widgets.confirm_dialog import ConfirmacaoDigitarDialog
075 | from app.views.widgets.pagination import PaginacaoWidget
076 | from app.views.widgets.search_bar import SearchBar
077 | from app.views.widgets.table_widget import TabelaPadrao, tornar_interativa
078 | from db import safe_commit as _safe_commit
079 |
080 | from config import ANEXOS_DIR
081 |
082 |
083 | def _criar_mascara_data(le: QLineEdit) -> Callable[[str], None]:
084 |     """Retorna callback que formata dd/mm/aaaa automaticamente no QLineEdit dado."""
085 |     def _mascarar(texto: str) -> None:
086 |         old_pos = le.cursorPosition()
087 |         digits = ''.join(c for c in texto if c.isdigit())[:8]
088 |         if not digits and texto:
089 |             le.blockSignals(True)
090 |             le.clear()
091 |             le.blockSignals(False)
092 |             return
093 |         partes = [digits[:2]]
094 |         if len(digits) > 2:
095 |             partes.append(digits[2:4])
096 |         if len(digits) > 4:
097 |             partes.append(digits[4:8])
098 |         nova = '/'.join(partes)
099 |         if nova != texto:
100 |             le.blockSignals(True)
101 |             le.setText(nova)
102 |             le.blockSignals(False)
103 |             le.setCursorPosition(old_pos + (len(nova) - len(texto)))

```

[illegible]

```

167 |         self.btn_nova.setToolTip("Criar nova ordem de serviço")
168 |         self.btn_nova.clicked.connect(self._nova)
169 |         header.addWidget(self.btn_nova)
170 |
171 |         self.btn_manut = QPushButton("Manutenções")
172 |         self.btn_manut.setStyleSheet(ESTILO_BOTAO_SECUNDARIO)
173 |         self.btn_manut.setToolTip("Filtrar apenas manutenções")
174 |         self.btn_manut.clicked.connect(lambda: self._filtrar_tipo("MANUTENCAO"))
175 |         header.addWidget(self.btn_manut)
176 |
177 |         self.btn_mov = QPushButton("Movimentações")
178 |         self.btn_mov.setStyleSheet(ESTILO_BOTAO_SECUNDARIO)
179 |         self.btn_mov.setToolTip("Filtrar apenas movimentações")
180 |         self.btn_mov.clicked.connect(lambda: self._filtrar_tipo("MOVIMENTACAO"))
181 |         header.addWidget(self.btn_mov)
182 |
183 |         self.btn_todas = QPushButton("Todas")
184 |         self.btn_todas.setStyleSheet(ESTILO_BOTAO_SECUNDARIO)
185 |         self.btn_todas.setToolTip("Mostrar todas as ordens de serviço")
186 |         self.btn_todas.clicked.connect(lambda: self._filtrar_tipo("TODAS"))
187 |         header.addWidget(self.btn_todas)
188 |
189 |         self.btn_atualizar = QPushButton("Atualizar")
190 |         self.btn_atualizar.setStyleSheet(ESTILO_BOTAO_SECUNDARIO)
191 |         self.btn_atualizar.setToolTip("Recarregar lista de ordens de serviço")
192 |         self.btn_atualizar.clicked.connect(self.recarregar)
193 |         header.addWidget(self.btn_atualizar)
194 |
195 |         layout.addLayout(header)
196 |
197 |         self.search = SearchBar(placeholder="Buscar por patrimônio, descrição...", glass=True)
198 |         self.search.textChanged().connect(self._filtrar_busca)
199 |         layout.addWidget(self.search)
200 |
201 |         cards = QHBoxLayout()
202 |         cards.setSpacing(12)
203 |
204 |         self.card_total = CardMiniClicavel("\u0001f4ca", "Total", "0", "#6366f1",
205 |                                             ao_clicar=lambda: self._filtrar_tipo("TODAS"))
206 |         cards.addWidget(self.card_total)
207 |
208 |         self.card_andamento = CardMiniClicavel("\u0001f504", "Em Atendimento", "0", "#6366f1",
209 |                                                  ao_clicar=lambda: self._filtrar_status("Em Atendimento"))
210 |         cards.addWidget(self.card_andamento)
211 |
212 |         self.card_pendentes = CardMiniClicavel("\u23f3", "Aberta", "0", "#94a3b8",
213 |                                                  ao_clicar=lambda: self._filtrar_status("Aberta"))
214 |         cards.addWidget(self.card_pendentes)
215 |
216 |         self.card_concluidas = CardMiniClicavel("\u2705", "Concluído", "0", "#10b981",
217 |                                                  ao_clicar=lambda: self._filtrar_status("Concluído"))
218 |         cards.addWidget(self.card_concluidas)
219 |
220 |         layout.addLayout(cards)
221 |
222 |         self.tabela = TabelaPadrao(["Data/Hora", "Patrimônio", "Tipo", "Descrição", "Peças", "Urgência", "O
223 |         self.tabela.cellDoubleClicked.connect(self._detalhes)
224 |         layout.addWidget(self.tabela)
225 |
226 |         self._paginacao = PaginacaoWidget()
227 |         self._paginacao.pagina_alterada.connect(lambda p: self._carregar())
228 |         layout.addWidget(self._paginacao)
229 |

```

```

230 |         self._filtro_status_atual = None
231 |         self._filtro_busca_atual = None
232 |
233 |     def recarregar(self, filtro_tipo: str | None = None) -> None:
234 |         if filtro_tipo:
235 |             self._filtro_tipo_atual = filtro_tipo
236 |             self._filtro_status_atual = None
237 |             self._filtro_busca_atual = None
238 |             self._carregar()
239 |
240 |     def _carregar(self) -> None:
241 |         if self._filtro_status_atual:
242 |             atividades = self.activity_service.listar_por_status(
243 |                 self._filtro_status_atual,
244 |                 limite=self._paginacao.limit, offset=self._paginacao.offset,
245 |             )
246 |             total = self.activity_service.contar_por_status(self._filtro_status_atual)
247 |         elif self._filtro_busca_atual:
248 |             atividades = self.activity_service.buscar_por_filtro_busca(
249 |                 self._filtro_busca_atual,
250 |                 limite=self._paginacao.limit, offset=self._paginacao.offset,
251 |             )
252 |             total = len(atividades)
253 |         else:
254 |             atividades = self.activity_service.listar(
255 |                 filtro_tipo=self._filtro_tipo_atual,
256 |                 limite=self._paginacao.limit, offset=self._paginacao.offset,
257 |             )
258 |             total = self.activity_service.contar_total()
259 |         self._atividades = atividades
260 |         self._paginacao.configurar(total, pagina_atual=self._paginacao.pagina,
261 |                                     itens_por_pagina=self._paginacao.limit)
262 |         self._preencher_tabela(atividades)
263 |         self._atualizar_cards()
264 |
265 |     def _preencher_tabela(self, atividades: list[Any]) -> None:
266 |         self.tabela.limpar()
267 |         if not atividades:
268 |             return
269 |
270 |         printer_ids = list({a.printer_id for a in atividades if a.printer_id})
271 |         mapa_pat = self.printer_service.mapa_patrimonio(printer_ids)
272 |
273 |         self.tabela.setRowCount(len(atividades))
274 |         for row, atv in enumerate(atividades):
275 |             patrimonio = mapa_pat.get(atv.printer_id, atv.printer_id[:8] if atv.printer_id else "-")
276 |             cor_tipo = "#f59e0b" if atv.kind == "MANUTENCAO" else "#6366f1"
277 |
278 |             data_item = QTableWidgetItem(formatar_data_hora(atv.event_at))
279 |             data_item.setTextAlignment(Qt.AlignCenter)
280 |
281 |             pat_item = QTableWidgetItem(patrimonio)
282 |             pat_item.setTextAlignment(Qt.AlignCenter)
283 |
284 |             cor_kind = "#f59e0b" if atv.kind == "MANUTENCAO" else "#6366f1"
285 |             self.tabela.definir_badge(row, 2, atv.kind or "-", cor_kind)
286 |
287 |             desc_item = QTableWidgetItem(atv.notes or "-")
288 |
289 |             pecas_item = QTableWidgetItem(atv.parts_used or "-")
290 |
291 |             urg_val = getattr(atv, 'urgencia', '') or 'Normal'
292 |             cor_urg = URGENCIA_CORES.get(urg_val, "#717182")

```



```

293 |         self.tabela.definir_badge(row, 5, urg_val, cor_urg)
294 |
295 |         orig_item = QTableWidgetItem(atv.from_location or "-")
296 |         orig_item.setTextAlignment(Qt.AlignCenter)
297 |
298 |         dest_item = QTableWidgetItem(atv.to_location or "-")
299 |         dest_item.setTextAlignment(Qt.AlignCenter)
300 |
301 |         nome_tecnico = "-"
302 |         if atv.tecnico_id and self.technician_service:
303 |             tec = self.technician_service.buscar_por_id(atv.tecnico_id)
304 |             if tec:
305 |                 nome_tecnico = tec.nome_exibicao
306 |         tecnico_item = QTableWidgetItem(nome_tecnico)
307 |         tecnico_item.setTextAlignment(Qt.AlignCenter)
308 |
309 |         tem_vinculo = bool(atv.os_vinculada_id) or bool(atv.oses_filhas)
310 |         self.tabela.definir_badge(row, 9, "■" if tem_vinculo else "", "#818cf8" if tem_vinculo else "tr
311 |
312 |         self.tabela.setItem(row, 0, data_item)
313 |         self.tabela.setItem(row, 1, pat_item)
314 |         self.tabela.setItem(row, 3, desc_item)
315 |         self.tabela.setItem(row, 4, pecas_item)
316 |         self.tabela.setItem(row, 6, orig_item)
317 |         self.tabela.setItem(row, 7, dest_item)
318 |         self.tabela.setItem(row, 8, tecnico_item)
319 |
320 |     self.tabela.redimensionar()
321 |
322 | def _filtrar_tipo(self, tipo: str) -> None:
323 |     self.recarregar(filtro_tipo=tipo)
324 |
325 | def _filtrar_status(self, status: str) -> None:
326 |     self._filtro_status_atual = status
327 |     self._filtro_busca_atual = None
328 |     self._carregar()
329 |
330 | def _filtrar_busca(self, texto: str) -> None:
331 |     self._filtro_status_atual = None
332 |     self._filtro_busca_atual = texto.strip() if texto.strip() else None
333 |     self._carregar()
334 |
335 | def _atualizar_cards(self) -> None:
336 |     total = self.activity_service.contar_total()
337 |     andamento = self.activity_service.contar_por_status("Em Atendimento")
338 |     pendentes = self.activity_service.contar_por_status("Aberta")
339 |     concluidas = self.activity_service.contar_por_status("Concluido")
340 |     self.card_total.atualizar_valor(total)
341 |     self.card_andamento.atualizar_valor(andamento)
342 |     self.card_pendentes.atualizar_valor(pendentes)
343 |     self.card_concluidas.atualizar_valor(concluidas)
344 |
345 | def _nova(self) -> None:
346 |     dialog, campos = self._criar_form_dialog("Nova OS")
347 |     if dialog.exec() != QDialog.Accepted:
348 |         return
349 |
350 |     patrimonio = campos["printer"].currentText().strip()
351 |     if not patrimonio:
352 |         QMessageBox.warning(self, "Aviso", "Selecione uma impressora.")
353 |         return
354 |
355 |     printer = self.printer_service.buscar_por_patrimonio(patrimonio)

```

```

356 |         if not printer:
357 |             QMessageBox.warning(self, "Aviso", f"Impressora '{patrimonio}' não encontrada.")
358 |             return
359 |
360 |         kind = campos["tipo"].currentText()
361 |         event_at = campos["data"].dateTime().toPython()
362 |         notes = campos["descricao"].toPlainText().strip()
363 |         sintoma_relatoado = campos["sintoma"].toPlainText().strip()
364 |         diagnostico_tecnico = campos["diagnostico"].toPlainText().strip()
365 |         solucao_aplicada = campos["solucao"].toPlainText().strip()
366 |         from_location = campos["origem"].currentText().strip()
367 |         to_location = campos["destino"].currentText().strip()
368 |         status_atividade = campos["status"].currentText()
369 |         urgencia = campos["urgencia"].currentText()
370 |
371 |         from_company_id = self._resolver_empresa(from_location)
372 |         to_company_id = self._resolver_empresa(to_location)
373 |         tecnico_id = self._resolver_tecnico(campos["tecnico"].currentText().strip())
374 |
375 |         inicio_qdt = campos["inicio"].dateTime()
376 |         fim_qdt = campos["fim"].dateTime()
377 |         inicio_atendimento = inicio_qdt.toPython() if inicio_qdt.isValid() else None
378 |         fim_atendimento = fim_qdt.toPython() if fim_qdt.isValid() else None
379 |         parts_used = campos["pecas"].toPlainText().strip()
380 |         os_vinculada_id = campos["os_vinculada"].currentData()
381 |
382 |         tbl = campos.get("checklist")
383 |         procedimentos = ""
384 |         if tbl:
385 |             dados = []
386 |             for r in range(tbl.rowCount()):
387 |                 chk = tbl.item(r, 0)
388 |                 nome = tbl.item(r, 1)
389 |                 if nome and nome.text().strip():
390 |                     dados.append({"nome": nome.text().strip(), "feito": bool(chk and chk.checkState() == Qt.Checked)})
391 |             procedimentos = json.dumps(dados, ensure_ascii=False)
392 |
393 |         try:
394 |             with transacao(self.session):
395 |                 atividade = self.activity_service.criar(
396 |                     printer_id=printer.id,
397 |                     kind=kind,
398 |                     notes=notes,
399 |                     parts_used=parts_used,
400 |                     sintoma_relatoado=sintoma_relatoado,
401 |                     diagnostico_tecnico=diagnostico_tecnico,
402 |                     solucao_aplicada=solucao_aplicada,
403 |                     inicio_atendimento=inicio_atendimento,
404 |                     fim_atendimento=fim_atendimento,
405 |                     from_location=from_location,
406 |                     to_location=to_location,
407 |                     status_atividade=status_atividade,
408 |                     event_at=event_at,
409 |                     tecnico_id=tecnico_id,
410 |                     procedimentos=procedimentos,
411 |                     urgencia=urgencia,
412 |                     os_vinculada_id=os_vinculada_id,
413 |                 )
414 |             self._criar_alerta_urgencia(printer, urgencia)
415 |             self._dar_baixa_estoque(parts_used, atividade_id=atividade.id)
416 |             self.activity_service.atualizar(
417 |                 atividade,
418 |                 from_company_id=from_company_id,

```

```

419 |         to_company_id=to_company_id,
420 |     )
421 |     fotos_pendentes = getattr(dialog, "pending_fotos", [])
422 |     for file_path, categoria in fotos_pendentes:
423 |         original_name = os.path.basename(file_path)
424 |         timestamp = dt.now().strftime("%Y%m%d_%H%M%S")
425 |         ext = os.path.splitext(original_name)[1]
426 |         stored_name = f"activity_{atividade.id}_{timestamp}_{original_name}"
427 |         dest = ANEXOS_DIR / stored_name
428 |         dest.parent.mkdir(parents=True, exist_ok=True)
429 |         shutil.copy2(file_path, str(dest))
430 |         anexo = Attachment(
431 |             entity_type="activity",
432 |             entity_id=atividade.id,
433 |             original_name=original_name,
434 |             file_path=str(dest),
435 |             categoria=categoria,
436 |         )
437 |         self.session.add(anexo)
438 |     self.recarregar()
439 | except Exception as e:
440 |     QMessageBox.critical(self, "Erro", f"Erro ao criar OS: {e}")
441 |
442 | def editar_atividade_por_id(self, activity_id: int) -> None:
443 |     atividade = self.activity_service.buscar_por_id(activity_id)
444 |     if not atividade:
445 |         return
446 |     self._atividades = [atividade]
447 |     self._preencher_tabela(self._atividades)
448 |     self._abrir_edicao(atividade)
449 |
450 | def _concluir_os_dialog(self, atividade: Any) -> None:
451 |     printer = self.printer_service.buscar_por_id(atividade.printer_id)
452 |     printer_str = f"{printer.patrimonio} - {printer.modelo}" if printer else "-"
453 |
454 |     dialog = QDialog(self)
455 |     dialog.setWindowTitle("Concluir OS")
456 |     dialog.setMinimumSize(500, 480)
457 |     dialog.setStyleSheet(ESTILO_DIALOG)
458 |
459 |     root = QVBoxLayout(dialog)
460 |     root.setContentsMargins(0, 0, 0, 0)
461 |     root.setSpacing(0)
462 |
463 |     scroll = QScrollArea()
464 |     scroll.setWidgetResizable(True)
465 |     scroll.setHorizontalScrollBarPolicy(Qt.ScrollBarAlwaysOff)
466 |     scroll.setStyleSheet("QScrollArea { border: none; background: transparent; }")
467 |
468 |     container = QWidget()
469 |     container.setStyleSheet("background: transparent;")
470 |     layout = QVBoxLayout(container)
471 |     layout.setContentsMargins(24, 20, 24, 20)
472 |     layout.setSpacing(20)
473 |
474 |     equip_box, equip_layout = group_box("Equipamento")
475 |     ef = QFormLayout()
476 |     ef.setSpacing(8)
477 |     ef.setLabelAlignment(Qt.AlignRight | Qt.AlignVCenter)
478 |     ef.addRow(campo_rotulo("Impressora"), campo_readonly(printer_str))
479 |     ef.addRow(campo_rotulo("Tipo"), campo_readonly(atividade.kind or "-"))
480 |     if atividade.tecnico_id and self.technician_service:
481 |         tec = self.technician_service.buscar_por_id(atividade.tecnico_id)

```

```

482 |         ef.addRow(campo_rotulo("Técnico"), campo_readonly(tec.nome_exibicao if tec else "-"))
483 |     equip_layout.addLayout(ef)
484 |     layout.addWidget(equip_box)
485 |
486 |     concluir_box, concluir_layout = group_box("Finalização")
487 |
488 |     edt_solucao = QTextEdit()
489 |     edt_solucao.setStyleSheet(ESTILO_INPUT)
490 |     edt_solucao.setMaximumHeight(80)
491 |     edt_solucao.setPlainText(atividade.solucao_aplicada or "")
492 |
493 |     edt_fim = QDateTimeEdit()
494 |     edt_fim.setCalendarPopup(True)
495 |     edt_fim.setDisplayFormat("dd/MM/yyyy HH:mm")
496 |     edt_fim.setStyleSheet(ESTILO_INPUT)
497 |     edt_fim.setDateTime(QDateTime.currentDateTime())
498 |
499 |     edt_pecas = QTextEdit()
500 |     edt_pecas.setStyleSheet(ESTILO_INPUT)
501 |     edt_pecas.setMaximumHeight(60)
502 |     edt_pecas.setPlainText(atividade.parts_used or "")
503 |
504 |     estoque_combo = QComboBox()
505 |     estoque_combo.setEditable(True)
506 |     configurar_combo(estoque_combo)
507 |     cmp = estoque_combo.completer()
508 |     if cmp:
509 |         cmp.setFilterMode(Qt.MatchFlag.MatchContains)
510 |         cmp.setCaseSensitivity(Qt.CaseSensitivity.CaseInsensitive)
511 |     estoque_combo.addItem("-- Nenhuma --", None)
512 |     for p in self.part_service.listar_todas():
513 |         if p.quantidade_estoque > 0:
514 |             estoque_combo.addItem(f"{p.nome} ({p.quantidade_estoque} un.)", p.id)
515 |
516 |     def _add_pecas():
517 |         idx = estoque_combo.currentIndex()
518 |         if idx <= 0:
519 |             return
520 |         pid = estoque_combo.currentData()
521 |         if pid is None:
522 |             return
523 |         part = self.part_service.buscar_por_id(pid)
524 |         if part:
525 |             atual = edt_pecas.toPlainText().strip()
526 |             edt_pecas.setPlainText(f"{part.nome}" if not atual else f"{atual}, {part.nome}")
527 |     estoque_combo.currentIndexChanged.connect(lambda i: _add_pecas() if i > 0 else None)
528 |
529 |     tbl = QTableWidgetItem()
530 |     tbl.setColumnCount(2)
531 |     tbl.setHorizontalHeaderLabels(["Feito", "Procedimento"])
532 |     tornar_interativa(tbl)
533 |     tbl.horizontalHeader().setSectionResizeMode(0, QHeaderView.ResizeToContents)
534 |     tbl.verticalHeader().setVisible(False)
535 |     tbl.verticalHeader().setDefaultSectionSize(32)
536 |     tbl.setSelectionBehavior(QAbstractItemView.SelectRows)
537 |     tbl.setSelectionMode(QAbstractItemView.SingleSelection)
538 |     tbl.setStyleSheet("""
539 |         QTableWidgetItem { background: transparent; border: none; }
540 |         QTableWidgetItem::item { padding: 4px; color: #e2e8f0; font-size: 13px; }
541 |         QHeaderView::section { background: transparent; color: #a0a0b0; border: none; font-weight: 600; }
542 |         QTableWidgetItem QLineEdit {
543 |             background-color: #1e1e2f !important;
544 |             color: #ffffff !important;

```

```

545 |         border: 1px solid #3b82f6 !important;
546 |         border-radius: 4px;
547 |         padding: 2px 6px !important;
548 |         font-size: 13px;
549 |     }
550 |     QCheckBox::indicator {
551 |         width: 18px; height: 18px;
552 |         border: 2px solid #3b82f6; border-radius: 4px;
553 |         background-color: #1e1e2f;
554 |     }
555 |     QCheckBox::indicator:checked {
556 |         background-color: #3b82f6;
557 |     }
558 | """)
559 | if atividade.procedimentos:
560 |     try:
561 |         dados = json.loads(atividade.procedimentos)
562 |         for item in dados:
563 |             r = tbl.rowCount()
564 |             tbl.insertRow(r)
565 |             chk = QTableWidgetItem()
566 |             chk.setFlags(chk.flags() | Qt.ItemIsUserCheckable)
567 |             chk.setCheckState(Qt.Checked if item.get("feito") else Qt.Unchecked)
568 |             tbl.setItem(r, 0, chk)
569 |             tbl.setItem(r, 1, QTableWidgetItem(item.get("nome", "")))
570 |     except json.JSONDecodeError:
571 |         pass
572 | else:
573 |     itens_padrao = [
574 |         "Limpeza de laser", "Lubrificação do fusor",
575 |         "Troca de película", "Troca de rolo de pressão", "Teste de impressão",
576 |     ]
577 |     for nome in itens_padrao:
578 |         r = tbl.rowCount()
579 |         tbl.insertRow(r)
580 |         chk = QTableWidgetItem()
581 |         chk.setFlags(chk.flags() | Qt.ItemIsUserCheckable)
582 |         chk.setCheckState(Qt.Unchecked)
583 |         tbl.setItem(r, 0, chk)
584 |         tbl.setItem(r, 1, QTableWidgetItem(nome))
585 |
586 | btn_add = QPushButton("+ Adicionar")
587 | btn_add.setStyleSheet(ESTILO_BOTAO_SECUNDARIO)
588 | btn_add.setToolTip("Adicionar procedimento")
589 | btn_add.setCursor(Qt.PointingHandCursor)
590 | btn_rem = QPushButton("- Remover")
591 | btn_rem.setStyleSheet(ESTILO_BOTAO_ERRO)
592 | btn_rem.setToolTip("Remover selecionado")
593 | btn_rem.setCursor(Qt.PointingHandCursor)
594 |
595 | def _add():
596 |     r = tbl.rowCount()
597 |     tbl.insertRow(r)
598 |     chk = QTableWidgetItem()
599 |     chk.setFlags(chk.flags() | Qt.ItemIsUserCheckable)
600 |     chk.setCheckState(Qt.Unchecked)
601 |     tbl.setItem(r, 0, chk)
602 |     item_texto = QTableWidgetItem("")
603 |     tbl.setItem(r, 1, item_texto)
604 |     tbl.setCurrentCell(r, 1)
605 |     tbl.editItem(item_texto)
606 |
607 | def _rem():

```

```

608 |         r = tbl.currentRow()
609 |         if r >= 0:
610 |             tbl.removeRow(r)
611 |
612 | btn_add.clicked.connect(_add)
613 | btn_rem.clicked.connect(_rem)
614 |
615 | chk_agendar = QCheckBox("Agendar próxima manutenção da impressora")
616 | chk_agendar.setStyleSheet("color: #e2e8f0; font-size: 12px; spacing: 8px; margin-top: 4px;")
617 | chk_agendar.setToolTip("Define a data da próxima manutenção programada para esta impressora")
618 | chk_agendar.setChecked(True)
619 |
620 | edt_prox = QDateEdit()
621 | edt_prox.setCalendarPopup(True)
622 | edt_prox.setDisplayFormat("dd/MM/yyyy")
623 | edt_prox.setDate(QDate.currentDate().addMonths(3))
624 | edt_prox.setStyleSheet(ESTILO_INPUT)
625 | edt_prox.setEnabled(True)
626 | chk_agendar.toggled.connect(edt_prox.setEnabled)
627 |
628 | grid = QGridLayout()
629 | grid.setSpacing(8)
630 | grid.setHorizontalSpacing(16)
631 | grid.addWidget(input_label("Solução Aplicada"), 0, 0, 1, 2)
632 | grid.addWidget(edt_solucao, 1, 0, 1, 2)
633 | grid.addWidget(input_label("Fim Atendimento"), 2, 0)
634 | grid.addWidget(edt_fim, 3, 0)
635 | grid.addWidget(input_label("Peças Trocadas"), 4, 0, 1, 2)
636 | grid.addWidget(edt_pecas, 5, 0, 1, 2)
637 | grid.addWidget(input_label("Peça do Estoque"), 6, 0, 1, 2)
638 | grid.addWidget(estoque_combo, 7, 0, 1, 2)
639 | btn_reservar = QPushButton("■ Reservar")
640 | btn_reservar.setStyleSheet(ESTILO_BOTAO_SECUNDARIO)
641 | btn_reservar.setToolTip("Reservar esta peça para esta OS (baixa no estoque)")
642 | def _reservar():
643 |     idx = estoque_combo.currentIndex()
644 |     if idx <= 0:
645 |         return
646 |     pid = estoque_combo.currentData()
647 |     if pid is None:
648 |         return
649 |     with tratar_erro("reservar peça"):
650 |         res = self.part_service.criar_reserva(pid, atividade.id, quantidade=1)
651 |         if res:
652 |             ToastManager.mostrar(f"Peça reservada para OS #{atividade.id}.", "sucesso")
653 |             _atualizar_reservas()
654 |             estoque_combo.setCurrentIndex(0)
655 |             self.recarregar()
656 |         else:
657 |             ToastManager.mostrar("Estoque insuficiente para reservar.", "erro")
658 | btn_reservar.clicked.connect(_reservar)
659 | grid.addWidget(btn_reservar, 8, 0, 1, 2)
660 |
661 | # ■■ Reservas existentes ■■
662 | reservas_box, reservas_layout = group_box("Peças Reservadas")
663 | reservas_tabela = QTableWidgetItem()
664 | reservas_tabela.setColumnCount(5)
665 | reservas_tabela.setHorizontalHeaderLabels(["Peça", "Qtd", "Status", "Data", ""])
666 | reservas_tabela.setStyleSheet(ESTILO_TABELA_SIMPLES)
667 | reservas_tabela.setSelectionBehavior(QTableWidgetItem.SelectRows)
668 | reservas_tabela.setEditTriggers(QTableWidgetItem.NoEditTriggers)
669 | reservas_tabela.verticalHeader().setVisible(False)
670 | reservas_tabela.setAlternatingRowColors(True)

```

[illegible]

```

734 |     def _atualizar_galeria_concluir():
735 |         old = dialog._fotos_box_concluir
736 |         idx = layout.indexOf(old)
737 |         if idx >= 0:
738 |             layout.removeWidget(old)
739 |             old.deleteLater()
740 |         dialog._fotos_box_concluir, _ = self._criar_grupo_fotos(atividade.id)
741 |         layout.insertWidget(idx if idx >= 0 else layout.count(), dialog._fotos_box_concluir)
742 |
743 |     dialog.refresh_gallery_cb = _atualizar_galeria_concluir
744 |
745 |     fotos_upload_layout = QHBoxLayout()
746 |     fotos_upload_layout.setSpacing(8)
747 |     cmb_cat_concluir = QComboBox()
748 |     configurar_combo(cmb_cat_concluir)
749 |     cmb_cat_concluir.addItem(["antes", "depois", "peca"])
750 |     cmb_cat_concluir.setMinimumWidth(140)
751 |     btn_foto_concluir = QPushButton("■ Adicionar Foto")
752 |     btn_foto_concluir.setStyleSheet(ESTILO_BOTAO_SECUNDARIO)
753 |     btn_foto_concluir.setToolTip("Adicionar foto antes de concluir")
754 |     btn_foto_concluir.setCursor(Qt.PointingHandCursor)
755 |     btn_foto_concluir.clicked.connect(
756 |         lambda: (self._anexar_arquivo(atividade.id, cmb_cat_concluir.currentText(), dialog),
757 |                 _atualizar_galeria_concluir())
758 |     )
759 |     fotos_upload_layout.addWidget(cmb_cat_concluir)
760 |     fotos_upload_layout.addWidget(btn_foto_concluir)
761 |     fotos_upload_layout.addStretch()
762 |     layout.addLayout(fotos_upload_layout)
763 |
764 |     layout.addStretch()
765 |     scroll.setWidget(container)
766 |     root.addWidget(scroll, stretch=1)
767 |
768 |     btn_layout = QHBoxLayout()
769 |     btn_layout.setContentsMargins(24, 12, 24, 24)
770 |     btn_layout.setSpacing(10)
771 |
772 |     btn_concluir = QPushButton("■ Concluir OS")
773 |     btn_concluir.setStyleSheet(ESTILO_BOTAO_SUCESSO)
774 |     btn_concluir.setToolTip("Finalizar ordem de serviço")
775 |     btn_fechar = QPushButton("Fechar")
776 |     btn_fechar.setStyleSheet(ESTILO_BOTAO_FECHAR)
777 |     btn_fechar.setToolTip("Fechar sem concluir")
778 |
779 |     def _salvar():
780 |         solucao = edt_solucao.toPlainText().strip()
781 |         fim = edt_fim.dateTime().toPython()
782 |         pecas = edt_pecas.toPlainText().strip()
783 |
784 |         dados = []
785 |         for r in range(tbl.rowCount()):
786 |             chk = tbl.item(r, 0)
787 |             nome = tbl.item(r, 1)
788 |             if nome and nome.text().strip():
789 |                 dados.append({"nome": nome.text().strip(), "feito": bool(chk and chk.checkState() == Qt.Checked)})
790 |         procedimentos = json.dumps(dados, ensure_ascii=False)
791 |
792 |         try:
793 |             with transacao(self.session):
794 |                 self.activity_service.atualizar(
795 |                     atividade,
796 |                     solucao_aplicada=solucao,

```



```

860 |         mime_type=mime_map.get(ext, "application/octet-stream"),
861 |         size_bytes=os.path.getsize(file_path),
862 |         categoria=categoria,
863 |     )
864 |     self.session.add(attachment)
865 |     _safe_commit(self.session)
866 | except Exception as e:
867 |     self.session.rollback()
868 |     QMessageBox.critical(dialog, "Erro", f"Erro ao anexar arquivo:\n{e}")
869 |
870 | def _remover_anexo(self, anexo_id: int, dialog: QWidget) -> None:
871 |     if not ConfirmacaoDigitalDialog.confirmar(
872 |         "Remover Anexo", "Deseja realmente remover este anexo?",
873 |         dialog,
874 |     ):
875 |         return
876 |     try:
877 |         anexo = self.session.query(Attachment).filter_by(id=anexo_id).first()
878 |         if anexo:
879 |             if os.path.exists(anexo.file_path):
880 |                 os.remove(anexo.file_path)
881 |                 self.session.delete(anexo)
882 |                 _safe_commit(self.session)
883 |     except Exception as e:
884 |         self.session.rollback()
885 |         QMessageBox.critical(dialog, "Erro", f"Erro ao remover anexo:\n{e}")
886 |
887 | def _abrir_anexo(self, file_path: str) -> None:
888 |     if os.path.exists(file_path):
889 |         try:
890 |             os.startfile(file_path)
891 |         except Exception as e:
892 |             QMessageBox.warning(self, "Aviso", f"Não foi possível abrir o arquivo:\n{e}")
893 |     else:
894 |         QMessageBox.warning(self, "Aviso", "Arquivo não encontrado:\n" + file_path)
895 |
896 | def _criar_grupo_fotos(self, activity_id: int | None = None, *,
897 |     pending_fotos: list[tuple[str, str]] | None = None) -> tuple[QWidget, QVBoxLayout]:
898 |     fotos_box, fotos_layout = group_box("GALERIA DE FOTOS")
899 |
900 |     galeria_principal = QHBoxLayout()
901 |     galeria_principal.setSpacing(16)
902 |     galeria_principal.setContentsMargins(4, 4, 4, 4)
903 |
904 |     galeria_scroll = QScrollArea()
905 |     galeria_scroll.setWidgetResizable(True)
906 |     galeria_scroll.setHorizontalScrollBarPolicy(Qt.ScrollBarAsNeeded)
907 |     galeria_scroll.setVerticalScrollBarPolicy(Qt.ScrollBarAlwaysOff)
908 |     galeria_scroll.setStyleSheet("QScrollArea { border: none; background: transparent; }")
909 |     galeria_container = QWidget()
910 |     galeria_container.setStyleSheet("background: transparent;")
911 |     galeria_container.setLayout(galeria_principal)
912 |     galeria_scroll.setWidget(galeria_container)
913 |
914 |     categorias = {"antes": "Antes", "depois": "Depois", "peca": "Peça Quebrada"}
915 |     subtitulos = {"antes": "Antes da manutenção", "depois": "Após a manutenção", "peca": "Peça com defeito"}
916 |
917 |     estilo_card = """
918 |         QWidget {
919 |             background-color: #111111;
920 |             border: 1px solid #2a2a3e;
921 |             border-radius: 8px;
922 |         }

```

```

923 |     """
924 |     estilo_titulo_card = "color: #ffffff; font-size: 16px; font-weight: bold; border: none; background:
925 |     estilo_sub_card = "color: #717182; font-size: 12px; border: none; background: transparent;"
926 |     estilo_thumb = "border-radius: 6px; border: none; background: #1e1e2f;"
927 |     estilo_btn_visualizar = """
928 |         QPushButton {
929 |             background-color: #3b82f6; color: white; border: none; border-radius: 6px;
930 |             padding: 10px; font-size: 13px; font-weight: bold;
931 |         }
932 |         QPushButton:hover { background-color: #2563eb; }
933 |     """
934 |     estilo_btn_excluir = """
935 |         QPushButton {
936 |             background-color: #ef4444; color: white; border: none; border-radius: 6px;
937 |             padding: 10px; font-size: 13px; font-weight: bold;
938 |         }
939 |         QPushButton:hover { background-color: #dc2626; }
940 |     """
941 |     estilo_seta = """
942 |         QPushButton {
943 |             background-color: transparent; color: #818cf8;
944 |             border: 1px solid #2a2a3e; border-radius: 6px;
945 |             font-size: 18px; min-width: 36px; min-height: 36px;
946 |         }
947 |         QPushButton:hover { background-color: rgba(99,102,241,0.15); border-color: #6366f1; }
948 |         QPushButton:disabled { color: #3a3a4e; border-color: #1e1e2e; }
949 |     """
950 |     estilo_contador = "color: #717182; font-size: 11px; background: transparent;"
951 |
952 |     def _criar_card(chave: str, rotulo: str) -> QWidget:
953 |         card = QWidget()
954 |         card.setStyleSheet(estilo_card)
955 |         card.setMinimumWidth(400)
956 |         layout_card = QVBoxLayout(card)
957 |         layout_card.setContentsMargins(16, 16, 16, 16)
958 |         layout_card.setSpacing(12)
959 |         lbl_cat = QLabel(rotulo.upper())
960 |         lbl_cat.setStyleSheet(estilo_titulo_card)
961 |         layout_card.addWidget(lbl_cat)
962 |         lbl_sub = QLabel(subtitulos.get(chave, ""))
963 |         lbl_sub.setStyleSheet(estilo_sub_card)
964 |         layout_card.addWidget(lbl_sub)
965 |         return card
966 |
967 |     def _montar_pagina(arquivo_path: str, btn_remove: QPushButton | None) -> tuple[QWidget, QPushButton]:
968 |         page = QWidget()
969 |         pl = QVBoxLayout(page)
970 |         pl.setContentsMargins(0, 0, 0, 0)
971 |         pl.setSpacing(10)
972 |         pm = QPixmap(arquivo_path)
973 |         thumb = QLabel()
974 |         thumb.setPixmap(pm.scaled(350, 250, Qt.KeepAspectRatio, Qt.SmoothTransformation))
975 |         thumb.setStyleSheet(estilo_thumb)
976 |         thumb.setAlignment(Qt.AlignCenter)
977 |         pl.addWidget(thumb)
978 |         bv = QPushButton("Visualizar Original em Tamanho Completo")
979 |         bv.setStyleSheet(estilo_btn_visualizar)
980 |         bv.setCursor(Qt.PointingHandCursor)
981 |         bv.clicked.connect(lambda checked, f=arquivo_path: self._abrir_anexo(f))
982 |         pl.addWidget(bv)
983 |         if btn_remove:
984 |             pl.addWidget(btn_remove)
985 |         return page, bv, btn_remove

```



```

1112 |         atv = self.activity_service.buscar_por_id(activity_id)
1113 |         if atv:
1114 |             self._atividades.append(atv)
1115 |             self._detalhes(len(self._atividades) - 1)
1116 |
1117 |     def _detalhes(self, row: int) -> None:
1118 |         if row < 0 or row >= len(self._atividades):
1119 |             return
1120 |         atividade = self._atividades[row]
1121 |
1122 |         printer_str = "-"
1123 |         if atividade.printer_id:
1124 |             printer = self.printer_service.buscar_por_id(atividade.printer_id)
1125 |             if printer:
1126 |                 printer_str = f"{printer.patrimonio} - {printer.modelo}"
1127 |
1128 |         tecnico_str = "-"
1129 |         if atividade.tecnico_id and self.technician_service:
1130 |             tec = self.technician_service.buscar_por_id(atividade.tecnico_id)
1131 |             if tec:
1132 |                 tecnico_str = tec.nome_exibicao
1133 |
1134 |         dialog = QDialog(self)
1135 |         dialog.setWindowTitle("Ordem de Serviço")
1136 |         dialog.setMinimumSize(650, 500)
1137 |         dialog.setStyleSheet(ESTILO_DIALOG)
1138 |
1139 |         root = QVBoxLayout(dialog)
1140 |         root.setContentsMargins(0, 0, 0, 0)
1141 |         root.setSpacing(0)
1142 |
1143 |         scroll = QScrollArea()
1144 |         scroll.setWidgetResizable(True)
1145 |         scroll.setHorizontalScrollBarPolicy(Qt.ScrollBarAlwaysOff)
1146 |         scroll.setStyleSheet("QScrollArea { border: none; background: transparent; }")
1147 |
1148 |         container = QWidget()
1149 |         container.setStyleSheet("background: transparent;")
1150 |         layout = QVBoxLayout(container)
1151 |         layout.setContentsMargins(24, 20, 24, 20)
1152 |         layout.setSpacing(20)
1153 |
1154 |         equip_box, equip_layout = group_box("Equipamento")
1155 |         equip_form = QFormLayout()
1156 |         equip_form.setSpacing(8)
1157 |         equip_form.setLabelAlignment(Qt.AlignRight | Qt.AlignVCenter)
1158 |         equip_form.addRow(campo_rotulo("Impressora"), campo_readonly(printer_str))
1159 |         equip_form.addRow(campo_rotulo("Tipo"), campo_readonly(atividade.kind or "-"))
1160 |         equip_form.addRow(campo_rotulo("Técnico"), campo_readonly(tecnico_str))
1161 |         equip_layout.addLayout(equip_form)
1162 |         layout.addWidget(equip_box)
1163 |
1164 |         def _formatar_tempo(diff):
1165 |             total_seg = int(diff.total_seconds())
1166 |             h = total_seg // 3600
1167 |             m = (total_seg % 3600) // 60
1168 |             if h > 0:
1169 |                 return f"{h}h {m}min"
1170 |             return f"{m}min"
1171 |
1172 |         tempo_str = "-"
1173 |         if atividade.inicio_atendimento and atividade.fim_atendimento:
1174 |             diff = atividade.fim_atendimento - atividade.inicio_atendimento

```



```

1238 |         if atividade.procedimentos:
1239 |             try:
1240 |                 procedimentos = json.loads(atividade.procedimentos)
1241 |                 checklist_box, checklist_layout = group_box("Procedimentos Realizados")
1242 |                 for item in procedimentos:
1243 |                     nome = item.get("nome", "")
1244 |                     feito = item.get("feito", False)
1245 |                     icon = "■" if feito else "□"
1246 |                     lbl = QLabel(f"{icon} {nome}")
1247 |                     lbl.setStyleSheet("color: #c8c8d8; font-size: 12px; padding: 2px 0; background: transparent")
1248 |                     checklist_layout.addWidget(lbl)
1249 |                 layout.addWidget(checklist_box)
1250 |             except json.JSONDecodeError:
1251 |                 pass
1252 |
1253 |         if atividade.kind == "MOVIMENTACAO":
1254 |             mov_box, mov_layout = group_box("Movimentação")
1255 |             mov_form = QFormLayout()
1256 |             mov_form.setSpacing(8)
1257 |             mov_form.setLabelAlignment(Qt.AlignRight | Qt.AlignVCenter)
1258 |             mov_form.addRow(campo_rotulo("Origem"), campo_readonly(atividade.from_location or "-"))
1259 |             mov_form.addRow(campo_rotulo("Destino"), campo_readonly(atividade.to_location or "-"))
1260 |             mov_layout.addLayout(mov_form)
1261 |             layout.addWidget(mov_box)
1262 |
1263 |         fotos_box, _ = self._criar_grupo_fotos(atividade.id)
1264 |         layout.addWidget(fotos_box)
1265 |
1266 |         # ■■ Peças Reservadas ■■
1267 |         reservas = self.part_service.reservas_por_os(atividade.id)
1268 |         if reservas:
1269 |             res_box, res_layout = group_box("Peças Reservadas")
1270 |             res_tabela = QTableWidgetItem()
1271 |             res_tabela.setColumnCount(4)
1272 |             res_tabela.setHorizontalHeaderLabels(["Peça", "Qtd", "Status", "Data"])
1273 |             res_tabela.setStyleSheet(ESTILO_TABELA_SIMPLES)
1274 |             res_tabela.setSelectionBehavior(QTableWidgetItem.SelectRows)
1275 |             res_tabela.setEditTriggers(QTableWidgetItem.NoEditTriggers)
1276 |             res_tabela.verticalHeader().setVisible(False)
1277 |             res_tabela.setAlternatingRowColors(True)
1278 |             res_tabela.verticalHeader().setDefaultSectionSize(28)
1279 |             tornar_interativa(res_tabela)
1280 |             for i, r in enumerate(reservas):
1281 |                 res_tabela.setItem(i, 0, QTableWidgetItem(r.part.nome if r.part else "-"))
1282 |                 item_q = QTableWidgetItem(str(r.quantidade))
1283 |                 item_q.setTextAlignment(Qt.AlignCenter)
1284 |                 res_tabela.setItem(i, 1, item_q)
1285 |                 res_tabela.setItem(i, 2, QTableWidgetItem(r.status))
1286 |                 res_tabela.setItem(i, 3, QTableWidgetItem(formatar_data_hora(r.created_at)))
1287 |             res_layout.addWidget(res_tabela)
1288 |             layout.addWidget(res_box)
1289 |
1290 |         layout.addStretch()
1291 |         scroll.setWidget(container)
1292 |         root.addWidget(scroll, stretch=1)
1293 |
1294 |         btn_layout = QHBoxLayout()
1295 |         btn_layout.setContentsMargins(24, 12, 24, 24)
1296 |         btn_layout.setSpacing(10)
1297 |
1298 |         btn_editar = QPushButton("■ Editar")
1299 |         btn_editar.setStyleSheet(ESTILO_BOTAO_AVISO)
1300 |         btn_editar.setToolTip("Editar esta ordem de serviço")

```



```

1301 |
1302 |     btn_excluir = QPushButton("■ Excluir")
1303 |     btn_excluir.setStyleSheet(ESTILO_BOTAO_ERR0)
1304 |     btn_excluir.setToolTip("Excluir esta ordem de serviço (pode ser desfeito pela Lixeira)")
1305 |
1306 |     pode_concluir = atividade.status_atividade not in ("Concluído", "Verificada")
1307 |     btn_concluir = QPushButton("■ Concluir OS")
1308 |     btn_concluir.setStyleSheet(ESTILO_BOTAO_SUCESSO)
1309 |     btn_concluir.setToolTip("Finalizar esta ordem de serviço")
1310 |     btn_concluir.setVisible(pode_concluir)
1311 |
1312 |     btn_fechar = QPushButton("Fechar")
1313 |     btn_fechar.setStyleSheet(ESTILO_BOTAO_FECHAR)
1314 |     btn_fechar.setToolTip("Fechar detalhes")
1315 |
1316 |     def excluir():
1317 |         if ConfirmacaoDigitalDialog.confirmar(
1318 |             "Confirmar", "Deseja realmente excluir esta OS?",
1319 |             dialog,
1320 |         ):
1321 |             try:
1322 |                 self.activity_service.excluir(atividade)
1323 |                 ToastManager.mostrar(
1324 |                     "OS excluída.",
1325 |                     "aviso", duracao=8000,
1326 |                     acao=("Desfazer", lambda o=atividade, svc=self.activity_service, pag=self: (svc.re
1327 |                 )
1328 |                 dialog.accept()
1329 |                 self.recarregar()
1330 |             except Exception as e:
1331 |                 QMessageBox.critical(dialog, "Erro", f"Erro ao excluir: {e}")
1332 |
1333 |     def editar():
1334 |         dialog.accept()
1335 |         self._abrir_edicao(atividade)
1336 |
1337 |     def concluir():
1338 |         dialog.accept()
1339 |         self._concluir_os_dialog(atividade)
1340 |
1341 |     btn_editar.clicked.connect(editar)
1342 |     btn_excluir.clicked.connect(excluir)
1343 |     btn_concluir.clicked.connect(concluir)
1344 |     btn_fechar.clicked.connect(dialog.accept)
1345 |
1346 |     btn_layout.addWidget(btn_concluir)
1347 |     btn_layout.addWidget(btn_editar)
1348 |     btn_layout.addWidget(btn_excluir)
1349 |     btn_layout.addStretch()
1350 |     btn_layout.addWidget(btn_fechar)
1351 |
1352 |     root.addLayout(btn_layout)
1353 |
1354 |     dialog.exec()
1355 |
1356 | def _abrir_edicao(self, atividade: Any) -> None:
1357 |     dialog, campos = self._criar_form_dialog("Editar OS", atividade)
1358 |
1359 |     botoes = QHBoxLayout()
1360 |     botoes.setContentsMargins(24, 12, 24, 24)
1361 |     botoes.setSpacing(10)
1362 |
1363 |     def _btn(texto: str, estilo: str, tooltip: str) -> QPushButton:

```

```

1364 |         btn = QPushButton(texto)
1365 |         btn.setStyleSheet(estilo)
1366 |         btn.setToolTip(tooltip)
1367 |         btn.setCursor(Qt.PointingHandCursor)
1368 |         return btn
1369 |
1370 |     btn_salvar = _btn("Salvar", ESTILO_BOTAO_SUCESSO, "Salvar alterações da ordem de serviço")
1371 |     btn_excluir = _btn("Excluir", ESTILO_BOTAO_ERRO, "Excluir esta ordem de serviço (pode ser desfeito")
1372 |     btn_cancelar = _btn("Cancelar", ESTILO_BOTAO_FECHAR, "Descartar alterações e fechar")
1373 |     botoes.addWidget(btn_salvar)
1374 |     botoes.addWidget(btn_excluir)
1375 |     botoes.addStretch()
1376 |     botoes.addWidget(btn_cancelar)
1377 |
1378 |     form_layout = dialog.layout()
1379 |     form_layout.addLayout(botoes)
1380 |
1381 |     resultado = {"acao": None}
1382 |
1383 |     def salvar():
1384 |         patrimonio = campos["printer"].currentText().strip()
1385 |         if not patrimonio:
1386 |             QMessageBox.warning(dialog, "Aviso", "Selecione uma impressora.")
1387 |             return
1388 |         printer = self.printer_service.buscar_por_patrimonio(patrimonio)
1389 |         if not printer:
1390 |             QMessageBox.warning(dialog, "Aviso", f"Impressora '{patrimonio}' não encontrada.")
1391 |             return
1392 |
1393 |         kind = campos["tipo"].currentText()
1394 |         event_at = campos["data"].dateTime().toPython()
1395 |         notes = campos["descricao"].toPlainText().strip()
1396 |         parts_used = campos["pecas"].toPlainText().strip()
1397 |         sintoma_relatoado = campos["sintoma"].toPlainText().strip()
1398 |         diagnostico_tecnico = campos["diagnostico"].toPlainText().strip()
1399 |         solucao_aplicada = campos["solucao"].toPlainText().strip()
1400 |         from_location = campos["origem"].currentText().strip()
1401 |         to_location = campos["destino"].currentText().strip()
1402 |         status_atividade = campos["status"].currentText()
1403 |         urgencia = campos["urgencia"].currentText()
1404 |         from_company_id = self._resolver_empresa(from_location)
1405 |         to_company_id = self._resolver_empresa(to_location)
1406 |         tecnico_id = self._resolver_tecnico(campos["tecnico"].currentText().strip())
1407 |         inicio_qdt = campos["inicio"].dateTime()
1408 |         fim_qdt = campos["fim"].dateTime()
1409 |         inicio_atendimento = inicio_qdt.toPython() if inicio_qdt.isValid() else None
1410 |         fim_atendimento = fim_qdt.toPython() if fim_qdt.isValid() else None
1411 |         os_vinculada_id = campos["os_vinculada"].currentData()
1412 |
1413 |         tbl = campos.get("checklist")
1414 |         procedimentos = ""
1415 |         if tbl:
1416 |             dados = []
1417 |             for r in range(tbl.rowCount()):
1418 |                 chk = tbl.item(r, 0)
1419 |                 nome = tbl.item(r, 1)
1420 |                 if nome and nome.text().strip():
1421 |                     dados.append({"nome": nome.text().strip(), "feito": bool(chk and chk.checkState())})
1422 |             procedimentos = json.dumps(dados, ensure_ascii=False)
1423 |
1424 |         try:
1425 |             with transacao(self.session):
1426 |                 self.activity_service.atualizar(

```

```

1427 |         atividade,
1428 |         printer_id=printer.id,
1429 |         kind=kind,
1430 |         event_at=event_at,
1431 |         notes=notes,
1432 |         parts_used=parts_used,
1433 |         sintoma_relatoado=sintoma_relatoado,
1434 |         diagnostico_tecnico=diagnostico_tecnico,
1435 |         solucao_aplicada=solucao_aplicada,
1436 |         inicio_atendimento=inicio_atendimento,
1437 |         fim_atendimento=fim_atendimento,
1438 |         from_location=from_location,
1439 |         to_location=to_location,
1440 |         status_atividade=status_atividade,
1441 |         urgencia=urgencia,
1442 |         from_company_id=from_company_id,
1443 |         to_company_id=to_company_id,
1444 |         tecnico_id=tecnico_id,
1445 |         procedimentos=procedimentos,
1446 |         os_vinculada_id=os_vinculada_id,
1447 |     )
1448 |     self._dar_baixa_estoque(parts_used, activity_id=atividade.id)
1449 |     resultado["acao"] = "salvar"
1450 |     dialog.accept()
1451 |     self.recarregar()
1452 | except Exception as e:
1453 |     QMessageBox.critical(self, "Erro", f"Erro ao salvar: {e}")
1454 |
1455 | def excluir():
1456 |
1457 |     if ConfirmacaoDigitalDialog.confirmar(
1458 |         "Confirmar", "Deseja realmente excluir esta OS?",
1459 |         dialog,
1460 |     ):
1461 |         try:
1462 |             self.activity_service.excluir(atividade)
1463 |             ToastManager.mostrar(
1464 |                 "OS excluída.",
1465 |                 "aviso", duracao=8000,
1466 |                 acao=("Desfazer", lambda o=atividade, svc=self.activity_service, pag=self: (svc.re
1467 |             )
1468 |             resultado["acao"] = "excluir"
1469 |             dialog.accept()
1470 |             self.recarregar()
1471 |         except Exception as e:
1472 |             QMessageBox.critical(dialog, "Erro", f"Erro ao excluir: {e}")
1473 |
1474 | def cancelar():
1475 |     resultado["acao"] = "cancelar"
1476 |     dialog.reject()
1477 |
1478 | btn_salvar.clicked.connect(salvar)
1479 | btn_excluir.clicked.connect(excluir)
1480 | btn_cancelar.clicked.connect(cancelar)
1481 |
1482 | dialog.exec()
1483 |
1484 | def _criar_form_dialog(self, titulo: str, atividade: Any = None) -> tuple[QDialog, dict[str, Any]]:
1485 |     dialog = QDialog(self)
1486 |     dialog.setWindowTitle(titulo)
1487 |     dialog.setMinimumSize(800, 690)
1488 |     dialog.setStyleSheet(ESTILO_DIALOG)
1489 |

```

[illegible]

[illegible]

[illegible]

```

1679 |         pecas_grid.setSpacing(6)
1680 |         pecas_grid.addWidget(input_label("PEÇAS TROCADAS"))
1681 |         pecas_grid.addWidget(txt_pecas)
1682 |         pecas_grid.addWidget(input_label("ADICIONAR DO ESTOQUE"))
1683 |         pecas_grid.addWidget(estoque_combo_os)
1684 |
1685 |     if atividade:
1686 |         btn_reservar_edit = QPushButton("■ Reservar Peça")
1687 |         btn_reservar_edit.setStyleSheet(ESTILO_BOTAO_SECUNDARIO)
1688 |         def _reservar_edit():
1689 |             idx = estoque_combo_os.currentIndex()
1690 |             if idx <= 0:
1691 |                 return
1692 |             pid = estoque_combo_os.currentData()
1693 |             if pid is None:
1694 |                 return
1695 |             with tratar_erro("reservar peça"):
1696 |                 res = self.part_service.criar_reserva(pid, atividade.id, quantidade=1)
1697 |                 if res:
1698 |                     ToastManager.mostrar(f"Peça reservada para OS #{atividade.id}.", "sucesso")
1699 |                     _atualizar_reservas_edit()
1700 |                     estoque_combo_os.setCurrentIndex(0)
1701 |                     self.recarregar()
1702 |                 else:
1703 |                     ToastManager.mostrar("Estoque insuficiente para reservar.", "erro")
1704 |         btn_reservar_edit.clicked.connect(_reservar_edit)
1705 |         pecas_grid.addWidget(btn_reservar_edit)
1706 |
1707 |     # ■■ Reservas na edição ■■
1708 |     reservas_edit_tabela = QTableWidget()
1709 |     reservas_edit_tabela.setColumnCount(4)
1710 |     reservas_edit_tabela.setHorizontalHeaderLabels(["Peça", "Qtd", "Status", ""])
1711 |     reservas_edit_tabela.setStyleSheet(ESTILO_TABELA_SIMPLES)
1712 |     reservas_edit_tabela.setSelectionBehavior(QTableWidget.SelectRows)
1713 |     reservas_edit_tabela.setEditTriggers(QTableWidget.NoEditTriggers)
1714 |     reservas_edit_tabela.verticalHeader().setVisible(False)
1715 |     reservas_edit_tabela.setAlternatingRowColors(True)
1716 |     reservas_edit_tabela.verticalHeader().setDefaultSectionSize(26)
1717 |     tornar_interativa(reservas_edit_tabela)
1718 |     reservas_edit_tabela.horizontalHeader().setSectionResizeMode(1, QHeaderView.ResizeToContents)
1719 |     reservas_edit_tabela.horizontalHeader().setSectionResizeMode(2, QHeaderView.ResizeToContents)
1720 |     reservas_edit_tabela.horizontalHeader().setSectionResizeMode(3, QHeaderView.ResizeToContents)
1721 |     def _cancelar_reserva_edit(res_id: int):
1722 |         with tratar_erro("cancelar reserva"):
1723 |             self.part_service.cancelar_reserva(res_id)
1724 |             ToastManager.mostrar("Reserva cancelada. Estoque restaurado.", "sucesso")
1725 |             _atualizar_reservas_edit()
1726 |             self.recarregar()
1727 |     def _atualizar_reservas_edit():
1728 |         reservas = self.part_service.reservas_por_os(atividade.id)
1729 |         reservas_edit_tabela.setRowCount(len(reservas))
1730 |         for i, r in enumerate(reservas):
1731 |             reservas_edit_tabela.setItem(i, 0, QTableWidgetItem(r.part.nome if r.part else "-"))
1732 |             item_q = QTableWidgetItem(str(r.quantidade))
1733 |             item_q.setTextAlignment(Qt.AlignCenter)
1734 |             reservas_edit_tabela.setItem(i, 1, item_q)
1735 |             reservas_edit_tabela.setItem(i, 2, QTableWidgetItem(r.status))
1736 |             if r.status == "reservada":
1737 |                 btn_cancel = QPushButton("X")
1738 |                 btn_cancel.setFixedSize(24, 24)
1739 |                 btn_cancel.setStyleSheet("color: #ef4444; font-size: 12px; background: transparent")
1740 |                 btn_cancel.setToolTip("Cancelar reserva")
1741 |                 btn_cancel.clicked.connect(lambda _, rid=r.id: _cancelar_reserva_edit(rid))

```

```

1742 |         reservas_edit_tabela.setCellWidget(i, 3, btn_cancel)
1743 |         _atualizar_reservas_edit()
1744 |         pecas_grid.addWidget(reservas_edit_tabela)
1745 |
1746 |     pecas_layout.addLayout(pecas_grid)
1747 |
1748 |     checklist_box, checklist_layout = group_box("CHECKLIST")
1749 |     tbl = QTableWidget()
1750 |     tbl.setColumnCount(2)
1751 |     tbl.setHorizontalHeaderLabels(["Feito", "Procedimento"])
1752 |     tornar_interativa(tbl)
1753 |     tbl.horizontalHeader().setSectionResizeMode(0, QHeaderView.ResizeToContents)
1754 |     tbl.verticalHeader().setVisible(False)
1755 |     tbl.verticalHeader().setDefaultSectionSize(32)
1756 |     tbl.setSelectionBehavior(QAbstractItemView.SelectRows)
1757 |     tbl.setSelectionMode(QAbstractItemView.SingleSelection)
1758 |     tbl.setStyleSheet("""
1759 |         QTableWidget { background: transparent; border: none; }
1760 |         QTableWidget::item { padding: 4px; color: #e2e8f0; font-size: 13px; }
1761 |         QHeaderView::section { background: transparent; color: #a0a0b0; border: none; font-weight: 600; }
1762 |         QTableWidget QLineEdit {
1763 |             background-color: #1e1e2f !important;
1764 |             color: #ffffff !important;
1765 |             border: 1px solid #3b82f6 !important;
1766 |             border-radius: 4px;
1767 |             padding: 2px 6px !important;
1768 |             font-size: 13px;
1769 |         }
1770 |         QCheckBox::indicator {
1771 |             width: 18px; height: 18px;
1772 |             border: 2px solid #3b82f6; border-radius: 4px;
1773 |             background-color: #1e1e2f;
1774 |         }
1775 |         QCheckBox::indicator:checked {
1776 |             background-color: #3b82f6;
1777 |         }
1778 |     """)
1779 |     itens_padrao = ["Limpeza de laser", "Lubrificação do fusor", "Troca de película", "Troca de rolo d
1780 |     for nome in itens_padrao:
1781 |         r = tbl.rowCount()
1782 |         tbl.insertRow(r)
1783 |         chk = QTableWidgetItem()
1784 |         chk.setFlags(chk.flags() | Qt.ItemIsUserCheckable)
1785 |         chk.setCheckState(Qt.Unchecked)
1786 |         tbl.setItem(r, 0, chk)
1787 |         tbl.setItem(r, 1, QTableWidgetItem(nome))
1788 |     tbl.setMinimumHeight(120)
1789 |     tbl.setMaximumHeight(140)
1790 |
1791 |     btn_add = QPushButton("+ Adicionar")
1792 |     btn_add.setStyleSheet(ESTILO_BOTAO_SECUNDARIO)
1793 |     btn_add.setCursor(Qt.PointingHandCursor)
1794 |     btn_rem = QPushButton("- Remover")
1795 |     btn_rem.setStyleSheet(ESTILO_BOTAO_ERRO)
1796 |     btn_rem.setCursor(Qt.PointingHandCursor)
1797 |
1798 |     def _add_procedimento():
1799 |         r = tbl.rowCount()
1800 |         tbl.insertRow(r)
1801 |         chk = QTableWidgetItem()
1802 |         chk.setFlags(chk.flags() | Qt.ItemIsUserCheckable)
1803 |         chk.setCheckState(Qt.Unchecked)
1804 |         tbl.setItem(r, 0, chk)

```


[illegible]

```

1931 |         upload_layout.setSpacing(8)
1932 |         cmb_categoria = QComboBox()
1933 |         configurar_combo(cmb_categoria)
1934 |         cmb_categoria.addItem(["antes", "depois", "peca"])
1935 |         cmb_categoria.setMinimumWidth(140)
1936 |
1937 |         def _do_upload_edit():
1938 |             self._anexar_arquivo(atividade.id, cmb_categoria.currentText(), dialog)
1939 |             _atualizar_galeria()
1940 |
1941 |         btn_upload = QPushButton("■ Adicionar Foto")
1942 |         btn_upload.setStyleSheet(ESTILO_BOTAO_SECUNDARIO)
1943 |         btn_upload.setToolTip("Selecionar imagem para anexar à OS")
1944 |         btn_upload.setCursor(Qt.PointingHandCursor)
1945 |
1946 |         if atividade:
1947 |             btn_upload.clicked.connect(_do_upload_edit)
1948 |         else:
1949 |             dialog.pending_fotos = []
1950 |             def _abrir_pendente(_categoria):
1951 |                 path, _ = QFileDialog.getOpenFileName(
1952 |                     dialog, "Selecionar Arquivo", "", "Imagens (*.png *.jpg *.jpeg *.bmp);;Todos (*.*)"
1953 |                 )
1954 |                 if path:
1955 |                     dialog.pending_fotos.append((path, _categoria))
1956 |                     _atualizar_galeria()
1957 |             btn_upload.clicked.connect(
1958 |                 lambda: _abrir_pendente(cmb_categoria.currentText())
1959 |             )
1960 |
1961 |         upload_layout.addWidget(cmb_categoria)
1962 |         upload_layout.addWidget(btn_upload)
1963 |         upload_layout.addStretch()
1964 |         fotos_content.addLayout(upload_layout)
1965 |
1966 |         # ■■ Galeria abaixo ■■
1967 |         def _atualizar_galeria():
1968 |             old = dialog._fotos_box if hasattr(dialog, '_fotos_box') else None
1969 |             idx = fotos_content.indexOf(old) if old else 0
1970 |             if old:
1971 |                 fotos_content.removeWidget(old)
1972 |                 old.deleteLater()
1973 |             pending = getattr(dialog, 'pending_fotos', None) if not atividade else None
1974 |             dialog._fotos_box, _ = self._criar_grupo_fotos(
1975 |                 atividade.id if atividade else None,
1976 |                 pending_fotos=pending,
1977 |             )
1978 |             fotos_content.insertWidget(idx if idx >= 0 else 0, dialog._fotos_box)
1979 |
1980 |         dialog.refresh_gallery_cb = _atualizar_galeria
1981 |         _atualizar_galeria()
1982 |
1983 |         fotos_content.addStretch()
1984 |         tab2_scroll.setWidget(tab2_container)
1985 |         tabs.addTab(tab2_scroll, "■ Galeria / Fotos")
1986 |
1987 |         outer.addWidget(tabs, stretch=1)
1988 |
1989 |         campos = {
1990 |             "printer": cmb_printer,
1991 |             "tipo": cmb_tipo,
1992 |             "data": edt_data,
1993 |             "inicio": edt_inicio,

```

```

1994 |         "fim": edt_fim,
1995 |         "sintoma": txt_sintoma,
1996 |         "diagnostico": txt_diagnostico,
1997 |         "solucao": txt_solucao,
1998 |         "descricao": txt_descricao,
1999 |         "pecas": txt_pecas,
2000 |         "origem": cmb_origem,
2001 |         "destino": cmb_destino,
2002 |         "tecnico": cmb_tecnico,
2003 |         "status": cmb_status,
2004 |         "urgencia": cmb_urgencia,
2005 |         "checklist": tbl,
2006 |         "os_vinculada": cmb_os_vinculada,
2007 |     }
2008 |
2009 |     def _toggle_campos_conclusao(status_text: str) -> None:
2010 |         bloq = status_text in ("Aberta", "Aguardando Peça", "Técnico Designado", "Em Deslocamento")
2011 |         edt_fim.setEnabled(not bloq)
2012 |         txt_solucao.setEnabled(not bloq)
2013 |         txt_descricao.setEnabled(not bloq)
2014 |         txt_pecas.setEnabled(not bloq)
2015 |         estoque_combo_os.setEnabled(not bloq)
2016 |         cmb_status.currentTextChanged.connect(_toggle_campos_conclusao)
2017 |
2018 |     if atividade:
2019 |         self._preencher_campos(atividade, campos)
2020 |
2021 |     _toggle_campos_conclusao(cmb_status.currentText())
2022 |
2023 |     if not atividade:
2024 |         btn_layout = QHBoxLayout()
2025 |         btn_layout.setContentsMargins(24, 12, 24, 24)
2026 |         btn_box = QDialogButtonBox(QDialogButtonBox.Save | QDialogButtonBox.Cancel)
2027 |         btn_box.button(QDialogButtonBox.Save).setText("Salvar")
2028 |         btn_box.button(QDialogButtonBox.Save).setStyleSheet(ESTILO_BOTAO_SUCESSO)
2029 |         btn_box.button(QDialogButtonBox.Cancel).setText("Cancelar")
2030 |         btn_box.button(QDialogButtonBox.Cancel).setStyleSheet(ESTILO_BOTAO_FECHAR)
2031 |         btn_box.accepted.connect(dialog.accept)
2032 |         btn_box.rejected.connect(dialog.reject)
2033 |         btn_layout.addStretch()
2034 |         btn_layout.addWidget(btn_box)
2035 |         outer.addLayout(btn_layout)
2036 |
2037 |     return dialog, campos
2038 |
2039 |     def _preencher_campos(self, atividade: Any, campos: dict[str, Any]) -> None:
2040 |         printer = self.printer_service.buscar_por_id(atividade.printer_id)
2041 |         if printer:
2042 |             idx = campos["printer"].findText(printer.patrimonio)
2043 |             if idx >= 0:
2044 |                 campos["printer"].setCurrentIndex(idx)
2045 |             else:
2046 |                 campos["printer"].setCurrentText(printer.patrimonio)
2047 |
2048 |         if atividade.os_vinculada_id:
2049 |             vinc = self.activity_service.buscar_por_id(atividade.os_vinculada_id)
2050 |             if vinc:
2051 |                 idx = campos["os_vinculada"].findData(vinc.id)
2052 |                 if idx >= 0:
2053 |                     campos["os_vinculada"].setCurrentIndex(idx)
2054 |
2055 |         idx_tipo = campos["tipo"].findText(atividade.kind)
2056 |         if idx_tipo >= 0:

```

```

2057 |         campos["tipo"].setCurrentIndex(idx_tipo)
2058 |
2059 |     campos["data"].setDateTime(QDateTime(atividade.event_at))
2060 |     if atividade.inicio_atendimento:
2061 |         campos["inicio"].setDateTime(QDateTime(atividade.inicio_atendimento))
2062 |     if atividade.fim_atendimento:
2063 |         campos["fim"].setDateTime(QDateTime(atividade.fim_atendimento))
2064 |     campos["sintoma"].setPlainText(atividade.sintoma_relatoado or "")
2065 |     campos["diagnostico"].setPlainText(atividade.diagnostico_tecnico or "")
2066 |     campos["solucao"].setPlainText(atividade.solucao_aplicada or "")
2067 |     campos["descricao"].setPlainText(atividade.notes or "")
2068 |     campos["pecas"].setPlainText(atividade.parts_used or "")
2069 |
2070 |     if atividade.from_location:
2071 |         idx_orig = campos["origem"].findText(atividade.from_location)
2072 |         if idx_orig >= 0:
2073 |             campos["origem"].setCurrentIndex(idx_orig)
2074 |         else:
2075 |             campos["origem"].setCurrentText(atividade.from_location)
2076 |
2077 |     if atividade.to_location:
2078 |         idx_dest = campos["destino"].findText(atividade.to_location)
2079 |         if idx_dest >= 0:
2080 |             campos["destino"].setCurrentIndex(idx_dest)
2081 |         else:
2082 |             campos["destino"].setCurrentText(atividade.to_location)
2083 |
2084 |     if atividade.tecnico_id and self.technician_service:
2085 |         tec = self.technician_service.buscar_por_id(atividade.tecnico_id)
2086 |         if tec:
2087 |             idx_tec = campos["tecnico"].findText(tec.nome_exibicao)
2088 |             if idx_tec >= 0:
2089 |                 campos["tecnico"].setCurrentIndex(idx_tec)
2090 |             else:
2091 |                 campos["tecnico"].setCurrentText(tec.nome_exibicao)
2092 |
2093 |     idx_st = campos["status"].findText(atividade.status_atividade or "Aberta",
2094 |                                         Qt.MatchFixedString)
2095 |     if idx_st >= 0:
2096 |         campos["status"].setCurrentIndex(idx_st)
2097 |
2098 |     urgencia = getattr(atividade, 'urgencia', '')
2099 |     if urgencia in URGENCIAS:
2100 |         idx_urg = campos["urgencia"].findText(urgencia)
2101 |         if idx_urg >= 0:
2102 |             campos["urgencia"].setCurrentIndex(idx_urg)
2103 |
2104 |     tbl = campos.get("checklist")
2105 |     if tbl and atividade.procedimentos:
2106 |         try:
2107 |             dados = json.loads(atividade.procedimentos)
2108 |             tbl.setRowCount(0)
2109 |             for item in dados:
2110 |                 r = tbl.rowCount()
2111 |                 tbl.insertRow(r)
2112 |                 chk = QTableWidgetItem()
2113 |                 chk.setFlags(chk.flags() | Qt.ItemIsUserCheckable)
2114 |                 chk.setCheckState(Qt.Checked if item.get("feito") else Qt.Unchecked)
2115 |                 tbl.setItem(r, 0, chk)
2116 |                 tbl.setItem(r, 1, QTableWidgetItem(item.get("nome", "")))
2117 |         except json.JSONDecodeError:
2118 |             pass
2119 |

```

```

2120 |     def _resolver_empresa(self, nome: str) -> Any | None:
2121 |         if not nome or not self.company_service:
2122 |             return None
2123 |         empresa = self.company_service.buscar_por_nome(nome)
2124 |         return empresa.id if empresa else None
2125 |
2126 |     def _resolver_tecnico(self, nome: str) -> int | None:
2127 |         if not nome or not self.technician_service:
2128 |             return None
2129 |         tecnicos = self.technician_service.listar_ativos()
2130 |         for t in tecnicos:
2131 |             if t.nome_exibicao == nome or t.nome_completo == nome:
2132 |                 return t.id
2133 |         return None
2134 |
2135 |     def _dar_baixa_estoque(self, pecas_texto: str, activity_id: int | None = None) -> None:
2136 |         if not pecas_texto:
2137 |             return
2138 |         with tratar_erro("dar baixa no estoque"):
2139 |             self.part_service.retirar_estoque_por_nome(pecas_texto, activity_id=activity_id)
2140 |
2141 |     def _criar_alerta_urgencia(self, printer: Any, urgencia: str) -> None:
2142 |         if not self.alert_service or urgencia not in ("Alta", "Crítica"):
2143 |             return
2144 |         from app.models import Alert
2145 |         existente = self.alert_service.session.query(Alert).filter(
2146 |             Alert.printer_id == printer.id,
2147 |             Alert.tipo == "urgencia",
2148 |             Alert.resolvido == False
2149 |         ).first()
2150 |         if existente:
2151 |             return
2152 |         try:
2153 |             self.alert_service.criar(
2154 |                 printer_id=printer.id,
2155 |                 tipo="urgencia",
2156 |                 titulo=f"OS urgente: {printer.patrimonio}",
2157 |                 descricao=f"Ordem de serviço com urgência {urgencia} criada para {printer.patrimonio} ({pr
2158 |             )
2159 |         except Exception as e:
2160 |             log.warning("Erro ao criar alerta de urgência: %s", e)

```

=== ARQUIVO: .lapp\views\pages\parts_page.py ===

```

001 | from __future__ import annotations
002 |
003 | from typing import Any
004 |
005 | from PySide6.QtCore import Qt, Signal
006 | from PySide6.QtGui import QColor, QIntValidator
007 | from PySide6.QtWidgets import (
008 |     QComboBox,
009 |     QDialog,
010 |     QDialogButtonBox,
011 |     QFormLayout,
012 |     QFrame,
013 |     QGridLayout,
014 |     QHBoxLayout,
015 |     QHeaderView,
016 |     QLabel,
017 |     QLineEdit,

```

```

018 |     QListWidget,
019 |     QListWidgetItem,
020 |     QMessageBox,
021 |     QPushButton,
022 |     QScrollArea,
023 |     QSpinBox,
024 |     QTableWidget,
025 |     QTableWidgetItem,
026 |     QTextEdit,
027 |     QVBoxLayout,
028 |     QWidget,
029 | )
030 |
031 | from app.utils.ui_helpers import tratar_erro
032 | from app.utils.validacao import ValidadorCampo, obrigatorio
033 | from app.views.styles.theme import (
034 |     COR,
035 |     ESTILO_BOTAO_AVISO,
036 |     ESTILO_BOTAO_ERRO,
037 |     ESTILO_BOTAO_FECHAR,
038 |     ESTILO_BOTAO_PRIMARIO,
039 |     ESTILO_BOTAO_SECUNDARIO,
040 |     ESTILO_BOTAO_SUCESSO,
041 |     ESTILO_SUBTITULO,
042 |     ESTILO_TABELA_SIMPLES,
043 |     campo_readonly,
044 |     campo_rotulo,
045 |     configurar_combo,
046 |     ESTILO_DIALOG,
047 |     ESTILO_INPUT,
048 |     ESTILO_INPUT_READONLY,
049 |     ESTILO_TITULO_PAGINA,
050 |     group_box,
051 |     input_label,
052 | )
053 | from app.views.widgets import ToastManager
054 | from app.views.widgets.card_widget import CardMiniClicavel
055 | from app.views.widgets.confirm_dialog import ConfirmacaoDigitalDialog
056 | from app.views.widgets.import_dialog import ImportDialog
057 | from app.views.widgets.pagination import PaginacaoWidget
058 | from app.views.widgets.search_bar import SearchBar
059 | from app.views.widgets.table_widget import TabelaPadrao, tornar_interativa
060 |
061 |
062 | class MultiModelWidget(QWidget):
063 |     """Widget para selecionar multiplos modelos compativeis."""
064 |
065 |     def __init__(self, modelos_disponiveis: list[str] | None = None, parent: QWidget | None = None) -> None:
066 |         super().__init__(parent)
067 |         layout = QVBoxLayout(self)
068 |         layout.setContentsMargins(0, 0, 0, 0)
069 |         layout.setSpacing(4)
070 |
071 |         add_row = QHBoxLayout()
072 |         add_row.setSpacing(6)
073 |         self._combo = QComboBox()
074 |         self._combo.setEditable(True)
075 |         self._combo.setInsertPolicy(QComboBox.NoInsert)
076 |         self._combo.completer().setFilterMode(Qt.MatchFlag.MatchContains)
077 |         self._combo.completer().setCaseSensitivity(Qt.CaseSensitivity.CaseInsensitive)
078 |         self._combo.setMinimumWidth(200)
079 |         self._combo.setStyleSheet(
080 |             "QComboBox { background-color: #1e1e2e; color: #e8e8f0;"

```

```

081 |         " border: 1px solid #2a2a3e; border-radius: 6px;"
082 |         " padding: 4px 8px; font-size: 12px; min-height: 24px; }"
083 |         "QComboBox:hover { border: 1px solid #6366f1; }"
084 |     )
085 |     if modelos_disponiveis:
086 |         self._combo.addItem(modelos_disponiveis)
087 |     add_row.addWidget(self._combo, stretch=1)
088 |
089 |     btn_add = QPushButton("Adicionar")
090 |     btn_add.setFixedHeight(32)
091 |     btn_add.setCursor(Qt.PointingHandCursor)
092 |     btn_add.setToolTip("Adicionar modelo selecionado")
093 |     btn_add.setStyleSheet(
094 |         "QPushButton { background: #22c55e; color: white; border: none;"
095 |         " border-radius: 6px; font-size: 12px; font-weight: bold;"
096 |         " font-family: 'Segoe UI', Arial, sans-serif;"
097 |         " padding: 0 14px; }"
098 |         "QPushButton:hover { background: #16a34a; }"
099 |         "QPushButton:pressed { background: #15803d; }"
100 |     )
101 |     btn_add.clicked.connect(self._adicionar)
102 |     add_row.addWidget(btn_add)
103 |
104 |     btn_rem = QPushButton("Remover")
105 |     btn_rem.setFixedHeight(32)
106 |     btn_rem.setCursor(Qt.PointingHandCursor)
107 |     btn_rem.setToolTip("Remover modelo selecionado na lista")
108 |     btn_rem.setStyleSheet(
109 |         "QPushButton { background: #ef4444; color: white; border: none;"
110 |         " border-radius: 6px; font-size: 12px; font-weight: bold;"
111 |         " font-family: 'Segoe UI', Arial, sans-serif;"
112 |         " padding: 0 14px; }"
113 |         "QPushButton:hover { background: #dc2626; }"
114 |         "QPushButton:pressed { background: #b91c1c; }"
115 |     )
116 |     btn_rem.clicked.connect(self._remover)
117 |     add_row.addWidget(btn_rem)
118 |
119 |     layout.addLayout(add_row)
120 |
121 |     self._lista = QListWidget()
122 |     self._lista.setMaximumHeight(100)
123 |     self._lista.setDragDropMode(QListWidget.NoDragDrop)
124 |     self._lista.setStyleSheet(
125 |         "QListWidget { background-color: #1e1e2e; color: #e8e8f0;"
126 |         " border: 1px solid #2a2a3e; border-radius: 6px;"
127 |         " padding: 4px; font-size: 12px; }"
128 |         "QListWidget::item { padding: 4px 8px; border-radius: 4px; margin: 1px 0; }"
129 |         "QListWidget::item:selected { background-color: rgba(99,102,241,0.3); color: #c7d2fe; }"
130 |         "QListWidget::item:hover { background-color: rgba(99,102,241,0.15); }"
131 |     )
132 |     layout.addWidget(self._lista)
133 |
134 |     self._lbl_count = QLabel("0 modelo(s)")
135 |     self._lbl_count.setStyleSheet("color: #6c7086; font-size: 11px; background: transparent;")
136 |     layout.addWidget(self._lbl_count)
137 |
138 |     le = self._combo.lineEdit()
139 |     if le:
140 |         le.returnPressed.connect(self._adicionar)
141 |
142 | def _adicionar(self) -> None:
143 |     texto = self._combo.currentText().strip()

```



```

144 |         if not texto:
145 |             return
146 |         existentes = [self._lista.item(i).text() for i in range(self._lista.count())]
147 |         if texto not in existentes:
148 |             self._lista.addItem(texto)
149 |             self._atualizar_contador()
150 |             self._combo.setCurrentText("")
151 |
152 |     def _remover(self) -> None:
153 |         row = self._lista.currentRow()
154 |         if row >= 0:
155 |             self._lista.takeItem(row)
156 |             self._atualizar_contador()
157 |
158 |     def _atualizar_contador(self) -> None:
159 |         n = self._lista.count()
160 |         self._lbl_count.setText(f"{n} modelo(s)")
161 |
162 |     def modelos(self) -> list[str]:
163 |         return [self._lista.item(i).text() for i in range(self._lista.count())]
164 |
165 |     def set_modelos(self, modelos: list[str]) -> None:
166 |         self._lista.clear()
167 |         for m in modelos:
168 |             if m.strip():
169 |                 self._lista.addItem(m.strip())
170 |             self._atualizar_contador()
171 |
172 |     def modelo_string(self) -> str:
173 |         return "/".join(self.modelos())
174 |
175 |     def set_modelo_string(self, texto: str) -> None:
176 |         if not texto:
177 |             self._lista.clear()
178 |             self._atualizar_contador()
179 |             return
180 |         partes = [t.strip() for t in texto.split("/") if t.strip()]
181 |         self.set_modelos(partes)
182 |
183 |
184 | class PartsPage(QWidget):
185 |     abrir_atividade: Signal = Signal(int)
186 |
187 |     session: Any
188 |     part_service: Any
189 |     printer_service: Any
190 |     activity_service: Any | None
191 |     _filtro_atual: str | None
192 |     _card_filtro: str | None
193 |     _partes_visiveis: list[Any]
194 |     search: SearchBar
195 |     btn_nova: QPushButton
196 |     card_total: CardMiniClicavel
197 |     card_em_estoque: CardMiniClicavel
198 |     card_sem_estoque: CardMiniClicavel
199 |     tabela: TabelaPadrao
200 |     _paginacao: PaginacaoWidget
201 |
202 |     def __init__(self, session: Any, part_service: Any, printer_service: Any, activity_service: Any | None
203 |         super().__init__(parent)
204 |         self.session = session
205 |         self.part_service = part_service
206 |         self.printer_service = printer_service

```

```

207 |         self.activity_service = activity_service
208 |
209 |         layout = QVBoxLayout(self)
210 |         layout.setContentsMargins(24, 24, 24, 24)
211 |         layout.setSpacing(16)
212 |
213 |         header = QHBoxLayout()
214 |         header.setSpacing(10)
215 |
216 |         titulo = QLabel("\u0001f527 Peças")
217 |         titulo.setStyleSheet(ESTILO_TITULO_PAGINA)
218 |         sub = QLabel("Gerencie o estoque de peças")
219 |         sub.setStyleSheet(ESTILO_SUBTITULO)
220 |
221 |         titulo_col = QVBoxLayout()
222 |         titulo_col.setSpacing(2)
223 |         titulo_col.addWidget(titulo)
224 |         titulo_col.addWidget(sub)
225 |         header.addLayout(titulo_col)
226 |         header.addStretch()
227 |
228 |         self.search = SearchBar(placeholder="Buscar por nome, código ou modelo...", glass=True)
229 |         self.search.textChanged().connect(lambda texto: self._filtrar(texto))
230 |         header.addWidget(self.search)
231 |
232 |         self.btn_nova = QPushButton("\u2795 Nova Peça")
233 |         self.btn_nova.setStyleSheet(ESTILO_BOTAO_PRIMARIO)
234 |         self.btn_nova.setToolTip("Cadastrar nova peça no estoque")
235 |         self.btn_nova.clicked.connect(self._nova)
236 |         header.addWidget(self.btn_nova)
237 |
238 |         btn_importar = QPushButton(" Importar")
239 |         btn_importar.setStyleSheet(ESTILO_BOTAO_AVISO)
240 |         btn_importar.setToolTip("Importar peças de arquivo CSV ou XLSX")
241 |         btn_importar.clicked.connect(lambda: self._importar())
242 |         header.addWidget(btn_importar)
243 |
244 |         layout.addLayout(header)
245 |
246 |         cards = QHBoxLayout()
247 |         cards.setSpacing(12)
248 |         self.card_total = CardMiniClicavel("\u0001f4e6", "Total Peças", "0", COR["roxo"], ao_clicar=lambda: self._total())
249 |         cards.addWidget(self.card_total)
250 |         self.card_em_estoque = CardMiniClicavel("\u2705", "Em Estoque", "0", COR["sucesso"], ao_clicar=lambda: self._em_estoque())
251 |         cards.addWidget(self.card_em_estoque)
252 |         self.card_sem_estoque = CardMiniClicavel("\u274c", "Sem Estoque", "0", COR["erro"], ao_clicar=lambda: self._sem_estoque())
253 |         cards.addWidget(self.card_sem_estoque)
254 |         layout.addLayout(cards)
255 |
256 |         self._filtro_atual = None
257 |         self._card_filtro = None
258 |         self._partes_visiveis = []
259 |         self.tabela = TabelaPadrao(["Código", "Nome", "Marca", "Modelo Compatível", "Categoria", "Estoque",
260 |                                     "Descrição", "Valor", "Data de Entrada", "Data de Saída"])
261 |         self.tabela.cellDoubleClicked.connect(self._detalhes)
262 |         layout.addWidget(self.tabela)
263 |
264 |         self._paginacao = PaginacaoWidget()
265 |         self._paginacao.pagina_alterada.connect(lambda p: self._carregar())
266 |         layout.addWidget(self._paginacao)
267 |
268 |         self._carregar()
269 |
270 | def recarregar(self) -> None:

```

```

270 |         texto = self.search.texto().strip() if hasattr(self, 'search') else ""
271 |         self._filtro_atual = texto if texto else None
272 |         self._card_filtro = None
273 |         self._carregar()
274 |
275 |     def _clicar_card(self, card: str | None) -> None:
276 |         if self._card_filtro == card:
277 |             self._card_filtro = None
278 |         else:
279 |             self._card_filtro = card
280 |             self._carregar()
281 |
282 |     def _atualizar_estado_cards(self) -> None:
283 |         for card, filtro in [
284 |             (self.card_total, None),
285 |             (self.card_em_estoque, "em_estoque"),
286 |             (self.card_sem_estoque, "sem_estoque"),
287 |         ]:
288 |             ativo = self._card_filtro == filtro
289 |             card.setStyleSheet(
290 |                 f"QFrame#miniCard {{ background-color: {'rgba(99,102,241,0.15)'} if ativo else 'rgba(20,20,30,0.15)'}; border: 1px solid {'#6366f1' if ativo else 'rgba(42,42,62,0.5)'}; border-radius: 12px; padding: 5px; }"
291 |                 f"QFrame#miniCard:hover {{ background-color: {'rgba(99,102,241,0.25)'} if ativo else 'rgba(20,20,30,0.25)'}; border: 1px solid {'#6366f1' if ativo else 'rgba(42,42,62,0.8)'}; border-radius: 12px; padding: 5px; }"
292 |                 f"border: 1px solid {'#6366f1' if ativo else 'rgba(42,42,62,0.8)'}; border-radius: 12px; padding: 5px; }"
293 |             )
294 |
295 |
296 |     def _carregar(self) -> None:
297 |         filtro = self._filtro_atual
298 |         todas = self.part_service.listar_todas(filtro=filtro)
299 |
300 |         total = len(todas)
301 |         em_estoque = sum(1 for p in todas if p.quantidade_estoque > 0)
302 |         sem_estoque = sum(1 for p in todas if p.quantidade_estoque <= 0)
303 |         soma_estoque = sum(p.quantidade_estoque for p in todas)
304 |
305 |         self.card_total.atualizar_valor(total)
306 |         self.card_em_estoque.atualizar_valor(soma_estoque)
307 |         self.card_sem_estoque.atualizar_valor(sem_estoque)
308 |         self._atualizar_estado_cards()
309 |
310 |         if self._card_filtro == "em_estoque":
311 |             todas = [p for p in todas if p.quantidade_estoque > 0]
312 |         elif self._card_filtro == "sem_estoque":
313 |             todas = [p for p in todas if p.quantidade_estoque <= 0]
314 |
315 |         total_filtrado = len(todas)
316 |         self._paginacao.configurar(total_filtrado, pagina_atual=self._paginacao.pagina,
317 |                                     itens_por_pagina=self._paginacao.limit)
318 |         inicio = self._paginacao.offset
319 |         pecas = todas[inicio:inicio + self._paginacao.limit]
320 |
321 |         self._partes_visiveis = pecas
322 |         self.tabela.limpar()
323 |         self.tabela.setRowCount(len(pecas))
324 |
325 |         for i, p in enumerate(pecas):
326 |             items = [
327 |                 (p.codigo, None),
328 |                 (p.nome, None),
329 |                 (p.marca, None),
330 |                 (p.modelo_compativel, None),
331 |                 (p.categoria, None),
332 |                 (str(p.quantidade_estoque), None),

```



```

711 |         f_layout.setContentsMargins(20, 0, 20, 0)
712 |         f_layout.setSpacing(10)
713 |
714 |         btn_salvar = QPushButton("\U0001f4be Salvar")
715 |         btn_salvar.setToolTip("Salvar alterações")
716 |         btn_salvar.setCursor(Qt.PointingHandCursor)
717 |         btn_salvar.setStyleSheet(ESTILO_BOTAO_SUCESSO)
718 |         btn_salvar.setMinimumWidth(120)
719 |
720 |         btn_cancelar = QPushButton("Cancelar")
721 |         btn_cancelar.setToolTip("Descartar e fechar")
722 |         btn_cancelar.setCursor(Qt.PointingHandCursor)
723 |         btn_cancelar.setStyleSheet(ESTILO_BOTAO_FECHAR)
724 |
725 |         f_layout.addStretch()
726 |         f_layout.addWidget(btn_cancelar)
727 |         f_layout.addWidget(btn_salvar)
728 |         root.addWidget/footer)
729 |
730 |     def salvar():
731 |         nome_val = edit_nome.text().strip()
732 |         desc_val = edit_descricao.toPlainText().strip()
733 |         modelo_val = multi_modelo.modelo_string()
734 |         try:
735 |             qtd_val = int(edit_qtd.text().strip())
736 |         except ValueError:
737 |             qtd_val = 0
738 |         try:
739 |             min_val = int(edit_minimo.text().strip())
740 |         except ValueError:
741 |             min_val = 1
742 |         with tratar_erro("atualizar peça"):
743 |             self.part_service.atualizar(
744 |                 peca, nome=nome_val, descricao=desc_val,
745 |                 modelo_compativel=modelo_val, quantidade_estoque=qtd_val,
746 |                 estoque_minimo=min_val, marca=edit_marca.text().strip(),
747 |                 categoria=edit_categoria.text().strip(),
748 |                 fornecedor=edit_fornecedor.text().strip(),
749 |             )
750 |             dialog.accept()
751 |             if parent_dialog:
752 |                 parent_dialog.accept()
753 |             idx = next((i for i, p in enumerate(self._partes_visiveis) if p.id == peca.id), -1)
754 |             if idx >= 0:
755 |                 self._detalhes(idx)
756 |             else:
757 |                 self.recarregar()
758 |
759 |         btn_salvar.clicked.connect(salvar)
760 |         btn_cancelar.clicked.connect(dialog.reject)
761 |
762 |         dialog.exec()
763 |
764 |     def _detalhes(self, row: int) -> None:
765 |         if row < 0 or row >= len(self._partes_visiveis):
766 |             return
767 |         peca = self._partes_visiveis[row]
768 |
769 |         dialog = QDialog(self)
770 |         dialog.setWindowTitle(peca.nome)
771 |         dialog.setStyleSheet(ESTILO_DIALOG)
772 |         dialog.setMinimumSize(650, 500)
773 |

```

```

774 |         layout = QVBoxLayout(dialog)
775 |         layout.setContentsMargins(0, 0, 0, 0)
776 |         layout.setSpacing(0)
777 |
778 |         scroll = QScrollArea()
779 |         scroll.setWidgetResizable(True)
780 |         scroll.setHorizontalScrollBarPolicy(Qt.ScrollBarAlwaysOff)
781 |         scroll.setStyleSheet("QScrollArea { border: none; background: transparent; }")
782 |
783 |         container = QWidget()
784 |         container.setStyleSheet("background: transparent;")
785 |         content = QVBoxLayout(container)
786 |         content.setContentsMargins(24, 20, 24, 20)
787 |         content.setSpacing(20)
788 |
789 |         id_box, id_layout = group_box("Identificação")
790 |         id_form = QFormLayout()
791 |         id_form.setLabelAlignment(Qt.AlignRight)
792 |         id_form.setSpacing(8)
793 |         id_form.addRow(campo_rotulo("Código"), campo_readonly(peca.codigo))
794 |         id_form.addRow(campo_rotulo("Nome"), campo_readonly(peca.nome))
795 |         id_form.addRow(campo_rotulo("Descrição"), campo_readonly(peca.descricao))
796 |         modelos = [m.strip() for m in (peca.modelo_compativel or "").split("/") if m.strip()]
797 |         if modelos:
798 |             tags_layout = QHBoxLayout()
799 |             tags_layout.setSpacing(4)
800 |             for m in modelos:
801 |                 tag = QLabel(m)
802 |                 tag.setStyleSheet(
803 |                     "QLabel { background: rgba(99,102,241,0.2); color: #c7d2fe;"
804 |                     " border: 1px solid #6366f1; border-radius: 8px;"
805 |                     " padding: 3px 10px; font-size: 11px; }"
806 |                 )
807 |                 tag.setFixedHeight(24)
808 |                 tags_layout.addWidget(tag)
809 |             tags_layout.addStretch()
810 |             id_form.addRow(campo_rotulo("Modelos"), tags_layout)
811 |         else:
812 |             id_form.addRow(campo_rotulo("Modelos"), campo_readonly(""))
813 |         id_form.addRow(campo_rotulo("Marca"), campo_readonly(peca.marca or ""))
814 |         id_form.addRow(campo_rotulo("Categoria"), campo_readonly(peca.categoria or ""))
815 |         id_form.addRow(campo_rotulo("Fornecedor"), campo_readonly(peca.fornecedor or ""))
816 |         id_layout.addLayout(id_form)
817 |         content.addWidget(id_box)
818 |
819 |         est_box, est_layout = group_box("Estoque")
820 |         est_form = QFormLayout()
821 |         est_form.setLabelAlignment(Qt.AlignRight)
822 |         est_form.setSpacing(8)
823 |         est_form.addRow(campo_rotulo("Quantidade"), campo_readonly(str(peca.quantidade_estoque)))
824 |         est_form.addRow(campo_rotulo("Estoque Mín."), campo_readonly(str(peca.estoque_minimo)))
825 |         if peca.quantidade_estoque <= peca.estoque_minimo:
826 |             warn = QLabel("■■■ Estoque baixo! A quantidade está no ou abaixo do mínimo.")
827 |             warn.setStyleSheet("color: #ef4444; font-size: 11px; font-weight: 600; background: transparent;")
828 |             est_layout.addWidget(warn)
829 |         est_layout.addLayout(est_form)
830 |         # ■■■ Receber Estoque ■■■
831 |         btn_receber = QPushButton("■ Receber Estoque")
832 |         btn_receber.setStyleSheet(ESTILO_BOTAO_SUCESSO)
833 |         btn_receber.clicked.connect(lambda: self._receber_estoque(peca, dialog))
834 |         est_layout.addWidget(btn_receber)
835 |         content.addWidget(est_box)
836 |

```

```

837 |         # ■■ Movimentações Recentes ■■
838 |         mov_box, mov_layout = group_box("Movimentações Recentes")
839 |         mov_tabela = QTableWidgetItem()
840 |         mov_tabela.setColumnCount(6)
841 |         mov_tabela.setHorizontalHeaderLabels(["Data/Hora", "Tipo", "Qtd", "Saldo", "Observação", ""])
842 |         mov_tabela.setStyleSheet(ESTILO_TABELA_SIMPLES)
843 |         mov_tabela.setSelectionBehavior(QTableWidgetItem.SelectRows)
844 |         mov_tabela.setEditTriggers(QTableWidgetItem.NoEditTriggers)
845 |         mov_tabela.verticalHeader().setVisible(False)
846 |         mov_tabela.setAlternatingRowColors(True)
847 |         mov_tabela.verticalHeader().setDefaultSectionSize(30)
848 |         tornar_interativa(mov_tabela)
849 |         mov_tabela.horizontalHeader().setSectionResizeMode(5, QHeaderView.ResizeToContents)
850 |         from app.utils.helpers import formatar_data_hora
851 |         movimentos = self.part_service.movimentacoes(peca.id, limite=20)
852 |         mov_tabela.setRowCount(len(movimentos))
853 |         for i, m in enumerate(movimentos):
854 |             mov_tabela.setItem(i, 0, QTableWidgetItem(formatar_data_hora(m.created_at)))
855 |             mov_tabela.setItem(i, 1, QTableWidgetItem(m.tipo))
856 |             item_qtd = QTableWidgetItem(str(m.quantidade))
857 |             item_qtd.setTextAlignment(Qt.AlignCenter)
858 |             mov_tabela.setItem(i, 2, item_qtd)
859 |             item_saldo = QTableWidgetItem(str(m.saldo_posterior))
860 |             item_saldo.setTextAlignment(Qt.AlignCenter)
861 |             mov_tabela.setItem(i, 3, item_saldo)
862 |             mov_tabela.setItem(i, 4, QTableWidgetItem(m.observacao or ""))
863 |             btn_lixeira = QPushButton("■")
864 |             btn_lixeira.setFixedSize(28, 28)
865 |             btn_lixeira.setStyleSheet("""
866 |                 QPushButton {
867 |                     background: transparent;
868 |                     border: none;
869 |                     font-size: 14px;
870 |                     color: #ef4444;
871 |                 }
872 |                 QPushButton:hover {
873 |                     background: rgba(239, 68, 68, 0.15);
874 |                     border-radius: 4px;
875 |                 }
876 |             """)
877 |             btn_lixeira.setToolTip("Estornar esta movimentação")
878 |             mov_id = m.id
879 |             btn_lixeira.clicked.connect(lambda checked, mid=mov_id: self._estornar_movimento(mid, peca, dia))
880 |             mov_tabela.setCellWidget(i, 5, btn_lixeira)
881 |         mov_layout.addWidget(mov_tabela)
882 |         content.addWidget(mov_box)
883 |
884 |         # ■■ Requisição de Compra Pendente ■■
885 |         reqs = self.part_service.listar_requisicoes(status="pendente", limite=10)
886 |         reqs_destas = [r for r in reqs if r.part_id == peca.id]
887 |         if reqs_destas:
888 |             req_box, req_layout = group_box("Requisição de Compra Pendente")
889 |             for r in reqs_destas:
890 |                 lbl_req = QLabel(f"■■ #{r.id} - {r.quantidade_sugerida} un. ({r.observacao or ''})")
891 |                 lbl_req.setStyleSheet("color: #f59e0b; font-size: 12px; background: transparent; padding: 2px;")
892 |                 req_layout.addWidget(lbl_req)
893 |             content.addWidget(req_box)
894 |
895 |         content.addStretch()
896 |         scroll.setWidget(container)
897 |         layout.addWidget(scroll, stretch=1)
898 |
899 |         btn_layout = QHBoxLayout()

```

```

900 |         btn_layout.setContentsMargins(24, 12, 24, 16)
901 |         btn_layout.setSpacing(10)
902 |
903 |         btn_editar = QPushButton("■ Editar")
904 |         btn_editar.setStyleSheet(ESTILO_BOTAO_AVISO)
905 |         btn_editar.setToolTip("Editar esta peça")
906 |         btn_editar.clicked.connect(lambda: self._abrir_edicao(peca, dialog))
907 |
908 |         btn_excluir = QPushButton("■ Excluir")
909 |         btn_excluir.setStyleSheet(ESTILO_BOTAO_ERRO)
910 |         btn_excluir.setToolTip("Excluir esta peça (pode ser desfeito pela Lixeira)")
911 |         btn_excluir.clicked.connect(lambda: self._confirmar_exclusao(peca, dialog))
912 |
913 |         btn_layout.addWidget(btn_editar)
914 |         btn_layout.addWidget(btn_excluir)
915 |
916 |         if self.activity_service:
917 |             btn_historico = QPushButton("■ Histórico de Uso")
918 |             btn_historico.setStyleSheet(ESTILO_BOTAO_SECUNDARIO)
919 |             btn_historico.setToolTip("Ver histórico de uso desta peça em ordens de serviço")
920 |             btn_historico.clicked.connect(lambda: self._mostrar_uso(peca))
921 |             btn_layout.addWidget(btn_historico)
922 |
923 |         btn_layout.addStretch()
924 |
925 |         btn_fechar = QPushButton("Fechar")
926 |         btn_fechar.setStyleSheet(ESTILO_BOTAO_FECHAR)
927 |         btn_fechar.setToolTip("Fechar")
928 |         btn_fechar.clicked.connect(dialog.accept)
929 |         btn_layout.addWidget(btn_fechar)
930 |
931 |         layout.addLayout(btn_layout)
932 |
933 |         dialog.exec()
934 |
935 |     def _estornar_movimento(self, movimento_id: int, peca: Any, parent_dialog: QDialog) -> None:
936 |         from app.views.widgets.confirm_dialog import ConfirmacaoDigitalDialog
937 |         resp = ConfirmacaoDigitalDialog.confirmar(
938 |             "Estornar Movimentação",
939 |             f"Tem certeza que deseja estornar esta movimentação?\n\n0 estoque da peça \"{peca.nome}\" será",
940 |             parent=parent_dialog,
941 |             palavra_chave="ESTORNAR",
942 |         )
943 |         if not resp:
944 |             return
945 |         with tratar_erro("estornar movimentação"):
946 |             self.part_service.estornar_movimento(movimento_id)
947 |             ToastManager.mostrar("Movimentação estornada com sucesso!", tipo="sucesso")
948 |             parent_dialog.accept()
949 |             self._detalhes(self._partes_visiveis.index(peca) if peca in self._partes_visiveis else 0)
950 |
951 |     def _receber_estoque(self, peca: Any, parent_dialog: QDialog) -> None:
952 |         dialog = QDialog(parent_dialog)
953 |         dialog.setWindowTitle(f"Receber Estoque – {peca.nome}")
954 |         dialog.setStyleSheet(ESTILO_DIALOG)
955 |         dialog.setMinimumWidth(400)
956 |         layout = QVBoxLayout(dialog)
957 |         layout.setSpacing(12)
958 |
959 |         lbl = QLabel(f"Estoque atual: <b>{peca.quantidade_estoque}</b> un. | Mínimo: <b>{peca.estoque_min}</b>")
960 |         lbl.setStyleSheet("color: #c8c8e0; font-size: 13px; background: transparent;")
961 |         layout.addWidget(lbl)
962 |

```

```

963 |         qtd_layout = QFormLayout()
964 |         qtd_spin = QSpinBox()
965 |         qtd_spin.setRange(1, 99999)
966 |         qtd_spin.setValue(1)
967 |         qtd_spin.setStyleSheet(ESTILO_INPUT)
968 |         qtd_layout.addRow(campo_rotulo("Quantidade a receber:"), qtd_spin)
969 |         layout.addLayout(qtd_layout)
970 |
971 |         obs_input = QLineEdit()
972 |         obs_input.setStyleSheet(ESTILO_INPUT)
973 |         obs_input.setPlaceholderText("Observação (opcional)")
974 |         layout.addWidget(QLabel("Observação:"))
975 |         layout.addWidget(obs_input)
976 |
977 |         botoes = QHBoxLayout()
978 |         btn_confirmar = QPushButton("■ Receber")
979 |         btn_confirmar.setStyleSheet(ESTILO_BOTAO_SUCESSO)
980 |         btn_confirmar.clicked.connect(lambda: self._confirmar_recebimento(peca, qtd_spin, obs_input, dialog))
981 |         botoes.addWidget(btn_confirmar)
982 |         btn_cancelar = QPushButton("Cancelar")
983 |         btn_cancelar.setStyleSheet(ESTILO_BOTAO_FECHAR)
984 |         btn_cancelar.clicked.connect(dialog.reject)
985 |         botoes.addWidget(btn_cancelar)
986 |         layout.addLayout(botoes)
987 |
988 |         dialog.exec()
989 |
990 | def _confirmar_recebimento(self, peca: Any, qtd_spin: QSpinBox, obs_input: QLineEdit, dialog: QDialog,
991 |                             qtd = qtd_spin.value()
992 |                             obs = obs_input.text().strip()
993 |                             with tratar_erro("receber estoque"):
994 |                                 self.part_service.receber_estoque(peca, qtd, observacao=obs)
995 |                                 ToastManager.mostrar(f"{qtd} unidade(s) de '{peca.nome}' adicionada(s) ao estoque.", "sucesso")
996 |                                 dialog.accept()
997 |                                 parent_dialog.accept()
998 |                                 self.recarregar()
999 |
1000 | def _confirmar_exclusao(self, peca, dialog: QDialog) -> None:
1001 |     if ConfirmacaoDigitarDialog.confirmar(
1002 |         "Confirmar", f"Excluir a peça '{peca.nome}'?",
1003 |         dialog,
1004 |     ):
1005 |         with tratar_erro("excluir peça"):
1006 |             self.part_service.excluir(peca)
1007 |             ToastManager.mostrar(
1008 |                 f"Peça '{peca.nome}' excluída.",
1009 |                 "aviso", duracao=8000,
1010 |                 acao=("Desfazer", lambda o=peca, svc=self.part_service, pag=self: (svc.restaurar(o), pag)),
1011 |             )
1012 |             dialog.accept()
1013 |             self.recarregar()
1014 |
1015 | def _mostrar_uso(self, peca: Any) -> None:
1016 |     from app.utils.helpers import formatar_data_hora
1017 |     diretas, relacionadas = self.activity_service.listar_por_peca_com_relacionadas(peca.nome, limite=2)
1018 |     if not diretas:
1019 |         QMessageBox.information(self, "Histórico de Uso", f"Nenhum uso encontrado para '{peca.nome}'.")
1020 |         return
1021 |
1022 |     dialog = QDialog(self)
1023 |     dialog.setWindowTitle(f"Uso da peça: {peca.nome}")
1024 |     dialog.setMinimumSize(780, 500)
1025 |     dialog.setStyleSheet(ESTILO_DIALOG)

```

```

1026 |
1027 |     layout = QVBoxLayout(dialog)
1028 |     layout.setSpacing(12)
1029 |
1030 |     def preencher_tabela(atividades):
1031 |         tabela = QTableWidgetItem()
1032 |         tabela.setColumnCount(5)
1033 |         tabela.setHorizontalHeaderLabels(["Data/Hora", "Tipo", "Impressora", "Origem", "Destino"])
1034 |         tabela.horizontalHeader().setStretchLastSection(True)
1035 |         tabela.setSelectionBehavior(QTableWidgetItem.SelectRows)
1036 |         tabela.setEditTriggers(QTableWidgetItem.NoEditTriggers)
1037 |         tabela.setAlternatingRowColors(True)
1038 |         tabela.setRowCount(len(atividades))
1039 |         tabela.verticalHeader().setVisible(False)
1040 |         for i, a in enumerate(atividades):
1041 |             pat = a.printer.patrimonio if a.printer else "-"
1042 |             tabela.setItem(i, 0, QTableWidgetItem(formatar_data_hora(a.event_at)))
1043 |             tabela.setItem(i, 1, QTableWidgetItem("Manutenção" if a.kind == "MANUTENCAO" else "Movimen
1044 |             tabela.setItem(i, 2, QTableWidgetItem(pat))
1045 |             tabela.setItem(i, 3, QTableWidgetItem(a.from_location or "-"))
1046 |             tabela.setItem(i, 4, QTableWidgetItem(a.to_location or "-"))
1047 |         tornar_interativa(tabela)
1048 |         tabela.resizeColumnsToContents()
1049 |         return tabela
1050 |
1051 |     box, box_layout = group_box("Histórico de Uso")
1052 |
1053 |     box_layout.addWidget(QLabel(f"<b style='font-size:14px'>■ Uso direto – {len(diretas)} registro(s)<b style='font-size:14px'>■ Outras atividades nas mesmas impress
1054 |     tab_diretas = preencher_tabela(diretas)
1055 |     tab_diretas.cellDoubleClicked.connect(lambda r, c: self._abrir_atividade_historico(diretas[r].id,
1056 |     box_layout.addWidget(tab_diretas)
1057 |
1058 |     if relacionadas:
1059 |         box_layout.addWidget(QLabel(f"<b style='font-size:14px'>■ Outras atividades nas mesmas impress
1060 |         tab_rel = preencher_tabela(relacionadas)
1061 |         tab_rel.cellDoubleClicked.connect(lambda r, c: self._abrir_atividade_historico(relacionadas[r]
1062 |         box_layout.addWidget(tab_rel)
1063 |
1064 |     layout.addWidget(box)
1065 |
1066 |     btn_fechar = QPushButton("Fechar")
1067 |     btn_fechar.setStyleSheet(ESTILO_BOTAO_FECHAR)
1068 |     btn_fechar.setToolTip("Fechar janela")
1069 |     btn_fechar.clicked.connect(dialog.accept)
1070 |     layout.addWidget(btn_fechar, alignment=Qt.AlignCenter)
1071 |
1072 |     dialog.exec()
1073 |
1074 |     def _importar(self) -> None:
1075 |         dialog: ImportDialog = ImportDialog(self, "parts", "Peças", self.part_service, self.session)
1076 |         if dialog.exec() == ImportDialog.Accepted:
1077 |             self.recarregar()
1078 |
1079 |     def _abrir_atividade_historico(self, activity_id: int, dialog: QDialog) -> None:
1080 |         self.abrir_atividade.emit(activity_id)

```

=== ARQUIVO: .lapp\views\pages\printers_page.py ===

```

001 | from __future__ import annotations
002 |
003 | import os

```

```

004 | import shutil
005 | from datetime import datetime as dt
006 | from pathlib import Path
007 | from typing import Any
008 |
009 | from PySide6.QtCore import QDate, Qt
010 | from PySide6.QtGui import QColor, QPixmap
011 | from PySide6.QtWidgets import (
012 |     QAbstractItemView,
013 |     QComboBox,
014 |     QCompleter,
015 |     QDateEdit,
016 |     QDialog,
017 |     QFileDialog,
018 |     QFrame,
019 |     QGridLayout,
020 |     QHBoxLayout,
021 |     QHeaderView,
022 |     QLabel,
023 |     QLineEdit,
024 |     QMessageBox,
025 |     QPushButton,
026 |     QScrollArea,
027 |     QSizePolicy,
028 |     QTableWidget,
029 |     QTableWidgetItem,
030 |     QTabWidget,
031 |     QTextEdit,
032 |     QVBoxLayout,
033 |     QWidget,
034 | )
035 |
036 | from app.utils.helpers import formatar_data, formatar_data_hora, limpar_local, parse_data
037 | from app.utils.ui_helpers import tratar_erro
038 | from app.utils.validacao import ValidadorCampo, obrigatorio, minimo
039 | from app.views.styles.theme import (
040 |     COR,
041 |     ESTILO_BOTAO_AVISO,
042 |     ESTILO_BOTAO_ERRO,
043 |     ESTILO_BOTAO_FECHAR,
044 |     ESTILO_BOTAO_PRIMARIO,
045 |     ESTILO_BOTAO_SECUNDARIO,
046 |     ESTILO_BOTAO_SUCESSO,
047 |     configurar_combo,
048 |     configurar_combo_colorido,
049 |     ESTILO_DIALOG,
050 |     ESTILO_INPUT,
051 |     ESTILO_INPUT_READONLY,
052 |     ESTILO_LABEL_CAMPO,
053 |     ESTILO_SUBTITULO,
054 |     ESTILO_TITULO_PAGINA,
055 |     STATUS_CORES,
056 | )
057 | from app.views.styles.theme import URGENCIAS, URGENCIA_CORES, SLA_DIAS
058 | from app.views.widgets import ToastManager
059 | from app.views.widgets.confirm_dialog import ConfirmacaoDigitalDialog
060 | from app.views.widgets.import_dialog import ImportDialog
061 | from app.views.widgets.pagination import PaginacaoWidget
062 | from app.views.widgets.search_bar import SearchBar
063 | from app.views.widgets.table_widget import TabelaPadrao
064 | from app.services.maintenance_scheduler import (
065 |     DIAS_AVISO_OPCOES,
066 |     PAGINAS_OPCOES,

```



```

445 | def _campo_texto(self, placeholder: str, max_len: int) -> QLineEdit:
446 |     le = QLineEdit()
447 |     le.setPlaceholderText(placeholder)
448 |     le.setMaxLength(max_len)
449 |     le.setStyleSheet(ESTILO_INPUT)
450 |     return le
451 |
452 | def _err_lbl(self) -> QLabel:
453 |     lbl = QLabel()
454 |     lbl.setStyleSheet(
455 |         "color: #ef4444; font-size: 9px; background: transparent;"
456 |         " padding: 0; margin: 0; border: none;"
457 |     )
458 |     lbl.hide()
459 |     return lbl
460 |
461 | def _mascara_data(self, texto: str) -> None:
462 |     le = self.sender()
463 |     if not isinstance(le, QLineEdit):
464 |         return
465 |     old_pos = le.cursorPosition()
466 |     digits = "".join(c for c in texto if c.isdigit())[:8]
467 |     if not digits and texto:
468 |         le.blockSignals(True)
469 |         le.clear()
470 |         le.blockSignals(False)
471 |         return
472 |     partes = [digits[:2]]
473 |     if len(digits) > 2:
474 |         partes.append(digits[2:4])
475 |     if len(digits) > 4:
476 |         partes.append(digits[4:8])
477 |     nova = "/".join(partes)
478 |     if nova != texto:
479 |         le.blockSignals(True)
480 |         le.setText(nova)
481 |         le.blockSignals(False)
482 |         le.setCursorPosition(old_pos + (len(nova) - len(texto)))
483 |
484 | def _ajustar_urgencia_por_status(self, status: str) -> None:
485 |     bloqueado = status in ("Operacional", "Em uso")
486 |     self.urgencia_combo.setEnabled(not bloqueado)
487 |     if bloqueado:
488 |         self.urgencia_combo.setCurrentIndex(-1)
489 |         self.urgencia_combo.setToolTip("Urgência bloqueada para status Operacional ou Em uso")
490 |         self.urgencia_combo.setStyleSheet(
491 |             "QComboBox { background-color: #1a1a2a; color: #4a4a5e; border: 1px solid #2a2a3e; }"
492 |             " QComboBox::drop-down { border: none; }"
493 |             " QComboBox QAbstractItemView { color: #4a4a5e; }"
494 |         )
495 |     else:
496 |         self.urgencia_combo.setToolTip("")
497 |         configurar_combo(self.urgencia_combo)
498 |
499 | def _preencher(self, p: Any) -> None:
500 |     self.pat.setText(p.patrimonio or "")
501 |     self.serial.setText(p.serial or "")
502 |     self.modelo.setText(p.modelo or "")
503 |     self.marca.setText(p.marca or "")
504 |     if p.status:
505 |         self.status.setCurrentText(p.status)
506 |     if p.tipo in ["Laser", "Jato de tinta", "Multifuncional"]:
507 |         self.tipo.setCurrentText(p.tipo)

```

[illegible]


```

949 |         printer.proxima_revisao.day,
950 |     ))
951 | else:
952 |     self.date_edit.setDate(QDate.currentDate().addMonths(3))
953 |     self.date_edit.setMinimumDate(QDate.currentDate())
954 |     self.date_edit.setStyleSheet(ESTILO_INPUT)
955 |     root.addWidget(self.date_edit)
956 |
957 |     label_urgencia = QLabel("Nível de urgência:")
958 |     label_urgencia.setStyleSheet("color: #94949f; font-size: 11px; font-weight: 600; background: transparent;")
959 |     root.addWidget(label_urgencia)
960 |
961 |     self.urgencia_combo = QComboBox()
962 |     self.urgencia_combo.addItem(URGENCIAS)
963 |     configurar_combo(self.urgencia_combo)
964 |     configurar_combo_colorido(self.urgencia_combo, URGENCIA_CORES)
965 |     if printer.urgencia_prox_manutencao in URGENCIAS:
966 |         self.urgencia_combo.setCurrentText(printer.urgencia_prox_manutencao)
967 |     if printer.status in ("Operacional", "Em uso"):
968 |         self.urgencia_combo.setEnabled(False)
969 |         self.urgencia_combo.setToolTip("Urgência bloqueada para impressoras com status Operacional ou Em uso")
970 |         self.urgencia_combo.setStyleSheet(
971 |             self.urgencia_combo.styleSheet()
972 |             + " QComboBox:disabled { background-color: #1e1e2e; color: #6b7280;"
973 |             + " border: 1px solid #ef4444; }"
974 |         )
975 |     self.urgencia_combo.currentTextChanged.connect(self._atualizar_sla)
976 |     root.addWidget(self.urgencia_combo)
977 |
978 |     self.sla_label = QLabel()
979 |     self.sla_label.setStyleSheet("color: #94949f; font-size: 11px; background: transparent;")
980 |     root.addWidget(self.sla_label)
981 |     self._atualizar_sla(self.urgencia_combo.currentText())
982 |
983 |     root.addStretch()
984 |
985 |     btn_layout = QHBoxLayout()
986 |     btn_layout.setSpacing(10)
987 |
988 |     btn_salvar = QPushButton("Salvar Agendamento")
989 |     btn_salvar.setToolTip("Salvar a data de agendamento")
990 |     btn_salvar.setCursor(Qt.PointingHandCursor)
991 |     btn_salvar.setStyleSheet(ESTILO_BOTAO_SUCESSO)
992 |     btn_salvar.clicked.connect(self._salvar)
993 |
994 |     btn_remove = QPushButton("Remover Agendamento")
995 |     btn_remove.setToolTip("Cancelar o agendamento existente")
996 |     btn_remove.setCursor(Qt.PointingHandCursor)
997 |     btn_remove.setStyleSheet(ESTILO_BOTAO_ERRO)
998 |     btn_remove.clicked.connect(self._remove)
999 |
1000 |     btn_cancelar = QPushButton("Cancelar")
1001 |     btn_cancelar.setToolTip("Fechar sem alterar")
1002 |     btn_cancelar.setCursor(Qt.PointingHandCursor)
1003 |     btn_cancelar.setStyleSheet(ESTILO_BOTAO_FECHAR)
1004 |     btn_cancelar.clicked.connect(self.reject)
1005 |
1006 |     btn_layout.addWidget(btn_salvar)
1007 |     btn_layout.addWidget(btn_remove)
1008 |     btn_layout.addStretch()
1009 |     btn_layout.addWidget(btn_cancelar)
1010 |     root.addLayout(btn_layout)
1011 |

```

```

1012 |     def _atualizar_sla(self, urgencia: str) -> None:
1013 |         dias = SLA_DIAS.get(urgencia, 0)
1014 |         cor = URGENCIA_CORES.get(urgencia, "#94949f")
1015 |         if dias == 1:
1016 |             texto = f"SLA: até {dias} dia útil"
1017 |         else:
1018 |             texto = f"SLA: até {dias} dias úteis"
1019 |         self.sla_label.setText(texto)
1020 |         self.sla_label.setStyleSheet(
1021 |             f"color: {cor}; font-size: 11px; font-weight: 600; background: transparent;"
1022 |         )
1023 |
1024 |     def _salvar(self) -> None:
1025 |         self._data = self.date_edit.date()
1026 |         self._urgencia = self.urgencia_combo.currentText()
1027 |         self.accept()
1028 |
1029 |     def _remover(self) -> None:
1030 |         self._data = None
1031 |         self._urgencia = ""
1032 |         self.accept()
1033 |
1034 |     def data_agendada(self) -> QDate | None:
1035 |         return self._data
1036 |
1037 |     def urgencia(self) -> str:
1038 |         return self._urgencia
1039 |
1040 |
1041 | def _obter_printer_service(widget: QWidget):
1042 |     """Sobe na hierarquia de widgets até achar o PrintersPage com printer_service."""
1043 |     parent = widget.parent()
1044 |     while parent is not None:
1045 |         if hasattr(parent, 'printer_service'):
1046 |             return parent.printer_service
1047 |         parent = parent.parent()
1048 |     return None
1049 |
1050 |
1051 | class _PrinterDetailDialog(QDialog):
1052 |     """Dialog de visualização detalhada de uma impressora."""
1053 |
1054 |     def __init__(
1055 |         self,
1056 |         printer: Any,
1057 |         activity_service: Any,
1058 |         printer_location_service: Any,
1059 |         part_service: Any,
1060 |         main_window: Any = None,
1061 |         scheduler: MaintenanceScheduler | None = None,
1062 |         parent: QWidget | None = None,
1063 |     ) -> None:
1064 |         super().__init__(parent)
1065 |         self._printer = printer
1066 |         self._activity_service = activity_service
1067 |         self._printer_location_service = printer_location_service
1068 |         self._part_service = part_service
1069 |         self._main_window = main_window
1070 |         self._scheduler = scheduler
1071 |
1072 |         self.setWindowTitle(f"Impressora \u2014 {printer.patrimonio}")
1073 |         self.setMinimumSize(820, 600)
1074 |         self.setStyleSheet(ESTILO_DIALOG)

```


[illegible]

```

1138 |         pat_lbl = QLabel(p.patrimonio)
1139 |         pat_lbl.setStyleSheet(
1140 |             "color: #e8e8f0; font-size: 20px; font-weight: 700;"
1141 |             " background: transparent; letter-spacing: -0.3px;"
1142 |         )
1143 |         info_col.addWidget(pat_lbl)
1144 |
1145 |         modelo_row = QHBoxLayout()
1146 |         modelo_row.setSpacing(8)
1147 |         if p.marca:
1148 |             marca_lbl = QLabel(p.marca)
1149 |             marca_lbl.setStyleSheet(
1150 |                 "color: #717182; font-size: 13px; background: transparent;"
1151 |             )
1152 |             modelo_row.addWidget(marca_lbl)
1153 |             dot = QLabel("\u00b7")
1154 |             dot.setStyleSheet("color: #3a3a50; background: transparent;")
1155 |             modelo_row.addWidget(dot)
1156 |         if p.modelo:
1157 |             modelo_lbl = QLabel(p.modelo)
1158 |             modelo_lbl.setStyleSheet(
1159 |                 "color: #94949f; font-size: 13px; background: transparent;"
1160 |             )
1161 |             modelo_row.addWidget(modelo_lbl)
1162 |         modelo_row.addStretch()
1163 |         info_col.addLayout(modelo_row)
1164 |         layout.addLayout(info_col, stretch=1)
1165 |
1166 |         # Status e local (à direita)
1167 |         meta_col = QVBoxLayout()
1168 |         meta_col.setSpacing(6)
1169 |         meta_col.setAlignment(Qt.AlignRight | Qt.AlignVCenter)
1170 |
1171 |         status_lbl = _badge_status(p.status or "-")
1172 |         status_lbl.setAlignment(Qt.AlignRight)
1173 |         meta_col.addWidget(status_lbl)
1174 |
1175 |         if p.local_atual:
1176 |             local_lbl = QLabel(f"\u00001f4cd {p.local_atual}")
1177 |             local_lbl.setStyleSheet(
1178 |                 "color: #717182; font-size: 12px; background: transparent;"
1179 |             )
1180 |             local_lbl.setAlignment(Qt.AlignRight)
1181 |             meta_col.addWidget(local_lbl)
1182 |
1183 |         layout.addLayout(meta_col)
1184 |         return header
1185 |
1186 |     def _build_tab_geral(self) -> QWidget:
1187 |         tab = QWidget()
1188 |         tab.setStyleSheet("background: transparent;")
1189 |
1190 |         scroll = QScrollArea()
1191 |         scroll.setWidgetResizable(True)
1192 |         scroll.setHorizontalScrollBarPolicy(Qt.ScrollBarAlwaysOff)
1193 |         scroll.setStyleSheet("QScrollArea { border: none; background: transparent; }")
1194 |
1195 |         container = QWidget()
1196 |         container.setStyleSheet("background: transparent;")
1197 |         layout = QVBoxLayout(container)
1198 |         layout.setContentsMargins(24, 20, 24, 20)
1199 |         layout.setSpacing(20)
1200 |

```

[illegible]

```

1264 |         man_grid.addWidget(_campo_rotulo("Última Revisão"), 0, 1)
1265 |         ult_val = formatar_data(p.ultima_revisao) if p.ultima_revisao else "-"
1266 |         man_grid.addWidget(_campo_readonly(ult_val), 1, 1)
1267 |
1268 |         man_grid.setColumnStretch(0, 1)
1269 |         man_grid.setColumnStretch(1, 1)
1270 |
1271 |         man_grid.addWidget(_campo_rotulo("Próxima Manutenção"), 2, 0)
1272 |         data_prox = p.proxima_revisao
1273 |         if data_prox:
1274 |             dias = (data_prox - dt.now()).days
1275 |             if dias < 0:
1276 |                 cor_prox = "#f38ba8"
1277 |                 txt_prox = f"{formatar_data(data_prox)} (VENCIDA há {abs(dias)} dias)"
1278 |             elif dias <= 15:
1279 |                 cor_prox = "#f9e2af"
1280 |                 txt_prox = f"{formatar_data(data_prox)} (em {dias} dias)"
1281 |             elif dias <= 30:
1282 |                 cor_prox = "#f9e2af"
1283 |                 txt_prox = f"{formatar_data(data_prox)} ({dias} dias)"
1284 |             else:
1285 |                 cor_prox = "#a6e3a1"
1286 |                 txt_prox = formatar_data(data_prox)
1287 |         else:
1288 |             cor_prox = "#717182"
1289 |             txt_prox = "Não agendada"
1290 |
1291 |         prox_lbl = QLabel(txt_prox)
1292 |         prox_lbl.setToolTip("Data agendada para a próxima manutenção")
1293 |         prox_lbl.setTextInteractionFlags(Qt.TextSelectableByMouse)
1294 |         prox_lbl.setStyleSheet(
1295 |             f"color: {cor_prox}; font-size: 13px; font-weight: 600;"
1296 |             f" background: transparent; padding: 0;"
1297 |         )
1298 |         man_grid.addWidget(prox_lbl, 3, 0)
1299 |
1300 |         man_grid.addWidget(_campo_rotulo("Urgência"), 2, 1)
1301 |         urgencia = getattr(p, 'urgencia_prox_manutencao', '')
1302 |         if urgencia in URGENCIA_CORES:
1303 |             sla_dias = SLA_DIAS.get(urgencia, 0)
1304 |             cor_urg = URGENCIA_CORES[urgencia]
1305 |             urg_txt = f"{urgencia} - SLA: {sla_dias} dia(s) útil(eis)"
1306 |         else:
1307 |             cor_urg = "#717182"
1308 |             urg_txt = "-"
1309 |         urg_lbl = QLabel(urg_txt)
1310 |         urg_lbl.setToolTip("Nível de urgência e SLA para esta manutenção")
1311 |         urg_lbl.setTextInteractionFlags(Qt.TextSelectableByMouse)
1312 |         urg_lbl.setStyleSheet(
1313 |             f"color: {cor_urg}; font-size: 13px; font-weight: 600;"
1314 |             f" background: transparent; padding: 0;"
1315 |         )
1316 |         man_grid.addWidget(urg_lbl, 3, 1)
1317 |
1318 |         btn_agendar = QPushButton("Agendar" if not data_prox else "Reagendar")
1319 |         btn_agendar.setToolTip("Definir ou alterar a data da próxima manutenção")
1320 |         btn_agendar.setCursor(Qt.PointingHandCursor)
1321 |         btn_agendar.setStyleSheet(ESTILO_BOTAO_SECUNDARIO)
1322 |         btn_agendar.clicked.connect(lambda: self._agendar_manutencao(prox_lbl))
1323 |
1324 |         btn_row = QHBoxLayout()
1325 |         btn_row.setSpacing(8)
1326 |         btn_row.addWidget(btn_agendar)

```

[illegible]

[illegible]

```

1453 |         self._main_window.pagina_os._abrir_edicao(a)
1454 |     elif a.kind == "MOVIMENTACAO":
1455 |         if hasattr(self._main_window, 'pagina_transferencias'):
1456 |             self._main_window.pagina_transferencias._abrir_edicao(a)
1457 |     self.recarregar()
1458 |
1459 | def _agendar_manutencao(self, prox_lbl: QLabel) -> None:
1460 |     dlg = _AgendarManutencaoDialog(self._printer, self)
1461 |     if dlg.exec() == QDialog.Accepted:
1462 |         data = dlg.data_agendada()
1463 |         urgencia = dlg.urgencia()
1464 |         printer_service = _obter_printer_service(self)
1465 |         if printer_service:
1466 |             printer_service.atualizar(
1467 |                 self._printer,
1468 |                 proxima_revisao=data.toPython() if data else None,
1469 |                 urgencia_prox_manutencao=urgencia,
1470 |             )
1471 |             self._atualizar_label_manutencao(prox_lbl)
1472 |
1473 | def _remover_agendamento(self, prox_lbl: QLabel, btn_agendar: QPushButton, btn_remover: QPushButton) -> None:
1474 |     printer_service = _obter_printer_service(self)
1475 |     if printer_service:
1476 |         printer_service.atualizar(self._printer, proxima_revisao=None, urgencia_prox_manutencao="Normal")
1477 |         prox_lbl.setText("Não agendada")
1478 |         prox_lbl.setStyleSheet("color: #717182; font-size: 13px; font-weight: 600; background: transparent;")
1479 |         btn_agendar.setText("Agendar")
1480 |         btn_remover.hide()
1481 |
1482 | def _atualizar_label_manutencao(self, prox_lbl: QLabel) -> None:
1483 |     data_prox = self._printer.proxima_revisao
1484 |     urgencia = getattr(self._printer, 'urgencia_prox_manutencao', '')
1485 |     badge = ""
1486 |     if urgencia and urgencia in URGENCIA_CORES:
1487 |         badge = f" [{urgencia}]"
1488 |     if data_prox:
1489 |         dias = (data_prox - dt.now()).days
1490 |         if dias < 0:
1491 |             cor = "#f38ba8"
1492 |             txt = f"{formatar_data(data_prox)} (VENCIDA há {abs(dias)} dias){badge}"
1493 |         elif dias <= 15:
1494 |             cor = "#f9e2af"
1495 |             txt = f"{formatar_data(data_prox)} (em {dias} dias){badge}"
1496 |         elif dias <= 30:
1497 |             cor = "#f9e2af"
1498 |             txt = f"{formatar_data(data_prox)} ({dias} dias){badge}"
1499 |         else:
1500 |             cor = "#a6e3a1"
1501 |             txt = f"{formatar_data(data_prox)}{badge}"
1502 |     else:
1503 |         cor = "#717182"
1504 |         txt = "Não agendada"
1505 |     prox_lbl.setText(txt)
1506 |     prox_lbl.setStyleSheet(
1507 |         f"color: {cor}; font-size: 13px; font-weight: 600;"
1508 |         f"background: transparent; padding: 0;"
1509 |     )
1510 |
1511 | def _nova_programacao(self, rec_box: Any) -> None:
1512 |     if not self._scheduler:
1513 |         return
1514 |     dlg = _NovaProgramacaoDialog(self._printer, self)
1515 |     if dlg.exec() == QDialog.Accepted:

```

```

1516 |         dados = dlg.dados()
1517 |         if dados["tipo"] == "periodo":
1518 |             self._scheduler.agendar_periodo(
1519 |                 printer_id=self._printer.id,
1520 |                 intervalo_dias=dados["intervalo_dias"],
1521 |                 dias_aviso=dados["dias_aviso"],
1522 |                 observacao=dados["observacao"],
1523 |             )
1524 |         else:
1525 |             self._scheduler.agendar_paginas(
1526 |                 printer_id=self._printer.id,
1527 |                 intervalo_paginas=dados["intervalo_paginas"],
1528 |                 contador_inicial=dados["contador_inicial"],
1529 |                 dias_aviso=dados["dias_aviso"],
1530 |                 observacao=dados["observacao"],
1531 |             )
1532 |         self._recarregar_programacao(rec_box)
1533 |
1534 |     def _cancelar_agendamento(self, schedule_id: int, rec_box: Any) -> None:
1535 |         if not self._scheduler:
1536 |             return
1537 |         self._scheduler.cancelar(schedule_id)
1538 |         self._recarregar_programacao(rec_box)
1539 |
1540 |     def _deletar_agendamento(self, schedule_id: int, rec_box: Any) -> None:
1541 |         if not self._scheduler:
1542 |             return
1543 |         if not ConfirmacaoDigitarDialog.confirmar(
1544 |             "Confirmar", "Excluir permanentemente esta programação recorrente?",
1545 |             self,
1546 |         ):
1547 |             return
1548 |         self._scheduler.deletar(schedule_id)
1549 |         self._recarregar_programacao(rec_box)
1550 |
1551 |     def _recarregar_programacao(self, rec_box: Any) -> None:
1552 |         novas_schedules = self._scheduler.listar_por_printer(self._printer.id) if self._scheduler else []
1553 |         inner = getattr(self, "_rec_inner", None)
1554 |         if inner is None:
1555 |             return
1556 |         self._limpar_layout(inner)
1557 |         if not novas_schedules:
1558 |             lbl_sem = QLabel("Nenhuma programação recorrente ativa.")
1559 |             lbl_sem.setStyleSheet("color: #717182; font-size: 13px; background: transparent;")
1560 |             inner.addWidget(lbl_sem)
1561 |         else:
1562 |             for sch in novas_schedules:
1563 |                 row_w = QHBoxLayout()
1564 |                 if sch.tipo == "periodo":
1565 |                     info = f"■ A cada {sch.intervalo_dias} dias | Aviso: {sch.dias_aviso} dia(s)"
1566 |                 else:
1567 |                     cnt = sch.contador_atual or 0
1568 |                     meta = sch.proxima_meta_paginas or 0
1569 |                     info = f"■ A cada {sch.intervalo_paginas} páginas | Atual: {cnt:,} | Meta: {meta:,}"
1570 |                 lbl_info = QLabel(info)
1571 |                 lbl_info.setStyleSheet("color: #cdd6f4; font-size: 13px; background: transparent;")
1572 |                 row_w.addWidget(lbl_info, 1)
1573 |
1574 |                 if sch.ativo:
1575 |                     btn_cancelar = QPushButton("Cancelar")
1576 |                     btn_cancelar.setStyleSheet(ESTILO_BOTAO_ERRO)
1577 |                     btn_cancelar.setCursor(Qt.PointingHandCursor)
1578 |                     btn_cancelar.clicked.connect(

```



```

1579 |         lambda checked, sid=sch.id, rb=rec_box: self._cancelar_agendamento(sid, rb)
1580 |     )
1581 |     row_w.addWidget(btn_cancelar)
1582 | else:
1583 |     lbl_inativo = QLabel("(cancelado)")
1584 |     lbl_inativo.setStyleSheet("color: #717182; font-size: 12px; background: transparent;")
1585 |     row_w.addWidget(lbl_inativo)
1586 |
1587 |     btn_apagar = QPushButton("■ Apagar")
1588 |     btn_apagar.setStyleSheet(ESTILO_BOTAO_ERR0)
1589 |     btn_apagar.setCursor(Qt.PointingHandCursor)
1590 |     btn_apagar.clicked.connect(
1591 |         lambda checked, sid=sch.id, rb=rec_box: self._deletar_agendamento(sid, rb)
1592 |     )
1593 |     row_w.addWidget(btn_apagar)
1594 |     inner.addLayout(row_w)
1595 |
1596 |     btn_nova_prog = QPushButton("+ Nova Programação")
1597 |     btn_nova_prog.setStyleSheet(ESTILO_BOTAO_SECUNDARIO)
1598 |     btn_nova_prog.setCursor(Qt.PointingHandCursor)
1599 |     btn_nova_prog.clicked.connect(lambda: self._nova_programacao(rec_box))
1600 |     inner.addWidget(btn_nova_prog)
1601 |
1602 | def _limpar_layout(self, layout: QVBoxLayout) -> None:
1603 |     while layout.count():
1604 |         item = layout.takeAt(0)
1605 |         if item.widget():
1606 |             item.widget().deleteLater()
1607 |         elif item.layout():
1608 |             self._limpar_layout(item.layout())
1609 |
1610 | def _build_tab_locais(self) -> QWidget:
1611 |     tab = QWidget()
1612 |     tab.setStyleSheet("background: transparent;")
1613 |     layout = QVBoxLayout(tab)
1614 |     layout.setContentsMargins(20, 16, 20, 16)
1615 |     layout.setSpacing(10)
1616 |
1617 |     atividades = [
1618 |         a for a in self._activity_service.listar_por_impressora(self._printer.id)
1619 |         if a.kind == "MOVIMENTACAO" and a.to_location
1620 |     ] if self._activity_service else []
1621 |
1622 |     if not atividades:
1623 |         empty = QLabel("Nenhum registro de localização encontrado.")
1624 |         empty.setStyleSheet("color: #717182; font-size: 13px; padding: 20px; background: transparent;")
1625 |         empty.setAlignment(Qt.AlignCenter)
1626 |         layout.addWidget(empty)
1627 |         return tab
1628 |
1629 |     t = TabelaPadrao(["Data/Hora", "Origem", "Destino", "Observação", "Status"])
1630 |     t.setRowCount(len(atividades))
1631 |     t.verticalHeader().setVisible(False)
1632 |     for i, a in enumerate(atividades):
1633 |         data_fmt = formatar_data_hora(a.event_at) if a.event_at else "-"
1634 |         t.setItem(i, 0, QTableWidgetItem(data_fmt))
1635 |         t.setItem(i, 1, QTableWidgetItem(a.from_location or "-"))
1636 |         t.setItem(i, 2, QTableWidgetItem(a.to_location or "-"))
1637 |         t.setItem(i, 3, QTableWidgetItem(a.notes or "-"))
1638 |         t.setItem(i, 4, QTableWidgetItem(a.status_atividade or "-"))
1639 |     t.redimensionar()
1640 |     layout.addWidget(t)
1641 |

```

```

def _abrir_transferencia(row: int, col: int) -> None:
    if row < 0 or row >= len(atividades):
        return
    a = atividades[row]
    if hasattr(self._main_window, 'pagina_transferencias'):
        self._main_window.pagina_transferencias._abrir_edicao(a, self)

t.cellDoubleClicked.connect(_abrir_transferencia)
return tab

def recarregar(self) -> None:
    if not hasattr(self, '_tabs'):
        return
    while self._tabs.count() > 1:
        w = self._tabs.widget(1)
        if w:
            w.deleteLater()
        else:
            break
    self._tabs.addTab(self._build_tab_atividades(), "\U0001f527 Atividades")
    self._tabs.addTab(self._build_tab_locais(), "\U0001f4cd Histórico de Locais")

# ■■ Página Principal ████████████████████████████████████████████████████████████

class PrintersPage(QWidget):
    session: Any
    printer_service: Any
    company_service: Any
    technician_service: Any
    activity_service: Any
    part_service: Any
    printer_location_service: Any
    _filtro_atual: str | None
    search: SearchBar
    tabela: TabelaPadrao
    _paginacao: PaginacaoWidget
    _impressoras_visiveis: list[Any]

    def __init__(
        self,
        session: Any,
        printer_service: Any,
        company_service: Any,
        technician_service: Any,
        activity_service: Any,
        part_service: Any,
        printer_location_service: Any = None,
        scheduler: MaintenanceScheduler | None = None,
        monitor_service: Any = None,
        parent: QWidget | None = None,
    ) -> None:
        super().__init__(parent)
        self.session = session
        self.printer_service = printer_service
        self.company_service = company_service
        self.technician_service = technician_service
        self.activity_service = activity_service
        self.part_service = part_service
        self.printer_location_service = printer_location_service
        self.scheduler = scheduler
        self.monitor_service = monitor_service
        self._impressoras_visiveis = []

```

```

1705 | self._filtro_atual = None
1706 |
1707 | layout = QVBoxLayout(self)
1708 | layout.setContentsMargins(24, 24, 24, 24)
1709 | layout.setSpacing(16)
1710 |
1711 | # ■■ Header ████████████████████████████████████████████
1712 | header = QHBoxLayout()
1713 | header.setSpacing(10)
1714 |
1715 | titulo_col = QVBoxLayout()
1716 | titulo_col.setSpacing(2)
1717 | titulo = QLabel("Impressoras")
1718 | titulo.setStyleSheet(ESTILO_TITULO_PAGINA)
1719 | titulo_col.addWidget(titulo)
1720 | sub = QLabel("Gerencie todas as impressoras cadastradas")
1721 | sub.setStyleSheet(ESTILO_SUBTITULO)
1722 | titulo_col.addWidget(sub)
1723 | header.addLayout(titulo_col)
1724 | header.addStretch()
1725 |
1726 | self.search = SearchBar(placeholder="Buscar por patrimônio, modelo, serial ou local...")
1727 | self.search.setMinimumWidth(360)
1728 | self.search.textChanged().connect(lambda t: self.filtrar(t))
1729 | header.addWidget(self.search)
1730 |
1731 | btn_nova = QPushButton("\u2795 Nova Impressora")
1732 | btn_nova.setToolTip("Cadastrar uma nova impressora")
1733 | btn_nova.setCursor(Qt.PointingHandCursor)
1734 | btn_nova.setStyleSheet(ESTILO_BOTAO_PRIMARIO)
1735 | btn_nova.clicked.connect(self._nova)
1736 | header.addWidget(btn_nova)
1737 |
1738 | btn_importar = QPushButton("\U0001f4e5 Importar")
1739 | btn_importar.setToolTip("Importar impressoras de um arquivo CSV ou XLSX")
1740 | btn_importar.setCursor(Qt.PointingHandCursor)
1741 | btn_importar.setStyleSheet(ESTILO_BOTAO_SECUNDARIO)
1742 | btn_importar.clicked.connect(lambda: self._importar())
1743 | header.addWidget(btn_importar)
1744 |
1745 | btn_atualizar = QPushButton("\U0001f504 Atualizar")
1746 | btn_atualizar.setToolTip("Recarregar a lista de impressoras")
1747 | btn_atualizar.setCursor(Qt.PointingHandCursor)
1748 | btn_atualizar.setStyleSheet(ESTILO_BOTAO_SECUNDARIO)
1749 | btn_atualizar.clicked.connect(self.recarregar)
1750 | header.addWidget(btn_atualizar)
1751 |
1752 | layout.addLayout(header)
1753 |
1754 | # ■■ Tabela ████████████████████████████████████████████
1755 | colunas = ["Patrimônio", "Serial", "Modelo", "Marca", "Status", "Rede", "Local Atual", "Atividades"]
1756 | self.tabela = TabelaPadrao(colunas)
1757 | self.tabela.cellDoubleClicked.connect(self._detalhes)
1758 | layout.addWidget(self.tabela)
1759 |
1760 | self._paginacao = PaginacaoWidget()
1761 | self._paginacao.pagina_alterada.connect(lambda p: self._carregar())
1762 | layout.addWidget(self._paginacao)
1763 |
1764 | self._carregar()
1765 |
1766 | def recarregar(self) -> None:
1767 |     texto = self.search.texto().strip() if hasattr(self, 'search') else ""

```

[illegible]

[illegible]

[illegible]


```

003 | from typing import Any
004 |
005 | from PySide6.QtCore import Qt
006 | from PySide6.QtGui import QColor
007 | from PySide6.QtWidgets import (
008 |     QComboBox,
009 |     QHBoxLayout,
010 |     QHeaderView,
011 |     QLabel,
012 |     QPushButton,
013 |     QTableWidget,
014 |     QTableWidgetItem,
015 |     QVBoxLayout,
016 |     QWidget,
017 | )
018 |
019 | from app.utils.helpers import formatar_data_hora
020 | from app.utils.ui_helpers import tratar_erro
021 | from app.views.styles.theme import (
022 |     ESTILO_BOTAO_AVISO,
023 |     ESTILO_BOTAO_ERRO,
024 |     ESTILO_BOTAO_SECUNDARIO,
025 |     ESTILO_BOTAO_SUCESSO,
026 |     ESTILO_TABELA_SIMPLES,
027 |     ESTILO_TITULO_PAGINA,
028 |     configurar_combo,
029 | )
030 | from app.views.widgets import ToastManager
031 | from app.views.widgets.table_widget import tornar_interativa
032 |
033 |
034 | class PurchaseRequisitionsPage(QWidget):
035 |     session: Any
036 |     part_service: Any
037 |     _filtro_status: str | None
038 |     _requisicoes: list[Any]
039 |     combo_filtro: QComboBox
040 |     tabela: QTableWidget
041 |     btn_recarregar: QPushButton
042 |
043 |     def __init__(self, session: Any, part_service: Any, parent: QWidget | None = None) -> None:
044 |         super().__init__(parent)
045 |         self.session = session
046 |         self.part_service = part_service
047 |         self._filtro_status = None
048 |
049 |         layout = QVBoxLayout(self)
050 |         layout.setContentsMargins(24, 24, 24, 24)
051 |         layout.setSpacing(16)
052 |
053 |         header = QHBoxLayout()
054 |         header.setSpacing(10)
055 |
056 |         titulo = QLabel("\U0001f6d2 Requisições de Compra")
057 |         titulo.setStyleSheet(ESTILO_TITULO_PAGINA)
058 |         header.addWidget(titulo)
059 |         header.addStretch()
060 |
061 |         self.combo_filtro = QComboBox()
062 |         configurar_combo(self.combo_filtro)
063 |         self.combo_filtro.addItem("Todas", "pendente", "aprovada", "recebida", "cancelada"])
064 |         self.combo_filtro.currentIndexChanged.connect(lambda: self._filtrar())
065 |         header.addWidget(self.combo_filtro)

```



```

066 |
067 |     self.btn_recarregar = QPushButton("\U0001f504 Recarregar")
068 |     self.btn_recarregar.setStyleSheet(ESTILO_BOTA0_SECUNDARIO)
069 |     self.btn_recarregar.setToolTip("Recarregar lista de requisições")
070 |     self.btn_recarregar.clicked.connect(self._carregar)
071 |     header.addWidget(self.btn_recarregar)
072 |
073 |     layout.addLayout(header)
074 |
075 |     self.tabela = QTableWidget()
076 |     self.tabela.setColumnCount(7)
077 |     self.tabela.setHorizontalHeaderLabels(["#", "Peça", "Qtd Sugerida", "Status", "Observação", "Criada"])
078 |     self.tabela.setStyleSheet(ESTILO_TABELA_SIMPLES)
079 |     self.tabela.setSelectionBehavior(QTableWidget.SelectRows)
080 |     self.tabela.setEditTriggers(QTableWidget.NoEditTriggers)
081 |     self.tabela.verticalHeader().setVisible(False)
082 |     self.tabela.setAlternatingRowColors(True)
083 |     self.tabela.verticalHeader().setDefaultSectionSize(36)
084 |     layout.addWidget(self.tabela)
085 |
086 |     self._requisicoes = []
087 |     self._carregar()
088 |
089 | def recarregar(self) -> None:
090 |     self._carregar()
091 |
092 | def _filtrar(self) -> None:
093 |     texto = self.combo_filtro.currentText()
094 |     self._filtro_status = texto if texto != "Todas" else None
095 |     self._carregar()
096 |
097 | def _carregar(self) -> None:
098 |     requisicoes = self.part_service.listar_requisicoes(status=self._filtro_status, limite=500)
099 |     self._requisicoes = requisicoes
100 |     self.tabela.setRowCount(len(requisicoes))
101 |
102 |     status_cores = {
103 |         "pendente": "#f59e0b",
104 |         "aprovada": "#3b82f6",
105 |         "recebida": "#10b981",
106 |         "cancelada": "#717182",
107 |     }
108 |
109 |     for i, r in enumerate(requisicoes):
110 |         items = [
111 |             (str(r.id), None),
112 |             (r.part.nome if r.part else "-", None),
113 |             (str(r.quantidade_sugerida), None),
114 |             (r.status, None),
115 |             (r.observacao or "", None),
116 |             (formatar_data_hora(r.created_at), None),
117 |         ]
118 |         for j, (texto, _) in enumerate(items):
119 |             item = QTableWidgetItem(texto)
120 |             item.setTextAlignment(Qt.AlignCenter if j in (2,) else Qt.AlignLeft)
121 |             self.tabela.setItem(i, j, item)
122 |
123 |         cor = status_cores.get(r.status, "#e8e8f0")
124 |         self.tabela.item(i, 3).setForeground(QColor(cor))
125 |
126 |         container = QWidget()
127 |         row_layout = QHBoxLayout(container)
128 |         row_layout.setContentsMargins(4, 2, 4, 2)

```

```

129 |         row_layout.setSpacing(4)
130 |
131 |     def make_handler(req_id: int, acao: str) -> Any:
132 |         def handler() -> None:
133 |             if acao == "aprovar":
134 |                 self._aprovar(req_id)
135 |             elif acao == "receber":
136 |                 self._receber(req_id)
137 |             elif acao == "cancelar":
138 |                 self._cancelar(req_id)
139 |         return handler
140 |
141 |     if r.status == "pendente":
142 |         btn_aprovar = QPushButton("Aprovar")
143 |         btn_aprovar.setStyleSheet(ESTILO_BOTAO_SUCESSO)
144 |         btn_aprovar.setFixedHeight(28)
145 |         btn_aprovar.clicked.connect(make_handler(r.id, "aprovar"))
146 |         row_layout.addWidget(btn_aprovar)
147 |
148 |         btn_receber = QPushButton("Receber")
149 |         btn_receber.setStyleSheet(ESTILO_BOTAO_AVISO)
150 |         btn_receber.setFixedHeight(28)
151 |         btn_receber.clicked.connect(make_handler(r.id, "receber"))
152 |         row_layout.addWidget(btn_receber)
153 |
154 |         btn_cancelar = QPushButton("Cancelar")
155 |         btn_cancelar.setStyleSheet(ESTILO_BOTAO_ERR0)
156 |         btn_cancelar.setFixedHeight(28)
157 |         btn_cancelar.clicked.connect(make_handler(r.id, "cancelar"))
158 |         row_layout.addWidget(btn_cancelar)
159 |
160 |     elif r.status == "aprovada":
161 |         btn_receber = QPushButton("Receber")
162 |         btn_receber.setStyleSheet(ESTILO_BOTAO_AVISO)
163 |         btn_receber.setFixedHeight(28)
164 |         btn_receber.clicked.connect(make_handler(r.id, "receber"))
165 |         row_layout.addWidget(btn_receber)
166 |
167 |         btn_cancelar = QPushButton("Cancelar")
168 |         btn_cancelar.setStyleSheet(ESTILO_BOTAO_ERR0)
169 |         btn_cancelar.setFixedHeight(28)
170 |         btn_cancelar.clicked.connect(make_handler(r.id, "cancelar"))
171 |         row_layout.addWidget(btn_cancelar)
172 |
173 |     elif r.status == "recebida":
174 |         pass
175 |
176 |     elif r.status == "cancelada":
177 |         pass
178 |
179 |     self.tabela.setCellWidget(i, 6, container)
180 |
181 |     tornar_interativa(self.tabela)
182 |     self.tabela.horizontalHeader().setSectionResizeMode(0, QHeaderView.ResizeToContents)
183 |     self.tabela.horizontalHeader().setSectionResizeMode(2, QHeaderView.ResizeToContents)
184 |     self.tabela.horizontalHeader().setSectionResizeMode(3, QHeaderView.ResizeToContents)
185 |     self.tabela.horizontalHeader().setSectionResizeMode(5, QHeaderView.ResizeToContents)
186 |     self.tabela.horizontalHeader().setSectionResizeMode(6, QHeaderView.ResizeToContents)
187 |
188 | def _executar_acao(self, req_id: int, acao: str) -> None:
189 |     rotulos = {"aprovar": "aprovada", "receber": "recebida", "cancelar": "cancelada"}
190 |     metodos = {
191 |         "aprovar": self.part_service.aprovar_requisicao,

```

```

192 |         "receber": self.part_service.receber_requisicao,
193 |         "cancelar": self.part_service.cancelar_requisicao,
194 |     }
195 |     with tratar_erro(f"{acao} requisição"):
196 |         metodos[acao](req_id)
197 |         ToastManager.mostrar(f"Requisição #{req_id} {rotulos[acao]}.", "sucesso")
198 |         self._carregar()
199 |
200 |     def _aprovar(self, req_id: int) -> None:
201 |         self._executar_acao(req_id, "aprovar")
202 |
203 |     def _receber(self, req_id: int) -> None:
204 |         self._executar_acao(req_id, "receber")
205 |
206 |     def _cancelar(self, req_id: int) -> None:
207 |         self._executar_acao(req_id, "cancelar")

```

=== ARQUIVO: .lapplviews\pages\reports_page.py ===

```

001 | from __future__ import annotations
002 |
003 | from typing import Any
004 |
005 | import matplotlib
006 | from PySide6.QtCore import Qt
007 | from PySide6.QtWidgets import QGridLayout, QHBoxLayout, QLabel, QPushButton, QVBoxLayout, QWidget
008 |
009 | matplotlib.use('QtAgg')
010 | from matplotlib.backends.backend_qtagg import FigureCanvasQTAgg as FigureCanvas
011 | from matplotlib.figure import Figure
012 |
013 | from app.views.relatorio_dialog import RelatorioDialog
014 | from app.views.styles.theme import COR, ESTILO_TITULO_PAGINA, group_box, input_label, campo_rotulo, campo_r
015 |
016 | BG: str = COR["fundo"]
017 | CARD: str = COR["fundo_card"]
018 | BORDA: str = COR["borda"]
019 | TEXTO: str = COR["texto"]
020 | AZUL: str = COR["azul"]
021 | VERDE: str = COR["sucesso"]
022 | AMARELO: str = COR["aviso"]
023 | ROXO: str = COR["roxo"]
024 | CINZA: str = COR["texto_label"]
025 |
026 |
027 | class ReportsPage(QWidget):
028 |     _session: Any
029 |     _printer_service: Any
030 |     _activity_service: Any
031 |     _company_service: Any
032 |     canvas_status: FigureCanvas | None
033 |     canvas_modelos: FigureCanvas | None
034 |     canvas_atividades: FigureCanvas | None
035 |     canvas_pecas: FigureCanvas | None
036 |     btn_relatorio: QPushButton
037 |     grid: QGridLayout
038 |
039 |     def __init__(self, session: Any, printer_service: Any, activity_service: Any, company_service: Any, par
040 |         super().__init__(parent)
041 |         self._session = session
042 |         self._printer_service = printer_service

```

```

043 |         self._activity_service = activity_service
044 |         self._company_service = company_service
045 |
046 |         self.canvas_status = None
047 |         self.canvas_modelos = None
048 |         self.canvas_atividades = None
049 |         self.canvas_pecas = None
050 |
051 |         layout = QVBoxLayout(self)
052 |         layout.setContentsMargins(30, 30, 30, 30)
053 |         layout.setSpacing(16)
054 |
055 |         header = QHBoxLayout()
056 |         titulo = QLabel("■ Relatórios e Gráficos")
057 |         titulo.setStyleSheet(ESTILO_TITULO_PAGINA)
058 |         header.addWidget(titulo)
059 |         header.addStretch()
060 |         layout.addLayout(header)
061 |
062 |         self.btn_relatorio = QPushButton("Gerar Relatório Personalizado")
063 |         self.btn_relatorio.setToolTip("Abrir diálogo para gerar relatório personalizado")
064 |         self.btn_relatorio.setMinimumHeight(56)
065 |         self.btn_relatorio.setCursor(Qt.PointingHandCursor)
066 |         self.btn_relatorio.setStyleSheet("""
067 |             QPushButton {
068 |                 background: qlineargradient(x1:0, y1:0, x2:1, y2:1,
069 |                     stop:0 #a78bfa, stop:1 #7c3aed);
070 |                 color: #ffffff;
071 |                 border: none; border-radius: 12px;
072 |                 padding: 16px 24px; font-size: 15px; font-weight: 700;
073 |                 text-align: center;
074 |             }
075 |             QPushButton:hover {
076 |                 background: qlineargradient(x1:0, y1:0, x2:1, y2:1,
077 |                     stop:0 #c4b5fd, stop:1 #a78bfa);
078 |             }
079 |         """)
080 |         self.btn_relatorio.clicked.connect(self._abrir_relatorio)
081 |         layout.addWidget(self.btn_relatorio)
082 |
083 |         self.grid = QGridLayout()
084 |         self.grid.setSpacing(16)
085 |         layout.addLayout(self.grid)
086 |
087 |     def recarregar(self) -> None:
088 |         for i in reversed(range(self.grid.count())):
089 |             item = self.grid.itemAt(i)
090 |             if item and item.widget():
091 |                 item.widget().deleteLater()
092 |
093 |         self.canvas_status = self._criar_grafico_pizza()
094 |         self.canvas_modelos = self._criar_grafico_barras()
095 |         self.canvas_atividades = self._criar_grafico_linhas()
096 |         self.canvas_pecas = self._criar_grafico_barras_horiz()
097 |
098 |         CANVAS_STYLE = f"""
099 |             QWidget {{
100 |                 background-color: {CARD};
101 |                 border-radius: 12px;
102 |                 border: 1px solid {BORDA};
103 |             }}
104 |         """
105 |

```

```

106 |         for canvas in [self.canvas_status, self.canvas_modelos, self.canvas_atividades, self.canvas_pecas]:
107 |             canvas.setStyleSheet(CANVAS_STYLE)
108 |
109 |         self.grid.addWidget(self.canvas_status, 0, 0)
110 |         self.grid.addWidget(self.canvas_modelos, 0, 1)
111 |         self.grid.addWidget(self.canvas_atividades, 1, 0)
112 |         self.grid.addWidget(self.canvas_pecas, 1, 1)
113 |
114 |     def _estilo_ax(self, ax: Any, titulo: str, cor_titulo: str = AMARELO) -> None:
115 |         ax.set_title(titulo, color=cor_titulo, fontsize=12, fontweight='bold')
116 |         ax.tick_params(colors=CINZA)
117 |         ax.spines['bottom'].set_color(BORDA)
118 |         ax.spines['left'].set_color(BORDA)
119 |         ax.spines['top'].set_visible(False)
120 |         ax.spines['right'].set_visible(False)
121 |         ax.xaxis.label.set_color(CINZA)
122 |         ax.yaxis.label.set_color(CINZA)
123 |
124 |     def _criar_grafico_pizza(self) -> FigureCanvas:
125 |         fig = Figure(facecolor=BG)
126 |         ax = fig.add_subplot(111)
127 |         ax.set_facecolor(BG)
128 |
129 |         dados = self._printer_service.agrupar_por_status()
130 |         if dados:
131 |             labels = [d[0] for d in dados]
132 |             valores = [d[1] for d in dados]
133 |             cores = [AMARELO, AZUL, VERDE, ROXO, '#f0883e', '#f38ba8', CINZA]
134 |             ax.pie(valores, labels=labels, autopct='%1.1f%%', startangle=90,
135 |                   colors=cores[:len(valores)], textprops={'color': TEXTO, 'fontsize': 9})
136 |         else:
137 |             ax.text(0.5, 0.5, 'Sem dados', ha='center', va='center', color=CINZA, fontsize=12)
138 |
139 |         self._estilo_ax(ax, 'Impressoras por Status')
140 |         canvas = FigureCanvas(fig)
141 |         return canvas
142 |
143 |     def _criar_grafico_barras(self) -> FigureCanvas:
144 |         fig = Figure(facecolor=BG)
145 |         ax = fig.add_subplot(111)
146 |         ax.set_facecolor(BG)
147 |
148 |         dados = self._printer_service.top_modelos(5)
149 |         if dados:
150 |             labels = [d[0] if d[0] else 'N/A' for d in dados]
151 |             valores = [d[1] for d in dados]
152 |             ax.barh(labels[:-1], valores[:-1], color=AZUL, edgecolor=BORDA)
153 |         else:
154 |             ax.text(0.5, 0.5, 'Sem dados', ha='center', va='center', color=CINZA, fontsize=12)
155 |             ax.set_xlim(0, 1)
156 |
157 |         self._estilo_ax(ax, 'Top 5 Modelos')
158 |         canvas = FigureCanvas(fig)
159 |         return canvas
160 |
161 |     def _criar_grafico_linhas(self) -> FigureCanvas:
162 |         fig = Figure(facecolor=BG)
163 |         ax = fig.add_subplot(111)
164 |         ax.set_facecolor(BG)
165 |
166 |         dados = self._activity_service.contar_ultimos_6_meses()
167 |         if dados and dados.get('labels'):
168 |             meses = dados['labels']

```

```

169 |         manut = dados['manut']
170 |         mov = dados['mov']
171 |         ax.plot(meses, manut, marker='o', color=AMARELO, label='Manutenção', linewidth=2)
172 |         ax.plot(meses, mov, marker='s', color=AZUL, label='Movimentação', linewidth=2)
173 |         ax.legend(frameon=False, labelcolor=TEXT0)
174 |     else:
175 |         ax.text(0.5, 0.5, 'Sem dados', ha='center', va='center', color=CINZA, fontsize=12)
176 |         ax.set_xlim(0, 1)
177 |
178 |     self._estilo_ax(ax, 'Atividades por Mês (últimos 6 meses)')
179 |     canvas = FigureCanvas(fig)
180 |     return canvas
181 |
182 | def _criar_grafico_barras_horiz(self) -> FigureCanvas:
183 |     fig = Figure(facecolor=BG)
184 |     ax = fig.add_subplot(111)
185 |     ax.set_facecolor(BG)
186 |
187 |     dados = self._activity_service.top_pecas_trocadas(8)
188 |     if dados:
189 |         labels = [d[0] for d in dados]
190 |         valores = [d[1] for d in dados]
191 |         ax.barh(labels[::-1], valores[::-1], color=VERDE, edgecolor=BORDA)
192 |     else:
193 |         ax.text(0.5, 0.5, 'Sem dados', ha='center', va='center', color=CINZA, fontsize=12)
194 |         ax.set_xlim(0, 1)
195 |
196 |     self._estilo_ax(ax, 'Peças Mais Trocadas')
197 |     canvas = FigureCanvas(fig)
198 |     return canvas
199 |
200 | def _abrir_relatorio(self) -> None:
201 |     dialogo = RelatorioDialog(self._session, self._printer_service, self._company_service, self._activi
202 |     dialogo.exec()

```

=== ARQUIVO: .lapp\views\pages\technicians_page.py ===

```

001 | from __future__ import annotations
002 |
003 | from typing import Any
004 |
005 | from PySide6.QtCore import Qt
006 | from PySide6.QtWidgets import (
007 |     QDialog,
008 |     QDialogButtonBox,
009 |     QFormLayout,
010 |     QFrame,
011 |     QHBoxLayout,
012 |     QLabel,
013 |     QLineEdit,
014 |     QMessageBox,
015 |     QPushButton,
016 |     QScrollArea,
017 |     QTableWidgetItem,
018 |     QVBoxLayout,
019 |     QWidget,
020 | )
021 |
022 | from app.views.styles.theme import (
023 |     ESTILO_BOTAO_AVISO,
024 |     ESTILO_BOTAO_ERRO,

```

```

025 |     ESTILO_BOTAO_FECHAR,
026 |     ESTILO_BOTAO_PRIMARIO,
027 |     ESTILO_BOTAO_SUCESSO,
028 |     ESTILO_DIALOG,
029 |     ESTILO_INPUT,
030 |     ESTILO_TITULO_PAGINA,
031 |     group_box,
032 |     input_label,
033 |     campo_rotulo,
034 |     campo_readonly,
035 | )
036 | from app.utils.validacao import ValidadorCampo, obrigatorio, email
037 | from app.views.widgets import ToastManager
038 | from app.views.widgets.confirm_dialog import ConfirmacaoDigitalDialog
039 | from app.views.widgets.pagination import PaginacaoWidget
040 | from app.views.widgets.table_widget import TabelaPadrao
041 |
042 |
043 | class _TechnicianDialog(QDialog):
044 |     input_nome: QLineEdit
045 |     input_exibicao: QLineEdit
046 |     input_telefone: QLineEdit
047 |     input_email: QLineEdit
048 |     btn_salvar: QPushButton
049 |     btn_excluir: QPushButton
050 |     _excluir_confirmado: bool
051 |
052 |     def __init__(self, parent: QWidget | None = None, dados: dict[str, Any] | None = None) -> None:
053 |         super().__init__(parent)
054 |         self.setWindowTitle("Editar Técnico" if dados else "Novo Técnico")
055 |         self.setMinimumWidth(400)
056 |         self.setStyleSheet(ESTILO_DIALOG)
057 |
058 |         layout = QVBoxLayout(self)
059 |         layout.setSpacing(12)
060 |         layout.setContentsMargins(20, 20, 20, 20)
061 |
062 |         # ■■ Identificação ■■
063 |         id_box, id_layout = group_box("Identificação")
064 |         id_form = QFormLayout()
065 |         id_form.setLabelAlignment(Qt.AlignRight)
066 |
067 |         self.input_nome = QLineEdit()
068 |         self.input_nome.setStyleSheet(ESTILO_INPUT)
069 |         self.input_nome.setPlaceholderText("Nome completo do técnico")
070 |         self.input_nome.setMaxLength(150)
071 |
072 |         self.input_exibicao = QLineEdit()
073 |         self.input_exibicao.setStyleSheet(ESTILO_INPUT)
074 |         self.input_exibicao.setPlaceholderText("Nome de exibição (apelido)")
075 |         self.input_exibicao.setMaxLength(80)
076 |
077 |         id_form.addRow("Nome Completo:", self.input_nome)
078 |
079 |         erro_nome = QLabel()
080 |         erro_nome.setStyleSheet("color: #ef4444; font-size: 10px; background: transparent;")
081 |         erro_nome.hide()
082 |         id_form.addRow("", erro_nome)
083 |
084 |         id_form.addRow("Nome Exibição:", self.input_exibicao)
085 |         id_layout.addLayout(id_form)
086 |         layout.addWidget(id_box)
087 |

```

```

088 |         # ■■ Contato ■■
089 |         ct_box, ct_layout = group_box("Contato")
090 |         ct_form = QFormLayout()
091 |         ct_form.setLabelAlignment(Qt.AlignRight)
092 |
093 |         self.input_telefone = QLineEdit()
094 |         self.input_telefone.setStyleSheet(ESTILO_INPUT)
095 |         self.input_telefone.setPlaceholderText("(81) 99999-9999")
096 |         self.input_telefone.setMaxLength(20)
097 |
098 |         self.input_email = QLineEdit()
099 |         self.input_email.setStyleSheet(ESTILO_INPUT)
100 |         self.input_email.setPlaceholderText("email@exemplo.com")
101 |         self.input_email.setMaxLength(120)
102 |
103 |         ct_form.addRow("Telefone:", self.input_telefone)
104 |         ct_form.addRow("Email:", self.input_email)
105 |
106 |         erro_email = QLabel()
107 |         erro_email.setStyleSheet("color: #ef4444; font-size: 10px; background: transparent;")
108 |         erro_email.hide()
109 |         ct_form.addRow("", erro_email)
110 |
111 |         ct_layout.addLayout(ct_form)
112 |         layout.addWidget(ct_box)
113 |
114 |         ValidadorCampo(self.input_nome, obrigatorio, erro_nome)
115 |         ValidadorCampo(self.input_email, email, erro_email)
116 |
117 |         layout.addStretch()
118 |
119 |         if dados:
120 |             self.input_nome.setText(dados.get("nome_completo", ""))
121 |             self.input_exibicao.setText(dados.get("nome_exibicao", ""))
122 |             self.input_telefone.setText(dados.get("telefone", ""))
123 |             self.input_email.setText(dados.get("email", ""))
124 |
125 |             botoes = QHBoxLayout()
126 |             self.btn_salvar = QPushButton("■ Salvar")
127 |             self.btn_salvar.setToolTip("Salvar dados do técnico")
128 |             self.btn_salvar.setStyleSheet(ESTILO_BOTAO_SUCESSO)
129 |             self.btn_salvar.clicked.connect(self.accept)
130 |
131 |             self.btn_excluir = QPushButton("■■ Excluir")
132 |             self.btn_excluir.setToolTip("Excluir este técnico (pode ser desfeito pela Lixeira)")
133 |             self.btn_excluir.setStyleSheet(ESTILO_BOTAO_ERRO)
134 |             self.btn_excluir.clicked.connect(self._confirmar_exclusao)
135 |
136 |             botoes.addWidget(self.btn_salvar)
137 |             botoes.addWidget(self.btn_excluir)
138 |             layout.addLayout(botoes)
139 |         else:
140 |             botoes = QDialogButtonBox(QDialogButtonBox.Ok | QDialogButtonBox.Cancel)
141 |             botoes.accepted.connect(self.accept)
142 |             botoes.rejected.connect(self.reject)
143 |             botoes.button(QDialogButtonBox.Ok).setText("■ Salvar")
144 |             botoes.button(QDialogButtonBox.Ok).setToolTip("Salvar novo técnico")
145 |             botoes.button(QDialogButtonBox.Cancel).setText("Cancelar")
146 |             botoes.button(QDialogButtonBox.Cancel).setToolTip("Cancelar")
147 |             botoes.button(QDialogButtonBox.Ok).setStyleSheet(ESTILO_BOTAO_SUCESSO)
148 |             botoes.button(QDialogButtonBox.Cancel).setStyleSheet(ESTILO_BOTAO_AVISO)
149 |             layout.addWidget(botoes)
150 |

```



```

151 |         self._excluir_confirmado = False
152 |
153 |     def dados(self) -> dict[str, str]:
154 |         return {
155 |             "nome_completo": self.input_nome.text().strip(),
156 |             "nome_exibicao": self.input_exibicao.text().strip(),
157 |             "telefone": self.input_telefone.text().strip(),
158 |             "email": self.input_email.text().strip(),
159 |         }
160 |
161 |     def _confirmar_exclusao(self) -> None:
162 |         if ConfirmacaoDigitarDialog.confirmar(
163 |             "Confirmar Exclusão",
164 |             "Tem certeza que deseja excluir este técnico?",
165 |             self,
166 |         ):
167 |             self._excluir_confirmado = True
168 |             self.accept()
169 |
170 |     def excluir_confirmado(self) -> bool:
171 |         return self._excluir_confirmado
172 |
173 |
174 | class _EspecialidadesDialog(QDialog):
175 |     def __init__(self, tecnico: Any, printer_service: Any, technician_service: Any, parent: QWidget | None
176 |         super().__init__(parent)
177 |         self._technician_service = technician_service
178 |         self._printer_service = printer_service
179 |         self._tecnico = tecnico
180 |         self.setWindowTitle(f"Especialidades – {tecnico.nome_exibicao or tecnico.nome_completo}")
181 |         self.setMinimumWidth(500)
182 |         self.setMinimumHeight(450)
183 |         self.setStyleSheet(ESTILO_DIALOG)
184 |         layout = QVBoxLayout(self)
185 |         layout.setContentsMargins(20, 20, 20, 20)
186 |         layout.setSpacing(12)
187 |
188 |         lbl = QLabel("Modelos de impressoras que este técnico pode atender:")
189 |         lbl.setStyleSheet("color: #94949f; font-size: 12px; background: transparent;")
190 |         lbl.setWordWrap(True)
191 |         layout.addWidget(lbl)
192 |
193 |         # ■■ Input para novo modelo ■■
194 |         add_frame = QFrame()
195 |         add_frame.setStyleSheet("QFrame { background: transparent; border: none; }")
196 |         add_layout = QHBoxLayout(add_frame)
197 |         add_layout.setContentsMargins(0, 0, 0, 0)
198 |         add_layout.setSpacing(8)
199 |
200 |         self._novo_modelo_input = QLineEdit()
201 |         self._novo_modelo_input.setStyleSheet(ESTILO_INPUT)
202 |         self._novo_modelo_input.setPlaceholderText("Digite um novo modelo (ex: HP M425dn)...")
203 |         add_layout.addWidget(self._novo_modelo_input)
204 |
205 |         btn_adicionar = QPushButton("+ Adicionar")
206 |         btn_adicionar.setStyleSheet(ESTILO_BOTAO_SUCESSO)
207 |         btn_adicionar.clicked.connect(self._adicionar_modelo)
208 |         add_layout.addWidget(btn_adicionar)
209 |         layout.addWidget(add_frame)
210 |
211 |         # ■■ Lista de modelos disponíveis (do banco) ■■
212 |         lbl_disponiveis = QLabel("Modelos existentes no sistema:")
213 |         lbl_disponiveis.setStyleSheet("color: #717182; font-size: 11px; background: transparent; font-weight: normal;")

```

```

214 |         layout.addWidget(lbl_disponiveis)
215 |
216 |         scroll = QScrollArea()
217 |         scroll.setWidgetResizable(True)
218 |         scroll.setStyleSheet("QScrollArea { border: 1px solid #2a2a3e; border-radius: 8px; background: tran
219 |         container = QWidget()
220 |         container.setStyleSheet("background: transparent;")
221 |         self._check_layout = QVBoxLayout(container)
222 |         self._check_layout.setContentsMargins(8, 8, 8, 8)
223 |         self._check_layout.setSpacing(2)
224 |         scroll.setWidget(container)
225 |         layout.addWidget(scroll, 1)
226 |
227 |         self._checkboxes: dict[str, QWidget] = {}
228 |         modelos_tec = set(technician_service.listar_especialidades(tecnico.id))
229 |
230 |         from collections import OrderedDict
231 |         seen = OrderedDict()
232 |         for p in printer_service.listar_todos():
233 |             modelo = (p.modelo or p.marca or "Sem modelo").strip()
234 |             if modelo and modelo not in seen:
235 |                 seen[modelo] = True
236 |
237 |         for modelo in seen:
238 |             self._adicionar_checkbox(modelo, modelo in modelos_tec)
239 |
240 |         self._check_layout.addStretch()
241 |
242 |         botoes = QHBoxLayout()
243 |         btn_salvar = QPushButton("■ Salvar")
244 |         btn_salvar.setStyleSheet(ESTILO_BOTAO_SUCESSO)
245 |         btn_salvar.clicked.connect(self._salvar)
246 |         botoes.addWidget(btn_salvar)
247 |         btn_cancelar = QPushButton("Cancelar")
248 |         btn_cancelar.setStyleSheet(ESTILO_BOTAO_FECHAR)
249 |         btn_cancelar.clicked.connect(self.reject)
250 |         botoes.addWidget(btn_cancelar)
251 |         layout.addLayout(botoes)
252 |
253 |     def _adicionar_checkbox(self, modelo: str, checked: bool = False) -> None:
254 |         from PySide6.QtWidgets import QCheckBox
255 |         cb = QCheckBox(modelo)
256 |         cb.setChecked(checked)
257 |         cb.setStyleSheet("color: #e8e8f0; font-size: 12px; background: transparent;")
258 |         self._checkboxes[modelo] = cb
259 |         self._check_layout.insertWidget(self._check_layout.count() - 1, cb)
260 |
261 |     def _adicionar_modelo(self) -> None:
262 |         modelo = self._novo_modelo_input.text().strip()
263 |         if not modelo:
264 |             return
265 |         if modelo not in self._checkboxes:
266 |             self._adicionar_checkbox(modelo, True)
267 |             self._novo_modelo_input.clear()
268 |
269 |     def _salvar(self) -> None:
270 |         selected = [m for m, cb in self._checkboxes.items() if cb.isChecked()]
271 |         atuais = set(self._technician_service.listar_especialidades(self._tecnico.id))
272 |         to_add = [m for m in selected if m not in atuais]
273 |         to_remove = [m for m in atuais if m not in selected]
274 |         if to_remove:
275 |             self._technician_service.remover_especialidades(self._tecnico.id, to_remove)
276 |         if to_add:

```

```

277 |         self._technician_service.adicionar_especialidades(self._tecnico.id, to_add)
278 |     self.accept()
279 |
280 |
281 | class TechniciansPage(QWidget):
282 |     COLUNAS: list[str] = ["Nome Completo", "Exibição", "Telefone", "Email", "Ativo"]
283 |
284 |     _session: Any
285 |     _technician_service: Any
286 |     _printer_service: Any
287 |     _tecnicos_visiveis: list[Any]
288 |     btn_novo: QPushButton
289 |     tabela: TabelaPadrao
290 |     _paginacao: PaginacaoWidget
291 |
292 |     def __init__(self, session: Any, technician_service: Any, printer_service: Any | None = None, parent: Q
293 |         super().__init__(parent)
294 |         self._session = session
295 |         self._technician_service = technician_service
296 |         self._printer_service = printer_service
297 |
298 |         layout = QVBoxLayout(self)
299 |         layout.setContentsMargins(24, 24, 24, 24)
300 |         layout.setSpacing(16)
301 |
302 |         header = QHBoxLayout()
303 |         titulo = QLabel("■■■ Técnicos")
304 |         titulo.setStyleSheet(ESTILO_TITULO_PAGINA)
305 |         header.addWidget(titulo)
306 |         header.addStretch()
307 |
308 |         self.btn_novo = QPushButton("■ Novo Técnico")
309 |         self.btn_novo.setToolTip("Cadastrar novo técnico")
310 |         self.btn_novo.setStyleSheet(ESTILO_BOTAO_SUCESSO)
311 |         self.btn_novo.clicked.connect(self._novo)
312 |         header.addWidget(self.btn_novo)
313 |
314 |         layout.addLayout(header)
315 |
316 |         self._tecnicos_visiveis = []
317 |         self.tabela = TabelaPadrao(self.COLUNAS)
318 |         self.tabela.cellDoubleClicked.connect(self._detalhes)
319 |         layout.addWidget(self.tabela, 1)
320 |
321 |         self._paginacao = PaginacaoWidget()
322 |         self._paginacao.pagina_alterada.connect(lambda p: self._carregar())
323 |         layout.addWidget(self._paginacao)
324 |
325 |         self._carregar()
326 |
327 |     def recarregar(self) -> None:
328 |         self._carregar()
329 |
330 |     def _carregar(self) -> None:
331 |         tecnicos = self._technician_service.listar_todos(
332 |             limite=self._paginacao.limit, offset=self._paginacao.offset
333 |         )
334 |         total = self._technician_service.contar_todos()
335 |         self._paginacao.configurar(total, pagina_atual=self._paginacao.pagina,
336 |             itens_por_pagina=self._paginacao.limit)
337 |         self._tecnicos_visiveis = tecnicos
338 |         self.tabela.limpar()
339 |         self.tabela.setRowCount(len(tecnicos))

```

```

340 |         for i, t in enumerate(tecnicos):
341 |             self.tabela.setItem(i, 0, QTableWidgetItem(t.nome_completo))
342 |             self.tabela.setItem(i, 1, QTableWidgetItem(t.nome_exibicao))
343 |             self.tabela.setItem(i, 2, QTableWidgetItem(t.telefone or ""))
344 |             self.tabela.setItem(i, 3, QTableWidgetItem(t.email or ""))
345 |             ativo = "■" if t.ativo else "■"
346 |             item_ativo = QTableWidgetItem(ativo)
347 |             item_ativo.setTextAlignment(Qt.AlignCenter)
348 |             self.tabela.setItem(i, 4, item_ativo)
349 |         self.tabela.redimensionar()
350 |
351 |     def _novo(self) -> None:
352 |         dialogo = _TechnicianDialog(self)
353 |         if dialogo.exec() == QDialog.Accepted:
354 |             dados = dialogo.dados()
355 |             if not dados["nome_completo"]:
356 |                 QMessageBox.warning(self, "Aviso", "O campo Nome Completo é obrigatório.")
357 |                 return
358 |             try:
359 |                 self._technician_service.criar(**dados)
360 |                 self.recarregar()
361 |             except Exception as e:
362 |                 QMessageBox.critical(self, "Erro", f"Erro ao criar técnico:\n{e}")
363 |
364 |     def _detalhes(self, row: int) -> None:
365 |         if row < 0 or row >= len(self._tecnicos_visiveis):
366 |             return
367 |         tecnico = self._tecnicos_visiveis[row]
368 |
369 |         dialogo = QDialog(self)
370 |         dialogo.setWindowTitle(tecnico.nome_exibicao or "Técnico")
371 |         dialogo.setMinimumSize(500, 400)
372 |         dialogo.setStyleSheet(ESTILO_DIALOG)
373 |
374 |         root = QVBoxLayout(dialogo)
375 |         root.setContentsMargins(0, 0, 0, 0)
376 |         root.setSpacing(0)
377 |
378 |         scroll = QScrollArea()
379 |         scroll.setWidgetResizable(True)
380 |         scroll.setHorizontalScrollBarPolicy(Qt.ScrollBarAlwaysOff)
381 |         scroll.setStyleSheet("QScrollArea { border: none; background: transparent; }")
382 |
383 |         container = QWidget()
384 |         container.setStyleSheet("background: transparent;")
385 |         layout = QVBoxLayout(container)
386 |         layout.setContentsMargins(24, 20, 24, 20)
387 |         layout.setSpacing(20)
388 |
389 |         # ■■ Identificação ■■
390 |         id_box, id_layout = group_box("Identificação")
391 |         id_form = QFormLayout()
392 |         id_form.setLabelAlignment(Qt.AlignRight)
393 |         id_form.addRow(campo_rotulo("Nome Completo"), campo_readonly(tecnico.nome_completo))
394 |         id_form.addRow(campo_rotulo("Nome Exibição"), campo_readonly(tecnico.nome_exibicao))
395 |         id_layout.addLayout(id_form)
396 |         layout.addWidget(id_box)
397 |
398 |         # ■■ Contato ■■
399 |         ct_box, ct_layout = group_box("Contato")
400 |         ct_form = QFormLayout()
401 |         ct_form.setLabelAlignment(Qt.AlignRight)
402 |         ct_form.addRow(campo_rotulo("Telefone"), campo_readonly(tecnico.telefone or "-"))

```

```

403 |         ct_form.addRow(campo_rotulo("Email"), campo_readonly(tecnico.email or "-"))
404 |         ct_layout.addLayout(ct_form)
405 |         layout.addWidget(ct_box)
406 |
407 |         # ■■ Especialidades ■■
408 |         esp_box, esp_layout = group_box("Especialidades")
409 |         esp_inner = QVBoxLayout()
410 |         esp_inner.setSpacing(4)
411 |         modelos = self._technician_service.listar_especialidades(tecnico.id)
412 |         if modelos:
413 |             for m in modelos:
414 |                 lbl = QLabel(f"■ {m}")
415 |                 lbl.setStyleSheet("color: #c8c8e0; font-size: 12px; background: transparent; padding: 2px 0")
416 |                 esp_inner.addWidget(lbl)
417 |         else:
418 |             lbl = QLabel("Nenhuma especialidade cadastrada.")
419 |             lbl.setStyleSheet("color: #717182; font-size: 12px; background: transparent; font-style: italic")
420 |             esp_inner.addWidget(lbl)
421 |         if self._printer_service:
422 |             btn_gerir_esp = QPushButton("■ Gerenciar Especialidades")
423 |             btn_gerir_esp.setStyleSheet(ESTILO_BOTAO_PRIMARIO)
424 |             btn_gerir_esp.clicked.connect(lambda: (dialog.accept(), self._gerir_especialidades(tecnico)))
425 |             esp_inner.addWidget(btn_gerir_esp)
426 |         esp_layout.addLayout(esp_inner)
427 |         layout.addWidget(esp_box)
428 |
429 |         layout.addStretch()
430 |         scroll.setWidget(container)
431 |         root.addWidget(scroll, 1)
432 |
433 |         # ■■ Botões ■■
434 |         botoes = QHBoxLayout()
435 |         botoes.setSpacing(10)
436 |         botoes.setContentsMargins(24, 12, 24, 12)
437 |
438 |         btn_editar = QPushButton("■ Editar")
439 |         btn_editar.setStyleSheet(ESTILO_BOTAO_AVISO)
440 |         btn_editar.clicked.connect(lambda: (dialog.accept(), self._editar(row)))
441 |
442 |         btn_excluir = QPushButton("■ Excluir")
443 |         btn_excluir.setStyleSheet(ESTILO_BOTAO_ERRO)
444 |         btn_excluir.clicked.connect(lambda: self._excluir_do_detalhes(tecnico, dialog))
445 |
446 |         btn_fechar = QPushButton("Fechar")
447 |         btn_fechar.setStyleSheet(ESTILO_BOTAO_FECHAR)
448 |         btn_fechar.clicked.connect(dialog.accept)
449 |
450 |         botoes.addWidget(btn_editar)
451 |         botoes.addWidget(btn_excluir)
452 |         botoes.addStretch()
453 |         botoes.addWidget(btn_fechar)
454 |         root.addLayout(botoes)
455 |
456 |         dialog.exec()
457 |
458 |     def _gerir_especialidades(self, tecnico: Any) -> None:
459 |         if not self._printer_service:
460 |             return
461 |         dialog = _EspecialidadesDialog(tecnico, self._printer_service, self._technician_service, self)
462 |         dialog.exec()
463 |         self.recarregar()
464 |
465 |     def _excluir_do_detalhes(self, tecnico: Any, dialog: QDialog) -> None:

```

```

466 |         from app.views.widgets.confirm_dialog import ConfirmacaoDigitalDialog
467 |         if ConfirmacaoDigitalDialog.confirmar(
468 |             "Confirmar Exclusão",
469 |             "Tem certeza que deseja excluir este técnico?",
470 |             dialog,
471 |         ):
472 |             try:
473 |                 self._technician_service.excluir(tecnico)
474 |                 ToastManager.mostrar(
475 |                     f"Técnico '{tecnico.nome_exibicao or tecnico.nome_completo}' excluído.",
476 |                     "aviso", duracao=8000,
477 |                     acao=("Desfazer", lambda o=tecnico, svc=self._technician_service, pag=self: (svc.restaurar(
478 |                         o,
479 |                     dialog.accept()
480 |                     self.recarregar()
481 |             except Exception as e:
482 |                 QMessageBox.critical(dialog, "Erro", f"Erro ao excluir técnico:\n{e}")
483 |
484 |     def _editar(self, row: int) -> None:
485 |         if row < 0 or row >= len(self._tecnicos_visiveis):
486 |             return
487 |         tecnico = self._tecnicos_visiveis[row]
488 |         dados = {
489 |             "nome_completo": tecnico.nome_completo,
490 |             "nome_exibicao": tecnico.nome_exibicao,
491 |             "telefone": tecnico.telefone or "",
492 |             "email": tecnico.email or "",
493 |         }
494 |         dialogo = _TechnicianDialog(self, dados)
495 |         if dialogo.exec() == QDialog.Accepted:
496 |             if dialogo.excluir_confirmado():
497 |                 try:
498 |                     self._technician_service.excluir(tecnico)
499 |                     ToastManager.mostrar(
500 |                         f"Técnico '{tecnico.nome_exibicao or tecnico.nome_completo}' excluído.",
501 |                         "aviso", duracao=8000,
502 |                         acao=("Desfazer", lambda o=tecnico, svc=self._technician_service, pag=self: (svc.restaurar(
503 |                             o,
504 |                         self.recarregar()
505 |                 except Exception as e:
506 |                     QMessageBox.critical(self, "Erro", f"Erro ao excluir técnico:\n{e}")
507 |             else:
508 |                 novos_dados = dialogo.dados()
509 |                 if not novos_dados["nome_completo"]:
510 |                     QMessageBox.warning(self, "Aviso", "O campo Nome Completo é obrigatório.")
511 |                     return
512 |                 try:
513 |                     self._technician_service.atualizar(tecnico, **novos_dados)
514 |                     self.recarregar()
515 |                 except Exception as e:
516 |                     QMessageBox.critical(self, "Erro", f"Erro ao atualizar técnico:\n{e}")

```

=== ARQUIVO: .\app\views\pages\technician_agenda_page.py ===

```

001 | from __future__ import annotations
002 |
003 | from collections import Counter, defaultdict
004 | from datetime import datetime as dt, timedelta
005 | from typing import Any
006 |
007 | from PySide6.QtCore import Qt, QDate

```

```

008 | from PySide6.QtWidgets import (
009 |     QComboBox,
010 |     QDateEdit,
011 |     QFrame,
012 |     QGroupBox,
013 |     QHBoxLayout,
014 |     QHeaderView,
015 |     QLabel,
016 |     QPushButton,
017 |     QTableWidgetItem,
018 |     QTableWidgetItem,
019 |     QTabWidget,
020 |     QVBoxLayout,
021 |     QWidget,
022 | )
023 |
024 | from app.views.styles.theme import (
025 |     ESTILO_BOTAO_SECUNDARIO,
026 |     ESTILO_BOTAO_SUCESSO,
027 |     ESTILO_TABELA_SIMPLES,
028 |     ESTILO_TITULO_PAGINA,
029 |     configurar_combo,
030 | )
031 | from app.views.widgets.table_widget import tornar_interativa
032 |
033 |
034 | class _StatsCard(QFrame):
035 |     lbl_valor: QLabel
036 |
037 |     def __init__(self, titulo: str, valor: str, cor: str) -> None:
038 |         super().__init__()
039 |         self.setStyleSheet(
040 |             f"QFrame {{ background-color: #14141f; border: 1px solid #2a2a3e;"
041 |             f" border-radius: 12px; padding: 16px; }}"
042 |         )
043 |         layout = QVBoxLayout(self)
044 |         layout.setContentsMargins(16, 14, 16, 14)
045 |         layout.setSpacing(4)
046 |         lbl_titulo = QLabel(titulo)
047 |         lbl_titulo.setStyleSheet("color: #717182; font-size: 11px; font-weight: 600; background: transparent;")
048 |         self.lbl_valor = QLabel(valor)
049 |         self.lbl_valor.setStyleSheet(
050 |             f"color: {cor}; font-size: 26px; font-weight: 700; background: transparent; letter-spacing: -0.5px;"
051 |         )
052 |         layout.addWidget(lbl_titulo)
053 |         layout.addWidget(self.lbl_valor)
054 |
055 |     def set_valor(self, valor: str) -> None:
056 |         self.lbl_valor.setText(valor)
057 |
058 |
059 | class TechnicianAgendaPage(QWidget):
060 |     _session: Any
061 |     _technician_service: Any
062 |     _activity_service: Any
063 |     _printer_service: Any
064 |     _tecnico_combo: QComboBox
065 |     _date_picker: QDateEdit
066 |     _card_capacidade: _StatsCard
067 |     _card_agendadas: _StatsCard
068 |     _card_andamento: _StatsCard
069 |     _tabs: QTabWidget
070 |     _tab_agenda: QWidget

```

```

071 |     _tab_produtividade: QWidget
072 |     _tabela_agenda: QTableWidgetItem
073 |     _tabela_produtividade: QTableWidgetItem
074 |     _rota_label: QLabel
075 |
076 |     def __init__(self, session: Any, technician_service: Any, activity_service: Any,
077 |                  printer_service: Any, parent: QWidget | None = None) -> None:
078 |         super().__init__(parent)
079 |         self._session = session
080 |         self._technician_service = technician_service
081 |         self._activity_service = activity_service
082 |         self._printer_service = printer_service
083 |         self._init_ui()
084 |
085 |     def _init_ui(self) -> None:
086 |         layout = QVBoxLayout(self)
087 |         layout.setContentsMargins(24, 24, 24, 24)
088 |         layout.setSpacing(16)
089 |
090 |         header = QHBoxLayout()
091 |         titulo = QLabel("\U0001f4c5 Agenda do Técnico")
092 |         titulo.setStyleSheet(ESTILO_TITULO_PAGINA)
093 |         header.addWidget(titulo)
094 |         header.addStretch()
095 |         layout.addLayout(header)
096 |
097 |         filtros = QHBoxLayout()
098 |         filtros.setSpacing(10)
099 |
100 |         lbl_tec = QLabel("Técnico:")
101 |         lbl_tec.setStyleSheet("color: #94949f; font-size: 13px; font-weight: 500; background: transparent;")
102 |         filtros.addWidget(lbl_tec)
103 |
104 |         self._tecnico_combo = QComboBox()
105 |         configurar_combo(self._tecnico_combo)
106 |         self._tecnico_combo.setMinimumWidth(260)
107 |         self._tecnico_combo.currentIndexChanged.connect(self._ao_trocar_tecnico)
108 |         filtros.addWidget(self._tecnico_combo)
109 |
110 |         lbl_data = QLabel("Data:")
111 |         lbl_data.setStyleSheet("color: #94949f; font-size: 13px; font-weight: 500; background: transparent;")
112 |         filtros.addWidget(lbl_data)
113 |
114 |         self._date_picker = QDateEdit()
115 |         self._date_picker.setCalendarPopup(True)
116 |         self._date_picker.setDate(QDate.currentDate())
117 |         self._date_picker.setDisplayFormat("dd/MM/yyyy")
118 |         self._date_picker.dateChanged.connect(self._recarregar_agenda)
119 |         filtros.addWidget(self._date_picker)
120 |
121 |         btn_atualizar = QPushButton("\u21bb Atualizar")
122 |         btn_atualizar.setStyleSheet(ESTILO_BOTAO_SUCESSO)
123 |         btn_atualizar.clicked.connect(self._recarregar_agenda)
124 |         filtros.addStretch()
125 |         filtros.addWidget(btn_atualizar)
126 |
127 |         layout.addLayout(filtros)
128 |
129 |         cards = QHBoxLayout()
130 |         cards.setSpacing(12)
131 |         self._card_capacidade = _StatsCard("Capacidade", "0 / 0", "#6366f1")
132 |         self._card_agendadas = _StatsCard("OS no Dia", "0", "#3b82f6")
133 |         self._card_andamento = _StatsCard("Em Andamento", "0", "#f59e0b")

```



```

134 |         cards.addWidget(self._card_capacidade)
135 |         cards.addWidget(self._card_agendadas)
136 |         cards.addWidget(self._card_andamento)
137 |         layout.addLayout(cards)
138 |
139 |         self._rota_label = QLabel()
140 |         self._rota_label.setStyleSheet(
141 |             "color: #94949f; font-size: 12px; background: transparent; padding: 4px 0;"
142 |         )
143 |         self._rota_label.setWordWrap(True)
144 |         layout.addWidget(self._rota_label)
145 |
146 |         self._tabs = QTabWidget()
147 |         layout.addWidget(self._tabs, 1)
148 |
149 |         self._tab_agenda = self._criar_tabela_agenda()
150 |         self._tabs.addTab(self._tab_agenda, "\U0001f4cb Agenda do Dia")
151 |
152 |         self._tab_produtividade = self._criar_tabela_produtividade()
153 |         self._tabs.addTab(self._tab_produtividade, "\U0001f4ca Produtividade")
154 |
155 |     def _criar_tabela_agenda(self) -> QWidget:
156 |         tab = QWidget()
157 |         layout = QVBoxLayout(tab)
158 |         self._tabela_agenda = QTableWidgetItem()
159 |         self._tabela_agenda.setColumnCount(8)
160 |         self._tabela_agenda.setHorizontalHeaderLabels(
161 |             ["Ordem", "Patrimônio", "Local", "Tipo", "Descrição", "Status", "Urgência", "Recibo"]
162 |         )
163 |         self._tabela_agenda.setStyleSheet(ESTILO_TABELA_SIMPLES)
164 |         self._tabela_agenda.setSelectionBehavior(QTableWidgetItem.SelectRows)
165 |         self._tabela_agenda.setEditTriggers(QTableWidgetItem.NoEditTriggers)
166 |         self._tabela_agenda.verticalHeader().setVisible(False)
167 |         self._tabela_agenda.setAlternatingRowColors(True)
168 |         self._tabela_agenda.verticalHeader().setDefaultSectionSize(40)
169 |         tornar_interativa(self._tabela_agenda)
170 |         layout.addWidget(self._tabela_agenda)
171 |         return tab
172 |
173 |     def _criar_tabela_produtividade(self) -> QWidget:
174 |         tab = QWidget()
175 |         layout = QVBoxLayout(tab)
176 |
177 |         filtros_periodo = QHBoxLayout()
178 |         filtros_periodo.setSpacing(10)
179 |
180 |         lbl_inicio = QLabel("Início:")
181 |         lbl_inicio.setStyleSheet("color: #94949f; font-size: 13px; font-weight: 500; background: transparent;")
182 |         filtros_periodo.addWidget(lbl_inicio)
183 |
184 |         hoje = QDate.currentDate()
185 |         self._periodo_inicio = QDateEdit()
186 |         self._periodo_inicio.setCalendarPopup(True)
187 |         self._periodo_inicio.setDate(hoje.addMonths(-1))
188 |         self._periodo_inicio.setDisplayFormat("dd/MM/yyyy")
189 |         filtros_periodo.addWidget(self._periodo_inicio)
190 |
191 |         lbl_fim = QLabel("Fim:")
192 |         lbl_fim.setStyleSheet("color: #94949f; font-size: 13px; font-weight: 500; background: transparent;")
193 |         filtros_periodo.addWidget(lbl_fim)
194 |
195 |         self._periodo_fim = QDateEdit()
196 |         self._periodo_fim.setCalendarPopup(True)

```

```

197 |         self._periodo_fim.setDate(hoje)
198 |         self._periodo_fim.setDisplayFormat("dd/MM/yyyy")
199 |         filtros_periodo.addWidget(self._periodo_fim)
200 |
201 |         btn_filtrar = QPushButton("Filtrar")
202 |         btn_filtrar.setStyleSheet(ESTILO_BOTAO_SUCESSO)
203 |         btn_filtrar.clicked.connect(self._recarregar_produtividade)
204 |         filtros_periodo.addWidget(btn_filtrar)
205 |
206 |         filtros_periodo.addStretch()
207 |         layout.addLayout(filtros_periodo)
208 |
209 |         cards_prod = QHBoxLayout()
210 |         cards_prod.setSpacing(12)
211 |         self._card_concluidas = _StatsCard("OS Concluídas", "0", "#10b981")
212 |         self._card_pecas = _StatsCard("Peças Usadas", "0", "#f59e0b")
213 |         self._card_tempo_medio = _StatsCard("Tempo Médio", "0min", "#6366f1")
214 |         cards_prod.addWidget(self._card_concluidas)
215 |         cards_prod.addWidget(self._card_pecas)
216 |         cards_prod.addWidget(self._card_tempo_medio)
217 |         layout.addLayout(cards_prod)
218 |
219 |         self._tabela_produtividade = QTableWidgetItem()
220 |         self._tabela_produtividade.setColumnCount(7)
221 |         self._tabela_produtividade.setHorizontalHeaderLabels(
222 |             ["Data/Hora", "Patrimônio", "Descrição", "Peças", "Início", "Fim", "Duração"]
223 |         )
224 |         self._tabela_produtividade.setStyleSheet(ESTILO_TABELA_SIMPLES)
225 |         self._tabela_produtividade.setSelectionBehavior(QTableWidgetItem.SelectRows)
226 |         self._tabela_produtividade.setEditTriggers(QTableWidgetItem.NoEditTriggers)
227 |         self._tabela_produtividade.verticalHeader().setVisible(False)
228 |         self._tabela_produtividade.setAlternatingRowColors(True)
229 |         self._tabela_produtividade.verticalHeader().setDefaultSectionSize(40)
230 |         tornar_interativa(self._tabela_produtividade)
231 |         layout.addWidget(self._tabela_produtividade, 1)
232 |         return tab
233 |
234 |     def recarregar(self) -> None:
235 |         tecnicos = self._technician_service.listar_todos()
236 |         current_text = self._tecnico_combo.currentText()
237 |
238 |         self._tecnico_combo.blockSignals(True)
239 |         self._tecnico_combo.clear()
240 |         self._tecnico_combo.addItem("-- Selecione um técnico --", None)
241 |         for t in tecnicos:
242 |             display = t.nome_exibicao or t.nome_completo
243 |             self._tecnico_combo.addItem(display, t.id)
244 |
245 |         if current_text:
246 |             idx = self._tecnico_combo.findText(current_text)
247 |             if idx >= 0:
248 |                 self._tecnico_combo.setCurrentIndex(idx)
249 |         self._tecnico_combo.blockSignals(False)
250 |
251 |         if self._tecnico_combo.currentData() is not None:
252 |             self._recarregar_agenda()
253 |
254 |     def _ao_trocar_tecnico(self) -> None:
255 |         self._recarregar_agenda()
256 |
257 |     def _recarregar_agenda(self) -> None:
258 |         tech_id = self._tecnico_combo.currentData()
259 |         if tech_id is None:

```

```

260 |         self._tabela_agenda.setRowCount(0)
261 |         self._card_capacidade.set_valor("0 / 0")
262 |         self._card_agendadas.set_valor("0")
263 |         self._card_andamento.set_valor("0")
264 |         self._rota_label.setText("")
265 |         return
266 |
267 |     qdate = self._date_picker.date()
268 |     data = dt(qdate.year(), qdate.month(), qdate.day())
269 |
270 |     tecnico = self._technician_service.buscar_por_id(tech_id)
271 |     capacidade = tecnico.capacidade_diaria if tecnico else 5
272 |
273 |     atividades = self._activity_service.listar_por_tecnico_e_data(tech_id, data)
274 |     em_andamento = self._activity_service.contar_por_tecnico_por_status(tech_id, "Em Atendimento")
275 |     total_no_dia = len(atividades)
276 |
277 |     self._card_capacidade.set_valor(f"{total_no_dia} / {capacidade}")
278 |     self._card_agendadas.set_valor(str(total_no_dia))
279 |     self._card_andamento.set_valor(str(em_andamento))
280 |
281 |     self._preencher_agenda(atividades)
282 |     self._sugerir_rota(atividades)
283 |
284 | def _sugerir_rota(self, atividades: list) -> None:
285 |     if not atividades:
286 |         self._rota_label.setText("Nenhuma OS agendada para este dia.")
287 |         return
288 |
289 |     locais = []
290 |     for a in atividades:
291 |         local = a.from_location or "Sem local"
292 |         locais.append(local)
293 |
294 |     ordem = list(dict.fromkeys(locais))
295 |     setores_contagem = Counter(locais)
296 |     resumo = " \u2192 ".join(f"{l}" for l in ordem)
297 |     detalhes = "; ".join(f"{l}: {setores_contagem[l]} OS" for l in ordem)
298 |     self._rota_label.setText(
299 |         f"\U0001f9ed Rota Sugerida: {resumo} | {detalhes}"
300 |     )
301 |
302 | def _preencher_agenda(self, atividades: list) -> None:
303 |     if not atividades:
304 |         self._tabela_agenda.setRowCount(0)
305 |         return
306 |
307 |     locais = []
308 |     for a in atividades:
309 |         locais.append(a.from_location or "Sem local")
310 |     ordem_locais = list(dict.fromkeys(locais))
311 |
312 |     local_para_numero = {l: i + 1 for i, l in enumerate(ordem_locais)}
313 |
314 |     linhas = []
315 |     for a in atividades:
316 |         local = a.from_location or "Sem local"
317 |         ordem = local_para_numero[local]
318 |         linhas.append((ordem, local, a))
319 |
320 |     linhas.sort(key=lambda x: (x[0], x[2].event_at or dt.min))
321 |
322 |     self._tabela_agenda.setRowCount(0)

```

```

323 |         self._tabela_agenda.setRowCount(len(linhas))
324 |         for i, (ordem, local, a) in enumerate(linhas):
325 |             printer = self._printer_service.buscar_por_id(a.printer_id) if hasattr(self, '_printer_service')
326 |             pat = printer.patrimonio if printer else a.printer_id
327 |
328 |             ordem_item = QTableWidgetItem(str(ordem))
329 |             ordem_item.setTextAlignment(Qt.AlignCenter)
330 |             self._tabela_agenda.setItem(i, 0, ordem_item)
331 |             self._tabela_agenda.setItem(i, 1, QTableWidgetItem(pat))
332 |             self._tabela_agenda.setItem(i, 2, QTableWidgetItem(local))
333 |             self._tabela_agenda.setItem(i, 3, QTableWidgetItem(a.kind or ""))
334 |             self._tabela_agenda.setItem(i, 4, QTableWidgetItem((a.notes or "")[:80]))
335 |             self._tabela_agenda.setItem(i, 5, QTableWidgetItem(a.status_atividade or ""))
336 |             self._tabela_agenda.setItem(i, 6, QTableWidgetItem(a.urgencia or ""))
337 |             self._tabela_agenda.setItem(i, 7, QTableWidgetItem(a.numero_recibo or ""))
338 |
339 |     def _recarregar_produtividade(self) -> None:
340 |         tech_id = self._tecnico_combo.currentData()
341 |         if tech_id is None:
342 |             self._tabela_produtividade.setRowCount(0)
343 |             self._card_concluidas.set_valor("0")
344 |             self._card_pecas.set_valor("0")
345 |             self._card_tempo_medio.set_valor("0min")
346 |             return
347 |
348 |         q_inicio = self._periodo_inicio.date()
349 |         q_fim = self._periodo_fim.date()
350 |         inicio = dt(q_inicio.year(), q_inicio.month(), q_inicio.day(), 0, 0, 0)
351 |         fim = dt(q_fim.year(), q_fim.month(), q_fim.day(), 23, 59, 59)
352 |
353 |         concluidas = self._activity_service.listar_concluidas_por_tecnico_no_periodo(tech_id, inicio, fim)
354 |
355 |         total_pecas = 0
356 |         duracoes = []
357 |         for a in concluidas:
358 |             if a.parts_used:
359 |                 total_pecas += len([p for p in a.parts_used.split(",") if p.strip()])
360 |             if a.inicio_atendimento and a.fim_atendimento:
361 |                 diff = (a.fim_atendimento - a.inicio_atendimento).total_seconds()
362 |                 if diff > 0:
363 |                     duracoes.append(diff)
364 |
365 |         tempo_medio = sum(duracoes) / len(duracoes) if duracoes else 0
366 |         horas = int(tempo_medio // 3600)
367 |         minutos = int((tempo_medio % 3600) // 60)
368 |         if horas > 0:
369 |             tempo_str = f"{horas}h {minutos}min"
370 |         else:
371 |             tempo_str = f"{minutos}min"
372 |
373 |         self._card_concluidas.set_valor(str(len(concluidas)))
374 |         self._card_pecas.set_valor(str(total_pecas))
375 |         self._card_tempo_medio.set_valor(tempo_str)
376 |
377 |         self._tabela_produtividade.setRowCount(0)
378 |         self._tabela_produtividade.setRowCount(len(concluidas))
379 |         for i, a in enumerate(concluidas):
380 |             printer = self._printer_service.buscar_por_id(a.printer_id) if hasattr(self, '_printer_service')
381 |             pat = printer.patrimonio if printer else a.printer_id
382 |
383 |         def fmt(dt_val):
384 |             return dt_val.strftime("%d/%m/%Y %H:%M") if dt_val else "-"
385 |

```

```

386 |         duracao = "-"
387 |         if a.inicio_atendimento and a.fim_atendimento:
388 |             diff = (a.fim_atendimento - a.inicio_atendimento).total_seconds()
389 |             h = int(diff // 3600)
390 |             m = int((diff % 3600) // 60)
391 |             duracao = f"{h}h {m}min" if h > 0 else f"{m}min"
392 |
393 |         self._tabela_produtividade.setItem(i, 0, QTableWidgetItem(fmt(a.fim_atendimento or a.event_at))
394 |         self._tabela_produtividade.setItem(i, 1, QTableWidgetItem(pat))
395 |         self._tabela_produtividade.setItem(i, 2, QTableWidgetItem((a.notes or "")[:80]))
396 |         self._tabela_produtividade.setItem(i, 3, QTableWidgetItem(a.parts_used or ""))
397 |         self._tabela_produtividade.setItem(i, 4, QTableWidgetItem(fmt(a.inicio_atendimento)))
398 |         self._tabela_produtividade.setItem(i, 5, QTableWidgetItem(fmt(a.fim_atendimento)))
399 |         self._tabela_produtividade.setItem(i, 6, QTableWidgetItem(duracao))

```

=== ARQUIVO: .\app\views\pages\technician_history_page.py ===

```

001 | from __future__ import annotations
002 |
003 | from app.models.base import utcnow
004 | from datetime import datetime as dt
005 | from typing import Any
006 |
007 | from PySide6.QtCore import Qt, Signal
008 | from PySide6.QtWidgets import (
009 |     QComboBox,
010 |     QFrame,
011 |     QGroupBox,
012 |     QHBoxLayout,
013 |     QHeaderView,
014 |     QLabel,
015 |     QLineEdit,
016 |     QPushButton,
017 |     QTableWidgetItem,
018 |     QTableWidgetItem,
019 |     QTabWidget,
020 |     QVBoxLayout,
021 |     QWidget,
022 | )
023 |
024 | from app.models import Activity, Printer, User
025 | from app.views.styles.theme import (
026 |     ATIVIDADE_CORES,
027 |     ESTILO_BOTA0_SECUNDARIO,
028 |     ESTILO_BOTA0_SUCESSO,
029 |     ESTILO_INPUT,
030 |     ESTILO_LABEL_VALOR,
031 |     ESTILO_TABELA_SIMPLES,
032 |     ESTILO_TITULO_PAGINA,
033 |     STATUS_ATIVIDADE_OPCOES,
034 |     TIPO_ATIVIDADE_CORES,
035 |     configurar_combo,
036 |     configurar_combo_colorido,
037 | )
038 | from app.utils.helpers import formatar_data_hora
039 | from app.views.widgets.table_widget import tornar_interativa
040 |
041 |
042 | class _StatsCard(QFrame):
043 |     lbl_valor: QLabel
044 |

```

```

045 | def __init__(self, titulo: str, valor: int, cor: str) -> None:
046 |     super().__init__()
047 |     self.setStyleSheet(
048 |         f"QFrame {{ background-color: #14141f; border: 1px solid #2a2a3e;"
049 |         f" border-radius: 12px; padding: 16px; }}"
050 |     )
051 |     layout = QVBoxLayout(self)
052 |     layout.setContentsMargins(16, 14, 16, 14)
053 |     layout.setSpacing(4)
054 |     lbl_titulo = QLabel(titulo)
055 |     lbl_titulo.setStyleSheet("color: #717182; font-size: 11px; font-weight: 600; background: transparent;")
056 |     self.lbl_valor = QLabel(str(valor))
057 |     self.lbl_valor.setStyleSheet(
058 |         f"color: {cor}; font-size: 26px; font-weight: 700; background: transparent; letter-spacing: -0.5px;"
059 |     )
060 |     layout.addWidget(lbl_titulo)
061 |     layout.addWidget(self.lbl_valor)
062 |
063 | def set_valor(self, valor: int) -> None:
064 |     self.lbl_valor.setText(str(valor))
065 |
066 |
067 | class TechnicianHistoryPage(QWidget):
068 |     abrir_os = Signal(int)
069 |     _session: Any
070 |     _technician_service: Any
071 |     _activity_service: Any
072 |     _user_service: Any
073 |     _login_history_service: Any
074 |     _printer_service: Any
075 |     _current_tech_id: Any | None
076 |     _tecnico_combo: QComboBox
077 |     _status_combo: QComboBox
078 |     _tipo_combo: QComboBox
079 |     _card_total: _StatsCard
080 |     _card_andamento: _StatsCard
081 |     _card_concluidas: _StatsCard
082 |     _card_mov: _StatsCard
083 |     _tabs: QTabWidget
084 |     _tab_atividades: QWidget
085 |     _tab_andamento: QWidget
086 |     _tab_mov: QWidget
087 |     _tab_login: QWidget
088 |     _tabela_atividades: QTableWidget
089 |     _tabela_andamento: QTableWidget
090 |     _tabela_mov: QTableWidget
091 |     _tabela_login: QTableWidget
092 |     _assoc_label: QLabel
093 |     _assoc_combo: QComboBox
094 |     _assoc_btn: QPushButton
095 |     _tab_os_abertas: QWidget
096 |     _tab_pecas: QWidget
097 |     _tabela_os_abertas: QTableWidget
098 |     _tabela_pecas: QTableWidget
099 |     _peca_search_input: QLineEdit
100 |     _tabela_quem_usou: QTableWidget
101 |
102 | def __init__(self, session: Any, technician_service: Any, activity_service: Any, user_service: Any, login_history_service: Any, printer_service: Any, parent: QWidget | None = None) -> None:
103 |     super().__init__(parent)
104 |     self._session = session
105 |     self._technician_service = technician_service
106 |     self._activity_service = activity_service
107 |

```

```

108 |         self._user_service = user_service
109 |         self._login_history_service = login_history_service
110 |         self._printer_service = printer_service
111 |         self._current_tech_id = None
112 |         self._init_ui()
113 |
114 |     def _init_ui(self) -> None:
115 |         layout = QVBoxLayout(self)
116 |         layout.setContentsMargins(24, 24, 24, 24)
117 |         layout.setSpacing(16)
118 |
119 |         header = QHBoxLayout()
120 |         titulo = QLabel("\U0001f4dc Histórico de Técnicos")
121 |         titulo.setStyleSheet(ESTILO_TITULO_PAGINA)
122 |         header.addWidget(titulo)
123 |         header.addStretch()
124 |         layout.addLayout(header)
125 |
126 |         filtros = QHBoxLayout()
127 |         filtros.setSpacing(10)
128 |
129 |         lbl_tec = QLabel("Técnico:")
130 |         lbl_tec.setStyleSheet("color: #94949f; font-size: 13px; font-weight: 500; background: transparent;")
131 |         filtros.addWidget(lbl_tec)
132 |
133 |         self._tecnico_combo = QComboBox()
134 |         configurar_combo(self._tecnico_combo)
135 |         self._tecnico_combo.setMinimumWidth(260)
136 |         self._tecnico_combo.currentIndexChanged.connect(self._ao_trocar_tecnico)
137 |         filtros.addWidget(self._tecnico_combo)
138 |
139 |         lbl_status = QLabel("Status:")
140 |         lbl_status.setStyleSheet("color: #94949f; font-size: 13px; font-weight: 500; background: transparent;")
141 |         filtros.addWidget(lbl_status)
142 |
143 |         self._status_combo = QComboBox()
144 |         configurar_combo(self._status_combo)
145 |         self._status_combo.addItem(["Todos"] + STATUS_ATIVIDADE_OPCOES)
146 |         configurar_combo_colorido(self._status_combo, ATIVIDADE_CORES)
147 |         self._status_combo.currentIndexChanged.connect(self._aplicar_filtros)
148 |         filtros.addWidget(self._status_combo)
149 |
150 |         lbl_tipo = QLabel("Tipo:")
151 |         lbl_tipo.setStyleSheet("color: #94949f; font-size: 13px; font-weight: 500; background: transparent;")
152 |         filtros.addWidget(lbl_tipo)
153 |
154 |         self._tipo_combo = QComboBox()
155 |         configurar_combo(self._tipo_combo)
156 |         self._tipo_combo.addItem(["Todos", "MANUTENCAO", "MOVIMENTACAO"])
157 |         configurar_combo_colorido(self._tipo_combo, TIPO_ATIVIDADE_CORES)
158 |         self._tipo_combo.currentIndexChanged.connect(self._aplicar_filtros)
159 |         filtros.addWidget(self._tipo_combo)
160 |
161 |         btn_atualizar = QPushButton("\u21bb Atualizar")
162 |         btn_atualizar.setStyleSheet(ESTILO_BOTAO_SUCESSO)
163 |         btn_atualizar.clicked.connect(self.recarregar)
164 |         filtros.addStretch()
165 |         filtros.addWidget(btn_atualizar)
166 |
167 |         layout.addLayout(filtros)
168 |
169 |         cards = QHBoxLayout()
170 |         cards.setSpacing(12)

```

```

171 |         self._card_total = _StatsCard("Total de OS", 0, "#6366f1")
172 |         self._card_andamento = _StatsCard("Em Atendimento", 0, "#6366f1")
173 |         self._card_concluidas = _StatsCard("Concluídas", 0, "#10b981")
174 |         self._card_mov = _StatsCard("Movimentações", 0, "#3b82f6")
175 |         cards.addWidget(self._card_total)
176 |         cards.addWidget(self._card_andamento)
177 |         cards.addWidget(self._card_concluidas)
178 |         cards.addWidget(self._card_mov)
179 |         layout.addLayout(cards)
180 |
181 |         self._tabs = QTabWidget()
182 |         layout.addWidget(self._tabs, 1)
183 |
184 |         self._tab_atividades = self._criar_tabela_atividades()
185 |         self._tabs.addTab(self._tab_atividades, "\u2699\u2014 Atividades")
186 |
187 |         self._tab_andamento = self._criar_tabela_andamento()
188 |         self._tabs.addTab(self._tab_andamento, "\u23f3 Em Andamento")
189 |
190 |         self._tab_mov = self._criar_tabela_movimentacoes()
191 |         self._tabs.addTab(self._tab_mov, "\U0001f69a Movimentações")
192 |
193 |         self._tab_login = self._criar_tabela_login()
194 |         self._tabs.addTab(self._tab_login, "\U0001f511 Sessões")
195 |
196 |         self._tab_os_abertas = self._criar_tabela_os_abertas()
197 |         self._tabs.addTab(self._tab_os_abertas, "\U0001f4cc OS Abertas")
198 |
199 |         self._tab_pecas = self._criar_tabela_pecas()
200 |         self._tabs.addTab(self._tab_pecas, "\U0001f527 Peças")
201 |
202 |         self._tabela_atividades.cellDoubleClicked.connect(lambda r, c: self._ao_duplo_clicar_os(r, self._ta
203 |         self._tabela_andamento.cellDoubleClicked.connect(lambda r, c: self._ao_duplo_clicar_os(r, self._ta
204 |         self._tabela_os_abertas.cellDoubleClicked.connect(lambda r, c: self._ao_duplo_clicar_os(r, self._ta
205 |
206 |     def _criar_tabela_atividades(self) -> QWidget:
207 |         tab = QWidget()
208 |         layout = QVBoxLayout(tab)
209 |         self._tabela_atividades = QTableWidgetItem()
210 |         self._tabela_atividades.setColumnCount(8)
211 |         self._tabela_atividades.setHorizontalHeaderLabels(
212 |             ["Data/Hora", "Patrimônio", "Tipo", "Descrição", "Peças", "Origem/Destino", "Status", "Recibo"]
213 |         )
214 |         self._tabela_atividades.setStyleSheet(ESTILO_TABELA_SIMPLES)
215 |         self._tabela_atividades.setSelectionBehavior(QTableWidgetItem.SelectRows)
216 |         self._tabela_atividades.setEditTriggers(QTableWidgetItem.NoEditTriggers)
217 |         self._tabela_atividades.verticalHeader().setVisible(False)
218 |         self._tabela_atividades.setAlternatingRowColors(True)
219 |         self._tabela_atividades.verticalHeader().setDefaultSectionSize(40)
220 |         tornar_interativa(self._tabela_atividades)
221 |         layout.addWidget(self._tabela_atividades)
222 |         return tab
223 |
224 |     def _criar_tabela_andamento(self) -> QWidget:
225 |         tab = QWidget()
226 |         layout = QVBoxLayout(tab)
227 |         lbl = QLabel("Máquinas sendo consertadas agora pelo técnico selecionado:")
228 |         lbl.setStyleSheet("color: #94949f; font-size: 12px; background: transparent;")
229 |         layout.addWidget(lbl)
230 |         self._tabela_andamento = QTableWidgetItem()
231 |         self._tabela_andamento.setColumnCount(7)
232 |         self._tabela_andamento.setHorizontalHeaderLabels(
233 |             ["Data/Hora", "Patrimônio", "Modelo", "Descrição", "Peças", "Local", "Recibo"]

```



```

234 |         )
235 |         self._tabela_andamento.setStyleSheet(ESTILO_TABELA_SIMPLES)
236 |         self._tabela_andamento.setSelectionBehavior(QTableWidget.SelectRows)
237 |         self._tabela_andamento.setEditTriggers(QTableWidget.NoEditTriggers)
238 |         self._tabela_andamento.verticalHeader().setVisible(False)
239 |         self._tabela_andamento.setAlternatingRowColors(True)
240 |         self._tabela_andamento.verticalHeader().setDefaultSectionSize(40)
241 |         tornar_interativa(self._tabela_andamento)
242 |         layout.addWidget(self._tabela_andamento)
243 |         return tab
244 |
245 |     def _criar_tabela_movimentacoes(self) -> QWidget:
246 |         tab = QWidget()
247 |         layout = QVBoxLayout(tab)
248 |         self._tabela_mov = QTableWidget()
249 |         self._tabela_mov.setColumnCount(7)
250 |         self._tabela_mov.setHorizontalHeaderLabels(
251 |             ["Data/Hora", "Patrimônio", "Peças", "Origem", "Destino", "Status", "Recibo"]
252 |         )
253 |         self._tabela_mov.setStyleSheet(ESTILO_TABELA_SIMPLES)
254 |         self._tabela_mov.setSelectionBehavior(QTableWidget.SelectRows)
255 |         self._tabela_mov.setEditTriggers(QTableWidget.NoEditTriggers)
256 |         self._tabela_mov.verticalHeader().setVisible(False)
257 |         self._tabela_mov.setAlternatingRowColors(True)
258 |         self._tabela_mov.verticalHeader().setDefaultSectionSize(40)
259 |         tornar_interativa(self._tabela_mov)
260 |         layout.addWidget(self._tabela_mov)
261 |         return tab
262 |
263 |     def _criar_tabela_login(self) -> QWidget:
264 |         tab = QWidget()
265 |         layout = QVBoxLayout(tab)
266 |
267 |         assoc_frame = QFrame()
268 |         assoc_frame.setStyleSheet(
269 |             "QFrame { background-color: #14141f; border: 1px solid #2a2a3e;"
270 |             " border-radius: 8px; padding: 8px; }"
271 |         )
272 |         assoc_layout = QHBoxLayout(assoc_frame)
273 |         assoc_layout.setContentsMargins(12, 8, 12, 8)
274 |         assoc_layout.setSpacing(8)
275 |
276 |         assoc_label = QLabel("Usuário associado:")
277 |         assoc_label.setStyleSheet("color: #94949f; font-size: 12px; font-weight: 600; background: transparent;")
278 |         assoc_layout.addWidget(assoc_label)
279 |
280 |         self._assoc_label = QLabel("-")
281 |         self._assoc_label.setStyleSheet("color: #10b981; font-size: 12px; font-weight: 600; background: transparent;")
282 |         assoc_layout.addWidget(self._assoc_label)
283 |
284 |         assoc_layout.addSpacing(12)
285 |
286 |         self._assoc_combo = QComboBox()
287 |         configurar_combo(self._assoc_combo)
288 |         self._assoc_combo.setMinimumWidth(200)
289 |         assoc_layout.addWidget(self._assoc_combo)
290 |
291 |         self._assoc_btn = QPushButton("Salvar Associação")
292 |         self._assoc_btn.setStyleSheet(ESTILO_BOTAO_SUCESSO)
293 |         self._assoc_btn.clicked.connect(self._salvar_associacao)
294 |         assoc_layout.addWidget(self._assoc_btn)
295 |
296 |         assoc_layout.addStretch()

```

```

297 |         layout.addWidget(assoc_frame)
298 |
299 |         self._tabela_login = QTableWidgetItem()
300 |         self._tabela_login.setColumnCount(5)
301 |         self._tabela_login.setHorizontalHeaderLabels(
302 |             ["Usuário", "Perfil", "Login", "Logout", "Duração"]
303 |         )
304 |         self._tabela_login.setStyleSheet(ESTILO_TABELA_SIMPLES)
305 |         self._tabela_login.setSelectionBehavior(QTableWidgetItem.SelectRows)
306 |         self._tabela_login.setEditTriggers(QTableWidgetItem.NoEditTriggers)
307 |         self._tabela_login.verticalHeader().setVisible(False)
308 |         self._tabela_login.setAlternatingRowColors(True)
309 |         self._tabela_login.verticalHeader().setDefaultSectionSize(40)
310 |         tornar_interativa(self._tabela_login)
311 |         layout.addWidget(self._tabela_login)
312 |         return tab
313 |
314 |     def _criar_tabela_os_abertas(self) -> QWidget:
315 |         tab = QWidget()
316 |         layout = QVBoxLayout(tab)
317 |         lbl = QLabel("Ordens de serviço em aberto (não concluídas) para este técnico:")
318 |         lbl.setStyleSheet("color: #94949f; font-size: 12px; background: transparent;")
319 |         lbl.setWordWrap(True)
320 |         layout.addWidget(lbl)
321 |         self._tabela_os_abertas = QTableWidgetItem()
322 |         self._tabela_os_abertas.setColumnCount(7)
323 |         self._tabela_os_abertas.setHorizontalHeaderLabels(
324 |             ["Data/Hora", "Patrimônio", "Local", "Status", "Descrição", "Urgência", "Recibo"]
325 |         )
326 |         self._tabela_os_abertas.setStyleSheet(ESTILO_TABELA_SIMPLES)
327 |         self._tabela_os_abertas.setSelectionBehavior(QTableWidgetItem.SelectRows)
328 |         self._tabela_os_abertas.setEditTriggers(QTableWidgetItem.NoEditTriggers)
329 |         self._tabela_os_abertas.verticalHeader().setVisible(False)
330 |         self._tabela_os_abertas.setAlternatingRowColors(True)
331 |         self._tabela_os_abertas.verticalHeader().setDefaultSectionSize(40)
332 |         tornar_interativa(self._tabela_os_abertas)
333 |         layout.addWidget(self._tabela_os_abertas)
334 |         return tab
335 |
336 |     def _criar_tabela_pecas(self) -> QWidget:
337 |         tab = QWidget()
338 |         layout = QVBoxLayout(tab)
339 |
340 |         lbl = QLabel("Peças mais usadas por este técnico:")
341 |         lbl.setStyleSheet("color: #94949f; font-size: 12px; background: transparent;")
342 |         layout.addWidget(lbl)
343 |
344 |         self._tabela_pecas = QTableWidgetItem()
345 |         self._tabela_pecas.setColumnCount(2)
346 |         self._tabela_pecas.setHorizontalHeaderLabels(["Peça", "Vezes Usada"])
347 |         self._tabela_pecas.setStyleSheet(ESTILO_TABELA_SIMPLES)
348 |         self._tabela_pecas.setSelectionBehavior(QTableWidgetItem.SelectRows)
349 |         self._tabela_pecas.setEditTriggers(QTableWidgetItem.NoEditTriggers)
350 |         self._tabela_pecas.verticalHeader().setVisible(False)
351 |         self._tabela_pecas.setAlternatingRowColors(True)
352 |         self._tabela_pecas.verticalHeader().setDefaultSectionSize(36)
353 |         tornar_interativa(self._tabela_pecas)
354 |         self._tabela_pecas.horizontalHeader().setSectionResizeMode(1, QHeaderView.ResizeToContents)
355 |         layout.addWidget(self._tabela_pecas)
356 |
357 |         layout.addSpacing(12)
358 |
359 |         search_frame = QFrame()

```

```

360 |         search_frame.setStyleSheet("QFrame { background: transparent; border: none; }")
361 |         search_layout = QHBoxLayout(search_frame)
362 |         search_layout.setContentsMargins(0, 0, 0, 0)
363 |
364 |         search_lbl = QLabel("Quem usou esta peça:")
365 |         search_lbl.setStyleSheet("color: #94949f; font-size: 12px; font-weight: 600; background: transparent")
366 |         search_layout.addWidget(search_lbl)
367 |
368 |         self._peca_search_input = QLineEdit()
369 |         self._peca_search_input.setStyleSheet(ESTILO_INPUT)
370 |         self._peca_search_input.setPlaceholderText("Digite o nome da peça...")
371 |         search_layout.addWidget(self._peca_search_input)
372 |
373 |         btn_search = QPushButton("Buscar")
374 |         btn_search.setStyleSheet(ESTILO_BOTAO_SUCESSO)
375 |         btn_search.clicked.connect(self._buscar_quem_usou_peca)
376 |         search_layout.addWidget(btn_search)
377 |         search_layout.addStretch()
378 |         layout.addWidget(search_frame)
379 |
380 |         self._tabela_quem_usou = QTableWidgetItem()
381 |         self._tabela_quem_usou.setColumnCount(5)
382 |         self._tabela_quem_usou.setHorizontalHeaderLabels(
383 |             ["Técnico", "Data/Hora", "Patrimônio", "Peças", "Descrição"]
384 |         )
385 |         self._tabela_quem_usou.setStyleSheet(ESTILO_TABELA_SIMPLES)
386 |         self._tabela_quem_usou.setSelectionBehavior(QTableWidgetItem.SelectRows)
387 |         self._tabela_quem_usou.setEditTriggers(QTableWidgetItem.NoEditTriggers)
388 |         self._tabela_quem_usou.verticalHeader().setVisible(False)
389 |         self._tabela_quem_usou.setAlternatingRowColors(True)
390 |         self._tabela_quem_usou.verticalHeader().setDefaultSectionSize(40)
391 |         tornar_interativa(self._tabela_quem_usou)
392 |         layout.addWidget(self._tabela_quem_usou)
393 |         return tab
394 |
395 |     def recarregar(self) -> None:
396 |         tecnicos = self._technician_service.listar_todos()
397 |         current_text = self._tecnico_combo.currentText()
398 |
399 |         self._tecnico_combo.blockSignals(True)
400 |         self._tecnico_combo.clear()
401 |         self._tecnico_combo.addItem("-- Selecione um técnico --", None)
402 |         for t in tecnicos:
403 |             display = t.nome_exibicao or t.nome_completo
404 |             self._tecnico_combo.addItem(display, t.id)
405 |
406 |         if current_text:
407 |             idx = self._tecnico_combo.findText(current_text)
408 |             if idx >= 0:
409 |                 self._tecnico_combo.setCurrentIndex(idx)
410 |         self._tecnico_combo.blockSignals(False)
411 |
412 |         if self._tecnico_combo.currentData() is not None:
413 |             self._ao_trocar_tecnico()
414 |
415 |     def _ao_trocar_tecnico(self) -> None:
416 |         self._current_tech_id = self._tecnico_combo.currentData()
417 |         if self._current_tech_id is None:
418 |             self._limpar_tabelas()
419 |             return
420 |         self._atualizar_stats()
421 |         self._aplicar_filtros()
422 |

```

```

423 | def _aplicar_filtros(self) -> None:
424 |     tech_id = self._current_tech_id
425 |     if tech_id is None:
426 |         return
427 |
428 |     status = self._status_combo.currentText()
429 |     tipo = self._tipo_combo.currentText()
430 |
431 |     if status == "Todos":
432 |         atividades = self._activity_service.listar_por_tecnico(tech_id)
433 |     else:
434 |         atividades = self._activity_service.listar_por_tecnico_e_status(tech_id, status)
435 |
436 |     if tipo != "Todos":
437 |         atividades = [a for a in atividades if a.kind == tipo]
438 |
439 |     self._preencher_atividades(atividades)
440 |     self._preencher_andamento(tech_id)
441 |     self._preencher_movimentacoes(tech_id)
442 |     self._preencher_login(tech_id)
443 |     self._preencher_os_abertas(tech_id)
444 |     self._preencher_pecas(tech_id)
445 |
446 | def _atualizar_stats(self) -> None:
447 |     tech_id = self._current_tech_id
448 |     if tech_id is None:
449 |         for card in [self._card_total, self._card_andamento, self._card_concluidas, self._card_mov]:
450 |             card.set_valor(0)
451 |         return
452 |
453 |     total = self._activity_service.contar_por_tecnico(tech_id)
454 |     andamento = self._activity_service.contar_por_tecnico_por_status(tech_id, "Em Atendimento")
455 |     concluidas = self._activity_service.contar_por_tecnico_por_status(tech_id, "Concluido")
456 |     mov = len(self._activity_service.listar_por_tecnico_e_tipo(tech_id, "MOVIMENTACAO"))
457 |
458 |     self._card_total.set_valor(total)
459 |     self._card_andamento.set_valor(andamento)
460 |     self._card_concluidas.set_valor(concluidas)
461 |     self._card_mov.set_valor(mov)
462 |
463 | def _preencher_atividades(self, atividades: list[Any]) -> None:
464 |     self._tabela_atividades.setRowCount(0)
465 |     self._tabela_atividades.setRowCount(len(atividades))
466 |     for i, a in enumerate(atividades):
467 |         printer = self._printer_service.buscar_por_id(a.printer_id)
468 |         pat = printer.patrimonio if printer else a.printer_id
469 |         item0 = QTableWidgetItem(formatar_data_hora(a.event_at))
470 |         item0.setData(Qt.UserRole, a.id)
471 |         self._tabela_atividades.setItem(i, 0, item0)
472 |         self._tabela_atividades.setItem(i, 1, QTableWidgetItem(pat))
473 |         self._tabela_atividades.setItem(i, 2, QTableWidgetItem(a.kind))
474 |         self._tabela_atividades.setItem(i, 3, QTableWidgetItem((a.notes or "")[:60]))
475 |         self._tabela_atividades.setItem(i, 4, QTableWidgetItem(a.parts_used or ""))
476 |         od = f"{a.from_location} \u2192 {a.to_location}" if a.from_location or a.to_location else "-"
477 |         self._tabela_atividades.setItem(i, 5, QTableWidgetItem(od))
478 |         self._tabela_atividades.setItem(i, 6, QTableWidgetItem(a.status_atividade or ""))
479 |         self._tabela_atividades.setItem(i, 7, QTableWidgetItem(a.numero_recibo or ""))
480 |
481 | def _preencher_andamento(self, tech_id: int) -> None:
482 |     atividades = self._activity_service.listar_por_tecnico_e_status(tech_id, "Em Atendimento")
483 |     self._tabela_andamento.setRowCount(0)
484 |     self._tabela_andamento.setRowCount(len(atividades))
485 |     for i, a in enumerate(atividades):

```

```

486 |         printer = self._printer_service.buscar_por_id(a.printer_id)
487 |         pat = printer.patrimonio if printer else a.printer_id
488 |         modelo = printer.modelo if printer else ""
489 |         item0 = QTableWidgetItem(formatar_data_hora(a.event_at))
490 |         item0.setData(Qt.UserRole, a.id)
491 |         self._tabela_andamento.setItem(i, 0, item0)
492 |         self._tabela_andamento.setItem(i, 1, QTableWidgetItem(pat))
493 |         self._tabela_andamento.setItem(i, 2, QTableWidgetItem(modelo))
494 |         self._tabela_andamento.setItem(i, 3, QTableWidgetItem((a.notes or "")[:60]))
495 |         self._tabela_andamento.setItem(i, 4, QTableWidgetItem(a.parts_used or ""))
496 |         local = a.from_location or printer.local_atual or "-"
497 |         self._tabela_andamento.setItem(i, 5, QTableWidgetItem(local))
498 |         self._tabela_andamento.setItem(i, 6, QTableWidgetItem(a.numero_recibo or ""))
499 |
500 |     def _preencher_movimentacoes(self, tech_id: int) -> None:
501 |         atividades = self._activity_service.listar_por_tecnico_e_tipo(tech_id, "MOVIMENTACAO")
502 |         self._tabela_mov.setRowCount(0)
503 |         self._tabela_mov.setRowCount(len(atividades))
504 |         for i, a in enumerate(atividades):
505 |             printer = self._printer_service.buscar_por_id(a.printer_id)
506 |             pat = printer.patrimonio if printer else a.printer_id
507 |             self._tabela_mov.setItem(i, 0, QTableWidgetItem(formatar_data_hora(a.event_at)))
508 |             self._tabela_mov.setItem(i, 1, QTableWidgetItem(pat))
509 |             self._tabela_mov.setItem(i, 2, QTableWidgetItem(a.parts_used or ""))
510 |             self._tabela_mov.setItem(i, 3, QTableWidgetItem(a.from_location or "-"))
511 |             self._tabela_mov.setItem(i, 4, QTableWidgetItem(a.to_location or "-"))
512 |             self._tabela_mov.setItem(i, 5, QTableWidgetItem(a.status_atividade or ""))
513 |             self._tabela_mov.setItem(i, 6, QTableWidgetItem(a.numero_recibo or ""))
514 |
515 |     def _preencher_login(self, tech_id: int) -> None:
516 |         tecnico = self._technician_service.buscar_por_id(tech_id)
517 |         if not tecnico:
518 |             self._tabela_login.setRowCount(0)
519 |             self._assoc_label.setText("-")
520 |             self._assoc_combo.clear()
521 |             return
522 |
523 |         usuarios = self._user_service.listar_todos()
524 |         self._assoc_combo.blockSignals(True)
525 |         self._assoc_combo.clear()
526 |         self._assoc_combo.addItem("-- Nenhum --", None)
527 |         for u in usuarios:
528 |             self._assoc_combo.addItem(f"{u.nome} ({u.perfil})", u.id)
529 |         self._assoc_combo.blockSignals(False)
530 |
531 |         user_match = None
532 |         if tecnico.user_id is not None:
533 |             for u in usuarios:
534 |                 if u.id == tecnico.user_id:
535 |                     user_match = u
536 |                     break
537 |             if user_match is None:
538 |                 tecnico.user_id = None
539 |
540 |         if user_match is None:
541 |             for u in usuarios:
542 |                 if tecnico.nome_completo.lower() in u.nome.lower() or \
543 |                    (tecnico.nome_exibicao and tecnico.nome_exibicao.lower() in u.nome.lower()):
544 |                     user_match = u
545 |                     self._technician_service.associar_usuario(tech_id, u.id)
546 |                     break
547 |
548 |         if user_match is None:

```

```

549 |         self._tabela_login.setRowCount(0)
550 |         self._tabela_login.setRowCount(1)
551 |         self._tabela_login.setSpan(0, 0, 1, 5)
552 |         lbl_empty = QLabel(
553 |             "Nenhuma sessão – associe um usuário do sistema acima."
554 |         )
555 |         lbl_empty.setStyleSheet("color: #717182; font-size: 12px; background: transparent;")
556 |         lbl_empty.setAlignment(Qt.AlignCenter)
557 |         self._tabela_login.setCellWidget(0, 0, lbl_empty)
558 |         self._assoc_label.setStyleSheet("color: #ef4444; font-size: 12px; font-weight: 600; background: tra
559 |         self._assoc_label.setText("Nenhum")
560 |         self._assoc_combo.setCurrentIndex(0)
561 |         return
562 |
563 |     for i in range(self._assoc_combo.count()):
564 |         if self._assoc_combo.itemData(i) == user_match.id:
565 |             self._assoc_combo.setCurrentIndex(i)
566 |             break
567 |
568 |     self._assoc_label.setStyleSheet("color: #10b981; font-size: 12px; font-weight: 600; background: tra
569 |     self._assoc_label.setText(user_match.nome)
570 |
571 |     sessoes = self._login_history_service.listar_por_usuario(user_match.id)
572 |     self._tabela_login.setRowCount(0)
573 |     self._tabela_login.setRowCount(len(sessoes))
574 |     for i, s in enumerate(sessoes):
575 |         self._tabela_login.setItem(i, 0, QTableWidgetItem(user_match.nome))
576 |         self._tabela_login.setItem(i, 1, QTableWidgetItem(user_match.perfil.capitalize()))
577 |         self._tabela_login.setItem(i, 2, QTableWidgetItem(formatar_data_hora(s.login_at)))
578 |         logout = formatar_data_hora(s.logout_at) if s.logout_at else "Em sessão"
579 |         self._tabela_login.setItem(i, 3, QTableWidgetItem(logout))
580 |         if s.logout_at:
581 |             diff = s.logout_at - s.login_at
582 |             horas = int(diff.total_seconds() // 3600)
583 |             minutos = int((diff.total_seconds() % 3600) // 60)
584 |             duracao = f"{horas}h {minutos}min"
585 |         else:
586 |             diff = utcnow() - s.login_at
587 |             horas = int(diff.total_seconds() // 3600)
588 |             minutos = int((diff.total_seconds() % 3600) // 60)
589 |             duracao = f"{horas}h {minutos}min (em andamento)"
590 |         self._tabela_login.setItem(i, 4, QTableWidgetItem(duracao))
591 |
592 |     def _preencher_os_abertas(self, tech_id: int) -> None:
593 |         atividades = self._activity_service.listar_os_abertas_por_tecnico(tech_id)
594 |         self._tabela_os_abertas.setRowCount(0)
595 |         self._tabela_os_abertas.setRowCount(len(atividades))
596 |         for i, a in enumerate(atividades):
597 |             printer = self._printer_service.buscar_por_id(a.printer_id)
598 |             pat = printer.patrimonio if printer else a.printer_id
599 |             item0 = QTableWidgetItem(formatar_data_hora(a.event_at))
600 |             item0.setData(Qt.UserRole, a.id)
601 |             self._tabela_os_abertas.setItem(i, 0, item0)
602 |             self._tabela_os_abertas.setItem(i, 1, QTableWidgetItem(pat))
603 |             self._tabela_os_abertas.setItem(i, 2, QTableWidgetItem(a.from_location or ""))
604 |             self._tabela_os_abertas.setItem(i, 3, QTableWidgetItem(a.status_atividade or ""))
605 |             self._tabela_os_abertas.setItem(i, 4, QTableWidgetItem((a.notes or "")[:60]))
606 |             self._tabela_os_abertas.setItem(i, 5, QTableWidgetItem(a.urgencia or ""))
607 |             self._tabela_os_abertas.setItem(i, 6, QTableWidgetItem(a.numero_recibo or ""))
608 |
609 |     def _preencher_pecas(self, tech_id: int) -> None:
610 |         pecas = self._activity_service.listar_pecas_por_tecnico(tech_id)
611 |         self._tabela_pecas.setRowCount(0)

```

```

612 |         self._tabela_pecas.setRowCount(len(pecas))
613 |         for i, (nome, qtd) in enumerate(pecas):
614 |             self._tabela_pecas.setItem(i, 0, QTableWidgetItem(nome))
615 |             item = QTableWidgetItem(str(qtd))
616 |             item.setTextAlignment(Qt.AlignCenter)
617 |             self._tabela_pecas.setItem(i, 1, item)
618 |
619 |     def _buscar_quem_usou_peca(self) -> None:
620 |         nome = self._peca_search_input.text().strip()
621 |         if not nome:
622 |             return
623 |         resultados = self._activity_service.listar_tecnicos_por_peca(nome)
624 |         self._tabela_quem_usou.setRowCount(0)
625 |         self._tabela_quem_usou.setRowCount(len(resultados))
626 |         for i, a in enumerate(resultados):
627 |             printer = self._printer_service.buscar_por_id(a.printer_id)
628 |             pat = printer.patrimonio if printer else a.printer_id
629 |             tech_nome = a.technician.nome_exibicao or a.technician.nome_completo if a.technician else "-"
630 |             self._tabela_quem_usou.setItem(i, 0, QTableWidgetItem(tech_nome))
631 |             self._tabela_quem_usou.setItem(i, 1, QTableWidgetItem(formatar_data_hora(a.event_at)))
632 |             self._tabela_quem_usou.setItem(i, 2, QTableWidgetItem(pat))
633 |             self._tabela_quem_usou.setItem(i, 3, QTableWidgetItem(a.parts_used or ""))
634 |             self._tabela_quem_usou.setItem(i, 4, QTableWidgetItem((a.notes or "")[:80]))
635 |
636 |     def _salvar_associacao(self) -> None:
637 |         tech_id = self._current_tech_id
638 |         if tech_id is None:
639 |             return
640 |         user_id = self._assoc_combo.currentData()
641 |         self._technician_service.associar_usuario(tech_id, user_id)
642 |         self._preencher_login(tech_id)
643 |
644 |     def _ao_duplo_clicar_os(self, row: int, tabela: QTableWidgetItem) -> None:
645 |         item = tabela.item(row, 0)
646 |         if item is None:
647 |             return
648 |         activity_id = item.data(Qt.UserRole)
649 |         if activity_id is not None:
650 |             self.abrir_os.emit(activity_id)
651 |
652 |     def _limpar_tabelas(self) -> None:
653 |         self._tabela_atividades.setRowCount(0)
654 |         self._tabela_andamento.setRowCount(0)
655 |         self._tabela_mov.setRowCount(0)
656 |         self._tabela_login.setRowCount(0)
657 |         self._tabela_os_abertas.setRowCount(0)
658 |         self._tabela_pecas.setRowCount(0)
659 |         self._tabela_quem_usou.setRowCount(0)
660 |         for card in [self._card_total, self._card_andamento, self._card_concluidas, self._card_mov]:
661 |             card.set_valor(0)

```

=== ARQUIVO: .\app\views\pages\transfers_page.py ===

```

001 | from __future__ import annotations
002 |
003 | import logging
004 | from datetime import datetime as dt
005 | from typing import Any
006 |
007 | from PySide6.QtCore import Qt
008 | from PySide6.QtGui import QColor

```

```

009 | from PySide6.QtWidgets import (
010 |     QComboBox,
011 |     QDialog,
012 |     QDialogButtonBox,
013 |     QFormLayout,
014 |     QFrame,
015 |     QHBoxLayout,
016 |     QLabel,
017 |     QLineEdit,
018 |     QMessageBox,
019 |     QPushButton,
020 |     QTextEdit,
021 |     QVBoxLayout,
022 |     QWidget,
023 | )
024 |
025 | from app.models import Company
026 | from app.utils.helpers import formatar_data_hora
027 | from app.utils.ui_helpers import tratar_erro
028 | from app.views.styles.theme import (
029 |     ATIVIDADE_CORES,
030 |     ESTILO_BOTAO_AVISO,
031 |     ESTILO_BOTAO_ERRO,
032 |     ESTILO_BOTAO_FECHAR,
033 |     ESTILO_BOTAO_PRIMARIO,
034 |     ESTILO_BOTAO_SECUNDARIO,
035 |     ESTILO_DIALOG,
036 |     ESTILO_INPUT,
037 |     ESTILO_SUBTITULO,
038 |     ESTILO_TABELA_SIMPLES,
039 |     ESTILO_TITULO_PAGINA,
040 |     campo_readonly,
041 |     campo_rotulo,
042 |     configurar_combo,
043 |     group_box,
044 |     input_label,
045 | )
046 | from app.views.widgets.pagination import PaginacaoWidget
047 | from app.views.widgets.search_bar import SearchBar
048 | from app.views.widgets.table_widget import TabelaPadrao, tornar_interativa
049 |
050 | log = logging.getLogger(__name__)
051 |
052 | ITENS_POR_PAGINA = 20
053 |
054 |
055 | class TransfersPage(QWidget):
056 |     session: Any
057 |     printer_service: Any
058 |     activity_service: Any
059 |     company_service: Any
060 |     _atividades_visiveis: list[Any]
061 |     tabela: TabelaPadrao
062 |     _paginacao: PaginacaoWidget
063 |     _filtro_status: str
064 |     _filtro_busca: str
065 |
066 |     def __init__(self, session: Any, printer_service: Any, activity_service: Any, company_service: Any, par
067 |         super().__init__(parent)
068 |         self.session = session
069 |         self.printer_service = printer_service
070 |         self.activity_service = activity_service
071 |         self.company_service = company_service

```



```

072 |         self._filtro_status = "Todos"
073 |         self._filtro_busca = ""
074 |         self._atividades_visiveis = []
075 |         self._init_ui()
076 |
077 |     def _init_ui(self) -> None:
078 |         layout = QVBoxLayout(self)
079 |         layout.setContentsMargins(24, 24, 24, 24)
080 |         layout.setSpacing(16)
081 |
082 |         titulo = QLabel("■ Transferências")
083 |         titulo.setStyleSheet(ESTILO_TITULO_PAGINA)
084 |         layout.addWidget(titulo)
085 |
086 |         toolbar = QHBoxLayout()
087 |         toolbar.setSpacing(8)
088 |
089 |         self.btn_todos = QPushButton("Todos")
090 |         self.btn_abertas = QPushButton("Abertas")
091 |         self.btn_concluidas = QPushButton("Concluídas")
092 |         for btn in (self.btn_todos, self.btn_abertas, self.btn_concluidas):
093 |             btn.setCursor(Qt.PointingHandCursor)
094 |             btn.setMinimumHeight(32)
095 |             btn.setStyleSheet(ESTILO_BOTAO_SECUNDARIO)
096 |         self.btn_todos.setStyleSheet(ESTILO_BOTAO_PRIMARIO)
097 |         self.btn_todos.clicked.connect(lambda: self._filtrar_status("Todos"))
098 |         self.btn_abertas.clicked.connect(lambda: self._filtrar_status("Aberta"))
099 |         self.btn_concluidas.clicked.connect(lambda: self._filtrar_status("Concluido"))
100 |
101 |         toolbar.addWidget(self.btn_todos)
102 |         toolbar.addWidget(self.btn_abertas)
103 |         toolbar.addWidget(self.btn_concluidas)
104 |         toolbar.addStretch()
105 |
106 |         self.lbl_contador = QLabel("")
107 |         self.lbl_contador.setStyleSheet(ESTILO_SUBTITULO)
108 |         toolbar.addWidget(self.lbl_contador)
109 |         layout.addLayout(toolbar)
110 |
111 |         self.search = SearchBar()
112 |         self.search.setMinimumWidth(250)
113 |         self.search.buscado.connect(self._on_busca)
114 |         layout.addWidget(self.search)
115 |
116 |         self.tabela = TabelaPadrao(
117 |             colunas=["Patrimônio", "De", "Para", "Status", "Data"],
118 |             larguras=[120, 200, 200, 100, 120],
119 |         )
120 |         tornar_interativa(self.tabela)
121 |         self.tabela.cellDoubleClicked.connect(self._on_duplo_clique)
122 |         layout.addWidget(self.tabela, stretch=1)
123 |
124 |         self._paginacao = PaginacaoWidget()
125 |         self._paginacao.mudou_pagina.connect(self._renderizar_pagina)
126 |         layout.addWidget(self._paginacao)
127 |
128 |         self.recarregar()
129 |
130 |     def recarregar(self) -> None:
131 |         try:
132 |             todas = self.activity_service.listar(filtro_tipo="MOVIMENTACAO", limite=500)
133 |             self._todas_atividades = todas
134 |             self._aplicar_filtros()

```

```

135 |         except Exception:
136 |             log.exception("Erro ao carregar transferências")
137 |             self._todas_atividades = []
138 |             self._atividades_visiveis = []
139 |             self._renderizar_tabela()
140 |
141 |     def _filtrar_status(self, status: str) -> None:
142 |         self._filtro_status = status
143 |         for btn in (self.btn_todos, self.btn_abertas, self.btn_concluidas):
144 |             btn.setStyleSheet(ESTILO_BOTAO_SECUNDARIO)
145 |         if status == "Todos":
146 |             self.btn_todos.setStyleSheet(ESTILO_BOTAO_PRIMARIO)
147 |         elif status == "Aberta":
148 |             self.btn_abertas.setStyleSheet(ESTILO_BOTAO_PRIMARIO)
149 |         else:
150 |             self.btn_concluidas.setStyleSheet(ESTILO_BOTAO_PRIMARIO)
151 |         self._aplicar_filtros()
152 |
153 |     def _on_busca(self, texto: str) -> None:
154 |         self._filtro_busca = texto.lower().strip()
155 |         self._aplicar_filtros()
156 |
157 |     def _aplicar_filtros(self) -> None:
158 |         resultado = self._todas_atividades
159 |         if self._filtro_status and self._filtro_status != "Todos":
160 |             resultado = [a for a in resultado if a.status_atividade == self._filtro_status]
161 |         if self._filtro_busca:
162 |             resultado = [a for a in resultado if self._matches_busca(a)]
163 |         self._atividades_visiveis = resultado
164 |         self._paginacao.configurar(len(self._atividades_visiveis), ITENS_POR_PAGINA)
165 |         self._renderizar_pagina(1)
166 |
167 |     def _matches_busca(self, a: Any) -> bool:
168 |         termo = self._filtro_busca
169 |         printer = self.printer_service.buscar_por_id(a.printer_id) if a.printer_id else None
170 |         campos = [
171 |             a.numero_recibo or "",
172 |             a.responsavel or "",
173 |             a.from_location or "",
174 |             a.to_location or "",
175 |             a.notes or "",
176 |             a.status_atividade or "",
177 |             printer.patrimonio if printer else "",
178 |             printer.modelo if printer else "",
179 |         ]
180 |         return any(termo in c.lower() for c in campos)
181 |
182 |     def _renderizar_pagina(self, pagina: int) -> None:
183 |         inicio = (pagina - 1) * ITENS_POR_PAGINA
184 |         fim = inicio + ITENS_POR_PAGINA
185 |         paginados = self._atividades_visiveis[inicio:fim]
186 |         total = len(self._atividades_visiveis)
187 |         self.lbl_contador.setText(f"{total} transferência(s)")
188 |         self._atividades_pagina = paginados
189 |         self._renderizar_tabela()
190 |
191 |     def _renderizar_tabela(self) -> None:
192 |         atividades = getattr(self, '_atividades_pagina', [])
193 |         self.tabela.setRowCount(len(atividades))
194 |         for i, a in enumerate(atividades):
195 |             printer = self.printer_service.buscar_por_id(a.printer_id) if a.printer_id else None
196 |             patrimonio = printer.patrimonio if printer else "-"
197 |             empresa_de = ""

```

```

198 |         empresa_para = ""
199 |         if a.from_company_id:
200 |             c = self.session.query(Company).filter(Company.id == a.from_company_id).first()
201 |             empresa_de = c.nome if c else str(a.from_company_id)
202 |         if a.to_company_id:
203 |             c = self.session.query(Company).filter(Company.id == a.to_company_id).first()
204 |             empresa_para = c.nome if c else str(a.to_company_id)
205 |         de = a.from_location or empresa_de or "-"
206 |         para = a.to_location or empresa_para or "-"
207 |         status = a.status_atividade or "-"
208 |         data = formatar_data_hora(a.event_at) if a.event_at else "-"
209 |
210 |         self.tabela.setItem(i, 0, self._item(patrimonio))
211 |         self.tabela.setItem(i, 1, self._item(de))
212 |         self.tabela.setItem(i, 2, self._item(para))
213 |         item_status = self._item(status)
214 |         cor = ATIVIDADE_CORES.get(status)
215 |         if cor:
216 |             item_status.setForeground(QColor(cor))
217 |         self.tabela.setItem(i, 3, item_status)
218 |         self.tabela.setItem(i, 4, self._item(data))
219 |     self.tabela.redimensionar()
220 |
221 |     def _item(self, texto: str):
222 |         from PySide6.QtWidgets import QTableWidgetItem
223 |         item = QTableWidgetItem(texto)
224 |         item.setFlags(item.flags() & ~Qt.ItemIsEditable)
225 |         return item
226 |
227 |     def _on_duplo_clique(self, row: int, _col: int) -> None:
228 |         if 0 <= row < len(self._atividades_pagina):
229 |             self._abrir_edicao(self._atividades_pagina[row])
230 |
231 |     def _abrir_edicao(self, activity: Any, source: Any | None = None) -> None:
232 |         try:
233 |             printer = self.printer_service.buscar_por_id(activity.printer_id) if activity.printer_id else None
234 |             dlg = _EdicaoTransferenciaDialog(
235 |                 activity=activity,
236 |                 printer=printer,
237 |                 activity_service=self.activity_service,
238 |                 printer_service=self.printer_service,
239 |                 company_service=self.company_service,
240 |                 parent=self,
241 |             )
242 |             if dlg.exec() == QDialog.Accepted:
243 |                 self.recarregar()
244 |         except Exception:
245 |             log.exception("Erro ao abrir edição de transferência")
246 |             QMessageBox.warning(self, "Erro", "Não foi possível abrir a edição.")
247 |
248 |
249 |     class _EdicaoTransferenciaDialog(QDialog):
250 |         def __init__(self, activity: Any, printer: Any, activity_service: Any, printer_service: Any, company_service: Any):
251 |             super().__init__(parent)
252 |             self.activity = activity
253 |             self.printer = printer
254 |             self.activity_service = activity_service
255 |             self.printer_service = printer_service
256 |             self.company_service = company_service
257 |             self.setWindowTitle("Editar Transferência")
258 |             self.setMinimumWidth(520)
259 |             self.setMinimumHeight(420)
260 |             self.setStyleSheet(ESTILO_DIALOG)

```

```

261 |         self._init_ui()
262 |
263 |     def _init_ui(self) -> None:
264 |         layout = QVBoxLayout(self)
265 |         layout.setSpacing(12)
266 |
267 |         patr = self.printer.patrimonio if self.printer else "-"
268 |         modelo = self.printer.modelo if self.printer else ""
269 |         lbl_info = QLabel(f"Transferência – {patr} {modelo}")
270 |         lbl_info.setStyleSheet(ESTILO_SUBTITULO)
271 |         layout.addWidget(lbl_info)
272 |
273 |         form = QFormLayout()
274 |         form.setSpacing(10)
275 |
276 |         self.txt_de = QLineEdit(self.activity.from_location or "")
277 |         self.txt_de.setPlaceholderText("Local de origem")
278 |         self.txt_de.setStyleSheet(ESTILO_INPUT)
279 |         form.addRow("De:", self.txt_de)
280 |
281 |         self.txt_para = QLineEdit(self.activity.to_location or "")
282 |         self.txt_para.setPlaceholderText("Local de destino")
283 |         self.txt_para.setStyleSheet(ESTILO_INPUT)
284 |         form.addRow("Para:", self.txt_para)
285 |
286 |         self.txt_responsavel = QLineEdit(self.activity.responsavel or "")
287 |         self.txt_responsavel.setPlaceholderText("Responsável")
288 |         self.txt_responsavel.setStyleSheet(ESTILO_INPUT)
289 |         form.addRow("Responsável:", self.txt_responsavel)
290 |
291 |         self.txt_recibo = QLineEdit(self.activity.numero_recibo or "")
292 |         self.txt_recibo.setPlaceholderText("Nº recibo")
293 |         self.txt_recibo.setStyleSheet(ESTILO_INPUT)
294 |         form.addRow("Recibo:", self.txt_recibo)
295 |
296 |         self.combo_status = QComboBox()
297 |         self.combo_status.addItem("Aberta", "Em Deslocamento", "Concluido"])
298 |         self.combo_status.setStyleSheet(ESTILO_INPUT)
299 |         configurar_combo(self.combo_status)
300 |         idx = self.combo_status.findText(self.activity.status_atividade or "Aberta")
301 |         if idx >= 0:
302 |             self.combo_status.setCurrentIndex(idx)
303 |         form.addRow("Status:", self.combo_status)
304 |
305 |         self.txt_notas = QTextEdit()
306 |         self.txt_notas.setPlainText(self.activity.notes or "")
307 |         self.txt_notas.setPlaceholderText("Observações...")
308 |         self.txt_notas.setStyleSheet(ESTILO_INPUT)
309 |         self.txt_notas.setMaximumHeight(80)
310 |         form.addRow("Notas:", self.txt_notas)
311 |
312 |         layout.addLayout(form)
313 |
314 |         botoes = QDialogButtonBox()
315 |         btn_salvar = botoes.addButton("Salvar", QDialogButtonBox.AcceptRole)
316 |         btn_salvar.setStyleSheet(ESTILO_BOTAO_PRIMARIO)
317 |         btn_salvar.setCursor(Qt.PointingHandCursor)
318 |         btn_cancelar = botoes.addButton("Cancelar", QDialogButtonBox.RejectRole)
319 |         btn_cancelar.setStyleSheet(ESTILO_BOTAO_SECUNDARIO)
320 |         btn_cancelar.setCursor(Qt.PointingHandCursor)
321 |         botoes.accepted.connect(self._salvar)
322 |         botoes.rejected.connect(self.reject)
323 |         layout.addWidget(botoes)

```



```

016 | "Normal": "#f9e2af",
017 | "Alta": "#fab387",
018 | "Crítica": "#f38ba8",
019 | }
020 |
021 | STATUS_CORES: dict[str, str] = {
022 |     "Operacional": "#34d399",
023 |     "Em uso": "#34d399",
024 |     "Em manutenção": "#f97316",
025 |     "Manutenção": "#f97316",
026 |     "Aguardando peça": "#f59e0b",
027 |     "Parada": "#ef4444",
028 |     "Sucata": "#717182",
029 | }
030 |
031 | ATIVIDADE_CORES: dict[str, str] = {
032 |     "Aberta": "#94a3b8",
033 |     "Aguardando Peça": "#f59e0b",
034 |     "Técnico Designado": "#3b82f6",
035 |     "Em Deslocamento": "#8b5cf6",
036 |     "Em Manutenção": "#ef4444",
037 |     "Em Atendimento": "#6366f1",
038 |     "Aguardando Aprovação": "#f97316",
039 |     "Concluído": "#10b981",
040 |     "Verificada": "#34d399",
041 | }
042 |
043 | TIPO_ATIVIDADE_CORES: dict[str, str] = {
044 |     "Todos": "#717182",
045 |     "MANUTENCAO": "#f97316",
046 |     "MOVIMENTACAO": "#3b82f6",
047 | }
048 |
049 | TIPO_CLIENTE_CORES: dict[str, str] = {
050 |     "Cliente": "#6366f1",
051 |     "Filial": "#34d399",
052 |     "Parceiro": "#f59e0b",
053 | }
054 |
055 | PERFIL_CORES: dict[str, str] = {
056 |     "admin": "#ef4444",
057 |     "tecnico": "#3b82f6",
058 |     "visualizador": "#717182",
059 | }
060 |
061 | SIM_NAO_CORES: dict[str, str] = {
062 |     "Sim": "#10b981",
063 |     "Não": "#ef4444",
064 | }
065 |
066 | CORES_GRAFICO: list[str] = ["#6366f1", "#10b981", "#f59e0b", "#ec4899", "#8b5cf6", "#3b82f6", "#06b6d4", "#f9e2af", "#fab387", "#f38ba8", "#717182", "#34d399", "#f97316", "#f59e0b", "#ef4444", "#6366f1", "#94a3b8", "#34d399", "#3b82f6", "#8b5cf6", "#ef4444", "#6366f1", "#f97316", "#10b981", "#34d399", "#f59e0b", "#ec4899", "#8b5cf6", "#3b82f6", "#06b6d4", "#f9e2af", "#fab387", "#f38ba8", "#717182", "#34d399", "#f97316", "#f59e0b", "#ef4444", "#6366f1", "#94a3b8", "#34d399", "#3b82f6", "#8b5cf6", "#ef4444", "#6366f1", "#f97316", "#10b981", "#34d399", "#f59e0b", "#ec4899", "#8b5cf6", "#3b82f6", "#06b6d4", "#f9e2af", "#fab387", "#f38ba8", "#717182", "#34d399", "#f97316", "#f59e0b", "#ef4444", "#6366f1", "#94a3b8", "#34d399", "#3b82f6", "#8b5cf6", "#ef4444", "#6366f1", "#f97316", "#10b981", "#34d399", "#f59e0b", "#ec4899", "#8b5cf6", "#3b82f6", "#06b6d4", "#f9e2af", "#fab387", "#f38ba8", "#717182", "#34d399", "#f97316", "#f59e0b", "#ef4444", "#6366f1", "#94a3b8", "#34d399", "#3b82f6", "#8b5cf6", "#ef4444", "#6366f1", "#f97316", "#10b981", "#34d399", "#f59e0b", "#ec4899", "#8b5cf6", "#3b82f6", "#06b6d4", "#f9e2af", "#fab387", "#f38ba8", "#717182", "#34d399", "#f97316", "#f59e0b", "#ef4444", "#6366f1", "#94a3b8", "#34d399", "#3b82f6", "#8b5cf6", "#ef4444", "#6366f1", "#f97316", "#10b981", "#34d399", "#f59e0b", "#ec4899", "#8b5cf6", "#3b82f6", "#06b6d4", "#f9e2af", "#fab387", "#f38ba8", "#717182", "#34d399", "#f97316", "#f59e0b", "#ef4444", "#6366f1", "#94a3b8", "#34d399", "#3b82f6", "#8b5cf6", "#ef4444", "#6366f1", "#f97316", "#10b981", "#34d399", "#f59e0b", "#ec4899", "#8b5cf6", "#3b82f6", "#06b6d4", "#f9e2af", "#fab387", "#f38ba8", "#717182", "#34d399", "#f97316", "#f59e0b", "#ef4444", "#6366f1", "#94a3b8", "#34d399", "#3b82f6", "#8b5cf6", "#ef4444", "#6366f1", "#f97316", "#10b981", "#34d399", "#f59e0b", "#ec4899", "#8b5cf6", "#3b82f6", "#06b6d4", "#f9e2af", "#fab387", "#f38ba8", "#717182", "#34d399", "#f97316", "#f59e0b", "#ef4444", "#6366f1", "#94a3b8", "#34d399", "#3b82f6", "#8b5cf6", "#ef4444", "#6366f1", "#f97316", "#10b981", "#34d399", "#f59e0b", "#ec4899", "#8b5cf6", "#3b82f6", "#06b6d4", "#f9e2af", "#fab387", "#f38ba8", "#717182", "#34d399", "#f97316", "#f59e0b", "#ef4444", "#6366f1", "#94a3b8", "#34d399", "#3b82f6", "#8b5cf6", "#ef4444", "#6366f1", "#f97316", "#10b981", "#34d399", "#f59e0b", "#ec4899", "#8b5cf6", "#3b82f6", "#06b6d4", "#f9e2af", "#fab387", "#f38ba8", "#717182", "#34d399", "#f97316", "#f59e0b", "#ef4444", "#6366f1", "#94a3b8", "#34d399", "#3b82f6", "#8b5cf6", "#ef4444", "#6366f1", "#f97316", "#10b981", "#34d399", "#f59e0b", "#ec4899", "#8b5cf6", "#3b82f6", "#06b6d4", "#f9e2af", "#fab387", "#f38ba8", "#717182", "#34d399", "#f97316", "#f59e0b", "#ef4444", "#6366f1", "#94a3b8", "#34d399", "#3b82f6", "#8b5cf6", "#ef4444", "#6366f1", "#f97316", "#10b981", "#34d399", "#f59e0b", "#ec4899", "#8b5cf6", "#3b82f6", "#06b6d4", "#f9e2af", "#fab387", "#f38ba8", "#717182", "#34d399", "#f97316", "#f59e0b", "#ef4444", "#6366f1", "#94a3b8", "#34d399", "#3b82f6", "#8b5cf6", "#ef4444", "#6366f1", "#f97316", "#10b981", "#34d399", "#f59e0b", "#ec4899", "#8b5cf6", "#3b82f6", "#06b6d4", "#f9e2af", "#fab387", "#f38ba8", "#717182", "#34d399", "#f97316", "#f59e0b", "#ef4444", "#6366f1", "#94a3b8", "#34d399", "#3b82f6", "#8b5cf6", "#ef4444", "#6366f1", "#f97316", "#10b981", "#34d399", "#f59e0b", "#ec4899", "#8b5cf6", "#3b82f6", "#06b6d4", "#f9e2af", "#fab387", "#f38ba8", "#717182", "#34d399", "#f97316", "#f59e0b", "#ef4444", "#6366f1", "#94a3b8", "#34d399", "#3b82f6", "#8b5cf6", "#ef4444", "#6366f1", "#f97316", "#10b981", "#34d399", "#f59e0b", "#ec4899", "#8b5cf6", "#3b82f6", "#06b6d4", "#f9e2af", "#fab387", "#f38ba8", "#717182", "#34d399", "#f97316", "#f59e0b", "#ef4444", "#6366f1", "#94a3b8", "#34d399", "#3b82f6", "#8b5cf6", "#ef4444", "#6366f1", "#f97316", "#10b981", "#34d399", "#f59e0b", "#ec4899", "#8b5cf6", "#3b82f6", "#06b6d4", "#f9e2af", "#fab387", "#f38ba8", "#717182", "#34d399", "#f97316", "#f59e0b", "#ef4444", "#6366f1", "#94a3b8", "#34d399", "#3b82f6", "#8b5cf6", "#ef4444", "#6366f1", "#f97316", "#10b981", "#34d399", "#f59e0b", "#ec4899", "#8b5cf6", "#3b82f6", "#06b6d4", "#f9e2af", "#fab387", "#f38ba8", "#717182", "#34d399", "#f97316", "#f59e0b", "#ef4444", "#6366f1", "#94a3b8", "#34d399", "#3b82f6", "#8b5cf6", "#ef4444", "#6366f1", "#f97316", "#10b981", "#34d399", "#f59e0b", "#ec4899", "#8b5cf6", "#3b82f6", "#06b6d4", "#f9e2af", "#fab387", "#f38ba8", "#717182", "#34d399", "#f97316", "#f59e0b", "#ef4444", "#6366f1", "#94a3b8", "#34d399", "#3b82f6", "#8b5cf6", "#ef4444", "#6366f1", "#f97316", "#10b981", "#34d399", "#f59e0b", "#ec4899", "#8b5cf6", "#3b82f6", "#06b6d4", "#f9e2af", "#fab387", "#f38ba8", "#717182", "#34d399", "#f97316", "#f59e0b", "#ef4444", "#6366f1", "#94a3b8", "#34d399", "#3b82f6", "#8b5cf6", "#ef4444", "#6366f1", "#f97316", "#10b981", "#34d399", "#f59e0b",
```

```

079 |     "azul": "#3b82f6",
080 |     "azul_escuro": "#2563eb",
081 |     "hover": "#1e1e2e",
082 |     "sucesso": "#10b981",
083 |     "erro": "#ef4444",
084 |     "aviso": "#f59e0b",
085 |     "status_ok": "#34d399",
086 |     "status_alerta": "#f59e0b",
087 |     "status_ruim": "#ef4444",
088 |     "sidebar": "#0f0f16",
089 |     "topbar": "#0f0f16",
090 |     "texto_muted": "#94949f",
091 |     "texto_label": "#717182",
092 |     "nav_text": "#717182",
093 |     "nav_active_bg": "rgba(99, 102, 241, 0.12)",
094 |     "nav_active_text": "#6366f1",
095 | }
096 |
097 | CLARO = {
098 |     "fundo": "#f8fafc",
099 |     "fundo_card": "ffffff",
100 |     "fundo_input": "#f3f3f5",
101 |     "tabela_bg": "ffffff",
102 |     "borda": "#e2e8f0",
103 |     "texto": "#0f172a",
104 |     "texto_sec": "#475569",
105 |     "roxo": "#6366f1",
106 |     "roxo_escuro": "#4f46e5",
107 |     "azul": "#3b82f6",
108 |     "azul_escuro": "#2563eb",
109 |     "hover": "#f1f5f9",
110 |     "sucesso": "#10b981",
111 |     "erro": "#ef4444",
112 |     "aviso": "#f59e0b",
113 |     "status_ok": "#34d399",
114 |     "status_alerta": "#f59e0b",
115 |     "status_ruim": "#ef4444",
116 |     "sidebar": "ffffff",
117 |     "topbar": "ffffff",
118 |     "texto_muted": "#717182",
119 |     "texto_label": "#94949f",
120 |     "nav_text": "#64748b",
121 |     "nav_active_bg": "#eef2ff",
122 |     "nav_active_text": "#4f46e5",
123 | }
124 |
125 | _TEMA: str = "dark"
126 | COR: dict[str, str] = dict(ESCURO)
127 |
128 |
129 | def alternar() -> None:
130 |     global _TEMA
131 |     _TEMA = "light" if _TEMA == "dark" else "dark"
132 |     paleta: dict[str, str] = CLARO if _TEMA == "light" else ESCURO
133 |     COR.clear()
134 |     COR.update(paleta)
135 |
136 |
137 | def atual() -> str:
138 |     return _TEMA
139 |
140 |
141 | # ■■ ESTILOS (estáticos, tema escuro – Design System 2026) ■■

```

```

142 | # Inline stylesheets são LIMPOS no toggle via limpar_estilos().
143 | # O QSS global cuida da aparência correta em cada tema.
144 |
145 | ESTILO_BOTAO_PRIMARIO = """"\
146 |     QPushButton {
147 |         background: #6366f1; color: #ffffff;
148 |         border: none; border-radius: 8px;
149 |         padding: 9px 18px; font-size: 11pt; font-weight: 600;
150 |     }
151 |     QPushButton:hover { background: #4f46e5; }
152 |     QPushButton:pressed { background: #4338ca; }
153 | """"
154 |
155 | ESTILO_BOTAO_SUCESSO = """"\
156 |     QPushButton {
157 |         background: #3b82f6; color: #ffffff;
158 |         border: none; border-radius: 8px;
159 |         padding: 9px 18px; font-size: 11pt; font-weight: 600;
160 |     }
161 |     QPushButton:hover { background: #2563eb; }
162 |     QPushButton:pressed { background: #1d4ed8; }
163 | """"
164 |
165 | ESTILO_BOTAO_AVISO = """"\
166 |     QPushButton {
167 |         background: #f97316; color: #ffffff;
168 |         border: none; border-radius: 8px;
169 |         padding: 9px 18px; font-size: 11pt; font-weight: 600;
170 |     }
171 |     QPushButton:hover { background: #ea580c; }
172 |     QPushButton:pressed { background: #c2410c; }
173 | """"
174 |
175 | ESTILO_BOTAO_ERRO = """"\
176 |     QPushButton {
177 |         background: #ef4444; color: #ffffff;
178 |         border: none; border-radius: 8px;
179 |         padding: 9px 18px; font-size: 11pt; font-weight: 600;
180 |     }
181 |     QPushButton:hover { background: #dc2626; }
182 |     QPushButton:pressed { background: #b91c1c; }
183 | """"
184 |
185 | ESTILO_BOTAO_SECUNDARIO = """"\
186 |     QPushButton {
187 |         background-color: transparent; color: #e8e8f0;
188 |         border: 1px solid #2a2a3e; border-radius: 8px;
189 |         padding: 9px 18px; font-size: 11pt; font-weight: 600;
190 |     }
191 |     QPushButton:hover { background-color: #1e1e2e; color: #e8e8f0; border-color: #6366f1; }
192 | """"
193 |
194 | ESTILO_BOTAO_FECHAR = """"\
195 |     QPushButton {
196 |         background-color: transparent; color: #717182;
197 |         border: 1px solid #2a2a3e; border-radius: 8px;
198 |         padding: 9px 18px; font-size: 11pt; font-weight: 500;
199 |     }
200 |     QPushButton:hover { background-color: #1e1e2e; color: #e8e8f0; border-color: #3a3a50; }
201 | """"
202 |
203 | ESTILO_INPUT = """"\
204 |     QLineEdit, QTextEdit, QDateEdit, QTimeEdit, QDateTimeEdit, QComboBox {

```



```

205 |         background-color: #1e1e2f; color: #e2e8f0;
206 |         border: 2px solid #3b4261; border-radius: 8px;
207 |         padding: 4px 10px; font-size: 13px;
208 |     }
209 |     QLineEdit, QDateEdit, QTimeEdit, QDateTimeEdit, QComboBox { min-height: 26px; }
210 |     QLineEdit:focus, QTextEdit:focus, QDateEdit:focus, QTimeEdit:focus, QDateTimeEdit:focus, QComboBox:focus {
211 |         border: 2px solid #3b82f6; background-color: #1a1b26; color: #ffffff;
212 |     }
213 |     QLineEdit:disabled, QTextEdit:disabled, QDateEdit:disabled, QTimeEdit:disabled, QDateTimeEdit:disabled,
214 |         background-color: #14141f; color: #555568; border: 2px solid #1e1e2e;
215 |     }
216 |     QDateEdit::drop-down, QTimeEdit::drop-down, QDateTimeEdit::drop-down {
217 |         border: none; width: 30px; background: transparent;
218 |     }
219 |     QDateEdit::down-arrow, QTimeEdit::down-arrow, QDateTimeEdit::down-arrow {
220 |         image: none; border-left: 5px solid transparent; border-right: 5px solid transparent;
221 |         border-top: 6px solid #717182; width: 0; height: 0; margin-right: 8px;
222 |     }
223 |     QDateEdit:hover::down-arrow, QTimeEdit:hover::down-arrow, QDateTimeEdit:hover::down-arrow {
224 |         border-top-color: #6366f1;
225 |     }
226 |     QCalendarWidget {
227 |         background-color: #1e1e2e; color: #e8e8f0;
228 |         border: 1px solid #2a2a3e; border-radius: 8px;
229 |     }
230 |     QCalendarWidget QToolButton {
231 |         color: #e8e8f0; background: transparent;
232 |         padding: 6px 12px; border-radius: 6px; font-weight: 600;
233 |     }
234 |     QCalendarWidget QToolButton:hover { background: rgba(99, 102, 241, 0.15); }
235 |     QCalendarWidget QMenu { background-color: #1e1e2e; color: #e8e8f0; border: 1px solid #2a2a3e; }
236 |     QCalendarWidget QSpinBox { background-color: #1e1e2e; color: #e8e8f0; border: 1px solid #2a2a3e; border-
237 |     QCalendarWidget QTableView { border: none; background-color: #16162a; selection-background-color: rgba(
238 |     QCalendarWidget QTableView::item:hover { background-color: rgba(99, 102, 241, 0.15); }
239 | """"
240 |
241 | ESTILO_INPUT_READONLY = """"\
242 |     QLineEdit, QTextEdit {
243 |         background-color: #14141f; color: #717182;
244 |         border: 1px solid #2a2a3e; border-radius: 8px;
245 |         padding: 8px 12px; font-size: 13px;
246 |     }
247 | """"
248 |
249 | ESTILO_COMBO = """"\
250 |     QComboBox {
251 |         background-color: #1e1e2e; color: #e8e8f0;
252 |         border: 1px solid #2a2a3e; border-radius: 8px;
253 |         padding: 8px 12px; font-size: 13px;
254 |         min-height: 20px;
255 |     }
256 |     QComboBox:hover { border-color: #3a3a50; }
257 |     QComboBox:focus { border-color: #6366f1; }
258 |     QComboBox:on { background-color: #1e1e2e; border-color: #6366f1; }
259 |     QComboBox:disabled { background-color: #14141f; color: #3a3a50; border-color: #2a2a3e; }
260 |     QComboBox QLineEdit { background-color: #1e1e2e; color: #e8e8f0; border: none; }
261 |     QComboBox::drop-down { border: none; width: 30px; background: transparent; }
262 |     QComboBox::down-arrow { image: none; border-left: 5px solid transparent; border-right: 5px solid transp
263 |     QComboBox:hover::down-arrow { border-top-color: #6366f1; }
264 |     QComboBox QAbstractItemView {
265 |         background-color: #1e1e2e; color: #e8e8f0;
266 |         border: 1px solid #2a2a3e; border-top: none;
267 |         border-bottom-left-radius: 8px; border-bottom-right-radius: 8px;

```

```

268 |         selection-background-color: rgba(99, 102, 241, 0.15);
269 |         selection-color: #818cf8; padding: 4px; outline: none;
270 |     }
271 |     QComboBox QAbstractItemView::item {
272 |         padding: 8px 12px; min-height: 28px; border-radius: 6px;
273 |         background-color: #1e1e2e; color: #e8e8f0;
274 |     }
275 |     QComboBox QAbstractItemView::item:hover {
276 |         background-color: rgba(99, 102, 241, 0.15); color: #818cf8;
277 |     }
278 |     QComboBox QAbstractItemView::item:selected {
279 |         background-color: rgba(99, 102, 241, 0.25); color: #a5b4fc;
280 |     }
281 |     QComboBox QAbstractItemView::viewport { background-color: #1e1e2e; }
282 |     QComboBox QScrollBar:vertical {
283 |         background: #1e1e2e; width: 8px; margin: 0;
284 |         border: none; border-radius: 4px;
285 |     }
286 |     QComboBox QScrollBar::handle:vertical {
287 |         background: #3a3a50; min-height: 30px; border-radius: 4px;
288 |     }
289 |     QComboBox QScrollBar::handle:vertical:hover { background: #52527a; }
290 |     QComboBox QScrollBar::add-line:vertical, QComboBox QScrollBar::sub-line:vertical {
291 |         height: 0; background: none; border: none;
292 |     }
293 |     QComboBox QScrollBar::add-page:vertical, QComboBox QScrollBar::sub-page:vertical {
294 |         background: none;
295 |     }
296 | ""
297 |
298 | ESTILO_TABELA = """\
299 |     QWidget {
300 |         background-color: rgba(20, 20, 31, 0.5); color: #e8e8f0;
301 |         alternate-background-color: transparent;
302 |         border: 1px solid rgba(42, 42, 62, 0.5); border-radius: 12px;
303 |         gridline-color: transparent; selection-background-color: rgba(99, 102, 241, 0.15);
304 |         font-size: 13px;
305 |     }
306 |     QWidget::item { padding: 11px 16px; border-bottom: 1px solid rgba(42, 42, 62, 0.3); }
307 |     QWidget::item:hover { background-color: rgba(30, 30, 46, 0.5); color: #e8e8f0; }
308 |     QWidget::item:selected { background-color: rgba(99, 102, 241, 0.15); color: #818cf8; font-weight:
309 |     QHeaderView::section {
310 |         background-color: transparent; color: #717182;
311 |         font-weight: 700; padding: 10px 16px;
312 |         border: none; border-bottom: 1px solid rgba(42, 42, 62, 0.5);
313 |         font-size: 10px; letter-spacing: 0.8px; text-transform: uppercase;
314 |     }
315 |     QHeaderView::section:first { border-top-left-radius: 11px; }
316 |     QHeaderView::section:last { border-top-right-radius: 11px; }
317 |     QHeaderView::section:hover { background-color: rgba(30, 30, 46, 0.5); color: #94949f; }
318 |     QWidget QTableCornerButton::section { background-color: transparent; border: none; }
319 | ""
320 |
321 | ESTILO_TABELA_SIMPLES = """\
322 |     QWidget {
323 |         background-color: #14141f; color: #e8e8f0;
324 |         border: 1px solid #2a2a3e; border-radius: 10px;
325 |         gridline-color: transparent; font-size: 12px;
326 |     }
327 |     QWidget::item { padding: 8px; }
328 |     QHeaderView::section {
329 |         background-color: #14141f; color: #717182;
330 |         font-weight: 700; padding: 8px; border: none;

```

```

331 |         border-bottom: 1px solid #2a2a3e;
332 |     }
333 | """"
334 |
335 | ESTILO_DIALOG = """"\
336 |     QDialog { background-color: #14141f; }
337 |     QLabel { color: #e8e8f0; font-size: 12px; background: transparent; }
338 |     QLineEdit, QTextEdit {
339 |         background-color: #1e1e2e; color: #e8e8f0;
340 |         border: 1px solid #2a2a3e; border-radius: 8px;
341 |         padding: 8px 12px; font-size: 13px;
342 |     }
343 |     QLineEdit:hover, QTextEdit:hover { border-color: #3a3a50; }
344 |     QLineEdit:focus, QTextEdit:focus { border-color: #6366f1; }
345 | """"
346 |
347 | ESTILO_TITULO_PAGINA = "color: #e8e8f0; font-size: 22px; font-weight: 700; background: transparent; letter-
348 | ESTILO_SUBTITULO = "color: #94949f; font-size: 13px; background: transparent;"
349 |
350 | ESTILO_VIDRO = "background-color: rgba(20, 20, 31, 0.5); border: 1px solid rgba(42, 42, 62, 0.5); border-ra
351 |
352 | ESTILO_LABEL_VALOR = "font-weight: 600; background: transparent;"
353 | ESTILO_LABEL_CAMPO = "color: #3b82f6; font-weight: 600; font-size: 12px;"
354 |
355 |
356 | def _cor_rgba(hex_color: str, alpha: float = 1.0) -> str:
357 |     h: str = hex_color.lstrip("#")
358 |     r: int
359 |     g: int
360 |     b: int
361 |     r, g, b = int(h[0:2], 16), int(h[2:4], 16), int(h[4:6], 16)
362 |     return f"rgba({r},{g},{b},{alpha})"
363 |
364 | def estilo_botao_outline(cor: str, cor_hover: str | None = None, bg_hover: str | None = None) -> str:
365 |     if cor_hover is None:
366 |         cor_hover = cor
367 |     if bg_hover is None:
368 |         bg_hover = cor
369 |     return f""""
370 |     QPushButton {{
371 |         background-color: {_cor_rgba(cor, 0.10)}; color: {cor};
372 |         border: 1px solid {_cor_rgba(cor, 0.30)}; border-radius: 8px;
373 |         padding: 9px 18px; font-size: 11pt; font-weight: 600;
374 |     }}
375 |     QPushButton:hover {{
376 |         background-color: {bg_hover}; color: #ffffff;
377 |         border: 1px solid {cor_hover};
378 |     }}
379 | """"
380 |
381 |
382 | def configurar_combo(combo: QComboBox) -> None:
383 |     """"Garante fundo escuro e habilita digitação em qualquer QComboBox.
384 |     editable + read-only (contorna bug do Fusion que ignora
385 |     background-color via stylesheet em combos não-editáveis).""""
386 |     combo.setEditable(True)
387 |     combo.setInsertPolicy(QComboBox.NoInsert)
388 |     combo.setMinimumHeight(38)
389 |
390 |     le = combo.lineEdit()
391 |     if le:
392 |         le.setAlignment(Qt.AlignmentFlag.AlignLeft)
393 |         le.setStyleSheet(

```

```

394 |         "background-color: #1e1e2e; color: #e8e8f0;"
395 |         " border: none;"
396 |     )
397 |
398 | combo.setStyleSheet(
399 |     "QComboBox { background-color: #1e1e2e; color: #e8e8f0;"
400 |     " border: 1px solid #2a2a3e; border-radius: 8px;"
401 |     " padding: 0px 12px; font-size: 13px;"
402 |     " min-height: 36px; }"
403 |     "QComboBox:hover { border-color: #3a3a50; }"
404 |     "QComboBox:focus { border-color: #6366f1; }"
405 |     "QComboBox:on { background-color: #1e1e2e; border-color: #6366f1; }"
406 |     "QComboBox QLineEdit { background-color: #1e1e2e; color: #e8e8f0;"
407 |     " border: none; }"
408 |     "QComboBox::drop-down { border: none; width: 30px;"
409 |     " background: transparent; }"
410 |     "QComboBox::down-arrow {"
411 |     " image: none; border-left: 5px solid transparent;"
412 |     " border-right: 5px solid transparent;"
413 |     " border-top: 6px solid #717182; width: 0; height: 0;"
414 |     " margin-right: 8px; }"
415 |     "QComboBox:disabled { background-color: #14141f;"
416 |     " color: #3a3a50; }"
417 | )
418 |
419 | orig: Callable[[[], Any] = combo.showPopup
420 | def _popup() -> None:
421 |     orig()
422 |     v = combo.view()
423 |     if v:
424 |         from PySide6.QtWidgets import QStyleFactory
425 |         v.setStyle(QStyleFactory.create("Fusion"))
426 |         v.setStyleSheet(
427 |             "QListView { background-color: #1e1e2e; color: #e8e8f0; }"
428 |             "QListView::item { background-color: #1e1e2e; color: #e8e8f0;"
429 |             " padding: 8px 12px; }"
430 |             "QListView::item:hover {"
431 |             " background-color: rgba(99, 102, 241, 0.15);"
432 |             " color: #818cf8; }"
433 |             "QListView::item:selected {"
434 |             " background-color: rgba(99, 102, 241, 0.25);"
435 |             " color: #a5b4fc; }"
436 |         )
437 |     combo.showPopup = _popup
438 |
439 |
440 | def input_label(texto: str) -> QLabel:
441 |     """Label para campos de formulário (cinza, uppercase)."""
442 |     lbl = QLabel(texto)
443 |     lbl.setStyleSheet(
444 |         "color: #94949f; font-size: 10px; font-weight: 600;"
445 |         " background: transparent; letter-spacing: 0.5px;"
446 |         " text-transform: uppercase;"
447 |     )
448 |     return lbl
449 |
450 |
451 | def group_box(titulo: str) -> tuple[QWidget, QVBoxLayout]:
452 |     """Retorna um widget agrupado com título azul (#6366f1) e layout interno."""
453 |     box = QWidget()
454 |     box.setStyleSheet("background: transparent;")
455 |     outer = QVBoxLayout(box)
456 |     outer.setContentsMargins(0, 0, 0, 0)

```

```

457 |     outer.setSpacing(6)
458 |     title_lbl = QLabel(titulo.upper())
459 |     title_lbl.setStyleSheet(
460 |         "color: #6366f1; font-size: 9px; font-weight: 700;"
461 |         " letter-spacing: 1.2px; background: transparent;"
462 |     )
463 |     outer.addWidget(title_lbl)
464 |     return box, outer
465 |
466 |
467 | def configurar_combo_colorido(combo: QComboBox, mapa_cores: dict[str, str]) -> None:
468 |     """Conecta currentTextChanged do combo para atualizar borda e cor do texto."""
469 |     def _atualizar(texto: str):
470 |         cor = mapa_cores.get(texto, "#e8e8f0")
471 |         combo.setStyleSheet(
472 |             "QComboBox { background-color: #1e1e2e; color: #e8e8f0;"
473 |             f" border: 2px solid {cor}; border-radius: 8px;"
474 |             " padding: 0px 12px; font-size: 13px; min-height: 36px; }"
475 |             "QComboBox:hover { border-color: #3a3a50; }"
476 |             f"QComboBox:focus {{ border-color: {cor}; }}"
477 |             f"QComboBox:on {{ background-color: #1e1e2e; border-color: {cor}; }}"
478 |             "QComboBox::drop-down { border: none; width: 30px; background: transparent; }"
479 |             "QComboBox::down-arrow { image: none; border-left: 5px solid transparent;"
480 |             " border-right: 5px solid transparent; border-top: 6px solid #717182;"
481 |             " width: 0; height: 0; }"
482 |             "QComboBox:hover::down-arrow { border-top-color: #6366f1; }"
483 |             "QComboBox QAbstractItemView { background-color: #1e1e2e; color: #e8e8f0;"
484 |             " border: 1px solid #2a2a3e; border-top: none;"
485 |             " selection-background-color: rgba(99, 102, 241, 0.15);"
486 |             " selection-color: #818cf8; outline: none; }"
487 |         )
488 |         le = combo.lineEdit()
489 |         if le:
490 |             le.setStyleSheet(f"background-color: #1e1e2e; color: {cor}; border: none; padding: 0; min-height: 36px;")
491 |         combo.currentTextChanged.connect(_atualizar)
492 |         _atualizar(combo.currentText())
493 |
494 |
495 | def campo_rotulo(texto: str) -> QLabel:
496 |     """Rótulo em negrito para exibição de campo em detalhes."""
497 |     lbl = QLabel(texto)
498 |     lbl.setStyleSheet(
499 |         "color: #717182; font-size: 9px; font-weight: 700;"
500 |         " letter-spacing: 0.8px; background: transparent;"
501 |     )
502 |     return lbl
503 |
504 |
505 | def campo_readonly(texto: str) -> QLabel:
506 |     """Valor readonly para exibição de campo em detalhes."""
507 |     lbl = QLabel(texto)
508 |     lbl.setWordWrap(True)
509 |     lbl.setStyleSheet(
510 |         "color: #c8c8e0; font-size: 12px; font-weight: 500;"
511 |         " background: transparent; padding: 0; margin: 0;"
512 |     )
513 |     return lbl

```

=== ARQUIVO: .lapp\views\styles\theme_manager.py ===

```

001 | from __future__ import annotations

```

```

002 |
003 | import os
004 |
005 | from PySide6.QtWidgets import QApplication, QWidget
006 |
007 | import app.views.styles.theme as theme
008 |
009 | _THEME_DIR = os.path.join(os.path.dirname(__file__), "..", "..", "..", "themes")
010 |
011 |
012 | class TemaManager:
013 |     @classmethod
014 |     def atual(cls) -> str:
015 |         return theme.atual()
016 |
017 |     @classmethod
018 |     def alternar(cls) -> str:
019 |         theme.alternar()
020 |         cls._aplicar_qss()
021 |         return theme.atual()
022 |
023 |     @classmethod
024 |     def _aplicar_qss(cls) -> None:
025 |         app: QApplication | None = QApplication.instance()
026 |         if not app:
027 |             return
028 |         qss_file: str = "dark_premium.qss" if theme.atual() == "dark" else "light.qss"
029 |         path: str = os.path.join(_THEME_DIR, qss_file)
030 |         if os.path.exists(path):
031 |             with open(path, "r", encoding="utf-8") as f:
032 |                 app.setStyleSheet(f.read())
033 |
034 |     @classmethod
035 |     def limpar_estilos(cls, root: QWidget) -> None:
036 |         fila: list[QWidget] = [root]
037 |         while fila:
038 |             w: QWidget = fila.pop(0)
039 |             w.setStyleSheet("")
040 |             for child in w.findChildren(QWidget):
041 |                 fila.append(child)
042 |
043 |     @classmethod
044 |     def reestilizar_paginas(cls, widgets: list[QWidget]) -> None:
045 |         cls._aplicar_qss()
046 |         for w in widgets:
047 |             cls.limpar_estilos(w)
048 |             if hasattr(w, "recarregar"):
049 |                 w.recarregar()

```

=== ARQUIVO: .lapp\views\styles__init__.py ===

=== ARQUIVO: .lapp\views\widgets\badge_widget.py ===

```

001 | from __future__ import annotations
002 |
003 | from typing import Optional
004 |
005 | from PySide6.QtCore import Qt

```

```

006 | from PySide6.QtWidgets import QHBoxLayout, QLabel, QWidget
007 |
008 |
009 | class BadgeWidget(QWidget):
010 |     def __init__(self, texto: str, cor: Optional[str] = None, parent: Optional[QWidget] = None) -> None:
011 |         super().__init__(parent)
012 |         if cor is None:
013 |             cor = "#6366f1"
014 |
015 |         self.setAttribute(Qt.WA_StyledBackground, False)
016 |         self.setStyleSheet("background: transparent; border: none;")
017 |
018 |         layout = QHBoxLayout(self)
019 |         layout.setContentsMargins(0, 0, 0, 0)
020 |         layout.setSpacing(5)
021 |
022 |         dot = QLabel()
023 |         dot.setFixedSize(7, 7)
024 |         dot.setStyleSheet(
025 |             f"background-color: {cor}; border-radius: 4px; border: none;"
026 |         )
027 |         layout.addWidget(dot)
028 |
029 |         label = QLabel(texto)
030 |         label.setStyleSheet(
031 |             f"color: {cor}; font-size: 11px; font-weight: 600; "
032 |             f"background: transparent; border: none; padding: 0;"
033 |         )
034 |         layout.addWidget(label)

```

=== ARQUIVO: .lapp\views\widgets\base_dialog.py ===

```

001 | from __future__ import annotations
002 |
003 | from typing import Optional
004 |
005 | from PySide6.QtCore import Qt
006 | from PySide6.QtWidgets import QDialog, QHBoxLayout, QPushButton
007 |
008 | import app.views.styles.theme as theme
009 |
010 |
011 | class DialogoBase(QDialog):
012 |     def __init__(self, titulo: str, parent: Optional[QDialog] = None, largura: int = 500, altura: int = 500):
013 |         super().__init__(parent)
014 |         self.setWindowTitle(titulo)
015 |         self.setMinimumSize(largura, altura)
016 |         c = theme.COR
017 |         self.setStyleSheet(f"QDialog {{ background-color: {c['fundo']}; }}")
018 |
019 |     def botao_salvar(self, texto: str = "■ Salvar") -> QPushButton:
020 |         btn = QPushButton(texto)
021 |         btn.setCursor(Qt.PointingHandCursor)
022 |         c = theme.COR
023 |         btn.setStyleSheet(f"""
024 |             QPushButton {{
025 |                 background: qlineargradient(x1:0,y1:0,x2:1,y2:1,stop:0 {c['azul']},stop:1 {c['azul']})dd);
026 |                 color: #ffffff;
027 |                 border: none; border-radius: 8px;
028 |                 padding: 10px 20px; font-size: 13px; font-weight: 600;
029 |             }}

```

```

030 |         QPushButton:hover {{ background: qlineargradient(x1:0,y1:0,x2:1,y2:1,stop:0 #93c5fd,stop:1 {c['
031 |         ""})
032 |         return btn
033 |
034 |     def botao_excluir(self, texto: str = "■ Excluir") -> QPushButton:
035 |         btn = QPushButton(texto)
036 |         btn.setCursor(Qt.PointingHandCursor)
037 |         c = theme.COR
038 |         btn.setStyleSheet(f"""
039 |             QPushButton {{
040 |                 background: qlineargradient(x1:0,y1:0,x2:1,y2:1,stop:0 #f87171,stop:1 #ef4444);
041 |                 color: #ffffff;
042 |                 border: none; border-radius: 8px;
043 |                 padding: 10px 20px; font-size: 13px; font-weight: 600;
044 |             }}
045 |             QPushButton:hover {{ background: qlineargradient(x1:0,y1:0,x2:1,y2:1,stop:0 #fca5a5,stop:1 #f87
046 |             ""})
047 |         return btn
048 |
049 |     def botao_cancelar(self, texto: str = "Cancelar") -> QPushButton:
050 |         c = theme.COR
051 |         btn = QPushButton(texto)
052 |         btn.setCursor(Qt.PointingHandCursor)
053 |         btn.setStyleSheet(f"""
054 |             QPushButton {{
055 |                 background-color: transparent; color: {c['texto_sec']};
056 |                 border: 1px solid {c['borda']}; border-radius: 8px;
057 |                 padding: 10px 20px; font-size: 13px; font-weight: 500;
058 |             }}
059 |             QPushButton:hover {{ background-color: {c['hover']}; color: {c['texto']}; }}
060 |         """)
061 |         btn.clicked.connect(self.reject)
062 |         return btn
063 |
064 |     def botoes_layout(self) -> QHBoxLayout:
065 |         layout = QHBoxLayout()
066 |         layout.setSpacing(10)
067 |         return layout

```

=== ARQUIVO: .\app\views\widgets\card_widget.py ===

```

001 | from __future__ import annotations
002 |
003 | from collections.abc import Callable
004 | from typing import Any, Optional
005 |
006 | from PySide6.QtCore import Qt
007 | from PySide6.QtGui import QMouseEvent
008 | from PySide6.QtWidgets import QFrame, QHBoxLayout, QLabel, QVBoxLayout
009 |
010 | from app.utils.effects import sombra_card
011 |
012 | ESTILOS_CARD = {
013 |     "azul": {
014 |         "card": "background: qlineargradient(x1:0,y1:0,x2:1,y2:1,stop:0 rgba(99,102,241,0.10),stop:1 rgba(7
015 |         "icon_bg": "background: rgba(99,102,241,0.20);",
016 |         "icon_color": "#6366f1",
017 |     },
018 |     "verde": {
019 |         "card": "background: qlineargradient(x1:0,y1:0,x2:1,y2:1,stop:0 rgba(16,185,129,0.10),stop:1 rgba(5
020 |         "icon_bg": "background: rgba(16,185,129,0.20);",

```



```

021 |         "icon_color": "#10b981",
022 |     },
023 |     "amarelo": {
024 |         "card": "background: qlineargradient(x1:0,y1:0,x2:1,y2:1,stop:0 rgba(249,115,22,0.10),stop:1 rgba(249,115,22,0.20));",
025 |         "icon_bg": "background: rgba(249,115,22,0.20);",
026 |         "icon_color": "#f97316",
027 |     },
028 |     "rosa": {
029 |         "card": "background: qlineargradient(x1:0,y1:0,x2:1,y2:1,stop:0 rgba(236,72,153,0.10),stop:1 rgba(236,72,153,0.20));",
030 |         "icon_bg": "background: rgba(236,72,153,0.20);",
031 |         "icon_color": "#ec4899",
032 |     },
033 |     "roxo": {
034 |         "card": "background: qlineargradient(x1:0,y1:0,x2:1,y2:1,stop:0 rgba(139,92,246,0.10),stop:1 rgba(139,92,246,0.20));",
035 |         "icon_bg": "background: rgba(139,92,246,0.20);",
036 |         "icon_color": "#8b5cf6",
037 |     },
038 |     "blue": {
039 |         "card": "background: qlineargradient(x1:0,y1:0,x2:1,y2:1,stop:0 rgba(59,130,246,0.10),stop:1 rgba(59,130,246,0.20));",
040 |         "icon_bg": "background: rgba(59,130,246,0.20);",
041 |         "icon_color": "#3b82f6",
042 |     },
043 | }
044 |
045 |
046 | class CardWidget(QFrame):
047 |     valor_label: QLabel
048 |
049 |     def __init__(self, icon: str, titulo: str, valor_inicial: str = "0", estilo: str = "azul", callback: Op):
050 |         super().__init__(parent)
051 |         e = ESTILOS_CARD.get(estilo, ESTILOS_CARD["azul"])
052 |         sombra_card(self)
053 |
054 |         self.setStyleSheet(
055 |             f"QFrame#cardFrame {{ border-radius: 12px; padding: 20px; {e['card']} }}"
056 |             f"QFrame#cardFrame:hover {{ border-color: {e['icon_color']}; }}"
057 |         )
058 |         self.setObjectName("cardFrame")
059 |         self.setMinimumHeight(130)
060 |
061 |         layout = QVBoxLayout(self)
062 |         layout.setContentsMargins(0, 0, 0, 0)
063 |         layout.setSpacing(12)
064 |
065 |         header = QHBoxLayout()
066 |         header.setSpacing(12)
067 |
068 |         icon_label = QLabel(icon)
069 |         icon_label.setFixedSize(48, 48)
070 |         icon_label.setAlignment(Qt.AlignCenter)
071 |         icon_label.setStyleSheet(
072 |             f"QLabel {{ {e['icon_bg']} border-radius: 12px; "
073 |             f"font-size: 22px; color: {e['icon_color']}; "
074 |             f"border: none; }}"
075 |         )
076 |         icon_label.setAttribute(Qt.WA_TransparentForMouseEvents)
077 |         header.addWidget(icon_label)
078 |         header.addStretch()
079 |         layout.addLayout(header)
080 |
081 |         titulo_label = QLabel(titulo)
082 |         titulo_label.setStyleSheet(
083 |             "font-size: 13px; color: #94949f; background: transparent; border: none;"

```

```

084 |         )
085 |         layout.addWidget(titulo_label)
086 |
087 |         self.valor_label = QLabel(valor_inicial)
088 |         self.valor_label.setStyleSheet(
089 |             f"font-size: 30px; font-weight: 700; color: #e8e8f0; "
090 |             f"background: transparent; border: none;"
091 |         )
092 |         layout.addWidget(self.valor_label)
093 |
094 |         layout.addStretch()
095 |
096 |         if callback:
097 |             self.setCursor(Qt.PointingHandCursor)
098 |             self.mousePressEvent = lambda event: callback()
099 |
100 |     def atualizar_valor(self, valor: Any) -> None:
101 |         self.valor_label.setText(str(valor))
102 |
103 |
104 | def _rgba(hex_color, alpha=1.0):
105 |     h = hex_color.lstrip("#")
106 |     r, g, b = int(h[0:2], 16), int(h[2:4], 16), int(h[4:6], 16)
107 |     return f"rgba({r},{g},{b},{alpha})"
108 |
109 |
110 | class CardMiniWidget(QFrame):
111 |     valor_label: QLabel
112 |
113 |     def __init__(self, icon: str, titulo: str, valor_inicial: str = "0", cor: str = "#6366f1", parent: Optional[QWidget] = None):
114 |         super().__init__(parent)
115 |         self.setStyleSheet(
116 |             f"QFrame#miniCard {{ background-color: rgba(20,20,31,0.5); "
117 |             f"border: 1px solid rgba(42,42,62,0.5); border-radius: 12px; padding: 16px; }}"
118 |         )
119 |         self.setObjectName("miniCard")
120 |         self.setMinimumHeight(90)
121 |
122 |         layout = QVBoxLayout(self)
123 |         layout.setContentsMargins(0, 0, 0, 0)
124 |         layout.setSpacing(8)
125 |
126 |         icon_label = QLabel(icon)
127 |         icon_label.setFixedSize(36, 36)
128 |         icon_label.setAlignment(Qt.AlignCenter)
129 |         icon_label.setStyleSheet(
130 |             f"background: {_rgba(cor, 0.15)}; border-radius: 8px; "
131 |             f"font-size: 16px; color: {cor}; border: none;"
132 |         )
133 |         icon_label.setAttribute(Qt.WA_TransparentForMouseEvents)
134 |         layout.addWidget(icon_label)
135 |
136 |         title_label = QLabel(titulo)
137 |         title_label.setStyleSheet(
138 |             "font-size: 11px; color: #94949f; background: transparent; border: none; font-weight: 500;"
139 |         )
140 |         title_label.setAttribute(Qt.WA_TransparentForMouseEvents)
141 |         layout.addWidget(title_label)
142 |
143 |         self.valor_label = QLabel(valor_inicial)
144 |         self.valor_label.setStyleSheet(
145 |             f"font-size: 24px; font-weight: 700; color: {cor}; "
146 |             f"background: transparent; border: none;"

```

```

147 |         )
148 |         self.valor_label.setAttribute(Qt.WA_TransparentForMouseEvents)
149 |         layout.addWidget(self.valor_label)
150 |
151 |     def atualizar_valor(self, valor: Any) -> None:
152 |         self.valor_label.setText(str(valor))
153 |
154 |
155 | class CardMiniClicavel(CardMiniWidget):
156 |     def __init__(self, icon: str, titulo: str, valor_inicial: str, cor: str, ao_clicar: Optional[Callable[[
157 |         super().__init__(icon, titulo, valor_inicial, cor, parent)
158 |         self.setCursor(Qt.PointingHandCursor)
159 |         self.setStyleSheet(
160 |             f"QFrame#miniCard {{ background-color: rgba(20,20,31,0.5); "
161 |             f"border: 1px solid rgba(42,42,62,0.5); border-radius: 12px; padding: 16px; }}"
162 |             f"QFrame#miniCard:hover {{ background-color: rgba(30,30,46,0.6); "
163 |             f"border: 1px solid rgba(42,42,62,0.8); border-radius: 12px; padding: 16px; }}"
164 |         )
165 |         if ao_clicar:
166 |             self.mousePressEvent = lambda event: ao_clicar()

```

=== ARQUIVO: .\app\views\widgets\chart_widget.py ===

```

001 | from __future__ import annotations
002 |
003 | from typing import Any, Optional
004 |
005 | from matplotlib.backends.backend_qtagg import FigureCanvasQTagg
006 | from matplotlib.figure import Figure
007 |
008 | from app.views.styles.theme import COR, CORES_GRAFICO
009 |
010 | FUNDO_GRAFICO = (20/255, 20/255, 31/255, 0.5)
011 |
012 |
013 | class ChartWidget(FigureCanvasQTagg):
014 |     fig: Figure
015 |     titulo: str
016 |     ax: Any
017 |
018 |     def __init__(self, titulo: str, width: int = 5, height: float = 3.2, dpi: int = 100) -> None:
019 |         self.fig = Figure(figsize=(width, height), dpi=dpi)
020 |         self.fig.patch.set_facecolor(FUNDO_GRAFICO)
021 |         super().__init__(self.fig)
022 |         self.titulo = titulo
023 |         self.ax = self.fig.add_subplot(111)
024 |         self.ax.set_facecolor(FUNDO_GRAFICO)
025 |         self._estilo_eixos()
026 |         self._draw_empty()
027 |
028 |     def _estilo_eixos(self) -> None:
029 |         self.ax.tick_params(colors=COR["texto_sec"], labelsz=9)
030 |         for spine in self.ax.spines.values():
031 |             spine.set_color(COR["borda"])
032 |             spine.set_linewidth(0.5)
033 |         self.ax.set_title(self.titulo, color=COR["texto"], fontsize=11, fontweight="bold", pad=12)
034 |
035 |     def _draw_empty(self) -> None:
036 |         self.ax.text(0.5, 0.5, "Sem dados", color=COR["texto_muted"], ha="center", va="center", fontsize=12)
037 |         self.ax.set_xlim(0, 1)
038 |         self.ax.set_ylim(0, 1)

```

```

039 |         self.draw()
040 |
041 |     def limpar(self) -> None:
042 |         self.ax.clear()
043 |         self.ax.set_facecolor(FUNDO_GRAFICO)
044 |         self._estilo_eixos()
045 |
046 |
047 | class PizzaChart(ChartWidget):
048 |     def __init__(self, *args: Any, **kwargs: Any) -> None:
049 |         super().__init__(*args, **kwargs)
050 |
051 |     def atualizar(self, labels: list[str], valores: list[float]) -> None:
052 |         self.limpar()
053 |         total = sum(valores)
054 |         if total == 0:
055 |             self._draw_empty()
056 |             return
057 |         wedges, texts, autotexts = self.ax.pie(
058 |             valores,
059 |             labels=labels,
060 |             autopct="%1.1f%%",
061 |             colors=CORES_GRAFICO[: len(labels)],
062 |             textprops={"color": COR["texto"], "fontsize": 9},
063 |             pctdistance=0.75,
064 |             wedgeprops={"linewidth": 1, "edgecolor": FUNDO_GRAFICO},
065 |             startangle=90,
066 |         )
067 |         for t in autotexts:
068 |             t.set_color("white")
069 |             t.set_fontweight("bold")
070 |         self.draw()
071 |
072 |
073 | class BarChart(ChartWidget):
074 |     def __init__(self, *args: Any, **kwargs: Any) -> None:
075 |         super().__init__(*args, **kwargs)
076 |
077 |     def atualizar(self, labels: list[str], valores: list[float]) -> None:
078 |         self.limpar()
079 |         if not valores:
080 |             self._draw_empty()
081 |             return
082 |         bars = self.ax.bar(labels, valores, color=CORES_GRAFICO[: len(valores)], edgecolor=None, linewidth=1)
083 |         self.ax.set_xlabel("")
084 |         for i, (bar, v) in enumerate(zip(bars, valores)):
085 |             self.ax.text(bar.get_x() + bar.get_width() / 2, bar.get_height() + 0.3, str(v),
086 |                           ha="center", va="bottom", color=COR["texto_sec"], fontsize=8)
087 |         self.fig.autofmt_xdate(rotation=30, ha="right")
088 |         self.draw()
089 |
090 |
091 | class LineChart(ChartWidget):
092 |     def __init__(self, *args: Any, **kwargs: Any) -> None:
093 |         super().__init__(*args, **kwargs)
094 |
095 |     def atualizar(self, labels: list[str], valores: list[float]) -> None:
096 |         self.limpar()
097 |         if not valores:
098 |             self._draw_empty()
099 |             return
100 |         self.ax.plot(labels, valores, color=COR["roxo"], marker="o", linewidth=2, markersize=4)
101 |         self.ax.fill_between(range(len(valores)), valores, alpha=0.1, color=COR["roxo"])

```

```

102 |         self.ax.set_xlabel("")
103 |         self.fig.autofmt_xdate(rotation=30, ha="right")
104 |         self.draw()

```

=== ARQUIVO: .\appp\views\widgets\confirm_dialog.py ===

```

001 | from __future__ import annotations
002 |
003 | from typing import Optional
004 |
005 | from PySide6.QtCore import Qt
006 | from PySide6.QtWidgets import (
007 |     QDialog,
008 |     QHBoxLayout,
009 |     QLabel,
010 |     QLineEdit,
011 |     QPushButton,
012 |     QVBoxLayout,
013 |     QFrame,
014 | )
015 |
016 | from app.views.styles.theme import (
017 |     COR,
018 |     ESTILO_BOTAO_ERRO,
019 |     ESTILO_BOTAO_SECUNDARIO,
020 |     ESTILO_INPUT,
021 | )
022 |
023 | _ESTILO_BOTAO_CONFIRMAR = ESTILO_BOTAO_ERRO + """"\
024 |     QPushButton:disabled {
025 |         background: #3a3a50; color: #717182;
026 |         border: 1px solid #2a2a3e; border-radius: 8px;
027 |     }
028 | """"
029 |
030 |
031 | class ConfirmacaoDigitarDialog(QDialog):
032 |     palavra_chave: str
033 |     confirmado: bool
034 |     input: QLineEdit
035 |     btn_cancelar: QPushButton
036 |     btn_confirmar: QPushButton
037 |
038 |     def __init__(
039 |         self,
040 |         titulo: str,
041 |         mensagem: str,
042 |         parent: Optional[QDialog] = None,
043 |         palavra_chave: str = "EXCLUIR",
044 |     ) -> None:
045 |         super().__init__(parent)
046 |         self.setWindowTitle(titulo)
047 |         self.setModal(True)
048 |         self.setMinimumWidth(420)
049 |         self.palavra_chave = palavra_chave.upper()
050 |         self.confirmado = False
051 |         self._setup_ui(mensagem)
052 |
053 |     def _setup_ui(self, mensagem: str) -> None:
054 |         self.setStyleSheet(
055 |             f"QDialog {{ background-color: {COR['fundo_card']}; }}"

```

```

056 |         )
057 |
058 |         layout = QVBoxLayout(self)
059 |         layout.setSpacing(16)
060 |         layout.setContentsMargins(24, 24, 24, 24)
061 |
062 |         header = QHBoxLayout()
063 |         icon = QLabel("\U000026A0\U0000FE0F")
064 |         icon.setStyleSheet("font-size: 28px; background: transparent;")
065 |         title = QLabel(self.windowTitle())
066 |         title.setStyleSheet(
067 |             f"font-size: 16px; font-weight: 700; color: {COR['erro']};"
068 |             " background: transparent;"
069 |         )
070 |         header.addWidget(icon)
071 |         header.addWidget(title, 1)
072 |         layout.addLayout(header)
073 |
074 |         msg = QLabel(mensagem)
075 |         msg.setWordWrap(True)
076 |         msg.setStyleSheet(
077 |             f"color: {COR['texto_sec']}; font-size: 12px;"
078 |             " background: transparent;"
079 |         )
080 |         layout.addWidget(msg)
081 |
082 |         sep = QFrame()
083 |         sep.setFrameShape(QFrame.HLine)
084 |         sep.setStyleSheet(
085 |             f"background: {COR['borda']}; max-height: 1px; border: none;"
086 |         )
087 |         layout.addWidget(sep)
088 |
089 |         instr = QLabel(
090 |             f'Digite <b style="color:{COR["erro"]};">{self.palavra_chave}</b>'
091 |             " para confirmar:"
092 |         )
093 |         instr.setWordWrap(True)
094 |         instr.setStyleSheet(
095 |             f"color: {COR['texto']}; font-size: 12px; background: transparent;"
096 |         )
097 |         layout.addWidget(instr)
098 |
099 |         self.input = QLineEdit()
100 |         self.input.setMaxLength(30)
101 |         self.input.setStyleSheet(ESTILO_INPUT)
102 |         self.input.setPlaceholderText(self.palavra_chave)
103 |         layout.addWidget(self.input)
104 |
105 |         botoes = QHBoxLayout()
106 |         botoes.addStretch()
107 |
108 |         self.btn_cancelar = QPushButton("Cancelar")
109 |         self.btn_cancelar.setStyleSheet(ESTILO_BOTAO_SECUNDARIO)
110 |         self.btn_cancelar.setCursor(Qt.PointingHandCursor)
111 |         self.btn_cancelar.clicked.connect(self.reject)
112 |
113 |         self.btn_confirmar = QPushButton("Confirmar")
114 |         self.btn_confirmar.setStyleSheet(_ESTILO_BOTAO_CONFIRMAR)
115 |         self.btn_confirmar.setCursor(Qt.PointingHandCursor)
116 |         self.btn_confirmar.setEnabled(False)
117 |         self.btn_confirmar.clicked.connect(self._confirmar)
118 |

```

```

119 |         botoes.addWidget(self.btn_cancelar)
120 |         botoes.addWidget(self.btn_confirmar)
121 |         layout.addLayout(botoes)
122 |
123 |         self.input.textChanged.connect(self._verificar)
124 |         self.input.returnPressed.connect(self._confirmar)
125 |
126 |     def _verificar(self, texto: str) -> None:
127 |         self.btn_confirmar.setEnabled(texto.strip().upper() == self.palavra_chave)
128 |
129 |     def _confirmar(self) -> None:
130 |         if not self.btn_confirmar.isEnabled():
131 |             return
132 |         self.confirmando = True
133 |         self.accept()
134 |
135 |     @classmethod
136 |     def confirmar(
137 |         cls,
138 |         titulo: str,
139 |         mensagem: str,
140 |         parent: Optional[QDialog] = None,
141 |         palavra_chave: str = "EXCLUIR",
142 |     ) -> bool:
143 |         dlg = cls(titulo, mensagem, parent, palavra_chave)
144 |         dlg.exec()
145 |         return dlg.confirmando

```

=== ARQUIVO: .\app\views\widgets\import_dialog.py ===

```

001 | from __future__ import annotations
002 |
003 | from typing import Any, Optional
004 |
005 | from PySide6.QtCore import Qt
006 | from PySide6.QtWidgets import (
007 |     QComboBox,
008 |     QDialog,
009 |     QFileDialog,
010 |     QFormLayout,
011 |     QHBoxLayout,
012 |     QHeaderView,
013 |     QLabel,
014 |     QLineEdit,
015 |     QMessageBox,
016 |     QProgressBar,
017 |     QPushButton,
018 |     QScrollArea,
019 |     QTableWidget,
020 |     QTableWidgetItem,
021 |     QVBoxLayout,
022 |     QWidget,
023 | )
024 |
025 | from app.utils.importer import (
026 |     CAMPOS_EDITAVEIS,
027 |     Importador,
028 |     ImportResult,
029 |     detectar_mapeamento,
030 |     parse_csv,
031 |     parse_xlsx,

```

```

032 | )
033 | from app.views.styles.theme import (
034 |     ESTILO_BOTAO_AVIS0,
035 |     ESTILO_BOTAO_ERRO,
036 |     ESTILO_BOTAO_FECHAR,
037 |     ESTILO_BOTAO_PRIMARIO,
038 |     ESTILO_BOTAO_SUCESSO,
039 |     ESTILO_DIALOG,
040 |     ESTILO_INPUT,
041 |     ESTILO_INPUT_READONLY,
042 |     ESTILO_TABELA_SIMPLES,
043 |     ESTILO_TITULO_PAGINA,
044 | )
045 | from app.views.widgets.table_widget import tornar_interativa
046 |
047 |
048 | class ImportDialog(QDialog):
049 |     _entity: str
050 |     _entity_nome: str
051 |     _service: Any
052 |     _session: Any
053 |     _caminho: str
054 |     _cabecalho: list[str]
055 |     _dados: list[list[str]]
056 |     _mapa_colunas: list[Optional[str]]
057 |     _combo_mapas: list[QComboBox]
058 |     _lbl_arquivo: QLineEdit
059 |     _lbl_encoding: QLabel
060 |     _combo_encoding: QComboBox
061 |     _lbl_delim: QLabel
062 |     _combo_delim: QComboBox
063 |     _lbl_sheet: QLabel
064 |     _combo_sheet: QComboBox
065 |     _lbl_total_linhas: QLabel
066 |     _preview_table: QTableWidget
067 |     _map_widget: QWidget
068 |     _map_layout: QVBoxLayout
069 |     _progress: QProgressBar
070 |     _lbl_status: QLabel
071 |     _btn_importar: QPushButton
072 |
073 |     def __init__(self, parent: Optional[QWidget], entity: str, entity_nome: str, service: Any, session: Any):
074 |         super().__init__(parent)
075 |         self._entity = entity
076 |         self._entity_nome = entity_nome
077 |         self._service = service
078 |         self._session = session
079 |         self._caminho = ""
080 |         self._cabecalho = []
081 |         self._dados = []
082 |         self._mapa_colunas = []
083 |         self._combo_mapas = []
084 |
085 |         self.setWindowTitle(f"Importar {entity_nome}")
086 |         self.setMinimumSize(800, 720)
087 |         self.setStyleSheet(ESTILO_DIALOG)
088 |
089 |         self._setup_ui()
090 |
091 |     def _setup_ui(self) -> None:
092 |         layout = QVBoxLayout(self)
093 |         layout.setSpacing(12)
094 |         layout.setContentsMargins(20, 20, 20, 20)

```



```

095 |
096 |     titulo = QLabel(f"■ Importar {self._entity_nome}")
097 |     titulo.setStyleSheet(ESTILO_TITULO_PAGINA)
098 |     layout.addWidget(titulo)
099 |
100 |     # File selection
101 |     file_layout = QHBoxLayout()
102 |     self._lbl_arquivo = QLineEdit()
103 |     self._lbl_arquivo.setReadOnly(True)
104 |     self._lbl_arquivo.setPlaceholderText("Selecione um arquivo CSV ou XLSX...")
105 |     self._lbl_arquivo.setStyleSheet(ESTILO_INPUT_READONLY)
106 |     file_layout.addWidget(self._lbl_arquivo)
107 |
108 |     btn_selecionar = QPushButton("■ Selecionar")
109 |     btn_selecionar.setStyleSheet(ESTILO_BOTA0_PRIMARIO)
110 |     btn_selecionar.clicked.connect(self._selecionar_arquivo)
111 |     file_layout.addWidget(btn_selecionar)
112 |     layout.addLayout(file_layout)
113 |
114 |     # Options (encoding, delimiter) - shown conditionally
115 |     opts_layout = QHBoxLayout()
116 |     opts_layout.setSpacing(8)
117 |
118 |     self._lbl_encoding = QLabel("Codificação:")
119 |     self._lbl_encoding.setStyleSheet("color: #c8c8d8; font-size: 12px; background: transparent;")
120 |     self._combo_encoding = QComboBox()
121 |     self._combo_encoding.addItem("UTF-8")
122 |     self._combo_encoding.addItem("ISO-8859-1")
123 |     self._combo_encoding.addItem("Windows-1252")
124 |     self._combo_encoding.addItem("UTF-16")
125 |     self._combo_encoding.setStyleSheet(self._estilo_combo_opt())
126 |     opts_layout.addWidget(self._lbl_encoding)
127 |     opts_layout.addWidget(self._combo_encoding)
128 |
129 |     self._lbl_delim = QLabel("Delimitador:")
130 |     self._lbl_delim.setStyleSheet("color: #c8c8d8; font-size: 12px; background: transparent;")
131 |     self._combo_delim = QComboBox()
132 |     self._combo_delim.addItem(";")
133 |     self._combo_delim.addItem(",")
134 |     self._combo_delim.addItem("|")
135 |     self._combo_delim.addItem("\t")
136 |     self._combo_delim.setStyleSheet(self._estilo_combo_opt())
137 |     opts_layout.addWidget(self._lbl_delim)
138 |     opts_layout.addWidget(self._combo_delim)
139 |
140 |     self._lbl_sheet = QLabel("Planilha:")
141 |     self._lbl_sheet.setStyleSheet("color: #c8c8d8; font-size: 12px; background: transparent;")
142 |     self._combo_sheet = QComboBox()
143 |     self._combo_sheet.setStyleSheet(self._estilo_combo_opt())
144 |     opts_layout.addWidget(self._lbl_sheet)
145 |     opts_layout.addWidget(self._combo_sheet)
146 |
147 |     opts_layout.addStretch()
148 |     btn_analisar = QPushButton("■ Analisar")
149 |     btn_analisar.setStyleSheet(ESTILO_BOTA0_AVISO)
150 |     btn_analisar.clicked.connect(self._analisar)
151 |     opts_layout.addWidget(btn_analisar)
152 |     layout.addLayout(opts_layout)
153 |
154 |     # Preview header
155 |     preview_header = QHBoxLayout()
156 |     lbl_preview = QLabel("Pré-visualização:")
157 |     lbl_preview.setStyleSheet("color: #94949f; font-size: 12px; background: transparent;")
158 |     preview_header.addWidget(lbl_preview)
159 |     preview_header.addStretch()
160 |     self._lbl_total_linhas = QLabel("")
161 |     self._lbl_total_linhas.setStyleSheet("color: #475569; font-size: 12px; background: transparent;")
162 |     preview_header.addWidget(self._lbl_total_linhas)
163 |     layout.addLayout(preview_header)

```

```

158 |
159 | # Preview table
160 | self._preview_table = QTableWidgetItem()
161 | self._preview_table.setStyleSheet(ESTILO_TABELA_SIMPLES)
162 | self._preview_table.setEditTriggers(QTableWidgetItem.NoEditTriggers)
163 | self._preview_table.setAlternatingRowColors(True)
164 | self._preview_table.verticalHeader().setVisible(False)
165 | self._preview_table.setMaximumHeight(450)
166 | layout.addWidget(self._preview_table)
167 |
168 | # Column mapping
169 | lbl_map = QLabel("Mapeamento de Colunas:")
170 | lbl_map.setStyleSheet("color: #94949f; font-size: 12px; background: transparent;")
171 | layout.addWidget(lbl_map)
172 |
173 | scroll = QScrollArea()
174 | scroll.setWidgetResizable(True)
175 | scroll.setMaximumHeight(140)
176 | scroll.setStyleSheet("QScrollArea { border: none; background: transparent; }")
177 |
178 | self._map_widget = QWidget()
179 | self._map_layout = QVBoxLayout(self._map_widget)
180 | self._map_layout.setContentsMargins(0, 0, 0, 0)
181 | self._map_layout.setSpacing(4)
182 | scroll.setWidget(self._map_widget)
183 | layout.addWidget(scroll)
184 |
185 | # Progress
186 | self._progress = QProgressBar()
187 | self._progress.setVisible(False)
188 | self._progress.setStyleSheet("""
189 |     QProgressBar { background-color: #1e1e2e; border: 1px solid #2a2a3e;
190 |                   border-radius: 6px; text-align: center; color: #e8e8f0;
191 |                   font-size: 11px; height: 20px; }
192 |     QProgressBar::chunk { background-color: #6366f1; border-radius: 5px; }
193 | """)
194 | layout.addWidget(self._progress)
195 |
196 | # Status
197 | self._lbl_status = QLabel("")
198 | self._lbl_status.setStyleSheet("color: #475569; font-size: 12px; background: transparent;")
199 | layout.addWidget(self._lbl_status)
200 |
201 | # Buttons
202 | btn_layout = QHBoxLayout()
203 | btn_layout.addStretch()
204 |
205 | self._btn_importar = QPushButton("■ Importar")
206 | self._btn_importar.setStyleSheet(ESTILO_BOTAO_SUCESSO)
207 | self._btn_importar.setEnabled(False)
208 | self._btn_importar.clicked.connect(self._importar)
209 | btn_layout.addWidget(self._btn_importar)
210 |
211 | btn_fechar = QPushButton("Fechar")
212 | btn_fechar.setStyleSheet(ESTILO_BOTAO_FECHAR)
213 | btn_fechar.clicked.connect(self.reject)
214 | btn_layout.addWidget(btn_fechar)
215 |
216 | layout.addLayout(btn_layout)
217 |
218 | # Initial visibility
219 | self._combo_sheet.setVisible(False)
220 | self._lbl_sheet.setVisible(False)

```

```

221 |
222 | def _estilo_combo_opt(self) -> str:
223 |     return (
224 |         "QComboBox { background-color: #1e1e2e; color: #e8e8f0;"
225 |         " border: 1px solid #2a2a3e; border-radius: 6px;"
226 |         " padding: 4px 8px; font-size: 11px; min-height: 20px; }"
227 |     )
228 |
229 | def _selecionar_arquivo(self) -> None:
230 |     path, _ = QFileDialog.getOpenFileName(
231 |         self, "Selecionar Arquivo", "",
232 |         "Arquivos Suportados (*.csv *.xlsx);;CSV (*.csv);;Excel (*.xlsx);;Todos (*.*)",
233 |     )
234 |     if not path:
235 |         return
236 |     self._caminho = path
237 |     self._lbl_arquivo.setText(path)
238 |     is_csv = path.lower().endswith(".csv")
239 |     self._lbl_encoding.setVisible(is_csv)
240 |     self._combo_encoding.setVisible(is_csv)
241 |     self._lbl_delim.setVisible(is_csv)
242 |     self._combo_delim.setVisible(is_csv)
243 |     self._lbl_sheet.setVisible(not is_csv)
244 |     self._combo_sheet.setVisible(not is_csv)
245 |     self._combo_sheet.clear()
246 |     self._btn_importar.setEnabled(False)
247 |     self._preview_table.setRowCount(0)
248 |     self._preview_table.setColumnCount(0)
249 |
250 |     if not is_csv:
251 |         try:
252 |             from openpyxl import load_workbook
253 |             wb = load_workbook(path, read_only=True)
254 |             self._combo_sheet.addItem(wb.sheetnames)
255 |             wb.close()
256 |         except Exception as e:
257 |             QMessageBox.warning(self, "Aviso", f"Não foi possível ler o arquivo:\n{e}")
258 |
259 | def _analisar(self) -> None:
260 |     if not self._caminho:
261 |         QMessageBox.warning(self, "Aviso", "Selecione um arquivo primeiro.")
262 |         return
263 |
264 |     try:
265 |         is_csv = self._caminho.lower().endswith(".csv")
266 |         if is_csv:
267 |             encoding = self._combo_encoding.currentText()
268 |             delim = self._combo_delim.currentText()
269 |             if delim == "\\t":
270 |                 delim = "\t"
271 |             cabecalho, dados = parse_csv(self._caminho, encoding, delim)
272 |         else:
273 |             sheet = self._combo_sheet.currentText() or None
274 |             cabecalho, dados = parse_xlsx(self._caminho, sheet)
275 |
276 |         if not cabecalho:
277 |             QMessageBox.warning(self, "Aviso", "Arquivo vazio ou cabeçalho não encontrado.")
278 |             return
279 |
280 |         self._cabecalho = cabecalho
281 |         self._dados = dados
282 |         total = len(dados)
283 |         self._lbl_total_linhas.setText(f"{total} linha(s) de dados")

```

```

284 |
285 |     # Auto-detect mapping
286 |     mapa = detectar_mapeamento(cabecalho, self._entity)
287 |     self._mapa_colunas = [mapa.get(i) for i in range(len(cabecalho))]
288 |
289 |     # Preview
290 |     self._preview_table.setColumnCount(len(cabecalho))
291 |     self._preview_table.setHorizontalHeaderLabels(cabecalho)
292 |     n_preview = len(dados)
293 |     self._preview_table.setRowCount(n_preview)
294 |     for i in range(n_preview):
295 |         for j in range(len(cabecalho)):
296 |             val = dados[i][j] if j < len(dados[i]) else ""
297 |             item = QTableWidgetItem(val)
298 |             item.setForeground(Qt.gray if not val else Qt.white)
299 |             self._preview_table.setItem(i, j, item)
300 |     tornar_interativa(self._preview_table)
301 |
302 |     # Build mapping combos
303 |     self._construir_mapeamento(cabecalho)
304 |
305 |     self._btn_importar.setEnabled(True)
306 |     self._lbl_status.setText("■ Arquivo analisado. Verifique o mapeamento e clique em Importar.")
307 |     self._lbl_status.setStyleSheet("color: #34d399; font-size: 12px; background: transparent;")
308 |
309 | except Exception as e:
310 |     QMessageBox.critical(self, "Erro", f"Erro ao analisar arquivo:\n{e}")
311 |     self._lbl_status.setText(f"■ Erro: {e}")
312 |     self._lbl_status.setStyleSheet("color: #f87171; font-size: 12px; background: transparent;")
313 |
314 | def _construir_mapeamento(self, cabecalho: list[str]) -> None:
315 |     # Clear old mapping widgets
316 |     while self._map_layout.count():
317 |         item = self._map_layout.takeAt(0)
318 |         if item.widget():
319 |             item.widget().deleteLater()
320 |     self._combo_mapas.clear()
321 |
322 |     campos = CAMPOS_EDITAVEIS.get(self._entity, [])
323 |     opcoes = ["■ Ignorar ■"] + campos
324 |
325 |     for i, nome_col in enumerate(cabecalho):
326 |         row = QHBoxLayout()
327 |         row.setSpacing(8)
328 |
329 |         label = QLabel(f"{nome_col} →")
330 |         label.setStyleSheet("color: #c8c8d8; font-size: 12px; background: transparent; min-width: 120px")
331 |         row.addWidget(label)
332 |
333 |         combo = QComboBox()
334 |         combo.addItem(opcoes)
335 |         combo.setStyleSheet(
336 |             "QComboBox { background-color: #1e1e2e; color: #e8e8f0;"
337 |             " border: 1px solid #2a2a3e; border-radius: 6px;"
338 |             " padding: 4px 8px; font-size: 11px; min-height: 22px; }"
339 |         )
340 |         mapped = self._mapa_colunas[i] if i < len(self._mapa_colunas) else None
341 |         if mapped and mapped in campos:
342 |             idx = opcoes.index(mapped)
343 |             combo.setCurrentIndex(idx)
344 |         combo.currentIndexChanged.connect(self._atualizar_mapa)
345 |         row.addWidget(combo)
346 |         self._combo_mapas.append(combo)

```

```

347 |
348 |         self._map_layout.addLayout(row)
349 |
350 | def _atualizar_mapa(self) -> None:
351 |     for i, combo in enumerate(self._combo_mapas):
352 |         texto = combo.currentText()
353 |         if texto == "■ Ignorar ■":
354 |             self._mapa_colunas[i] = None
355 |         else:
356 |             self._mapa_colunas[i] = texto
357 |
358 | def _importar(self) -> None:
359 |     if not self._dados:
360 |         return
361 |
362 |     self._btn_importar.setEnabled(False)
363 |     self._progress.setVisible(True)
364 |     self._progress.setMaximum(0)
365 |     self._progress.setValue(0)
366 |     self._lbl_status.setText("Importando...")
367 |     self._lbl_status.setStyleSheet("color: #fbbf24; font-size: 12px; background: transparent;")
368 |
369 |     importador = Importador(self._entity, self._service, self._session)
370 |     resultado = importador.importar_linhas(
371 |         self._dados, self._mapa_colunas, self._cabecalho
372 |     )
373 |
374 |     self._progress.setMaximum(100)
375 |     self._progress.setValue(100)
376 |
377 |     # Show result
378 |     if resultado.erros:
379 |         msg_erros = "\n".join(
380 |             f" Linha {num}: {err}" for num, err in resultado.erros[:20]
381 |         )
382 |         if len(resultado.erros) > 20:
383 |             msg_erros += f"\n ... e mais {len(resultado.erros) - 20} erro(s)"
384 |         msg = (
385 |             f"■ {resultado.importados} de {resultado.total} registros importados.\n\n"
386 |             f"■ {len(resultado.erros)} erro(s):\n{msg_erros}"
387 |         )
388 |         self._lbl_status.setText(f"■ {resultado.importados} importados, {len(resultado.erros)} erro(s).")
389 |         self._lbl_status.setStyleSheet("color: #fbbf24; font-size: 12px; background: transparent;")
390 |         QMessageBox.warning(self, "Resultado da Importação", msg)
391 |     else:
392 |         self._lbl_status.setText(f"■ {resultado.importados} registros importados com sucesso!")
393 |         self._lbl_status.setStyleSheet("color: #34d399; font-size: 12px; background: transparent;")
394 |         QMessageBox.information(
395 |             self, "Importação Concluída",
396 |             f"{resultado.importados} registro(s) importado(s) com sucesso!"
397 |         )
398 |
399 |     self._progress.setVisible(False)
400 |     self._btn_importar.setEnabled(True)
401 |
402 |     if resultado.importados > 0:
403 |         self.accept()

```

=== ARQUIVO: .laplviews\widgets\loading_overlay.py ===

```

001 | from __future__ import annotations

```

```

002 |
003 | from typing import Optional
004 |
005 | from PySide6.QtCore import Qt, QTimer
006 | from PySide6.QtGui import QColor, QPainter, QResizeEvent
007 | from PySide6.QtWidgets import QLabel, QVBoxLayout, QWidget
008 |
009 |
010 | class SpinnerWidget(QWidget):
011 |     _angulo: int
012 |
013 |     def __init__(self, parent: Optional[QWidget] = None) -> None:
014 |         super().__init__(parent)
015 |         self.setFixedSize(40, 40)
016 |         self._angulo = 0
017 |
018 |     def paintEvent(self, event: QResizeEvent) -> None:
019 |         p = QPainter(self)
020 |         p.setRenderHint(QPainter.Antialiasing)
021 |         rect = self.rect().adjusted(3, 3, -3, -3)
022 |         p.setPen(Qt.NoPen)
023 |         p.setBrush(QColor("#2e2e4a"))
024 |         p.drawPie(rect, 0, 360 * 16)
025 |         p.setBrush(QColor("#a78bfa"))
026 |         p.drawPie(rect, self._angulo * 16, 90 * 16)
027 |         p.end()
028 |
029 |
030 | class LoadingOverlay(QWidget):
031 |     spinner: SpinnerWidget
032 |     label: QLabel
033 |     _timer: QTimer
034 |
035 |     def __init__(self, parent: Optional[QWidget] = None, text: str = "Carregando...") -> None:
036 |         super().__init__(parent)
037 |         self.setAttribute(Qt.WA_TransparentForMouseEvents, False)
038 |         self.setStyleSheet("background-color: rgba(10, 10, 18, 180);")
039 |         self.hide()
040 |
041 |         layout = QVBoxLayout(self)
042 |         layout.setAlignment(Qt.AlignCenter)
043 |
044 |         self.spinner = SpinnerWidget()
045 |         layout.addWidget(self.spinner, alignment=Qt.AlignCenter)
046 |
047 |         self.label = QLabel(text)
048 |         self.label.setAlignment(Qt.AlignCenter)
049 |         self.label.setStyleSheet(
050 |             "color: #a78bfa; font-size: 15px; font-weight: 600;"
051 |             " background: transparent; padding: 8px;"
052 |         )
053 |         layout.addWidget(self.label)
054 |
055 |         self._timer = QTimer(self)
056 |         self._timer.timeout.connect(self._rotacionar)
057 |
058 |     def _rotacionar(self) -> None:
059 |         self.spinner._angulo = (self.spinner._angulo + 30) % 360
060 |         self.spinner.update()
061 |
062 |     def mostrar(self, text: Optional[str] = None) -> None:
063 |         if text:
064 |             self.label.setText(text)

```

```

065 |         self.setFixedSize(self.parent().size())
066 |         self.raise_()
067 |         self._timer.start(50)
068 |         self.show()
069 |
070 |     def ocultar(self) -> None:
071 |         self._timer.stop()
072 |         self.hide()
073 |
074 |     def resizeEvent(self, event: QResizeEvent) -> None:
075 |         if self.parent():
076 |             self.setFixedSize(self.parent().size())
077 |             super().resizeEvent(event)

```

=== ARQUIVO: .lapp\views\widgets\pagination.py ===

```

001 | from __future__ import annotations
002 |
003 | from typing import Optional
004 |
005 | from PySide6.QtCore import Qt, Signal
006 | from PySide6.QtWidgets import (
007 |     QComboBox, QHBoxLayout, QLabel, QPushButton, QWidget,
008 | )
009 |
010 |
011 | class PaginacaoWidget(QWidget):
012 |     pagina_alterada = Signal(int)
013 |
014 |     _pagina_atual: int
015 |     _total_paginas: int
016 |     _total_registros: int
017 |     _itens_por_pagina: int
018 |     _btn_anterior: QPushButton
019 |     _label_info: QLabel
020 |     _btn_proximo: QPushButton
021 |     _combo_itens: QComboBox
022 |
023 |     def __init__(self, parent: Optional[QWidget] = None) -> None:
024 |         super().__init__(parent)
025 |         self._pagina_atual = 1
026 |         self._total_paginas = 1
027 |         self._total_registros = 0
028 |         self._itens_por_pagina = 50
029 |
030 |         layout = QHBoxLayout(self)
031 |         layout.setContentsMargins(0, 4, 0, 4)
032 |         layout.setSpacing(8)
033 |
034 |         self._btn_anterior = QPushButton("« Anterior")
035 |         self._btn_anterior.setStyleSheet(self._estilo_botao())
036 |         self._btn_anterior.clicked.connect(self._anterior)
037 |
038 |         self._label_info = QLabel("Página 1 de 1")
039 |         self._label_info.setStyleSheet("color: #94949f; font-size: 12px; background: transparent;")
040 |
041 |         self._btn_proximo = QPushButton("Próximo »")
042 |         self._btn_proximo.setStyleSheet(self._estilo_botao())
043 |         self._btn_proximo.clicked.connect(self._proximo)
044 |
045 |         self._combo_itens = QComboBox()

```

```

046 |         self._combo_itens.addItem("20", "50", "100", "200"])
047 |         self._combo_itens.setCurrentText("50")
048 |         self._combo_itens.currentTextChanged.connect(self._itens_por_pagina_changed)
049 |         self._combo_itens.setStyleSheet(
050 |             "QComboBox { background-color: #1e1e2e; color: #e8e8f0;"
051 |             " border: 1px solid #2a2a3e; border-radius: 6px;"
052 |             " padding: 4px 8px; font-size: 11px; min-height: 20px; }"
053 |         )
054 |
055 |         layout.addWidget(self._btn_anterior)
056 |         layout.addWidget(self._label_info)
057 |         layout.addWidget(self._btn_proximo)
058 |         layout.addStretch()
059 |         lbl = QLabel("Itens/página:")
060 |         lbl.setStyleSheet("color: #717182; font-size: 11px; background: transparent;")
061 |         layout.addWidget(lbl)
062 |         layout.addWidget(self._combo_itens)
063 |
064 |         self._atualizar_botoes()
065 |
066 |     def configurar(self, total_registros: int, pagina_atual: int = 1, itens_por_pagina: int = 50) -> None:
067 |         self._total_registros = total_registros
068 |         self._pagina_atual = pagina_atual
069 |         self._itens_por_pagina = itens_por_pagina
070 |         self._total_paginas = max(1, (total_registros + itens_por_pagina - 1) // itens_por_pagina)
071 |         self._combo_itens.blockSignals(True)
072 |         self._combo_itens.setCurrentText(str(itens_por_pagina))
073 |         self._combo_itens.blockSignals(False)
074 |         self._atualizar_label()
075 |         self._atualizar_botoes()
076 |
077 |     def _anterior(self) -> None:
078 |         if self._pagina_atual > 1:
079 |             self._pagina_atual -= 1
080 |             self._atualizar_label()
081 |             self._atualizar_botoes()
082 |             self.pagina_alterada.emit(self._pagina_atual)
083 |
084 |     def _proximo(self) -> None:
085 |         if self._pagina_atual < self._total_paginas:
086 |             self._pagina_atual += 1
087 |             self._atualizar_label()
088 |             self._atualizar_botoes()
089 |             self.pagina_alterada.emit(self._pagina_atual)
090 |
091 |     def _itens_por_pagina_changed(self, valor: str) -> None:
092 |         self._itens_por_pagina = int(valor)
093 |         self._total_paginas = max(1, (self._total_registros + self._itens_por_pagina - 1) // self._itens_po
094 |         self._pagina_atual = 1
095 |         self._atualizar_label()
096 |         self._atualizar_botoes()
097 |         self.pagina_alterada.emit(self._pagina_atual)
098 |
099 |     def _atualizar_label(self) -> None:
100 |         texto = f"Página {self._pagina_atual} de {self._total_paginas}"
101 |         if self._total_registros > 0:
102 |             texto += f" ({self._total_registros} registro{'s' if self._total_registros != 1 else ''})"
103 |         self._label_info.setText(texto)
104 |
105 |     def _atualizar_botoes(self) -> None:
106 |         self._btn_anterior.setEnabled(self._pagina_atual > 1)
107 |         self._btn_proximo.setEnabled(self._pagina_atual < self._total_paginas)
108 |

```



```

109 |     def _estilo_botao(self) -> str:
110 |         return (
111 |             "QPushButton { background-color: #1e1e2e; color: #c8c8d8;"
112 |             " border: 1px solid #2a2a3e; border-radius: 6px;"
113 |             " padding: 6px 14px; font-size: 11px; font-weight: 600; }"
114 |             "QPushButton:hover { background-color: #2a2a3e; color: #e8e8f0; border-color: #6366f1; }"
115 |             "QPushButton:disabled { color: #3a3a50; border-color: #1e1e2e; }"
116 |         )
117 |
118 |     @property
119 |     def offset(self) -> int:
120 |         return (self._pagina_atual - 1) * self._itens_por_pagina
121 |
122 |     @property
123 |     def limit(self) -> int:
124 |         return self._itens_por_pagina
125 |
126 |     @property
127 |     def pagina(self) -> int:
128 |         return self._pagina_atual

```

=== ARQUIVO: .\app\views\widgets\search_bar.py ===

```

001 | from __future__ import annotations
002 |
003 | from PySide6.QtCore import Qt
004 | from PySide6.QtWidgets import QFrame, QHBoxLayout, QLabel, QLineEdit
005 |
006 |
007 | class SearchBar(QFrame):
008 |     _glass: bool
009 |     _padrao: str
010 |     _borda: str
011 |     input: QLineEdit
012 |
013 |     def __init__(self, placeholder: str = "Buscar...", glass: bool = False, parent: Optional[QFrame] = None):
014 |         super().__init__(parent)
015 |         self._glass = glass
016 |         self._padrao = "rgba(20,20,31,0.5)" if glass else "#1e1e2e"
017 |         self._borda = "rgba(42,42,62,0.5)" if glass else "#2a2a3e"
018 |
019 |         self.setStyleSheet(
020 |             f"QFrame#searchFrame {{ background: {self._padrao}; "
021 |             f"border: 1px solid {self._borda}; border-radius: 10px; }}"
022 |         )
023 |         self.setObjectName("searchFrame")
024 |         min_h = 48 if glass else 40
025 |         self.setMinimumHeight(min_h)
026 |         self.setMaximumHeight(min_h)
027 |
028 |         layout = QHBoxLayout(self)
029 |         layout.setContentsMargins(14, 0, 14, 0)
030 |         layout.setSpacing(8)
031 |
032 |         icon = QLabel("\U0001F50D")
033 |         icon.setStyleSheet("font-size: 14px; background: transparent; color: #94949f;")
034 |         layout.addWidget(icon)
035 |
036 |         self.input = QLineEdit()
037 |         self.input.setPlaceholderText(placeholder)
038 |         self.input.setStyleSheet(

```

```

039 |         "QLineEdit { background: transparent; color: #e8e8f0; "
040 |         "border: none; font-size: 11pt; }"
041 |         "QLineEdit::placeholder { color: #717182; }"
042 |     )
043 |     layout.addWidget(self.input)
044 |
045 |     def textChanged(self) -> Any:
046 |         return self.input.textChanged
047 |
048 |     def texto(self) -> str:
049 |         return self.input.text()
050 |
051 |     def limpar(self) -> None:
052 |         self.input.clear()

```

=== ARQUIVO: .\app\views\widgets\table_widget.py ===

```

001 | from __future__ import annotations
002 |
003 | from typing import Optional
004 |
005 | from PySide6.QtCore import Qt, QEvent, QObject
006 | from PySide6.QtGui import QColor, QMouseEvent
007 | from PySide6.QtWidgets import QAbstractItemView, QHeaderView, QSizePolicy, QTableWidgetItem
008 |
009 | from app.views.styles.theme import ESTILO_TABELA
010 | from app.views.widgets.badge_widget import BadgeWidget
011 |
012 |
013 | class _InteractiveHeaderView(QHeaderView):
014 |     """Header que começa com Stretch e troca pra Interactive no primeiro arrasto."""
015 |
016 |     def __init__(self, parent=None):
017 |         super().__init__(Qt.Horizontal, parent)
018 |         self._user_resized = False
019 |
020 |     def resetar(self) -> None:
021 |         self._user_resized = False
022 |         for i in range(self.count()):
023 |             self.setSectionResizeMode(i, QHeaderView.Stretch)
024 |
025 |     def mousePressEvent(self, event: QMouseEvent) -> None:
026 |         if not self._user_resized and event.button() == Qt.LeftButton:
027 |             idx = self.logicalIndexAt(event.pos())
028 |             if idx >= 0:
029 |                 self._user_resized = True
030 |                 for i in range(self.count()):
031 |                     self.setSectionResizeMode(i, QHeaderView.Interactive)
032 |                 super().mousePressEvent(event)
033 |
034 |
035 | class TabelaPadrao(QTableWidget):
036 |     def __init__(self, colunas: list[str], parent: Optional[QTableWidget] = None) -> None:
037 |         super().__init__(parent)
038 |         self.setColumnCount(len(colunas))
039 |         self.setHorizontalHeaderLabels(colunas)
040 |         self.setStyleSheet(ESTILO_TABELA)
041 |         header = _InteractiveHeaderView()
042 |         header.setDefaultSectionSize(120)
043 |         for i in range(len(colunas)):
044 |             header.setSectionResizeMode(i, QHeaderView.Stretch)

```

```

045 |         self.setHorizontalHeader(header)
046 |         self.setSelectionBehavior(QAbstractItemView.SelectRows)
047 |         self.setEditTriggers(QAbstractItemView.NoEditTriggers)
048 |         self.verticalHeader().setVisible(False)
049 |         self.setShowGrid(False)
050 |         self.setAlternatingRowColors(True)
051 |         self.setSizePolicy(QSizePolicy.Expanding, QSizePolicy.Expanding)
052 |
053 |     def showEvent(self, event) -> None:
054 |         super().showEvent(event)
055 |         header = self.horizontalHeader()
056 |         if isinstance(header, _InteractiveHeaderView):
057 |             header.resetar()
058 |
059 |     def redimensionar(self) -> None:
060 |         self.verticalHeader().setDefaultSectionSize(44)
061 |
062 |     def item_colorido(self, texto: str, cor_hex: str) -> QTableWidgetItem:
063 |         item = QTableWidgetItem(texto)
064 |         item.setForeground(QColor(cor_hex))
065 |         return item
066 |
067 |     def item_centralizado(self, texto: str) -> QTableWidgetItem:
068 |         item = QTableWidgetItem(texto)
069 |         item.setTextAlignment(Qt.AlignCenter)
070 |         return item
071 |
072 |     def badge_item(self, texto: str, cor: str) -> BadgeWidget:
073 |         badge = BadgeWidget(texto, cor)
074 |         return badge
075 |
076 |     def definir_badge(self, row: int, col: int, texto: str, cor: str) -> None:
077 |         badge = BadgeWidget(texto, cor)
078 |         self.setCellWidget(row, col, badge)
079 |
080 |     def limpar(self) -> None:
081 |         self.setRowCount(0)
082 |
083 |
084 | def tornar_interativa(tabela: QTableWidgetItem) -> None:
085 |     """Converte qualquer QTableWidgetItem para ter colunas arrastáveis com reset ao mostrar a tela."""
086 |     header = _InteractiveHeaderView()
087 |     header.setDefaultSectionSize(120)
088 |     n = tabela.columnCount()
089 |     for i in range(n):
090 |         header.setSectionResizeMode(i, QHeaderView.Stretch)
091 |     tabela.setHorizontalHeader(header)
092 |     original_show = tabela.showEvent
093 |
094 |     def _show_event(event):
095 |         original_show(event)
096 |         header.resetar()
097 |
098 |     tabela.showEvent = _show_event

```

=== ARQUIVO: .lapp\views\widgets\toast.py ===

```

001 | from __future__ import annotations
002 |
003 | from collections.abc import Callable
004 | from typing import Any, ClassVar, Optional

```

```

005 |
006 | from PySide6.QtCore import QEasingCurve, QPoint, QPropertyAnimation, Qt, QTimer
007 | from PySide6.QtGui import QResizeEvent
008 | from PySide6.QtWidgets import QFrame, QHBoxLayout, QLabel, QPushButton, QWidget
009 |
010 |
011 | class ToastManager:
012 |     _container: ClassVar[Optional[_ToastContainer]] = None
013 |
014 |     @classmethod
015 |     def instalar(cls, parent: QWidget) -> None:
016 |         if cls._container is None:
017 |             cls._container = _ToastContainer(parent)
018 |
019 |     @classmethod
020 |     def mostrar(cls, mensagem: str, tipo: str = "info", duracao: int = 4000, persistente: bool = False, acao: Optional[Callable[[], Any]] = None) -> None:
021 |         if cls._container:
022 |             cls._container.adicionar(mensagem, tipo, duracao, persistente, acao)
023 |
024 |     @classmethod
025 |     def info(cls, mensagem: str, persistente: bool = False, acao: Optional[tuple[str, Callable[[], Any]]] = None) -> None:
026 |         cls.mostrar(mensagem, "info", persistente=persistente, acao=acao)
027 |
028 |     @classmethod
029 |     def sucesso(cls, mensagem: str, persistente: bool = False, acao: Optional[tuple[str, Callable[[], Any]]] = None) -> None:
030 |         cls.mostrar(mensagem, "sucesso", persistente=persistente, acao=acao)
031 |
032 |     @classmethod
033 |     def aviso(cls, mensagem: str, persistente: bool = False, acao: Optional[tuple[str, Callable[[], Any]]] = None) -> None:
034 |         cls.mostrar(mensagem, "aviso", persistente=persistente, acao=acao)
035 |
036 |     @classmethod
037 |     def erro(cls, mensagem: str, persistente: bool = False, acao: Optional[tuple[str, Callable[[], Any]]] = None) -> None:
038 |         cls.mostrar(mensagem, "erro", persistente=persistente, acao=acao)
039 |
040 |     @classmethod
041 |     def limpar_persistentes(cls) -> None:
042 |         if cls._container:
043 |             cls._container._limpar_persistentes()
044 |
045 |     @classmethod
046 |     def atualizar_status(cls, pendentes: int) -> None:
047 |         cls.limpar_persistentes()
048 |         if pendentes:
049 |             cls.aviso(f"{pendentes} alerta(s) pendente(s)", persistente=True)
050 |         else:
051 |             cls.sucesso("Nenhum alerta pendente")
052 |
053 |
054 | CORES_TOAST = {
055 |     "info": {"bg": "#1e1e3a", "borda": "#a78bfa", "icone": "■"},
056 |     "sucesso": {"bg": "#0a2e1a", "borda": "#34d399", "icone": "✓"},
057 |     "aviso": {"bg": "#2e2a0a", "borda": "#fbbf24", "icone": "■"},
058 |     "erro": {"bg": "#2e0a0a", "borda": "#f87171", "icone": "X"},
059 | }
060 |
061 |
062 | class _ToastContainer(QWidget):
063 |     _toasts: list[_Toast]
064 |     _margem: int
065 |     _espaco: int
066 |
067 |     def __init__(self, parent: QWidget) -> None:

```

```

068 |         super().__init__(parent)
069 |         self.setStyleSheet("background: transparent;")
070 |         self._toasts = []
071 |         self._margem = 20
072 |         self._espaco = 8
073 |         self.setFixedWidth(360)
074 |         self._reposicionar_container()
075 |         self.show()
076 |
077 |     def _reposicionar_container(self) -> None:
078 |         p = self.parent()
079 |         if p:
080 |             self.move(p.width() - self.width() - self._margem, 10)
081 |             self.raise_()
082 |
083 |     def adicionar(self, mensagem: str, tipo: str, duracao: int, persistente: bool = False, acao: Optional[tuple[str, Callable[[], Any]]] = None) -> None:
084 |         toast = _Toast(self, mensagem, tipo, duracao, persistente, acao)
085 |         self._toasts.append(toast)
086 |         toast.mostrar()
087 |         self._reposicionar_toasts()
088 |
089 |     def remover(self, toast: _Toast) -> None:
090 |         if toast in self._toasts:
091 |             self._toasts.remove(toast)
092 |             toast.deleteLater()
093 |             self._reposicionar_toasts()
094 |
095 |     def _limpar_persistentes(self) -> None:
096 |         for t in list(self._toasts):
097 |             if t._persistente:
098 |                 self._toasts.remove(t)
099 |                 t.deleteLater()
100 |
101 |     def _reposicionar_toasts(self) -> None:
102 |         y = 0
103 |         for t in self._toasts:
104 |             if t.isVisible():
105 |                 t.move(0, y)
106 |                 y += t.height() + self._espaco
107 |         self.setFixedHeight(y if y > 0 else 0)
108 |
109 |     def resizeEvent(self, event: QResizeEvent) -> None:
110 |         super().resizeEvent(event)
111 |         self._reposicionar_container()
112 |
113 |
114 | class _Toast(QFrame):
115 |     _container: _ToastContainer
116 |     _duracao: int
117 |     _persistente: bool
118 |     _acao: Optional[tuple[str, Callable[[], Any]]]
119 |     _anim_entrada: Optional[QPropertyAnimation]
120 |     _anim_saida: Optional[QPropertyAnimation]
121 |     _timer: QTimer
122 |
123 |     def __init__(self, container: _ToastContainer, mensagem: str, tipo: str, duracao: int, persistente: bool = False, acao: Optional[tuple[str, Callable[[], Any]]] = None) -> None:
124 |         super().__init__(container)
125 |         self._container = container
126 |         self._duracao = duracao
127 |         self._persistente = persistente
128 |         cor = CORES_TOAST.get(tipo, CORES_TOAST["info"])
129 |
130 |         self._acao = acao

```

```

131 |
132 |         if acao:
133 |             self.setFixedWidth(400)
134 |         else:
135 |             self.setFixedWidth(340)
136 |         self.setStyleSheet(f"""
137 |             _Toast {{
138 |                 background-color: {cor['bg']};
139 |                 border: 1px solid {cor['borda']};
140 |                 border-radius: 10px;
141 |             }}
142 |         """)
143 |
144 |         layout = QHBoxLayout(self)
145 |         layout.setContentsMargins(12, 10, 12, 10)
146 |         layout.setSpacing(10)
147 |
148 |         lbl_icone = QLabel(cor["icone"])
149 |         lbl_icone.setStyleSheet(f"font-size: 16px; color: {cor['borda']}; background: transparent;")
150 |         lbl_icone.setFixedWidth(20)
151 |         layout.addWidget(lbl_icone)
152 |
153 |         lbl_msg = QLabel(mensagem)
154 |         lbl_msg.setWordWrap(True)
155 |         lbl_msg.setStyleSheet("color: #e2e8f0; font-size: 12px; background: transparent;")
156 |         layout.addWidget(lbl_msg, 1)
157 |
158 |         if acao:
159 |             texto, callback = acao
160 |             btn_acao = QPushButton(texto)
161 |             btn_acao.setFixedHeight(28)
162 |             btn_acao.setCursor(Qt.PointingHandCursor)
163 |             btn_acao.setStyleSheet(
164 |                 "QPushButton { background: rgba(99,102,241,0.15); color: #818cf8;"
165 |                 " border: 1px solid rgba(99,102,241,0.3); border-radius: 6px;"
166 |                 " font-size: 11px; font-weight: 700; padding: 0 12px; }"
167 |                 " QPushButton: hover { background: rgba(99,102,241,0.25); color: #a5b4fc; }"
168 |             )
169 |             btn_acao.clicked.connect(lambda: (callback(), self.fechar()))
170 |             layout.addWidget(btn_acao)
171 |
172 |         btn_fechar = QPushButton("X")
173 |         btn_fechar.setFixedSize(24, 24)
174 |         btn_fechar.setCursor(Qt.PointingHandCursor)
175 |         btn_fechar.setStyleSheet(
176 |             "QPushButton { background: rgba(255,255,255,0.08); color: #94a3b8; border: none; border-radius:"
177 |             " QPushButton: hover { background: rgba(248,113,113,0.3); color: #f87171; }"
178 |         )
179 |         btn_fechar.clicked.connect(self.fechar)
180 |         layout.addWidget(btn_fechar)
181 |
182 |         self._anim_entrada = None
183 |         self._anim_saida = None
184 |         self._timer = QTimer(self)
185 |         self._timer.setSingleShot(True)
186 |         self._timer.timeout.connect(self.fechar)
187 |
188 |     def mostrar(self) -> None:
189 |         self.adjustSize()
190 |         self._container._reposicionar_container()
191 |
192 |         idx = len(self._container._toasts) - 1
193 |         y = idx * (self.height() + self._container._espaco)

```

```

194 |         self.move(360, y)
195 |         self.show()
196 |
197 |         self._anim_entrada = QPropertyAnimation(self, b"pos")
198 |         self._anim_entrada.setDuration(300)
199 |         self._anim_entrada.setEasingCurve(QEasingCurve.OutCubic)
200 |         self._anim_entrada.setStartValue(QPoint(360, y))
201 |         self._anim_entrada.setEndValue(QPoint(0, y))
202 |         self._anim_entrada.start()
203 |
204 |         self._container.setFixedHeight(y + self.height())
205 |
206 |         if not self._persistente:
207 |             self._timer.start(self._duracao)
208 |
209 |     def fechar(self) -> None:
210 |         self._timer.stop()
211 |         self._anim_saida = QPropertyAnimation(self, b"pos")
212 |         self._anim_saida.setDuration(250)
213 |         self._anim_saida.setEasingCurve(QEasingCurve.InCubic)
214 |         self._anim_saida.setStartValue(self.pos())
215 |         self._anim_saida.setEndValue(QPoint(360, self.pos().y()))
216 |         self._anim_saida.finished.connect(self._finalizar)
217 |         self._anim_saida.start()
218 |
219 |     def _finalizar(self) -> None:
220 |         self._container.remove(self)

```

=== ARQUIVO: .\app\views\widgets__init__.py ===

```

001 | from __future__ import annotations
002 |
003 | from .badge_widget import BadgeWidget
004 | from .card_widget import CardWidget
005 | from .chart_widget import BarChart, LineChart, PizzaChart
006 | from .confirm_dialog import ConfirmacaoDigitarDialog
007 | from .loading_overlay import LoadingOverlay
008 | from .pagination import PaginacaoWidget
009 | from .search_bar import SearchBar
010 | from .table_widget import TabelaPadrao
011 | from .toast import ToastManager

```

=== ARQUIVO: .\dist\ControleImpressoras__internal\alembic\env.py ===

```

001 | import sys
002 | from logging.config import fileConfig
003 | from pathlib import Path
004 |
005 | from alembic import context
006 | from sqlalchemy import pool
007 |
008 | sys.path.insert(0, str(Path(__file__).parent.parent))
009 |
010 | config = context.config
011 |
012 | if config.config_file_name is not None:
013 |     fileConfig(config.config_file_name)
014 |
015 | from app.models.base import Base

```

```

016 | from db import ENGINE
017 |
018 | target_metadata = Base.metadata
019 |
020 |
021 | def run_migrations_offline() -> None:
022 |     url = config.get_main_option("sqlalchemy.url")
023 |     context.configure(
024 |         url=url,
025 |         target_metadata=target_metadata,
026 |         literal_binds=True,
027 |         dialect_opts={"paramstyle": "named"},
028 |         render_as_batch=True,
029 |     )
030 |     with context.begin_transaction():
031 |         context.run_migrations()
032 |
033 |
034 | def run_migrations_online() -> None:
035 |     with ENGINE.connect() as connection:
036 |         context.configure(
037 |             connection=connection,
038 |             target_metadata=target_metadata,
039 |             render_as_batch=True,
040 |         )
041 |         with context.begin_transaction():
042 |             context.run_migrations()
043 |
044 |
045 | if context.is_offline_mode():
046 |     run_migrations_offline()
047 | else:
048 |     run_migrations_online()

```

=== ARQUIVO: .\dist\ControleImpressoras\internal\alembic\versions\01c2af25a419_add_inicio_fim_atendimento.py ===

```

001 | """add_inicio_fim_atendimento
002 |
003 | Revision ID: 01c2af25a419
004 | Revises: 370e4ecbc5dc
005 | Create Date: 2026-06-05 18:08:33.665588
006 |
007 | """
008 | from typing import Sequence, Union
009 |
010 | from alembic import op
011 | import sqlalchemy as sa
012 |
013 |
014 | revision: str = '01c2af25a419'
015 | down_revision: Union[str, Sequence[str], None] = '370e4ecbc5dc'
016 | branch_labels: Union[str, Sequence[str], None] = None
017 | depends_on: Union[str, Sequence[str], None] = None
018 |
019 |
020 | def upgrade() -> None:
021 |     with op.batch_alter_table('activities', schema=None) as batch_op:
022 |         batch_op.add_column(sa.Column('inicio_atendimento', sa.DateTime(), nullable=True))
023 |         batch_op.add_column(sa.Column('fim_atendimento', sa.DateTime(), nullable=True))

```



```

024 |
025 |
026 | def downgrade() -> None:
027 |     with op.batch_alter_table('activities', schema=None) as batch_op:
028 |         batch_op.drop_column('inicio_atendimento')
029 |         batch_op.drop_column('fim_atendimento')

```

=== ARQUIVO: .\dist\ControleImpressoras\internal\alembic\versions\0df5fd127c31_add_notificado_fields_to_alert.py ===

```

001 | """add_notificado_fields_to_alert
002 |
003 | Revision ID: 0df5fd127c31
004 | Revises: 43f9d2337797
005 | Create Date: 2026-06-02 16:47:18.668008
006 |
007 | """
008 | from typing import Sequence, Union
009 |
010 | from alembic import op
011 | import sqlalchemy as sa
012 |
013 |
014 | revision: str = '0df5fd127c31'
015 | down_revision: Union[str, None] = '43f9d2337797'
016 | branch_labels: Union[str, Sequence[str], None] = None
017 | depends_on: Union[str, Sequence[str], None] = None
018 |
019 |
020 | def upgrade() -> None:
021 |     with op.batch_alter_table('alerts') as batch_op:
022 |         batch_op.add_column(sa.Column('notificado', sa.Boolean(), nullable=False, server_default=sa.text('0')))
023 |         batch_op.add_column(sa.Column('notificado_em', sa.DateTime(), nullable=True))
024 |
025 |
026 | def downgrade() -> None:
027 |     with op.batch_alter_table('alerts') as batch_op:
028 |         batch_op.drop_column('notificado_em')
029 |         batch_op.drop_column('notificado')

```

=== ARQUIVO: .\dist\ControleImpressoras\internal\alembic\versions\1a2b3c4d5e6f_add_ultima_revisao_to_printers.py ===

```

001 | """add ultima_revisao to printers
002 |
003 | Revision ID: 1a2b3c4d5e6f
004 | Revises: 01c2af25a419
005 | Create Date: 2026-06-09 18:15:00.000000
006 |
007 | """
008 | from typing import Sequence, Union
009 |
010 | from alembic import op
011 | import sqlalchemy as sa
012 |
013 |
014 | revision: str = '1a2b3c4d5e6f'

```

```

015 | down_revision: Union[str, Sequence[str], None] = '01c2af25a419'
016 | branch_labels: Union[str, Sequence[str], None] = None
017 | depends_on: Union[str, Sequence[str], None] = None
018 |
019 |
020 | def upgrade() -> None:
021 |     with op.batch_alter_table('printers', schema=None) as batch_op:
022 |         batch_op.add_column(sa.Column('ultima_revisao', sa.DateTime(), nullable=True))
023 |
024 |
025 | def downgrade() -> None:
026 |     with op.batch_alter_table('printers', schema=None) as batch_op:
027 |         batch_op.drop_column('ultima_revisao')

```

=== ARQUIVO: .\dist\ControleImpressoras\internal\alembic\versions\2b3c4d5e6f7a_add_urgencia_to_printers.py ===

```

001 | """add_urgencia_prox_manutencao to printers
002 |
003 | Revision ID: 2b3c4d5e6f7a
004 | Revises: 1a2b3c4d5e6f
005 | Create Date: 2026-06-09 18:30:00.000000
006 |
007 | """
008 | from typing import Sequence, Union
009 |
010 | from alembic import op
011 | import sqlalchemy as sa
012 |
013 |
014 | revision: str = '2b3c4d5e6f7a'
015 | down_revision: Union[str, Sequence[str], None] = '1a2b3c4d5e6f'
016 | branch_labels: Union[str, Sequence[str], None] = None
017 | depends_on: Union[str, Sequence[str], None] = None
018 |
019 |
020 | def upgrade() -> None:
021 |     with op.batch_alter_table('printers', schema=None) as batch_op:
022 |         batch_op.add_column(sa.Column('urgencia_prox_manutencao', sa.String(20), nullable=False, server_def
023 |
024 |
025 | def downgrade() -> None:
026 |     with op.batch_alter_table('printers', schema=None) as batch_op:
027 |         batch_op.drop_column('urgencia_prox_manutencao')

```

=== ARQUIVO: .\dist\ControleImpressoras\internal\alembic\versions\370e4ecbc5dc_add_diagnostico_solucao_to_activities.py ===

```

001 | """add_diagnostico_solucao_to_activities
002 |
003 | Revision ID: 370e4ecbc5dc
004 | Revises: e2ea9a0e5059
005 | Create Date: 2026-06-05 18:04:38.775529
006 |
007 | """
008 | from typing import Sequence, Union
009 |

```

```

010 | from alembic import op
011 | import sqlalchemy as sa
012 |
013 |
014 | revision: str = '370e4ecbc5dc'
015 | down_revision: Union[str, Sequence[str], None] = 'e2ea9a0e5059'
016 | branch_labels: Union[str, Sequence[str], None] = None
017 | depends_on: Union[str, Sequence[str], None] = None
018 |
019 |
020 | def upgrade() -> None:
021 |     with op.batch_alter_table('activities', schema=None) as batch_op:
022 |         batch_op.add_column(sa.Column('diagnostico_tecnico', sa.Text(), nullable=False, server_default=""))
023 |         batch_op.add_column(sa.Column('solucao_aplicada', sa.Text(), nullable=False, server_default=""))
024 |
025 |
026 | def downgrade() -> None:
027 |     with op.batch_alter_table('activities', schema=None) as batch_op:
028 |         batch_op.drop_column('diagnostico_tecnico')
029 |         batch_op.drop_column('solucao_aplicada')

```

=== ARQUIVO: .\dist\ControleImpressoras\internal\alembic\versions\3c4d5e6f7a8b_add_urgencia_to_activities.py ===

```

001 | """add urgencia to activities
002 |
003 | Revision ID: 3c4d5e6f7a8b
004 | Revises: 2b3c4d5e6f7a
005 | Create Date: 2026-06-09 19:00:00.000000
006 |
007 | """
008 | from typing import Sequence, Union
009 |
010 | from alembic import op
011 | import sqlalchemy as sa
012 |
013 |
014 | revision: str = '3c4d5e6f7a8b'
015 | down_revision: Union[str, Sequence[str], None] = '2b3c4d5e6f7a'
016 | branch_labels: Union[str, Sequence[str], None] = None
017 | depends_on: Union[str, Sequence[str], None] = None
018 |
019 |
020 | def upgrade() -> None:
021 |     with op.batch_alter_table('activities', schema=None) as batch_op:
022 |         batch_op.add_column(sa.Column('urgencia', sa.String(20), nullable=False, server_default='Normal'))
023 |
024 |
025 | def downgrade() -> None:
026 |     with op.batch_alter_table('activities', schema=None) as batch_op:
027 |         batch_op.drop_column('urgencia')

```

=== ARQUIVO: .\dist\ControleImpressoras\internal\alembic\versions\3df1b9c20c20_add_procedimentos_to_activities.py ===

```

001 | """add_procedimentos_to_activities
002 |

```

```

003 | Revision ID: 3df1b9c20c20
004 | Revises: 4113ad29b4c6
005 | Create Date: 2026-06-05 17:15:07.351949
006 |
007 | """
008 | from typing import Sequence, Union
009 |
010 | from alembic import op
011 | import sqlalchemy as sa
012 |
013 |
014 | # revision identifiers, used by Alembic.
015 | revision: str = '3df1b9c20c20'
016 | down_revision: Union[str, Sequence[str], None] = '4113ad29b4c6'
017 | branch_labels: Union[str, Sequence[str], None] = None
018 | depends_on: Union[str, Sequence[str], None] = None
019 |
020 |
021 | def upgrade() -> None:
022 |     with op.batch_alter_table('activities', schema=None) as batch_op:
023 |         batch_op.add_column(sa.Column('procedimentos', sa.Text(), nullable=False, server_default=""))
024 |
025 |
026 | def downgrade() -> None:
027 |     with op.batch_alter_table('activities', schema=None) as batch_op:
028 |         batch_op.drop_column('procedimentos')

```

=== ARQUIVO: .\dist\ControleImpressoras\internal\alembic\versions\4113ad29b4c6_add_responsavel_to_activities.py ===

```

001 | """add_responsavel_to_activities
002 |
003 | Revision ID: 4113ad29b4c6
004 | Revises: b2c3d4e5f6a7
005 | Create Date: 2026-06-05 16:20:56.050336
006 |
007 | """
008 | from typing import Sequence, Union
009 |
010 | from alembic import op
011 | import sqlalchemy as sa
012 |
013 |
014 | # revision identifiers, used by Alembic.
015 | revision: str = '4113ad29b4c6'
016 | down_revision: Union[str, Sequence[str], None] = 'b2c3d4e5f6a7'
017 | branch_labels: Union[str, Sequence[str], None] = None
018 | depends_on: Union[str, Sequence[str], None] = None
019 |
020 |
021 | def upgrade() -> None:
022 |     with op.batch_alter_table('activities', schema=None) as batch_op:
023 |         batch_op.add_column(sa.Column('responsavel', sa.String(length=120), nullable=False, server_default=
024 |
025 |
026 | def downgrade() -> None:
027 |     with op.batch_alter_table('activities', schema=None) as batch_op:
028 |         batch_op.drop_column('responsavel')

```

=== ARQUIVO: .\dist\ControleImpressoras\internal\alembic\versions\43f9d2337797_add_deleted_at_soft_delete.py ===

```
001 | """add deleted_at columns for soft delete
002 |
003 | Revision ID: 43f9d2337797
004 | Revises: 860330a5ed5a
005 | Create Date: 2026-06-02 16:02:04.880542
006 |
007 | """
008 | from typing import Sequence, Union
009 |
010 | from alembic import op
011 | import sqlalchemy as sa
012 |
013 |
014 | revision: str = '43f9d2337797'
015 | down_revision: Union[str, Sequence[str], None] = '860330a5ed5a'
016 | branch_labels: Union[str, Sequence[str], None] = None
017 | depends_on: Union[str, Sequence[str], None] = None
018 |
019 |
020 | TABELAS = ["printers", "activities", "companies", "parts", "technicians", "alerts", "transfers", "users"]
021 |
022 |
023 | def upgrade() -> None:
024 |     for tabela in TABELAS:
025 |         with op.batch_alter_table(tabela, schema=None) as batch_op:
026 |             batch_op.add_column(sa.Column("deleted_at", sa.DateTime(), nullable=True))
027 |
028 |
029 | def downgrade() -> None:
030 |     for tabela in TABELAS:
031 |         with op.batch_alter_table(tabela, schema=None) as batch_op:
032 |             batch_op.drop_column("deleted_at")
```

=== ARQUIVO: .\dist\ControleImpressoras\internal\alembic\versions\7899a6741eeb_add_part_movements_reservations_requisitions.py ===

```
001 | """add part_movements, part_reservations, purchase_requisitions
002 |
003 | Revision ID: 7899a6741eeb
004 | Revises: f6a7b8c9d0e1
005 | Create Date: 2026-06-15 14:00:00.000000
006 |
007 | """
008 | from typing import Sequence, Union
009 |
010 | from alembic import op
011 | import sqlalchemy as sa
012 |
013 |
014 | revision: str = '7899a6741eeb'
015 | down_revision: Union[str, Sequence[str], None] = 'f6a7b8c9d0e1'
016 | branch_labels: Union[str, Sequence[str], None] = None
017 | depends_on: Union[str, Sequence[str], None] = None
018 |
019 |
020 | def upgrade() -> None:
```

```

021 |     op.create_table('part_movements',
022 |         sa.Column('id', sa.Integer(), autoincrement=True, nullable=False),
023 |         sa.Column('part_id', sa.Integer(), sa.ForeignKey('parts.id'), nullable=False, index=True),
024 |         sa.Column('activity_id', sa.Integer(), sa.ForeignKey('activities.id'), nullable=True, index=True),
025 |         sa.Column('tipo', sa.String(length=30), nullable=False),
026 |         sa.Column('quantidade', sa.Integer(), nullable=False),
027 |         sa.Column('saldo_anterior', sa.Integer(), nullable=False, server_default='0'),
028 |         sa.Column('saldo_posterior', sa.Integer(), nullable=False, server_default='0'),
029 |         sa.Column('observacao', sa.Text(), nullable=False, server_default=''),
030 |         sa.Column('created_at', sa.DateTime(), nullable=True),
031 |         sa.PrimaryKeyConstraint('id'),
032 |     )
033 |
034 |     op.create_table('part_reservations',
035 |         sa.Column('id', sa.Integer(), autoincrement=True, nullable=False),
036 |         sa.Column('part_id', sa.Integer(), sa.ForeignKey('parts.id'), nullable=False, index=True),
037 |         sa.Column('activity_id', sa.Integer(), sa.ForeignKey('activities.id'), nullable=False, index=True),
038 |         sa.Column('quantidade', sa.Integer(), nullable=False, server_default='1'),
039 |         sa.Column('status', sa.String(length=20), nullable=False, server_default='reservada'),
040 |         sa.Column('created_at', sa.DateTime(), nullable=True),
041 |         sa.Column('updated_at', sa.DateTime(), nullable=True),
042 |         sa.PrimaryKeyConstraint('id'),
043 |     )
044 |
045 |     op.create_table('purchase_requisitions',
046 |         sa.Column('id', sa.Integer(), autoincrement=True, nullable=False),
047 |         sa.Column('part_id', sa.Integer(), sa.ForeignKey('parts.id'), nullable=False, index=True),
048 |         sa.Column('quantidade_sugerida', sa.Integer(), nullable=False, server_default='1'),
049 |         sa.Column('status', sa.String(length=20), nullable=False, server_default='pendente'),
050 |         sa.Column('observacao', sa.Text(), nullable=False, server_default=''),
051 |         sa.Column('created_at', sa.DateTime(), nullable=True),
052 |         sa.Column('updated_at', sa.DateTime(), nullable=True),
053 |         sa.PrimaryKeyConstraint('id'),
054 |     )
055 |
056 |
057 | def downgrade() -> None:
058 |     op.drop_table('purchase_requisitions')
059 |     op.drop_table('part_reservations')
060 |     op.drop_table('part_movements')

```

**=== ARQUIVO: .\dist\ControleImpressoras\internal\alembic\versions\860330a5e
d5a_sync_models.py ===**

```

001 | """sync models with DB
002 |
003 | Revision ID: 860330a5ed5a
004 | Revises: d46315348779
005 | Create Date: 2026-06-01 11:24:43.623550
006 |
007 | """
008 | from typing import Sequence, Union
009 |
010 | from alembic import op
011 | import sqlalchemy as sa
012 |
013 |
014 | revision: str = '860330a5ed5a'
015 | down_revision: Union[str, Sequence[str], None] = 'd46315348779'
016 | branch_labels: Union[str, Sequence[str], None] = None

```

```

017 | depends_on: Union[str, Sequence[str], None] = None
018 |
019 |
020 | def upgrade() -> None:
021 |     op.execute("PRAGMA foreign_keys=OFF")
022 |
023 |     # Fix index names to match model __table_args__
024 |     op.drop_index('ix_parts_codigo', table_name='parts')
025 |     op.create_index('ix_part_codigo', 'parts', ['codigo'], unique=False)
026 |
027 |     op.drop_index('ix_companies_cnpj', table_name='companies')
028 |     op.create_index('ix_company_cnpj', 'companies', ['cnpj'], unique=False)
029 |
030 |     op.drop_index('ix_transfers_numero_os', table_name='transfers')
031 |     op.create_index('ix_transfer_numero_os', 'transfers', ['numero_os'], unique=False)
032 |
033 |     op.drop_index('ix_transfers_tipo', table_name='transfers')
034 |     op.create_index('ix_transfer_tipo', 'transfers', ['tipo'], unique=False)
035 |
036 |     op.drop_index('ix_users_username', table_name='users')
037 |     op.create_index('ix_user_username', 'users', ['username'], unique=False)
038 |
039 |     # Add missing alert index
040 |     op.create_index('ix_alerts_printer_id', 'alerts', ['printer_id'], unique=False)
041 |
042 |     # Cleanup leftover temp table from failed migration
043 |     op.execute("DROP TABLE IF EXISTS _alembic_tmp_alerts")
044 |
045 |     # Users: drop empresa_id, add email unique constraint
046 |     with op.batch_alter_table('users', schema=None) as batch_op:
047 |         batch_op.drop_column('empresa_id')
048 |         batch_op.create_unique_constraint('uq_users_email', ['email'])
049 |
050 |     # Printers: tipo VARCHAR(50)->80, add patrimony unique, drop old columns
051 |     with op.batch_alter_table('printers', schema=None) as batch_op:
052 |         batch_op.alter_column('patrimonio',
053 |             existing_type=sa.VARCHAR(length=50),
054 |             type_=sa.String(length=80),
055 |             existing_nullable=False)
056 |         batch_op.create_unique_constraint('uq_printers_patrimonio', ['patrimonio'])
057 |         batch_op.drop_column('tecnico_revisao')
058 |         batch_op.drop_column('ultima_revisao_data')
059 |
060 |     # Alerts: titulo VARCHAR(150)->200
061 |     with op.batch_alter_table('alerts', schema=None) as batch_op:
062 |         batch_op.alter_column('titulo',
063 |             existing_type=sa.VARCHAR(length=150),
064 |             type_=sa.String(length=200),
065 |             existing_nullable=False)
066 |
067 |     op.execute("PRAGMA foreign_keys=ON")
068 |
069 |
070 | def downgrade() -> None:
071 |     op.execute("PRAGMA foreign_keys=OFF")
072 |
073 |     with op.batch_alter_table('alerts', schema=None) as batch_op:
074 |         batch_op.alter_column('titulo',
075 |             existing_type=sa.String(length=200),
076 |             type_=sa.VARCHAR(length=150),
077 |             existing_nullable=False)
078 |
079 |     with op.batch_alter_table('printers', schema=None) as batch_op:

```

```

080 |         batch_op.add_column(sa.Column('ultima_revisao_data', sa.DATETIME(), nullable=True))
081 |         batch_op.add_column(sa.Column('tecnico_revisao', sa.TEXT(), nullable=True))
082 |         batch_op.drop_constraint('uq_printers_patrimonio', type_='unique')
083 |         batch_op.alter_column('patrimonio',
084 |             existing_type=sa.String(length=80),
085 |             type_=sa.VARCHAR(length=50),
086 |             existing_nullable=False)
087 |
088 |     with op.batch_alter_table('users', schema=None) as batch_op:
089 |         batch_op.drop_constraint('uq_users_email', type_='unique')
090 |         batch_op.add_column(sa.Column('empresa_id', sa.INTEGER(), nullable=True))
091 |
092 |     op.drop_index('ix_alerts_printer_id', table_name='alerts')
093 |     op.drop_index('ix_user_username', table_name='users')
094 |     op.create_index('ix_users_username', 'users', ['username'], unique=False)
095 |     op.drop_index('ix_transfer_tipo', table_name='transfers')
096 |     op.create_index('ix_transfers_tipo', 'transfers', ['tipo'], unique=False)
097 |     op.drop_index('ix_transfer_numero_os', table_name='transfers')
098 |     op.create_index('ix_transfers_numero_os', 'transfers', ['numero_os'], unique=False)
099 |     op.drop_index('ix_company_cnpj', table_name='companies')
100 |     op.create_index('ix_companies_cnpj', 'companies', ['cnpj'], unique=False)
101 |     op.drop_index('ix_part_codigo', table_name='parts')
102 |     op.create_index('ix_parts_codigo', 'parts', ['codigo'], unique=False)
103 |
104 |     op.execute("PRAGMA foreign_keys=ON")

```

=== ARQUIVO: .\dist\ControleImpressoras\internal\alembic\versions\8a7b6c5d4e3f_add_activity_id_to_printer_locations.py ===

```

001 | """Add activity_id to printer_locations
002 |
003 | Revision ID: 8a7b6c5d4e3f
004 | Revises: 7899a6741eeb
005 | Create Date: 2026-07-01 12:00:00.000000
006 |
007 | """
008 | from typing import Sequence, Union
009 |
010 | from alembic import op
011 | import sqlalchemy as sa
012 |
013 |
014 | revision: str = '8a7b6c5d4e3f'
015 | down_revision: Union[str, None] = '7899a6741eeb'
016 | branch_labels: Union[str, Sequence[str], None] = None
017 | depends_on: Union[str, Sequence[str], None] = None
018 |
019 |
020 | def upgrade() -> None:
021 |     with op.batch_alter_table('printer_locations', schema=None) as batch_op:
022 |         batch_op.add_column(sa.Column('activity_id', sa.Integer(), nullable=True))
023 |         batch_op.create_foreign_key('fk_printer_locations_activity', 'activities', ['activity_id'], ['id'])
024 |
025 |
026 | def downgrade() -> None:
027 |     with op.batch_alter_table('printer_locations', schema=None) as batch_op:
028 |         batch_op.drop_constraint('fk_printer_locations_activity', type_='foreignkey')
029 |         batch_op.drop_column('activity_id')

```


=== ARQUIVO: .\dist\ControleImpressoras\internal\alembic\versions\9a3b1c2d4e5f_add_part_id_data_agendada_to_alerts.py ===

```
001 | """add_part_id_data_agendada_to_alerts
002 |
003 | Revision ID: 9a3b1c2d4e5f
004 | Revises: 0df5fd127c31
005 | Create Date: 2026-06-03 11:30:00.000000
006 |
007 | """
008 | from typing import Sequence, Union
009 |
010 | from alembic import op
011 | import sqlalchemy as sa
012 |
013 |
014 | revision: str = '9a3b1c2d4e5f'
015 | down_revision: Union[str, None] = '0df5fd127c31'
016 | branch_labels: Union[str, Sequence[str], None] = None
017 | depends_on: Union[str, Sequence[str], None] = None
018 |
019 |
020 | def upgrade() -> None:
021 |     with op.batch_alter_table('alerts') as batch_op:
022 |         batch_op.alter_column('printer_id', existing_type=sa.String(36), nullable=True)
023 |         batch_op.add_column(sa.Column('part_id', sa.Integer(), nullable=True))
024 |         batch_op.add_column(sa.Column('data_agendada', sa.DateTime(), nullable=True))
025 |         batch_op.create_index('ix_alert_part_id', ['part_id'])
026 |         batch_op.create_foreign_key('fk_alert_part_id', 'parts', ['part_id'], ['id'])
027 |
028 |
029 | def downgrade() -> None:
030 |     with op.batch_alter_table('alerts') as batch_op:
031 |         batch_op.drop_constraint('fk_alert_part_id', type_='foreignkey')
032 |         batch_op.drop_index('ix_alert_part_id')
033 |         batch_op.drop_column('data_agendada')
034 |         batch_op.drop_column('part_id')
035 |         batch_op.alter_column('printer_id', existing_type=sa.String(36), existing_nullable=True, nullable=False)
```

=== ARQUIVO: .\dist\ControleImpressoras\internal\alembic\versions\1a1b2c3d4e5f6_add_foto_path_to_printers.py ===

```
001 | """add_foto_path_to_printers
002 |
003 | Revision ID: 1a1b2c3d4e5f6
004 | Revises: 9a3b1c2d4e5f
005 | Create Date: 2026-06-04 10:30:00.000000
006 |
007 | """
008 | from typing import Sequence, Union
009 |
010 | from alembic import op
011 | import sqlalchemy as sa
012 |
013 |
014 | revision: str = '1a1b2c3d4e5f6'
015 | down_revision: Union[str, None] = '9a3b1c2d4e5f'
016 | branch_labels: Union[str, Sequence[str], None] = None
017 | depends_on: Union[str, Sequence[str], None] = None
```

```

018 |
019 |
020 | def upgrade() -> None:
021 |     with op.batch_alter_table('printers') as batch_op:
022 |         batch_op.add_column(sa.Column('foto_path', sa.String(255), nullable=True))
023 |
024 |
025 | def downgrade() -> None:
026 |     with op.batch_alter_table('printers') as batch_op:
027 |         batch_op.drop_column('foto_path')

```

=== ARQUIVO: .\dist\ControleImpressoras\internal\alembic\versions\la44fc2747c28_add_performance_indexes.py ===

```

001 | """add performance indexes
002 |
003 | Revision ID: a44fc2747c28
004 | Revises: 8a7b6c5d4e3f
005 | Create Date: 2026-07-10 12:00:00.000000
006 |
007 | """
008 | from typing import Sequence, Union
009 |
010 | from alembic import op
011 | import sqlalchemy as sa
012 |
013 |
014 | revision: str = 'a44fc2747c28'
015 | down_revision: Union[str, Sequence[str], None] = '8a7b6c5d4e3f'
016 | branch_labels: Union[str, Sequence[str], None] = None
017 | depends_on: Union[str, Sequence[str], None] = None
018 |
019 |
020 | def upgrade() -> None:
021 |     # --- printers ---
022 |     with op.batch_alter_table("printers", schema=None) as batch_op:
023 |         batch_op.create_index("ix_printers_status", ["status"])
024 |         batch_op.create_index("ix_printers_deleted_at", ["deleted_at"])
025 |
026 |     # --- activities ---
027 |     with op.batch_alter_table("activities", schema=None) as batch_op:
028 |         batch_op.create_index("ix_activities_deleted_at", ["deleted_at"])
029 |         batch_op.create_index("ix_activities_event_at", ["event_at"])
030 |         batch_op.create_index("ix_activities_tecnico_id", ["tecnico_id"])
031 |         batch_op.create_index("ix_activities_tecnico_id_event_at", ["tecnico_id", "event_at"])
032 |         batch_op.create_index("ix_activities_printer_id_event_at", ["printer_id", "event_at"])
033 |         batch_op.create_index("ix_activities_status_event_at", ["status_atividade", "event_at"])
034 |
035 |     # --- alerts ---
036 |     with op.batch_alter_table("alerts", schema=None) as batch_op:
037 |         batch_op.create_index("ix_alerts_deleted_at", ["deleted_at"])
038 |         batch_op.create_index("ix_alerts_resolvido", ["resolvido"])
039 |         batch_op.create_index("ix_alerts_resolvido_created_at", ["resolvido", "created_at"])
040 |         batch_op.create_index("ix_alerts_tipo_resolvido", ["tipo", "resolvido"])
041 |
042 |     # --- companies ---
043 |     with op.batch_alter_table("companies", schema=None) as batch_op:
044 |         batch_op.create_index("ix_companies_deleted_at", ["deleted_at"])
045 |
046 |     # --- parts ---

```

```

047 | with op.batch_alter_table("parts", schema=None) as batch_op:
048 |     batch_op.create_index("ix_parts_deleted_at", ["deleted_at"])
049 |     batch_op.create_index("ix_parts_nome", ["nome"])
050 |
051 | # --- technicians ---
052 | with op.batch_alter_table("technicians", schema=None) as batch_op:
053 |     batch_op.create_index("ix_technicians_deleted_at", ["deleted_at"])
054 |
055 | # --- users ---
056 | with op.batch_alter_table("users", schema=None) as batch_op:
057 |     batch_op.create_index("ix_users_deleted_at", ["deleted_at"])
058 |
059 | # --- FK columns sem indice ---
060 | with op.batch_alter_table("login_history", schema=None) as batch_op:
061 |     batch_op.create_index("ix_login_history_user_id", ["user_id"])
062 |
063 | with op.batch_alter_table("technician_specialties", schema=None) as batch_op:
064 |     batch_op.create_index("ix_technician_specialties_technician_id", ["technician_id"])
065 |
066 | with op.batch_alter_table("printer_locations", schema=None) as batch_op:
067 |     batch_op.create_index("ix_printer_locations_activity_id", ["activity_id"])
068 |
069 | with op.batch_alter_table("attachments", schema=None) as batch_op:
070 |     batch_op.create_index("ix_attachments_uploader_id", ["uploader_id"])
071 |
072 | with op.batch_alter_table("audit_log", schema=None) as batch_op:
073 |     batch_op.create_index("ix_audit_log_user_id", ["user_id"])
074 |     batch_op.create_index("ix_audit_log_created_at", ["created_at"])
075 |     batch_op.create_index("ix_audit_log_tabela_alvo", ["tabela_alvo"])
076 |
077 |
078 | def downgrade() -> None:
079 |     with op.batch_alter_table("printers", schema=None) as batch_op:
080 |         batch_op.drop_index("ix_printers_status")
081 |         batch_op.drop_index("ix_printers_deleted_at")
082 |
083 |     with op.batch_alter_table("activities", schema=None) as batch_op:
084 |         batch_op.drop_index("ix_activities_deleted_at")
085 |         batch_op.drop_index("ix_activities_event_at")
086 |         batch_op.drop_index("ix_activities_tecnico_id")
087 |         batch_op.drop_index("ix_activities_tecnico_id_event_at")
088 |         batch_op.drop_index("ix_activities_printer_id_event_at")
089 |         batch_op.drop_index("ix_activities_status_event_at")
090 |
091 |     with op.batch_alter_table("alerts", schema=None) as batch_op:
092 |         batch_op.drop_index("ix_alerts_deleted_at")
093 |         batch_op.drop_index("ix_alerts_resolvido")
094 |         batch_op.drop_index("ix_alerts_resolvido_created_at")
095 |         batch_op.drop_index("ix_alerts_tipo_resolvido")
096 |
097 |     with op.batch_alter_table("companies", schema=None) as batch_op:
098 |         batch_op.drop_index("ix_companies_deleted_at")
099 |
100 |     with op.batch_alter_table("parts", schema=None) as batch_op:
101 |         batch_op.drop_index("ix_parts_deleted_at")
102 |         batch_op.drop_index("ix_parts_nome")
103 |
104 |     with op.batch_alter_table("technicians", schema=None) as batch_op:
105 |         batch_op.drop_index("ix_technicians_deleted_at")
106 |
107 |     with op.batch_alter_table("users", schema=None) as batch_op:
108 |         batch_op.drop_index("ix_users_deleted_at")
109 |

```



```

041 |         nullable=True,
042 |         existing_server_default=sa.text('(CURRENT_TIMESTAMP)'))
043 | batch_op.alter_column('ativo',
044 |         existing_type=sa.BOOLEAN(),
045 |         nullable=True,
046 |         existing_server_default=sa.text('1'))
047 | batch_op.alter_column('perfil',
048 |         existing_type=sa.VARCHAR(length=20),
049 |         nullable=True,
050 |         existing_server_default=sa.text("'visualizador'"))
051 | batch_op.alter_column('username',
052 |         existing_type=sa.VARCHAR(length=50),
053 |         nullable=True)
054 | batch_op.alter_column('id',
055 |         existing_type=sa.INTEGER(),
056 |         nullable=True,
057 |         autoincrement=True)
058 |
059 | with op.batch_alter_table('transfers', schema=None) as batch_op:
060 |     batch_op.drop_constraint('fk_transfers_to_company', type_='foreignkey')
061 |     batch_op.drop_constraint('fk_transfers_printer', type_='foreignkey')
062 |     batch_op.drop_constraint('fk_transfers_from_company', type_='foreignkey')
063 |     batch_op.alter_column('created_at',
064 |         existing_type=sa.DATETIME(),
065 |         nullable=True,
066 |         existing_server_default=sa.text('(CURRENT_TIMESTAMP)'))
067 |     batch_op.alter_column('observacao',
068 |         existing_type=sa.TEXT(),
069 |         nullable=True,
070 |         existing_server_default=sa.text("(')"))
071 |     batch_op.alter_column('responsavel_recebimento',
072 |         existing_type=sa.VARCHAR(length=120),
073 |         nullable=True)
074 |     batch_op.alter_column('responsavel_entrega',
075 |         existing_type=sa.VARCHAR(length=120),
076 |         nullable=True)
077 |     batch_op.alter_column('tipo',
078 |         existing_type=sa.VARCHAR(length=20),
079 |         nullable=True)
080 |     batch_op.alter_column('numero_os',
081 |         existing_type=sa.VARCHAR(length=50),
082 |         nullable=True)
083 |     batch_op.alter_column('printer_id',
084 |         existing_type=sa.VARCHAR(length=36),
085 |         nullable=True)
086 |     batch_op.alter_column('id',
087 |         existing_type=sa.INTEGER(),
088 |         nullable=True,
089 |         autoincrement=True)
090 |
091 | with op.batch_alter_table('printers', schema=None) as batch_op:
092 |     batch_op.drop_constraint('fk_printers_company', type_='foreignkey')
093 |     batch_op.alter_column('tecnico',
094 |         existing_type=sa.VARCHAR(length=120),
095 |         nullable=True,
096 |         existing_server_default=sa.text("(')"))
097 |     batch_op.alter_column('mac_address',
098 |         existing_type=sa.VARCHAR(length=17),
099 |         nullable=True,
100 |         existing_server_default=sa.text("(')"))
101 |     batch_op.alter_column('ip_rede',
102 |         existing_type=sa.VARCHAR(length=45),
103 |         nullable=True,

```

```

104 |         existing_server_default=sa.text("('')"))
105 |     batch_op.alter_column('tipo',
106 |         existing_type=sa.VARCHAR(length=20),
107 |         nullable=True,
108 |         existing_server_default=sa.text("('')"))
109 |     batch_op.alter_column('marca',
110 |         existing_type=sa.VARCHAR(length=80),
111 |         nullable=True,
112 |         existing_server_default=sa.text("('')"))
113 |
114 |     with op.batch_alter_table('parts', schema=None) as batch_op:
115 |         batch_op.alter_column('modelo_compativel',
116 |             existing_type=sa.VARCHAR(length=200),
117 |             nullable=True,
118 |             existing_server_default=sa.text("('')"))
119 |
120 |     with op.batch_alter_table('companies', schema=None) as batch_op:
121 |         batch_op.alter_column('created_at',
122 |             existing_type=sa.DATETIME(),
123 |             nullable=True,
124 |             existing_server_default=sa.text('(CURRENT_TIMESTAMP)'))
125 |         batch_op.alter_column('tipo',
126 |             existing_type=sa.VARCHAR(length=20),
127 |             nullable=True,
128 |             existing_server_default=sa.text("'cliente'"))
129 |         batch_op.alter_column('email',
130 |             existing_type=sa.VARCHAR(length=120),
131 |             nullable=True)
132 |         batch_op.alter_column('telefone',
133 |             existing_type=sa.VARCHAR(length=20),
134 |             nullable=True)
135 |         batch_op.alter_column('uf',
136 |             existing_type=sa.VARCHAR(length=2),
137 |             nullable=True)
138 |         batch_op.alter_column('cidade',
139 |             existing_type=sa.VARCHAR(length=100),
140 |             nullable=True)
141 |         batch_op.alter_column('endereco',
142 |             existing_type=sa.VARCHAR(length=255),
143 |             nullable=True)
144 |         batch_op.alter_column('cnpj',
145 |             existing_type=sa.VARCHAR(length=18),
146 |             nullable=True)
147 |         batch_op.alter_column('id',
148 |             existing_type=sa.INTEGER(),
149 |             nullable=True,
150 |             autoincrement=True)
151 |
152 |     with op.batch_alter_table('attachments', schema=None) as batch_op:
153 |         batch_op.drop_constraint('fk_attachments_uploader', type_='foreignkey')
154 |         batch_op.alter_column('created_at',
155 |             existing_type=sa.DATETIME(),
156 |             nullable=True,
157 |             existing_server_default=sa.text('(CURRENT_TIMESTAMP)'))
158 |         batch_op.alter_column('categoria',
159 |             existing_type=sa.VARCHAR(length=50),
160 |             nullable=True)
161 |         batch_op.alter_column('size_bytes',
162 |             existing_type=sa.INTEGER(),
163 |             nullable=True)
164 |         batch_op.alter_column('mime_type',
165 |             existing_type=sa.VARCHAR(length=100),
166 |             nullable=True)

```

```

167 |         batch_op.alter_column('file_path',
168 |                               existing_type=sa.VARCHAR(length=500),
169 |                               nullable=True)
170 |         batch_op.alter_column('original_name',
171 |                               existing_type=sa.VARCHAR(length=255),
172 |                               nullable=True)
173 |         batch_op.alter_column('filename',
174 |                               existing_type=sa.VARCHAR(length=255),
175 |                               nullable=True)
176 |         batch_op.alter_column('entity_id',
177 |                               existing_type=sa.VARCHAR(length=36),
178 |                               nullable=True)
179 |         batch_op.alter_column('entity_type',
180 |                               existing_type=sa.VARCHAR(length=50),
181 |                               nullable=True)
182 |         batch_op.alter_column('id',
183 |                               existing_type=sa.INTEGER(),
184 |                               nullable=True,
185 |                               autoincrement=True)
186 |
187 |     with op.batch_alter_table('alerts', schema=None) as batch_op:
188 |         batch_op.drop_constraint('fk_alerts_printer', type_='foreignkey')
189 |         batch_op.drop_constraint('fk_alerts_resolved_by', type_='foreignkey')
190 |         batch_op.drop_index(batch_op.f('ix_alerts_part_id'))
191 |         batch_op.create_index(batch_op.f('ix_alert_part_id'), ['part_id'], unique=False)
192 |         batch_op.alter_column('created_at',
193 |                               existing_type=sa.DATETIME(),
194 |                               nullable=True,
195 |                               existing_server_default=sa.text('(CURRENT_TIMESTAMP)'))
196 |         batch_op.alter_column('resolvido',
197 |                               existing_type=sa.BOOLEAN(),
198 |                               nullable=True,
199 |                               existing_server_default=sa.text('0'))
200 |         batch_op.alter_column('titulo',
201 |                               existing_type=sa.VARCHAR(length=200),
202 |                               nullable=True)
203 |         batch_op.alter_column('tipo',
204 |                               existing_type=sa.VARCHAR(length=20),
205 |                               nullable=True)
206 |         batch_op.alter_column('id',
207 |                               existing_type=sa.INTEGER(),
208 |                               nullable=True,
209 |                               autoincrement=True)
210 |
211 |     with op.batch_alter_table('activities', schema=None) as batch_op:
212 |         batch_op.drop_constraint('fk_activities_to_company', type_='foreignkey')
213 |         batch_op.drop_constraint('fk_activities_technician', type_='foreignkey')
214 |         batch_op.drop_constraint('fk_activities_from_company', type_='foreignkey')
215 |         batch_op.drop_constraint('fk_activities_os_vinculada', type_='foreignkey')
216 |         batch_op.drop_constraint('fk_activities_printer', type_='foreignkey')
217 |         batch_op.drop_index(batch_op.f('ix_activities_os_vinculada_id'))
218 |         batch_op.alter_column('custo_servico',
219 |                               existing_type=sa.FLOAT(),
220 |                               nullable=True,
221 |                               existing_server_default=sa.text('(0.0)'))
222 |         batch_op.alter_column('numero_recibo',
223 |                               existing_type=sa.VARCHAR(length=50),
224 |                               nullable=True,
225 |                               existing_server_default=sa.text("('')"))
226 |         batch_op.drop_column('os_vinculada_id')
227 |
228 |     # ### end Alembic commands ###

```

=== ARQUIVO: .\dist\ControleImpressoras\internal\alembic\versions\aa953b7ee9b_add_deleted_at_to_part_movements.py ===

```
001 | """add_deleted_at_to_part_movements
002 |
003 | Revision ID: aa953b7eee9b
004 | Revises: b5f6a7c8d9e0
005 | Create Date: 2026-07-15 17:12:43.424357
006 |
007 | """
008 | from typing import Sequence, Union
009 |
010 | from alembic import op
011 | import sqlalchemy as sa
012 |
013 |
014 | # revision identifiers, used by Alembic.
015 | revision: str = 'aa953b7eee9b'
016 | down_revision: Union[str, Sequence[str], None] = 'b5f6a7c8d9e0'
017 | branch_labels: Union[str, Sequence[str], None] = None
018 | depends_on: Union[str, Sequence[str], None] = None
019 |
020 |
021 | def upgrade() -> None:
022 |     with op.batch_alter_table('part_movements', schema=None) as batch_op:
023 |         batch_op.add_column(sa.Column('deleted_at', sa.DateTime(), nullable=True))
024 |
025 | def downgrade() -> None:
026 |     with op.batch_alter_table('part_movements', schema=None) as batch_op:
027 |         batch_op.drop_column('deleted_at')
```

=== ARQUIVO: .\dist\ControleImpressoras\internal\alembic\versions\b1c2d3e4f5a6_create_maintenance_schedules.py ===

```
001 | """create maintenance_schedules table
002 |
003 | Revision ID: b1c2d3e4f5a6
004 | Revises: a605d5c373df
005 | Create Date: 2026-06-12 12:00:00.000000
006 |
007 | """
008 | from typing import Sequence, Union
009 |
010 | from alembic import op
011 | import sqlalchemy as sa
012 |
013 |
014 | revision: str = 'b1c2d3e4f5a6'
015 | down_revision: Union[str, Sequence[str], None] = 'a605d5c373df'
016 | branch_labels: Union[str, Sequence[str], None] = None
017 | depends_on: Union[str, Sequence[str], None] = None
018 |
019 |
020 | def upgrade() -> None:
021 |     op.create_table('maintenance_schedules',
022 |         sa.Column('id', sa.Integer(), autoincrement=True, nullable=False),
023 |         sa.Column('printer_id', sa.String(length=36), nullable=False),
024 |         sa.Column('tipo', sa.String(length=20), nullable=False),
025 |         sa.Column('intervalo_dias', sa.Integer(), nullable=True),
```



```

026 |         sa.Column('intervalo_paginas', sa.Integer(), nullable=True),
027 |         sa.Column('contador_inicial', sa.Integer(), nullable=True),
028 |         sa.Column('contador_atual', sa.Integer(), nullable=True),
029 |         sa.Column('ultima_geracao', sa.DateTime(), nullable=True),
030 |         sa.Column('proxima_geracao', sa.DateTime(), nullable=True),
031 |         sa.Column('proxima_meta_paginas', sa.Integer(), nullable=True),
032 |         sa.Column('dias_aviso', sa.Integer(), nullable=False, server_default=sa.text('7')),
033 |         sa.Column('observacao', sa.Text(), nullable=False, server_default=''),
034 |         sa.Column('ativo', sa.Boolean(), nullable=False, server_default=sa.text('1')),
035 |         sa.Column('created_at', sa.DateTime(), nullable=False),
036 |         sa.Column('updated_at', sa.DateTime(), nullable=False),
037 |         sa.ForeignKeyConstraint(['printer_id'], ['printers.id'], ),
038 |         sa.PrimaryKeyConstraint('id'),
039 |     )
040 |     with op.batch_alter_table('maintenance_schedules', schema=None) as batch_op:
041 |         batch_op.create_index('ix_maintenance_schedules_printer_id', ['printer_id'])
042 |
043 |
044 | def downgrade() -> None:
045 |     with op.batch_alter_table('maintenance_schedules', schema=None) as batch_op:
046 |         batch_op.drop_index('ix_maintenance_schedules_printer_id')
047 |     op.drop_table('maintenance_schedules')

```

=== ARQUIVO: .\dist\ControleImpressoras\internal\alembic\versions\b2c3d4e5f6a7_add_printer_locations.py ===

```

001 | """add_printer_locations
002 |
003 | Revision ID: b2c3d4e5f6a7
004 | Revises: a1b2c3d4e5f6
005 | Create Date: 2026-06-04 11:00:00.000000
006 |
007 | """
008 | from typing import Sequence, Union
009 |
010 | from alembic import op
011 | import sqlalchemy as sa
012 |
013 |
014 | revision: str = 'b2c3d4e5f6a7'
015 | down_revision: Union[str, None] = 'a1b2c3d4e5f6'
016 | branch_labels: Union[str, Sequence[str], None] = None
017 | depends_on: Union[str, Sequence[str], None] = None
018 |
019 |
020 | def upgrade() -> None:
021 |     op.create_table('printer_locations',
022 |         sa.Column('id', sa.Integer(), autoincrement=True, nullable=False),
023 |         sa.Column('printer_id', sa.String(36), nullable=False),
024 |         sa.Column('local', sa.String(120), nullable=False),
025 |         sa.Column('data', sa.DateTime(), nullable=True),
026 |         sa.Column('observacao', sa.Text(), nullable=False),
027 |         sa.Column('created_at', sa.DateTime(), nullable=False),
028 |         sa.ForeignKeyConstraint(['printer_id'], ['printers.id'], ),
029 |         sa.PrimaryKeyConstraint('id')
030 |     )
031 |     op.create_index('ix_printer_locations_printer_id', 'printer_locations', ['printer_id'])
032 |
033 |
034 | def downgrade() -> None:

```

```
035 |     op.drop_index('ix_printer_locations_printer_id', table_name='printer_locations')
036 |     op.drop_table('printer_locations')
```

=== ARQUIVO: .\dist\ControleImpressoras\internal\alembic\versions\b5f6a7c8d9e0_create_printer_health.py ===

```
001 | """create printer_health table
002 |
003 | Revision ID: b5f6a7c8d9e0
004 | Revises: a44fc2747c28
005 | Create Date: 2026-07-10 18:00:00.000000
006 | """
007 | from __future__ import annotations
008 |
009 | from typing import Sequence
010 |
011 | import sqlalchemy as sa
012 | from alembic import op
013 |
014 | revision: str = "b5f6a7c8d9e0"
015 | down_revision: str | None = "a44fc2747c28"
016 | branch_labels: str | Sequence[str] | None = None
017 | depends_on: str | Sequence[str] | None = None
018 |
019 |
020 | def upgrade() -> None:
021 |     op.create_table(
022 |         "printer_health",
023 |         sa.Column("id", sa.Integer(), autoincrement=True, nullable=False),
024 |         sa.Column("printer_id", sa.String(36), nullable=False),
025 |         sa.Column("timestamp", sa.DateTime(), nullable=False),
026 |         sa.Column("is_online", sa.Integer(), nullable=False, server_default=sa.text("0")),
027 |         sa.Column("ping_ms", sa.Float(), nullable=True),
028 |         sa.Column("toner_level", sa.Integer(), nullable=True),
029 |         sa.Column("toner_max", sa.Integer(), nullable=True),
030 |         sa.Column("drum_life", sa.Integer(), nullable=True),
031 |         sa.Column("drum_max", sa.Integer(), nullable=True),
032 |         sa.Column("page_count", sa.Integer(), nullable=True),
033 |         sa.Column("error_code", sa.String(50), nullable=True),
034 |         sa.Column("error_message", sa.String(255), nullable=True),
035 |         sa.Column("raw_data", sa.Text(), nullable=True),
036 |         sa.Column("created_at", sa.DateTime(), nullable=False),
037 |         sa.PrimaryKeyConstraint("id"),
038 |         sa.ForeignKeyConstraint(["printer_id"], ["printers.id"], ondelete="CASCADE"),
039 |     )
040 |     op.create_index(op.f("ix_printer_health_printer_id"), "printer_health", ["printer_id"])
041 |     op.create_index(op.f("ix_printer_health_timestamp"), "printer_health", ["timestamp"])
042 |     op.create_index(
043 |         "ix_printer_health_printer_timestamp",
044 |         "printer_health",
045 |         ["printer_id", "timestamp"],
046 |     )
047 |
048 |
049 | def downgrade() -> None:
050 |     op.drop_index("ix_printer_health_printer_timestamp", table_name="printer_health")
051 |     op.drop_index(op.f("ix_printer_health_timestamp"), table_name="printer_health")
052 |     op.drop_index(op.f("ix_printer_health_printer_id"), table_name="printer_health")
053 |     op.drop_table("printer_health")
```



```

026 |         existing_nullable=False,
027 |         existing_server_default=sa.text("(')"))
028 |
029 |     # fix status_atividade default
030 |     with op.batch_alter_table('activities', schema=None) as batch_op:
031 |         batch_op.alter_column('status_atividade',
032 |             existing_type=sa.VARCHAR(length=30),
033 |             existing_server_default=sa.text("'concluida'"),
034 |             nullable=False)
035 |
036 |     # indexes
037 |     op.create_index(op.f('ix_parts_codigo'), 'parts', ['codigo'], unique=False)
038 |     op.create_index(op.f('ix_companies_cnpj'), 'companies', ['cnpj'], unique=False)
039 |     op.create_index(op.f('ix_activities_kind'), 'activities', ['kind'], unique=False)
040 |     op.create_index(op.f('ix_activities_numero_recibo'), 'activities', ['numero_recibo'], unique=False)
041 |     op.create_index(op.f('ix_activities_status_atividade'), 'activities', ['status_atividade'], unique=False)
042 |     op.create_index(op.f('ix_transfers_numero_os'), 'transfers', ['numero_os'], unique=False)
043 |     op.create_index(op.f('ix_transfers_tipo'), 'transfers', ['tipo'], unique=False)
044 |     op.create_index(op.f('ix_users_username'), 'users', ['username'], unique=False)
045 |
046 |
047 | def downgrade() -> None:
048 |     op.drop_index(op.f('ix_users_username'), table_name='users')
049 |     op.drop_index(op.f('ix_transfers_tipo'), table_name='transfers')
050 |     op.drop_index(op.f('ix_transfers_numero_os'), table_name='transfers')
051 |     op.drop_index(op.f('ix_activities_status_atividade'), table_name='activities')
052 |     op.drop_index(op.f('ix_activities_numero_recibo'), table_name='activities')
053 |     op.drop_index(op.f('ix_activities_kind'), table_name='activities')
054 |     op.drop_index(op.f('ix_companies_cnpj'), table_name='companies')
055 |     op.drop_index(op.f('ix_parts_codigo'), table_name='parts')
056 |
057 |     with op.batch_alter_table('printers', schema=None) as batch_op:
058 |         batch_op.alter_column('ip_rede',
059 |             existing_type=sa.String(length=45),
060 |             type_=sa.VARCHAR(length=15),
061 |             existing_nullable=False,
062 |             existing_server_default=sa.text("(')"))

```

=== ARQUIVO: .\dist\ControleImpressoras\internal\alembic\versions\d4e5f6a7b8c9_add_user_id_to_technicians.py ===

```

001 | """add user_id to technicians for user association
002 |
003 | Revision ID: d4e5f6a7b8c9
004 | Revises: c3d4e5f6a7b8
005 | Create Date: 2026-06-15 10:00:00.000000
006 |
007 | """
008 | from typing import Sequence, Union
009 |
010 | from alembic import op
011 | import sqlalchemy as sa
012 |
013 |
014 | revision: str = 'd4e5f6a7b8c9'
015 | down_revision: Union[str, Sequence[str], None] = 'c3d4e5f6a7b8'
016 | branch_labels: Union[str, Sequence[str], None] = None
017 | depends_on: Union[str, Sequence[str], None] = None
018 |
019 |

```

```

020 | def upgrade() -> None:
021 |     with op.batch_alter_table('technicians', schema=None) as batch_op:
022 |         batch_op.add_column(sa.Column('user_id', sa.Integer(), nullable=True))
023 |         batch_op.create_foreign_key('fk_technicians_user_id', 'users', ['user_id'], ['id'])
024 |
025 |
026 | def downgrade() -> None:
027 |     with op.batch_alter_table('technicians', schema=None) as batch_op:
028 |         batch_op.drop_constraint('fk_technicians_user_id', type_='foreignkey')
029 |         batch_op.drop_column('user_id')

```

=== ARQUIVO: .\dist\ControleImpressoras\internal\alembic\versions\ded4cb6c701a_add_estoque_minimo_to_part.py ===

```

001 | """add_estoque_minimo_to_part
002 |
003 | Revision ID: ded4cb6c701a
004 | Revises: initial
005 | Create Date: 2026-05-18 14:57:26.514563
006 |
007 | """
008 | from typing import Sequence, Union
009 |
010 | from alembic import op
011 | import sqlalchemy as sa
012 |
013 |
014 | revision: str = 'ded4cb6c701a'
015 | down_revision: Union[str, Sequence[str], None] = 'initial'
016 | branch_labels: Union[str, Sequence[str], None] = None
017 | depends_on: Union[str, Sequence[str], None] = None
018 |
019 |
020 | def upgrade() -> None:
021 |     with op.batch_alter_table('parts', schema=None) as batch_op:
022 |         batch_op.add_column(sa.Column('estoque_minimo', sa.Integer(), nullable=False, server_default=sa.text('0')))
023 |
024 |
025 | def downgrade() -> None:
026 |     with op.batch_alter_table('parts', schema=None) as batch_op:
027 |         batch_op.drop_column('estoque_minimo')

```

=== ARQUIVO: .\dist\ControleImpressoras\internal\alembic\versions\e2ea9a0e5059_add_sintoma_relato_to_activities.py ===

```

001 | """add_sintoma_relato_to_activities
002 |
003 | Revision ID: e2ea9a0e5059
004 | Revises: 3df1b9c20c20
005 | Create Date: 2026-06-05 17:42:45.220479
006 |
007 | """
008 | from typing import Sequence, Union
009 |
010 | from alembic import op
011 | import sqlalchemy as sa
012 |

```

```

013 |
014 | # revision identifiers, used by Alembic.
015 | revision: str = 'e2ea9a0e5059'
016 | down_revision: Union[str, Sequence[str], None] = '3df1b9c20c20'
017 | branch_labels: Union[str, Sequence[str], None] = None
018 | depends_on: Union[str, Sequence[str], None] = None
019 |
020 |
021 | def upgrade() -> None:
022 |     with op.batch_alter_table('activities', schema=None) as batch_op:
023 |         batch_op.add_column(sa.Column('sintoma_relato', sa.Text(), nullable=False, server_default=''))
024 |
025 |
026 | def downgrade() -> None:
027 |     with op.batch_alter_table('activities', schema=None) as batch_op:
028 |         batch_op.drop_column('sintoma_relato')

```

=== ARQUIVO: .\dist\ControleImpressoras\internal\alembic\versions\le5f6a7b8c9d0_create_technician_specialties.py ===

```

001 | """create technician_specialties table
002 |
003 | Revision ID: e5f6a7b8c9d0
004 | Revises: d4e5f6a7b8c9
005 | Create Date: 2026-06-15 11:00:00.000000
006 |
007 | """
008 | from typing import Sequence, Union
009 |
010 | from alembic import op
011 | import sqlalchemy as sa
012 |
013 |
014 | revision: str = 'e5f6a7b8c9d0'
015 | down_revision: Union[str, Sequence[str], None] = 'd4e5f6a7b8c9'
016 | branch_labels: Union[str, Sequence[str], None] = None
017 | depends_on: Union[str, Sequence[str], None] = None
018 |
019 |
020 | def upgrade() -> None:
021 |     op.create_table('technician_specialties',
022 |         sa.Column('technician_id', sa.Integer(), nullable=False),
023 |         sa.Column('printer_id', sa.String(length=36), nullable=False),
024 |         sa.ForeignKeyConstraint(['technician_id'], ['technicians.id'], ),
025 |         sa.ForeignKeyConstraint(['printer_id'], ['printers.id'], ),
026 |         sa.PrimaryKeyConstraint('technician_id', 'printer_id'),
027 |     )
028 |
029 |
030 | def downgrade() -> None:
031 |     op.drop_table('technician_specialties')

```

=== ARQUIVO: .\dist\ControleImpressoras\internal\alembic\versions\lf1a2b3c4d5e6_add_marca_categoria_fornecedor_to_parts.py ===

```

001 | """add_marca_categoria_fornecedor_to_parts
002 |

```

```

003 | Revision ID: f1a2b3c4d5e6
004 | Revises: aa953b7eee9b
005 | Create Date: 2026-07-23 10:00:00.000000
006 |
007 | """
008 | from typing import Sequence, Union
009 |
010 | from alembic import op
011 | import sqlalchemy as sa
012 |
013 |
014 | revision: str = 'f1a2b3c4d5e6'
015 | down_revision: Union[str, Sequence[str], None] = 'aa953b7eee9b'
016 | branch_labels: Union[str, Sequence[str], None] = None
017 | depends_on: Union[str, Sequence[str], None] = None
018 |
019 |
020 | def upgrade() -> None:
021 |     op.execute("PRAGMA foreign_keys = OFF")
022 |     with op.batch_alter_table('parts', schema=None) as batch_op:
023 |         batch_op.add_column(sa.Column('marca', sa.String(100), server_default=''))
024 |         batch_op.add_column(sa.Column('categoria', sa.String(100), server_default=''))
025 |         batch_op.add_column(sa.Column('fornecedor', sa.String(150), server_default=''))
026 |     op.execute("PRAGMA foreign_keys = ON")
027 |
028 |
029 | def downgrade() -> None:
030 |     op.execute("PRAGMA foreign_keys = OFF")
031 |     with op.batch_alter_table('parts', schema=None) as batch_op:
032 |         batch_op.drop_column('fornecedor')
033 |         batch_op.drop_column('categoria')
034 |         batch_op.drop_column('marca')
035 |     op.execute("PRAGMA foreign_keys = ON")

```

=== ARQUIVO: .\dist\ControleImpressoras\internal\alembic\versions\lf6a7b8c9d0e1_refactor_technician_specialties.py ===

```

001 | """refactor technician_specialties to use modelo instead of printer_id
002 |
003 | Revision ID: f6a7b8c9d0e1
004 | Revises: e5f6a7b8c9d0
005 | Create Date: 2026-06-15 12:00:00.000000
006 |
007 | """
008 | from typing import Sequence, Union
009 |
010 | from alembic import op
011 | import sqlalchemy as sa
012 |
013 |
014 | revision: str = 'f6a7b8c9d0e1'
015 | down_revision: Union[str, Sequence[str], None] = 'e5f6a7b8c9d0'
016 | branch_labels: Union[str, Sequence[str], None] = None
017 | depends_on: Union[str, Sequence[str], None] = None
018 |
019 |
020 | def upgrade() -> None:
021 |     op.drop_table('technician_specialties')
022 |     op.create_table('technician_specialties',
023 |         sa.Column('id', sa.Integer(), autoincrement=True, nullable=False),

```

```

024 |         sa.Column('technician_id', sa.Integer(), sa.ForeignKey('technicians.id'), nullable=False),
025 |         sa.Column('modelo', sa.String(length=120), nullable=False),
026 |         sa.PrimaryKeyConstraint('id'),
027 |     )
028 |
029 |
030 | def downgrade() -> None:
031 |     op.drop_table('technician_specialties')
032 |     op.create_table('technician_specialties',
033 |         sa.Column('technician_id', sa.Integer(), nullable=False),
034 |         sa.Column('printer_id', sa.String(length=36), nullable=False),
035 |         sa.ForeignKeyConstraint(['technician_id'], ['technicians.id'], ),
036 |         sa.ForeignKeyConstraint(['printer_id'], ['printers.id'], ),
037 |         sa.PrimaryKeyConstraint('technician_id', 'printer_id'),
038 |     )

```

=== ARQUIVO:

.\dist\ControleImpressoras\internal\alembic\versions\initial_schema.py ===

```

001 | """initial_schema
002 |
003 | Revision ID: initial
004 | Revises:
005 | Create Date: 2026-05-18
006 | """
007 |
008 | from typing import Sequence, Union
009 | from alembic import op
010 | import sqlalchemy as sa
011 |
012 | revision: str = "initial"
013 | down_revision: Union[str, Sequence[str], None] = None
014 | branch_labels: Union[str, Sequence[str], None] = None
015 | depends_on: Union[str, Sequence[str], None] = None
016 |
017 |
018 | def upgrade() -> None:
019 |     pass
020 |
021 |
022 | def downgrade() -> None:
023 |     pass

```

=== ARQUIVO:

.\dist\ControleImpressoras\internal\pysnmp\smi\mibs\ASN1-ENUMERATION.py

===

```

001 | #
002 | # This file is part of pysnmp software.
003 | #
004 | # Copyright (c) 2005-2019, Ilya Etingof <etingof@gmail.com>
005 | # License: http://snmplabs.com/pysnmp/license.html
006 | #
007 | from pyasn1.type import namedval
008 |
009 | mibBuilder.exportSymbols(
010 |     'ASN1-ENUMERATION',

```



```
001 |     NamedValues=namedval.NamedValues
002 | )
```

=== ARQUIVO:

.\dist\ControleImpressoras_internal\pysnmp\smi\mibs\ASN1-REFINEMENT.py

===

```
001 | #
002 | # This file is part of pysnmp software.
003 | #
004 | # Copyright (c) 2005-2019, Ilya Etingof <etingof@gmail.com>
005 | # License: http://snmplabs.com/pysnmp/license.html
006 | #
007 | from pyasn1.type import constraint
008 |
009 | mibBuilder.exportSymbols(
010 |     'ASN1-REFINEMENT',
011 |     ConstraintsUnion=constraint.ConstraintsUnion,
012 |     ConstraintsIntersection=constraint.ConstraintsIntersection,
013 |     SingleValueConstraint=constraint.SingleValueConstraint,
014 |     ValueRangeConstraint=constraint.ValueRangeConstraint,
015 |     ValueSizeConstraint=constraint.ValueSizeConstraint
016 | )
```

=== ARQUIVO: .\dist\ControleImpressoras_internal\pysnmp\smi\mibs\ASN1.py

===

```
001 | #
002 | # This file is part of pysnmp software.
003 | #
004 | # Copyright (c) 2005-2019, Ilya Etingof <etingof@gmail.com>
005 | # License: http://snmplabs.com/pysnmp/license.html
006 | #
007 | from pyasn1.type import univ
008 | from pysnmp.proto import rfc1902
009 |
010 | mibBuilder.exportSymbols(
011 |     'ASN1',
012 |     ObjectIdentifier=univ.ObjectIdentifier,
013 |     # Instead of using base ASN.1 types we use SNMPv2 SMI ones to make
014 |     # SMI objects type-compatible with SNMP protocol values
015 |     Integer=rfc1902.Integer32,
016 |     OctetString=rfc1902.OctetString
017 | )
```

=== ARQUIVO:

.\dist\ControleImpressoras_internal\pysnmp\smi\mibs\INET-ADDRESS-MIB.py

===

```
001 | #
002 | # This file is part of pysnmp software.
003 | #
004 | # Copyright (c) 2005-2019, Ilya Etingof <etingof@gmail.com>
```

```

005 | # License: http://snmplabs.com/pysnmp/license.html
006 | #
007 | # PySNMP MIB module INET-ADDRESS-MIB (http://snmplabs.com/pysnmp)
008 | # ASN.1 source http://mibs.snmplabs.com:80/asn1/INET-ADDRESS-MIB
009 | # Produced by pysmi-0.1.2 at Sat Apr 15 23:36:33 2017
010 | # On host grommit.local platform Darwin version 16.4.0 by user ilya
011 | # Using Python version 3.4.2 (v3.4.2:ab2c023a9432, Oct 5 2014, 20:42:22)
012 | #
013 | from pysnmp.smi import error
014 |
015 | ObjectIdentifier, Integer, OctetString = mibBuilder.importSymbols("ASN1", "ObjectIdentifier", "Integer", "0
016 | NamedValues, = mibBuilder.importSymbols("ASN1-ENUMERATION", "NamedValues")
017 | ValueRangeConstraint, ValueSizeConstraint, ConstraintsIntersection, ConstraintsUnion, SingleValueConstraint
018 | NotificationGroup, ModuleCompliance = mibBuilder.importSymbols("SNMPv2-CONF", "NotificationGroup", "ModuleC
019 | Gauge32, iso, Bits, Integer32, MibIdentifier, TimeTicks, MibScalar, MibTable, MibTableRow, MibTableColumn,
020 | TextualConvention, DisplayString = mibBuilder.importSymbols("SNMPv2-TC", "TextualConvention", "DisplayStrin
021 | inetAddressMIB = ModuleIdentity((1, 3, 6, 1, 2, 1, 76))
022 | if mibBuilder.loadTexts: inetAddressMIB.setRevisions(('2005-02-04 00:00', '2002-05-09 00:00', '2000-06-08 0
023 | if mibBuilder.loadTexts: inetAddressMIB.setLastUpdated('200502040000Z')
024 | if mibBuilder.loadTexts: inetAddressMIB.setOrganization('IETF Operations and Management Area')
025 | if mibBuilder.loadTexts: inetAddressMIB.setContactInfo('Juergen Schoenwaelder (Editor) International Univer
026 | if mibBuilder.loadTexts: inetAddressMIB.setDescription('This MIB module defines textual conventions for rep
027 | class InetAddressType(TextualConvention, Integer32):
028 |     description = 'A value that represents a type of Internet address. unknown(0) An unknown address type.
029 |     status = 'current'
030 |     subtypeSpec = Integer32.subtypeSpec + ConstraintsUnion(SingleValueConstraint(0, 1, 2, 3, 4, 16))
031 |     namedValues = NamedValues(("unknown", 0), ("ipv4", 1), ("ipv6", 2), ("ipv4z", 3), ("ipv6z", 4), ("dns",
032 |
033 | class InetAddressIPv4(TextualConvention, OctetString):
034 |     description = 'Represents an IPv4 network address: Octets Contents Encoding 1-4 IPv4 address network-by
035 |     status = 'current'
036 |     displayHint = '1d.1d.1d.1d'
037 |     subtypeSpec = OctetString.subtypeSpec + ValueSizeConstraint(4, 4)
038 |
039 | class InetAddressIPv6(TextualConvention, OctetString):
040 |     description = 'Represents an IPv6 network address: Octets Contents Encoding 1-16 IPv6 address network-b
041 |     status = 'current'
042 |     displayHint = '2x:2x:2x:2x:2x:2x:2x:2x'
043 |     subtypeSpec = OctetString.subtypeSpec + ValueSizeConstraint(16, 16)
044 |
045 | class InetAddressIPv4z(TextualConvention, OctetString):
046 |     description = 'Represents a non-global IPv4 network address, together with its zone index: Octets Conte
047 |     status = 'current'
048 |     displayHint = '1d.1d.1d.1d%4d'
049 |     subtypeSpec = OctetString.subtypeSpec + ValueSizeConstraint(8, 8)
050 |
051 | class InetAddressIPv6z(TextualConvention, OctetString):
052 |     description = 'Represents a non-global IPv6 network address, together with its zone index: Octets Conte
053 |     status = 'current'
054 |     displayHint = '2x:2x:2x:2x:2x:2x:2x:2x%4d'
055 |     subtypeSpec = OctetString.subtypeSpec + ValueSizeConstraint(20, 20)
056 |
057 | class InetAddressDNS(TextualConvention, OctetString):
058 |     description = 'Represents a DNS domain name. The name SHOULD be fully qualified whenever possible. The
059 |     status = 'current'
060 |     displayHint = '255a'
061 |     subtypeSpec = OctetString.subtypeSpec + ValueSizeConstraint(1, 255)
062 |
063 | # https://tools.ietf.org/html/rfc4001#section-4.1
064 |
065 | class InetAddress(TextualConvention, OctetString):
066 |     description = "Denotes a generic Internet address. An InetAddress value is always interpreted within th
067 |     status = 'current'

```

```

068 |     subtypeSpec = OctetString.subtypeSpec + ValueSizeConstraint(0, 255)
069 |
070 |     typeMap = {
071 |         InetAddressType.namedValues.getValue("ipv4"): InetAddressIPv4(),
072 |         InetAddressType.namedValues.getValue("ipv6"): InetAddressIPv6(),
073 |         InetAddressType.namedValues.getValue("ipv4z"): InetAddressIPv4z(),
074 |         InetAddressType.namedValues.getValue("ipv6z"): InetAddressIPv6z(),
075 |         InetAddressType.namedValues.getValue("dns"): InetAddressDNS()
076 |     }
077 |
078 |     @classmethod
079 |     def cloneFromName(cls, value, impliedFlag, parentRow, parentIndices):
080 |         for parentIndex in reversed(parentIndices):
081 |             if isinstance(parentIndex, InetAddressType):
082 |                 try:
083 |                     return parentRow.setFromName(cls.typeMap[int(parentIndex)], value, impliedFlag, parentI
084 |                 except KeyError:
085 |                     pass
086 |
087 |             raise error.SmiError('%s object encountered without preceding InetAddressType-like index: %r' % (cl
088 |
089 |     def cloneAsName(self, impliedFlag, parentRow, parentIndices):
090 |         for parentIndex in reversed(parentIndices):
091 |             if isinstance(parentIndex, InetAddressType):
092 |                 try:
093 |                     return parentRow.getAsName(self.typeMap[int(parentIndex)].clone(self.asOctets()).decode(
094 |                 except KeyError:
095 |                     pass
096 |
097 |             raise error.SmiError('%s object encountered without preceding InetAddressType-like index: %r' % (se
098 |
099 | class InetAddressPrefixLength(TextualConvention, Unsigned32):
100 |     description = "Denotes the length of a generic Internet network address prefix. A value of n correspond
101 |     status = 'current'
102 |     displayHint = 'd'
103 |     subtypeSpec = Unsigned32.subtypeSpec + ValueRangeConstraint(0, 2040)
104 |
105 | class InetPortNumber(TextualConvention, Unsigned32):
106 |     reference = 'STD 6 (RFC 768), STD 7 (RFC 793) and RFC 2960'
107 |     description = 'Represents a 16 bit port number of an Internet transport layer protocol. Port numbers ar
108 |     status = 'current'
109 |     displayHint = 'd'
110 |     subtypeSpec = Unsigned32.subtypeSpec + ValueRangeConstraint(0, 65535)
111 |
112 | class InetAutonomousSystemNumber(TextualConvention, Unsigned32):
113 |     reference = 'RFC 1771, RFC 1930'
114 |     description = "Represents an autonomous system number that identifies an Autonomous System (AS). An AS
115 |     status = 'current'
116 |     displayHint = 'd'
117 |
118 | class InetScopeType(TextualConvention, Integer32):
119 |     reference = 'RFC 3513'
120 |     description = 'Represents a scope type. This textual convention can be used in cases where a MIB has to
121 |     status = 'current'
122 |     subtypeSpec = Integer32.subtypeSpec + ConstraintsUnion(SingleValueConstraint(1, 2, 3, 4, 5, 8, 14))
123 |     namedValues = NamedValues(("interfaceLocal", 1), ("linkLocal", 2), ("subnetLocal", 3), ("adminLocal", 4
124 |
125 | class InetZoneIndex(TextualConvention, Unsigned32):
126 |     reference = 'RFC4007'
127 |     description = 'A zone index identifies an instance of a zone of a specific scope. The zone index MUST d
128 |     status = 'current'
129 |     displayHint = 'd'
130 |

```

```

131 | class InetVersion(TextualConvention, Integer32):
132 |     reference = 'RFC 791, RFC 2460'
133 |     description = 'A value representing a version of the IP protocol. unknown(0) An unknown or unspecified
134 |     status = 'current'
135 |     subtypeSpec = Integer32.subtypeSpec + ConstraintsUnion(SingleValueConstraint(0, 1, 2))
136 |     namedValues = NamedValues(("unknown", 0), ("ipv4", 1), ("ipv6", 2))
137 |
138 | mibBuilder.exportSymbols("INET-ADDRESS-MIB", inetAddressMIB=inetAddressMIB, InetVersion=InetVersion, InetAd

```

=== ARQUIVO:

.\dist\ControleImpressoras_internal\pysnmp\smi\mibs\PYSNMP-MIB.py ===

```

001 | #
002 | # This file is part of pysnmp software.
003 | #
004 | # Copyright (c) 2005-2019, Ilya Etingof <etingof@gmail.com>
005 | # License: http://snmplabs.com/pysnmp/license.html
006 | #
007 | # PySNMP MIB module PYSNMP-MIB (http://snmplabs.com/pysnmp)
008 | # ASN.1 source http://mibs.snmplabs.com:80/asn1/PYSNMP-MIB
009 | # Produced by pysmi-0.1.3 at Mon Apr 17 11:46:02 2017
010 | # On host grommit.local platform Darwin version 16.4.0 by user ilya
011 | # Using Python version 3.4.2 (v3.4.2:ab2c023a9432, Oct 5 2014, 20:42:22)
012 | #
013 | Integer, OctetString, ObjectIdentifier = mibBuilder.importSymbols("ASN1", "Integer", "OctetString", "Object
014 | NamedValues, = mibBuilder.importSymbols("ASN1-ENUMERATION", "NamedValues")
015 | SingleValueConstraint, ValueRangeConstraint, ConstraintsIntersection, ValueSizeConstraint, ConstraintsUnion
016 | NotificationGroup, ModuleCompliance = mibBuilder.importSymbols("SNMPv2-CONF", "NotificationGroup", "ModuleC
017 | ModuleIdentity, iso, MibScalar, MibTable, MibTableRow, MibTableColumn, Gauge32, NotificationType, IpAddress
018 | TextualConvention, DisplayString = mibBuilder.importSymbols("SNMPv2-TC", "TextualConvention", "DisplayStrin
019 | pysnmp = ModuleIdentity((1, 3, 6, 1, 4, 1, 20408))
020 | if mibBuilder.loadTexts: pysnmp.setRevisions(('2017-04-14 00:00', '2005-05-14 00:00',))
021 | if mibBuilder.loadTexts: pysnmp.setLastUpdated('201704140000Z')
022 | if mibBuilder.loadTexts: pysnmp.setOrganization('The PySNMP Project')
023 | if mibBuilder.loadTexts: pysnmp.setContactInfo('E-mail: Ilya Etingof <etingof@gmail.com> GitHub: https://gi
024 | if mibBuilder.loadTexts: pysnmp.setDescription('PySNMP top-level MIB tree infrastructure')
025 | pysnmpObjects = MibIdentifier((1, 3, 6, 1, 4, 1, 20408, 1))
026 | pysnmpExamples = MibIdentifier((1, 3, 6, 1, 4, 1, 20408, 2))
027 | pysnmpEnumerations = MibIdentifier((1, 3, 6, 1, 4, 1, 20408, 3))
028 | pysnmpModuleIDs = MibIdentifier((1, 3, 6, 1, 4, 1, 20408, 3, 1))
029 | pysnmpAgentOIDs = MibIdentifier((1, 3, 6, 1, 4, 1, 20408, 3, 2))
030 | pysnmpDomains = MibIdentifier((1, 3, 6, 1, 4, 1, 20408, 3, 3))
031 | pysnmpExperimental = MibIdentifier((1, 3, 6, 1, 4, 1, 20408, 9999))
032 | pysnmpNotificationPrefix = MibIdentifier((1, 3, 6, 1, 4, 1, 20408, 4))
033 | pysnmpNotifications = MibIdentifier((1, 3, 6, 1, 4, 1, 20408, 4, 0))
034 | pysnmpNotificationObjects = MibIdentifier((1, 3, 6, 1, 4, 1, 20408, 4, 1))
035 | pysnmpConformance = MibIdentifier((1, 3, 6, 1, 4, 1, 20408, 5))
036 | pysnmpCompliances = MibIdentifier((1, 3, 6, 1, 4, 1, 20408, 5, 1))
037 | pysnmpGroups = MibIdentifier((1, 3, 6, 1, 4, 1, 20408, 5, 2))
038 | mibBuilder.exportSymbols("PYSNMP-MIB", pysnmpCompliances=pysnmpCompliances, pysnmpObjects=pysnmpObjects, py

```

=== ARQUIVO:

.\dist\ControleImpressoras_internal\pysnmp\smi\mibs\PYSNMP-SOURCE-MIB.py

===

```

001 | #
002 | # This file is part of pysnmp software.

```

```

003 | #
004 | # Copyright (c) 2005-2019, Ilya Etingof <etingof@gmail.com>
005 | # License: http://snmplabs.com/pysnmp/license.html
006 | #
007 | # PySNMP MIB module PYSNMP-SOURCE-MIB (http://snmplabs.com/pysnmp)
008 | # ASN.1 source http://mibs.snmplabs.com:80/asn1/PYSNMP-SOURCE-MIB
009 | # Produced by pysmi-0.1.3 at Mon Apr 17 11:46:02 2017
010 | # On host grommit.local platform Darwin version 16.4.0 by user ilya
011 | # Using Python version 3.4.2 (v3.4.2:ab2c023a9432, Oct 5 2014, 20:42:22)
012 | #
013 | Integer, OctetString, ObjectIdentifier = mibBuilder.importSymbols("ASN1", "Integer", "OctetString", "Object
014 | NamedValues, = mibBuilder.importSymbols("ASN1-ENUMERATION", "NamedValues")
015 | SingleValueConstraint, ValueRangeConstraint, ConstraintsIntersection, ValueSizeConstraint, ConstraintsUnion
016 | pysnmpModuleIDs, = mibBuilder.importSymbols("PYSNMP-MIB", "pysnmpModuleIDs")
017 | snmpTargetAddrEntry, = mibBuilder.importSymbols("SNMP-TARGET-MIB", "snmpTargetAddrEntry")
018 | NotificationGroup, ModuleCompliance = mibBuilder.importSymbols("SNMPv2-CONF", "NotificationGroup", "ModuleC
019 | ModuleIdentity, iso, MibScalar, MibTable, MibTableRow, MibTableColumn, Gauge32, NotificationType, IPAddress
020 | TextualConvention, DisplayString, TAddress = mibBuilder.importSymbols("SNMPv2-TC", "TextualConvention", "Di
021 | pysnmpSourceMIB = ModuleIdentity((1, 3, 6, 1, 4, 1, 20408, 3, 1, 8))
022 | if mibBuilder.loadTexts: pysnmpSourceMIB.setRevisions(('2017-04-14 00:00', '2015-01-16 00:00',))
023 | if mibBuilder.loadTexts: pysnmpSourceMIB.setLastUpdated('201704140000Z')
024 | if mibBuilder.loadTexts: pysnmpSourceMIB.setOrganization('The PySNMP Project')
025 | if mibBuilder.loadTexts: pysnmpSourceMIB.setContactInfo('E-mail: Ilya Etingof <etingof@gmail.com> GitHub: h
026 | if mibBuilder.loadTexts: pysnmpSourceMIB.setDescription('This MIB module defines implementation specific ob
027 | pysnmpSourceMIBObjects = MibIdentifier((1, 3, 6, 1, 4, 1, 20408, 3, 1, 8, 1))
028 | pysnmpSourceMIBConformance = MibIdentifier((1, 3, 6, 1, 4, 1, 20408, 3, 1, 8, 2))
029 | snmpSourceAddrTable = MibTable((1, 3, 6, 1, 4, 1, 20408, 3, 1, 8, 1, 1), )
030 | if mibBuilder.loadTexts: snmpSourceAddrTable.setStatus('current')
031 | if mibBuilder.loadTexts: snmpSourceAddrTable.setDescription('A table of transport addresses to be used as a
032 | snmpSourceAddrEntry = MibTableRow((1, 3, 6, 1, 4, 1, 20408, 3, 1, 8, 1, 1), )
033 | snmpTargetAddrEntry.registerAugmentations(("PYSNMP-SOURCE-MIB", "snmpSourceAddrEntry"))
034 | snmpSourceAddrEntry.setIndexNames(*snmpTargetAddrEntry.getIndexNames())
035 | if mibBuilder.loadTexts: snmpSourceAddrEntry.setStatus('current')
036 | if mibBuilder.loadTexts: snmpSourceAddrEntry.setDescription('A transport address to be used as a source in
037 | snmpSourceAddrTAddress = MibTableColumn((1, 3, 6, 1, 4, 1, 20408, 3, 1, 8, 1, 1, 1), TAddress()).setMaxA
038 | if mibBuilder.loadTexts: snmpSourceAddrTAddress.setStatus('current')
039 | if mibBuilder.loadTexts: snmpSourceAddrTAddress.setDescription('This object contains a transport address. T
040 | pysnmpSourceMIBCompliances = MibIdentifier((1, 3, 6, 1, 4, 1, 20408, 3, 1, 8, 2, 1))
041 | pysnmpSourceMIBGroups = MibIdentifier((1, 3, 6, 1, 4, 1, 20408, 3, 1, 8, 2, 2))
042 | mibBuilder.exportSymbols("PYSNMP-SOURCE-MIB", pysnmpSourceMIBConformance=pysnmpSourceMIBConformance, pysnmp

```

=== ARQUIVO:

.\dist\ControleImpressoras\internal\pysnmp\smi\mibs\PYSNMP-USM-MIB.py ===

```

001 | #
002 | # This file is part of pysnmp software.
003 | #
004 | # Copyright (c) 2005-2019, Ilya Etingof <etingof@gmail.com>
005 | # License: http://snmplabs.com/pysnmp/license.html
006 | #
007 | # PySNMP MIB module PYSNMP-USM-MIB (http://snmplabs.com/pysnmp)
008 | # ASN.1 source http://mibs.snmplabs.com:80/asn1/PYSNMP-USM-MIB
009 | # Produced by pysmi-0.1.3 at Mon Apr 17 11:46:02 2017
010 | # On host grommit.local platform Darwin version 16.4.0 by user ilya
011 | # Using Python version 3.4.2 (v3.4.2:ab2c023a9432, Oct 5 2014, 20:42:22)
012 | #
013 | Integer, OctetString, ObjectIdentifier = mibBuilder.importSymbols("ASN1", "Integer", "OctetString", "Object
014 | NamedValues, = mibBuilder.importSymbols("ASN1-ENUMERATION", "NamedValues")
015 | SingleValueConstraint, ValueRangeConstraint, ConstraintsIntersection, ValueSizeConstraint, ConstraintsUnion
016 | pysnmpModuleIDs, = mibBuilder.importSymbols("PYSNMP-MIB", "pysnmpModuleIDs")

```

```

017 | SnmpAdminString, = mibBuilder.importSymbols("SNMP-FRAMEWORK-MIB", "SnmpAdminString")
018 | usmUserEntry, = mibBuilder.importSymbols("SNMP-USER-BASED-SM-MIB", "usmUserEntry")
019 | NotificationGroup, ModuleCompliance = mibBuilder.importSymbols("SNMPv2-CONF", "NotificationGroup", "ModuleC
020 | ModuleIdentity, iso, MibScalar, MibTable, MibTableRow, MibTableColumn, Gauge32, NotificationType, IPAddress
021 | RowStatus, DisplayString, TextualConvention = mibBuilder.importSymbols("SNMPv2-TC", "RowStatus", "DisplaySt
022 | pysnmpUsmMIB = ModuleIdentity((1, 3, 6, 1, 4, 1, 20408, 3, 1, 1))
023 | if mibBuilder.loadTexts: pysnmpUsmMIB.setRevisions(('2017-04-14 00:00', '2005-05-14 00:00',))
024 | if mibBuilder.loadTexts: pysnmpUsmMIB.setLastUpdated('201704140000Z')
025 | if mibBuilder.loadTexts: pysnmpUsmMIB.setOrganization('The PySNMP Project')
026 | if mibBuilder.loadTexts: pysnmpUsmMIB.setContactInfo('E-mail: Ilya Etingof <etingof@gmail.com> GitHub: http
027 | if mibBuilder.loadTexts: pysnmpUsmMIB.setDescription('This MIB module defines objects specific to User Secu
028 | pysnmpUsmMIBObjects = MibIdentifier((1, 3, 6, 1, 4, 1, 20408, 3, 1, 1, 1))
029 | pysnmpUsmMIBConformance = MibIdentifier((1, 3, 6, 1, 4, 1, 20408, 3, 1, 1, 2))
030 | pysnmpUsmCfg = MibIdentifier((1, 3, 6, 1, 4, 1, 20408, 3, 1, 1, 1, 1))
031 | pysnmpUsmDiscoverable = MibScalar((1, 3, 6, 1, 4, 1, 20408, 3, 1, 1, 1, 1, 1), Integer32().subtype(subtypeS
032 | if mibBuilder.loadTexts: pysnmpUsmDiscoverable.setStatus('current')
033 | if mibBuilder.loadTexts: pysnmpUsmDiscoverable.setDescription('Whether SNMP engine would support its discov
034 | pysnmpUsmDiscovery = MibScalar((1, 3, 6, 1, 4, 1, 20408, 3, 1, 1, 1, 1, 2), Integer32().subtype(subtypeSpec
035 | if mibBuilder.loadTexts: pysnmpUsmDiscovery.setStatus('current')
036 | if mibBuilder.loadTexts: pysnmpUsmDiscovery.setDescription('Whether SNMP engine would try to figure out the
037 | pysnmpUsmKeyType = MibScalar((1, 3, 6, 1, 4, 1, 20408, 3, 1, 1, 1, 1, 3), Integer32().subtype(subtypeSpec=C
038 | if mibBuilder.loadTexts: pysnmpUsmKeyType.setStatus('current')
039 | if mibBuilder.loadTexts: pysnmpUsmKeyType.setDescription('When configuring USM user, the value of this enum
040 | pysnmpUsmUser = MibIdentifier((1, 3, 6, 1, 4, 1, 20408, 3, 1, 1, 1, 3))
041 | pysnmpUsmSecretTable = MibTable((1, 3, 6, 1, 4, 1, 20408, 3, 1, 1, 1, 2), )
042 | if mibBuilder.loadTexts: pysnmpUsmSecretTable.setStatus('current')
043 | if mibBuilder.loadTexts: pysnmpUsmSecretTable.setDescription("The table of USM users passphrases configured
044 | pysnmpUsmSecretEntry = MibTableRow((1, 3, 6, 1, 4, 1, 20408, 3, 1, 1, 1, 2, 1), ).setIndexNames((1, "PYSNMP
045 | if mibBuilder.loadTexts: pysnmpUsmSecretEntry.setStatus('current')
046 | if mibBuilder.loadTexts: pysnmpUsmSecretEntry.setDescription('Information about a particular USM user crede
047 | pysnmpUsmSecretUserName = MibTableColumn((1, 3, 6, 1, 4, 1, 20408, 3, 1, 1, 1, 2, 1, 1), SnmpAdminString().
048 | if mibBuilder.loadTexts: pysnmpUsmSecretUserName.setStatus('current')
049 | if mibBuilder.loadTexts: pysnmpUsmSecretUserName.setDescription('The username string for which a row in thi
050 | pysnmpUsmSecretAuthKey = MibTableColumn((1, 3, 6, 1, 4, 1, 20408, 3, 1, 1, 1, 2, 1, 2), OctetString('\x00\x
051 | if mibBuilder.loadTexts: pysnmpUsmSecretAuthKey.setStatus('current')
052 | if mibBuilder.loadTexts: pysnmpUsmSecretAuthKey.setDescription("User's authentication passphrase used for l
053 | pysnmpUsmSecretPrivKey = MibTableColumn((1, 3, 6, 1, 4, 1, 20408, 3, 1, 1, 1, 2, 1, 3), OctetString('\x00\x
054 | if mibBuilder.loadTexts: pysnmpUsmSecretPrivKey.setStatus('current')
055 | if mibBuilder.loadTexts: pysnmpUsmSecretPrivKey.setDescription("User's encryption passphrase used for local
056 | pysnmpUsmSecretStatus = MibTableColumn((1, 3, 6, 1, 4, 1, 20408, 3, 1, 1, 1, 2, 1, 4), RowStatus()).setMaxA
057 | if mibBuilder.loadTexts: pysnmpUsmSecretStatus.setStatus('current')
058 | if mibBuilder.loadTexts: pysnmpUsmSecretStatus.setDescription('Table status')
059 | pysnmpUsmKeyTable = MibTable((1, 3, 6, 1, 4, 1, 20408, 3, 1, 1, 1, 3), )
060 | if mibBuilder.loadTexts: pysnmpUsmKeyTable.setStatus('current')
061 | if mibBuilder.loadTexts: pysnmpUsmKeyTable.setDescription("The table of USM users localized keys configured
062 | pysnmpUsmKeyEntry = MibTableRow((1, 3, 6, 1, 4, 1, 20408, 3, 1, 1, 1, 3, 1), )
063 | usmUserEntry.registerAugmentations(("PYSNMP-USM-MIB", "pysnmpUsmKeyEntry"))
064 | pysnmpUsmKeyEntry.setIndexNames(*usmUserEntry.getIndexNames())
065 | if mibBuilder.loadTexts: pysnmpUsmKeyEntry.setStatus('current')
066 | if mibBuilder.loadTexts: pysnmpUsmKeyEntry.setDescription('Information about a particular USM user credenti
067 | pysnmpUsmKeyAuthLocalized = MibTableColumn((1, 3, 6, 1, 4, 1, 20408, 3, 1, 1, 1, 3, 1, 1), OctetString('\x0
068 | if mibBuilder.loadTexts: pysnmpUsmKeyAuthLocalized.setStatus('current')
069 | if mibBuilder.loadTexts: pysnmpUsmKeyAuthLocalized.setDescription("User's localized key used for authentica
070 | pysnmpUsmKeyPrivLocalized = MibTableColumn((1, 3, 6, 1, 4, 1, 20408, 3, 1, 1, 1, 3, 1, 2), OctetString('\x0
071 | if mibBuilder.loadTexts: pysnmpUsmKeyPrivLocalized.setStatus('current')
072 | if mibBuilder.loadTexts: pysnmpUsmKeyPrivLocalized.setDescription("User's localized key used for encryption
073 | pysnmpUsmKeyAuth = MibTableColumn((1, 3, 6, 1, 4, 1, 20408, 3, 1, 1, 1, 3, 1, 3), OctetString('\x00\x00\x00
074 | if mibBuilder.loadTexts: pysnmpUsmKeyAuth.setStatus('current')
075 | if mibBuilder.loadTexts: pysnmpUsmKeyAuth.setDescription("User's non-localized key used for authentication.
076 | pysnmpUsmKeyPriv = MibTableColumn((1, 3, 6, 1, 4, 1, 20408, 3, 1, 1, 1, 3, 1, 4), OctetString('\x00\x00\x00
077 | if mibBuilder.loadTexts: pysnmpUsmKeyPriv.setStatus('current')
078 | if mibBuilder.loadTexts: pysnmpUsmKeyPriv.setDescription("User's non-localized key used for encryption.")
079 | pysnmpUsmMIBCompliances = MibIdentifier((1, 3, 6, 1, 4, 1, 20408, 3, 1, 1, 2, 1))

```

```
080 | pysnmpUsmMIBGroups = MibIdentifier((1, 3, 6, 1, 4, 1, 20408, 3, 1, 1, 2, 2))
081 | mibBuilder.exportSymbols("PYSNMP-USM-MIB", pysnmpUsmCfg=pysnmpUsmCfg, pysnmpUsmDiscoverable=pysnmpUsmDiscoverable)
```

=== ARQUIVO:

.\dist\ControleImpressoras_internal\pysnmp\smi\mibs\RFC1158-MIB.py ===

```
001 | #
002 | # This file is part of pysnmp software.
003 | #
004 | # Copyright (c) 2005-2019, Ilya Etingof <etingof@gmail.com>
005 | # License: http://snmplabs.com/pysnmp/license.html
006 | #
007 | # PySNMP MIB module RFC1158-MIB (http://snmplabs.com/pysnmp)
008 | # ASN.1 source http://mibs.snmplabs.com:80/asn1/RFC1158-MIB
009 | # Produced by pysmi-0.1.3 at Mon Apr 17 12:12:07 2017
010 | # On host grommit.local platform Darwin version 16.4.0 by user ilya
011 | # Using Python version 3.4.2 (v3.4.2:ab2c023a9432, Oct 5 2014, 20:42:22)
012 | #
013 | # It is a stripped version of MIB that contains only symbols that is
014 | # unique to SMIV1 and have no analogues in SMIV2
015 | #
016 | Integer32, MibScalar, MibTable, MibTableRow, MibTableColumn, TimeTicks, iso, Gauge32, MibIdentifier, Bits,
017 | snmpInBadTypes = MibScalar((1, 3, 6, 1, 2, 1, 11, 7), Counter32()).setMaxAccess("readonly")
018 | if mibBuilder.loadTexts: snmpInBadTypes.setStatus('mandatory')
019 | snmpOutReadOnlys = MibScalar((1, 3, 6, 1, 2, 1, 11, 23), Counter32()).setMaxAccess("readonly")
020 | if mibBuilder.loadTexts: snmpOutReadOnlys.setStatus('mandatory')
021 | mibBuilder.exportSymbols("RFC1158-MIB", snmpOutReadOnlys=snmpOutReadOnlys, snmpInBadTypes=snmpInBadTypes)
```

=== ARQUIVO:

.\dist\ControleImpressoras_internal\pysnmp\smi\mibs\RFC1213-MIB.py ===

```
001 | #
002 | # This file is part of pysnmp software.
003 | #
004 | # Copyright (c) 2005-2019, Ilya Etingof <etingof@gmail.com>
005 | # License: http://snmplabs.com/pysnmp/license.html
006 | #
007 | # PySNMP MIB module RFC1213-MIB (http://snmplabs.com/pysnmp)
008 | # ASN.1 source http://mibs.snmplabs.com:80/asn1/RFC1213-MIB
009 | # Produced by pysmi-0.1.3 at Mon Apr 17 12:12:07 2017
010 | # On host grommit.local platform Darwin version 16.4.0 by user ilya
011 | # Using Python version 3.4.2 (v3.4.2:ab2c023a9432, Oct 5 2014, 20:42:22)
012 | #
013 | # It is a stripped version of MIB that contains only symbols that is
014 | # unique to SMIV1 and have no analogues in SMIV2
015 | #
016 | OctetString, ObjectIdentifier, Integer = mibBuilder.importSymbols("ASN1", "OctetString", "ObjectIdentifier")
017 | NamedValues, = mibBuilder.importSymbols("ASN1-ENUMERATION", "NamedValues")
018 | SingleValueConstraint, ConstraintsIntersection, ValueSizeConstraint, ConstraintsUnion, ValueRangeConstraint
019 | ModuleCompliance, NotificationGroup = mibBuilder.importSymbols("SNMPv2-CONF", "ModuleCompliance", "Notifica
020 | IpAddress, TimeTicks, NotificationType, ModuleIdentity, Integer32, Counter32, MibIdentifier, ObjectIdentity
021 | DisplayString, PhysAddress = mibBuilder.importSymbols("SNMPv2-TC", "DisplayString", "PhysAddress")
022 | from pysnmp.proto.rfc1155 import NetworkAddress
023 |
024 | at = MibIdentifier((1, 3, 6, 1, 2, 1, 3))
025 | ip = MibIdentifier((1, 3, 6, 1, 2, 1, 4))
026 | epg = MibIdentifier((1, 3, 6, 1, 2, 1, 8))
```

```

027 | atTable = MibTable((1, 3, 6, 1, 2, 1, 3, 1), )
028 | if mibBuilder.loadTexts: atTable.setStatus('deprecated')
029 | if mibBuilder.loadTexts: atTable.setDescription("The Address Translation tables contain the NetworkAddress
030 | atEntry = MibTableRow((1, 3, 6, 1, 2, 1, 3, 1, 1), ).setIndexNames((0, "RFC1213-MIB", "atIfIndex"), (0, "RF
031 | if mibBuilder.loadTexts: atEntry.setStatus('deprecated')
032 | if mibBuilder.loadTexts: atEntry.setDescription("Each entry contains one NetworkAddress to `physical' addre
033 | atIfIndex = MibTableColumn((1, 3, 6, 1, 2, 1, 3, 1, 1), Integer32()).setMaxAccess("readwrite")
034 | if mibBuilder.loadTexts: atIfIndex.setStatus('deprecated')
035 | if mibBuilder.loadTexts: atIfIndex.setDescription("The interface on which this entry's equivalence is effec
036 | atPhysAddress = MibTableColumn((1, 3, 6, 1, 2, 1, 3, 1, 1, 2), PhysAddress()).setMaxAccess("readwrite")
037 | if mibBuilder.loadTexts: atPhysAddress.setStatus('deprecated')
038 | if mibBuilder.loadTexts: atPhysAddress.setDescription("The media-dependent `physical' address. Setting this
039 | atNetAddress = MibTableColumn((1, 3, 6, 1, 2, 1, 3, 1, 1, 3), NetworkAddress()).setMaxAccess("readwrite")
040 | if mibBuilder.loadTexts: atNetAddress.setStatus('deprecated')
041 | if mibBuilder.loadTexts: atNetAddress.setDescription("The NetworkAddress (e.g., the IP address) correspondi
042 | ipForwarding = MibScalar((1, 3, 6, 1, 2, 1, 4, 1), Integer32().subtype(subtypeSpec=ConstraintsUnion(SingleV
043 | if mibBuilder.loadTexts: ipForwarding.setStatus('mandatory')
044 | if mibBuilder.loadTexts: ipForwarding.setDescription("The indication of whether this entity is acting as an
045 | ipDefaultTTL = MibScalar((1, 3, 6, 1, 2, 1, 4, 2), Integer32()).setMaxAccess("readwrite")
046 | if mibBuilder.loadTexts: ipDefaultTTL.setStatus('mandatory')
047 | if mibBuilder.loadTexts: ipDefaultTTL.setDescription('The default value inserted into the Time-To-Live fiel
048 | ipInReceives = MibScalar((1, 3, 6, 1, 2, 1, 4, 3), Counter32()).setMaxAccess("readonly")
049 | if mibBuilder.loadTexts: ipInReceives.setStatus('mandatory')
050 | if mibBuilder.loadTexts: ipInReceives.setDescription('The total number of input datagrams received from int
051 | ipInHdrErrors = MibScalar((1, 3, 6, 1, 2, 1, 4, 4), Counter32()).setMaxAccess("readonly")
052 | if mibBuilder.loadTexts: ipInHdrErrors.setStatus('mandatory')
053 | if mibBuilder.loadTexts: ipInHdrErrors.setDescription('The number of input datagrams discarded due to error
054 | ipInAddrErrors = MibScalar((1, 3, 6, 1, 2, 1, 4, 5), Counter32()).setMaxAccess("readonly")
055 | if mibBuilder.loadTexts: ipInAddrErrors.setStatus('mandatory')
056 | if mibBuilder.loadTexts: ipInAddrErrors.setDescription("The number of input datagrams discarded because the
057 | ipForwDatagrams = MibScalar((1, 3, 6, 1, 2, 1, 4, 6), Counter32()).setMaxAccess("readonly")
058 | if mibBuilder.loadTexts: ipForwDatagrams.setStatus('mandatory')
059 | if mibBuilder.loadTexts: ipForwDatagrams.setDescription('The number of input datagrams for which this entit
060 | ipInUnknownProtos = MibScalar((1, 3, 6, 1, 2, 1, 4, 7), Counter32()).setMaxAccess("readonly")
061 | if mibBuilder.loadTexts: ipInUnknownProtos.setStatus('mandatory')
062 | if mibBuilder.loadTexts: ipInUnknownProtos.setDescription('The number of locally-addressed datagrams receiv
063 | ipInDiscards = MibScalar((1, 3, 6, 1, 2, 1, 4, 8), Counter32()).setMaxAccess("readonly")
064 | if mibBuilder.loadTexts: ipInDiscards.setStatus('mandatory')
065 | if mibBuilder.loadTexts: ipInDiscards.setDescription('The number of input IP datagrams for which no problem
066 | ipInDelivers = MibScalar((1, 3, 6, 1, 2, 1, 4, 9), Counter32()).setMaxAccess("readonly")
067 | if mibBuilder.loadTexts: ipInDelivers.setStatus('mandatory')
068 | if mibBuilder.loadTexts: ipInDelivers.setDescription('The total number of input datagrams successfully deli
069 | ipOutRequests = MibScalar((1, 3, 6, 1, 2, 1, 4, 10), Counter32()).setMaxAccess("readonly")
070 | if mibBuilder.loadTexts: ipOutRequests.setStatus('mandatory')
071 | if mibBuilder.loadTexts: ipOutRequests.setDescription('The total number of IP datagrams which local IP user
072 | ipOutDiscards = MibScalar((1, 3, 6, 1, 2, 1, 4, 11), Counter32()).setMaxAccess("readonly")
073 | if mibBuilder.loadTexts: ipOutDiscards.setStatus('mandatory')
074 | if mibBuilder.loadTexts: ipOutDiscards.setDescription('The number of output IP datagrams for which no probl
075 | ipOutNoRoutes = MibScalar((1, 3, 6, 1, 2, 1, 4, 12), Counter32()).setMaxAccess("readonly")
076 | if mibBuilder.loadTexts: ipOutNoRoutes.setStatus('mandatory')
077 | if mibBuilder.loadTexts: ipOutNoRoutes.setDescription("The number of IP datagrams discarded because no rout
078 | ipReasmTimeout = MibScalar((1, 3, 6, 1, 2, 1, 4, 13), Integer32()).setMaxAccess("readonly")
079 | if mibBuilder.loadTexts: ipReasmTimeout.setStatus('mandatory')
080 | if mibBuilder.loadTexts: ipReasmTimeout.setDescription('The maximum number of seconds which received fragme
081 | ipReasmReqds = MibScalar((1, 3, 6, 1, 2, 1, 4, 14), Counter32()).setMaxAccess("readonly")
082 | if mibBuilder.loadTexts: ipReasmReqds.setStatus('mandatory')
083 | if mibBuilder.loadTexts: ipReasmReqds.setDescription('The number of IP fragments received which needed to b
084 | ipReasmOKs = MibScalar((1, 3, 6, 1, 2, 1, 4, 15), Counter32()).setMaxAccess("readonly")
085 | if mibBuilder.loadTexts: ipReasmOKs.setStatus('mandatory')
086 | if mibBuilder.loadTexts: ipReasmOKs.setDescription('The number of IP datagrams successfully re- assembled.'
087 | ipReasmFails = MibScalar((1, 3, 6, 1, 2, 1, 4, 16), Counter32()).setMaxAccess("readonly")
088 | if mibBuilder.loadTexts: ipReasmFails.setStatus('mandatory')
089 | if mibBuilder.loadTexts: ipReasmFails.setDescription('The number of failures detected by the IP re- assembl

```



```

090 | ipFragOKs = MibScalar((1, 3, 6, 1, 2, 1, 4, 17), Counter32()).setMaxAccess("readonly")
091 | if mibBuilder.loadTexts: ipFragOKs.setStatus('mandatory')
092 | if mibBuilder.loadTexts: ipFragOKs.setDescription('The number of IP datagrams that have been successfully f
093 | ipFragFails = MibScalar((1, 3, 6, 1, 2, 1, 4, 18), Counter32()).setMaxAccess("readonly")
094 | if mibBuilder.loadTexts: ipFragFails.setStatus('mandatory')
095 | if mibBuilder.loadTexts: ipFragFails.setDescription('The number of IP datagrams that have been discarded be
096 | ipFragCreates = MibScalar((1, 3, 6, 1, 2, 1, 4, 19), Counter32()).setMaxAccess("readonly")
097 | if mibBuilder.loadTexts: ipFragCreates.setStatus('mandatory')
098 | if mibBuilder.loadTexts: ipFragCreates.setDescription('The number of IP datagram fragments that have been g
099 | ipAddrTable = MibTable((1, 3, 6, 1, 2, 1, 4, 20), )
100 | if mibBuilder.loadTexts: ipAddrTable.setStatus('mandatory')
101 | if mibBuilder.loadTexts: ipAddrTable.setDescription('The table of addressing information relevant to this e
102 | ipAddrEntry = MibTableRow((1, 3, 6, 1, 2, 1, 4, 20, 1), ).setIndexNames((0, "RFC1213-MIB", "ipAdEntAddr"))
103 | if mibBuilder.loadTexts: ipAddrEntry.setStatus('mandatory')
104 | if mibBuilder.loadTexts: ipAddrEntry.setDescription('The addressing information for one of this entity's IP
105 | ipAdEntAddr = MibTableColumn((1, 3, 6, 1, 2, 1, 4, 20, 1, 1), IPAddress()).setMaxAccess("readonly")
106 | if mibBuilder.loadTexts: ipAdEntAddr.setStatus('mandatory')
107 | if mibBuilder.loadTexts: ipAdEntAddr.setDescription('The IP address to which this entry's addressing inform
108 | ipAdEntIfIndex = MibTableColumn((1, 3, 6, 1, 2, 1, 4, 20, 1, 2), Integer32()).setMaxAccess("readonly")
109 | if mibBuilder.loadTexts: ipAdEntIfIndex.setStatus('mandatory')
110 | if mibBuilder.loadTexts: ipAdEntIfIndex.setDescription('The index value which uniquely identifies the inter
111 | ipAdEntNetMask = MibTableColumn((1, 3, 6, 1, 2, 1, 4, 20, 1, 3), IPAddress()).setMaxAccess("readonly")
112 | if mibBuilder.loadTexts: ipAdEntNetMask.setStatus('mandatory')
113 | if mibBuilder.loadTexts: ipAdEntNetMask.setDescription('The subnet mask associated with the IP address of t
114 | ipAdEntBcastAddr = MibTableColumn((1, 3, 6, 1, 2, 1, 4, 20, 1, 4), Integer32()).setMaxAccess("readonly")
115 | if mibBuilder.loadTexts: ipAdEntBcastAddr.setStatus('mandatory')
116 | if mibBuilder.loadTexts: ipAdEntBcastAddr.setDescription('The value of the least-significant bit in the IP
117 | ipAdEntReasmMaxSize = MibTableColumn((1, 3, 6, 1, 2, 1, 4, 20, 1, 5), Integer32().subtype(subtypeSpec=Value
118 | if mibBuilder.loadTexts: ipAdEntReasmMaxSize.setStatus('mandatory')
119 | if mibBuilder.loadTexts: ipAdEntReasmMaxSize.setDescription('The size of the largest IP datagram which this
120 | ipRouteTable = MibTable((1, 3, 6, 1, 2, 1, 4, 21), )
121 | if mibBuilder.loadTexts: ipRouteTable.setStatus('mandatory')
122 | if mibBuilder.loadTexts: ipRouteTable.setDescription('This entity's IP Routing table.')
123 | ipRouteEntry = MibTableRow((1, 3, 6, 1, 2, 1, 4, 21, 1), ).setIndexNames((0, "RFC1213-MIB", "ipRouteDest"))
124 | if mibBuilder.loadTexts: ipRouteEntry.setStatus('mandatory')
125 | if mibBuilder.loadTexts: ipRouteEntry.setDescription('A route to a particular destination.')
126 | ipRouteDest = MibTableColumn((1, 3, 6, 1, 2, 1, 4, 21, 1, 1), IPAddress()).setMaxAccess("readwrite")
127 | if mibBuilder.loadTexts: ipRouteDest.setStatus('mandatory')
128 | if mibBuilder.loadTexts: ipRouteDest.setDescription('The destination IP address of this route. An entry wit
129 | ipRouteIfIndex = MibTableColumn((1, 3, 6, 1, 2, 1, 4, 21, 1, 2), Integer32()).setMaxAccess("readwrite")
130 | if mibBuilder.loadTexts: ipRouteIfIndex.setStatus('mandatory')
131 | if mibBuilder.loadTexts: ipRouteIfIndex.setDescription('The index value which uniquely identifies the local
132 | ipRouteMetric1 = MibTableColumn((1, 3, 6, 1, 2, 1, 4, 21, 1, 3), Integer32()).setMaxAccess("readwrite")
133 | if mibBuilder.loadTexts: ipRouteMetric1.setStatus('mandatory')
134 | if mibBuilder.loadTexts: ipRouteMetric1.setDescription('The primary routing metric for this route. The sema
135 | ipRouteMetric2 = MibTableColumn((1, 3, 6, 1, 2, 1, 4, 21, 1, 4), Integer32()).setMaxAccess("readwrite")
136 | if mibBuilder.loadTexts: ipRouteMetric2.setStatus('mandatory')
137 | if mibBuilder.loadTexts: ipRouteMetric2.setDescription('An alternate routing metric for this route. The sem
138 | ipRouteMetric3 = MibTableColumn((1, 3, 6, 1, 2, 1, 4, 21, 1, 5), Integer32()).setMaxAccess("readwrite")
139 | if mibBuilder.loadTexts: ipRouteMetric3.setStatus('mandatory')
140 | if mibBuilder.loadTexts: ipRouteMetric3.setDescription('An alternate routing metric for this route. The sem
141 | ipRouteMetric4 = MibTableColumn((1, 3, 6, 1, 2, 1, 4, 21, 1, 6), Integer32()).setMaxAccess("readwrite")
142 | if mibBuilder.loadTexts: ipRouteMetric4.setStatus('mandatory')
143 | if mibBuilder.loadTexts: ipRouteMetric4.setDescription('An alternate routing metric for this route. The sem
144 | ipRouteNextHop = MibTableColumn((1, 3, 6, 1, 2, 1, 4, 21, 1, 7), IPAddress()).setMaxAccess("readwrite")
145 | if mibBuilder.loadTexts: ipRouteNextHop.setStatus('mandatory')
146 | if mibBuilder.loadTexts: ipRouteNextHop.setDescription('The IP address of the next hop of this route. (In t
147 | ipRouteType = MibTableColumn((1, 3, 6, 1, 2, 1, 4, 21, 1, 8), Integer32().subtype(subtypeSpec=ConstraintsUn
148 | if mibBuilder.loadTexts: ipRouteType.setStatus('mandatory')
149 | if mibBuilder.loadTexts: ipRouteType.setDescription('The type of route. Note that the values direct(3) and
150 | ipRouteProto = MibTableColumn((1, 3, 6, 1, 2, 1, 4, 21, 1, 9), Integer32().subtype(subtypeSpec=ConstraintsU
151 | if mibBuilder.loadTexts: ipRouteProto.setStatus('mandatory')
152 | if mibBuilder.loadTexts: ipRouteProto.setDescription('The routing mechanism via which this route was learne

```

```

153 | ipRouteAge = MibTableColumn((1, 3, 6, 1, 2, 1, 4, 21, 1, 10), Integer32()).setMaxAccess("readwrite")
154 | if mibBuilder.loadTexts: ipRouteAge.setStatus('mandatory')
155 | if mibBuilder.loadTexts: ipRouteAge.setDescription("The number of seconds since this route was last updated")
156 | ipRouteMask = MibTableColumn((1, 3, 6, 1, 2, 1, 4, 21, 1, 11), IpAddress()).setMaxAccess("readwrite")
157 | if mibBuilder.loadTexts: ipRouteMask.setStatus('mandatory')
158 | if mibBuilder.loadTexts: ipRouteMask.setDescription('Indicate the mask to be logical-ANDed with the destination')
159 | ipRouteMetric5 = MibTableColumn((1, 3, 6, 1, 2, 1, 4, 21, 1, 12), Integer32()).setMaxAccess("readwrite")
160 | if mibBuilder.loadTexts: ipRouteMetric5.setStatus('mandatory')
161 | if mibBuilder.loadTexts: ipRouteMetric5.setDescription("An alternate routing metric for this route. The semantics are")
162 | ipRouteInfo = MibTableColumn((1, 3, 6, 1, 2, 1, 4, 21, 1, 13), ObjectIdentifier()).setMaxAccess("readonly")
163 | if mibBuilder.loadTexts: ipRouteInfo.setStatus('mandatory')
164 | if mibBuilder.loadTexts: ipRouteInfo.setDescription("A reference to MIB definitions specific to the particular")
165 | ipNetToMediaTable = MibTable((1, 3, 6, 1, 2, 1, 4, 22), )
166 | if mibBuilder.loadTexts: ipNetToMediaTable.setStatus('mandatory')
167 | if mibBuilder.loadTexts: ipNetToMediaTable.setDescription('The IP Address Translation table used for mapping')
168 | ipNetToMediaEntry = MibTableRow((1, 3, 6, 1, 2, 1, 4, 22, 1), ).setIndexNames((0, "RFC1213-MIB", "ipNetToMediaTable"))
169 | if mibBuilder.loadTexts: ipNetToMediaEntry.setStatus('mandatory')
170 | if mibBuilder.loadTexts: ipNetToMediaEntry.setDescription("Each entry contains one IpAddress to `physical'")
171 | ipNetToMediaIfIndex = MibTableColumn((1, 3, 6, 1, 2, 1, 4, 22, 1, 1), Integer32()).setMaxAccess("readwrite")
172 | if mibBuilder.loadTexts: ipNetToMediaIfIndex.setStatus('mandatory')
173 | if mibBuilder.loadTexts: ipNetToMediaIfIndex.setDescription("The interface on which this entry's equivalence")
174 | ipNetToMediaPhysAddress = MibTableColumn((1, 3, 6, 1, 2, 1, 4, 22, 1, 2), PhysAddress()).setMaxAccess("readonly")
175 | if mibBuilder.loadTexts: ipNetToMediaPhysAddress.setStatus('mandatory')
176 | if mibBuilder.loadTexts: ipNetToMediaPhysAddress.setDescription("The media-dependent `physical' address.")
177 | ipNetToMediaNetAddress = MibTableColumn((1, 3, 6, 1, 2, 1, 4, 22, 1, 3), IpAddress()).setMaxAccess("readwrite")
178 | if mibBuilder.loadTexts: ipNetToMediaNetAddress.setStatus('mandatory')
179 | if mibBuilder.loadTexts: ipNetToMediaNetAddress.setDescription("The IpAddress corresponding to the media-dependent")
180 | ipNetToMediaType = MibTableColumn((1, 3, 6, 1, 2, 1, 4, 22, 1, 4), Integer32().subtype(subtypeSpec=Constraint(1, 255)))
181 | if mibBuilder.loadTexts: ipNetToMediaType.setStatus('mandatory')
182 | if mibBuilder.loadTexts: ipNetToMediaType.setDescription('The type of mapping. Setting this object to the value')
183 | ipRoutingDiscards = MibScalar((1, 3, 6, 1, 2, 1, 4, 23), Counter32()).setMaxAccess("readonly")
184 | if mibBuilder.loadTexts: ipRoutingDiscards.setStatus('mandatory')
185 | if mibBuilder.loadTexts: ipRoutingDiscards.setDescription('The number of routing entries which were chosen')
186 | icmpInMsgs = MibScalar((1, 3, 6, 1, 2, 1, 5, 1), Counter32()).setMaxAccess("readonly")
187 | if mibBuilder.loadTexts: icmpInMsgs.setStatus('mandatory')
188 | if mibBuilder.loadTexts: icmpInMsgs.setDescription('The total number of ICMP messages which the entity received')
189 | icmpInErrors = MibScalar((1, 3, 6, 1, 2, 1, 5, 2), Counter32()).setMaxAccess("readonly")
190 | if mibBuilder.loadTexts: icmpInErrors.setStatus('mandatory')
191 | if mibBuilder.loadTexts: icmpInErrors.setDescription('The number of ICMP messages which the entity received')
192 | icmpInDestUnreachs = MibScalar((1, 3, 6, 1, 2, 1, 5, 3), Counter32()).setMaxAccess("readonly")
193 | if mibBuilder.loadTexts: icmpInDestUnreachs.setStatus('mandatory')
194 | if mibBuilder.loadTexts: icmpInDestUnreachs.setDescription('The number of ICMP Destination Unreachable messages received')
195 | icmpInTimeExcds = MibScalar((1, 3, 6, 1, 2, 1, 5, 4), Counter32()).setMaxAccess("readonly")
196 | if mibBuilder.loadTexts: icmpInTimeExcds.setStatus('mandatory')
197 | if mibBuilder.loadTexts: icmpInTimeExcds.setDescription('The number of ICMP Time Exceeded messages received')
198 | icmpInParmProbs = MibScalar((1, 3, 6, 1, 2, 1, 5, 5), Counter32()).setMaxAccess("readonly")
199 | if mibBuilder.loadTexts: icmpInParmProbs.setStatus('mandatory')
200 | if mibBuilder.loadTexts: icmpInParmProbs.setDescription('The number of ICMP Parameter Problem messages received')
201 | icmpInSrcQuenchs = MibScalar((1, 3, 6, 1, 2, 1, 5, 6), Counter32()).setMaxAccess("readonly")
202 | if mibBuilder.loadTexts: icmpInSrcQuenchs.setStatus('mandatory')
203 | if mibBuilder.loadTexts: icmpInSrcQuenchs.setDescription('The number of ICMP Source Quench messages received')
204 | icmpInRedirects = MibScalar((1, 3, 6, 1, 2, 1, 5, 7), Counter32()).setMaxAccess("readonly")
205 | if mibBuilder.loadTexts: icmpInRedirects.setStatus('mandatory')
206 | if mibBuilder.loadTexts: icmpInRedirects.setDescription('The number of ICMP Redirect messages received.')
207 | icmpInEchos = MibScalar((1, 3, 6, 1, 2, 1, 5, 8), Counter32()).setMaxAccess("readonly")
208 | if mibBuilder.loadTexts: icmpInEchos.setStatus('mandatory')
209 | if mibBuilder.loadTexts: icmpInEchos.setDescription('The number of ICMP Echo (request) messages received.')
210 | icmpInEchoReps = MibScalar((1, 3, 6, 1, 2, 1, 5, 9), Counter32()).setMaxAccess("readonly")
211 | if mibBuilder.loadTexts: icmpInEchoReps.setStatus('mandatory')
212 | if mibBuilder.loadTexts: icmpInEchoReps.setDescription('The number of ICMP Echo Reply messages received.')
213 | icmpInTimestamps = MibScalar((1, 3, 6, 1, 2, 1, 5, 10), Counter32()).setMaxAccess("readonly")
214 | if mibBuilder.loadTexts: icmpInTimestamps.setStatus('mandatory')
215 | if mibBuilder.loadTexts: icmpInTimestamps.setDescription('The number of ICMP Timestamp (request) messages received')

```

```

216 | icmpInTimestampReps = MibScalar((1, 3, 6, 1, 2, 1, 5, 11), Counter32()).setMaxAccess("readonly")
217 | if mibBuilder.loadTexts: icmpInTimestampReps.setStatus('mandatory')
218 | if mibBuilder.loadTexts: icmpInTimestampReps.setDescription('The number of ICMP Timestamp Reply messages received')
219 | icmpInAddrMasks = MibScalar((1, 3, 6, 1, 2, 1, 5, 12), Counter32()).setMaxAccess("readonly")
220 | if mibBuilder.loadTexts: icmpInAddrMasks.setStatus('mandatory')
221 | if mibBuilder.loadTexts: icmpInAddrMasks.setDescription('The number of ICMP Address Mask Request messages received')
222 | icmpInAddrMaskReps = MibScalar((1, 3, 6, 1, 2, 1, 5, 13), Counter32()).setMaxAccess("readonly")
223 | if mibBuilder.loadTexts: icmpInAddrMaskReps.setStatus('mandatory')
224 | if mibBuilder.loadTexts: icmpInAddrMaskReps.setDescription('The number of ICMP Address Mask Reply messages received')
225 | icmpOutMsgs = MibScalar((1, 3, 6, 1, 2, 1, 5, 14), Counter32()).setMaxAccess("readonly")
226 | if mibBuilder.loadTexts: icmpOutMsgs.setStatus('mandatory')
227 | if mibBuilder.loadTexts: icmpOutMsgs.setDescription('The total number of ICMP messages which this entity has sent')
228 | icmpOutErrors = MibScalar((1, 3, 6, 1, 2, 1, 5, 15), Counter32()).setMaxAccess("readonly")
229 | if mibBuilder.loadTexts: icmpOutErrors.setStatus('mandatory')
230 | if mibBuilder.loadTexts: icmpOutErrors.setDescription('The number of ICMP messages which this entity did not send')
231 | icmpOutDestUnreachs = MibScalar((1, 3, 6, 1, 2, 1, 5, 16), Counter32()).setMaxAccess("readonly")
232 | if mibBuilder.loadTexts: icmpOutDestUnreachs.setStatus('mandatory')
233 | if mibBuilder.loadTexts: icmpOutDestUnreachs.setDescription('The number of ICMP Destination Unreachable messages sent')
234 | icmpOutTimeExcds = MibScalar((1, 3, 6, 1, 2, 1, 5, 17), Counter32()).setMaxAccess("readonly")
235 | if mibBuilder.loadTexts: icmpOutTimeExcds.setStatus('mandatory')
236 | if mibBuilder.loadTexts: icmpOutTimeExcds.setDescription('The number of ICMP Time Exceeded messages sent')
237 | icmpOutParmProbs = MibScalar((1, 3, 6, 1, 2, 1, 5, 18), Counter32()).setMaxAccess("readonly")
238 | if mibBuilder.loadTexts: icmpOutParmProbs.setStatus('mandatory')
239 | if mibBuilder.loadTexts: icmpOutParmProbs.setDescription('The number of ICMP Parameter Problem messages sent')
240 | icmpOutSrcQuenchs = MibScalar((1, 3, 6, 1, 2, 1, 5, 19), Counter32()).setMaxAccess("readonly")
241 | if mibBuilder.loadTexts: icmpOutSrcQuenchs.setStatus('mandatory')
242 | if mibBuilder.loadTexts: icmpOutSrcQuenchs.setDescription('The number of ICMP Source Quench messages sent')
243 | icmpOutRedirects = MibScalar((1, 3, 6, 1, 2, 1, 5, 20), Counter32()).setMaxAccess("readonly")
244 | if mibBuilder.loadTexts: icmpOutRedirects.setStatus('mandatory')
245 | if mibBuilder.loadTexts: icmpOutRedirects.setDescription('The number of ICMP Redirect messages sent. For a')
246 | icmpOutEchos = MibScalar((1, 3, 6, 1, 2, 1, 5, 21), Counter32()).setMaxAccess("readonly")
247 | if mibBuilder.loadTexts: icmpOutEchos.setStatus('mandatory')
248 | if mibBuilder.loadTexts: icmpOutEchos.setDescription('The number of ICMP Echo (request) messages sent')
249 | icmpOutEchoReps = MibScalar((1, 3, 6, 1, 2, 1, 5, 22), Counter32()).setMaxAccess("readonly")
250 | if mibBuilder.loadTexts: icmpOutEchoReps.setStatus('mandatory')
251 | if mibBuilder.loadTexts: icmpOutEchoReps.setDescription('The number of ICMP Echo Reply messages sent')
252 | icmpOutTimestamps = MibScalar((1, 3, 6, 1, 2, 1, 5, 23), Counter32()).setMaxAccess("readonly")
253 | if mibBuilder.loadTexts: icmpOutTimestamps.setStatus('mandatory')
254 | if mibBuilder.loadTexts: icmpOutTimestamps.setDescription('The number of ICMP Timestamp (request) messages sent')
255 | icmpOutTimestampReps = MibScalar((1, 3, 6, 1, 2, 1, 5, 24), Counter32()).setMaxAccess("readonly")
256 | if mibBuilder.loadTexts: icmpOutTimestampReps.setStatus('mandatory')
257 | if mibBuilder.loadTexts: icmpOutTimestampReps.setDescription('The number of ICMP Timestamp Reply messages sent')
258 | icmpOutAddrMasks = MibScalar((1, 3, 6, 1, 2, 1, 5, 25), Counter32()).setMaxAccess("readonly")
259 | if mibBuilder.loadTexts: icmpOutAddrMasks.setStatus('mandatory')
260 | if mibBuilder.loadTexts: icmpOutAddrMasks.setDescription('The number of ICMP Address Mask Request messages sent')
261 | icmpOutAddrMaskReps = MibScalar((1, 3, 6, 1, 2, 1, 5, 26), Counter32()).setMaxAccess("readonly")
262 | if mibBuilder.loadTexts: icmpOutAddrMaskReps.setStatus('mandatory')
263 | if mibBuilder.loadTexts: icmpOutAddrMaskReps.setDescription('The number of ICMP Address Mask Reply messages sent')
264 | tcpRtoAlgorithm = MibScalar((1, 3, 6, 1, 2, 1, 6, 1), Integer32()).subtype(subtypeSpec=ConstraintsUnion(SingleValue, Range))
265 | if mibBuilder.loadTexts: tcpRtoAlgorithm.setStatus('mandatory')
266 | if mibBuilder.loadTexts: tcpRtoAlgorithm.setDescription('The algorithm used to determine the timeout value')
267 | tcpRtoMin = MibScalar((1, 3, 6, 1, 2, 1, 6, 2), Integer32()).setMaxAccess("readonly")
268 | if mibBuilder.loadTexts: tcpRtoMin.setStatus('mandatory')
269 | if mibBuilder.loadTexts: tcpRtoMin.setDescription('The minimum value permitted by a TCP implementation for')
270 | tcpRtoMax = MibScalar((1, 3, 6, 1, 2, 1, 6, 3), Integer32()).setMaxAccess("readonly")
271 | if mibBuilder.loadTexts: tcpRtoMax.setStatus('mandatory')
272 | if mibBuilder.loadTexts: tcpRtoMax.setDescription('The maximum value permitted by a TCP implementation for')
273 | tcpMaxConn = MibScalar((1, 3, 6, 1, 2, 1, 6, 4), Integer32()).setMaxAccess("readonly")
274 | if mibBuilder.loadTexts: tcpMaxConn.setStatus('mandatory')
275 | if mibBuilder.loadTexts: tcpMaxConn.setDescription('The limit on the total number of TCP connections the entity has')
276 | tcpActiveOpens = MibScalar((1, 3, 6, 1, 2, 1, 6, 5), Counter32()).setMaxAccess("readonly")
277 | if mibBuilder.loadTexts: tcpActiveOpens.setStatus('mandatory')
278 | if mibBuilder.loadTexts: tcpActiveOpens.setDescription('The number of times TCP connections have made a direct')

```

```

279 | tcpPassiveOpens = MibScalar((1, 3, 6, 1, 2, 1, 6, 6), Counter32()).setMaxAccess("readonly")
280 | if mibBuilder.loadTexts: tcpPassiveOpens.setStatus('mandatory')
281 | if mibBuilder.loadTexts: tcpPassiveOpens.setDescription('The number of times TCP connections have made a di
282 | tcpAttemptFails = MibScalar((1, 3, 6, 1, 2, 1, 6, 7), Counter32()).setMaxAccess("readonly")
283 | if mibBuilder.loadTexts: tcpAttemptFails.setStatus('mandatory')
284 | if mibBuilder.loadTexts: tcpAttemptFails.setDescription('The number of times TCP connections have made a di
285 | tcpEstabResets = MibScalar((1, 3, 6, 1, 2, 1, 6, 8), Counter32()).setMaxAccess("readonly")
286 | if mibBuilder.loadTexts: tcpEstabResets.setStatus('mandatory')
287 | if mibBuilder.loadTexts: tcpEstabResets.setDescription('The number of times TCP connections have made a di
288 | tcpCurrEstab = MibScalar((1, 3, 6, 1, 2, 1, 6, 9), Gauge32()).setMaxAccess("readonly")
289 | if mibBuilder.loadTexts: tcpCurrEstab.setStatus('mandatory')
290 | if mibBuilder.loadTexts: tcpCurrEstab.setDescription('The number of TCP connections for which the current s
291 | tcpInSegs = MibScalar((1, 3, 6, 1, 2, 1, 6, 10), Counter32()).setMaxAccess("readonly")
292 | if mibBuilder.loadTexts: tcpInSegs.setStatus('mandatory')
293 | if mibBuilder.loadTexts: tcpInSegs.setDescription('The total number of segments received, including those r
294 | tcpOutSegs = MibScalar((1, 3, 6, 1, 2, 1, 6, 11), Counter32()).setMaxAccess("readonly")
295 | if mibBuilder.loadTexts: tcpOutSegs.setStatus('mandatory')
296 | if mibBuilder.loadTexts: tcpOutSegs.setDescription('The total number of segments sent, including those on c
297 | tcpRetransSegs = MibScalar((1, 3, 6, 1, 2, 1, 6, 12), Counter32()).setMaxAccess("readonly")
298 | if mibBuilder.loadTexts: tcpRetransSegs.setStatus('mandatory')
299 | if mibBuilder.loadTexts: tcpRetransSegs.setDescription('The total number of segments retransmitted - that i
300 | tcpConnTable = MibTable((1, 3, 6, 1, 2, 1, 6, 13), )
301 | if mibBuilder.loadTexts: tcpConnTable.setStatus('mandatory')
302 | if mibBuilder.loadTexts: tcpConnTable.setDescription('A table containing TCP connection-specific informatio
303 | tcpConnEntry = MibTableRow((1, 3, 6, 1, 2, 1, 6, 13, 1), ).setIndexNames((0, "RFC1213-MIB", "tcpConnLocalAd
304 | if mibBuilder.loadTexts: tcpConnEntry.setStatus('mandatory')
305 | if mibBuilder.loadTexts: tcpConnEntry.setDescription('Information about a particular current TCP connection
306 | tcpConnState = MibTableColumn((1, 3, 6, 1, 2, 1, 6, 13, 1, 1), Integer32().subtype(subtypeSpec=ConstraintsU
307 | if mibBuilder.loadTexts: tcpConnState.setStatus('mandatory')
308 | if mibBuilder.loadTexts: tcpConnState.setDescription('The state of this TCP connection. The only value whic
309 | tcpConnLocalAddress = MibTableColumn((1, 3, 6, 1, 2, 1, 6, 13, 1, 2), IpAddress()).setMaxAccess("readonly")
310 | if mibBuilder.loadTexts: tcpConnLocalAddress.setStatus('mandatory')
311 | if mibBuilder.loadTexts: tcpConnLocalAddress.setDescription('The local IP address for this TCP connection.
312 | tcpConnLocalPort = MibTableColumn((1, 3, 6, 1, 2, 1, 6, 13, 1, 3), Integer32().subtype(subtypeSpec=ValueRan
313 | if mibBuilder.loadTexts: tcpConnLocalPort.setStatus('mandatory')
314 | if mibBuilder.loadTexts: tcpConnLocalPort.setDescription('The local port number for this TCP connection.')
315 | tcpConnRemAddress = MibTableColumn((1, 3, 6, 1, 2, 1, 6, 13, 1, 4), IpAddress()).setMaxAccess("readonly")
316 | if mibBuilder.loadTexts: tcpConnRemAddress.setStatus('mandatory')
317 | if mibBuilder.loadTexts: tcpConnRemAddress.setDescription('The remote IP address for this TCP connection.')
318 | tcpConnRemPort = MibTableColumn((1, 3, 6, 1, 2, 1, 6, 13, 1, 5), Integer32().subtype(subtypeSpec=ValueRange
319 | if mibBuilder.loadTexts: tcpConnRemPort.setStatus('mandatory')
320 | if mibBuilder.loadTexts: tcpConnRemPort.setDescription('The remote port number for this TCP connection.')
321 | tcpInErrs = MibScalar((1, 3, 6, 1, 2, 1, 6, 14), Counter32()).setMaxAccess("readonly")
322 | if mibBuilder.loadTexts: tcpInErrs.setStatus('mandatory')
323 | if mibBuilder.loadTexts: tcpInErrs.setDescription('The total number of segments received in error (e.g., ba
324 | tcpOutRsts = MibScalar((1, 3, 6, 1, 2, 1, 6, 15), Counter32()).setMaxAccess("readonly")
325 | if mibBuilder.loadTexts: tcpOutRsts.setStatus('mandatory')
326 | if mibBuilder.loadTexts: tcpOutRsts.setDescription('The number of TCP segments sent containing the RST flag
327 | udpInDatagrams = MibScalar((1, 3, 6, 1, 2, 1, 7, 1), Counter32()).setMaxAccess("readonly")
328 | if mibBuilder.loadTexts: udpInDatagrams.setStatus('mandatory')
329 | if mibBuilder.loadTexts: udpInDatagrams.setDescription('The total number of UDP datagrams delivered to UDP
330 | udpNoPorts = MibScalar((1, 3, 6, 1, 2, 1, 7, 2), Counter32()).setMaxAccess("readonly")
331 | if mibBuilder.loadTexts: udpNoPorts.setStatus('mandatory')
332 | if mibBuilder.loadTexts: udpNoPorts.setDescription('The total number of received UDP datagrams for which th
333 | udpInErrors = MibScalar((1, 3, 6, 1, 2, 1, 7, 3), Counter32()).setMaxAccess("readonly")
334 | if mibBuilder.loadTexts: udpInErrors.setStatus('mandatory')
335 | if mibBuilder.loadTexts: udpInErrors.setDescription('The number of received UDP datagrams that could not be
336 | udpOutDatagrams = MibScalar((1, 3, 6, 1, 2, 1, 7, 4), Counter32()).setMaxAccess("readonly")
337 | if mibBuilder.loadTexts: udpOutDatagrams.setStatus('mandatory')
338 | if mibBuilder.loadTexts: udpOutDatagrams.setDescription('The total number of UDP datagrams sent from this e
339 | udpTable = MibTable((1, 3, 6, 1, 2, 1, 7, 5), )
340 | if mibBuilder.loadTexts: udpTable.setStatus('mandatory')
341 | if mibBuilder.loadTexts: udpTable.setDescription('A table containing UDP listener information.')

```

```

342 | udpEntry = MibTableRow((1, 3, 6, 1, 2, 1, 7, 5, 1), ).setIndexNames((0, "RFC1213-MIB", "udpLocalAddress"),
343 | if mibBuilder.loadTexts: udpEntry.setStatus('mandatory')
344 | if mibBuilder.loadTexts: udpEntry.setDescription('Information about a particular current UDP listener.')
345 | udpLocalAddress = MibTableColumn((1, 3, 6, 1, 2, 1, 7, 5, 1, 1), IPAddress()).setMaxAccess("readonly")
346 | if mibBuilder.loadTexts: udpLocalAddress.setStatus('mandatory')
347 | if mibBuilder.loadTexts: udpLocalAddress.setDescription('The local IP address for this UDP listener. In the
348 | udpLocalPort = MibTableColumn((1, 3, 6, 1, 2, 1, 7, 5, 1, 2), Integer32().subtype(subtypeSpec=ValueRangeCon
349 | if mibBuilder.loadTexts: udpLocalPort.setStatus('mandatory')
350 | if mibBuilder.loadTexts: udpLocalPort.setDescription('The local port number for this UDP listener.')
351 | egpInMsgs = MibScalar((1, 3, 6, 1, 2, 1, 8, 1), Counter32()).setMaxAccess("readonly")
352 | if mibBuilder.loadTexts: egpInMsgs.setStatus('mandatory')
353 | if mibBuilder.loadTexts: egpInMsgs.setDescription('The number of EGP messages received without error.')
354 | egpInErrors = MibScalar((1, 3, 6, 1, 2, 1, 8, 2), Counter32()).setMaxAccess("readonly")
355 | if mibBuilder.loadTexts: egpInErrors.setStatus('mandatory')
356 | if mibBuilder.loadTexts: egpInErrors.setDescription('The number of EGP messages received that proved to be
357 | egpOutMsgs = MibScalar((1, 3, 6, 1, 2, 1, 8, 3), Counter32()).setMaxAccess("readonly")
358 | if mibBuilder.loadTexts: egpOutMsgs.setStatus('mandatory')
359 | if mibBuilder.loadTexts: egpOutMsgs.setDescription('The total number of locally generated EGP messages.')
360 | egpOutErrors = MibScalar((1, 3, 6, 1, 2, 1, 8, 4), Counter32()).setMaxAccess("readonly")
361 | if mibBuilder.loadTexts: egpOutErrors.setStatus('mandatory')
362 | if mibBuilder.loadTexts: egpOutErrors.setDescription('The number of locally generated EGP messages not sent
363 | egpNeighTable = MibTable((1, 3, 6, 1, 2, 1, 8, 5), )
364 | if mibBuilder.loadTexts: egpNeighTable.setStatus('mandatory')
365 | if mibBuilder.loadTexts: egpNeighTable.setDescription('The EGP neighbor table.')
366 | egpNeighEntry = MibTableRow((1, 3, 6, 1, 2, 1, 8, 5, 1), ).setIndexNames((0, "RFC1213-MIB", "egpNeighAddr")
367 | if mibBuilder.loadTexts: egpNeighEntry.setStatus('mandatory')
368 | if mibBuilder.loadTexts: egpNeighEntry.setDescription("Information about this entity's relationship with a
369 | egpNeighState = MibTableColumn((1, 3, 6, 1, 2, 1, 8, 5, 1, 1), Integer32().subtype(subtypeSpec=ConstraintsU
370 | if mibBuilder.loadTexts: egpNeighState.setStatus('mandatory')
371 | if mibBuilder.loadTexts: egpNeighState.setDescription("The EGP state of the local system with respect to th
372 | egpNeighAddr = MibTableColumn((1, 3, 6, 1, 2, 1, 8, 5, 1, 2), IPAddress()).setMaxAccess("readonly")
373 | if mibBuilder.loadTexts: egpNeighAddr.setStatus('mandatory')
374 | if mibBuilder.loadTexts: egpNeighAddr.setDescription("The IP address of this entry's EGP neighbor.")
375 | egpNeighAs = MibTableColumn((1, 3, 6, 1, 2, 1, 8, 5, 1, 3), Integer32()).setMaxAccess("readonly")
376 | if mibBuilder.loadTexts: egpNeighAs.setStatus('mandatory')
377 | if mibBuilder.loadTexts: egpNeighAs.setDescription('The autonomous system of this EGP peer. Zero should be
378 | egpNeighInMsgs = MibTableColumn((1, 3, 6, 1, 2, 1, 8, 5, 1, 4), Counter32()).setMaxAccess("readonly")
379 | if mibBuilder.loadTexts: egpNeighInMsgs.setStatus('mandatory')
380 | if mibBuilder.loadTexts: egpNeighInMsgs.setDescription('The number of EGP messages received without error f
381 | egpNeighInErrs = MibTableColumn((1, 3, 6, 1, 2, 1, 8, 5, 1, 5), Counter32()).setMaxAccess("readonly")
382 | if mibBuilder.loadTexts: egpNeighInErrs.setStatus('mandatory')
383 | if mibBuilder.loadTexts: egpNeighInErrs.setDescription('The number of EGP messages received from this EGP p
384 | egpNeighOutMsgs = MibTableColumn((1, 3, 6, 1, 2, 1, 8, 5, 1, 6), Counter32()).setMaxAccess("readonly")
385 | if mibBuilder.loadTexts: egpNeighOutMsgs.setStatus('mandatory')
386 | if mibBuilder.loadTexts: egpNeighOutMsgs.setDescription('The number of locally generated EGP messages to th
387 | egpNeighOutErrs = MibTableColumn((1, 3, 6, 1, 2, 1, 8, 5, 1, 7), Counter32()).setMaxAccess("readonly")
388 | if mibBuilder.loadTexts: egpNeighOutErrs.setStatus('mandatory')
389 | if mibBuilder.loadTexts: egpNeighOutErrs.setDescription('The number of locally generated EGP messages not s
390 | egpNeighInErrMsgs = MibTableColumn((1, 3, 6, 1, 2, 1, 8, 5, 1, 8), Counter32()).setMaxAccess("readonly")
391 | if mibBuilder.loadTexts: egpNeighInErrMsgs.setStatus('mandatory')
392 | if mibBuilder.loadTexts: egpNeighInErrMsgs.setDescription('The number of EGP-defined error messages receive
393 | egpNeighOutErrMsgs = MibTableColumn((1, 3, 6, 1, 2, 1, 8, 5, 1, 9), Counter32()).setMaxAccess("readonly")
394 | if mibBuilder.loadTexts: egpNeighOutErrMsgs.setStatus('mandatory')
395 | if mibBuilder.loadTexts: egpNeighOutErrMsgs.setDescription('The number of EGP-defined error messages sent t
396 | egpNeighStateUps = MibTableColumn((1, 3, 6, 1, 2, 1, 8, 5, 1, 10), Counter32()).setMaxAccess("readonly")
397 | if mibBuilder.loadTexts: egpNeighStateUps.setStatus('mandatory')
398 | if mibBuilder.loadTexts: egpNeighStateUps.setDescription('The number of EGP state transitions to the UP sta
399 | egpNeighStateDowns = MibTableColumn((1, 3, 6, 1, 2, 1, 8, 5, 1, 11), Counter32()).setMaxAccess("readonly")
400 | if mibBuilder.loadTexts: egpNeighStateDowns.setStatus('mandatory')
401 | if mibBuilder.loadTexts: egpNeighStateDowns.setDescription('The number of EGP state transitions from the UP
402 | egpNeighIntervalHello = MibTableColumn((1, 3, 6, 1, 2, 1, 8, 5, 1, 12), Integer32()).setMaxAccess("readonly")
403 | if mibBuilder.loadTexts: egpNeighIntervalHello.setStatus('mandatory')
404 | if mibBuilder.loadTexts: egpNeighIntervalHello.setDescription('The interval between EGP Hello command retra

```

```

405 | egpNeighIntervalPoll = MibTableColumn((1, 3, 6, 1, 2, 1, 8, 5, 1, 13), Integer32()).setMaxAccess("readonly")
406 | if mibBuilder.loadTexts: egpNeighIntervalPoll.setStatus('mandatory')
407 | if mibBuilder.loadTexts: egpNeighIntervalPoll.setDescription('The interval between EGP poll command retransmissions')
408 | egpNeighMode = MibTableColumn((1, 3, 6, 1, 2, 1, 8, 5, 1, 14), Integer32().subtype(subtypeSpec=ConstraintsUnion(ZeroOne, Integer32())))
409 | if mibBuilder.loadTexts: egpNeighMode.setStatus('mandatory')
410 | if mibBuilder.loadTexts: egpNeighMode.setDescription('The polling mode of this EGP entity, either passive or active')
411 | egpNeighEventTrigger = MibTableColumn((1, 3, 6, 1, 2, 1, 8, 5, 1, 15), Integer32().subtype(subtypeSpec=ConstraintsUnion(ZeroOne, Integer32())))
412 | if mibBuilder.loadTexts: egpNeighEventTrigger.setStatus('mandatory')
413 | if mibBuilder.loadTexts: egpNeighEventTrigger.setDescription('A control variable used to trigger operator-activated polling')
414 | egpAs = MibScalar((1, 3, 6, 1, 2, 1, 8, 6), Integer32()).setMaxAccess("readonly")
415 | if mibBuilder.loadTexts: egpAs.setStatus('mandatory')
416 | if mibBuilder.loadTexts: egpAs.setDescription('The autonomous system number of this EGP entity.')
417 | mibBuilder.exportSymbols("RFC1213-MIB", ipAdEntReasmMaxSize=ipAdEntReasmMaxSize, icmpOutAddrMasks=icmpOutAddrMasks)

```

=== ARQUIVO: .\dist\ControleImpressoras_internal\pysnmp\smi\mibs\SNMP-COMMUNITY-MIB.py ===

```

001 | #
002 | # This file is part of pysnmp software.
003 | #
004 | # Copyright (c) 2005-2019, Ilya Etingof <etingof@gmail.com>
005 | # License: http://snmplabs.com/pysnmp/license.html
006 | #
007 | # PySNMP MIB module SNMP-COMMUNITY-MIB (http://snmplabs.com/pysnmp)
008 | # ASN.1 source http://mibs.snmplabs.com:80/asn1/SNMP-COMMUNITY-MIB
009 | # Produced by pysmi-0.1.3 at Mon Apr 17 13:47:39 2017
010 | # On host grommit.local platform Darwin version 16.4.0 by user ilya
011 | # Using Python version 3.4.2 (v3.4.2:ab2c023a9432, Oct 5 2014, 20:42:22)
012 | #
013 | OctetString, ObjectIdentifier, Integer = mibBuilder.importSymbols("ASN1", "OctetString", "ObjectIdentifier")
014 | NamedValues, = mibBuilder.importSymbols("ASN1-ENUMERATION", "NamedValues")
015 | ConstraintsUnion, ConstraintsIntersection, ValueRangeConstraint, ValueSizeConstraint, SingleValueConstraint,
016 | SnmpAdminString, SnmpEngineID = mibBuilder.importSymbols("SNMP-FRAMEWORK-MIB", "SnmpAdminString", "SnmpEngineID",
017 | "snmpTargetAddrEntry", "SnmpTagValue")
018 | NotificationGroup, ObjectGroup, ModuleCompliance = mibBuilder.importSymbols("SNMPV2-CONF", "NotificationGroup", "ObjectGroup", "ModuleCompliance")
019 | Unsigned32, MibScalar, MibTable, MibTableRow, MibTableColumn, NotificationType, Bits, ModuleIdentity, TimeTicks,
020 | DisplayString, RowStatus, StorageType, TextualConvention = mibBuilder.importSymbols("SNMPV2-TC", "DisplayString", "TextualConvention", "RowStatus", "StorageType", "TimeTicks", "NotificationType", "NotificationGroup", "ObjectGroup", "ModuleCompliance")
021 | snmpCommunityMIB = ModuleIdentity((1, 3, 6, 1, 6, 3, 18))
022 | if mibBuilder.loadTexts: snmpCommunityMIB.setRevisions(('2003-08-06 00:00', '2000-03-06 00:00',))
023 | if mibBuilder.loadTexts: snmpCommunityMIB.setLastUpdated('200308060000Z')
024 | if mibBuilder.loadTexts: snmpCommunityMIB.setOrganization('SNMPv3 Working Group')
025 | if mibBuilder.loadTexts: snmpCommunityMIB.setContactInfo('WG-email: snmpv3@lists.tislabs.com Subscribe: majordomo@lists.tislabs.com')
026 | if mibBuilder.loadTexts: snmpCommunityMIB.setDescription('This MIB module defines objects to help support the SNMPv3 community string')
027 | snmpCommunityMIBObjects = MibIdentifier((1, 3, 6, 1, 6, 3, 18, 1))
028 | snmpCommunityMIBConformance = MibIdentifier((1, 3, 6, 1, 6, 3, 18, 2))
029 | snmpCommunityTable = MibTable((1, 3, 6, 1, 6, 3, 18, 1, 1), )
030 | if mibBuilder.loadTexts: snmpCommunityTable.setStatus('current')
031 | if mibBuilder.loadTexts: snmpCommunityTable.setDescription('The table of community strings configured in this MIB')
032 | snmpCommunityEntry = MibTableRow((1, 3, 6, 1, 6, 3, 18, 1, 1, 1), ).setIndexNames((1, "SNMP-COMMUNITY-MIB", "snmpCommunityIndex"))
033 | if mibBuilder.loadTexts: snmpCommunityEntry.setStatus('current')
034 | if mibBuilder.loadTexts: snmpCommunityEntry.setDescription('Information about a particular community string')
035 | snmpCommunityIndex = MibTableColumn((1, 3, 6, 1, 6, 3, 18, 1, 1, 1, 1), SnmpAdminString().subtype(subtypeSpec=ConstraintsUnion(ZeroOne, Integer32())))
036 | if mibBuilder.loadTexts: snmpCommunityIndex.setStatus('current')
037 | if mibBuilder.loadTexts: snmpCommunityIndex.setDescription('The unique index value of a row in this table.')
038 | snmpCommunityName = MibTableColumn((1, 3, 6, 1, 6, 3, 18, 1, 1, 1, 2), OctetString()).setMaxAccess("readcreate")
039 | if mibBuilder.loadTexts: snmpCommunityName.setStatus('current')
040 | if mibBuilder.loadTexts: snmpCommunityName.setDescription('The community string for which a row in this table is configured')
041 | snmpCommunitySecurityName = MibTableColumn((1, 3, 6, 1, 6, 3, 18, 1, 1, 1, 3), SnmpAdminString().subtype(subtypeSpec=ConstraintsUnion(ZeroOne, Integer32())))
042 | if mibBuilder.loadTexts: snmpCommunitySecurityName.setStatus('current')
043 | if mibBuilder.loadTexts: snmpCommunitySecurityName.setDescription('A human readable string representing the community string')

```



```

044 | snmpCommunityContextEngineID = MibTableColumn((1, 3, 6, 1, 6, 3, 18, 1, 1, 1, 4), SnmpEngineID()).setMaxAccess("noAccess")
045 | if mibBuilder.loadTexts: snmpCommunityContextEngineID.setStatus('current')
046 | if mibBuilder.loadTexts: snmpCommunityContextEngineID.setDescription('The contextEngineID indicating the location of the context')
047 | snmpCommunityContextName = MibTableColumn((1, 3, 6, 1, 6, 3, 18, 1, 1, 1, 5), SnmpAdminString().subtype(subtypeSpec=MaxSizeSpec(255)))
048 | if mibBuilder.loadTexts: snmpCommunityContextName.setStatus('current')
049 | if mibBuilder.loadTexts: snmpCommunityContextName.setDescription('The context in which management information is stored')
050 | snmpCommunityTransportTag = MibTableColumn((1, 3, 6, 1, 6, 3, 18, 1, 1, 1, 6), SnmpTagValue().clone(hexValue=0x00000000))
051 | if mibBuilder.loadTexts: snmpCommunityTransportTag.setStatus('current')
052 | if mibBuilder.loadTexts: snmpCommunityTransportTag.setDescription('This object specifies a set of transport parameters for this community')
053 | snmpCommunityStorageType = MibTableColumn((1, 3, 6, 1, 6, 3, 18, 1, 1, 1, 7), StorageType()).setMaxAccess("noAccess")
054 | if mibBuilder.loadTexts: snmpCommunityStorageType.setStatus('current')
055 | if mibBuilder.loadTexts: snmpCommunityStorageType.setDescription('The storage type for this conceptual row')
056 | snmpCommunityStatus = MibTableColumn((1, 3, 6, 1, 6, 3, 18, 1, 1, 1, 8), RowStatus()).setMaxAccess("readcreate")
057 | if mibBuilder.loadTexts: snmpCommunityStatus.setStatus('current')
058 | if mibBuilder.loadTexts: snmpCommunityStatus.setDescription('The status of this conceptual row in the snmpCommunityTable')
059 | snmpTargetAddrExtTable = MibTable((1, 3, 6, 1, 6, 3, 18, 1, 2), )
060 | if mibBuilder.loadTexts: snmpTargetAddrExtTable.setStatus('current')
061 | if mibBuilder.loadTexts: snmpTargetAddrExtTable.setDescription('The table of mask and maximum message size')
062 | snmpTargetAddrExtEntry = MibTableRow((1, 3, 6, 1, 6, 3, 18, 1, 2, 1), )
063 | snmpTargetAddrExtEntry.registerAugmentations(("SNMP-COMMUNITY-MIB", "snmpTargetAddrExtEntry"))
064 | snmpTargetAddrExtEntry.setIndexNames(*snmpTargetAddrExtEntry.getIndexNames())
065 | if mibBuilder.loadTexts: snmpTargetAddrExtEntry.setStatus('current')
066 | if mibBuilder.loadTexts: snmpTargetAddrExtEntry.setDescription('Information about a particular mask and maximum message size')
067 | snmpTargetAddrTMask = MibTableColumn((1, 3, 6, 1, 6, 3, 18, 1, 2, 1, 1), OctetString().subtype(subtypeSpec=MaxSizeSpec(255)))
068 | if mibBuilder.loadTexts: snmpTargetAddrTMask.setStatus('current')
069 | if mibBuilder.loadTexts: snmpTargetAddrTMask.setDescription('The mask value associated with an entry in the snmpTargetAddrExtTable')
070 | snmpTargetAddrMMS = MibTableColumn((1, 3, 6, 1, 6, 3, 18, 1, 2, 1, 2), Integer32().subtype(subtypeSpec=Constraints(0..2147483647)))
071 | if mibBuilder.loadTexts: snmpTargetAddrMMS.setStatus('current')
072 | if mibBuilder.loadTexts: snmpTargetAddrMMS.setDescription('The maximum message size value associated with an entry in the snmpTargetAddrExtTable')
073 | snmpTrapAddress = MibScalar((1, 3, 6, 1, 6, 3, 18, 1, 3), IpAddress()).setMaxAccess("accessiblefornotification")
074 | if mibBuilder.loadTexts: snmpTrapAddress.setStatus('current')
075 | if mibBuilder.loadTexts: snmpTrapAddress.setDescription('The value of the agent-addr field of a Trap PDU')
076 | snmpTrapCommunity = MibScalar((1, 3, 6, 1, 6, 3, 18, 1, 4), OctetString()).setMaxAccess("accessiblefornotification")
077 | if mibBuilder.loadTexts: snmpTrapCommunity.setStatus('current')
078 | if mibBuilder.loadTexts: snmpTrapCommunity.setDescription('The value of the community string field of an snmpTrap')
079 | snmpCommunityMIBCompliances = MibIdentifier((1, 3, 6, 1, 6, 3, 18, 2, 1))
080 | snmpCommunityMIBGroups = MibIdentifier((1, 3, 6, 1, 6, 3, 18, 2, 2))
081 | snmpCommunityMIBCompliance = ModuleCompliance((1, 3, 6, 1, 6, 3, 18, 2, 1, 1)).setObjects(("SNMP-COMMUNITY-MIB", "snmpCommunityMIBCompliance"))
082 | if mibBuilder.loadTexts: snmpCommunityMIBCompliance.setDescription('The compliance statement for SNMP engine')
083 | snmpProxyTrapForwardCompliance = ModuleCompliance((1, 3, 6, 1, 6, 3, 18, 2, 1, 2)).setObjects(("SNMP-COMMUNITY-MIB", "snmpProxyTrapForwardCompliance"))
084 | if mibBuilder.loadTexts: snmpProxyTrapForwardCompliance.setDescription('The compliance statement for SNMP engine')
085 | snmpCommunityMIBFullCompliance = ModuleCompliance((1, 3, 6, 1, 6, 3, 18, 2, 1, 3)).setObjects(("SNMP-COMMUNITY-MIB", "snmpCommunityMIBFullCompliance"))
086 | if mibBuilder.loadTexts: snmpCommunityMIBFullCompliance.setDescription('The compliance statement for SNMP engine')
087 | snmpCommunityTableGroup = ObjectGroup((1, 3, 6, 1, 6, 3, 18, 2, 2, 1)).setObjects(("SNMP-COMMUNITY-MIB", "snmpCommunityTableGroup"))
088 | if mibBuilder.loadTexts: snmpCommunityTableGroup.setDescription('A collection of objects providing for configuration')
089 | snmpProxyTrapForwardGroup = ObjectGroup((1, 3, 6, 1, 6, 3, 18, 2, 2, 3)).setObjects(("SNMP-COMMUNITY-MIB", "snmpProxyTrapForwardGroup"))
090 | if mibBuilder.loadTexts: snmpProxyTrapForwardGroup.setDescription('Objects which are used by proxy forwarding')
091 | mibBuilder.exportSymbols("SNMP-COMMUNITY-MIB", PYSNMP_MODULE_ID=snmpCommunityMIB, snmpCommunityMIBFullCompliance=snmpCommunityMIBFullCompliance)

```

=== ARQUIVO: .\dist\ControleImpressoras_internal\pysnmp\smi\mibs\SNMP-FRAMEWORK-MIB.py ===

```

001 | #
002 | # This file is part of pysnmp software.
003 | #
004 | # Copyright (c) 2005-2019, Ilya Etingof <etingof@gmail.com>
005 | # License: http://snmplabs.com/pysnmp/license.html
006 | #
007 | # PySNMP MIB module SNMP-FRAMEWORK-MIB (http://snmplabs.com/pysnmp)
008 | # ASN.1 source http://mibs.snmplabs.com:80/asn1/SNMP-FRAMEWORK-MIB

```

```

009 | # Produced by pysmi-0.1.3 at Mon Apr 17 13:59:41 2017
010 | # On host grommit.local platform Darwin version 16.4.0 by user ilya
011 | # Using Python version 3.4.2 (v3.4.2:ab2c023a9432, Oct 5 2014, 20:42:22)
012 | #
013 | import time
014 | try:
015 |     import os
016 | except ImportError:
017 |     pass
018 |
019 | OctetString, ObjectIdentifier, Integer = mibBuilder.importSymbols("ASN1", "OctetString", "ObjectIdentifier")
020 | NamedValues, = mibBuilder.importSymbols("ASN1-ENUMERATION", "NamedValues")
021 | ValueSizeConstraint, SingleValueConstraint, ValueRangeConstraint, ConstraintsUnion, ConstraintsIntersection
022 | ObjectGroup, NotificationGroup, ModuleCompliance = mibBuilder.importSymbols("SNMPv2-CONF", "ObjectGroup", "
023 | Gauge32, Unsigned32, MibIdentifier, ObjectIdentity, iso, Counter64, IpAddress, Counter32, ModuleIdentity, B
024 | DisplayString, TextualConvention = mibBuilder.importSymbols("SNMPv2-TC", "DisplayString", "TextualConventio
025 | snmpFrameworkMIB = ModuleIdentity((1, 3, 6, 1, 6, 3, 10))
026 | if mibBuilder.loadTexts: snmpFrameworkMIB.setRevisions(('2002-10-14 00:00', '1999-01-19 00:00', '1997-11-20
027 | if mibBuilder.loadTexts: snmpFrameworkMIB.setLastUpdated('200210140000Z')
028 | if mibBuilder.loadTexts: snmpFrameworkMIB.setOrganization('SNMPv3 Working Group')
029 | if mibBuilder.loadTexts: snmpFrameworkMIB.setContactInfo('WG-EMail: snmpv3@lists.tislabs.com Subscribe: snm
030 | if mibBuilder.loadTexts: snmpFrameworkMIB.setDescription('The SNMP Management Architecture MIB Copyright (C
031 |
032 | #
033 | # WARNING: some of the classes below are manually implemented
034 | #
035 |
036 | class SnmpAdminString(TextualConvention, OctetString):
037 |     description = 'An octet string containing administrative information, preferably in human-readable form
038 |     status = 'current'
039 |     displayHint = "255t"
040 |     subtypeSpec = OctetString.subtypeSpec + ValueSizeConstraint(0, 255)
041 |     encoding = 'utf-8'
042 |
043 |
044 | class SnmpEngineID(TextualConvention, OctetString):
045 |     description = "An SNMP engine's administratively-unique identifier. Objects of this type are for identi
046 |     status = 'current'
047 |     subtypeSpec = OctetString.subtypeSpec + ValueSizeConstraint(5, 32)
048 |     defaultValue = [128, 0, 79, 184, 5]
049 |     try:
050 |         # Attempt to base engine ID on local system name and properties
051 |         defaultValue += [ord(x) for x in os.uname()[1][:16]]
052 |     except Exception:
053 |         pass
054 |     try:
055 |         # Attempt to base engine ID on PID
056 |         defaultValue += [os.getpid() >> 8 & 0xff, os.getpid() & 0xff]
057 |     except Exception:
058 |         pass
059 |     # add pseudo-random text ID
060 |     defaultValue += [id(defaultValue) >> 8 & 0xff, id(defaultValue) & 0xff]
061 |     defaultValue = OctetString(defaultValue).asOctets()
062 |
063 |
064 | class SnmpEngineTime(Integer32):
065 |     def clone(self, *args, **kwargs):
066 |         if not args:
067 |             try:
068 |                 args = (time.time() - self,)
069 |             except Exception:
070 |                 pass
071 |         return Integer32.clone(self, *args, **kwargs)

```



```

072 |
073 |
074 | class SnmpSecurityModel(Integer32, TextualConvention):
075 |     description = "An identifier that uniquely identifies a Security Model of the Security Subsystem within
076 |     status = 'current'
077 |     subtypeSpec = Integer32.subtypeSpec + ValueRangeConstraint(0, 2147483647)
078 |
079 |
080 | class SnmpMessageProcessingModel(Integer32, TextualConvention):
081 |     description = 'An identifier that uniquely identifies a Message Processing Model of the Message Process
082 |     status = 'current'
083 |     subtypeSpec = Integer32.subtypeSpec + ValueRangeConstraint(0, 2147483647)
084 |
085 |
086 | class SnmpSecurityLevel(Integer32, TextualConvention):
087 |     description = 'A Level of Security at which SNMP messages can be sent or with which operations are bein
088 |     status = 'current'
089 |     subtypeSpec = Integer32.subtypeSpec + SingleValueConstraint(1, 2, 3, )
090 |     namedValues = NamedValues(("noAuthNoPriv", 1), ("authNoPriv", 2), ("authPriv", 3), )
091 |
092 | snmpFrameworkAdmin = MibIdentifier((1, 3, 6, 1, 6, 3, 10, 1))
093 | snmpFrameworkMIBObjects = MibIdentifier((1, 3, 6, 1, 6, 3, 10, 2))
094 | snmpFrameworkMIBConformance = MibIdentifier((1, 3, 6, 1, 6, 3, 10, 3))
095 | snmpEngine = MibIdentifier((1, 3, 6, 1, 6, 3, 10, 2, 1))
096 | snmpEngineID = MibScalar((1, 3, 6, 1, 6, 3, 10, 2, 1, 1), SnmpEngineID()).setMaxAccess("readonly")
097 | if mibBuilder.loadTexts: snmpEngineID.setStatus('current')
098 | if mibBuilder.loadTexts: snmpEngineID.setDescription("An SNMP engine's administratively-unique identifier.
099 | snmpEngineBoots = MibScalar((1, 3, 6, 1, 6, 3, 10, 2, 1, 2), Integer32().subtype(subtypeSpec=ValueRangeCons
100 | if mibBuilder.loadTexts: snmpEngineBoots.setStatus('current')
101 | if mibBuilder.loadTexts: snmpEngineBoots.setDescription('The number of times that the SNMP engine has (re-)
102 | snmpEngineTime = MibScalar((1, 3, 6, 1, 6, 3, 10, 2, 1, 3), SnmpEngineTime().subtype(subtypeSpec=ValueRange
103 | if mibBuilder.loadTexts: snmpEngineTime.setStatus('current')
104 | if mibBuilder.loadTexts: snmpEngineTime.setDescription("The number of seconds since the value of the snmpEn
105 | snmpEngineMaxMessageSize = MibScalar((1, 3, 6, 1, 6, 3, 10, 2, 1, 4), Integer32().subtype(subtypeSpec=Value
106 | if mibBuilder.loadTexts: snmpEngineMaxMessageSize.setStatus('current')
107 | if mibBuilder.loadTexts: snmpEngineMaxMessageSize.setDescription('The maximum length in octets of an SNMP m
108 | snmpAuthProtocols = ObjectIdentity((1, 3, 6, 1, 6, 3, 10, 1, 1))
109 | if mibBuilder.loadTexts: snmpAuthProtocols.setStatus('current')
110 | if mibBuilder.loadTexts: snmpAuthProtocols.setDescription('Registration point for standards-track authentic
111 | snmpPrivProtocols = ObjectIdentity((1, 3, 6, 1, 6, 3, 10, 1, 2))
112 | if mibBuilder.loadTexts: snmpPrivProtocols.setStatus('current')
113 | if mibBuilder.loadTexts: snmpPrivProtocols.setDescription('Registration point for standards-track privacy p
114 | snmpFrameworkMIBCompliances = MibIdentifier((1, 3, 6, 1, 6, 3, 10, 3, 1))
115 | snmpFrameworkMIBGroups = MibIdentifier((1, 3, 6, 1, 6, 3, 10, 3, 2))
116 | snmpFrameworkMIBCompliance = ModuleCompliance((1, 3, 6, 1, 6, 3, 10, 3, 1, 1)).setObjects(("SNMP-FRAMEWORK-
117 | if mibBuilder.loadTexts: snmpFrameworkMIBCompliance.setDescription('The compliance statement for SNMP engin
118 | snmpEngineGroup = ObjectGroup((1, 3, 6, 1, 6, 3, 10, 3, 2, 1)).setObjects(("SNMP-FRAMEWORK-MIB", "snmpEngin
119 | if mibBuilder.loadTexts: snmpEngineGroup.setDescription('A collection of objects for identifying and determ
120 | mibBuilder.exportSymbols("SNMP-FRAMEWORK-MIB", snmpFrameworkMIBConformance=snmpFrameworkMIBConformance, snm

```

=== ARQUIVO:

.\dist\ControleImpressoras_internal\pysnmp\smi\mibs\SNMP-MPD-MIB.py ===

```

001 | #
002 | # This file is part of pysnmp software.
003 | #
004 | # Copyright (c) 2005-2019, Ilya Etingof <etingof@gmail.com>
005 | # License: http://snmplabs.com/pysnmp/license.html
006 | #
007 | #

```

```

008 | # PySNMP MIB module SNMP-MPD-MIB (http://snmplabs.com/pysnmp)
009 | # ASN.1 source http://mibs.snmplabs.com:80/asn1/SNMP-MPD-MIB
010 | # Produced by pysmi-0.1.3 at Tue Apr 18 00:17:00 2017
011 | # On host grommit.local platform Darwin version 16.4.0 by user ilya
012 | # Using Python version 3.4.2 (v3.4.2:ab2c023a9432, Oct 5 2014, 20:42:22)
013 | #
014 | OctetString, ObjectIdentifier, Integer = mibBuilder.importSymbols("ASN1", "OctetString", "ObjectIdentifier")
015 | NamedValues, = mibBuilder.importSymbols("ASN1-ENUMERATION", "NamedValues")
016 | SingleValueConstraint, ConstraintsUnion, ValueSizeConstraint, ValueRangeConstraint, ConstraintsIntersection
017 | NotificationGroup, ModuleCompliance, ObjectGroup = mibBuilder.importSymbols("SNMPv2-CONF", "NotificationGro
018 | iso, Counter32, Unsigned32, Gauge32, MibIdentifier, TimeTicks, NotificationType, ObjectIdentity, Bits, Modu
019 | DisplayString, TextualConvention = mibBuilder.importSymbols("SNMPv2-TC", "DisplayString", "TextualConventio
020 | snmpMPDMIB = ModuleIdentity((1, 3, 6, 1, 6, 3, 11))
021 | if mibBuilder.loadTexts: snmpMPDMIB.setRevisions(('1999-05-04 16:36', '1997-09-30 00:00',))
022 | if mibBuilder.loadTexts: snmpMPDMIB.setLastUpdated('9905041636Z')
023 | if mibBuilder.loadTexts: snmpMPDMIB.setOrganization('SNMPv3 Working Group')
024 | if mibBuilder.loadTexts: snmpMPDMIB.setContactInfo('WG-EMail: snmpv3@lists.tislabs.com Subscribe: majordomo
025 | if mibBuilder.loadTexts: snmpMPDMIB.setDescription('The MIB for Message Processing and Dispatching')
026 | snmpMPDAdmin = MibIdentifier((1, 3, 6, 1, 6, 3, 11, 1))
027 | snmpMPDMIBObjects = MibIdentifier((1, 3, 6, 1, 6, 3, 11, 2))
028 | snmpMPDMIBConformance = MibIdentifier((1, 3, 6, 1, 6, 3, 11, 3))
029 | snmpMPDStats = MibIdentifier((1, 3, 6, 1, 6, 3, 11, 2, 1))
030 | snmpUnknownSecurityModels = MibScalar((1, 3, 6, 1, 6, 3, 11, 2, 1, 1), Counter32()).setMaxAccess("readonly")
031 | if mibBuilder.loadTexts: snmpUnknownSecurityModels.setStatus('current')
032 | if mibBuilder.loadTexts: snmpUnknownSecurityModels.setDescription('The total number of packets received by
033 | snmpInvalidMsgs = MibScalar((1, 3, 6, 1, 6, 3, 11, 2, 1, 2), Counter32()).setMaxAccess("readonly")
034 | if mibBuilder.loadTexts: snmpInvalidMsgs.setStatus('current')
035 | if mibBuilder.loadTexts: snmpInvalidMsgs.setDescription('The total number of packets received by the SNMP e
036 | snmpUnknownPDUHandlers = MibScalar((1, 3, 6, 1, 6, 3, 11, 2, 1, 3), Counter32()).setMaxAccess("readonly")
037 | if mibBuilder.loadTexts: snmpUnknownPDUHandlers.setStatus('current')
038 | if mibBuilder.loadTexts: snmpUnknownPDUHandlers.setDescription('The total number of packets received by the
039 | snmpMPDMIBCompliances = MibIdentifier((1, 3, 6, 1, 6, 3, 11, 3, 1))
040 | snmpMPDMIBGroups = MibIdentifier((1, 3, 6, 1, 6, 3, 11, 3, 2))
041 | snmpMPDCompliance = ModuleCompliance((1, 3, 6, 1, 6, 3, 11, 3, 1, 1)).setObjects(("SNMP-MPD-MIB", "snmpMPDG
042 | if mibBuilder.loadTexts: snmpMPDCompliance.setDescription('The compliance statement for SNMP entities which
043 | snmpMPDGroup = ObjectGroup((1, 3, 6, 1, 6, 3, 11, 3, 2, 1)).setObjects(("SNMP-MPD-MIB", "snmpUnknownSecurit
044 | if mibBuilder.loadTexts: snmpMPDGroup.setDescription('A collection of objects providing for remote monitori
045 | mibBuilder.exportSymbols("SNMP-MPD-MIB", snmpMPDMIBGroups=snmpMPDMIBGroups, PYSNMP_MODULE_ID=snmpMPDMIB, sn

```

=== ARQUIVO: .\dist\ControleImpressoras_internal\pysnmp\smi\mibs\SNMP-NOTIFICATION-MIB.py ===

```

001 | #
002 | # This file is part of pysnmp software.
003 | #
004 | # Copyright (c) 2005-2019, Ilya Etingof <etingof@gmail.com>
005 | # License: http://snmplabs.com/pysnmp/license.html
006 | #
007 | # PySNMP MIB module SNMP-NOTIFICATION-MIB (http://snmplabs.com/pysnmp)
008 | # ASN.1 source http://mibs.snmplabs.com:80/asn1/SNMP-NOTIFICATION-MIB
009 | # Produced by pysmi-0.1.3 at Tue Apr 18 00:41:37 2017
010 | # On host grommit.local platform Darwin version 16.4.0 by user ilya
011 | # Using Python version 3.4.2 (v3.4.2:ab2c023a9432, Oct 5 2014, 20:42:22)
012 | #
013 | OctetString, Integer, ObjectIdentifier = mibBuilder.importSymbols("ASN1", "OctetString", "Integer", "Object
014 | NamedValues, = mibBuilder.importSymbols("ASN1-ENUMERATION", "NamedValues")
015 | ValueRangeConstraint, ConstraintsIntersection, SingleValueConstraint, ValueSizeConstraint, ConstraintsUnion
016 | SnmpAdminString, = mibBuilder.importSymbols("SNMP-FRAMEWORK-MIB", "SnmpAdminString")
017 | snmpTargetParamsName, SnmpTagValue = mibBuilder.importSymbols("SNMP-TARGET-MIB", "snmpTargetParamsName", "S
018 | NotificationGroup, ObjectGroup, ModuleCompliance = mibBuilder.importSymbols("SNMPv2-CONF", "NotificationGro

```

```

019 | TimeTicks, IpAddress, Integer32, snmpModules, MibScalar, MibTable, MibTableRow, MibTableColumn, Bits, Count
020 | DisplayString, TextualConvention, RowStatus, StorageType = mibBuilder.importSymbols("SNMPv2-TC", "DisplaySt
021 | snmpNotificationMIB = ModuleIdentity((1, 3, 6, 1, 6, 3, 13))
022 | if mibBuilder.loadTexts: snmpNotificationMIB.setRevisions(('2002-10-14 00:00', '1998-08-04 00:00', '1997-07
023 | if mibBuilder.loadTexts: snmpNotificationMIB.setLastUpdated('200210140000Z')
024 | if mibBuilder.loadTexts: snmpNotificationMIB.setOrganization('IETF SNMPv3 Working Group')
025 | if mibBuilder.loadTexts: snmpNotificationMIB.setContactInfo('WG-email: snmpv3@lists.tislabs.com Subscribe:
026 | if mibBuilder.loadTexts: snmpNotificationMIB.setDescription('This MIB module defines MIB objects which prov
027 | snmpNotifyObjects = MibIdentifier((1, 3, 6, 1, 6, 3, 13, 1))
028 | snmpNotifyConformance = MibIdentifier((1, 3, 6, 1, 6, 3, 13, 3))
029 | snmpNotifyTable = MibTable((1, 3, 6, 1, 6, 3, 13, 1, 1), )
030 | if mibBuilder.loadTexts: snmpNotifyTable.setStatus('current')
031 | if mibBuilder.loadTexts: snmpNotifyTable.setDescription('This table is used to select management targets wh
032 | snmpNotifyEntry = MibTableRow((1, 3, 6, 1, 6, 3, 13, 1, 1, 1), ).setIndexNames((1, "SNMP-NOTIFICATION-MIB",
033 | if mibBuilder.loadTexts: snmpNotifyEntry.setStatus('current')
034 | if mibBuilder.loadTexts: snmpNotifyEntry.setDescription('An entry in this table selects a set of management
035 | snmpNotifyName = MibTableColumn((1, 3, 6, 1, 6, 3, 13, 1, 1, 1, 1), SnmpAdminString().subtype(subtypeSpec=V
036 | if mibBuilder.loadTexts: snmpNotifyName.setStatus('current')
037 | if mibBuilder.loadTexts: snmpNotifyName.setDescription('The locally arbitrary, but unique identifier associ
038 | snmpNotifyTag = MibTableColumn((1, 3, 6, 1, 6, 3, 13, 1, 1, 1, 2), SnmpTagValue()).setMaxAccess("readcreate
039 | if mibBuilder.loadTexts: snmpNotifyTag.setStatus('current')
040 | if mibBuilder.loadTexts: snmpNotifyTag.setDescription('This object contains a single tag value which is use
041 | snmpNotifyType = MibTableColumn((1, 3, 6, 1, 6, 3, 13, 1, 1, 1, 3), Integer32().subtype(subtypeSpec=Constra
042 | if mibBuilder.loadTexts: snmpNotifyType.setStatus('current')
043 | if mibBuilder.loadTexts: snmpNotifyType.setDescription('This object determines the type of notification to
044 | snmpNotifyStorageType = MibTableColumn((1, 3, 6, 1, 6, 3, 13, 1, 1, 1, 4), StorageType().clone('nonVolatile
045 | if mibBuilder.loadTexts: snmpNotifyStorageType.setStatus('current')
046 | if mibBuilder.loadTexts: snmpNotifyStorageType.setDescription('The storage type for this conceptual row. Co
047 | snmpNotifyRowStatus = MibTableColumn((1, 3, 6, 1, 6, 3, 13, 1, 1, 1, 5), RowStatus()).setMaxAccess("readcre
048 | if mibBuilder.loadTexts: snmpNotifyRowStatus.setStatus('current')
049 | if mibBuilder.loadTexts: snmpNotifyRowStatus.setDescription('The status of this conceptual row. To create a
050 | snmpNotifyFilterProfileTable = MibTable((1, 3, 6, 1, 6, 3, 13, 1, 2), )
051 | if mibBuilder.loadTexts: snmpNotifyFilterProfileTable.setStatus('current')
052 | if mibBuilder.loadTexts: snmpNotifyFilterProfileTable.setDescription('This table is used to associate a not
053 | snmpNotifyFilterProfileEntry = MibTableRow((1, 3, 6, 1, 6, 3, 13, 1, 2, 1), ).setIndexNames((1, "SNMP-TARGE
054 | if mibBuilder.loadTexts: snmpNotifyFilterProfileEntry.setStatus('current')
055 | if mibBuilder.loadTexts: snmpNotifyFilterProfileEntry.setDescription('An entry in this table indicates the
056 | snmpNotifyFilterProfileName = MibTableColumn((1, 3, 6, 1, 6, 3, 13, 1, 2, 1, 1), SnmpAdminString().subtype(
057 | if mibBuilder.loadTexts: snmpNotifyFilterProfileName.setStatus('current')
058 | if mibBuilder.loadTexts: snmpNotifyFilterProfileName.setDescription('The name of the filter profile to be u
059 | snmpNotifyFilterProfileStorageType = MibTableColumn((1, 3, 6, 1, 6, 3, 13, 1, 2, 1, 2), StorageType().clone('n
060 | if mibBuilder.loadTexts: snmpNotifyFilterProfileStorageType.setStatus('current')
061 | if mibBuilder.loadTexts: snmpNotifyFilterProfileStorageType.setDescription('The storage type for this conceptu
062 | snmpNotifyFilterProfileRowStatus = MibTableColumn((1, 3, 6, 1, 6, 3, 13, 1, 2, 1, 3), RowStatus()).setMaxAc
063 | if mibBuilder.loadTexts: snmpNotifyFilterProfileRowStatus.setStatus('current')
064 | if mibBuilder.loadTexts: snmpNotifyFilterProfileRowStatus.setDescription('The status of this conceptual row
065 | snmpNotifyFilterTable = MibTable((1, 3, 6, 1, 6, 3, 13, 1, 3), )
066 | if mibBuilder.loadTexts: snmpNotifyFilterTable.setStatus('current')
067 | if mibBuilder.loadTexts: snmpNotifyFilterTable.setDescription('The table of filter profiles. Filter profile
068 | snmpNotifyFilterEntry = MibTableRow((1, 3, 6, 1, 6, 3, 13, 1, 3, 1), ).setIndexNames((0, "SNMP-NOTIFICATION
069 | if mibBuilder.loadTexts: snmpNotifyFilterEntry.setStatus('current')
070 | if mibBuilder.loadTexts: snmpNotifyFilterEntry.setDescription('An element of a filter profile. Entries in t
071 | snmpNotifyFilterSubtree = MibTableColumn((1, 3, 6, 1, 6, 3, 13, 1, 3, 1, 1), ObjectIdentifier())
072 | if mibBuilder.loadTexts: snmpNotifyFilterSubtree.setStatus('current')
073 | if mibBuilder.loadTexts: snmpNotifyFilterSubtree.setDescription('The MIB subtree which, when combined with
074 | snmpNotifyFilterMask = MibTableColumn((1, 3, 6, 1, 6, 3, 13, 1, 3, 1, 2), OctetString().subtype(subtypeSpec
075 | if mibBuilder.loadTexts: snmpNotifyFilterMask.setStatus('current')
076 | if mibBuilder.loadTexts: snmpNotifyFilterMask.setDescription('The bit mask which, in combination with the c
077 | snmpNotifyFilterType = MibTableColumn((1, 3, 6, 1, 6, 3, 13, 1, 3, 1, 3), Integer32().subtype(subtypeSpec=C
078 | if mibBuilder.loadTexts: snmpNotifyFilterType.setStatus('current')
079 | if mibBuilder.loadTexts: snmpNotifyFilterType.setDescription('This object indicates whether the family of f
080 | snmpNotifyFilterStorageType = MibTableColumn((1, 3, 6, 1, 6, 3, 13, 1, 3, 1, 4), StorageType().clone('nonVo
081 | if mibBuilder.loadTexts: snmpNotifyFilterStorageType.setStatus('current')

```

```

0082 | if mibBuilder.loadTexts: snmpNotifyFilterStorageType.setDescription("The storage type for this conceptual r
0083 | snmpNotifyFilterRowStatus = MibTableColumn((1, 3, 6, 1, 6, 3, 13, 1, 3, 1, 5), RowStatus()).setMaxAccess("r
0084 | if mibBuilder.loadTexts: snmpNotifyFilterRowStatus.setStatus('current')
0085 | if mibBuilder.loadTexts: snmpNotifyFilterRowStatus.setDescription('The status of this conceptual row. To cr
0086 | snmpNotifyCompliances = MibIdentifier((1, 3, 6, 1, 6, 3, 13, 3, 1))
0087 | snmpNotifyGroups = MibIdentifier((1, 3, 6, 1, 6, 3, 13, 3, 2))
0088 | snmpNotifyBasicCompliance = ModuleCompliance((1, 3, 6, 1, 6, 3, 13, 3, 1, 1)).setObjects(("SNMP-TARGET-MIB"
0089 | if mibBuilder.loadTexts: snmpNotifyBasicCompliance.setDescription('The compliance statement for minimal SNM
0090 | snmpNotifyBasicFiltersCompliance = ModuleCompliance((1, 3, 6, 1, 6, 3, 13, 3, 1, 2)).setObjects(("SNMP-TARG
0091 | if mibBuilder.loadTexts: snmpNotifyBasicFiltersCompliance.setDescription('The compliance statement for SNMP
0092 | snmpNotifyFullCompliance = ModuleCompliance((1, 3, 6, 1, 6, 3, 13, 3, 1, 3)).setObjects(("SNMP-TARGET-MIB",
0093 | if mibBuilder.loadTexts: snmpNotifyFullCompliance.setDescription('The compliance statement for SNMP entitie
0094 | snmpNotifyGroup = ObjectGroup((1, 3, 6, 1, 6, 3, 13, 3, 2, 1)).setObjects(("SNMP-NOTIFICATION-MIB", "snmpNo
0095 | if mibBuilder.loadTexts: snmpNotifyGroup.setDescription('A collection of objects for selecting which manage
0096 | snmpNotifyFilterGroup = ObjectGroup((1, 3, 6, 1, 6, 3, 13, 3, 2, 2)).setObjects(("SNMP-NOTIFICATION-MIB", "
0097 | if mibBuilder.loadTexts: snmpNotifyFilterGroup.setDescription('A collection of objects providing remote con
0098 | mibBuilder.exportSymbols("SNMP-NOTIFICATION-MIB", PYSNMP_MODULE_ID=snmpNotificationMIB, snmpNotifyFilterPro

```

=== ARQUIVO:

.\dist\ControleImpressoras_internal\pysnmp\smi\mibs\SNMP-PROXY-MIB.py ===

```

0001 | #
0002 | # This file is part of pysnmp software.
0003 | #
0004 | # Copyright (c) 2005-2019, Ilya Etingof <etingof@gmail.com>
0005 | # License: http://snmplabs.com/pysnmp/license.html
0006 | #
0007 | # PySNMP MIB module SNMP-PROXY-MIB (http://snmplabs.com/pysnmp)
0008 | # ASN.1 source http://mibs.snmplabs.com:80/asn1/SNMP-PROXY-MIB
0009 | # Produced by pysmi-0.1.3 at Tue Apr 18 00:43:17 2017
0010 | # On host grommit.local platform Darwin version 16.4.0 by user ilya
0011 | # Using Python version 3.4.2 (v3.4.2:ab2c023a9432, Oct 5 2014, 20:42:22)
0012 | #
0013 | ObjectIdentifier, Integer, OctetString = mibBuilder.importSymbols("ASN1", "ObjectIdentifier", "Integer", "O
0014 | NamedValues, = mibBuilder.importSymbols("ASN1-ENUMERATION", "NamedValues")
0015 | SingleValueConstraint, ValueSizeConstraint, ConstraintsUnion, ValueRangeConstraint, ConstraintsIntersection
0016 | SnmpAdminString, SnmpEngineID = mibBuilder.importSymbols("SNMP-FRAMEWORK-MIB", "SnmpAdminString", "SnmpEngi
0017 | SnmpTagValue, = mibBuilder.importSymbols("SNMP-TARGET-MIB", "SnmpTagValue")
0018 | ModuleCompliance, NotificationGroup, ObjectGroup = mibBuilder.importSymbols("SNMPv2-CONF", "ModuleComplianc
0019 | ModuleIdentity, Counter64, Gauge32, Integer32, Bits, MibScalar, MibTable, MibTableRow, MibTableColumn, MibI
0020 | TextualConvention, RowStatus, DisplayString, StorageType = mibBuilder.importSymbols("SNMPv2-TC", "TextualCo
0021 | snmpProxyMIB = ModuleIdentity((1, 3, 6, 1, 6, 3, 14))
0022 | if mibBuilder.loadTexts: snmpProxyMIB.setRevisions(('1998-08-04 00:00', '1997-07-14 00:00',))
0023 | if mibBuilder.loadTexts: snmpProxyMIB.setLastUpdated('9808040000Z')
0024 | if mibBuilder.loadTexts: snmpProxyMIB.setOrganization('IETF SNMPv3 Working Group')
0025 | if mibBuilder.loadTexts: snmpProxyMIB.setContactInfo('WG-email: snmpv3@lists.tislabs.com Subscribe: majordomo
0026 | if mibBuilder.loadTexts: snmpProxyMIB.setDescription('This MIB module defines MIB objects which provide mec
0027 | snmpProxyObjects = MibIdentifier((1, 3, 6, 1, 6, 3, 14, 1))
0028 | snmpProxyConformance = MibIdentifier((1, 3, 6, 1, 6, 3, 14, 3))
0029 | snmpProxyTable = MibTable((1, 3, 6, 1, 6, 3, 14, 1, 2), )
0030 | if mibBuilder.loadTexts: snmpProxyTable.setStatus('current')
0031 | if mibBuilder.loadTexts: snmpProxyTable.setDescription('The table of translation parameters used by proxy f
0032 | snmpProxyEntry = MibTableRow((1, 3, 6, 1, 6, 3, 14, 1, 2, 1), ).setIndexNames((1, "SNMP-PROXY-MIB", "snmpPr
0033 | if mibBuilder.loadTexts: snmpProxyEntry.setStatus('current')
0034 | if mibBuilder.loadTexts: snmpProxyEntry.setDescription('A set of translation parameters used by a proxy for
0035 | snmpProxyName = MibTableColumn((1, 3, 6, 1, 6, 3, 14, 1, 2, 1, 1), SnmpAdminString().subtype(subtypeSpec=Va
0036 | if mibBuilder.loadTexts: snmpProxyName.setStatus('current')
0037 | if mibBuilder.loadTexts: snmpProxyName.setDescription('The locally arbitrary, but unique identifier associa
0038 | snmpProxyType = MibTableColumn((1, 3, 6, 1, 6, 3, 14, 1, 2, 1, 2), Integer32().subtype(subtypeSpec=Constrai
0039 | if mibBuilder.loadTexts: snmpProxyType.setStatus('current')

```

```

040 | if mibBuilder.loadTexts: snmpProxyType.setDescription('The type of message that may be forwarded using the
041 | snmpProxyContextEngineID = MibTableColumn((1, 3, 6, 1, 6, 3, 14, 1, 2, 1, 3), SnmpEngineID()).setMaxAccess(
042 | if mibBuilder.loadTexts: snmpProxyContextEngineID.setStatus('current')
043 | if mibBuilder.loadTexts: snmpProxyContextEngineID.setDescription('The contextEngineID contained in messages
044 | snmpProxyContextName = MibTableColumn((1, 3, 6, 1, 6, 3, 14, 1, 2, 1, 4), SnmpAdminString()).setMaxAccess("
045 | if mibBuilder.loadTexts: snmpProxyContextName.setStatus('current')
046 | if mibBuilder.loadTexts: snmpProxyContextName.setDescription('The contextName contained in messages that ma
047 | snmpProxyTargetParamsIn = MibTableColumn((1, 3, 6, 1, 6, 3, 14, 1, 2, 1, 5), SnmpAdminString()).setMaxAcces
048 | if mibBuilder.loadTexts: snmpProxyTargetParamsIn.setStatus('current')
049 | if mibBuilder.loadTexts: snmpProxyTargetParamsIn.setDescription('This object selects an entry in the snmpTa
050 | snmpProxySingleTargetOut = MibTableColumn((1, 3, 6, 1, 6, 3, 14, 1, 2, 1, 6), SnmpAdminString()).setMaxAcce
051 | if mibBuilder.loadTexts: snmpProxySingleTargetOut.setStatus('current')
052 | if mibBuilder.loadTexts: snmpProxySingleTargetOut.setDescription('This object selects a management target d
053 | snmpProxyMultipleTargetOut = MibTableColumn((1, 3, 6, 1, 6, 3, 14, 1, 2, 1, 7), SnmpTagValue()).setMaxAcces
054 | if mibBuilder.loadTexts: snmpProxyMultipleTargetOut.setStatus('current')
055 | if mibBuilder.loadTexts: snmpProxyMultipleTargetOut.setDescription('This object selects a set of management
056 | snmpProxyStorageType = MibTableColumn((1, 3, 6, 1, 6, 3, 14, 1, 2, 1, 8), StorageType()).clone('nonVolatile')
057 | if mibBuilder.loadTexts: snmpProxyStorageType.setStatus('current')
058 | if mibBuilder.loadTexts: snmpProxyStorageType.setDescription('The storage type of this conceptual row.')
059 | snmpProxyRowStatus = MibTableColumn((1, 3, 6, 1, 6, 3, 14, 1, 2, 1, 9), RowStatus()).setMaxAccess("readcrea
060 | if mibBuilder.loadTexts: snmpProxyRowStatus.setStatus('current')
061 | if mibBuilder.loadTexts: snmpProxyRowStatus.setDescription('The status of this conceptual row. To create a
062 | snmpProxyCompliances = MibIdentifier((1, 3, 6, 1, 6, 3, 14, 3, 1))
063 | snmpProxyGroups = MibIdentifier((1, 3, 6, 1, 6, 3, 14, 3, 2))
064 | snmpProxyCompliance = ModuleCompliance((1, 3, 6, 1, 6, 3, 14, 3, 1, 1)).setObjects(("SNMP-TARGET-MIB", "snm
065 | if mibBuilder.loadTexts: snmpProxyCompliance.setDescription('The compliance statement for SNMP entities whi
066 | snmpProxyGroup = ObjectGroup((1, 3, 6, 1, 6, 3, 14, 3, 2, 3)).setObjects(("SNMP-PROXY-MIB", "snmpProxyType"
067 | if mibBuilder.loadTexts: snmpProxyGroup.setDescription('A collection of objects providing remote configurat
068 | mibBuilder.exportSymbols("SNMP-PROXY-MIB", snmpProxyObjects=snmpProxyObjects, snmpProxyType=snmpProxyType,

```

=== ARQUIVO:

.\dist\ControleImpressoras_internal\pysnmp\smi\mibs\SNMP-TARGET-MIB.py

===

```

001 | #
002 | # This file is part of pysnmp software.
003 | #
004 | # Copyright (c) 2005-2019, Ilya Etingof <etingof@gmail.com>
005 | # License: http://snmplabs.com/pysnmp/license.html
006 | #
007 | # PySNMP MIB module SNMP-TARGET-MIB (http://snmplabs.com/pysnmp)
008 | # ASN.1 source http://mibs.snmplabs.com:80/asn1/SNMP-TARGET-MIB
009 | # Produced by pysmi-0.1.3 at Tue Apr 18 00:44:13 2017
010 | # On host grommit.local platform Darwin version 16.4.0 by user ilya
011 | # Using Python version 3.4.2 (v3.4.2:ab2c023a9432, Oct 5 2014, 20:42:22)
012 | #
013 | ObjectIdentifier, Integer, OctetString = mibBuilder.importSymbols("ASN1", "ObjectIdentifier", "Integer", "O
014 | NamedValues, = mibBuilder.importSymbols("ASN1-ENUMERATION", "NamedValues")
015 | ConstraintsIntersection, ValueSizeConstraint, ConstraintsUnion, SingleValueConstraint, ValueRangeConstraint
016 | SnmpAdminString, SnmpSecurityLevel, SnmpMessageProcessingModel, SnmpSecurityModel = mibBuilder.importSymbol
017 | ModuleCompliance, ObjectGroup, NotificationGroup = mibBuilder.importSymbols("SNMPv2-CONF", "ModuleComplianc
018 | MibIdentifier, MibScalar, MibTable, MibTableRow, MibTableColumn, iso, Counter32, ObjectIdentity, Bits, Noti
019 | TimeInterval, RowStatus, TextualConvention, TestAndIncr, TAddress, StorageType, TDomain, DisplayString = mi
020 | snmpTargetMIB = ModuleIdentity((1, 3, 6, 1, 6, 3, 12))
021 | if mibBuilder.loadTexts: snmpTargetMIB.setRevisions(('2002-10-14 00:00', '1998-08-04 00:00', '1997-07-14 00
022 | if mibBuilder.loadTexts: snmpTargetMIB.setLastUpdated('200210140000Z')
023 | if mibBuilder.loadTexts: snmpTargetMIB.setOrganization('IETF SNMPv3 Working Group')
024 | if mibBuilder.loadTexts: snmpTargetMIB.setContactInfo('WG-email: snmpv3@lists.tislabs.com Subscribe: majord
025 | if mibBuilder.loadTexts: snmpTargetMIB.setDescription('This MIB module defines MIB objects which provide me
026 | snmpTargetObjects = MibIdentifier((1, 3, 6, 1, 6, 3, 12, 1))

```



```

027 | snmpTargetConformance = MibIdentifier((1, 3, 6, 1, 6, 3, 12, 3))
028 |
029 | #
030 | # WARNING: some of the classes below are manually implemented
031 | #
032 |
033 | class SnmpTagList(TextualConvention, OctetString):
034 |     description = "An octet string containing a list of tag values. Tag values are preferably in human-readable form"
035 |     status = 'current'
036 |     displayHint = "255t"
037 |     subtypeSpec = OctetString.subtypeSpec + ValueSizeConstraint(0, 255)
038 |     encoding = 'utf-8'
039 |     _delimiters = (' ', '\n', '\t', '\t')
040 |
041 |     def prettyIn(self, value):
042 |         inDelim = True
043 |         for v in str(value):
044 |             if v in self._delimiters:
045 |                 if inDelim:
046 |                     raise error.SmiError('Leading or multiple delimiters not allowed in tag list %r' % value)
047 |                     inDelim = True
048 |                 else:
049 |                     inDelim = False
050 |             if value and inDelim:
051 |                 raise error.SmiError('Dangling delimiter not allowed in tag list %r' % value)
052 |         return OctetString.prettyIn(self, value)
053 |
054 |
055 | class SnmpTagValue(TextualConvention, OctetString):
056 |     description = "An octet string containing a tag value. Tag values are preferably in human-readable form"
057 |     status = 'current'
058 |     displayHint = "255t"
059 |     subtypeSpec = OctetString.subtypeSpec + ValueSizeConstraint(0, 255)
060 |     encoding = 'utf-8'
061 |     _delimiters = (' ', '\n', '\t', '\t')
062 |
063 |     def prettyIn(self, value):
064 |         for v in str(value):
065 |             if v in self._delimiters:
066 |                 raise error.SmiError('Delimiters not allowed in tag value')
067 |         return OctetString.prettyIn(self, value)
068 |
069 | snmpTargetSpinLock = MibScalar((1, 3, 6, 1, 6, 3, 12, 1, 1), TestAndIncr()).setMaxAccess("readwrite")
070 | if mibBuilder.loadTexts: snmpTargetSpinLock.setStatus('current')
071 | if mibBuilder.loadTexts: snmpTargetSpinLock.setDescription('This object is used to facilitate modification of the target')
072 | snmpTargetAddrTable = MibTable((1, 3, 6, 1, 6, 3, 12, 1, 2), )
073 | if mibBuilder.loadTexts: snmpTargetAddrTable.setStatus('current')
074 | if mibBuilder.loadTexts: snmpTargetAddrTable.setDescription('A table of transport addresses to be used in the generation of transport addresses')
075 | snmpTargetAddrEntry = MibTableRow((1, 3, 6, 1, 6, 3, 12, 1, 2, 1), ).setIndexNames((1, "SNMP-TARGET-MIB", "snmpTargetAddrTable"))
076 | if mibBuilder.loadTexts: snmpTargetAddrEntry.setStatus('current')
077 | if mibBuilder.loadTexts: snmpTargetAddrEntry.setDescription('A transport address to be used in the generation of transport addresses')
078 | snmpTargetAddrName = MibTableColumn((1, 3, 6, 1, 6, 3, 12, 1, 2, 1, 1), SnmpAdminString().subtype(subtypeSpec=MaximumSize(255))).setMaxAccess("readonly")
079 | if mibBuilder.loadTexts: snmpTargetAddrName.setStatus('current')
080 | if mibBuilder.loadTexts: snmpTargetAddrName.setDescription('The locally arbitrary, but unique identifier assigned to the transport address')
081 | snmpTargetAddrTDomain = MibTableColumn((1, 3, 6, 1, 6, 3, 12, 1, 2, 1, 2), TDomain()).setMaxAccess("readcreate")
082 | if mibBuilder.loadTexts: snmpTargetAddrTDomain.setStatus('current')
083 | if mibBuilder.loadTexts: snmpTargetAddrTDomain.setDescription('This object indicates the transport type of the transport address')
084 | snmpTargetAddrTAddress = MibTableColumn((1, 3, 6, 1, 6, 3, 12, 1, 2, 1, 3), TAddress()).setMaxAccess("readcreate")
085 | if mibBuilder.loadTexts: snmpTargetAddrTAddress.setStatus('current')
086 | if mibBuilder.loadTexts: snmpTargetAddrTAddress.setDescription('This object contains a transport address. The transport address is used to generate transport addresses')
087 | snmpTargetAddrTimeout = MibTableColumn((1, 3, 6, 1, 6, 3, 12, 1, 2, 1, 4), TimeInterval().clone(1500)).setMaxAccess("readcreate")
088 | if mibBuilder.loadTexts: snmpTargetAddrTimeout.setStatus('current')
089 | if mibBuilder.loadTexts: snmpTargetAddrTimeout.setDescription('This object should reflect the expected maximum time to receive a response')

```

```

090 | snmpTargetAddrRetryCount = MibTableColumn((1, 3, 6, 1, 6, 3, 12, 1, 2, 1, 5), Integer32().subtype(subtypeSp
091 | if mibBuilder.loadTexts: snmpTargetAddrRetryCount.setStatus('current')
092 | if mibBuilder.loadTexts: snmpTargetAddrRetryCount.setDescription('This object specifies a default number of
093 | snmpTargetAddrTagList = MibTableColumn((1, 3, 6, 1, 6, 3, 12, 1, 2, 1, 6), SnmpTagList()).setMaxAccess("rea
094 | if mibBuilder.loadTexts: snmpTargetAddrTagList.setStatus('current')
095 | if mibBuilder.loadTexts: snmpTargetAddrTagList.setDescription('This object contains a list of tag values wh
096 | snmpTargetAddrParams = MibTableColumn((1, 3, 6, 1, 6, 3, 12, 1, 2, 1, 7), SnmpAdminString().subtype(subtype
097 | if mibBuilder.loadTexts: snmpTargetAddrParams.setStatus('current')
098 | if mibBuilder.loadTexts: snmpTargetAddrParams.setDescription('The value of this object identifies an entry
099 | snmpTargetAddrStorageType = MibTableColumn((1, 3, 6, 1, 6, 3, 12, 1, 2, 1, 8), StorageType().clone('nonVola
100 | if mibBuilder.loadTexts: snmpTargetAddrStorageType.setStatus('current')
101 | if mibBuilder.loadTexts: snmpTargetAddrStorageType.setDescription('The storage type for this conceptual row
102 | snmpTargetAddrRowStatus = MibTableColumn((1, 3, 6, 1, 6, 3, 12, 1, 2, 1, 9), RowStatus()).setMaxAccess("rea
103 | if mibBuilder.loadTexts: snmpTargetAddrRowStatus.setStatus('current')
104 | if mibBuilder.loadTexts: snmpTargetAddrRowStatus.setDescription('The status of this conceptual row. To crea
105 | snmpTargetParamsTable = MibTable((1, 3, 6, 1, 6, 3, 12, 1, 3), )
106 | if mibBuilder.loadTexts: snmpTargetParamsTable.setStatus('current')
107 | if mibBuilder.loadTexts: snmpTargetParamsTable.setDescription('A table of SNMP target information to be use
108 | snmpTargetParamsEntry = MibTableRow((1, 3, 6, 1, 6, 3, 12, 1, 3, 1), ).setIndexNames((1, "SNMP-TARGET-MIB",
109 | if mibBuilder.loadTexts: snmpTargetParamsEntry.setStatus('current')
110 | if mibBuilder.loadTexts: snmpTargetParamsEntry.setDescription('A set of SNMP target information. Entries in
111 | snmpTargetParamsName = MibTableColumn((1, 3, 6, 1, 6, 3, 12, 1, 3, 1, 1), SnmpAdminString().subtype(subtype
112 | if mibBuilder.loadTexts: snmpTargetParamsName.setStatus('current')
113 | if mibBuilder.loadTexts: snmpTargetParamsName.setDescription('The locally arbitrary, but unique identifier
114 | snmpTargetParamsMPModel = MibTableColumn((1, 3, 6, 1, 6, 3, 12, 1, 3, 1, 2), SnmpMessageProcessingModel()).
115 | if mibBuilder.loadTexts: snmpTargetParamsMPModel.setStatus('current')
116 | if mibBuilder.loadTexts: snmpTargetParamsMPModel.setDescription('The Message Processing Model to be used wh
117 | snmpTargetParamsSecurityModel = MibTableColumn((1, 3, 6, 1, 6, 3, 12, 1, 3, 1, 3), SnmpSecurityModel().subt
118 | if mibBuilder.loadTexts: snmpTargetParamsSecurityModel.setStatus('current')
119 | if mibBuilder.loadTexts: snmpTargetParamsSecurityModel.setDescription('The Security Model to be used when g
120 | snmpTargetParamsSecurityName = MibTableColumn((1, 3, 6, 1, 6, 3, 12, 1, 3, 1, 4), SnmpAdminString()).setMax
121 | if mibBuilder.loadTexts: snmpTargetParamsSecurityName.setStatus('current')
122 | if mibBuilder.loadTexts: snmpTargetParamsSecurityName.setDescription('The securityName which identifies the
123 | snmpTargetParamsSecurityLevel = MibTableColumn((1, 3, 6, 1, 6, 3, 12, 1, 3, 1, 5), SnmpSecurityLevel()).set
124 | if mibBuilder.loadTexts: snmpTargetParamsSecurityLevel.setStatus('current')
125 | if mibBuilder.loadTexts: snmpTargetParamsSecurityLevel.setDescription('The Level of Security to be used whe
126 | snmpTargetParamsStorageType = MibTableColumn((1, 3, 6, 1, 6, 3, 12, 1, 3, 1, 6), StorageType().clone('nonVo
127 | if mibBuilder.loadTexts: snmpTargetParamsStorageType.setStatus('current')
128 | if mibBuilder.loadTexts: snmpTargetParamsStorageType.setDescription('The storage type for this conceptual r
129 | snmpTargetParamsRowStatus = MibTableColumn((1, 3, 6, 1, 6, 3, 12, 1, 3, 1, 7), RowStatus()).setMaxAccess("r
130 | if mibBuilder.loadTexts: snmpTargetParamsRowStatus.setStatus('current')
131 | if mibBuilder.loadTexts: snmpTargetParamsRowStatus.setDescription('The status of this conceptual row. To cr
132 | snmpUnavailableContexts = MibScalar((1, 3, 6, 1, 6, 3, 12, 1, 4), Counter32()).setMaxAccess("readonly")
133 | if mibBuilder.loadTexts: snmpUnavailableContexts.setStatus('current')
134 | if mibBuilder.loadTexts: snmpUnavailableContexts.setDescription('The total number of packets received by th
135 | snmpUnknownContexts = MibScalar((1, 3, 6, 1, 6, 3, 12, 1, 5), Counter32()).setMaxAccess("readonly")
136 | if mibBuilder.loadTexts: snmpUnknownContexts.setStatus('current')
137 | if mibBuilder.loadTexts: snmpUnknownContexts.setDescription('The total number of packets received by the SN
138 | snmpTargetCompliances = MibIdentifier((1, 3, 6, 1, 6, 3, 12, 3, 1))
139 | snmpTargetGroups = MibIdentifier((1, 3, 6, 1, 6, 3, 12, 3, 2))
140 | snmpTargetCommandResponderCompliance = ModuleCompliance((1, 3, 6, 1, 6, 3, 12, 3, 1, 1)).setObjects(("SNMP-
141 | if mibBuilder.loadTexts: snmpTargetCommandResponderCompliance.setDescription('The compliance statement for
142 | snmpTargetBasicGroup = ObjectGroup((1, 3, 6, 1, 6, 3, 12, 3, 2, 1)).setObjects(("SNMP-TARGET-MIB", "snmpTar
143 | if mibBuilder.loadTexts: snmpTargetBasicGroup.setDescription('A collection of objects providing basic remot
144 | snmpTargetResponseGroup = ObjectGroup((1, 3, 6, 1, 6, 3, 12, 3, 2, 2)).setObjects(("SNMP-TARGET-MIB", "snmp
145 | if mibBuilder.loadTexts: snmpTargetResponseGroup.setDescription('A collection of objects providing remote c
146 | snmpTargetCommandResponderGroup = ObjectGroup((1, 3, 6, 1, 6, 3, 12, 3, 2, 3)).setObjects(("SNMP-TARGET-MIB
147 | if mibBuilder.loadTexts: snmpTargetCommandResponderGroup.setDescription('A collection of objects required f
148 | mibBuilder.exportSymbols("SNMP-TARGET-MIB", snmpTargetAddrTimeout=snmpTargetAddrTimeout, SnmpTagValue=SnmpT

```

=== ARQUIVO: .\dist\ControleImpressoras_internal\pysnmp\smi\mibs\SNMP-USER-BASED-SM-3DES-MIB.py ===

```
001 | #
002 | # This file is part of pysnmp software.
003 | #
004 | # Copyright (c) 2005-2019, Ilya Etingof <etingof@gmail.com>
005 | # License: http://snmplabs.com/pysnmp/license.html
006 | #
007 | # PySNMP MIB module SNMP-USER-BASED-SM-3DES-MIB (http://snmplabs.com/pysnmp)
008 | # ASN.1 source http://mibs.snmplabs.com:80/asn1/SNMP-USER-BASED-SM-3DES-MIB
009 | # Produced by pysmi-0.1.3 at Tue Apr 18 00:47:21 2017
010 | # On host grommit.local platform Darwin version 16.4.0 by user ilya
011 | # Using Python version 3.4.2 (v3.4.2:ab2c023a9432, Oct 5 2014, 20:42:22)
012 | #
013 | Integer, ObjectIdentifier, OctetString = mibBuilder.importSymbols("ASN1", "Integer", "ObjectIdentifier", "O
014 | NamedValues, = mibBuilder.importSymbols("ASN1-ENUMERATION", "NamedValues")
015 | ValueRangeConstraint, ConstraintsUnion, ValueSizeConstraint, SingleValueConstraint, ConstraintsIntersection
016 | snmpPrivProtocols, = mibBuilder.importSymbols("SNMP-FRAMEWORK-MIB", "snmpPrivProtocols")
017 | ModuleCompliance, NotificationGroup = mibBuilder.importSymbols("SNMPv2-CONF", "ModuleCompliance", "Notifica
018 | Bits, MibScalar, MibTable, MibTableRow, MibTableColumn, Gauge32, IpAddress, TimeTicks, Integer32, Counter64
019 | TextualConvention, AutonomousType, DisplayString = mibBuilder.importSymbols("SNMPv2-TC", "TextualConvention
020 | snmpUsmMIB = ModuleIdentity((1, 3, 6, 1, 6, 3, 15))
021 | if mibBuilder.loadTexts: snmpUsmMIB.setRevisions(('1999-10-06 00:00',))
022 | if mibBuilder.loadTexts: snmpUsmMIB.setLastUpdated('9910060000Z')
023 | if mibBuilder.loadTexts: snmpUsmMIB.setOrganization('SNMPv3 Working Group')
024 | if mibBuilder.loadTexts: snmpUsmMIB.setContactInfo('WG-email: snmpv3@lists.tislabs.com Subscribe: majordomo
025 | if mibBuilder.loadTexts: snmpUsmMIB.setDescription('Extension to the SNMP User-based Security Model to supp
026 | usm3DESEDEPrivProtocol = ObjectIdentity((1, 3, 6, 1, 6, 3, 10, 1, 2, 3))
027 | if mibBuilder.loadTexts: usm3DESEDEPrivProtocol.setStatus('current')
028 | if mibBuilder.loadTexts: usm3DESEDEPrivProtocol.setDescription('The 3DES-EDE Symmetric Encryption Protocol
029 | if mibBuilder.loadTexts: usm3DESEDEPrivProtocol.setReference('- Data Encryption Standard, National Institut
030 | mibBuilder.exportSymbols("SNMP-USER-BASED-SM-3DES-MIB", snmpUsmMIB=snmpUsmMIB, PYSNMP_MODULE_ID=snmpUsmMIB,
```

=== ARQUIVO: .\dist\ControleImpressoras_internal\pysnmp\smi\mibs\SNMP-USER-BASED-SM-MIB.py ===

```
001 | #
002 | # This file is part of pysnmp software.
003 | #
004 | # Copyright (c) 2005-2019, Ilya Etingof <etingof@gmail.com>
005 | # License: http://snmplabs.com/pysnmp/license.html
006 | #
007 | # PySNMP MIB module SNMP-USER-BASED-SM-MIB (http://snmplabs.com/pysnmp)
008 | # ASN.1 source http://mibs.snmplabs.com:80/asn1/SNMP-USER-BASED-SM-MIB
009 | # Produced by pysmi-0.1.3 at Tue Apr 18 00:48:12 2017
010 | # On host grommit.local platform Darwin version 16.4.0 by user ilya
011 | # Using Python version 3.4.2 (v3.4.2:ab2c023a9432, Oct 5 2014, 20:42:22)
012 | #
013 | OctetString, Integer, ObjectIdentifier = mibBuilder.importSymbols("ASN1", "OctetString", "Integer", "Object
014 | NamedValues, = mibBuilder.importSymbols("ASN1-ENUMERATION", "NamedValues")
015 | ValueRangeConstraint, SingleValueConstraint, ValueSizeConstraint, ConstraintsIntersection, ConstraintsUnion
016 | SnmpAdminString, snmpAuthProtocols, snmpPrivProtocols, SnmpEngineID = mibBuilder.importSymbols("SNMP-FRAMEW
017 | ObjectGroup, ModuleCompliance, NotificationGroup = mibBuilder.importSymbols("SNMPv2-CONF", "ObjectGroup", "
018 | MibScalar, MibTable, MibTableRow, MibTableColumn, Counter64, Gauge32, iso, Counter32, NotificationType, Bit
019 | DisplayString, AutonomousType, RowStatus, StorageType, TextualConvention, RowPointer, TestAndIncr = mibBuil
020 | snmpUsmMIB = ModuleIdentity((1, 3, 6, 1, 6, 3, 15))
021 | if mibBuilder.loadTexts: snmpUsmMIB.setRevisions(('2002-10-16 00:00', '1999-01-20 00:00', '1997-11-20 00:00
022 | if mibBuilder.loadTexts: snmpUsmMIB.setLastUpdated('200210160000Z')
```



```

023 | if mibBuilder.loadTexts: snmpUsmMIB.setOrganization('SNMPv3 Working Group')
024 | if mibBuilder.loadTexts: snmpUsmMIB.setContactInfo('WG-email: snmpv3@lists.tislabs.com Subscribe: majordomo
025 | if mibBuilder.loadTexts: snmpUsmMIB.setDescription('The management information definitions for the SNMP Use
026 | usmMIBObjects = MibIdentifier((1, 3, 6, 1, 6, 3, 15, 1))
027 | usmMIBConformance = MibIdentifier((1, 3, 6, 1, 6, 3, 15, 2))
028 | usmNoAuthProtocol = ObjectIdentity((1, 3, 6, 1, 6, 3, 10, 1, 1, 1))
029 | if mibBuilder.loadTexts: usmNoAuthProtocol.setStatus('current')
030 | if mibBuilder.loadTexts: usmNoAuthProtocol.setDescription('No Authentication Protocol.')
031 | usmHMACMD5AuthProtocol = ObjectIdentity((1, 3, 6, 1, 6, 3, 10, 1, 1, 2))
032 | if mibBuilder.loadTexts: usmHMACMD5AuthProtocol.setStatus('current')
033 | if mibBuilder.loadTexts: usmHMACMD5AuthProtocol.setDescription('The HMAC-MD5-96 Digest Authentication Proto
034 | if mibBuilder.loadTexts: usmHMACMD5AuthProtocol.setReference('- H. Krawczyk, M. Bellare, R. Canetti HMAC: K
035 | usmHMACSHAAuthProtocol = ObjectIdentity((1, 3, 6, 1, 6, 3, 10, 1, 1, 3))
036 | if mibBuilder.loadTexts: usmHMACSHAAuthProtocol.setStatus('current')
037 | if mibBuilder.loadTexts: usmHMACSHAAuthProtocol.setDescription('The HMAC-SHA-96 Digest Authentication Proto
038 | if mibBuilder.loadTexts: usmHMACSHAAuthProtocol.setReference('- H. Krawczyk, M. Bellare, R. Canetti, HMAC:
039 | usmNoPrivProtocol = ObjectIdentity((1, 3, 6, 1, 6, 3, 10, 1, 2, 1))
040 | if mibBuilder.loadTexts: usmNoPrivProtocol.setStatus('current')
041 | if mibBuilder.loadTexts: usmNoPrivProtocol.setDescription('No Privacy Protocol.')
042 | usmDESPrivProtocol = ObjectIdentity((1, 3, 6, 1, 6, 3, 10, 1, 2, 2))
043 | if mibBuilder.loadTexts: usmDESPrivProtocol.setStatus('current')
044 | if mibBuilder.loadTexts: usmDESPrivProtocol.setDescription('The CBC-DES Symmetric Encryption Protocol.')
045 | if mibBuilder.loadTexts: usmDESPrivProtocol.setReference('- Data Encryption Standard, National Institute of
046 | class KeyChange(TextualConvention, OctetString):
047 |     description = "Every definition of an object with this syntax must identify a protocol P, a secret key
048 |     status = 'current'
049 |
050 | usmStats = MibIdentifier((1, 3, 6, 1, 6, 3, 15, 1, 1))
051 | usmStatsUnsupportedSecLevels = MibScalar((1, 3, 6, 1, 6, 3, 15, 1, 1, 1), Counter32()).setMaxAccess("readon
052 | if mibBuilder.loadTexts: usmStatsUnsupportedSecLevels.setStatus('current')
053 | if mibBuilder.loadTexts: usmStatsUnsupportedSecLevels.setDescription('The total number of packets received
054 | usmStatsNotInTimeWindows = MibScalar((1, 3, 6, 1, 6, 3, 15, 1, 1, 2), Counter32()).setMaxAccess("readonly")
055 | if mibBuilder.loadTexts: usmStatsNotInTimeWindows.setStatus('current')
056 | if mibBuilder.loadTexts: usmStatsNotInTimeWindows.setDescription("The total number of packets received by t
057 | usmStatsUnknownUserNames = MibScalar((1, 3, 6, 1, 6, 3, 15, 1, 1, 3), Counter32()).setMaxAccess("readonly")
058 | if mibBuilder.loadTexts: usmStatsUnknownUserNames.setStatus('current')
059 | if mibBuilder.loadTexts: usmStatsUnknownUserNames.setDescription('The total number of packets received by t
060 | usmStatsUnknownEngineIDs = MibScalar((1, 3, 6, 1, 6, 3, 15, 1, 1, 4), Counter32()).setMaxAccess("readonly")
061 | if mibBuilder.loadTexts: usmStatsUnknownEngineIDs.setStatus('current')
062 | if mibBuilder.loadTexts: usmStatsUnknownEngineIDs.setDescription('The total number of packets received by t
063 | usmStatsWrongDigests = MibScalar((1, 3, 6, 1, 6, 3, 15, 1, 1, 5), Counter32()).setMaxAccess("readonly")
064 | if mibBuilder.loadTexts: usmStatsWrongDigests.setStatus('current')
065 | if mibBuilder.loadTexts: usmStatsWrongDigests.setDescription("The total number of packets received by the S
066 | usmStatsDecryptionErrors = MibScalar((1, 3, 6, 1, 6, 3, 15, 1, 1, 6), Counter32()).setMaxAccess("readonly")
067 | if mibBuilder.loadTexts: usmStatsDecryptionErrors.setStatus('current')
068 | if mibBuilder.loadTexts: usmStatsDecryptionErrors.setDescription('The total number of packets received by t
069 | usmUser = MibIdentifier((1, 3, 6, 1, 6, 3, 15, 1, 2))
070 | usmUserSpinLock = MibScalar((1, 3, 6, 1, 6, 3, 15, 1, 2, 1), TestAndIncr()).setMaxAccess("readwrite")
071 | if mibBuilder.loadTexts: usmUserSpinLock.setStatus('current')
072 | if mibBuilder.loadTexts: usmUserSpinLock.setDescription('An advisory lock used to allow several cooperating
073 | usmUserTable = MibTable((1, 3, 6, 1, 6, 3, 15, 1, 2, 2), )
074 | if mibBuilder.loadTexts: usmUserTable.setStatus('current')
075 | if mibBuilder.loadTexts: usmUserTable.setDescription("The table of users configured in the SNMP engine's Lo
076 | usmUserEntry = MibTableRow((1, 3, 6, 1, 6, 3, 15, 1, 2, 2, 1), ).setIndexNames((0, "SNMP-USER-BASED-SM-MIB"
077 | if mibBuilder.loadTexts: usmUserEntry.setStatus('current')
078 | if mibBuilder.loadTexts: usmUserEntry.setDescription("A user configured in the SNMP engine's Local Configur
079 | usmUserEngineID = MibTableColumn((1, 3, 6, 1, 6, 3, 15, 1, 2, 2, 1, 1), SnmpEngineID())
080 | if mibBuilder.loadTexts: usmUserEngineID.setStatus('current')
081 | if mibBuilder.loadTexts: usmUserEngineID.setDescription("An SNMP engine's administratively-unique identifie
082 | usmUserName = MibTableColumn((1, 3, 6, 1, 6, 3, 15, 1, 2, 2, 1, 2), SnmpAdminString().subtype(subtypeSpec=V
083 | if mibBuilder.loadTexts: usmUserName.setStatus('current')
084 | if mibBuilder.loadTexts: usmUserName.setDescription('A human readable string representing the name of the u
085 | usmUserSecurityName = MibTableColumn((1, 3, 6, 1, 6, 3, 15, 1, 2, 2, 1, 3), SnmpAdminString()).setMaxAccess

```

```

0086 | if mibBuilder.loadTexts: usmUserSecurityName.setStatus('current')
0087 | if mibBuilder.loadTexts: usmUserSecurityName.setDescription('A human readable string representing the user
0088 | usmUserCloneFrom = MibTableColumn((1, 3, 6, 1, 6, 3, 15, 1, 2, 2, 1, 4), RowPointer()).setMaxAccess("readcr
0089 | if mibBuilder.loadTexts: usmUserCloneFrom.setStatus('current')
0090 | if mibBuilder.loadTexts: usmUserCloneFrom.setDescription("A pointer to another conceptual row in this usmUS
0091 | usmUserAuthProtocol = MibTableColumn((1, 3, 6, 1, 6, 3, 15, 1, 2, 2, 1, 5), AutonomousType().clone((1, 3, 6
0092 | if mibBuilder.loadTexts: usmUserAuthProtocol.setStatus('current')
0093 | if mibBuilder.loadTexts: usmUserAuthProtocol.setDescription("An indication of whether messages sent on beha
0094 | usmUserAuthKeyChange = MibTableColumn((1, 3, 6, 1, 6, 3, 15, 1, 2, 2, 1, 6), KeyChange().clone(hexValue="")
0095 | if mibBuilder.loadTexts: usmUserAuthKeyChange.setStatus('current')
0096 | if mibBuilder.loadTexts: usmUserAuthKeyChange.setDescription("An object, which when modified, causes the se
0097 | usmUserOwnAuthKeyChange = MibTableColumn((1, 3, 6, 1, 6, 3, 15, 1, 2, 2, 1, 7), KeyChange().clone(hexValue=
0098 | if mibBuilder.loadTexts: usmUserOwnAuthKeyChange.setStatus('current')
0099 | if mibBuilder.loadTexts: usmUserOwnAuthKeyChange.setDescription("Behaves exactly as usmUserAuthKeyChange, w
0100 | usmUserPrivProtocol = MibTableColumn((1, 3, 6, 1, 6, 3, 15, 1, 2, 2, 1, 8), AutonomousType().clone((1, 3, 6
0101 | if mibBuilder.loadTexts: usmUserPrivProtocol.setStatus('current')
0102 | if mibBuilder.loadTexts: usmUserPrivProtocol.setDescription("An indication of whether messages sent on beha
0103 | usmUserPrivKeyChange = MibTableColumn((1, 3, 6, 1, 6, 3, 15, 1, 2, 2, 1, 9), KeyChange().clone(hexValue="")
0104 | if mibBuilder.loadTexts: usmUserPrivKeyChange.setStatus('current')
0105 | if mibBuilder.loadTexts: usmUserPrivKeyChange.setDescription("An object, which when modified, causes the se
0106 | usmUserOwnPrivKeyChange = MibTableColumn((1, 3, 6, 1, 6, 3, 15, 1, 2, 2, 1, 10), KeyChange().clone(hexValue
0107 | if mibBuilder.loadTexts: usmUserOwnPrivKeyChange.setStatus('current')
0108 | if mibBuilder.loadTexts: usmUserOwnPrivKeyChange.setDescription("Behaves exactly as usmUserPrivKeyChange, w
0109 | usmUserPublic = MibTableColumn((1, 3, 6, 1, 6, 3, 15, 1, 2, 2, 1, 11), OctetString().subtype(subtypeSpec=Va
0110 | if mibBuilder.loadTexts: usmUserPublic.setStatus('current')
0111 | if mibBuilder.loadTexts: usmUserPublic.setDescription("A publicly-readable value which can be written as pa
0112 | usmUserStorageType = MibTableColumn((1, 3, 6, 1, 6, 3, 15, 1, 2, 2, 1, 12), StorageType().clone('nonVolatil
0113 | if mibBuilder.loadTexts: usmUserStorageType.setStatus('current')
0114 | if mibBuilder.loadTexts: usmUserStorageType.setDescription("The storage type for this conceptual row. Conce
0115 | usmUserStatus = MibTableColumn((1, 3, 6, 1, 6, 3, 15, 1, 2, 2, 1, 13), RowStatus()).setMaxAccess("readcreat
0116 | if mibBuilder.loadTexts: usmUserStatus.setStatus('current')
0117 | if mibBuilder.loadTexts: usmUserStatus.setDescription("The status of this conceptual row. Until instances o
0118 | usmMIBCompliances = MibIdentifier((1, 3, 6, 1, 6, 3, 15, 2, 1))
0119 | usmMIBGroups = MibIdentifier((1, 3, 6, 1, 6, 3, 15, 2, 2))
0120 | usmMIBCompliance = ModuleCompliance((1, 3, 6, 1, 6, 3, 15, 2, 1, 1)).setObjects(("SNMP-USER-BASED-SM-MIB",
0121 | if mibBuilder.loadTexts: usmMIBCompliance.setDescription('The compliance statement for SNMP engines which i
0122 | usmMIBBasicGroup = ObjectGroup((1, 3, 6, 1, 6, 3, 15, 2, 2, 1)).setObjects(("SNMP-USER-BASED-SM-MIB", "usmS
0123 | if mibBuilder.loadTexts: usmMIBBasicGroup.setDescription('A collection of objects providing for configurati
0124 | mibBuilder.exportSymbols("SNMP-USER-BASED-SM-MIB", usmUserStorageType=usmUserStorageType, PYSNMP_MODULE_ID=

```

=== ARQUIVO:

.\dist\ControleImpressoras_internal\pysnmp\smi\mibs\SNMP-USM-AES-MIB.py

===

```

0001 | #
0002 | # This file is part of pysnmp software.
0003 | #
0004 | # Copyright (c) 2005-2019, Ilya Etingof <etingof@gmail.com>
0005 | # License: http://snmplabs.com/pysnmp/license.html
0006 | #
0007 | # PySNMP MIB module SNMP-USM-AES-MIB (http://snmplabs.com/pysnmp)
0008 | # ASN.1 source http://mibs.snmplabs.com:80/asn1/SNMP-USM-AES-MIB
0009 | # Produced by pysmi-0.1.3 at Tue Apr 18 00:49:58 2017
0010 | # On host grommit.local platform Darwin version 16.4.0 by user ilya
0011 | # Using Python version 3.4.2 (v3.4.2:ab2c023a9432, Oct 5 2014, 20:42:22)
0012 | #
0013 | OctetString, Integer, ObjectIdentifier = mibBuilder.importSymbols("ASN1", "OctetString", "Integer", "Object
0014 | NamedValues, = mibBuilder.importSymbols("ASN1-ENUMERATION", "NamedValues")
0015 | ValueSizeConstraint, ConstraintsUnion, SingleValueConstraint, ValueRangeConstraint, ConstraintsIntersection
0016 | snmpPrivProtocols, = mibBuilder.importSymbols("SNMP-FRAMEWORK-MIB", "snmpPrivProtocols")

```

```

017 | NotificationGroup, ModuleCompliance = mibBuilder.importSymbols("SNMPv2-CONF", "NotificationGroup", "ModuleC
018 | Counter32, iso, ModuleIdentity, IPAddress, MibIdentifier, Integer32, TimeTicks, snmpModules, Bits, Gauge32,
019 | DisplayString, TextualConvention = mibBuilder.importSymbols("SNMPv2-TC", "DisplayString", "TextualConventio
020 | snmpUsmAesMIB = ModuleIdentity((1, 3, 6, 1, 6, 3, 20))
021 | if mibBuilder.loadTexts: snmpUsmAesMIB.setRevisions(('2004-06-14 00:00',))
022 | if mibBuilder.loadTexts: snmpUsmAesMIB.setLastUpdated('200406140000Z')
023 | if mibBuilder.loadTexts: snmpUsmAesMIB.setOrganization('IETF')
024 | if mibBuilder.loadTexts: snmpUsmAesMIB.setContactInfo('Uri Blumenthal Lucent Technologies / Bell Labs 67 Wh
025 | if mibBuilder.loadTexts: snmpUsmAesMIB.setDescription('Definitions of Object Identities needed for the use
026 | usmAesCfb128Protocol = ObjectIdentity((1, 3, 6, 1, 6, 3, 10, 1, 2, 4))
027 | if mibBuilder.loadTexts: usmAesCfb128Protocol.setStatus('current')
028 | if mibBuilder.loadTexts: usmAesCfb128Protocol.setDescription('The CFB128-AES-128 Privacy Protocol.')
029 | if mibBuilder.loadTexts: usmAesCfb128Protocol.setReference('- Specification for the ADVANCED ENCRYPTION STA
030 | mibBuilder.exportSymbols("SNMP-USM-AES-MIB", usmAesCfb128Protocol=usmAesCfb128Protocol, snmpUsmAesMIB=snmpU

```

=== ARQUIVO: .\dist\ControleImpressoras\internal\pysnmp\smi\mibs\SNMP-USM-HMAC-SHA2-MIB.py ===

```

001 | #
002 | # PySNMP MIB module SNMP-USM-HMAC-SHA2-MIB (http://snmplabs.com/pysnmp)
003 | # ASN.1 source http://mibs.snmplabs.com:80/asn1/SNMP-USM-HMAC-SHA2-MIB
004 | # Produced by pysmi-0.1.4 at Thu Aug 3 02:14:32 2017
005 | # On host grommit.local platform Darwin version 16.4.0 by user ilya
006 | # Using Python version 3.4.2 (v3.4.2:ab2c023a9432, Oct 5 2014, 20:42:22)
007 | #
008 | OctetString, Integer, ObjectIdentifier = mibBuilder.importSymbols("ASN1", "OctetString", "Integer", "Object
009 | NamedValues, = mibBuilder.importSymbols("ASN1-ENUMERATION", "NamedValues")
010 | ValueSizeConstraint, SingleValueConstraint, ConstraintsUnion, ConstraintsIntersection, ValueRangeConstraint
011 | snmpAuthProtocols, = mibBuilder.importSymbols("SNMP-FRAMEWORK-MIB", "snmpAuthProtocols")
012 | NotificationGroup, ModuleCompliance = mibBuilder.importSymbols("SNMPv2-CONF", "NotificationGroup", "ModuleC
013 | Counter32, Bits, MibScalar, MibTable, MibTableRow, MibTableColumn, Counter64, ObjectIdentity, TimeTicks, Ga
014 | TextualConvention, DisplayString = mibBuilder.importSymbols("SNMPv2-TC", "TextualConvention", "DisplayStrin
015 | snmpUsmHmacSha2MIB = ModuleIdentity((1, 3, 6, 1, 2, 1, 235))
016 | if mibBuilder.loadTexts: snmpUsmHmacSha2MIB.setRevisions(('2016-04-18 00:00', '2015-10-14 00:00',))
017 | if mibBuilder.loadTexts: snmpUsmHmacSha2MIB.setLastUpdated('201604180000Z')
018 | if mibBuilder.loadTexts: snmpUsmHmacSha2MIB.setOrganization('SNMPv3 Working Group')
019 | if mibBuilder.loadTexts: snmpUsmHmacSha2MIB.setContactInfo('WG email: OPSAWG@ietf.org Subscribe: https://ww
020 | if mibBuilder.loadTexts: snmpUsmHmacSha2MIB.setDescription('Definitions of Object Identities needed for the
021 | usmHMAC128SHA224AuthProtocol = ObjectIdentity((1, 3, 6, 1, 6, 3, 10, 1, 1, 4))
022 | if mibBuilder.loadTexts: usmHMAC128SHA224AuthProtocol.setStatus('current')
023 | if mibBuilder.loadTexts: usmHMAC128SHA224AuthProtocol.setDescription('The Authentication Protocol usmHMAC12
024 | if mibBuilder.loadTexts: usmHMAC128SHA224AuthProtocol.setReference('- Krawczyk, H., Bellare, M., and R. Can
025 | usmHMAC192SHA256AuthProtocol = ObjectIdentity((1, 3, 6, 1, 6, 3, 10, 1, 1, 5))
026 | if mibBuilder.loadTexts: usmHMAC192SHA256AuthProtocol.setStatus('current')
027 | if mibBuilder.loadTexts: usmHMAC192SHA256AuthProtocol.setDescription('The Authentication Protocol usmHMAC19
028 | if mibBuilder.loadTexts: usmHMAC192SHA256AuthProtocol.setReference('- Krawczyk, H., Bellare, M., and R. Can
029 | usmHMAC256SHA384AuthProtocol = ObjectIdentity((1, 3, 6, 1, 6, 3, 10, 1, 1, 6))
030 | if mibBuilder.loadTexts: usmHMAC256SHA384AuthProtocol.setStatus('current')
031 | if mibBuilder.loadTexts: usmHMAC256SHA384AuthProtocol.setDescription('The Authentication Protocol usmHMAC25
032 | if mibBuilder.loadTexts: usmHMAC256SHA384AuthProtocol.setReference('- Krawczyk, H., Bellare, M., and R. Can
033 | usmHMAC384SHA512AuthProtocol = ObjectIdentity((1, 3, 6, 1, 6, 3, 10, 1, 1, 7))
034 | if mibBuilder.loadTexts: usmHMAC384SHA512AuthProtocol.setStatus('current')
035 | if mibBuilder.loadTexts: usmHMAC384SHA512AuthProtocol.setDescription('The Authentication Protocol usmHMAC38
036 | if mibBuilder.loadTexts: usmHMAC384SHA512AuthProtocol.setReference('- Krawczyk, H., Bellare, M., and R. Can
037 | mibBuilder.exportSymbols("SNMP-USM-HMAC-SHA2-MIB", usmHMAC256SHA384AuthProtocol=usmHMAC256SHA384AuthProtoco

```

=== ARQUIVO: .\dist\ControleImpressoras_internal\pysnmp\smi\mibs\SNMP-VIEW-BASED-ACM-MIB.py ===

```
001 | #
002 | # This file is part of pysnmp software.
003 | #
004 | # Copyright (c) 2005-2019, Ilya Etingof <etingof@gmail.com>
005 | # License: http://snmplabs.com/pysnmp/license.html
006 | #
007 | # PySNMP MIB module SNMP-VIEW-BASED-ACM-MIB (http://snmplabs.com/pysnmp)
008 | # ASN.1 source http://mibs.snmplabs.com:80/asn1/SNMP-VIEW-BASED-ACM-MIB
009 | # Produced by pysmi-0.1.3 at Tue Apr 18 00:50:37 2017
010 | # On host grommit.local platform Darwin version 16.4.0 by user ilya
011 | # Using Python version 3.4.2 (v3.4.2:ab2c023a9432, Oct 5 2014, 20:42:22)
012 | #
013 | OctetString, Integer, ObjectIdentifier = mibBuilder.importSymbols("ASN1", "OctetString", "Integer", "ObjectIdentifier")
014 | NamedValues, = mibBuilder.importSymbols("ASN1-ENUMERATION", "NamedValues")
015 | ValueRangeConstraint, ValueSizeConstraint, ConstraintsIntersection, ConstraintsUnion, SingleValueConstraint, = mibBuilder.importSymbols("Constraints", "ValueRangeConstraint", "ValueSizeConstraint", "ConstraintsIntersection", "ConstraintsUnion", "SingleValueConstraint")
016 | SnmpAdminString, SnmpSecurityLevel, SnmpSecurityModel = mibBuilder.importSymbols("SNMP-FRAMEWORK-MIB", "SnmpAdminString", "SnmpSecurityLevel", "SnmpSecurityModel")
017 | NotificationGroup, ModuleCompliance, ObjectGroup = mibBuilder.importSymbols("SNMPv2-CONF", "NotificationGroup", "ModuleCompliance", "ObjectGroup")
018 | TimeTicks, Gauge32, Bits, snmpModules, IPAddress, Counter64, MibScalar, MibTable, MibTableRow, MibTableColumn, = mibBuilder.importSymbols("SNMPv2-SMI", "TimeTicks", "Gauge32", "Bits", "Counter64", "MibScalar", "MibTable", "MibTableRow", "MibTableColumn")
019 | TestAndIncr, RowStatus, TextualConvention, DisplayString, StorageType = mibBuilder.importSymbols("SNMPv2-TC", "TestAndIncr", "RowStatus", "TextualConvention", "DisplayString", "StorageType")
020 | snmpVacmMIB = ModuleIdentity((1, 3, 6, 1, 6, 3, 16))
021 | if mibBuilder.loadTexts: snmpVacmMIB.setRevisions(('2002-10-16 00:00', '1999-01-20 00:00', '1997-11-20 00:00'))
022 | if mibBuilder.loadTexts: snmpVacmMIB.setLastUpdated('200210160000Z')
023 | if mibBuilder.loadTexts: snmpVacmMIB.setOrganization('SNMPv3 Working Group')
024 | if mibBuilder.loadTexts: snmpVacmMIB.setContactInfo('WG-email: snmpv3@lists.tislabs.com Subscribe: majordomo@lists.tislabs.com')
025 | if mibBuilder.loadTexts: snmpVacmMIB.setDescription('The management information definitions for the View-based Access Control Model (VACM)')
026 | vacmMIBObjects = MibIdentifier((1, 3, 6, 1, 6, 3, 16, 1))
027 | vacmMIBConformance = MibIdentifier((1, 3, 6, 1, 6, 3, 16, 2))
028 | vacmContextTable = MibTable((1, 3, 6, 1, 6, 3, 16, 1, 1), )
029 | if mibBuilder.loadTexts: vacmContextTable.setStatus('current')
030 | if mibBuilder.loadTexts: vacmContextTable.setDescription('The table of locally available contexts. This table is used to identify the contexts that are available for use in the VACM')
031 | vacmContextEntry = MibTableRow((1, 3, 6, 1, 6, 3, 16, 1, 1), ).setIndexNames((0, "SNMP-VIEW-BASED-ACM-MIB-vacmContextTableEntryName"))
032 | if mibBuilder.loadTexts: vacmContextEntry.setStatus('current')
033 | if mibBuilder.loadTexts: vacmContextEntry.setDescription('Information about a particular context.')
034 | vacmContextName = MibTableColumn((1, 3, 6, 1, 6, 3, 16, 1, 1, 1), SnmpAdminString().subtype(subtypeSpec=ValueRangeConstraint(1, 255)))
035 | if mibBuilder.loadTexts: vacmContextName.setStatus('current')
036 | if mibBuilder.loadTexts: vacmContextName.setDescription('A human readable name identifying a particular context')
037 | # The RowStatus column is not present in the MIB
038 | vacmContextStatus = MibTableColumn((1, 3, 6, 1, 6, 3, 16, 1, 1, 2), RowStatus()).setMaxAccess("readcreate")
039 | if mibBuilder.loadTexts: vacmContextStatus.setStatus('current')
040 | if mibBuilder.loadTexts: vacmContextStatus.setDescription('The status of this conceptual row. Until instance creation, the status is "notReady".')
041 | vacmSecurityToGroupTable = MibTable((1, 3, 6, 1, 6, 3, 16, 1, 2), )
042 | if mibBuilder.loadTexts: vacmSecurityToGroupTable.setStatus('current')
043 | if mibBuilder.loadTexts: vacmSecurityToGroupTable.setDescription('This table maps a combination of security level and security model to a group name')
044 | vacmSecurityToGroupEntry = MibTableRow((1, 3, 6, 1, 6, 3, 16, 1, 2, 1), ).setIndexNames((0, "SNMP-VIEW-BASED-ACM-MIB-vacmSecurityToGroupTableEntryName"))
045 | if mibBuilder.loadTexts: vacmSecurityToGroupEntry.setStatus('current')
046 | if mibBuilder.loadTexts: vacmSecurityToGroupEntry.setDescription('An entry in this table maps the combination of security level and security model to a group name')
047 | vacmSecurityModel = MibTableColumn((1, 3, 6, 1, 6, 3, 16, 1, 2, 1, 1), SnmpSecurityModel().subtype(subtypeSpec=ValueRangeConstraint(1, 255)))
048 | if mibBuilder.loadTexts: vacmSecurityModel.setStatus('current')
049 | if mibBuilder.loadTexts: vacmSecurityModel.setDescription('The Security Model, by which the vacmSecurityName is interpreted')
050 | vacmSecurityName = MibTableColumn((1, 3, 6, 1, 6, 3, 16, 1, 2, 1, 2), SnmpAdminString().subtype(subtypeSpec=ValueRangeConstraint(1, 255)))
051 | if mibBuilder.loadTexts: vacmSecurityName.setStatus('current')
052 | if mibBuilder.loadTexts: vacmSecurityName.setDescription('The securityName for the principal, represented in UTF-8 string')
053 | vacmGroupName = MibTableColumn((1, 3, 6, 1, 6, 3, 16, 1, 2, 1, 3), SnmpAdminString().subtype(subtypeSpec=ValueRangeConstraint(1, 255)))
054 | if mibBuilder.loadTexts: vacmGroupName.setStatus('current')
055 | if mibBuilder.loadTexts: vacmGroupName.setDescription('The name of the group to which this entry (e.g., the vacmSecurityToGroupEntry) applies')
056 | vacmSecurityToGroupStorageType = MibTableColumn((1, 3, 6, 1, 6, 3, 16, 1, 2, 1, 4), StorageType().clone('no-create'))
057 | if mibBuilder.loadTexts: vacmSecurityToGroupStorageType.setStatus('current')
058 | if mibBuilder.loadTexts: vacmSecurityToGroupStorageType.setDescription('The storage type for this conceptual row')
059 | vacmSecurityToGroupStatus = MibTableColumn((1, 3, 6, 1, 6, 3, 16, 1, 2, 1, 5), RowStatus()).setMaxAccess("readcreate")
```

```

060 | if mibBuilder.loadTexts: vacmSecurityToGroupStatus.setStatus('current')
061 | if mibBuilder.loadTexts: vacmSecurityToGroupStatus.setDescription("The status of this conceptual row. Until
062 | vacmAccessTable = MibTable((1, 3, 6, 1, 6, 3, 16, 1, 4), )
063 | if mibBuilder.loadTexts: vacmAccessTable.setStatus('current')
064 | if mibBuilder.loadTexts: vacmAccessTable.setDescription("The table of access rights for groups. Each entry
065 | vacmAccessEntry = MibTableRow((1, 3, 6, 1, 6, 3, 16, 1, 4, 1), ).setIndexNames((0, "SNMP-VIEW-BASED-ACM-MIB
066 | if mibBuilder.loadTexts: vacmAccessEntry.setStatus('current')
067 | if mibBuilder.loadTexts: vacmAccessEntry.setDescription("An access right configured in the Local Configurati
068 | vacmAccessContextPrefix = MibTableColumn((1, 3, 6, 1, 6, 3, 16, 1, 4, 1, 1), SnmpAdminString().subtype(subt
069 | if mibBuilder.loadTexts: vacmAccessContextPrefix.setStatus('current')
070 | if mibBuilder.loadTexts: vacmAccessContextPrefix.setDescription("In order to gain the access rights allowed
071 | vacmAccessSecurityModel = MibTableColumn((1, 3, 6, 1, 6, 3, 16, 1, 4, 1, 2), SnmpSecurityModel())
072 | if mibBuilder.loadTexts: vacmAccessSecurityModel.setStatus('current')
073 | if mibBuilder.loadTexts: vacmAccessSecurityModel.setDescription("In order to gain the access rights allowed
074 | vacmAccessSecurityLevel = MibTableColumn((1, 3, 6, 1, 6, 3, 16, 1, 4, 1, 3), SnmpSecurityLevel())
075 | if mibBuilder.loadTexts: vacmAccessSecurityLevel.setStatus('current')
076 | if mibBuilder.loadTexts: vacmAccessSecurityLevel.setDescription("The minimum level of security required in
077 | vacmAccessContextMatch = MibTableColumn((1, 3, 6, 1, 6, 3, 16, 1, 4, 1, 4), Integer32().subtype(subtypeSpec
078 | if mibBuilder.loadTexts: vacmAccessContextMatch.setStatus('current')
079 | if mibBuilder.loadTexts: vacmAccessContextMatch.setDescription("If the value of this object is exact(1), th
080 | vacmAccessReadViewName = MibTableColumn((1, 3, 6, 1, 6, 3, 16, 1, 4, 1, 5), SnmpAdminString().subtype(subty
081 | if mibBuilder.loadTexts: vacmAccessReadViewName.setStatus('current')
082 | if mibBuilder.loadTexts: vacmAccessReadViewName.setDescription("The value of an instance of this object ide
083 | vacmAccessWriteViewName = MibTableColumn((1, 3, 6, 1, 6, 3, 16, 1, 4, 1, 6), SnmpAdminString().subtype(subt
084 | if mibBuilder.loadTexts: vacmAccessWriteViewName.setStatus('current')
085 | if mibBuilder.loadTexts: vacmAccessWriteViewName.setDescription("The value of an instance of this object id
086 | vacmAccessNotifyViewName = MibTableColumn((1, 3, 6, 1, 6, 3, 16, 1, 4, 1, 7), SnmpAdminString().subtype(sub
087 | if mibBuilder.loadTexts: vacmAccessNotifyViewName.setStatus('current')
088 | if mibBuilder.loadTexts: vacmAccessNotifyViewName.setDescription("The value of an instance of this object i
089 | vacmAccessStorageType = MibTableColumn((1, 3, 6, 1, 6, 3, 16, 1, 4, 1, 8), StorageType().clone('nonVolatile
090 | if mibBuilder.loadTexts: vacmAccessStorageType.setStatus('current')
091 | if mibBuilder.loadTexts: vacmAccessStorageType.setDescription("The storage type for this conceptual row. Co
092 | vacmAccessStatus = MibTableColumn((1, 3, 6, 1, 6, 3, 16, 1, 4, 1, 9), RowStatus()).setMaxAccess("readcreate
093 | if mibBuilder.loadTexts: vacmAccessStatus.setStatus('current')
094 | if mibBuilder.loadTexts: vacmAccessStatus.setDescription("The status of this conceptual row. The RowStatus
095 | vacmMIBViews = MibIdentifier((1, 3, 6, 1, 6, 3, 16, 1, 5))
096 | vacmViewSpinLock = MibScalar((1, 3, 6, 1, 6, 3, 16, 1, 5, 1), TestAndIncr()).setMaxAccess("readwrite")
097 | if mibBuilder.loadTexts: vacmViewSpinLock.setStatus('current')
098 | if mibBuilder.loadTexts: vacmViewSpinLock.setDescription("An advisory lock used to allow cooperating SNMP C
099 | vacmViewTreeFamilyTable = MibTable((1, 3, 6, 1, 6, 3, 16, 1, 5, 2), )
100 | if mibBuilder.loadTexts: vacmViewTreeFamilyTable.setStatus('current')
101 | if mibBuilder.loadTexts: vacmViewTreeFamilyTable.setDescription("Locally held information about families of
102 | vacmViewTreeFamilyEntry = MibTableRow((1, 3, 6, 1, 6, 3, 16, 1, 5, 2, 1), ).setIndexNames((0, "SNMP-VIEW-BA
103 | if mibBuilder.loadTexts: vacmViewTreeFamilyEntry.setStatus('current')
104 | if mibBuilder.loadTexts: vacmViewTreeFamilyEntry.setDescription("Information on a particular family of view
105 | vacmViewTreeFamilyViewName = MibTableColumn((1, 3, 6, 1, 6, 3, 16, 1, 5, 2, 1, 1), SnmpAdminString().subtyp
106 | if mibBuilder.loadTexts: vacmViewTreeFamilyViewName.setStatus('current')
107 | if mibBuilder.loadTexts: vacmViewTreeFamilyViewName.setDescription("The human readable name for a family of
108 | vacmViewTreeFamilySubtree = MibTableColumn((1, 3, 6, 1, 6, 3, 16, 1, 5, 2, 1, 2), ObjectIdentifier())
109 | if mibBuilder.loadTexts: vacmViewTreeFamilySubtree.setStatus('current')
110 | if mibBuilder.loadTexts: vacmViewTreeFamilySubtree.setDescription("The MIB subtree which when combined with
111 | vacmViewTreeFamilyMask = MibTableColumn((1, 3, 6, 1, 6, 3, 16, 1, 5, 2, 1, 3), OctetString().subtype(subtyp
112 | if mibBuilder.loadTexts: vacmViewTreeFamilyMask.setStatus('current')
113 | if mibBuilder.loadTexts: vacmViewTreeFamilyMask.setDescription("The bit mask which, in combination with the
114 | vacmViewTreeFamilyType = MibTableColumn((1, 3, 6, 1, 6, 3, 16, 1, 5, 2, 1, 4), Integer32().subtype(subtypeS
115 | if mibBuilder.loadTexts: vacmViewTreeFamilyType.setStatus('current')
116 | if mibBuilder.loadTexts: vacmViewTreeFamilyType.setDescription("Indicates whether the corresponding instanc
117 | vacmViewTreeFamilyStorageType = MibTableColumn((1, 3, 6, 1, 6, 3, 16, 1, 5, 2, 1, 5), StorageType().clone('
118 | if mibBuilder.loadTexts: vacmViewTreeFamilyStorageType.setStatus('current')
119 | if mibBuilder.loadTexts: vacmViewTreeFamilyStorageType.setDescription("The storage type for this conceptual
120 | vacmViewTreeFamilyStatus = MibTableColumn((1, 3, 6, 1, 6, 3, 16, 1, 5, 2, 1, 6), RowStatus()).setMaxAccess(
121 | if mibBuilder.loadTexts: vacmViewTreeFamilyStatus.setStatus('current')
122 | if mibBuilder.loadTexts: vacmViewTreeFamilyStatus.setDescription("The status of this conceptual row. The Ro

```

```

123 | vacmMIBCompliances = MibIdentifier((1, 3, 6, 1, 6, 3, 16, 2, 1))
124 | vacmMIBGroups = MibIdentifier((1, 3, 6, 1, 6, 3, 16, 2, 2))
125 | vacmMIBCompliance = ModuleCompliance((1, 3, 6, 1, 6, 3, 16, 2, 1, 1)).setObjects(("SNMP-VIEW-BASED-ACM-MIB"
126 | if mibBuilder.loadTexts: vacmMIBCompliance.setDescription('The compliance statement for SNMP engines which
127 | vacmBasicGroup = ObjectGroup((1, 3, 6, 1, 6, 3, 16, 2, 2, 1)).setObjects(("SNMP-VIEW-BASED-ACM-MIB", "vacmC
128 | if mibBuilder.loadTexts: vacmBasicGroup.setDescription('A collection of objects providing for remote config
129 | mibBuilder.exportSymbols("SNMP-VIEW-BASED-ACM-MIB", vacmSecurityModel=vacmSecurityModel, vacmSecurityToGrou

```

=== ARQUIVO:

.\dist\ControleImpressoras_internal\pysnmp\smi\mibs\SNMPv2-CONF.py ===

```

001 | #
002 | # This file is part of pysnmp software.
003 | #
004 | # Copyright (c) 2005-2019, Ilya Etingof <etingof@gmail.com>
005 | # License: http://snmplabs.com/pysnmp/license.html
006 | #
007 | # PySNMP MIB module SNMPv2-CONF (http://snmplabs.com/pysnmp)
008 | # ASN.1 source http://mibs.snmplabs.com:80/asn1/SNMPv2-CONF
009 | # Produced by pysmi-0.1.3 at Tue Apr 18 00:51:39 2017
010 | # On host grommit.local platform Darwin version 16.4.0 by user ilya
011 | # Using Python version 3.4.2 (v3.4.2:ab2c023a9432, Oct 5 2014, 20:42:22)
012 | #
013 | MibNode, = mibBuilder.importSymbols('SNMPv2-SMI', 'MibNode')
014 |
015 |
016 | class ObjectGroup(MibNode):
017 |     status = 'current'
018 |     objects = ()
019 |     description = ''
020 |
021 |     def getStatus(self):
022 |         return self.status
023 |
024 |     def setStatus(self, v):
025 |         self.status = v
026 |         return self
027 |
028 |     def getObjects(self):
029 |         return getattr(self, 'objects', ())
030 |
031 |     def setObjects(self, *args, **kwargs):
032 |         if kwargs.get('append'):
033 |             self.objects += args
034 |         else:
035 |             self.objects = args
036 |         return self
037 |
038 |     def getDescription(self):
039 |         return getattr(self, 'description', '')
040 |
041 |     def setDescription(self, v):
042 |         self.description = v
043 |         return self
044 |
045 |     def asn1Print(self):
046 |         return """\
047 | OBJECT-GROUP
048 |   OBJECTS { %s }
049 |   DESCRIPTION "%s"

```



```

050 | """ % ('', '.join([x for x in self.getObjects()])), self.getDescription())
051 |
052 |
053 | class NotificationGroup(MibNode):
054 |     status = 'current'
055 |     objects = ()
056 |     description = ''
057 |
058 |     def getStatus(self):
059 |         return self.status
060 |
061 |     def setStatus(self, v):
062 |         self.status = v
063 |         return self
064 |
065 |     def getObjects(self):
066 |         return getattr(self, 'objects', ())
067 |
068 |     def setObjects(self, *args, **kwargs):
069 |         if kwargs.get('append'):
070 |             self.objects += args
071 |         else:
072 |             self.objects = args
073 |         return self
074 |
075 |     def getDescription(self):
076 |         return getattr(self, 'description', '')
077 |
078 |     def setDescription(self, v):
079 |         self.description = v
080 |         return self
081 |
082 |     def asn1Print(self):
083 |         return """\
084 | NOTIFICATION-GROUP
085 |   NOTIFICATIONS { %s }
086 |   DESCRIPTION "%s"
087 | """ % ('', '.join([x for x in self.getObjects()])), self.getDescription())
088 |
089 |
090 | class ModuleCompliance(MibNode):
091 |     status = 'current'
092 |     objects = ()
093 |     description = ''
094 |
095 |     def getStatus(self):
096 |         return self.status
097 |
098 |     def setStatus(self, v):
099 |         self.status = v
100 |         return self
101 |
102 |     def getObjects(self):
103 |         return getattr(self, 'objects', ())
104 |
105 |     def setObjects(self, *args, **kwargs):
106 |         if kwargs.get('append'):
107 |             self.objects += args
108 |         else:
109 |             self.objects = args
110 |         return self
111 |
112 |     def getDescription(self):

```

```

113 |         return getattr(self, 'description', '')
114 |
115 |     def setDescription(self, v):
116 |         self.description = v
117 |         return self
118 |
119 |     def asn1Print(self):
120 |         return """\
121 | MODULE-COMPLIANCE
122 |   OBJECT { %s }
123 |   DESCRIPTION "%s"
124 | """ % ('', '.join([x for x in self.getObjects()])), self.getDescription())
125 |
126 |
127 | class AgentCapabilities(MibNode):
128 |     status = 'current'
129 |     description = ''
130 |     reference = ''
131 |     productRelease = ''
132 |
133 |     def getStatus(self):
134 |         return self.status
135 |
136 |     def setStatus(self, v):
137 |         self.status = v
138 |         return self
139 |
140 |     def getDescription(self):
141 |         return getattr(self, 'description', '')
142 |
143 |     def setDescription(self, v):
144 |         self.description = v
145 |         return self
146 |
147 |     def getReference(self):
148 |         return self.reference
149 |
150 |     def setReference(self, v):
151 |         self.reference = v
152 |         return self
153 |
154 |     def getProductRelease(self):
155 |         return self.productRelease
156 |
157 |     def setProductRelease(self, v):
158 |         self.productRelease = v
159 |         return self
160 |
161 |     # TODO: implement the rest of properties
162 |
163 |     def asn1Print(self):
164 |         return """\
165 | AGENT-CAPABILITIES
166 |   STATUS "%s"
167 |   PRODUCT-RELEASE "%s"
168 |   DESCRIPTION "%s"
169 | """ % (self.getStatus(), self.getProductRelease(), self.getDescription())
170 |
171 |
172 | mibBuilder.exportSymbols('SNMPv2-CONF', ObjectGroup=ObjectGroup, NotificationGroup=NotificationGroup, Modul

```


=== ARQUIVO:

.\dist\ControleImpressoras_internal\pysnmp\smi\mibs\SNMPv2-MIB.py ===

```
001 | #
002 | # This file is part of pysnmp software.
003 | #
004 | # Copyright (c) 2005-2019, Ilya Etingof <etingof@gmail.com>
005 | # License: http://snmplabs.com/pysnmp/license.html
006 | #
007 | # PySNMP MIB module SNMPv2-MIB (http://snmplabs.com/pysnmp)
008 | # ASN.1 source http://mibs.snmplabs.com:80/asn1/SNMPv2-MIB
009 | # Produced by pysmi-0.1.3 at Tue Apr 18 00:52:45 2017
010 | # On host grommit.local platform Darwin version 16.4.0 by user ilya
011 | # Using Python version 3.4.2 (v3.4.2:ab2c023a9432, Oct 5 2014, 20:42:22)
012 | #
013 | OctetString, ObjectIdentifier, Integer = mibBuilder.importSymbols("ASN1", "OctetString", "ObjectIdentifier")
014 | NamedValues, = mibBuilder.importSymbols("ASN1-ENUMERATION", "NamedValues")
015 | SingleValueConstraint, ValueSizeConstraint, ConstraintsIntersection, ValueRangeConstraint, ConstraintsUnion
016 | ObjectGroup, ModuleCompliance, NotificationGroup = mibBuilder.importSymbols("SNMPv2-CONF", "ObjectGroup", "
017 | mib_2, snmpModules, ModuleIdentity, Counter64, ObjectIdentity, Integer32, NotificationType, iso, MibScalar,
018 | TextualConvention, TestAndIncr, TimeStamp, DisplayString = mibBuilder.importSymbols("SNMPv2-TC", "TextualCo
019 | snmpMIB = ModuleIdentity((1, 3, 6, 1, 6, 3, 1))
020 | if mibBuilder.loadTexts: snmpMIB.setRevisions(('2000-08-09 20:17', '1995-11-09 00:00', '1993-04-01 00:00',))
021 | if mibBuilder.loadTexts: snmpMIB.setLastUpdated('200008092017Z')
022 | if mibBuilder.loadTexts: snmpMIB.setOrganization('IETF SNMPv3 Working Group')
023 | if mibBuilder.loadTexts: snmpMIB.setContactInfo('WG-EMail: snmpv3@tis.com Subscribe: majordomo@tis.com In m
024 | if mibBuilder.loadTexts: snmpMIB.setDescription('The MIB module for SNMP entities.')
025 | snmpMIBObjects = MibIdentifier((1, 3, 6, 1, 6, 3, 1, 1))
026 | system = MibIdentifier((1, 3, 6, 1, 2, 1, 1))
027 | sysDescr = MibScalar((1, 3, 6, 1, 2, 1, 1, 1), DisplayString().subtype(subtypeSpec=ValueSizeConstraint(0, 2
028 | if mibBuilder.loadTexts: sysDescr.setStatus('current')
029 | if mibBuilder.loadTexts: sysDescr.setDescription("A textual description of the entity. This value should in
030 | sysObjectID = MibScalar((1, 3, 6, 1, 2, 1, 1, 2), ObjectIdentifier()).setMaxAccess("readonly")
031 | if mibBuilder.loadTexts: sysObjectID.setStatus('current')
032 | if mibBuilder.loadTexts: sysObjectID.setDescription("The vendor's authoritative identification of the netwo
033 | sysUpTime = MibScalar((1, 3, 6, 1, 2, 1, 1, 3), TimeTicks()).setMaxAccess("readonly")
034 | if mibBuilder.loadTexts: sysUpTime.setStatus('current')
035 | if mibBuilder.loadTexts: sysUpTime.setDescription('The time (in hundredths of a second) since the network m
036 | sysContact = MibScalar((1, 3, 6, 1, 2, 1, 1, 4), DisplayString().subtype(subtypeSpec=ValueSizeConstraint(0,
037 | if mibBuilder.loadTexts: sysContact.setStatus('current')
038 | if mibBuilder.loadTexts: sysContact.setDescription('The textual identification of the contact person for th
039 | sysName = MibScalar((1, 3, 6, 1, 2, 1, 1, 5), DisplayString().subtype(subtypeSpec=ValueSizeConstraint(0, 25
040 | if mibBuilder.loadTexts: sysName.setStatus('current')
041 | if mibBuilder.loadTexts: sysName.setDescription("An administratively-assigned name for this managed node. B
042 | sysLocation = MibScalar((1, 3, 6, 1, 2, 1, 1, 6), DisplayString().subtype(subtypeSpec=ValueSizeConstraint(0
043 | if mibBuilder.loadTexts: sysLocation.setStatus('current')
044 | if mibBuilder.loadTexts: sysLocation.setDescription("The physical location of this node (e.g., 'telephone c
045 | sysServices = MibScalar((1, 3, 6, 1, 2, 1, 1, 7), Integer32().subtype(subtypeSpec=ValueRangeConstraint(0, 1
046 | if mibBuilder.loadTexts: sysServices.setStatus('current')
047 | if mibBuilder.loadTexts: sysServices.setDescription('A value which indicates the set of services that this
048 | sysORLastChange = MibScalar((1, 3, 6, 1, 2, 1, 1, 8), TimeStamp()).setMaxAccess("readonly")
049 | if mibBuilder.loadTexts: sysORLastChange.setStatus('current')
050 | if mibBuilder.loadTexts: sysORLastChange.setDescription('The value of sysUpTime at the time of the most rec
051 | sysORTable = MibTable((1, 3, 6, 1, 2, 1, 1, 9), )
052 | if mibBuilder.loadTexts: sysORTable.setStatus('current')
053 | if mibBuilder.loadTexts: sysORTable.setDescription('The (conceptual) table listing the capabilities of the
054 | sysOREntry = MibTableRow((1, 3, 6, 1, 2, 1, 1, 9, 1), ).setIndexNames((0, "SNMPv2-MIB", "sysORIndex"))
055 | if mibBuilder.loadTexts: sysOREntry.setStatus('current')
056 | if mibBuilder.loadTexts: sysOREntry.setDescription('An entry (conceptual row) in the sysORTable.')
057 | sysORIndex = MibTableColumn((1, 3, 6, 1, 2, 1, 1, 9, 1, 1), Integer32().subtype(subtypeSpec=ValueRangeConst
058 | if mibBuilder.loadTexts: sysORIndex.setStatus('current')
059 | if mibBuilder.loadTexts: sysORIndex.setDescription('The auxiliary variable used for identifying instances o
```

```

060 | sysORID = MibTableColumn((1, 3, 6, 1, 2, 1, 1, 9, 1, 2), ObjectIdentifier()).setMaxAccess("readonly")
061 | if mibBuilder.loadTexts: sysORID.setStatus('current')
062 | if mibBuilder.loadTexts: sysORID.setDescription('An authoritative identification of a capabilities statement')
063 | sysORDescr = MibTableColumn((1, 3, 6, 1, 2, 1, 1, 9, 1, 3), DisplayString()).setMaxAccess("readonly")
064 | if mibBuilder.loadTexts: sysORDescr.setStatus('current')
065 | if mibBuilder.loadTexts: sysORDescr.setDescription('A textual description of the capabilities identified by')
066 | sysORUpTime = MibTableColumn((1, 3, 6, 1, 2, 1, 1, 9, 1, 4), TimeStamp()).setMaxAccess("readonly")
067 | if mibBuilder.loadTexts: sysORUpTime.setStatus('current')
068 | if mibBuilder.loadTexts: sysORUpTime.setDescription('The value of sysUpTime at the time this conceptual row')
069 | snmp = MibIdentifier((1, 3, 6, 1, 2, 1, 11))
070 | snmpInPkts = MibScalar((1, 3, 6, 1, 2, 1, 11, 1), Counter32()).setMaxAccess("readonly")
071 | if mibBuilder.loadTexts: snmpInPkts.setStatus('current')
072 | if mibBuilder.loadTexts: snmpInPkts.setDescription('The total number of messages delivered to the SNMP entity')
073 | snmpInBadVersions = MibScalar((1, 3, 6, 1, 2, 1, 11, 3), Counter32()).setMaxAccess("readonly")
074 | if mibBuilder.loadTexts: snmpInBadVersions.setStatus('current')
075 | if mibBuilder.loadTexts: snmpInBadVersions.setDescription('The total number of SNMP messages which were deliv')
076 | snmpInBadCommunityNames = MibScalar((1, 3, 6, 1, 2, 1, 11, 4), Counter32()).setMaxAccess("readonly")
077 | if mibBuilder.loadTexts: snmpInBadCommunityNames.setStatus('current')
078 | if mibBuilder.loadTexts: snmpInBadCommunityNames.setDescription('The total number of community-based SNMP m')
079 | snmpInBadCommunityUses = MibScalar((1, 3, 6, 1, 2, 1, 11, 5), Counter32()).setMaxAccess("readonly")
080 | if mibBuilder.loadTexts: snmpInBadCommunityUses.setStatus('current')
081 | if mibBuilder.loadTexts: snmpInBadCommunityUses.setDescription('The total number of community-based SNMP me')
082 | snmpInASNParseErrs = MibScalar((1, 3, 6, 1, 2, 1, 11, 6), Counter32()).setMaxAccess("readonly")
083 | if mibBuilder.loadTexts: snmpInASNParseErrs.setStatus('current')
084 | if mibBuilder.loadTexts: snmpInASNParseErrs.setDescription('The total number of ASN.1 or BER errors encount')
085 | snmpEnableAuthenTraps = MibScalar((1, 3, 6, 1, 2, 1, 11, 30), Integer32().subtype(subtypeSpec=ConstraintsUn')
086 | if mibBuilder.loadTexts: snmpEnableAuthenTraps.setStatus('current')
087 | if mibBuilder.loadTexts: snmpEnableAuthenTraps.setDescription('Indicates whether the SNMP entity is permitt')
088 | snmpSilentDrops = MibScalar((1, 3, 6, 1, 2, 1, 11, 31), Counter32()).setMaxAccess("readonly")
089 | if mibBuilder.loadTexts: snmpSilentDrops.setStatus('current')
090 | if mibBuilder.loadTexts: snmpSilentDrops.setDescription('The total number of Confirmed Class PDUs (such as')
091 | snmpProxyDrops = MibScalar((1, 3, 6, 1, 2, 1, 11, 32), Counter32()).setMaxAccess("readonly")
092 | if mibBuilder.loadTexts: snmpProxyDrops.setStatus('current')
093 | if mibBuilder.loadTexts: snmpProxyDrops.setDescription('The total number of Confirmed Class PDUs (such as G')
094 | snmpTrap = MibIdentifier((1, 3, 6, 1, 6, 3, 1, 1, 4))
095 | snmpTrapOID = MibScalar((1, 3, 6, 1, 6, 3, 1, 1, 4, 1), ObjectIdentifier()).setMaxAccess("accessiblefornoti")
096 | if mibBuilder.loadTexts: snmpTrapOID.setStatus('current')
097 | if mibBuilder.loadTexts: snmpTrapOID.setDescription('The authoritative identification of the notification c')
098 | snmpTrapEnterprise = MibScalar((1, 3, 6, 1, 6, 3, 1, 1, 4, 3), ObjectIdentifier()).setMaxAccess("accessible")
099 | if mibBuilder.loadTexts: snmpTrapEnterprise.setStatus('current')
100 | if mibBuilder.loadTexts: snmpTrapEnterprise.setDescription('The authoritative identification of the enterpr')
101 | snmpTraps = MibIdentifier((1, 3, 6, 1, 6, 3, 1, 1, 5))
102 | coldStart = NotificationType((1, 3, 6, 1, 6, 3, 1, 1, 5, 1)).setObjects()
103 | if mibBuilder.loadTexts: coldStart.setStatus('current')
104 | if mibBuilder.loadTexts: coldStart.setDescription('A coldStart trap signifies that the SNMP entity, support')
105 | warmStart = NotificationType((1, 3, 6, 1, 6, 3, 1, 1, 5, 2)).setObjects()
106 | if mibBuilder.loadTexts: warmStart.setStatus('current')
107 | if mibBuilder.loadTexts: warmStart.setDescription('A warmStart trap signifies that the SNMP entity, support')
108 | authenticationFailure = NotificationType((1, 3, 6, 1, 6, 3, 1, 1, 5, 5)).setObjects()
109 | if mibBuilder.loadTexts: authenticationFailure.setStatus('current')
110 | if mibBuilder.loadTexts: authenticationFailure.setDescription('An authenticationFailure trap signifies that')
111 | snmpSet = MibIdentifier((1, 3, 6, 1, 6, 3, 1, 1, 6))
112 | snmpSetSerialNo = MibScalar((1, 3, 6, 1, 6, 3, 1, 1, 6, 1), TestAndIncr()).setMaxAccess("readwrite")
113 | if mibBuilder.loadTexts: snmpSetSerialNo.setStatus('current')
114 | if mibBuilder.loadTexts: snmpSetSerialNo.setDescription('An advisory lock used to allow several cooperating')
115 | snmpMIBConformance = MibIdentifier((1, 3, 6, 1, 6, 3, 1, 2))
116 | snmpMIBCompliances = MibIdentifier((1, 3, 6, 1, 6, 3, 1, 2, 1))
117 | snmpMIBGroups = MibIdentifier((1, 3, 6, 1, 6, 3, 1, 2, 2))
118 | snmpBasicCompliance = ModuleCompliance((1, 3, 6, 1, 6, 3, 1, 2, 1, 2)).setObjects(("SNMPv2-MIB", "snmpGroup")
119 | if mibBuilder.loadTexts: snmpBasicCompliance.setDescription('The compliance statement for SNMPv2 entities w')
120 | snmpBasicComplianceRev2 = ModuleCompliance((1, 3, 6, 1, 6, 3, 1, 2, 1, 3)).setObjects(("SNMPv2-MIB", "snmpG")
121 | if mibBuilder.loadTexts: snmpBasicComplianceRev2.setDescription('The compliance statement for SNMP entities')
122 | snmpGroup = ObjectGroup((1, 3, 6, 1, 6, 3, 1, 2, 2, 8)).setObjects(("SNMPv2-MIB", "snmpInPkts"), ("SNMPv2-M

```

```

123 | if mibBuilder.loadTexts: snmpGroup.setDescription('A collection of objects providing basic instrumentation
124 | snmpCommunityGroup = ObjectGroup((1, 3, 6, 1, 6, 3, 1, 2, 2, 9)).setObjects(("SNMPv2-MIB", "snmpInBadCommun
125 | if mibBuilder.loadTexts: snmpCommunityGroup.setDescription('A collection of objects providing basic instrum
126 | snmpSetGroup = ObjectGroup((1, 3, 6, 1, 6, 3, 1, 2, 2, 5)).setObjects(("SNMPv2-MIB", "snmpSetSerialNo"))
127 | if mibBuilder.loadTexts: snmpSetGroup.setDescription('A collection of objects which allow several cooperati
128 | systemGroup = ObjectGroup((1, 3, 6, 1, 6, 3, 1, 2, 2, 6)).setObjects(("SNMPv2-MIB", "sysDescr"), ("SNMPv2-M
129 | if mibBuilder.loadTexts: systemGroup.setDescription('The system group defines objects which are common to a
130 | snmpBasicNotificationsGroup = NotificationGroup((1, 3, 6, 1, 6, 3, 1, 2, 2, 7)).setObjects(("SNMPv2-MIB", "
131 | if mibBuilder.loadTexts: snmpBasicNotificationsGroup.setDescription('The basic notifications implemented by
132 | snmpWarmStartNotificationGroup = NotificationGroup((1, 3, 6, 1, 6, 3, 1, 2, 2, 11)).setObjects(("SNMPv2-MIB
133 | if mibBuilder.loadTexts: snmpWarmStartNotificationGroup.setDescription('An additional notification for an S
134 | snmpNotificationGroup = ObjectGroup((1, 3, 6, 1, 6, 3, 1, 2, 2, 12)).setObjects(("SNMPv2-MIB", "snmpTrapOID
135 | if mibBuilder.loadTexts: snmpNotificationGroup.setDescription('These objects are required for entities whic
136 | snmpOutPkts = MibScalar((1, 3, 6, 1, 2, 1, 11, 2), Counter32()).setMaxAccess("readonly")
137 | if mibBuilder.loadTexts: snmpOutPkts.setStatus('obsolete')
138 | if mibBuilder.loadTexts: snmpOutPkts.setDescription('The total number of SNMP Messages which were passed fr
139 | snmpInTooBigs = MibScalar((1, 3, 6, 1, 2, 1, 11, 8), Counter32()).setMaxAccess("readonly")
140 | if mibBuilder.loadTexts: snmpInTooBigs.setStatus('obsolete')
141 | if mibBuilder.loadTexts: snmpInTooBigs.setDescription('The total number of SNMP PDUs which were delivered t
142 | snmpInNoSuchNames = MibScalar((1, 3, 6, 1, 2, 1, 11, 9), Counter32()).setMaxAccess("readonly")
143 | if mibBuilder.loadTexts: snmpInNoSuchNames.setStatus('obsolete')
144 | if mibBuilder.loadTexts: snmpInNoSuchNames.setDescription('The total number of SNMP PDUs which were deliver
145 | snmpInBadValues = MibScalar((1, 3, 6, 1, 2, 1, 11, 10), Counter32()).setMaxAccess("readonly")
146 | if mibBuilder.loadTexts: snmpInBadValues.setStatus('obsolete')
147 | if mibBuilder.loadTexts: snmpInBadValues.setDescription('The total number of SNMP PDUs which were delivered
148 | snmpInReadOnlys = MibScalar((1, 3, 6, 1, 2, 1, 11, 11), Counter32()).setMaxAccess("readonly")
149 | if mibBuilder.loadTexts: snmpInReadOnlys.setStatus('obsolete')
150 | if mibBuilder.loadTexts: snmpInReadOnlys.setDescription('The total number valid SNMP PDUs which were delive
151 | snmpInGenErrs = MibScalar((1, 3, 6, 1, 2, 1, 11, 12), Counter32()).setMaxAccess("readonly")
152 | if mibBuilder.loadTexts: snmpInGenErrs.setStatus('obsolete')
153 | if mibBuilder.loadTexts: snmpInGenErrs.setDescription('The total number of SNMP PDUs which were delivered t
154 | snmpInTotalReqVars = MibScalar((1, 3, 6, 1, 2, 1, 11, 13), Counter32()).setMaxAccess("readonly")
155 | if mibBuilder.loadTexts: snmpInTotalReqVars.setStatus('obsolete')
156 | if mibBuilder.loadTexts: snmpInTotalReqVars.setDescription('The total number of MIB objects which have been
157 | snmpInTotalSetVars = MibScalar((1, 3, 6, 1, 2, 1, 11, 14), Counter32()).setMaxAccess("readonly")
158 | if mibBuilder.loadTexts: snmpInTotalSetVars.setStatus('obsolete')
159 | if mibBuilder.loadTexts: snmpInTotalSetVars.setDescription('The total number of MIB objects which have been
160 | snmpInGetRequests = MibScalar((1, 3, 6, 1, 2, 1, 11, 15), Counter32()).setMaxAccess("readonly")
161 | if mibBuilder.loadTexts: snmpInGetRequests.setStatus('obsolete')
162 | if mibBuilder.loadTexts: snmpInGetRequests.setDescription('The total number of SNMP Get-Request PDUs which
163 | snmpInGetNexts = MibScalar((1, 3, 6, 1, 2, 1, 11, 16), Counter32()).setMaxAccess("readonly")
164 | if mibBuilder.loadTexts: snmpInGetNexts.setStatus('obsolete')
165 | if mibBuilder.loadTexts: snmpInGetNexts.setDescription('The total number of SNMP Get-Next PDUs which have b
166 | snmpInSetRequests = MibScalar((1, 3, 6, 1, 2, 1, 11, 17), Counter32()).setMaxAccess("readonly")
167 | if mibBuilder.loadTexts: snmpInSetRequests.setStatus('obsolete')
168 | if mibBuilder.loadTexts: snmpInSetRequests.setDescription('The total number of SNMP Set-Request PDUs which
169 | snmpInGetResponses = MibScalar((1, 3, 6, 1, 2, 1, 11, 18), Counter32()).setMaxAccess("readonly")
170 | if mibBuilder.loadTexts: snmpInGetResponses.setStatus('obsolete')
171 | if mibBuilder.loadTexts: snmpInGetResponses.setDescription('The total number of SNMP Get-Response PDUs whic
172 | snmpInTraps = MibScalar((1, 3, 6, 1, 2, 1, 11, 19), Counter32()).setMaxAccess("readonly")
173 | if mibBuilder.loadTexts: snmpInTraps.setStatus('obsolete')
174 | if mibBuilder.loadTexts: snmpInTraps.setDescription('The total number of SNMP Trap PDUs which have been acc
175 | snmpOutTooBigs = MibScalar((1, 3, 6, 1, 2, 1, 11, 20), Counter32()).setMaxAccess("readonly")
176 | if mibBuilder.loadTexts: snmpOutTooBigs.setStatus('obsolete')
177 | if mibBuilder.loadTexts: snmpOutTooBigs.setDescription('The total number of SNMP PDUs which were generated
178 | snmpOutNoSuchNames = MibScalar((1, 3, 6, 1, 2, 1, 11, 21), Counter32()).setMaxAccess("readonly")
179 | if mibBuilder.loadTexts: snmpOutNoSuchNames.setStatus('obsolete')
180 | if mibBuilder.loadTexts: snmpOutNoSuchNames.setDescription('The total number of SNMP PDUs which were genera
181 | snmpOutBadValues = MibScalar((1, 3, 6, 1, 2, 1, 11, 22), Counter32()).setMaxAccess("readonly")
182 | if mibBuilder.loadTexts: snmpOutBadValues.setStatus('obsolete')
183 | if mibBuilder.loadTexts: snmpOutBadValues.setDescription('The total number of SNMP PDUs which were generate
184 | snmpOutGenErrs = MibScalar((1, 3, 6, 1, 2, 1, 11, 24), Counter32()).setMaxAccess("readonly")
185 | if mibBuilder.loadTexts: snmpOutGenErrs.setStatus('obsolete')

```

```

186 | if mibBuilder.loadTexts: snmpOutGenErrs.setDescription("The total number of SNMP PDUs which were generated
187 | snmpOutGetRequests = MibScalar((1, 3, 6, 1, 2, 1, 11, 25), Counter32()).setMaxAccess("readonly")
188 | if mibBuilder.loadTexts: snmpOutGetRequests.setStatus('obsolete')
189 | if mibBuilder.loadTexts: snmpOutGetRequests.setDescription('The total number of SNMP Get-Request PDUs which
190 | snmpOutGetNexts = MibScalar((1, 3, 6, 1, 2, 1, 11, 26), Counter32()).setMaxAccess("readonly")
191 | if mibBuilder.loadTexts: snmpOutGetNexts.setStatus('obsolete')
192 | if mibBuilder.loadTexts: snmpOutGetNexts.setDescription('The total number of SNMP Get-Next PDUs which have
193 | snmpOutSetRequests = MibScalar((1, 3, 6, 1, 2, 1, 11, 27), Counter32()).setMaxAccess("readonly")
194 | if mibBuilder.loadTexts: snmpOutSetRequests.setStatus('obsolete')
195 | if mibBuilder.loadTexts: snmpOutSetRequests.setDescription('The total number of SNMP Set-Request PDUs which
196 | snmpOutGetResponses = MibScalar((1, 3, 6, 1, 2, 1, 11, 28), Counter32()).setMaxAccess("readonly")
197 | if mibBuilder.loadTexts: snmpOutGetResponses.setStatus('obsolete')
198 | if mibBuilder.loadTexts: snmpOutGetResponses.setDescription('The total number of SNMP Get-Response PDUs whi
199 | snmpOutTraps = MibScalar((1, 3, 6, 1, 2, 1, 11, 29), Counter32()).setMaxAccess("readonly")
200 | if mibBuilder.loadTexts: snmpOutTraps.setStatus('obsolete')
201 | if mibBuilder.loadTexts: snmpOutTraps.setDescription('The total number of SNMP Trap PDUs which have been ge
202 | snmpObsoleteGroup = ObjectGroup((1, 3, 6, 1, 6, 3, 1, 2, 2, 10)).setObjects(("SNMPv2-MIB", "snmpOutPkts"),
203 | if mibBuilder.loadTexts: snmpObsoleteGroup.setDescription('A collection of objects from RFC 1213 made obsol
204 | mibBuilder.exportSymbols("SNMPv2-MIB", snmpOutBadValues=snmpOutBadValues, coldStart=coldStart, snmpOutPkts=

```

=== ARQUIVO:

.\dist\ControleImpressoras_internal\pysnmp\smi\mibs\SNMPv2-SMI.py ===

```

001 | #
002 | # This file is part of pysnmp software.
003 | #
004 | # Copyright (c) 2005-2019, Ilya Etingof <etingof@gmail.com>
005 | # License: http://snmplabs.com/pysnmp/license.html
006 | #
007 | import sys
008 | import traceback
009 | from pysnmp.smi.indices import OidOrderedDict
010 | from pysnmp.smi import exval, error
011 | from pysnmp.proto import rfc1902
012 | from pysnmp import cache, debug
013 | from pyasn1.type import univ
014 | from pyasn1.error import PyAsn1Error
015 |
016 | Integer, ObjectIdentifier = mibBuilder.importSymbols(
017 |     "ASN1", "Integer", "ObjectIdentifier"
018 | )
019 |
020 | (ConstraintsIntersection, ConstraintsUnion, SingleValueConstraint,
021 |  ValueRangeConstraint, ValueSizeConstraint) = mibBuilder.importSymbols(
022 |     "ASN1-REFINEMENT", "ConstraintsIntersection", "ConstraintsUnion",
023 |     "SingleValueConstraint", "ValueRangeConstraint", "ValueSizeConstraint"
024 | )
025 |
026 | # syntax of objects
027 |
028 | OctetString = rfc1902.OctetString
029 | Bits = rfc1902.Bits
030 | Integer32 = rfc1902.Integer32
031 | IPAddress = rfc1902.IPAddress
032 | Counter32 = rfc1902.Counter32
033 | Gauge32 = rfc1902.Gauge32
034 | Unsigned32 = rfc1902.Unsigned32
035 | TimeTicks = rfc1902.TimeTicks
036 | Opaque = rfc1902.Opaque
037 | Counter64 = rfc1902.Counter64

```

```

038 |
039 |
040 | class ExtUTCTime(OctetString):
041 |     subtypeSpec = OctetString.subtypeSpec + ConstraintsUnion(ValueSizeConstraint(11, 11), ValueSizeConstrai
042 |
043 |
044 | # MIB tree foundation class
045 |
046 | class MibNode(object):
047 |     label = ''
048 |
049 |     def __init__(self, name):
050 |         self.name = name
051 |
052 |     def __repr__(self):
053 |         return '%s(%r)' % (self.__class__.__name__, self.name)
054 |
055 |     def getName(self):
056 |         return self.name
057 |
058 |     def getLabel(self):
059 |         return self.label
060 |
061 |     def setLabel(self, label):
062 |         self.label = label
063 |         return self
064 |
065 |     def clone(self, name=None):
066 |         myClone = self.__class__(self.name)
067 |         if name is not None:
068 |             myClone.name = name
069 |         if self.label is not None:
070 |             myClone.label = self.label
071 |         return myClone
072 |
073 |
074 | # definitions for information modules
075 |
076 | class ModuleIdentity(MibNode):
077 |     status = 'current'
078 |     lastUpdated = ''
079 |     organization = ''
080 |     contactInfo = ''
081 |     description = ''
082 |     revisions = ()
083 |     revisionsDescriptions = ()
084 |
085 |     def getStatus(self):
086 |         return self.status
087 |
088 |     def setStatus(self, v):
089 |         self.status = v
090 |         return self
091 |
092 |     def getLastUpdated(self):
093 |         return self.lastUpdated
094 |
095 |     def setLastUpdated(self, v):
096 |         self.lastUpdated = v
097 |         return self
098 |
099 |     def getOrganization(self):
100 |         return self.organization

```

```

101 |
102 |     def setOrganization(self, v):
103 |         self.organization = v
104 |         return self
105 |
106 |     def getContactInfo(self):
107 |         return self.contactInfo
108 |
109 |     def setContactInfo(self, v):
110 |         self.contactInfo = v
111 |         return self
112 |
113 |     def getDescription(self):
114 |         return self.description
115 |
116 |     def setDescription(self, v):
117 |         self.description = v
118 |         return self
119 |
120 |     def getRevisions(self):
121 |         return self.revisions
122 |
123 |     def setRevisions(self, args):
124 |         self.revisions = args
125 |         return self
126 |
127 |     def getRevisionsDescriptions(self):
128 |         return self.revisionsDescriptions
129 |
130 |     def setRevisionsDescriptions(self, args):
131 |         self.revisionsDescriptions = args
132 |         return self
133 |
134 |     def asn1Print(self):
135 |         return """\
136 | MODULE-IDENTITY
137 |     LAST-UPDATED %s
138 |     ORGANIZATION "%s"
139 |     CONTACT-INFO "%s"
140 |     DESCRIPTION "%s"
141 |     %s"" % (self.getLastUpdated(),
142 |             self.getOrganization(),
143 |             self.getContactInfo(),
144 |             self.getDescription(),
145 |             ''.join(['REVISION "%s"\n' % x for x in self.getRevisions()]))
146 |
147 |
148 | class ObjectIdentity(MibNode):
149 |     status = 'current'
150 |     description = ''
151 |     reference = ''
152 |
153 |     def getStatus(self):
154 |         return self.status
155 |
156 |     def setStatus(self, v):
157 |         self.status = v
158 |         return self
159 |
160 |     def getDescription(self):
161 |         return self.description
162 |
163 |     def setDescription(self, v):

```

```

164 |         self.description = v
165 |         return self
166 |
167 |     def getReference(self):
168 |         return self.reference
169 |
170 |     def setReference(self, v):
171 |         self.reference = v
172 |         return self
173 |
174 |     def asn1Print(self):
175 |         return """\
176 | OBJECT-IDENTITY
177 | STATUS %s
178 | DESCRIPTION "%s"
179 | REFERENCE "%s"
180 | """ % (self.getStatus(),
181 |         self.getDescription(),
182 |         self.getReference())
183 |
184 |
185 | # definition for objects
186 |
187 | class NotificationType(MibNode):
188 |     objects = ()
189 |     status = 'current'
190 |     description = ''
191 |     reference = ''
192 |     # retained for compatibility
193 |     revisions = ()
194 |
195 |     def getObjects(self):
196 |         return self.objects
197 |
198 |     def setObjects(self, *args, **kwargs):
199 |         if kwargs.get('append'):
200 |             self.objects += args
201 |         else:
202 |             self.objects = args
203 |         return self
204 |
205 |     def getStatus(self):
206 |         return self.status
207 |
208 |     def setStatus(self, v):
209 |         self.status = v
210 |         return self
211 |
212 |     def getDescription(self):
213 |         return self.description
214 |
215 |     def setDescription(self, v):
216 |         self.description = v
217 |         return self
218 |
219 |     def getReference(self):
220 |         return self.reference
221 |
222 |     def setReference(self, v):
223 |         self.reference = v
224 |         return self
225 |
226 |     # This should not be here. Retained for compatibility.

```

```

227 |
228 |     def getRevisions(self):
229 |         return self.revisions
230 |
231 |     def setRevisions(self, v):
232 |         self.revisions = v
233 |         return self
234 |
235 |     def asn1Print(self):
236 |         return ""\
237 | NOTIFICATION-TYPE
238 |     OBJECTS { %s }
239 |     STATUS %s
240 |     DESCRIPTION "%s"
241 |     REFERENCE "%s"
242 |     """ % ('', '.join([x for x in self.getObjects()])),
243 |         self.getStatus(),
244 |         self.getDescription(),
245 |         self.getReference())
246 |
247 |
248 | class MibIdentifier(MibNode):
249 |     @staticmethod
250 |     def asn1Print():
251 |         return 'OBJECT IDENTIFIER'
252 |
253 |
254 | class ObjectType(MibNode):
255 |     units = ''
256 |     maxAccess = 'not-accessible'
257 |     status = 'current'
258 |     description = ''
259 |     reference = ''
260 |
261 |     def __init__(self, name, syntax=None):
262 |         MibNode.__init__(self, name)
263 |         self.syntax = syntax
264 |
265 |     # XXX
266 |     def __eq__(self, other):
267 |         return self.syntax == other
268 |
269 |     def __ne__(self, other):
270 |         return self.syntax != other
271 |
272 |     def __lt__(self, other):
273 |         return self.syntax < other
274 |
275 |     def __le__(self, other):
276 |         return self.syntax <= other
277 |
278 |     def __gt__(self, other):
279 |         return self.syntax > other
280 |
281 |     def __ge__(self, other):
282 |         return self.syntax >= other
283 |
284 |     def __repr__(self):
285 |         return '%s(%r, %r)' % (
286 |             self.__class__.__name__, self.name, self.syntax
287 |         )
288 |
289 |     def getSyntax(self):

```



```

290 |         return self.syntax
291 |
292 |     def setSyntax(self, v):
293 |         self.syntax = v
294 |         return self
295 |
296 |     def getUnits(self):
297 |         return self.units
298 |
299 |     def setUnits(self, v):
300 |         self.units = v
301 |         return self
302 |
303 |     def getMaxAccess(self):
304 |         return self.maxAccess
305 |
306 |     def setMaxAccess(self, v):
307 |         self.maxAccess = v
308 |         return self
309 |
310 |     def getStatus(self):
311 |         return self.status
312 |
313 |     def setStatus(self, v):
314 |         self.status = v
315 |         return self
316 |
317 |     def getDescription(self):
318 |         return self.description
319 |
320 |     def setDescription(self, v):
321 |         self.description = v
322 |         return self
323 |
324 |     def getReference(self):
325 |         return self.reference
326 |
327 |     def setReference(self, v):
328 |         self.reference = v
329 |         return self
330 |
331 |     def asn1Print(self):
332 |         return ""
333 |
334 | OBJECT-TYPE
335 |     SYNTAX %s
336 |     UNITS "%s"
337 |     MAX-ACCESS %s
338 |     STATUS %s
339 |     DESCRIPTION "%s"
340 |     REFERENCE "%s" "" % (self.getSyntax().__class__.__name__,
341 |                          self.getUnits(),
342 |                          self.getMaxAccess(),
343 |                          self.getStatus(),
344 |                          self.getDescription(),
345 |                          self.getReference())
346 |
347 | class MibTree(ObjectType):
348 |     branchVersionId = 0 # changes on tree structure change
349 |     maxAccess = 'not-accessible'
350 |
351 |     def __init__(self, name, syntax=None):
352 |         ObjectType.__init__(self, name, syntax)

```

```

353 |         self._vars = OidOrderedDict()
354 |
355 | # Subtrees registration
356 |
357 | def registerSubtrees(self, *subTrees):
358 |     self.branchVersionId += 1
359 |     for subTree in subTrees:
360 |         if subTree.name in self._vars:
361 |             raise error.SmiError(
362 |                 'MIB subtree %s already registered at %s' % (subTree.name, self)
363 |             )
364 |         self._vars[subTree.name] = subTree
365 |
366 | def unregisterSubtrees(self, *names):
367 |     self.branchVersionId += 1
368 |     for name in names:
369 |         # This may fail if you fill a table by exporting MibScalarInstances
370 |         # but later drop them through SNMP.
371 |         if name not in self._vars:
372 |             raise error.SmiError(
373 |                 'MIB subtree %s not registered at %s' % (name, self)
374 |             )
375 |         del self._vars[name]
376 |
377 | #
378 | # Tree traversal
379 | #
380 | # Missing branches are indicated by the NoSuchObjectError exception.
381 | # Although subtrees may indicate their missing branches by the
382 | # NoSuchInstanceError exception.
383 | #
384 |
385 | def getBranch(self, name, idx):
386 |     """Return a branch of this tree where the 'name' OID may reside"""
387 |     for keyLen in self._vars.getKeysLens():
388 |         subName = name[:keyLen]
389 |         if subName in self._vars:
390 |             return self._vars[subName]
391 |
392 |     raise error.NoSuchObjectError(name=name, idx=idx)
393 |
394 | def getNextBranch(self, name, idx=None):
395 |     # Start from the beginning
396 |     if self._vars:
397 |         first = list(self._vars.keys())[0]
398 |     else:
399 |         first = ()
400 |     if self._vars and name < first:
401 |         return self._vars[first]
402 |     else:
403 |         try:
404 |             return self._vars[self._vars.nextKey(name)]
405 |         except KeyError:
406 |             raise error.NoSuchObjectError(idx=idx, name=name)
407 |
408 | def getNode(self, name, idx=None):
409 |     """Return tree node found by name"""
410 |     if name == self.name:
411 |         return self
412 |     else:
413 |         return self.getBranch(name, idx).getNode(name, idx)
414 |
415 | def getNextNode(self, name, idx=None):

```

```

416 |         """Return tree node next to name"""
417 |     try:
418 |         nextNode = self.getBranch(name, idx)
419 |     except (error.NoSuchInstanceError, error.NoSuchObjectError):
420 |         return self.getNextBranch(name, idx)
421 |     else:
422 |         try:
423 |             return nextNode.getNextNode(name, idx)
424 |         except (error.NoSuchInstanceError, error.NoSuchObjectError):
425 |             try:
426 |                 return self._vars[self._vars.nextKey(nextNode.name)]
427 |             except KeyError:
428 |                 raise error.NoSuchObjectError(idx=idx, name=name)
429 |
430 | # MIB instrumentation
431 |
432 | # Read operation
433 |
434 | def readTest(self, name, val, idx, acInfo):
435 |     (acFun, acCtx) = acInfo
436 |     if name == self.name:
437 |         if acFun:
438 |             if self.maxAccess not in ('readonly',
439 |                                       'readwrite', 'readcreate') or \
440 |                 acFun(name, self.syntax, idx, 'read', acCtx):
441 |                 raise error.NoAccessError(idx=idx, name=name)
442 |         else:
443 |             try:
444 |                 node = self.getBranch(name, idx)
445 |             except (error.NoSuchInstanceError, error.NoSuchObjectError):
446 |                 return # missing object is not an error here
447 |             else:
448 |                 node.readTest(name, val, idx, acInfo)
449 |
450 | def readGet(self, name, val, idx, acInfo):
451 |     try:
452 |         node = self.getBranch(name, idx)
453 |     except (error.NoSuchInstanceError, error.NoSuchObjectError):
454 |         return name, exval.noSuchObject
455 |     else:
456 |         return node.readGet(name, val, idx, acInfo)
457 |
458 | # Read next operation is subtree-specific
459 |
460 | depthFirst, breadthFirst = 0, 1
461 |
462 | def readTestNext(self, name, val, idx, acInfo, oName=None):
463 |     if oName is None:
464 |         oName = name
465 |         topOfTheMib = True
466 |     else:
467 |         topOfTheMib = False
468 |     nextName = name
469 |     direction = self.depthFirst
470 |     while 1: # XXX linear search here
471 |         if direction == self.depthFirst:
472 |             direction = self.breadthFirst
473 |             try:
474 |                 node = self.getBranch(nextName, idx)
475 |             except (error.NoSuchInstanceError, error.NoSuchObjectError):
476 |                 continue
477 |         else:
478 |             try:

```

```

479 |         node = self.getNextBranch(nextName, idx)
480 |     except (error.NoSuchInstanceError, error.NoSuchObjectError):
481 |         if topOfTheMib:
482 |             return
483 |         raise
484 |     direction = self.depthFirst
485 |     nextName = node.name
486 |     try:
487 |         return node.readTestNext(nextName, val, idx, acInfo, oName)
488 |     except (error.NoAccessError, error.NoSuchInstanceError, error.NoSuchObjectError):
489 |         pass
490 |
491 | def readGetNext(self, name, val, idx, acInfo, oName=None):
492 |     if oName is None:
493 |         oName = name
494 |         topOfTheMib = True
495 |     else:
496 |         topOfTheMib = False
497 |     nextName = name
498 |     direction = self.depthFirst
499 |     while True: # XXX linear search ahead!
500 |         if direction == self.depthFirst:
501 |             direction = self.breadthFirst
502 |             try:
503 |                 node = self.getBranch(nextName, idx)
504 |             except (error.NoSuchInstanceError, error.NoSuchObjectError):
505 |                 continue
506 |         else:
507 |             try:
508 |                 node = self.getNextBranch(nextName, idx)
509 |             except (error.NoSuchInstanceError, error.NoSuchObjectError):
510 |                 if topOfTheMib:
511 |                     return name, exval.endOfMib
512 |                 raise
513 |             direction = self.depthFirst
514 |             nextName = node.name
515 |         try:
516 |             return node.readGetNext(nextName, val, idx, acInfo, oName)
517 |         except (error.NoAccessError, error.NoSuchInstanceError, error.NoSuchObjectError):
518 |             pass
519 |
520 | # Write operation
521 |
522 | def writeTest(self, name, val, idx, acInfo):
523 |     acFun, acCtx = acInfo
524 |     if name == self.name:
525 |         # Make sure variable is writable
526 |         if acFun:
527 |             if self.maxAccess not in ('readwrite', 'readcreate') or \
528 |                acFun(name, self.syntax, idx, 'write', acCtx):
529 |                 raise error.NotWritableError(idx=idx, name=name)
530 |     else:
531 |         node = self.getBranch(name, idx)
532 |         node.writeTest(name, val, idx, acInfo)
533 |
534 | def writeCommit(self, name, val, idx, acInfo):
535 |     self.getBranch(name, idx).writeCommit(name, val, idx, acInfo)
536 |
537 | def writeCleanup(self, name, val, idx, acInfo):
538 |     self.branchVersionId += 1
539 |     self.getBranch(name, idx).writeCleanup(name, val, idx, acInfo)
540 |
541 | def writeUndo(self, name, val, idx, acInfo):

```

```

542 |         self.getBranch(name, idx).writeUndo(name, val, idx, acInfo)
543 |
544 |
545 | class MibScalar(MibTree):
546 |     """Scalar MIB variable. Implements access control checking."""
547 |     maxAccess = 'readonly'
548 |
549 |     #
550 |     # Subtree traversal
551 |     #
552 |     # Missing branches are indicated by the NoSuchInstanceError exception.
553 |     #
554 |
555 |     def getBranch(self, name, idx):
556 |         try:
557 |             return MibTree.getBranch(self, name, idx)
558 |         except (error.NoSuchInstanceError, error.NoSuchObjectError):
559 |             raise error.NoSuchInstanceError(idx=idx, name=name)
560 |
561 |     def getNextBranch(self, name, idx=None):
562 |         try:
563 |             return MibTree.getNextBranch(self, name, idx)
564 |         except (error.NoSuchInstanceError, error.NoSuchObjectError):
565 |             raise error.NoSuchInstanceError(idx=idx, name=name)
566 |
567 |     def getNode(self, name, idx=None):
568 |         try:
569 |             return MibTree.getNode(self, name, idx)
570 |         except (error.NoSuchInstanceError, error.NoSuchObjectError):
571 |             raise error.NoSuchInstanceError(idx=idx, name=name)
572 |
573 |     def getNextNode(self, name, idx=None):
574 |         try:
575 |             return MibTree.getNextNode(self, name, idx)
576 |         except (error.NoSuchInstanceError, error.NoSuchObjectError):
577 |             raise error.NoSuchInstanceError(idx=idx, name=name)
578 |
579 |     # MIB instrumentation methods
580 |
581 |     # Read operation
582 |
583 |     def readTest(self, name, val, idx, acInfo):
584 |         (acFun, acCtx) = acInfo
585 |         if name == self.name:
586 |             raise error.NoAccessError(idx=idx, name=name)
587 |         if acFun:
588 |             if self.maxAccess not in ('readonly', 'readwrite',
589 |                                       'readcreate') or \
590 |                 acFun(name, self.syntax, idx, 'read', acCtx):
591 |                 raise error.NoAccessError(idx=idx, name=name)
592 |         MibTree.readTest(self, name, val, idx, acInfo)
593 |
594 |     def readGet(self, name, val, idx, acInfo):
595 |         try:
596 |             node = self.getBranch(name, idx)
597 |         except error.NoSuchInstanceError:
598 |             return name, exval.noSuchInstance
599 |         else:
600 |             return node.readGet(name, val, idx, acInfo)
601 |
602 |     def readTestNext(self, name, val, idx, acInfo, oName=None):
603 |         (acFun, acCtx) = acInfo
604 |         if acFun:

```

```

605 |         if self.maxAccess not in ('readonly', 'readwrite',
606 |                                   'readcreate') or \
607 |             acFun(name, self.syntax, idx, 'read', acCtx):
608 |             raise error.NoAccessError(idx=idx, name=name)
609 |         MibTree.readTestNext(self, name, val, idx, acInfo, oName)
610 |
611 |     def readGetNext(self, name, val, idx, acInfo, oName=None):
612 |         (acFun, acCtx) = acInfo
613 |         # have to duplicate AC here as *Next code above treats
614 |         # noAccess as a noSuchObject at the Test stage, goes on
615 |         # to Reading
616 |         if acFun:
617 |             if self.maxAccess not in ('readonly', 'readwrite',
618 |                                       'readcreate') or \
619 |                 acFun(name, self.syntax, idx, 'read', acCtx):
620 |                 raise error.NoAccessError(idx=idx, name=name)
621 |             return MibTree.readGetNext(self, name, val, idx, acInfo, oName)
622 |
623 |     # Two-phase commit implementation
624 |
625 |     def writeTest(self, name, val, idx, acInfo):
626 |         acFun, acCtx = acInfo
627 |         if name == self.name:
628 |             raise error.NoAccessError(idx=idx, name=name)
629 |         if acFun:
630 |             if self.maxAccess not in ('readwrite', 'readcreate') or \
631 |                 acFun(name, self.syntax, idx, 'write', acCtx):
632 |                 raise error.NotWritableError(idx=idx, name=name)
633 |             MibTree.writeTest(self, name, val, idx, acInfo)
634 |
635 |
636 | class MibScalarInstance(MibTree):
637 |     """Scalar MIB variable instance. Implements read/write operations."""
638 |
639 |     def __init__(self, typeName, instId, syntax):
640 |         MibTree.__init__(self, typeName + instId, syntax)
641 |         self.typeName = typeName
642 |         self.instId = instId
643 |         self.__oldSyntax = None
644 |
645 |     #
646 |     # Managed object value access methods
647 |     #
648 |
649 |     # noinspection PyUnusedLocal
650 |     def getValue(self, name, idx):
651 |         debug.logger & debug.flagIns and debug.logger('getValue: returning %r for %s' % (self.syntax, self.
652 |         return self.syntax.clone()
653 |
654 |     def setValue(self, value, name, idx):
655 |         if value is None:
656 |             value = univ.noValue
657 |         try:
658 |             if hasattr(self.syntax, 'setValue'):
659 |                 return self.syntax.setValue(value)
660 |             else:
661 |                 return self.syntax.clone(value)
662 |         except PyAsn1Error:
663 |             exc_t, exc_v, exc_tb = sys.exc_info()
664 |             debug.logger & debug.flagIns and debug.logger('setValue: %s=%r failed with traceback %s' % (
665 |                 self.name, value, traceback.format_exception(exc_t, exc_v, exc_tb)))
666 |             if isinstance(exc_v, error.TableRowManagement):
667 |                 raise exc_v

```

```

668 |         else:
669 |             raise error.WrongValueError(idx=idx, name=name, msg=exc_v)
670 |
671 |     #
672 |     # Subtree traversal
673 |     #
674 |     # Missing branches are indicated by the NoSuchInstanceError exception.
675 |     #
676 |
677 |     def getBranch(self, name, idx):
678 |         try:
679 |             return MibTree.getBranch(self, name, idx)
680 |         except (error.NoSuchInstanceError, error.NoSuchObjectError):
681 |             raise error.NoSuchInstanceError(idx=idx, name=name)
682 |
683 |     def getNextBranch(self, name, idx=None):
684 |         try:
685 |             return MibTree.getNextBranch(self, name, idx)
686 |         except (error.NoSuchInstanceError, error.NoSuchObjectError):
687 |             raise error.NoSuchInstanceError(idx=idx, name=name)
688 |
689 |     def getNode(self, name, idx=None):
690 |         # Recursion terminator
691 |         if name == self.name:
692 |             return self
693 |         raise error.NoSuchInstanceError(idx=idx, name=name)
694 |
695 |     def getNextNode(self, name, idx=None):
696 |         raise error.NoSuchInstanceError(idx=idx, name=name)
697 |
698 |     # MIB instrumentation methods
699 |
700 |     # Read operation
701 |
702 |     def readTest(self, name, val, idx, acInfo):
703 |         if name != self.name:
704 |             raise error.NoSuchInstanceError(idx=idx, name=name)
705 |
706 |     def readGet(self, name, val, idx, acInfo):
707 |         # Return current variable (name, value)
708 |         if name == self.name:
709 |             debug.logger & debug.flagIns and debug.logger('readGet: %s=%r' % (self.name, self.syntax))
710 |             return self.name, self.getValue(name, idx)
711 |         else:
712 |             raise error.NoSuchInstanceError(idx=idx, name=name)
713 |
714 |     def readTestNext(self, name, val, idx, acInfo, oName=None):
715 |         if name != self.name or name <= oName:
716 |             raise error.NoSuchInstanceError(idx=idx, name=name)
717 |
718 |     def readGetNext(self, name, val, idx, acInfo, oName=None):
719 |         if name == self.name and name > oName:
720 |             debug.logger & debug.flagIns and debug.logger('readGetNext: %s=%r' % (self.name, self.syntax))
721 |             return self.readGet(name, val, idx, acInfo)
722 |         else:
723 |             raise error.NoSuchInstanceError(idx=idx, name=name)
724 |
725 |     # Write operation: two-phase commit
726 |
727 |     # noinspection PyAttributeOutsideInit
728 |     def writeTest(self, name, val, idx, acInfo):
729 |         # Make sure write's allowed
730 |         if name == self.name:

```

```

731 |         try:
732 |             self.__newSyntax = self.setValue(val, name, idx)
733 |         except error.MibOperationError:
734 |             # SMI exceptions may carry additional content
735 |             why = sys.exc_info()[1]
736 |             if 'syntax' in why:
737 |                 self.__newSyntax = why['syntax']
738 |                 raise why
739 |             else:
740 |                 raise error.WrongValueError(idx=idx, name=name, msg=sys.exc_info()[1])
741 |         else:
742 |             raise error.NoSuchInstanceError(idx=idx, name=name)
743 |
744 |     def writeCommit(self, name, val, idx, acInfo):
745 |         # Backup original value
746 |         if self.__oldSyntax is None:
747 |             self.__oldSyntax = self.syntax
748 |         # Commit new value
749 |         self.syntax = self.__newSyntax
750 |
751 |     # noinspection PyAttributeOutsideInit
752 |     def writeCleanup(self, name, val, idx, acInfo):
753 |         self.branchVersionId += 1
754 |         debug.logger & debug.flagIns and debug.logger('writeCleanup: %s=%r' % (name, val))
755 |         # Drop previous value
756 |         self.__newSyntax = self.__oldSyntax = None
757 |
758 |     # noinspection PyAttributeOutsideInit
759 |     def writeUndo(self, name, val, idx, acInfo):
760 |         # Revive previous value
761 |         self.syntax = self.__oldSyntax
762 |         self.__newSyntax = self.__oldSyntax = None
763 |
764 |     # Table column instance specifics
765 |
766 |     # Create operation
767 |
768 |     # noinspection PyUnusedLocal,PyAttributeOutsideInit
769 |     def createTest(self, name, val, idx, acInfo):
770 |         if name == self.name:
771 |             try:
772 |                 self.__newSyntax = self.setValue(val, name, idx)
773 |             except error.MibOperationError:
774 |                 # SMI exceptions may carry additional content
775 |                 why = sys.exc_info()[1]
776 |                 if 'syntax' in why:
777 |                     self.__newSyntax = why['syntax']
778 |                 else:
779 |                     raise error.WrongValueError(idx=idx, name=name, msg=sys.exc_info()[1])
780 |             else:
781 |                 raise error.NoSuchInstanceError(idx=idx, name=name)
782 |
783 |     def createCommit(self, name, val, idx, acInfo):
784 |         if val is not None:
785 |             self.writeCommit(name, val, idx, acInfo)
786 |
787 |     def createCleanup(self, name, val, idx, acInfo):
788 |         self.branchVersionId += 1
789 |         debug.logger & debug.flagIns and debug.logger('createCleanup: %s=%r' % (name, val))
790 |         if val is not None:
791 |             self.writeCleanup(name, val, idx, acInfo)
792 |
793 |     def createUndo(self, name, val, idx, acInfo):

```



```

794 |         if val is not None:
795 |             self.writeUndo(name, val, idx, acInfo)
796 |
797 |     # Destroy operation
798 |
799 |     # noinspection PyUnusedLocal,PyAttributeOutsideInit
800 |     def destroyTest(self, name, val, idx, acInfo):
801 |         if name == self.name:
802 |             try:
803 |                 self.__newSyntax = self.setValue(val, name, idx)
804 |             except error.MibOperationError:
805 |                 # SMI exceptions may carry additional content
806 |                 why = sys.exc_info()[1]
807 |                 if 'syntax' in why:
808 |                     self.__newSyntax = why['syntax']
809 |             else:
810 |                 raise error.NoSuchInstanceError(idx=idx, name=name)
811 |
812 |     def destroyCommit(self, name, val, idx, acInfo):
813 |         pass
814 |
815 |     # noinspection PyUnusedLocal
816 |     def destroyCleanup(self, name, val, idx, acInfo):
817 |         self.branchVersionId += 1
818 |
819 |     def destroyUndo(self, name, val, idx, acInfo):
820 |         pass
821 |
822 |
823 | # Conceptual table classes
824 |
825 | class MibTableColumn(MibScalar):
826 |     """MIB table column. Manages a set of column instance variables"""
827 |     protoInstance = MibScalarInstance
828 |
829 |     def __init__(self, name, syntax):
830 |         MibScalar.__init__(self, name, syntax)
831 |         self.__createdInstances = {}
832 |         self.__destroyedInstances = {}
833 |         self.__rowOpWanted = {}
834 |
835 |     #
836 |     # Subtree traversal
837 |     #
838 |     # Missing leaves are indicated by the NoSuchInstanceError exception.
839 |     #
840 |
841 |     def getBranch(self, name, idx):
842 |         if name in self._vars:
843 |             return self._vars[name]
844 |         raise error.NoSuchInstanceError(name=name, idx=idx)
845 |
846 |     def setProtoInstance(self, protoInstance):
847 |         self.protoInstance = protoInstance
848 |
849 |     # Column creation (this should probably be converted into some state
850 |     # machine for clarity). Also, it might be a good idea to indicate
851 |     # defaulted cols creation in a clearer way than just a val == None.
852 |
853 |     def createTest(self, name, val, idx, acInfo):
854 |         (acFun, acCtx) = acInfo
855 |         # Make sure creation allowed, create a new column instance but
856 |         # do not replace the old one

```

```

857 |         if name == self.name:
858 |             raise error.NoAccessError(idx=idx, name=name)
859 |         if acFun:
860 |             if val is not None and self.maxAccess != 'readcreate' or \
861 |                 acFun(name, self.syntax, idx, 'write', acCtx):
862 |                 debug.logger & debug.flagACL and debug.logger(
863 |                     'createTest: %s=%r %s at %s' % (name, val, self.maxAccess, self.name))
864 |                 raise error.NoCreationError(idx=idx, name=name)
865 |             # Create instances if either it does not yet exist (row creation)
866 |             # or a value is passed (multiple OIDs in SET PDU)
867 |             if val is None and name in self.__createdInstances:
868 |                 return
869 |             self.__createdInstances[name] = self.protoInstance(
870 |                 self.name, name[len(self.name):], self.syntax.clone()
871 |             )
872 |             self.__createdInstances[name].createTest(name, val, idx, acInfo)
873 |
874 |     def createCommit(self, name, val, idx, acInfo):
875 |         # Commit new instance value
876 |         if name in self._vars: # XXX
877 |             if name in self.__createdInstances:
878 |                 self._vars[name].createCommit(name, val, idx, acInfo)
879 |             return
880 |             self.__createdInstances[name].createCommit(name, val, idx, acInfo)
881 |         # ...commit new column instance
882 |         self._vars[name], self.__createdInstances[name] = \
883 |             self.__createdInstances[name], self._vars.get(name)
884 |
885 |     def createCleanup(self, name, val, idx, acInfo):
886 |         # Drop previous column instance
887 |         self.branchVersionId += 1
888 |         if name in self.__createdInstances:
889 |             if self.__createdInstances[name] is not None:
890 |                 self.__createdInstances[name].createCleanup(name, val, idx,
891 |                                                             acInfo)
892 |             del self.__createdInstances[name]
893 |         elif name in self._vars:
894 |             self._vars[name].createCleanup(name, val, idx, acInfo)
895 |
896 |     def createUndo(self, name, val, idx, acInfo):
897 |         # Set back previous column instance, drop the new one
898 |         if name in self.__createdInstances:
899 |             self._vars[name] = self.__createdInstances[name]
900 |             del self.__createdInstances[name]
901 |             # Remove new instance on rollback
902 |             if self._vars[name] is None:
903 |                 del self._vars[name]
904 |             else:
905 |                 # Catch half-created instances (hackerish)
906 |                 try:
907 |                     self._vars[name] == 0
908 |                 except PyAsn1Error:
909 |                     del self._vars[name]
910 |                 else:
911 |                     self._vars[name].createUndo(name, val, idx, acInfo)
912 |
913 |     # Column destruction
914 |
915 |     def destroyTest(self, name, val, idx, acInfo):
916 |         (acFun, acCtx) = acInfo
917 |         # Make sure destruction is allowed
918 |         if name == self.name:
919 |             raise error.NoAccessError(idx=idx, name=name)

```

```

920 |         if name not in self._vars:
921 |             return
922 |         if acFun:
923 |             if val is not None and self.maxAccess != 'readcreate' or \
924 |                 acFun(name, self.syntax, idx, 'write', acCtx):
925 |                 raise error.NoAccessError(idx=idx, name=name)
926 |             self._vars[name].destroyTest(name, val, idx, acInfo)
927 |
928 |     def destroyCommit(self, name, val, idx, acInfo):
929 |         # Make a copy of column instance and take it off the tree
930 |         if name in self._vars:
931 |             self._vars[name].destroyCommit(name, val, idx, acInfo)
932 |             self.__destroyedInstances[name] = self._vars[name]
933 |             del self._vars[name]
934 |
935 |     def destroyCleanup(self, name, val, idx, acInfo):
936 |         # Drop instance copy
937 |         self.branchVersionId += 1
938 |         if name in self.__destroyedInstances:
939 |             self.__destroyedInstances[name].destroyCleanup(name, val,
940 |                                                             idx, acInfo)
941 |             debug.logger & debug.flagIns and debug.logger('destroyCleanup: %s=%r' % (name, val))
942 |             del self.__destroyedInstances[name]
943 |
944 |     def destroyUndo(self, name, val, idx, acInfo):
945 |         # Set back column instance
946 |         if name in self.__destroyedInstances:
947 |             self._vars[name] = self.__destroyedInstances[name]
948 |             self._vars[name].destroyUndo(name, val, idx, acInfo)
949 |             del self.__destroyedInstances[name]
950 |
951 |     # Set/modify column
952 |
953 |     def writeTest(self, name, val, idx, acInfo):
954 |         # Besides common checks, request row creation on no-instance
955 |         try:
956 |             # First try the instance
957 |             MibScalar.writeTest(self, name, val, idx, acInfo)
958 |             # ...otherwise proceed with creating new column
959 |         except (error.NoSuchInstanceError, error.RowCreationWanted):
960 |             excValue = sys.exc_info()[1]
961 |             if isinstance(excValue, error.RowCreationWanted):
962 |                 self.__rowOpWanted[name] = excValue
963 |             else:
964 |                 self.__rowOpWanted[name] = error.RowCreationWanted()
965 |                 self.createTest(name, val, idx, acInfo)
966 |         except error.RowDestructionWanted:
967 |             self.__rowOpWanted[name] = error.RowDestructionWanted()
968 |             self.destroyTest(name, val, idx, acInfo)
969 |         if name in self.__rowOpWanted:
970 |             debug.logger & debug.flagIns and debug.logger(
971 |                 '%s flagged by %s=%r, exception %s' % (self.__rowOpWanted[name], name, val, sys.exc_info()[1])
972 |             )
973 |             raise self.__rowOpWanted[name]
974 |
975 |     def __delegateWrite(self, subAction, name, val, idx, acInfo):
976 |         if name not in self.__rowOpWanted:
977 |             getattr(MibScalar, 'write' + subAction)(self, name, val, idx, acInfo)
978 |             return
979 |         if isinstance(self.__rowOpWanted[name], error.RowCreationWanted):
980 |             getattr(self, 'create' + subAction)(name, val, idx, acInfo)
981 |         if isinstance(self.__rowOpWanted[name], error.RowDestructionWanted):
982 |             getattr(self, 'destroy' + subAction)(name, val, idx, acInfo)

```

```

983 |     def writeCommit(self, name, val, idx, acInfo):
984 |         self.__delegateWrite('Commit', name, val, idx, acInfo)
985 |         if name in self.__rowOpWanted:
986 |             raise self.__rowOpWanted[name]
987 |
988 |     def writeCleanup(self, name, val, idx, acInfo):
989 |         self.branchVersionId += 1
990 |         self.__delegateWrite('Cleanup', name, val, idx, acInfo)
991 |         if name in self.__rowOpWanted:
992 |             e = self.__rowOpWanted[name]
993 |             del self.__rowOpWanted[name]
994 |             debug.logger & debug.flagIns and debug.logger('%s dropped by %s=%r' % (e, name, val))
995 |             raise e
996 |
997 |     def writeUndo(self, name, val, idx, acInfo):
998 |         if name in self.__rowOpWanted:
999 |             self.__rowOpWanted[name] = error.RowDestructionWanted()
1000 |             self.__delegateWrite('Undo', name, val, idx, acInfo)
1001 |             if name in self.__rowOpWanted:
1002 |                 e = self.__rowOpWanted[name]
1003 |                 del self.__rowOpWanted[name]
1004 |                 debug.logger & debug.flagIns and debug.logger('%s dropped by %s=%r' % (e, name, val))
1005 |                 raise e
1006 |
1007 |
1008 | class MibTableRow(MibTree):
1009 |     """MIB table row (SMI 'Entry'). Manages a set of table columns.
1010 |     Implements row creation/destruction.
1011 |     """
1012 |
1013 |     def __init__(self, name):
1014 |         MibTree.__init__(self, name)
1015 |         self.__idToIdxCache = cache.Cache()
1016 |         self.__idxToIdCache = cache.Cache()
1017 |         self.indexNames = ()
1018 |         self.augmentingRows = {}
1019 |
1020 |     # Table indices resolution. Handle almost all possible rfc1902 types
1021 |     # explicitly rather than by means of isSuperTypeOf() method because
1022 |     # some subtypes may be implicitly tagged what renders base tag
1023 |     # unavailable.
1024 |
1025 |     __intBaseTag = Integer.tagSet.getBaseTag()
1026 |     __strBaseTag = OctetString.tagSet.getBaseTag()
1027 |     __oidBaseTag = ObjectIdentifier.tagSet.getBaseTag()
1028 |     __ipaddrTagSet = IpAddress.tagSet
1029 |     __bitsBaseTag = Bits.tagSet.getBaseTag()
1030 |
1031 |     def setFromName(self, obj, value, impliedFlag=None, parentIndices=None):
1032 |         if not value:
1033 |             raise error.SmiError('Short OID for index %r' % (obj,))
1034 |         if hasattr(obj, 'cloneFromName'):
1035 |             return obj.cloneFromName(value, impliedFlag, parentRow=self, parentIndices=parentIndices)
1036 |         baseTag = obj.getTagSet().getBaseTag()
1037 |         if baseTag == self.__intBaseTag:
1038 |             return obj.clone(value[0]), value[1:]
1039 |         elif self.__ipaddrTagSet.isSuperTagSetOf(obj.getTagSet()):
1040 |             return obj.clone('.'.join([str(x) for x in value[:4]]), value[4:])
1041 |         elif baseTag == self.__strBaseTag:
1042 |             # rfc1902, 7.7
1043 |             if impliedFlag:
1044 |                 return obj.clone(tuple(value)), ()
1045 |             elif obj.isFixedLength():

```

```

1046 |         l = obj.getFixedLength()
1047 |         return obj.clone(tuple(value[:l])), value[l:]
1048 |     else:
1049 |         return obj.clone(tuple(value[1:value[0] + 1])), value[value[0] + 1:]
1050 | elif baseTag == self.__oidBaseTag:
1051 |     if impliedFlag:
1052 |         return obj.clone(value), ()
1053 |     else:
1054 |         return obj.clone(value[1:value[0] + 1]), value[value[0] + 1:]
1055 | # rfc2578, 7.1
1056 | elif baseTag == self.__bitsBaseTag:
1057 |     return obj.clone(tuple(value[1:value[0] + 1])), value[value[0] + 1:]
1058 | else:
1059 |     raise error.SmiError('Unknown value type for index %r' % (obj,))
1060 |
1061 | def getAsName(self, obj, impliedFlag=None, parentIndices=None):
1062 |     if hasattr(obj, 'cloneAsName'):
1063 |         return obj.cloneAsName(impliedFlag, parentRow=self, parentIndices=parentIndices)
1064 |     baseTag = obj.getTagSet().getBaseTag()
1065 |     if baseTag == self.__intBaseTag:
1066 |         # noinspection PyRedundantParentheses
1067 |         return (int(obj),)
1068 |     elif self.__ipaddrTagSet.isSuperTagSetOf(obj.getTagSet()):
1069 |         return obj.asNumbers()
1070 |     elif baseTag == self.__strBaseTag:
1071 |         if impliedFlag or obj.isFixedLength():
1072 |             initial = ()
1073 |         else:
1074 |             initial = (len(obj),)
1075 |         return initial + obj.asNumbers()
1076 |     elif baseTag == self.__oidBaseTag:
1077 |         if impliedFlag:
1078 |             return tuple(obj)
1079 |         else:
1080 |             return (len(obj),) + tuple(obj)
1081 |     # rfc2578, 7.1
1082 |     elif baseTag == self.__bitsBaseTag:
1083 |         return (len(obj),) + obj.asNumbers()
1084 |     else:
1085 |         raise error.SmiError('Unknown value type for index %r' % (obj,))
1086 |
1087 | # Fate sharing mechanics
1088 |
1089 | def announceManagementEvent(self, action, name, val, idx, acInfo):
1090 |     # Convert OID suffix into index vals
1091 |     instId = name[len(self.name) + 1:]
1092 |     baseIndices = []
1093 |     indices = []
1094 |     for impliedFlag, modName, symName in self.indexNames:
1095 |         mibObj, = mibBuilder.importSymbols(modName, symName)
1096 |         syntax, instId = self.setFromName(mibObj.syntax, instId,
1097 |                                           impliedFlag, indices)
1098 |
1099 |         if self.name == mibObj.name[:-1]:
1100 |             baseIndices.append((mibObj.name, syntax))
1101 |
1102 |         indices.append(syntax)
1103 |
1104 |     if instId:
1105 |         raise error.SmiError('Excessive instance identifier sub-OIDs left at %s: %s' % (self, instId))
1106 |
1107 |     if not baseIndices:
1108 |         return

```

[illegible]

```

1172 |         getattr(var, action)(name + nameSuffix, val, idx, acInfo)
1173 |
1174 |         debug.logger & debug.flagIns and debug.logger('__manageColumns: action %s name %s suffix %s %s'
1175 |             action, name, nameSuffix, name in indexVals and "index " or "", indexVals.get(name, val)))
1176 |
1177 |     def __delegate(self, subAction, name, val, idx, acInfo):
1178 |         # Relay operation request to column, expect row operation request.
1179 |         rowIsActive = False
1180 |         try:
1181 |             getattr(self.getBranch(name, idx), 'write' + subAction)(
1182 |                 name, val, idx, acInfo
1183 |             )
1184 |
1185 |         except error.RowCreationWanted:
1186 |             self.__manageColumns(
1187 |                 'create' + subAction, name[:len(self.name) + 1],
1188 |                 name[len(self.name) + 1:], None, idx, acInfo
1189 |             )
1190 |
1191 |             self.announceManagementEvent(
1192 |                 'create' + subAction, name, None, idx, acInfo
1193 |             )
1194 |
1195 |             # watch for RowStatus == 'stActive'
1196 |             rowIsActive = sys.exc_info()[1].get('syntax', 0) == 1
1197 |
1198 |         except error.RowDestructionWanted:
1199 |             self.__manageColumns(
1200 |                 'destroy' + subAction, name[:len(self.name) + 1],
1201 |                 name[len(self.name) + 1:], None, idx, acInfo
1202 |             )
1203 |
1204 |             self.announceManagementEvent(
1205 |                 'destroy' + subAction, name, None, idx, acInfo
1206 |             )
1207 |
1208 |         return rowIsActive
1209 |
1210 |     def writeTest(self, name, val, idx, acInfo):
1211 |         self.__delegate('Test', name, val, idx, acInfo)
1212 |
1213 |     def writeCommit(self, name, val, idx, acInfo):
1214 |         rowIsActive = self.__delegate('Commit', name, val, idx, acInfo)
1215 |         if rowIsActive:
1216 |             for mibNode in self._vars.values():
1217 |                 colNode = mibNode.getNode(mibNode.name + name[len(self.name) + 1:])
1218 |                 if not colNode.syntax.hasValue():
1219 |                     raise error.InconsistentValueError(msg='Row consistency check failed for %r' % colNode)
1220 |
1221 |     def writeCleanup(self, name, val, idx, acInfo):
1222 |         self.branchVersionId += 1
1223 |         self.__delegate('Cleanup', name, val, idx, acInfo)
1224 |
1225 |     def writeUndo(self, name, val, idx, acInfo):
1226 |         self.__delegate('Undo', name, val, idx, acInfo)
1227 |
1228 |     # Table row management
1229 |
1230 |     # Table row access by instance name
1231 |
1232 |     def getInstName(self, colId, instId):
1233 |         return self.name + (colId,) + instId
1234 |

```

```

1235 | # Table index management
1236 |
1237 | def getIndicesFromInstId(self, instId):
1238 |     """Return index values for instance identification"""
1239 |     if instId in self.__idToIdxCache:
1240 |         return self.__idToIdxCache[instId]
1241 |
1242 |     indices = []
1243 |     for impliedFlag, modName, symName in self.indexNames:
1244 |         mibObj, = mibBuilder.importSymbols(modName, symName)
1245 |         try:
1246 |             syntax, instId = self.setFromName(mibObj.syntax, instId, impliedFlag, indices)
1247 |         except PyAsn1Error:
1248 |             debug.logger & debug.flagIns and debug.logger('error resolving table indices at %s, %s: %s' %
1249 |                 indices, instId)
1250 |             instId = ()
1251 |             break
1252 |
1253 |     indices.append(syntax) # to avoid cyclic refs
1254 |
1255 |     if instId:
1256 |         raise error.SmiError(
1257 |             'Excessive instance identifier sub-OIDs left at %s: %s' %
1258 |             (self, instId)
1259 |         )
1260 |
1261 |     indices = tuple(indices)
1262 |     self.__idToIdxCache[instId] = indices
1263 |
1264 |     return indices
1265 |
1266 | def getInstIdFromIndices(self, *indices):
1267 |     """Return column instance identification from indices"""
1268 |     try:
1269 |         return self.__idxToIdCache[indices]
1270 |     except TypeError:
1271 |         cacheable = False
1272 |     except KeyError:
1273 |         cacheable = True
1274 |     idx = 0
1275 |     instId = ()
1276 |     parentIndices = []
1277 |     for impliedFlag, modName, symName in self.indexNames:
1278 |         if idx >= len(indices):
1279 |             break
1280 |         mibObj, = mibBuilder.importSymbols(modName, symName)
1281 |         syntax = mibObj.syntax.clone(indices[idx])
1282 |         instId += self.getAsName(syntax, impliedFlag, parentIndices)
1283 |         parentIndices.append(syntax)
1284 |         idx += 1
1285 |     if cacheable:
1286 |         self.__idxToIdCache[indices] = instId
1287 |     return instId
1288 |
1289 | # Table access by index
1290 |
1291 | def getInstNameByIndex(self, colId, *indices):
1292 |     """Build column instance name from components"""
1293 |     return self.name + (colId,) + self.getInstIdFromIndices(*indices)
1294 |
1295 | def getInstNamesByIndex(self, *indices):
1296 |     """Build column instance names from indices"""
1297 |     instNames = []

```



```

1298 |         for columnName in self._vars.keys():
1299 |             instNames.append(
1300 |                 self.getInstNameByIndex(*(columnName[-1],) + indices)
1301 |             )
1302 |
1303 |         return tuple(instNames)
1304 |
1305 |
1306 | class MibTable(MibTree):
1307 |     """MIB table. Manages a set of TableRow's"""
1308 |
1309 |     def __init__(self, name):
1310 |         MibTree.__init__(self, name)
1311 |
1312 |
1313 | zeroDotZero = ObjectIdentity((0, 0))
1314 |
1315 | # OID tree
1316 | itu_t = MibTree((0,)).setLabel('itu-t')
1317 | iso = MibTree((1,))
1318 | joint_iso_itu_t = MibTree((2,)).setLabel('joint-iso-itu-t')
1319 | org = MibIdentifier(iso.name + (3,))
1320 | dod = MibIdentifier(org.name + (6,))
1321 | internet = MibIdentifier(dod.name + (1,))
1322 | directory = MibIdentifier(internet.name + (1,))
1323 | mgmt = MibIdentifier(internet.name + (2,))
1324 | mib_2 = MibIdentifier(mgmt.name + (1,)).setLabel('mib-2')
1325 | transmission = MibIdentifier(mib_2.name + (10,))
1326 | experimental = MibIdentifier(internet.name + (3,))
1327 | private = MibIdentifier(internet.name + (4,))
1328 | enterprises = MibIdentifier(private.name + (1,))
1329 | security = MibIdentifier(internet.name + (5,))
1330 | snmpV2 = MibIdentifier(internet.name + (6,))
1331 |
1332 | snmpDomains = MibIdentifier(snmpV2.name + (1,))
1333 | snmpProxys = MibIdentifier(snmpV2.name + (2,))
1334 | snmpModules = MibIdentifier(snmpV2.name + (3,))
1335 |
1336 | mibBuilder.exportSymbols(
1337 |     'SNMPv2-SMI', MibNode=MibNode,
1338 |     Integer32=Integer32, Bits=Bits, IPAddress=IPAddress,
1339 |     Counter32=Counter32, Gauge32=Gauge32, Unsigned32=Unsigned32,
1340 |     TimeTicks=TimeTicks, Opaque=Opaque, Counter64=Counter64,
1341 |     ExtUTCTime=ExtUTCTime,
1342 |     ModuleIdentity=ModuleIdentity, ObjectIdentity=ObjectIdentity,
1343 |     NotificationType=NotificationType, MibScalar=MibScalar,
1344 |     MibScalarInstance=MibScalarInstance,
1345 |     MibIdentifier=MibIdentifier, MibTree=MibTree,
1346 |     MibTableColumn=MibTableColumn, MibTableRow=MibTableRow,
1347 |     MibTable=MibTable, zeroDotZero=zeroDotZero,
1348 |     itu_t=itu_t, iso=iso, joint_iso_itu_t=joint_iso_itu_t, org=org, dod=dod,
1349 |     internet=internet, directory=directory, mgmt=mgmt, mib_2=mib_2,
1350 |     transmission=transmission, experimental=experimental, private=private,
1351 |     enterprises=enterprises, security=security, snmpV2=snmpV2,
1352 |     snmpDomains=snmpDomains, snmpProxys=snmpProxys, snmpModules=snmpModules
1353 | )
1354 |
1355 | # XXX
1356 | # getAsName/setFromName goes out of MibRow?
1357 | # revisit getNextNode() -- needs optimization

```

=== ARQUIVO:

.\dist\ControleImpressoras_internal\pysnmp\smi\mibs\SNMPv2-TC.py ===

```
001 | #
002 | # This file is part of pysnmp software.
003 | #
004 | # Copyright (c) 2005-2019, Ilya Etingof <etingof@gmail.com>
005 | # License: http://snmplabs.com/pysnmp/license.html
006 | #
007 | import sys
008 | import inspect
009 | import string
010 | from pysnmp.smi.error import *
011 | from pysnmp import debug
012 | from pyasn1.type import univ
013 | from pyasn1.compat import octets
014 | from pyasn1.type.base import Asn1Item
015 |
016 | OctetString, Integer, ObjectIdentifier = mibBuilder.importSymbols('ASN1', 'OctetString', 'Integer', 'Object
017 | NamedValues, = mibBuilder.importSymbols("ASN1-ENUMERATION", "NamedValues")
018 | ConstraintsIntersection, ConstraintsUnion, SingleValueConstraint, ValueRangeConstraint, ValueSizeConstraint
019 | Counter32, Unsigned32, TimeTicks, Counter64 = mibBuilder.importSymbols('SNMPv2-SMI', 'Counter32', 'Unsigned
020 |
021 |
022 | class TextualConvention(object):
023 |     displayHint = ''
024 |     status = 'current'
025 |     description = ''
026 |     reference = ''
027 |     bits = ()
028 |     __integer = Integer()
029 |     __unsigned32 = Unsigned32()
030 |     __timeticks = TimeTicks()
031 |     __octetString = OctetString()
032 |
033 |     def getDisplayHint(self):
034 |         return self.displayHint
035 |
036 |     def getStatus(self):
037 |         return self.status
038 |
039 |     def getDescription(self):
040 |         return self.description
041 |
042 |     def getReference(self):
043 |         return self.reference
044 |
045 |     def getValue(self):
046 |         return self.clone()
047 |
048 |     def setValue(self, value):
049 |         if value is None:
050 |             value = univ.noValue
051 |         return self.clone(value)
052 |
053 |     def prettyOut(self, value): # override asn1 type method
054 |         """Implements DISPLAY-HINT evaluation"""
055 |         if self.displayHint and (self.__integer.isSuperTypeOf(self, matchConstraints=False) and not self.ge
056 |             self.__unsigned32.isSuperTypeOf(self, matchConstraints=False) or
057 |             self.__timeticks.isSuperTypeOf(self, matchConstraints=False)):
058 |             _ = lambda t, f=0: (t, f)
059 |             displayHintType, decimalPrecision = _(*self.displayHint.split('-'))
```

```

060 |         if displayHintType == 'x':
061 |             return '0x%x' % value
062 |         elif displayHintType == 'd':
063 |             try:
064 |                 return '%.*f' % (int(decimalPrecision), float(value) / pow(10, int(decimalPrecision)))
065 |             except Exception:
066 |                 raise SmiError(
067 |                     'float evaluation error: %s' % sys.exc_info()[1]
068 |                 )
069 |         elif displayHintType == 'o':
070 |             return '0%o' % value
071 |         elif displayHintType == 'b':
072 |             runningValue = value
073 |             outputValue = ['B']
074 |             while runningValue:
075 |                 outputValue.insert(0, '%d' % (runningValue & 0x01))
076 |                 runningValue >>= 1
077 |             return ''.join(outputValue)
078 |         else:
079 |             raise SmiError(
080 |                 'Unsupported numeric type spec "%s" at %s' % (displayHintType, self.__class__.__name__)
081 |             )
082 |     elif self.displayHint and self.__octetString.isSuperTypeOf(self, matchConstraints=False):
083 |         outputValue = ''
084 |         runningValue = OctetString(value).asOctets()
085 |         displayHint = self.displayHint
086 |         while runningValue and displayHint:
087 |             # 1
088 |             if displayHint[0] == '*':
089 |                 repeatIndicator = repeatCount = octets.oct2int(runningValue[0])
090 |                 displayHint = displayHint[1:]
091 |                 runningValue = runningValue[1:]
092 |             else:
093 |                 repeatCount = 1
094 |                 repeatIndicator = None
095 |
096 |             # 2
097 |             octetLength = ''
098 |             while displayHint and displayHint[0] in string.digits:
099 |                 octetLength += displayHint[0]
100 |                 displayHint = displayHint[1:]
101 |
102 |             # length is manatory, but people ignore that
103 |             if not octetLength:
104 |                 octetLength = len(runningValue)
105 |
106 |             try:
107 |                 octetLength = int(octetLength)
108 |             except Exception:
109 |                 raise SmiError(
110 |                     'Bad octet length: %s' % octetLength
111 |                 )
112 |
113 |             if not displayHint:
114 |                 raise SmiError(
115 |                     'Short octet length: %s' % self.displayHint
116 |                 )
117 |
118 |             # 3
119 |             displayFormat = displayHint[0]
120 |             displayHint = displayHint[1:]
121 |
122 |             # 4

```

```

123 |         if displayHint and displayHint[0] not in string.digits and displayHint[0] != '*':
124 |             displaySep = displayHint[0]
125 |             displayHint = displayHint[1:]
126 |         else:
127 |             displaySep = ''
128 |
129 |         # 5
130 |         if displayHint and displaySep and repeatIndicator is not None:
131 |             repeatTerminator = displayHint[0]
132 |             displaySep = ''
133 |             displayHint = displayHint[1:]
134 |         else:
135 |             repeatTerminator = None
136 |
137 |         while repeatCount:
138 |             repeatCount -= 1
139 |             if displayFormat == 'a':
140 |                 outputValue += runningValue[:octetLength].decode('ascii', 'ignore')
141 |             elif displayFormat == 't':
142 |                 outputValue += runningValue[:octetLength].decode('utf-8', 'ignore')
143 |             elif displayFormat in ('x', 'd', 'o'):
144 |                 number = 0
145 |                 numberString = runningValue[:octetLength]
146 |                 while numberString:
147 |                     number <= 8
148 |                     try:
149 |                         number |= octets.oct2int(numberString[0])
150 |                         numberString = numberString[1:]
151 |                     except Exception:
152 |                         raise SmiError(
153 |                             'Display format eval failure: %s: %s'
154 |                             % (numberString, sys.exc_info()[1])
155 |                         )
156 |                 if displayFormat == 'x':
157 |                     outputValue += '%02x' % number
158 |                 elif displayFormat == 'o':
159 |                     outputValue += '%03o' % number
160 |                 else:
161 |                     outputValue += '%d' % number
162 |             else:
163 |                 raise SmiError(
164 |                     'Unsupported display format char: %s' %
165 |                     displayFormat
166 |                 )
167 |             if runningValue and repeatTerminator:
168 |                 outputValue += repeatTerminator
169 |             runningValue = runningValue[octetLength:]
170 |         if runningValue and displaySep:
171 |             outputValue += displaySep
172 |         if not displayHint:
173 |             displayHint = self.displayHint
174 |
175 |         return outputValue
176 |
177 |         for base in inspect.getmro(self.__class__):
178 |             if not issubclass(base, TextualConvention) and issubclass(base, Asn1Item):
179 |                 return base.prettyOut(self, value)
180 |
181 |         raise SmiError('TEXTUAL-CONVENTION has no underlying SNMP base type')
182 |
183 |     def prettyIn(self, value): # override asn1 type method
184 |         """Implements DISPLAY-HINT parsing into base SNMP value
185 |

```

```

186 | Proper parsing seems impossible due to ambiguities.
187 | Here we are trying to do our best, but be prepared
188 | for failures on complicated DISPLAY-HINTs.
189 |
190 | Keep in mind that this parser only works with "text"
191 | input meaning `unicode` (Py2) or `str` (Py3).
192 | """
193 | for base in inspect.getmro(self.__class__):
194 |     if not issubclass(base, TextualConvention) and issubclass(base, Asn1Item):
195 |         break
196 | else:
197 |     raise SmiError('TEXTUAL-CONVENTION has no underlying SNMP base type')
198 |
199 | if self.displayHint and (self.__integer.isSuperTypeOf(self, matchConstraints=False) and
200 |                         self.getNamedValues() or
201 |                         self.__unsigned32.isSuperTypeOf(self, matchConstraints=False) or
202 |                         self.__timeticks.isSuperTypeOf(self, matchConstraints=False)):
203 |     value = str(value)
204 |
205 |     _ = lambda t, f=0: (t, f)
206 |     displayHintType, decimalPrecision = _(*self.displayHint.split('-'))
207 |     if displayHintType == 'x' and (value.startswith('0x') or value.startswith('-0x')):
208 |         try:
209 |             if value.startswith('-'):
210 |                 return base.prettyIn(self, -int(value[3:], 16))
211 |             else:
212 |                 return base.prettyIn(self, int(value[2:], 16))
213 |         except Exception:
214 |             raise SmiError(
215 |                 'integer evaluation error: %s' % sys.exc_info()[1]
216 |             )
217 |     elif displayHintType == 'd':
218 |         try:
219 |             return base.prettyIn(self, int(float(value) * 10**int(decimalPrecision)))
220 |         except Exception:
221 |             raise SmiError(
222 |                 'float evaluation error: %s' % sys.exc_info()[1]
223 |             )
224 |     elif displayHintType == 'o' and (value.startswith('0') or value.startswith('-0')):
225 |         try:
226 |             return base.prettyIn(self, int(value, 8))
227 |         except Exception:
228 |             raise SmiError(
229 |                 'octal evaluation error: %s' % sys.exc_info()[1]
230 |             )
231 |     elif displayHintType == 'b' and (value.startswith('B') or value.startswith('-B')):
232 |         negative = value.startswith('-')
233 |         if negative:
234 |             value = value[2:]
235 |         else:
236 |             value = value[1:]
237 |         value = [x != '0' and 1 or 0 for x in value]
238 |         binValue = 0
239 |         while value:
240 |             binValue <<= value[0]
241 |             del value[0]
242 |         return base.prettyIn(self, binValue)
243 |     else:
244 |         raise SmiError(
245 |             'Unsupported numeric type spec "%s" at %s' % (displayHintType, self.__class__.__name__)
246 |         )
247 |
248 | elif self.displayHint and self.__octetString.isSuperTypeOf(self, matchConstraints=False):

```

```

249 |         numBase = {
250 |             'x': 16,
251 |             'd': 10,
252 |             'o': 8
253 |         }
254 |         numDigits = {
255 |             'x': octets.str2octets(string.hexdigits),
256 |             'o': octets.str2octets(string.octdigits),
257 |             'd': octets.str2octets(string.digits)
258 |         }
259 |
260 |         # how do we know if object is initialized with display-hint
261 |         # formatted text? based on "text" input maybe?
262 |         # That boils down to `str` object on Py3 or `unicode` on Py2.
263 |         if octets.isStringType(value) and not octets.isOctetsType(value):
264 |             value = base.prettyIn(self, value)
265 |         else:
266 |             return base.prettyIn(self, value)
267 |
268 |         outputValue = octets.str2octets('')
269 |         runningValue = value
270 |         displayHint = self.displayHint
271 |
272 |         while runningValue and displayHint:
273 |             # 1 this information is totally lost, just fail explicitly
274 |             if displayHint[0] == '*':
275 |                 raise SmiError(
276 |                     'Can\'t parse "*" in DISPLAY-HINT (%s)' % self.__class__.__name__
277 |                 )
278 |
279 |             # 2 this becomes ambiguous when it comes to rendered value
280 |             octetLength = ''
281 |             while displayHint and displayHint[0] in string.digits:
282 |                 octetLength += displayHint[0]
283 |                 displayHint = displayHint[1:]
284 |
285 |             # length is mandatory but people ignore that
286 |             if not octetLength:
287 |                 octetLength = len(runningValue)
288 |
289 |             try:
290 |                 octetLength = int(octetLength)
291 |             except Exception:
292 |                 raise SmiError(
293 |                     'Bad octet length: %s' % octetLength
294 |                 )
295 |
296 |             if not displayHint:
297 |                 raise SmiError(
298 |                     'Short octet length: %s' % self.displayHint
299 |                 )
300 |
301 |             # 3
302 |             displayFormat = displayHint[0]
303 |             displayHint = displayHint[1:]
304 |
305 |             # 4 this is the lifesaver -- we could use it as an anchor
306 |             if displayHint and displayHint[0] not in string.digits and displayHint[0] != '*':
307 |                 displaySep = displayHint[0]
308 |                 displayHint = displayHint[1:]
309 |             else:
310 |                 displaySep = ''
311 |

```

```

312 |         # 5 is probably impossible to support
313 |
314 |         if displayFormat in ('a', 't'):
315 |             outputValue += runningValue[:octetLength]
316 |         elif displayFormat in numBase:
317 |             if displaySep:
318 |                 guessedOctetLength = runningValue.find(octets.str2octs(displaySep))
319 |                 if guessedOctetLength == -1:
320 |                     guessedOctetLength = len(runningValue)
321 |             else:
322 |                 for idx in range(len(runningValue)):
323 |                     if runningValue[idx] not in numDigits[displayFormat]:
324 |                         guessedOctetLength = idx
325 |                         break
326 |             else:
327 |                 guessedOctetLength = len(runningValue)
328 |
329 |             try:
330 |                 num = int(octets.octs2str(runningValue[:guessedOctetLength]), numBase[displayFormat])
331 |             except Exception:
332 |                 raise SmiError(
333 |                     'Display format eval failure: %s: %s'
334 |                     % (runningValue[:guessedOctetLength], sys.exc_info()[1])
335 |                 )
336 |
337 |             num_as_bytes = []
338 |             if num:
339 |                 while num:
340 |                     num_as_bytes.append(num & 0xFF)
341 |                     num >>= 8
342 |             else:
343 |                 num_as_bytes = [0]
344 |
345 |             while len(num_as_bytes) < octetLength:
346 |                 num_as_bytes.append(0)
347 |
348 |             num_as_bytes.reverse()
349 |
350 |             outputValue += octets.ints2octs(num_as_bytes)
351 |
352 |             if displaySep:
353 |                 guessedOctetLength += 1
354 |
355 |             octetLength = guessedOctetLength
356 |         else:
357 |             raise SmiError(
358 |                 'Unsupported display format char: %s' %
359 |                 displayFormat
360 |             )
361 |
362 |         runningValue = runningValue[octetLength:]
363 |
364 |         if not displayHint:
365 |             displayHint = self.displayHint
366 |
367 |         return base.prettyIn(self, outputValue)
368 |
369 |     else:
370 |         return base.prettyIn(self, value)
371 |
372 |
373 | class DisplayString(TextualConvention, OctetString):
374 |     description = "Represents textual information taken from the NVT ASCII character set, as defined in pag

```

```

375 |     status = 'current'
376 |     subtypeSpec = OctetString.subtypeSpec + ValueSizeConstraint(0, 255)
377 |     displayHint = "255a"
378 |
379 |
380 | class PhysAddress(TextualConvention, OctetString):
381 |     description = "Represents textual information taken from the NVT ASCII character set, as defined in pag
382 |     status = 'current'
383 |     displayHint = "1x:"
384 |
385 |
386 | class MacAddress(TextualConvention, OctetString):
387 |     description = "Represents an 802 MAC address represented in the `canonical' order defined by IEEE 802.1
388 |     status = 'current'
389 |     subtypeSpec = OctetString.subtypeSpec + ValueSizeConstraint(6, 6)
390 |     displayHint = "1x:"
391 |     fixedLength = 6
392 |
393 |
394 | class TruthValue(TextualConvention, Integer):
395 |     description = 'Represents a boolean value.'
396 |     status = 'current'
397 |     subtypeSpec = Integer.subtypeSpec + SingleValueConstraint(1, 2)
398 |     namedValues = NamedValues(('true', 1), ('false', 2))
399 |
400 |
401 | class TestAndIncr(TextualConvention, Integer):
402 |     description = "Represents integer-valued information used for atomic operations. When the management pr
403 |     status = 'current'
404 |     subtypeSpec = Integer.subtypeSpec + ValueRangeConstraint(0, 2147483647)
405 |     defaultValue = 0
406 |
407 |     def setValue(self, value):
408 |         if value is not None:
409 |             if value != self:
410 |                 raise InconsistentValueError()
411 |             value += 1
412 |             if value > 2147483646:
413 |                 value = 0
414 |         if value is None:
415 |             value = univ.noValue
416 |         return self.clone(value)
417 |
418 |
419 | class AutonomousType(TextualConvention, ObjectIdentifier):
420 |     description = 'Represents an independently extensible type identification value. It may, for example, i
421 |     status = 'current'
422 |
423 |
424 | class InstancePointer(TextualConvention, ObjectIdentifier):
425 |     description = 'A pointer to either a specific instance of a MIB object or a conceptual row of a MIB tab
426 |     status = 'obsolete'
427 |
428 |
429 | class VariablePointer(TextualConvention, ObjectIdentifier):
430 |     description = 'A pointer to a specific object instance. For example, sysContact.0 or ifInOctets.3.'
431 |     status = 'current'
432 |
433 |
434 | class RowPointer(TextualConvention, ObjectIdentifier):
435 |     description = 'Represents a pointer to a conceptual row. The value is the name of the instance of the f
436 |     status = 'current'
437 |

```



```

438 | class RowStatus(TextualConvention, Integer):
439 |     """A special kind of scalar MIB variable responsible for
440 |         MIB table row creation/destruction.
441 |     """
442 |     description = "The RowStatus textual convention is used to manage the creation and deletion of concepts"
443 |     status = 'current'
444 |     subtypeSpec = Integer.subtypeSpec + SingleValueConstraint(0, 1, 2, 3, 4, 5, 6)
445 |     namedValues = NamedValues(
446 |         ('notExists', 0), ('active', 1), ('notInService', 2), ('notReady', 3),
447 |         ('createAndGo', 4), ('createAndWait', 5), ('destroy', 6)
448 |     )
449 |     # Known row states
450 |     (stNotExists, stActive, stNotInService,
451 |      stNotReady, stCreateAndGo, stCreateAndWait,
452 |      stDestroy) = list(range(7))
453 |
454 |     # States transition matrix (see RFC-1903)
455 |     stateMatrix = {
456 |         # (new-state, current-state) -> (error, new-state)
457 |         (stCreateAndGo, stNotExists): (RowCreationWanted, stActive),
458 |         (stCreateAndGo, stNotReady): (InconsistentValueError, stNotReady),
459 |         (stCreateAndGo, stNotInService): (InconsistentValueError, stNotInService),
460 |         (stCreateAndGo, stActive): (InconsistentValueError, stActive),
461 |         #
462 |         (stCreateAndWait, stNotExists): (RowCreationWanted, stActive),
463 |         (stCreateAndWait, stNotReady): (InconsistentValueError, stNotReady),
464 |         (stCreateAndWait, stNotInService): (InconsistentValueError, stNotInService),
465 |         (stCreateAndWait, stActive): (InconsistentValueError, stActive),
466 |         #
467 |         (stActive, stNotExists): (InconsistentValueError, stNotExists),
468 |         (stActive, stNotReady): (InconsistentValueError, stNotReady),
469 |         (stActive, stNotInService): (None, stActive),
470 |         (stActive, stActive): (None, stActive),
471 |         #
472 |         (stNotInService, stNotExists): (InconsistentValueError, stNotExists),
473 |         (stNotInService, stNotReady): (InconsistentValueError, stNotReady),
474 |         (stNotInService, stNotInService): (None, stNotInService),
475 |         (stNotInService, stActive): (None, stActive),
476 |         #
477 |         (stDestroy, stNotExists): (RowDestructionWanted, stNotExists),
478 |         (stDestroy, stNotReady): (RowDestructionWanted, stNotExists),
479 |         (stDestroy, stNotInService): (RowDestructionWanted, stNotExists),
480 |         (stDestroy, stActive): (RowDestructionWanted, stNotExists),
481 |         # This is used on instantiation
482 |         (stNotExists, stNotExists): (None, stNotExists)
483 |     }
484 |
485 |     def setValue(self, value):
486 |         if value is None:
487 |             value = univ.noValue
488 |
489 |         value = self.clone(value)
490 |
491 |         # Run through states transition matrix,
492 |         # resolve new instance value
493 |         excValue, newState = self.stateMatrix.get(
494 |             (value.hasValue() and value or self.stNotExists,
495 |              self.hasValue() and self or self.stNotExists),
496 |             (MibOperationError, None)
497 |         )
498 |
499 |         if newState is None:
500 |             newState = univ.noValue

```

```

501 |
502 |         newState = self.clone(newState)
503 |
504 |         debug.logger & debug.flagIns and debug.logger(
505 |             'RowStatus state change from %r to %r produced new state %r, error indication %r' % (
506 |                 self, value, newState, excValue))
507 |
508 |         if excValue is not None:
509 |             excValue = excValue(
510 |                 msg='Exception at row state transition from %r to %r yields state %r and exception' % (
511 |                     self, value, newState), syntax=newState
512 |             )
513 |             raise excValue
514 |
515 |         return newState
516 |
517 |
518 | class TimeStamp(TextualConvention, TimeTicks):
519 |     description = 'The value of the sysUpTime object at which a specific occurrence happened. The specific
520 |     status = 'current'
521 |
522 |
523 | class TimeInterval(TextualConvention, Integer):
524 |     description = 'A period of time, measured in units of 0.01 seconds.'
525 |     status = 'current'
526 |     subtypeSpec = Integer.subtypeSpec + ValueRangeConstraint(0, 2147483647)
527 |
528 |
529 | class DateAndTime(TextualConvention, OctetString):
530 |     description = "A date-time specification. field octets contents range ----- 1 1-2
531 |     status = 'current'
532 |     subtypeSpec = OctetString.subtypeSpec + ValueSizeConstraint(8, 11)
533 |     displayHint = "2d-1d-1d,1d:1d:1d,1a1d:1d"
534 |
535 |
536 | class StorageType(TextualConvention, Integer):
537 |     description = "Describes the memory realization of a conceptual row. A row which is volatile(2) is lost
538 |     status = 'current'
539 |     subtypeSpec = Integer.subtypeSpec + SingleValueConstraint(1, 2, 3, 4, 5)
540 |     namedValues = NamedValues(
541 |         ('other', 1), ('volatile', 2), ('nonVolatile', 3),
542 |         ('permanent', 4), ('readOnly', 5)
543 |     )
544 |
545 |
546 | class TDomain(TextualConvention, ObjectIdentifier):
547 |     reference = 'The SNMPv2-TM MIB module is defined in RFC 1906.'
548 |     description = 'Denotes a kind of transport service. Some possible values, such as snmpUDPDomain, are de
549 |     status = 'current'
550 |
551 |
552 | class TAddress(TextualConvention, OctetString):
553 |     reference = 'The SNMPv2-TM MIB module is defined in RFC 1906.'
554 |     description = 'Denotes a transport service address. A TAddress value is always interpreted within the c
555 |     status = 'current'
556 |     subtypeSpec = OctetString.subtypeSpec + ValueSizeConstraint(1, 255)
557 |
558 |
559 | mibBuilder.exportSymbols(
560 |     'SNMPv2-TC', TextualConvention=TextualConvention,
561 |     DisplayString=DisplayString, PhysAddress=PhysAddress,
562 |     MacAddress=MacAddress, TruthValue=TruthValue, TestAndIncr=TestAndIncr,
563 |     AutonomousType=AutonomousType, InstancePointer=InstancePointer,

```

```

564 |     VariablePointer=VariablePointer, RowPointer=RowPointer,
565 |     RowStatus=RowStatus, TimeStamp=TimeStamp, TimeInterval=TimeInterval,
566 |     DateAndTime=DateAndTime, StorageType=StorageType, TDomain=TDomain,
567 |     TAddress=TAddress
568 | )

```

=== ARQUIVO:

.\dist\ControleImpressoras_internal\pysnmp\smi\mibs\SNMPv2-TM.py ===

```

001 | #
002 | # This file is part of pysnmp software.
003 | #
004 | # Copyright (c) 2005-2019, Ilya Etingof <etingof@gmail.com>
005 | # License: http://snmplabs.com/pysnmp/license.html
006 | #
007 | # PySNMP MIB module SNMPv2-TM (http://snmplabs.com/pysnmp)
008 | # ASN.1 source http://mibs.snmplabs.com:80/asn1/SNMPv2-TM
009 | # Produced by pysmi-0.1.3 at Tue Apr 18 01:01:19 2017
010 | # On host grommit.local platform Darwin version 16.4.0 by user ilya
011 | # Using Python version 3.4.2 (v3.4.2:ab2c023a9432, Oct 5 2014, 20:42:22)
012 | #
013 | try:
014 |     from socket import inet_ntop, inet_pton, AF_INET
015 | except ImportError:
016 |     from socket import inet_ntoa, inet_aton, AF_INET
017 |
018 |     inet_ntop = lambda x, y: inet_ntoa(y)
019 |     inet_pton = lambda x, y: inet_aton(y)
020 |
021 | from pyasn1.compat.octets import int2oct, oct2int
022 |
023 | OctetString, = mibBuilder.importSymbols('ASN1', 'OctetString')
024 | ConstraintsIntersection, ConstraintsUnion, SingleValueConstraint, ValueRangeConstraint, ValueSizeConstraint
025 | ModuleIdentity, MibIdentifier, ObjectIdentity, snmpModules, snmpDomains, snmpProxys = mibBuilder.importSymbols(
026 | TextualConvention, = mibBuilder.importSymbols('SNMPv2-TC', 'TextualConvention')
027 |
028 | snmpv2tm = ModuleIdentity((1, 3, 6, 1, 6, 3, 19))
029 | if mibBuilder.loadTexts: snmpv2tm.setRevisions(('2000-08-09 19:58', '1996-01-01 00:00', '1993-04-01 00:00',
030 | if mibBuilder.loadTexts: snmpv2tm.setLastUpdated('200008091958Z')
031 | if mibBuilder.loadTexts: snmpv2tm.setOrganization('IETF SNMPv3 Working Group')
032 | if mibBuilder.loadTexts: snmpv2tm.setContactInfo('WG-EMail: snmpv3@tis.com Subscribe: majordomo@tis.com In
033 | if mibBuilder.loadTexts: snmpv2tm.setDescription('The MIB module for SNMP transport mappings.')
034 | snmpUDPDomain = ObjectIdentity((1, 3, 6, 1, 6, 1, 1))
035 | if mibBuilder.loadTexts: snmpUDPDomain.setStatus('current')
036 | if mibBuilder.loadTexts: snmpUDPDomain.setDescription('The SNMP over UDP over IPv4 transport domain. The co
037 |
038 |
039 | class SnmpUDPAddress(TextualConvention, OctetString):
040 |     description = 'Represents a UDP over IPv4 address: octets contents encoding 1-4 IP-address network-byte
041 |     status = 'current'
042 |     subtypeSpec = OctetString.subtypeSpec + ValueSizeConstraint(6, 6)
043 |     displayHint = "1d.1d.1d.1d/2d"
044 |     fixedLength = 6
045 |
046 |     def prettyIn(self, value):
047 |         if isinstance(value, tuple):
048 |             # Wild hack -- need to implement TextualConvention.prettyIn
049 |             value = inet_pton(AF_INET, value[0]) + int2oct((value[1] >> 8) & 0xff) + int2oct(value[1] & 0xf
050 |             return OctetString.prettyIn(self, value)
051 |

```

```

052 | # Socket address syntax coercion
053 | def __asSocketAddress(self):
054 |     if not hasattr(self, '__tuple_value'):
055 |         v = self.asOctets()
056 |         self.__tuple_value = (
057 |             inet_ntop(AF_INET, v[:4]),
058 |             oct2int(v[4]) << 8 | oct2int(v[5])
059 |         )
060 |     return self.__tuple_value
061 |
062 | def __iter__(self):
063 |     return iter(self.__asSocketAddress())
064 |
065 | def __getitem__(self, item):
066 |     return self.__asSocketAddress()[item]
067 |
068 |
069 | snmpCLNSDomain = ObjectIdentity((1, 3, 6, 1, 6, 1, 2))
070 | if mibBuilder.loadTexts: snmpCLNSDomain.setStatus('current')
071 | if mibBuilder.loadTexts: snmpCLNSDomain.setDescription('The SNMP over CLNS transport domain. The correspond
072 | snmpCONSDomain = ObjectIdentity((1, 3, 6, 1, 6, 1, 3))
073 | if mibBuilder.loadTexts: snmpCONSDomain.setStatus('current')
074 | if mibBuilder.loadTexts: snmpCONSDomain.setDescription('The SNMP over CONS transport domain. The correspond
075 |
076 |
077 | class SnmpOSIAddress(TextualConvention, OctetString):
078 |     description = "Represents an OSI transport-address: octets contents encoding 1 length of NSAP 'n' as an
079 |     status = 'current'
080 |     subtypeSpec = OctetString.subtypeSpec + ValueSizeConstraint(1, 85)
081 |     displayHint = "*1x:/1x:"
082 |
083 | snmpDDPDomain = ObjectIdentity((1, 3, 6, 1, 6, 1, 4))
084 | if mibBuilder.loadTexts: snmpDDPDomain.setStatus('current')
085 | if mibBuilder.loadTexts: snmpDDPDomain.setDescription('The SNMP over DDP transport domain. The correspondin
086 |
087 |
088 | class SnmpNBPAddress(OctetString, TextualConvention):
089 |     description = "Represents an NBP name: octets contents encoding 1 length of object 'n' as an unsigned i
090 |     status = 'current'
091 |     subtypeSpec = OctetString.subtypeSpec + ValueSizeConstraint(3, 99)
092 |
093 | snmpIPXDomain = ObjectIdentity((1, 3, 6, 1, 6, 1, 5))
094 | if mibBuilder.loadTexts: snmpIPXDomain.setStatus('current')
095 | if mibBuilder.loadTexts: snmpIPXDomain.setDescription('The SNMP over IPX transport domain. The correspondin
096 | class SnmpIPXAddress(TextualConvention, OctetString):
097 |     description = 'Represents an IPX address: octets contents encoding 1-4 network-number network-byte orde
098 |     status = 'current'
099 |     subtypeSpec = OctetString.subtypeSpec + ValueSizeConstraint(12, 12)
100 |     displayHint = "4x.1x:1x:1x:1x:1x:1x:1x.2d"
101 |     fixedLength = 12
102 |
103 | rfc1157Proxy = MibIdentifier((1, 3, 6, 1, 6, 2, 1))
104 | rfc1157Domain = ObjectIdentity((1, 3, 6, 1, 6, 2, 1, 1))
105 | if mibBuilder.loadTexts: rfc1157Domain.setStatus('deprecated')
106 | if mibBuilder.loadTexts: rfc1157Domain.setDescription('The transport domain for SNMPv1 over UDP over IPv4.
107 |
108 | mibBuilder.exportSymbols("SNMPv2-TM", SnmpNBPAddress=SnmpNBPAddress, rfc1157Domain=rfc1157Domain, SnmpIPXAd

```

=== ARQUIVO: .\dist\ControleImpressoras_internal\pysnmp\smi\mibs\TRANSPORT-ADDRESS-MIB.py ===

```

001 | #
002 | # This file is part of pysnmp software.
003 | #
004 | # Copyright (c) 2005-2019, Ilya Etingof <etingof@gmail.com>
005 | # License: http://snmplabs.com/pysnmp/license.html
006 | #
007 | # PySNMP MIB module TRANSPORT-ADDRESS-MIB (http://snmplabs.com/pysnmp)
008 | # ASN.1 source http://mibs.snmplabs.com:80/asn1/TRANSPORT-ADDRESS-MIB
009 | # Produced by pysmi-0.1.3 at Tue Apr 18 01:08:18 2017
010 | # On host grommit.local platform Darwin version 16.4.0 by user ilya
011 | # Using Python version 3.4.2 (v3.4.2:ab2c023a9432, Oct 5 2014, 20:42:22)
012 | #
013 |
014 | from pyasn1.compat.octets import int2oct, oct2int
015 | from pysnmp import error
016 | from pysnmp.carrier import sockfix
017 | import socket
018 |
019 | has_ipv6 = socket.has_ipv6
020 |
021 | if hasattr(socket, 'inet_ntop') and hasattr(socket, 'inet_pton'):
022 |     inet_ntop = socket.inet_ntop
023 |     inet_pton = socket.inet_pton
024 | else:
025 |     import sys
026 |
027 |     if sys.platform != "win32":
028 |         from socket import inet_ntoa, inet_aton
029 |
030 |         inet_ntop = lambda x, y: inet_ntoa(y)
031 |         inet_pton = lambda x, y: inet_aton(y)
032 |         has_ipv6 = False
033 |     elif has_ipv6:
034 |         import struct # The case of old Python at old Windows
035 |
036 |
037 |     def inet_pton(address_family, ip_string):
038 |         if address_family == socket.AF_INET:
039 |             return socket.inet_aton(ip_string)
040 |         elif address_family != socket.AF_INET6:
041 |             raise socket.error(
042 |                 'Unknown address family %s' % (address_family,)
043 |             )
044 |
045 |         groups = ip_string.split(":")
046 |         spaces = groups.count('')
047 |
048 |         if '.' in groups[-1]:
049 |             groups[-1:] = ["%x" % x for x in struct.unpack("!HH", socket.inet_aton(groups[-1]))]
050 |
051 |         if spaces == 1:
052 |             idx = groups.index('')
053 |             groups[idx:idx + 1] = ['0'] * (8 - len(groups) + 1)
054 |         elif spaces == 2:
055 |             zeros = ['0'] * (8 - len(groups) + 2)
056 |             if ip_string.startswith('::'):
057 |                 groups[:2] = zeros
058 |             elif ip_string.endswith('::'):
059 |                 groups[-2:] = zeros
060 |             else:
061 |                 raise socket.error(
062 |                     'Invalid IPv6 address: "%s"' % (ip_string,)
063 |                 )

```

```

064 |         elif spaces == 3:
065 |             if ip_string != '::':
066 |                 raise socket.error(
067 |                     'Invalid IPv6 address: "%s"' % (ip_string,)
068 |                 )
069 |             return '\x00' * 16
070 |         elif spaces > 3:
071 |             raise socket.error(
072 |                 'Invalid IPv6 address: "%s"' % (ip_string,)
073 |             )
074 |
075 |         groups = [t for t in [int(t, 16) for t in groups] if t & 0xFFFF == t]
076 |
077 |         if len(groups) != 8:
078 |             raise socket.error(
079 |                 'Invalid IPv6 address: "%s"' % (ip_string,)
080 |             )
081 |
082 |         return struct.pack('!8H', *groups)
083 |
084 |
085 |     def inet_ntop(address_family, packed_ip):
086 |         if address_family == socket.AF_INET:
087 |             return socket.inet_ntop(packed_ip)
088 |         elif address_family != socket.AF_INET6:
089 |             raise socket.error(
090 |                 'Unknown address family %s' % (address_family,)
091 |             )
092 |
093 |         if len(packed_ip) != 16:
094 |             raise socket.error(
095 |                 'incorrect address length: %s' % len(packed_ip)
096 |             )
097 |
098 |         groups = list(struct.unpack('!8H', packed_ip))
099 |
100 |         cur_base = best_base = cur_len = best_len = -1
101 |
102 |         for idx in range(8):
103 |             if groups[idx]:
104 |                 if cur_base != -1:
105 |                     if best_base == -1 or cur_len > best_len:
106 |                         best_base, best_len = cur_base, cur_len
107 |                     cur_base = -1
108 |                 else:
109 |                     if cur_base == -1:
110 |                         cur_base, cur_len = idx, 1
111 |                     else:
112 |                         cur_len += 1
113 |
114 |         if cur_base != -1:
115 |             if best_base == -1 or cur_len > best_len:
116 |                 best_base, best_len = cur_base, cur_len
117 |
118 |         if best_base != -1 and best_len > 1:
119 |             groups[best_base:best_base + best_len] = [':']
120 |
121 |         if groups[0] == ':':
122 |             groups.insert(0, ':')
123 |         if groups[-1] == ':':
124 |             groups.append(':')
125 |
126 |         f = lambda x: x != ':' and '%x' % x or ''

```

```

127 |
128 |         return ':'.join([f(x) for x in groups])
129 |
130 | Integer, OctetString, ObjectIdentifier = mibBuilder.importSymbols("ASN1", "Integer", "OctetString", "Object
131 | NamedValues, = mibBuilder.importSymbols("ASN1-ENUMERATION", "NamedValues")
132 | ConstraintsIntersection, ConstraintsUnion, ValueRangeConstraint, ValueSizeConstraint, SingleValueConstraint
133 | NotificationGroup, ModuleCompliance = mibBuilder.importSymbols("SNMPv2-CONF", "NotificationGroup", "ModuleC
134 | Integer32, Unsigned32, MibIdentifier, Counter32, ModuleIdentity, MibScalar, MibTable, MibTableRow, MibTable
135 | DisplayString, TextualConvention = mibBuilder.importSymbols("SNMPv2-TC", "DisplayString", "TextualConventio
136 | transportAddressMIB = ModuleIdentity((1, 3, 6, 1, 2, 1, 100))
137 | if mibBuilder.loadTexts: transportAddressMIB.setRevisions(('2002-11-01 00:00',))
138 | if mibBuilder.loadTexts: transportAddressMIB.setLastUpdated('200211010000Z')
139 | if mibBuilder.loadTexts: transportAddressMIB.setOrganization('IETF Operations and Management Area')
140 | if mibBuilder.loadTexts: transportAddressMIB.setContactInfo('Juergen Schoenwaelder (Editor) TU Braunschweig
141 | if mibBuilder.loadTexts: transportAddressMIB.setDescription('This MIB module provides commonly used transpo
142 | transportDomains = MibIdentifier((1, 3, 6, 1, 2, 1, 100, 1))
143 | transportDomainUdpIpv4 = ObjectIdentity((1, 3, 6, 1, 2, 1, 100, 1, 1))
144 | if mibBuilder.loadTexts: transportDomainUdpIpv4.setStatus('current')
145 | if mibBuilder.loadTexts: transportDomainUdpIpv4.setDescription('The UDP over IPv4 transport domain. The cor
146 | transportDomainUdpIpv6 = ObjectIdentity((1, 3, 6, 1, 2, 1, 100, 1, 2))
147 | if mibBuilder.loadTexts: transportDomainUdpIpv6.setStatus('current')
148 | if mibBuilder.loadTexts: transportDomainUdpIpv6.setDescription('The UDP over IPv6 transport domain. The cor
149 | transportDomainUdpIpv4z = ObjectIdentity((1, 3, 6, 1, 2, 1, 100, 1, 3))
150 | if mibBuilder.loadTexts: transportDomainUdpIpv4z.setStatus('current')
151 | if mibBuilder.loadTexts: transportDomainUdpIpv4z.setDescription('The UDP over IPv4 transport domain. The co
152 | transportDomainUdpIpv6z = ObjectIdentity((1, 3, 6, 1, 2, 1, 100, 1, 4))
153 | if mibBuilder.loadTexts: transportDomainUdpIpv6z.setStatus('current')
154 | if mibBuilder.loadTexts: transportDomainUdpIpv6z.setDescription('The UDP over IPv6 transport domain. The co
155 | transportDomainTcpIpv4 = ObjectIdentity((1, 3, 6, 1, 2, 1, 100, 1, 5))
156 | if mibBuilder.loadTexts: transportDomainTcpIpv4.setStatus('current')
157 | if mibBuilder.loadTexts: transportDomainTcpIpv4.setDescription('The TCP over IPv4 transport domain. The cor
158 | transportDomainTcpIpv6 = ObjectIdentity((1, 3, 6, 1, 2, 1, 100, 1, 6))
159 | if mibBuilder.loadTexts: transportDomainTcpIpv6.setStatus('current')
160 | if mibBuilder.loadTexts: transportDomainTcpIpv6.setDescription('The TCP over IPv6 transport domain. The cor
161 | transportDomainTcpIpv4z = ObjectIdentity((1, 3, 6, 1, 2, 1, 100, 1, 7))
162 | if mibBuilder.loadTexts: transportDomainTcpIpv4z.setStatus('current')
163 | if mibBuilder.loadTexts: transportDomainTcpIpv4z.setDescription('The TCP over IPv4 transport domain. The co
164 | transportDomainTcpIpv6z = ObjectIdentity((1, 3, 6, 1, 2, 1, 100, 1, 8))
165 | if mibBuilder.loadTexts: transportDomainTcpIpv6z.setStatus('current')
166 | if mibBuilder.loadTexts: transportDomainTcpIpv6z.setDescription('The TCP over IPv6 transport domain. The co
167 | transportDomainSctpIpv4 = ObjectIdentity((1, 3, 6, 1, 2, 1, 100, 1, 9))
168 | if mibBuilder.loadTexts: transportDomainSctpIpv4.setStatus('current')
169 | if mibBuilder.loadTexts: transportDomainSctpIpv4.setDescription('The SCTP over IPv4 transport domain. The c
170 | transportDomainSctpIpv6 = ObjectIdentity((1, 3, 6, 1, 2, 1, 100, 1, 10))
171 | if mibBuilder.loadTexts: transportDomainSctpIpv6.setStatus('current')
172 | if mibBuilder.loadTexts: transportDomainSctpIpv6.setDescription('The SCTP over IPv6 transport domain. The c
173 | transportDomainSctpIpv4z = ObjectIdentity((1, 3, 6, 1, 2, 1, 100, 1, 11))
174 | if mibBuilder.loadTexts: transportDomainSctpIpv4z.setStatus('current')
175 | if mibBuilder.loadTexts: transportDomainSctpIpv4z.setDescription('The SCTP over IPv4 transport domain. The
176 | transportDomainSctpIpv6z = ObjectIdentity((1, 3, 6, 1, 2, 1, 100, 1, 12))
177 | if mibBuilder.loadTexts: transportDomainSctpIpv6z.setStatus('current')
178 | if mibBuilder.loadTexts: transportDomainSctpIpv6z.setDescription('The SCTP over IPv6 transport domain. The
179 | transportDomainLocal = ObjectIdentity((1, 3, 6, 1, 2, 1, 100, 1, 13))
180 | if mibBuilder.loadTexts: transportDomainLocal.setStatus('current')
181 | if mibBuilder.loadTexts: transportDomainLocal.setDescription('The Posix Local IPC transport domain. The cor
182 | transportDomainUdpDns = ObjectIdentity((1, 3, 6, 1, 2, 1, 100, 1, 14))
183 | if mibBuilder.loadTexts: transportDomainUdpDns.setStatus('current')
184 | if mibBuilder.loadTexts: transportDomainUdpDns.setDescription('The UDP transport domain using fully qualifi
185 | transportDomainTcpDns = ObjectIdentity((1, 3, 6, 1, 2, 1, 100, 1, 15))
186 | if mibBuilder.loadTexts: transportDomainTcpDns.setStatus('current')
187 | if mibBuilder.loadTexts: transportDomainTcpDns.setDescription('The TCP transport domain using fully qualifi
188 | transportDomainSctpDns = ObjectIdentity((1, 3, 6, 1, 2, 1, 100, 1, 16))
189 | if mibBuilder.loadTexts: transportDomainSctpDns.setStatus('current')

```

```

190 | if mibBuilder.loadTexts: transportDomainSctpDns.setDescription('The SCTP transport domain using fully quali
191 |
192 | class TransportDomain(TextualConvention, ObjectIdentifier):
193 |     description = 'A value that represents a transport domain. Some possible values, such as transportDomain
194 |     status = 'current'
195 |
196 | class TransportAddressType(TextualConvention, Integer32):
197 |     description = 'A value that represents a transport domain. This is the enumerated version of the transp
198 |     status = 'current'
199 |     subtypeSpec = Integer32.subtypeSpec + ConstraintsUnion(SingleValueConstraint(0, 1, 2, 3, 4, 5, 6, 7, 8,
200 |     namedValues = NamedValues(("unknown", 0), ("udpIpv4", 1), ("udpIpv6", 2), ("udpIpv4z", 3), ("udpIpv6z",
201 |
202 | class TransportAddress(TextualConvention, OctetString):
203 |     description = "Denotes a generic transport address. A TransportAddress value is always interpreted with
204 |     status = 'current'
205 |     subtypeSpec = OctetString.subtypeSpec + ValueSizeConstraint(0, 255)
206 |
207 | class TransportAddressIPv4(TextualConvention, OctetString):
208 |     description = 'Represents a transport address consisting of an IPv4 address and a port number (as used
209 |     status = 'current'
210 |     displayHint = '1d.1d.1d.1d:2d'
211 |     subtypeSpec = OctetString.subtypeSpec + ValueSizeConstraint(6, 6)
212 |     fixedLength = 6
213 |
214 |     def prettyIn(self, value):
215 |         if isinstance(value, tuple):
216 |             # Wild hack -- need to implement TextualConvention.prettyIn
217 |             value = inet_pton(socket.AF_INET, value[0]) + int2oct((value[1] >> 8) & 0xff) + int2oct(value[1
218 |             return OctetString.prettyIn(self, value)
219 |
220 |     # Socket address syntax coercion
221 |     def __asSocketAddress(self):
222 |         if not hasattr(self, '__tuple_value'):
223 |             v = self.asOctets()
224 |             self.__tuple_value = (
225 |                 inet_ntop(socket.AF_INET, v[:4]),
226 |                 oct2int(v[4]) << 8 | oct2int(v[5]),
227 |             )
228 |             return self.__tuple_value
229 |
230 |     def __iter__(self):
231 |         return iter(self.__asSocketAddress())
232 |
233 |     def __getitem__(self, item):
234 |         return self.__asSocketAddress()[item]
235 |
236 |
237 | class TransportAddressIPv6(TextualConvention, OctetString):
238 |     description = 'Represents a transport address consisting of an IPv6 address and a port number (as used
239 |     status = 'current'
240 |     displayHint = '0a[2x:2x:2x:2x:2x:2x:2x:2x]0a:2d'
241 |     subtypeSpec = OctetString.subtypeSpec + ValueSizeConstraint(18, 18)
242 |     fixedLength = 18
243 |
244 |     def prettyIn(self, value):
245 |         if not has_ipv6:
246 |             raise error.PySnmpError('IPv6 not supported by platform')
247 |         if isinstance(value, tuple):
248 |             value = inet_pton(socket.AF_INET6, value[0]) + int2oct((value[1] >> 8) & 0xff) + int2oct(value[1
249 |             return OctetString.prettyIn(self, value)
250 |
251 |     # Socket address syntax coercion
252 |     def __asSocketAddress(self):

```


=== ARQUIVO: .\dist\ControleImpressoras_internal\pysnmp\smi\mibs\instances__PYSNMP-USM-MIB.py ===

```
001 | #
002 | # This file is part of pysnmp software.
003 | #
004 | # Copyright (c) 2005-2019, Ilya Etingof <etingof@gmail.com>
005 | # License: http://snmplabs.com/pysnmp/license.html
006 | #
007 | MibScalarInstance, = mibBuilder.importSymbols('SNMPv2-SMI', 'MibScalarInstance')
008 |
009 | (pysnmpUsmDiscoverable,
010 |  pysnmpUsmDiscovery,
011 |  pysnmpUsmKeyType) = mibBuilder.importSymbols(
012 |      'PYSNMP-USM-MIB',
013 |      'pysnmpUsmDiscoverable',
014 |      'pysnmpUsmDiscovery',
015 |      'pysnmpUsmKeyType'
016 |  )
017 |
018 | __pysnmpUsmDiscoverable = MibScalarInstance(pysnmpUsmDiscoverable.name, (0,), pysnmpUsmDiscoverable.syntax)
019 | __pysnmpUsmDiscovery = MibScalarInstance(pysnmpUsmDiscovery.name, (0,), pysnmpUsmDiscovery.syntax)
020 | __pysnmpUsmKeyType = MibScalarInstance(pysnmpUsmKeyType.name, (0,), pysnmpUsmKeyType.syntax)
021 |
022 | mibBuilder.exportSymbols(
023 |     "__PYSNMP-USM-MIB",
024 |     pysnmpUsmDiscoverable=__pysnmpUsmDiscoverable,
025 |     pysnmpUsmDiscovery=__pysnmpUsmDiscovery,
026 |     pysnmpUsmKeyType=__pysnmpUsmKeyType
027 | )
```

=== ARQUIVO: .\dist\ControleImpressoras_internal\pysnmp\smi\mibs\instances__SNMP-FRAMEWORK-MIB.py ===

```
001 | #
002 | # This file is part of pysnmp software.
003 | #
004 | # Copyright (c) 2005-2019, Ilya Etingof <etingof@gmail.com>
005 | # License: http://snmplabs.com/pysnmp/license.html
006 | #
007 | import time
008 |
009 | MibScalarInstance, = mibBuilder.importSymbols('SNMPv2-SMI', 'MibScalarInstance')
010 |
011 | (snmpEngineID,
012 |  snmpEngineBoots,
013 |  snmpEngineTime,
014 |  snmpEngineMaxMessageSize) = mibBuilder.importSymbols(
015 |      'SNMP-FRAMEWORK-MIB',
016 |      'snmpEngineID',
017 |      'snmpEngineBoots',
018 |      'snmpEngineTime',
019 |      'snmpEngineMaxMessageSize'
020 |  )
021 |
022 | __snmpEngineID = MibScalarInstance(snmpEngineID.name, (0,), snmpEngineID.syntax)
023 | __snmpEngineBoots = MibScalarInstance(snmpEngineBoots.name, (0,), snmpEngineBoots.syntax.clone(1))
024 | __snmpEngineTime = MibScalarInstance(snmpEngineTime.name, (0,), snmpEngineTime.syntax.clone(int(time.time())))
025 | __snmpEngineMaxMessageSize = MibScalarInstance(snmpEngineMaxMessageSize.name, (0,),
```

```

026 |                                     snmpEngineMaxMessageSize.syntax.clone(4096))
027 |
028 | mibBuilder.exportSymbols(
029 |     '__SNMP-FRAMEWORK-MIB',
030 |     snmpEngineID=__snmpEngineID,
031 |     snmpEngineBoots=__snmpEngineBoots,
032 |     snmpEngineTime=__snmpEngineTime,
033 |     snmpEngineMaxMessageSize=__snmpEngineMaxMessageSize
034 | )

```

=== ARQUIVO: .\dist\ControleImpressoras_internal\pysnmp\smi\mibs\instances__SNMP-MPD-MIB.py ===

```

001 | #
002 | # This file is part of pysnmp software.
003 | #
004 | # Copyright (c) 2005-2019, Ilya Etingof <etingof@gmail.com>
005 | # License: http://snmplabs.com/pysnmp/license.html
006 | #
007 | MibScalarInstance, = mibBuilder.importSymbols('SNMPv2-SMI', 'MibScalarInstance')
008 |
009 | (snmpUnknownSecurityModels,
010 |  snmpInvalidMsgs,
011 |  snmpUnknownPDUHandlers) = mibBuilder.importSymbols(
012 |      'SNMP-MPD-MIB',
013 |      'snmpUnknownSecurityModels',
014 |      'snmpInvalidMsgs',
015 |      'snmpUnknownPDUHandlers',
016 |  )
017 |
018 | __snmpUnknownSecurityModels = MibScalarInstance(snmpUnknownSecurityModels.name, (0,),
019 |                                                  snmpUnknownSecurityModels.syntax.clone(0))
020 | __snmpInvalidMsgs = MibScalarInstance(snmpInvalidMsgs.name, (0,), snmpInvalidMsgs.syntax.clone(0))
021 | __snmpUnknownPDUHandlers = MibScalarInstance(snmpUnknownPDUHandlers.name, (0,), snmpUnknownPDUHandlers.syntax.clone(0))
022 |
023 | mibBuilder.exportSymbols(
024 |     '__SNMP-MPD-MIB',
025 |     snmpUnknownSecurityModels=__snmpUnknownSecurityModels,
026 |     snmpInvalidMsgs=__snmpInvalidMsgs,
027 |     snmpUnknownPDUHandlers=__snmpUnknownPDUHandlers
028 | )

```

=== ARQUIVO: .\dist\ControleImpressoras_internal\pysnmp\smi\mibs\instances__SNMP-TARGET-MIB.py ===

```

001 | #
002 | # This file is part of pysnmp software.
003 | #
004 | # Copyright (c) 2005-2019, Ilya Etingof <etingof@gmail.com>
005 | # License: http://snmplabs.com/pysnmp/license.html
006 | #
007 | MibScalarInstance, = mibBuilder.importSymbols(
008 |     'SNMPv2-SMI',
009 |     'MibScalarInstance'
010 | )
011 |
012 | (snmpTargetSpinLock,

```

```

013 | snmpUnavailableContexts,
014 | snmpUnknownContexts) = mibBuilder.importSymbols(
015 |     'SNMP-TARGET-MIB',
016 |     'snmpTargetSpinLock',
017 |     'snmpUnavailableContexts',
018 |     'snmpUnknownContexts'
019 | )
020 |
021 | __snmpTargetSpinLock = MibScalarInstance(snmpTargetSpinLock.name, (0,), snmpTargetSpinLock.syntax.clone(0))
022 | __snmpUnavailableContexts = MibScalarInstance(snmpUnavailableContexts.name, (0,),
023 |     snmpUnavailableContexts.syntax.clone(0))
024 | __snmpUnknownContexts = MibScalarInstance(snmpUnknownContexts.name, (0,), snmpUnknownContexts.syntax.clone(0))
025 |
026 | mibBuilder.exportSymbols(
027 |     '__SNMP-TARGET-MIB',
028 |     snmpTargetSpinLock=__snmpTargetSpinLock,
029 |     snmpUnavailableContexts=__snmpUnavailableContexts,
030 |     snmpUnknownContexts=__snmpUnknownContexts
031 | )

```

=== ARQUIVO: .\dist\ControleImpressoras_internal\pysnmp\smi\mibs\instances__SNMP-USER-BASED-SM-MIB.py ===

```

001 | #
002 | # This file is part of pysnmp software.
003 | #
004 | # Copyright (c) 2005-2019, Ilya Etingof <etingof@gmail.com>
005 | # License: http://snmplabs.com/pysnmp/license.html
006 | #
007 | MibScalarInstance, = mibBuilder.importSymbols('SNMPv2-SMI', 'MibScalarInstance')
008 |
009 | (usmStatsUnsupportedSecLevels,
010 |  usmStatsNotInTimeWindows,
011 |  usmStatsUnknownUserNames,
012 |  usmStatsUnknownEngineIDs,
013 |  usmStatsWrongDigests,
014 |  usmStatsDecryptionErrors,
015 |  usmUserSpinLock) = mibBuilder.importSymbols(
016 |      'SNMP-USER-BASED-SM-MIB',
017 |      'usmStatsUnsupportedSecLevels',
018 |      'usmStatsNotInTimeWindows',
019 |      'usmStatsUnknownUserNames',
020 |      'usmStatsUnknownEngineIDs',
021 |      'usmStatsWrongDigests',
022 |      'usmStatsDecryptionErrors',
023 |      'usmUserSpinLock'
024 | )
025 |
026 | __usmStatsUnsupportedSecLevels = MibScalarInstance(usmStatsUnsupportedSecLevels.name, (0,),
027 |     usmStatsUnsupportedSecLevels.syntax.clone(0))
028 | __usmStatsNotInTimeWindows = MibScalarInstance(usmStatsNotInTimeWindows.name, (0,),
029 |     usmStatsNotInTimeWindows.syntax.clone(0))
030 | __usmStatsUnknownUserNames = MibScalarInstance(usmStatsUnknownUserNames.name, (0,),
031 |     usmStatsUnknownUserNames.syntax.clone(0))
032 | __usmStatsUnknownEngineIDs = MibScalarInstance(usmStatsUnknownEngineIDs.name, (0,),
033 |     usmStatsUnknownEngineIDs.syntax.clone(0))
034 | __usmStatsWrongDigests = MibScalarInstance(usmStatsWrongDigests.name, (0,), usmStatsWrongDigests.syntax.clone(0))
035 | __usmStatsDecryptionErrors = MibScalarInstance(usmStatsDecryptionErrors.name, (0,),
036 |     usmStatsDecryptionErrors.syntax.clone(0))
037 | __usmUserSpinLock = MibScalarInstance(usmUserSpinLock.name, (0,), usmUserSpinLock.syntax.clone(0))

```

```

038 |
039 | mibBuilder.exportSymbols(
040 |     '__SNMP-USER-BASED-SM-MIB',
041 |     usmStatsUnsupportedSecLevels=__usmStatsUnsupportedSecLevels,
042 |     usmStatsNotInTimeWindows=__usmStatsNotInTimeWindows,
043 |     usmStatsUnknownUserNames=__usmStatsUnknownUserNames,
044 |     usmStatsUnknownEngineIDs=__usmStatsUnknownEngineIDs,
045 |     usmStatsWrongDigests=__usmStatsWrongDigests,
046 |     usmStatsDecryptionErrors=__usmStatsDecryptionErrors,
047 |     usmUserSpinLock=__usmUserSpinLock
048 | )

```

=== ARQUIVO: .\dist\ControleImpressoras_internal\pysnmp\smi\mibs\instances__SNMP-VIEW-BASED-ACM-MIB.py ===

```

001 | #
002 | # This file is part of pysnmp software.
003 | #
004 | # Copyright (c) 2005-2019, Ilya Etingof <etingof@gmail.com>
005 | # License: http://snmplabs.com/pysnmp/license.html
006 | #
007 | MibScalarInstance, = mibBuilder.importSymbols('SNMPv2-SMI', 'MibScalarInstance')
008 |
009 | vacmViewSpinLock, = mibBuilder.importSymbols('SNMP-VIEW-BASED-ACM-MIB', 'vacmViewSpinLock')
010 |
011 | __vacmViewSpinLock = MibScalarInstance(vacmViewSpinLock.name, (0,), vacmViewSpinLock.syntax)
012 |
013 | mibBuilder.exportSymbols(
014 |     "__SNMP-VIEW-BASED-ACM-MIB",
015 |     vacmViewSpinLock=__vacmViewSpinLock
016 | )

```

=== ARQUIVO: .\dist\ControleImpressoras_internal\pysnmp\smi\mibs\instances__SNMPv2-MIB.py ===

```

001 | #
002 | # This file is part of pysnmp software.
003 | #
004 | # Copyright (c) 2005-2019, Ilya Etingof <etingof@gmail.com>
005 | # License: http://snmplabs.com/pysnmp/license.html
006 | #
007 | from sys import version
008 | from time import time
009 | from pysnmp import __version__
010 |
011 | (MibScalarInstance,
012 |  TimeTicks) = mibBuilder.importSymbols(
013 |     'SNMPv2-SMI',
014 |     'MibScalarInstance',
015 |     'TimeTicks'
016 | )
017 |
018 | (sysDescr,
019 |  sysObjectID,
020 |  sysUpTime,
021 |  sysContact,
022 |  sysName,

```

```
023 | sysLocation,
024 | sysServices,
025 | sysORLastChange,
026 | snmpInPkts,
027 | snmpOutPkts,
028 | snmpInBadVersions,
029 | snmpInBadCommunityNames,
030 | snmpInBadCommunityUses,
031 | snmpInASNParseErrs,
032 | snmpInTooBigs,
033 | snmpInNoSuchNames,
034 | snmpInBadValues,
035 | snmpInReadOnlys,
036 | snmpInGenErrs,
037 | snmpInTotalReqVars,
038 | snmpInTotalSetVars,
039 | snmpInGetRequests,
040 | snmpInGetNexts,
041 | snmpInSetRequests,
042 | snmpInGetResponses,
043 | snmpInTraps,
044 | snmpOutTooBigs,
045 | snmpOutNoSuchNames,
046 | snmpOutBadValues,
047 | snmpOutGenErrs,
048 | snmpOutSetRequests,
049 | snmpOutGetResponses,
050 | snmpOutTraps,
051 | snmpEnableAuthenTraps,
052 | snmpSilentDrops,
053 | snmpProxyDrops,
054 | snmpTrapOID,
055 | coldStart,
056 | snmpSetSerialNo) = mibBuilder.importSymbols(
057 |     'SNMPv2-MIB',
058 |     'sysDescr',
059 |     'sysObjectID',
060 |     'sysUpTime',
061 |     'sysContact',
062 |     'sysName',
063 |     'sysLocation',
064 |     'sysServices',
065 |     'sysORLastChange',
066 |     'snmpInPkts',
067 |     'snmpOutPkts',
068 |     'snmpInBadVersions',
069 |     'snmpInBadCommunityNames',
070 |     'snmpInBadCommunityUses',
071 |     'snmpInASNParseErrs',
072 |     'snmpInTooBigs',
073 |     'snmpInNoSuchNames',
074 |     'snmpInBadValues',
075 |     'snmpInReadOnlys',
076 |     'snmpInGenErrs',
077 |     'snmpInTotalReqVars',
078 |     'snmpInTotalSetVars',
079 |     'snmpInGetRequests',
080 |     'snmpInGetNexts',
081 |     'snmpInSetRequests',
082 |     'snmpInGetResponses',
083 |     'snmpInTraps',
084 |     'snmpOutTooBigs',
085 |     'snmpOutNoSuchNames',
```

```

086 |     'snmpOutBadValues',
087 |     'snmpOutGenErrs',
088 |     'snmpOutSetRequests',
089 |     'snmpOutGetResponses',
090 |     'snmpOutTraps',
091 |     'snmpEnableAuthenTraps',
092 |     'snmpSilentDrops',
093 |     'snmpProxyDrops',
094 |     'snmpTrapOID',
095 |     'coldStart',
096 |     'snmpSetSerialNo'
097 | )
098 |
099 | __sysDescr = MibScalarInstance(sysDescr.name, (0,), sysDescr.syntax.clone(
100 |     "PySNMP engine version %, Python %" % (__version__, version.replace('\n', ' ').replace('\r', ' ')))
101 | __sysObjectID = MibScalarInstance(sysObjectID.name, (0,), sysObjectID.syntax.clone((1, 3, 6, 1, 4, 1, 20408
102 |
103 |
104 | class SysUpTime(TimeTicks):
105 |     createdAt = time()
106 |
107 |     def clone(self, **kwargs):
108 |         if 'value' not in kwargs:
109 |             kwargs['value'] = int((time() - self.createdAt) * 100)
110 |         return TimeTicks.clone(self, **kwargs)
111 |
112 |
113 | __sysUpTime = MibScalarInstance(sysUpTime.name, (0,), SysUpTime(0))
114 | __sysContact = MibScalarInstance(sysContact.name, (0,), sysContact.syntax.clone(''))
115 | __sysName = MibScalarInstance(sysName.name, (0,), sysName.syntax.clone(''))
116 | __sysLocation = MibScalarInstance(sysLocation.name, (0,), sysLocation.syntax.clone(''))
117 | __sysServices = MibScalarInstance(sysServices.name, (0,), sysServices.syntax.clone(0))
118 | __sysORLastChange = MibScalarInstance(sysORLastChange.name, (0,), sysORLastChange.syntax.clone(0))
119 | __snmpInPkts = MibScalarInstance(snmpInPkts.name, (0,), snmpInPkts.syntax.clone(0))
120 | __snmpOutPkts = MibScalarInstance(snmpOutPkts.name, (0,), snmpOutPkts.syntax.clone(0))
121 | __snmpInBadVersions = MibScalarInstance(snmpInBadVersions.name, (0,), snmpInBadVersions.syntax.clone(0))
122 | __snmpInBadCommunityNames = MibScalarInstance(snmpInBadCommunityNames.name, (0,),
123 |     snmpInBadCommunityNames.syntax.clone(0))
124 | __snmpInBadCommunityUses = MibScalarInstance(snmpInBadCommunityUses.name, (0,), snmpInBadCommunityUses.synt
125 | __snmpInASNParseErrs = MibScalarInstance(snmpInASNParseErrs.name, (0,), snmpInASNParseErrs.syntax.clone(0))
126 | __snmpInTooBigs = MibScalarInstance(snmpInTooBigs.name, (0,), snmpInTooBigs.syntax.clone(0))
127 | __snmpInNoSuchNames = MibScalarInstance(snmpInNoSuchNames.name, (0,), snmpInNoSuchNames.syntax.clone(0))
128 | __snmpInBadValues = MibScalarInstance(snmpInBadValues.name, (0,), snmpInBadValues.syntax.clone(0))
129 | __snmpInReadOnlyls = MibScalarInstance(snmpInReadOnlyls.name, (0,), snmpInReadOnlyls.syntax.clone(0))
130 | __snmpInGenErrs = MibScalarInstance(snmpInGenErrs.name, (0,), snmpInGenErrs.syntax.clone(0))
131 | __snmpInTotalReqVars = MibScalarInstance(snmpInTotalReqVars.name, (0,), snmpInTotalReqVars.syntax.clone(0))
132 | __snmpInTotalSetVars = MibScalarInstance(snmpInTotalSetVars.name, (0,), snmpInTotalSetVars.syntax.clone(0))
133 | __snmpInGetRequests = MibScalarInstance(snmpInGetRequests.name, (0,), snmpInGetRequests.syntax.clone(0))
134 | __snmpInGetNexts = MibScalarInstance(snmpInGetNexts.name, (0,), snmpInGetNexts.syntax.clone(0))
135 | __snmpInSetRequests = MibScalarInstance(snmpInSetRequests.name, (0,), snmpInSetRequests.syntax.clone(0))
136 | __snmpInGetResponses = MibScalarInstance(snmpInGetResponses.name, (0,), snmpInGetResponses.syntax.clone(0))
137 | __snmpInTraps = MibScalarInstance(snmpInTraps.name, (0,), snmpInTraps.syntax.clone(0))
138 | __snmpOutTooBigs = MibScalarInstance(snmpOutTooBigs.name, (0,), snmpOutTooBigs.syntax.clone(0))
139 | __snmpOutNoSuchNames = MibScalarInstance(snmpOutNoSuchNames.name, (0,), snmpOutNoSuchNames.syntax.clone(0))
140 | __snmpOutBadValues = MibScalarInstance(snmpOutBadValues.name, (0,), snmpOutBadValues.syntax.clone(0))
141 | __snmpOutGenErrs = MibScalarInstance(snmpOutGenErrs.name, (0,), snmpOutGenErrs.syntax.clone(0))
142 | __snmpOutSetRequests = MibScalarInstance(snmpOutSetRequests.name, (0,), snmpOutSetRequests.syntax.clone(0))
143 | __snmpOutGetResponses = MibScalarInstance(snmpOutGetResponses.name, (0,), snmpOutGetResponses.syntax.clone(
144 | __snmpOutTraps = MibScalarInstance(snmpOutTraps.name, (0,), snmpOutTraps.syntax.clone(0))
145 | __snmpEnableAuthenTraps = MibScalarInstance(snmpEnableAuthenTraps.name, (0,), snmpEnableAuthenTraps.syntax.
146 | __snmpSilentDrops = MibScalarInstance(snmpSilentDrops.name, (0,), snmpSilentDrops.syntax.clone(0))
147 | __snmpProxyDrops = MibScalarInstance(snmpProxyDrops.name, (0,), snmpProxyDrops.syntax.clone(0))
148 | __snmpTrapOID = MibScalarInstance(snmpTrapOID.name, (0,), snmpTrapOID.syntax.clone(coldStart.name))

```

```

149 | __snmpSetSerialNo = MibScalarInstance(snmpSetSerialNo.name, (0,), snmpSetSerialNo.syntax.clone(0))
150 |
151 | mibBuilder.exportSymbols(
152 |     "__SNMPv2-MIB",
153 |     sysDescr=__sysDescr,
154 |     sysObjectID=__sysObjectID,
155 |     sysUpTime=__sysUpTime,
156 |     sysContact=__sysContact,
157 |     sysName=__sysName,
158 |     sysLocation=__sysLocation,
159 |     sysServices=__sysServices,
160 |     sysORLastChange=__sysORLastChange,
161 |     snmpInPkts=__snmpInPkts,
162 |     snmpOutPkts=__snmpOutPkts,
163 |     snmpInBadVersions=__snmpInBadVersions,
164 |     snmpInBadCommunityNames=__snmpInBadCommunityNames,
165 |     snmpInBadCommunityUses=__snmpInBadCommunityUses,
166 |     snmpInASNParseErrs=__snmpInASNParseErrs,
167 |     snmpInTooBigs=__snmpInTooBigs,
168 |     snmpInNoSuchNames=__snmpInNoSuchNames,
169 |     snmpInBadValues=__snmpInBadValues,
170 |     snmpInReadOnly=__snmpInReadOnly,
171 |     snmpInGenErrs=__snmpInGenErrs,
172 |     snmpInTotalReqVars=__snmpInTotalReqVars,
173 |     snmpInTotalSetVars=__snmpInTotalSetVars,
174 |     snmpInGetRequests=__snmpInGetRequests,
175 |     snmpInGetNexts=__snmpInGetNexts,
176 |     snmpInSetRequests=__snmpInSetRequests,
177 |     snmpInGetResponses=__snmpInGetResponses,
178 |     snmpInTraps=__snmpInTraps,
179 |     snmpOutTooBigs=__snmpOutTooBigs,
180 |     snmpOutNoSuchNames=__snmpOutNoSuchNames,
181 |     snmpOutBadValues=__snmpOutBadValues,
182 |     snmpOutGenErrs=__snmpOutGenErrs,
183 |     snmpOutSetRequests=__snmpOutSetRequests,
184 |     snmpOutGetResponses=__snmpOutGetResponses,
185 |     snmpOutTraps=__snmpOutTraps,
186 |     snmpEnableAuthenTraps=__snmpEnableAuthenTraps,
187 |     snmpSilentDrops=__snmpSilentDrops,
188 |     snmpProxyDrops=__snmpProxyDrops,
189 |     snmpTrapOID=__snmpTrapOID,
190 |     snmpSetSerialNo=__snmpSetSerialNo
191 | )

```

=== ARQUIVO: .\scripts\organizar_pecas.py ===

```

001 | """
002 | Organiza a planilha de peças para importação no sistema.
003 | Separa descrição em nome + modelo, limpa dados e gera xlsx pronto.
004 | """
005 | import re
006 | import sys
007 | from pathlib import Path
008 |
009 | import openpyxl
010 | from openpyxl.styles import Alignment, Font, PatternFill, Border, Side
011 |
012 |
013 | # ■■ Palavras que indicam modelo/spec (não são nome da peça) ■■
014 | _MODELPattern = re.compile(
015 |     r"(?:^|\s)"

```



```

016 | r"("
017 | r"[A-Z]{1,3}\s?\d{2,5}(:[/\-\]\s?[A-Z]{0,3}\d{1,5})*" # HP P1102/M1132, SCX 4833/SCX 5637
018 | r"|P\d{3,5}(:[/\-\]\d{1,5})*" # P2035, P1102
019 | r"|\d{3,5}[A-Z]?" # 1010, 2035, 8157
020 | r"| [A-Z]\d{3,5}(:[/\-\]\d{1,5})*" # M1132, L4150, T650
021 | r"|SCX\s?\d{3,5}(:[/\-\]\s?[A-Z]{0,3}\d{1,5})*" # SCX 4833/SCX 5637/M4070
022 | r"|ML\s?\d{3,5}" # ML 2851
023 | r"|FS\s?\d{3,5}" # FS1035
024 | r"|X\d{3,5}" # X203
025 | r")"
026 | r"(?:\s*\([^\)]*\))?" # (PAR), (CONJUNTO)
027 | r"(?:\s+(?:METALICO|RESERVA|CONJUNTO))?" # sufixos descritivos
028 | )
029 |
030 |
031 | def _limpar_descricao(desc: str, marca: str) -> tuple[str, str]:
032 |     """Separa 'ROLO DE PRESSAO HP P1102/M1132' → ('ROLO DE PRESSAO', 'HP P1102/M1132')"""
033 |     desc = desc.strip()
034 |
035 |     # Remove marca do início se presente
036 |     if desc.upper().startswith(marca.upper()):
037 |         desc = desc[len(marca):].strip()
038 |
039 |     # Tenta extrair modelo do final
040 |     modelos_encontrados = list(_MODELPattern.finditer(desc))
041 |
042 |     if modelos_encontrados:
043 |         ultimo = modelos_encontrados[-1]
044 |         nome = desc[:ultimo.start()].strip()
045 |         modelo = desc[ultimo.start():].strip()
046 |
047 |         # Limpa nome: remove marca se sobrou
048 |         nome = re.sub(rf"\b{re.escape(marca)}\b", "", nome, flags=re.IGNORECASE).strip()
049 |         # Remove espaços duplos
050 |         nome = re.sub(r"\s+", " ", nome).strip()
051 |         modelo = re.sub(r"\s+", " ", modelo).strip()
052 |
053 |         # Remove marca duplicada do modelo (ex: "HP HP 4014" → "HP 4014")
054 |         modelo = re.sub(rf"^\b{re.escape(marca)}\s+{re.escape(marca)}\b", marca, modelo, flags=re.IGNORECASE)
055 |
056 |         # Se nome ficou vazio, usa a descrição toda como nome
057 |         if not nome:
058 |             nome = desc.strip()
059 |
060 |         return nome, modelo
061 |
062 |     # Sem modelo encontrado: tudo é nome
063 |     nome = re.sub(rf"\b{re.escape(marca)}\b", "", desc, flags=re.IGNORECASE).strip()
064 |     nome = re.sub(r"\s+", " ", nome).strip()
065 |     return nome or desc, ""
066 |
067 |
068 | def processar_arquivo(caminho: str) -> list[dict]:
069 |     """Lê todas as abas e retorna lista de dicts organizados."""
070 |     wb = openpyxl.load_workbook(caminho, read_only=True, data_only=True)
071 |     pecas = []
072 |
073 |     for sheet_name in wb.sheetnames:
074 |         if sheet_name.upper() == "INDICE":
075 |             continue
076 |
077 |         marca = sheet_name.strip()
078 |         ws = wb[sheet_name]

```

```

079 |         rows = list(ws.iter_rows(values_only=True))
080 |
081 |         # Encontra a linha do cabeçalho (contém "N", "QTD", etc.)
082 |         header_idx = None
083 |         for i, row in enumerate(rows):
084 |             if row and any(c and str(c).upper().strip() in ("N", "NUM", "Nº") for c in row if c):
085 |                 header_idx = i
086 |                 break
087 |
088 |         if header_idx is None:
089 |             continue
090 |
091 |         header = [str(c or "").upper().strip() for c in rows[header_idx]]
092 |
093 |         # Mapeia índices das colunas
094 |         col_map = {}
095 |         for i, h in enumerate(header):
096 |             if h in ("N", "NUM", "Nº"):
097 |                 col_map["n"] = i
098 |             elif h in ("QTD", "QTDE", "QUANTIDADE"):
099 |                 col_map["qtd"] = i
100 |             elif "COD" in h or "REFERENC" in h or "REF" in h:
101 |                 col_map["codigo"] = i
102 |             elif "DESCR" in h:
103 |                 col_map["descricao"] = i
104 |             elif "CATEG" in h:
105 |                 col_map["categoria"] = i
106 |             elif "STATUS" in h:
107 |                 col_map["status"] = i
108 |             elif "FORNEC" in h:
109 |                 col_map["fornecedor"] = i
110 |             elif "NOTA" in h:
111 |                 col_map["nota"] = i
112 |             elif "DATA" in h:
113 |                 col_map["data"] = i
114 |
115 |         # Processa linhas de dados
116 |         for row in rows[header_idx + 1:]:
117 |             if not row:
118 |                 continue
119 |
120 |             # Pula linhas de resumo/rodapé
121 |             n_val = row[col_map.get("n", 0)] if col_map.get("n") is not None else None
122 |             if n_val is None or not str(n_val).strip().isdigit():
123 |                 continue
124 |
125 |             qtd_val = row[col_map.get("qtd", 1)] if col_map.get("qtd") is not None else 0
126 |             try:
127 |                 qtd = int(float(str(qtd_val or 0)))
128 |             except (ValueError, TypeError):
129 |                 qtd = 0
130 |
131 |             cod_val = row[col_map.get("codigo", 3)] if col_map.get("codigo") is not None else ""
132 |             codigo = str(cod_val or "").strip()
133 |
134 |             desc_val = row[col_map.get("descricao", 4)] if col_map.get("descricao") is not None else ""
135 |             descricao = str(desc_val or "").strip()
136 |
137 |             cat_val = row[col_map.get("categoria", 5)] if col_map.get("categoria") is not None else ""
138 |             categoria = str(cat_val or "").strip()
139 |
140 |             status_val = row[col_map.get("status", 6)] if col_map.get("status") is not None else ""
141 |             status_planilha = str(status_val or "").strip()

```

```

142 |
143 |         fornec_val = row[col_map.get("fornecedor", 7)] if col_map.get("fornecedor") is not None else ""
144 |         fornecedor = str(fornec_val or "").strip()
145 |
146 |         if not descricao or descricao.upper() == "DESCRICAO":
147 |             continue
148 |
149 |         nome, modelo = _limpar_descricao(descricao, marca)
150 |
151 |         # Modelo compativel: marca + modelo, mas evita marca duplicada
152 |         if modelo:
153 |             if modelo.upper().startswith(marca.upper()):
154 |                 modelo_compat = modelo
155 |             else:
156 |                 modelo_compat = f"{marca} {modelo}".strip()
157 |         else:
158 |             modelo_compat = marca
159 |
160 |         # Status do sistema baseado no estoque
161 |         if qtd <= 0:
162 |             status_sistema = "Sem Estoque"
163 |         elif status_planilha.upper() == "BAIXO":
164 |             status_sistema = "Baixo"
165 |         else:
166 |             status_sistema = "Normal"
167 |
168 |         pecas.append({
169 |             "marca": marca,
170 |             "codigo": codigo if codigo and codigo.upper() != "DESCONHECIDO" else "",
171 |             "nome": nome,
172 |             "descricao": descricao,
173 |             "modelo": modelo,
174 |             "modelo_compativel": modelo_compat,
175 |             "quantidade_estoque": qtd,
176 |             "categoria": categoria,
177 |             "status_original": status_planilha,
178 |             "status_sistema": status_sistema,
179 |             "fornecedor": fornecedor if fornecedor != "-" else "",
180 |         })
181 |
182 |     wb.close()
183 |     return pecas
184 |
185 |
186 | def gerar_xlsx(pecas: list[dict], saida: str) -> None:
187 |     """Gera planilha limpa para importação."""
188 |     wb = openpyxl.Workbook()
189 |     ws = wb.active
190 |     ws.title = "Peças Importação"
191 |
192 |     # Estilos
193 |     header_font = Font(bold=True, color="FFFFFF", size=11)
194 |     header_fill = PatternFill(start_color="4472C4", end_color="4472C4", fill_type="solid")
195 |     header_align = Alignment(horizontal="center", vertical="center", wrap_text=True)
196 |     thin_border = Border(
197 |         left=Side(style="thin"), right=Side(style="thin"),
198 |         top=Side(style="thin"), bottom=Side(style="thin")
199 |     )
200 |
201 |     # Cabeçalho
202 |     colunas = ["Código", "Nome", "Descrição", "Modelo Compatível", "Quantidade",
203 |                "Estoque Mínimo", "Categoria", "Marca", "Fornecedor"]
204 |     for c, titulo in enumerate(colunas, 1):

```

```

205 |         cell = ws.cell(row=1, column=c, value=titulo)
206 |         cell.font = header_font
207 |         cell.fill = header_fill
208 |         cell.alignment = header_align
209 |         cell.border = thin_border
210 |
211 |     # Dados
212 |     for i, p in enumerate(pecas, 2):
213 |         dados = [
214 |             p["codigo"],
215 |             p["nome"],
216 |             p["descricao"],
217 |             p["modelo_compativel"],
218 |             p["quantidade_estoque"],
219 |             1, # estoque mínimo padrão
220 |             p["categoria"],
221 |             p["marca"],
222 |             p["fornecedor"],
223 |         ]
224 |         for c, val in enumerate(dados, 1):
225 |             cell = ws.cell(row=i, column=c, value=val)
226 |             cell.border = thin_border
227 |             if c in (5, 6):
228 |                 cell.alignment = Alignment(horizontal="center")
229 |
230 |     # Larguras das colunas
231 |     larguras = [15, 40, 50, 30, 12, 15, 25, 15, 25]
232 |     for c, larg in enumerate(larguras, 1):
233 |         ws.column_dimensions[openpyxl.utils.get_column_letter(c)].width = larg
234 |
235 |     # Congela primeira linha
236 |     ws.freeze_panes = "A2"
237 |
238 |     # Auto-filtro
239 |     ws.auto_filter.ref = ws.dimensions
240 |
241 |     wb.save(saida)
242 |     print(f"Planilha salva em: {saida}")
243 |     print(f"Total de peças: {len(pecas)}")
244 |
245 |
246 | if __name__ == "__main__":
247 |     entrada = r"C:\Users\Suporte Tecnico\Downloads\Planilha atualizada de pecas\PECAS_IMPRESSORAS_RECIFE_v2
248 |     saida = r"C:\Users\Suporte Tecnico\Downloads\Planilha atualizada de pecas\PECAS_IMPORTACAO_v2.xlsx"
249 |
250 |     if len(sys.argv) > 1:
251 |         entrada = sys.argv[1]
252 |     if len(sys.argv) > 2:
253 |         saida = sys.argv[2]
254 |
255 |     print(f"Lendo: {entrada}")
256 |     pecas = processar_arquivo(entrada)
257 |     print(f"Peças encontradas: {len(pecas)}")
258 |
259 |     # Mostra preview
260 |     print("\n=== Preview (10 primeiras) ===")
261 |     for p in pecas[:10]:
262 |         print(f"  Nome: {p['nome']:35s} | Modelo: {p['modelo_compativel']:25s} | Qtd: {p['quantidade_estoque']
263 |
264 |     print(f"\nGerando planilha de importação...")
265 |     gerar_xlsx(pecas, saida)

```

=== ARQUIVO: .ltests\confest.py ===

```
001 | import pytest
002 | from sqlalchemy import create_engine
003 | from sqlalchemy.orm import Session, sessionmaker
004 |
005 | from app.models.base import Base
006 | from app.models.user import User
007 | from app.utils.cache import invalidate_all
008 | from app.utils.security import hash_password
009 |
010 |
011 | @pytest.fixture(autouse=True)
012 | def _clear_cache() -> None:
013 |     invalidate_all()
014 |
015 |
016 | @pytest.fixture
017 | def db_session():
018 |     engine = create_engine("sqlite:///memory:", echo=False)
019 |     Base.metadata.create_all(engine)
020 |     TestSession = sessionmaker(bind=engine)
021 |     session = TestSession()
022 |     yield session
023 |     session.close()
024 |
025 |
026 | @pytest.fixture
027 | def admin_user(db_session: Session) -> User:
028 |     user = User(
029 |         nome="Admin Teste",
030 |         email="admin@teste.com",
031 |         username="admin",
032 |         senha_hash=hash_password("123456"),
033 |         perfil="admin",
034 |         ativo=True,
035 |     )
036 |     db_session.add(user)
037 |     db_session.commit()
038 |     return user
039 |
040 |
041 | @pytest.fixture
042 | def inactive_user(db_session: Session) -> User:
043 |     user = User(
044 |         nome="Inativo",
045 |         email="inativo@teste.com",
046 |         username="inativo",
047 |         senha_hash=hash_password("123456"),
048 |         perfil="visualizador",
049 |         ativo=False,
050 |     )
051 |     db_session.add(user)
052 |     db_session.commit()
053 |     return user
```

=== ARQUIVO: .ltests\test_models.py ===

```
001 | from datetime import datetime
002 |
```

```

003 | from app.models.company import Company
004 | from app.models.printer import Printer
005 | from app.models.user import User
006 | from app.utils.security import hash_password
007 |
008 |
009 | def test_create_user(db_session):
010 |     user = User(
011 |         nome="João",
012 |         email="joao@teste.com",
013 |         username="joao",
014 |         senha_hash=hash_password("123"),
015 |         perfil="tecnico",
016 |     )
017 |     db_session.add(user)
018 |     db_session.commit()
019 |
020 |     saved = db_session.query(User).filter_by(username="joao").first()
021 |     assert saved is not None
022 |     assert saved.nome == "João"
023 |     assert saved.perfil == "tecnico"
024 |     assert saved.ativo is True
025 |
026 |
027 | def test_create_printer(db_session):
028 |     printer = Printer(
029 |         patrimonio="PAT001",
030 |         modelo="HP LaserJet M404",
031 |         serial="ABC123",
032 |         status="Operacional",
033 |         local_atual="Matriz",
034 |     )
035 |     db_session.add(printer)
036 |     db_session.commit()
037 |
038 |     saved = db_session.query(Printer).filter_by(patrimonio="PAT001").first()
039 |     assert saved is not None
040 |     assert saved.modelo == "HP LaserJet M404"
041 |
042 |
043 | def test_printer_default_status(db_session):
044 |     printer = Printer(patrimonio="PAT002", modelo="Epson L3250")
045 |     db_session.add(printer)
046 |     db_session.commit()
047 |
048 |     assert printer.status == "Operacional"
049 |
050 |
051 | def test_login_admin_success(db_session, admin_user):
052 |     from app.utils.security import verify_password
053 |     user = db_session.query(User).filter_by(username="admin").first()
054 |     assert user is not None
055 |     assert user.ativo is True
056 |     assert verify_password("123456", user.senha_hash)
057 |
058 |
059 | def test_login_inactive_user_fails(db_session, inactive_user):
060 |     user = db_session.query(User).filter_by(username="inativo").first()
061 |     assert user is not None
062 |     assert user.ativo is False
063 |
064 |
065 | def test_login_wrong_password(db_session, admin_user):

```

```
066 | from app.utils.security import verify_password
067 | assert not verify_password("senha_errada", admin_user.senha_hash)
068 |
069 |
070 | def test_create_company(db_session):
071 |     company = Company(nome="Empresa Teste", tipo="filial")
072 |     db_session.add(company)
073 |     db_session.commit()
074 |
075 |     saved = db_session.query(Company).filter_by(nome="Empresa Teste").first()
076 |     assert saved is not None
077 |     assert saved.tipo == "filial"
078 |
079 |
080 | # ████████████████████████████████████████████████████████████████████████████
081 | # Edge cases - Model defaults & nullables
082 | # ████████████████████████████████████████████████████████████████████████████
083 |
084 | def test_printer_all_nullable_fields(db_session):
085 |     p = Printer(patrimonio="NULLS01", modelo="HP")
086 |     db_session.add(p)
087 |     db_session.commit()
088 |     assert p.serial == ""
089 |     assert p.marca == ""
090 |     assert p.tipo == ""
091 |     assert p.ip_rede == ""
092 |     assert p.observacao == ""
093 |     assert p.mac_address == ""
094 |     assert p.tecnico == ""
095 |
096 |
097 | def test_printer_soft_delete_default(db_session):
098 |     p = Printer(patrimonio="SDEL", modelo="HP")
099 |     db_session.add(p)
100 |     db_session.commit()
101 |     assert p.deleted_at is None
102 |
103 |
104 | def test_user_defaults(db_session):
105 |     from app.utils.security import hash_password
106 |     u = User(nome="Default", username="default", email="d@t.com", senha_hash=hash_password("123"))
107 |     db_session.add(u)
108 |     db_session.commit()
109 |     assert u.ativo is True
110 |     assert u.perfil == "visualizador"
111 |
112 |
113 | def test_technician_soft_delete_default(db_session):
114 |     from app.models.technician import Technician
115 |     t = Technician(nome_completo="Tec", telefone="81")
116 |     db_session.add(t)
117 |     db_session.commit()
118 |     assert t.deleted_at is None
119 |     assert t.ativo is True
120 |
121 |
122 | def test_part_default_estoque(db_session):
123 |     from app.models.part import Part
124 |     p = Part(codigo="PADRAO", nome="Peça Padrão")
125 |     db_session.add(p)
126 |     db_session.commit()
127 |     assert p.quantidade_estoque == 0
128 |     assert p.estoque_minimo == 1
```

```

129 |     assert p.deleted_at is None
130 |
131 |
132 | def test_alert_defaults(db_session):
133 |     from app.models.alert import Alert
134 |     a = Alert(tipo="info", titulo="Teste")
135 |     db_session.add(a)
136 |     db_session.commit()
137 |     assert a.resolvido is False
138 |     assert a.notificado is False
139 |     assert a.deleted_at is None
140 |     assert a.printer_id is None
141 |     assert a.part_id is None
142 |
143 |
144 | def test_attachment_defaults(db_session):
145 |     from app.models.attachment import Attachment
146 |     a = Attachment(entity_type="printer", entity_id=1, filename="test.pdf", original_name="test.pdf", mime_
147 |     db_session.add(a)
148 |     db_session.commit()
149 |     assert a.created_at is not None
150 |
151 |
152 | def test_login_history_defaults(db_session):
153 |     from app.models.login_history import LoginHistory
154 |     lh = LoginHistory(user_id=1, login_at=datetime.now())
155 |     db_session.add(lh)
156 |     db_session.commit()
157 |     assert lh.logout_at is None
158 |
159 |
160 | def test_audit_log_defaults(db_session):
161 |     from app.models.audit_log import AuditLog
162 |     log = AuditLog(user_id=1, acao="CRIAR", tabela_alvo="printers", registro_id="abc")
163 |     db_session.add(log)
164 |     db_session.commit()
165 |     assert log.dados_antes is None
166 |     assert log.dados_depois is None
167 |
168 |
169 | def test_printer_activity_relationship_empty(db_session):
170 |     p = Printer(patrimonio="NOREL", modelo="HP")
171 |     db_session.add(p)
172 |     db_session.commit()
173 |     assert p.activities == []
174 |
175 |
176 | def test_company_printers_relationship_empty(db_session):
177 |     c = Company(nome="Sem Impressoras")
178 |     db_session.add(c)
179 |     db_session.commit()
180 |     assert c.printers == []

```

=== ARQUIVO: .ltests/test_security.py ===

```

001 | import pytest
002 |
003 | from app.utils.security import hash_password, verify_password
004 |
005 |
006 | def test_hash_and_verify():

```



```

003 |
004 |
005 | def test_printer_service_criar(db_session):
006 |     from app.services.printer_service import PrinterService
007 |     svc = PrinterService(db_session)
008 |     p = svc.criar(patrimonio="SVC001", modelo="HP Test", status="Operacional")
009 |     assert p.patrimonio == "SVC001"
010 |     assert p.status == "Operacional"
011 |     assert p.id is not None
012 |
013 |
014 | def test_printer_service_listar(db_session):
015 |     from app.services.printer_service import PrinterService
016 |     svc = PrinterService(db_session)
017 |     svc.criar(patrimonio="LST001", modelo="Epson")
018 |     svc.criar(patrimonio="LST002", modelo="Brother")
019 |     todos = svc.listar_todos()
020 |     assert len(todos) >= 2
021 |
022 |
023 | def test_printer_service_buscar_por_patrimonio(db_session):
024 |     from app.services.printer_service import PrinterService
025 |     svc = PrinterService(db_session)
026 |     svc.criar(patrimonio="BUSCA01", modelo="Kyocera")
027 |     p = svc.buscar_por_patrimonio("BUSCA01")
028 |     assert p is not None
029 |     assert p.modelo == "Kyocera"
030 |
031 |
032 | def test_printer_service_atualizar(db_session):
033 |     from app.services.printer_service import PrinterService
034 |     svc = PrinterService(db_session)
035 |     p = svc.criar(patrimonio="UPD001", modelo="Xerox")
036 |     svc.atualizar(p, modelo="Xerox Altalink", local_atual="Filial B")
037 |     assert p.modelo == "Xerox Altalink"
038 |     assert p.local_atual == "Filial B"
039 |
040 |
041 | def test_printer_service_excluir(db_session):
042 |     from app.services.printer_service import PrinterService
043 |     svc = PrinterService(db_session)
044 |     p = svc.criar(patrimonio="DEL001", modelo="Samsung")
045 |     svc.excluir(p)
046 |     assert svc.buscar_por_patrimonio("DEL001") is None
047 |
048 |
049 | def test_printer_service_filtro(db_session):
050 |     from app.services.printer_service import PrinterService
051 |     svc = PrinterService(db_session)
052 |     svc.criar(patrimonio="FILTR001", modelo="HP")
053 |     svc.criar(patrimonio="FILTR002", modelo="Epson")
054 |     resultados = svc.listar_todos(filtro="FILTR0")
055 |     assert len(resultados) == 2
056 |     resultados = svc.listar_todos(filtro="XPT0")
057 |     assert len(resultados) == 0
058 |
059 |
060 | def test_company_service_criar(db_session):
061 |     from app.services.company_service import CompanyService
062 |     svc = CompanyService(db_session)
063 |     c = svc.criar(nome="Empresa Teste", cnpj="12.345.678/0001-00", tipo="matriz")
064 |     assert c.id is not None
065 |     assert c.nome == "Empresa Teste"

```

```

066 |
067 |
068 | def test_company_service_listar(db_session):
069 |     from app.services.company_service import CompanyService
070 |     svc = CompanyService(db_session)
071 |     svc.criar(nome="Empresa A")
072 |     svc.criar(nome="Empresa B")
073 |     assert len(svc.listar_todas()) >= 2
074 |
075 |
076 | def test_user_service_criar(db_session):
077 |     from app.services.user_service import UserService
078 |     svc = UserService(db_session)
079 |     u = svc.criar(nome="Usuário Teste", username="teste", email="teste@email.com", senha="123456", perfil="")
080 |     assert u.id is not None
081 |     assert u.perfil == "admin"
082 |     assert u.ativo is True
083 |
084 |
085 | def test_user_service_buscar(db_session):
086 |     from app.services.user_service import UserService
087 |     svc = UserService(db_session)
088 |     svc.criar(nome="Busca User", username="buscador", email="busca@email.com", senha="abc", perfil="tecnico")
089 |     u = svc.buscar_por_username_ou_email("buscador")
090 |     assert u is not None
091 |     assert u.nome == "Busca User"
092 |
093 |
094 | def test_part_service_criar(db_session):
095 |     from app.services.part_service import PartService
096 |     svc = PartService(db_session)
097 |     peca = svc.criar(codigo="TON001", nome="Toner HP 12A", quantidade=5)
098 |     assert peca.id is not None
099 |     assert peca.quantidade_estoque == 5
100 |
101 |
102 | def test_technician_service_criar(db_session):
103 |     from app.services.technician_service import TechnicianService
104 |     svc = TechnicianService(db_session)
105 |     t = svc.criar(nome_completo="João Técnico", telefone="81999999999")
106 |     assert t.id is not None
107 |     assert t.ativo is True
108 |
109 |
110 | def test_technician_service_listar_ativos(db_session):
111 |     from app.services.technician_service import TechnicianService
112 |     svc = TechnicianService(db_session)
113 |     svc.criar(nome_completo="Técnico Ativo")
114 |     t_inativo = svc.criar(nome_completo="Técnico Inativo")
115 |     t_inativo.ativo = False
116 |     safe_commit(db_session)
117 |     ativos = svc.listar_ativos()
118 |     assert all(t.ativo for t in ativos)
119 |
120 |
121 | def test_activity_service_criar(db_session):
122 |     from app.services.activity_service import ActivityService
123 |     from app.services.printer_service import PrinterService
124 |     p = PrinterService(db_session).criar(patrimonio="ACT001", modelo="HP")
125 |     svc = ActivityService(db_session)
126 |     a = svc.criar(printer_id=p.id, kind="MANUTENCAO", notes="Troca de toner")
127 |     assert a.id is not None
128 |     assert a.kind == "MANUTENCAO"

```

```

129 |
130 |
131 | def test_activity_service_listar_por_impessora(db_session):
132 |     from app.services.activity_service import ActivityService
133 |     from app.services.printer_service import PrinterService
134 |     p = PrinterService(db_session).criar(patrimonio="ACTLST", modelo="Epson")
135 |     svc = ActivityService(db_session)
136 |     svc.criar(printer_id=p.id, kind="MANUTENCAO")
137 |     svc.criar(printer_id=p.id, kind="MOVIMENTACAO")
138 |     atividades = svc.listar_por_impessora(p.id)
139 |     assert len(atividades) == 2
140 |
141 |
142 | def test_dashboard_service_resumo(db_session):
143 |     from app.services.dashboard_service import DashboardService
144 |     from app.services.printer_service import PrinterService
145 |     svc = PrinterService(db_session)
146 |     svc.criar(patrimonio="DASH01", modelo="HP", status="Operacional")
147 |     svc.criar(patrimonio="DASH02", modelo="Epson", status="Em manutenção")
148 |     dash = DashboardService(db_session)
149 |     r = dash.resumo()
150 |     assert r["total_impessoras"] >= 2
151 |     assert r["em_manutencao"] >= 1
152 |     assert r["operacionais"] >= 1
153 |
154 |
155 | def test_alert_service_criar(db_session):
156 |     from app.services.alert_service import AlertService
157 |     from app.services.printer_service import PrinterService
158 |     p = PrinterService(db_session).criar(patrimonio="ALERT01", modelo="HP")
159 |     svc = AlertService(db_session)
160 |     a = svc.criar(printer_id=p.id, tipo="revisao", titulo="Teste alerta")
161 |     assert a.id is not None
162 |     assert a.tipo == "revisao"
163 |     assert a.resolvido is False
164 |
165 |
166 | def test_alert_service_resolver(db_session):
167 |     from app.services.alert_service import AlertService
168 |     from app.services.printer_service import PrinterService
169 |     from app.services.user_service import UserService
170 |     p = PrinterService(db_session).criar(patrimonio="ALERT02", modelo="Epson")
171 |     usr = UserService(db_session).criar(nome="User", username="resolver", email="r@t.com", senha="123")
172 |     svc = AlertService(db_session)
173 |     a = svc.criar(printer_id=p.id, tipo="critico", titulo="Resolver")
174 |     svc.resolver(a, user_id=usr.id)
175 |     assert a.resolvido is True
176 |     assert a.resolvido_por == usr.id
177 |
178 |
179 | def test_alert_service_contar_pendentes(db_session):
180 |     from app.services.alert_service import AlertService
181 |     from app.services.printer_service import PrinterService
182 |     p = PrinterService(db_session).criar(patrimonio="ALERT03", modelo="Brother")
183 |     svc = AlertService(db_session)
184 |     svc.criar(printer_id=p.id, tipo="info", titulo="Pendente 1")
185 |     svc.criar(printer_id=p.id, tipo="info", titulo="Pendente 2")
186 |     a3 = svc.criar(printer_id=p.id, tipo="info", titulo="Resolvido 1")
187 |     svc.resolver(a3)
188 |     assert svc.contar_pendentes() == 2
189 |
190 |
191 | def test_alert_service_excluir(db_session):

```

```

192 | from app.services.alert_service import AlertService
193 | from app.services.printer_service import PrinterService
194 | p = PrinterService(db_session).criar(patrimonio="ALERT04", modelo="Xerox")
195 | svc = AlertService(db_session)
196 | a = svc.criar(printer_id=p.id, tipo="info", titulo="Excluir")
197 | svc.excluir(a)
198 | assert svc.buscar_por_id(a.id) is None
199 |
200 |
201 | def test_audit_service_log(db_session):
202 |     from app.services.audit_service import AuditService
203 |     svc = AuditService(db_session)
204 |     log = svc.log(user_id=1, acao="CRIAR", tabela_alvo="printers", registro_id="abc-123")
205 |     assert log.id is not None
206 |     assert log.acao == "CRIAR"
207 |     assert log.tabela_alvo == "printers"
208 |
209 |
210 | def test_audit_service_log_com_dados(db_session):
211 |     from app.services.audit_service import AuditService
212 |     svc = AuditService(db_session)
213 |     antes = {"status": "Operacional"}
214 |     depois = {"status": "Em manutenção"}
215 |     log = svc.log(user_id=1, acao="ATUALIZAR", tabela_alvo="printers",
216 |                  registro_id="abc", dados_antes=antes, dados_depois=depois)
217 |     assert log.dados_antes is not None
218 |     assert log.dados_depois is not None
219 |     import json
220 |     parsed = json.loads(log.dados_depois)
221 |     assert parsed["status"] == "Em manutenção"
222 |
223 |
224 | def test_audit_service_listar(db_session):
225 |     from app.services.audit_service import AuditService
226 |     svc = AuditService(db_session)
227 |     svc.log(user_id=1, acao="CRIAR", tabela_alvo="companies")
228 |     svc.log(user_id=1, acao="CRIAR", tabela_alvo="printers")
229 |     svc.log(user_id=2, acao="ATUALIZAR", tabela_alvo="printers")
230 |     assert len(svc.listar()) >= 3
231 |     assert len(svc.listar_por_tabela("printers")) >= 2
232 |     assert len(svc.listar_por_usuario(1)) >= 2
233 |
234 |
235 | # ████████████████████████████████████████████████████████████████████████████
236 | # Edge cases - PrinterService
237 | # ████████████████████████████████████████████████████████████████████████████
238 |
239 | def test_printer_criar_duplicate_patrimonio(db_session):
240 |     from app.services.printer_service import PrinterService
241 |     svc = PrinterService(db_session)
242 |     svc.criar(patrimonio="DUP001", modelo="HP")
243 |     with pytest.raises(Exception):
244 |         svc.criar(patrimonio="DUP001", modelo="Epson")
245 |
246 |
247 | def test_printer_criar_sem_patrimonio(db_session):
248 |     from app.services.printer_service import PrinterService
249 |     svc = PrinterService(db_session)
250 |     p = svc.criar(patrimonio="", modelo="HP")
251 |     assert p.id is not None
252 |
253 |
254 | def test_printer_criar_whitespace_patrimonio(db_session):

```

```

255 |     from app.services.printer_service import PrinterService
256 |     svc = PrinterService(db_session)
257 |     p = svc.criar(patrimonio=" ESPACO ", modelo="HP")
258 |     assert svc.buscar_por_patrimonio("ESPACO") is not None
259 |
260 |
261 | def test_printer_buscar_por_patrimonio_inexistente(db_session):
262 |     from app.services.printer_service import PrinterService
263 |     svc = PrinterService(db_session)
264 |     assert svc.buscar_por_patrimonio("NAO_EXISTE") is None
265 |
266 |
267 | def test_printer_buscar_por_patrimonio_vazio(db_session):
268 |     from app.services.printer_service import PrinterService
269 |     svc = PrinterService(db_session)
270 |     assert svc.buscar_por_patrimonio("") is None
271 |
272 |
273 | def test_printer_atualizar_com_none(db_session):
274 |     from app.services.printer_service import PrinterService
275 |     svc = PrinterService(db_session)
276 |     p = svc.criar(patrimonio="NONE001", modelo="HP")
277 |     svc.atualizar(p, proxima_revisao=None)
278 |     assert p.proxima_revisao is None
279 |
280 |
281 | def test_printer_excluir_duas_vezes(db_session):
282 |     from app.services.printer_service import PrinterService
283 |     svc = PrinterService(db_session)
284 |     p = svc.criar(patrimonio="DEL2X", modelo="HP")
285 |     svc.excluir(p)
286 |     assert svc.buscar_por_patrimonio("DEL2X") is None
287 |     svc.excluir(p)
288 |     assert svc.buscar_por_patrimonio("DEL2X") is None
289 |
290 |
291 | def test_printer_restaurar(db_session):
292 |     from app.services.printer_service import PrinterService
293 |     svc = PrinterService(db_session)
294 |     p = svc.criar(patrimonio="REST001", modelo="HP")
295 |     svc.excluir(p)
296 |     assert svc.buscar_por_patrimonio("REST001") is None
297 |     svc.restaurar(p)
298 |     assert svc.buscar_por_patrimonio("REST001") is not None
299 |
300 |
301 | def test_printer_restaurar_duas_vezes(db_session):
302 |     from app.services.printer_service import PrinterService
303 |     svc = PrinterService(db_session)
304 |     p = svc.criar(patrimonio="REST2X", modelo="HP")
305 |     svc.restaurar(p)
306 |     assert svc.buscar_por_patrimonio("REST2X") is not None
307 |     svc.restaurar(p)
308 |     assert svc.buscar_por_patrimonio("REST2X") is not None
309 |
310 |
311 | def test_printer_listar_com_filtro_vazio(db_session):
312 |     from app.services.printer_service import PrinterService
313 |     svc = PrinterService(db_session)
314 |     svc.criar(patrimonio="VZ01", modelo="HP")
315 |     todos = svc.listar_todos(filtro="")
316 |     assert len(todos) >= 1
317 |

```

```

318 | def test_printer_listar_com_filtro_muito_longo(db_session):
319 |     from app.services.printer_service import PrinterService
320 |     svc = PrinterService(db_session)
321 |     svc.criar(patrimonio="LGFLT", modelo="HP")
322 |     resultados = svc.listar_todos(filtro="a" * 500)
323 |     assert len(resultados) == 0
324 |
325 |
326 |
327 | def test_printer_listar_excluidos_vazio(db_session):
328 |     from app.services.printer_service import PrinterService
329 |     svc = PrinterService(db_session)
330 |     excluidos = svc.listar_excluidos()
331 |     assert isinstance(excluidos, list)
332 |
333 |
334 | def test_printer_listar_excluidos_com_restaurados(db_session):
335 |     from app.services.printer_service import PrinterService
336 |     svc = PrinterService(db_session)
337 |     p1 = svc.criar(patrimonio="EXCLST1", modelo="HP")
338 |     p2 = svc.criar(patrimonio="EXCLST2", modelo="Epson")
339 |     svc.excluir(p1)
340 |     svc.excluir(p2)
341 |     svc.restaurar(p1)
342 |     excluidos = svc.listar_excluidos()
343 |     pats = [e.patrimonio for e in excluidos]
344 |     assert "EXCLST2" in pats
345 |     assert "EXCLST1" not in pats
346 |
347 |
348 | def test_printer_contar_todos_sem_filtro(db_session):
349 |     from app.services.printer_service import PrinterService
350 |     svc = PrinterService(db_session)
351 |     svc.criar(patrimonio="CNT01", modelo="HP")
352 |     total = svc.contar_todos()
353 |     assert total >= 1
354 |
355 |
356 | # ████████████████████████████████████████████████████████████████████████████
357 | # Edge cases - CompanyService
358 | # ████████████████████████████████████████████████████████████████████████████
359 |
360 | def test_company_criar_sem_nome(db_session):
361 |     from app.services.company_service import CompanyService
362 |     svc = CompanyService(db_session)
363 |     c = svc.criar(nome="", cnpj="")
364 |     assert c.id is not None
365 |
366 |
367 | def test_company_criar_duplicate_nome(db_session):
368 |     from app.services.company_service import CompanyService
369 |     svc = CompanyService(db_session)
370 |     c1 = svc.criar(nome="Empresa Unica")
371 |     c2 = svc.criar(nome="Empresa Unica")
372 |     assert c1.id != c2.id
373 |
374 |
375 | def test_company_buscar_por_nome_inexistente(db_session):
376 |     from app.services.company_service import CompanyService
377 |     svc = CompanyService(db_session)
378 |     assert svc.buscar_por_nome("NAO_EXISTE") is None
379 |
380 |

```

```

381 | def test_company_listar_nomes_sem_empresas(db_session):
382 |     from app.services.company_service import CompanyService
383 |     svc = CompanyService(db_session)
384 |     nomes = svc.listar_nomes()
385 |     assert isinstance(nomes, list)
386 |
387 |
388 | # ████████████████████████████████████████████████████████████████████████████
389 | # Edge cases - UserService
390 | # ████████████████████████████████████████████████████████████████████████████
391 |
392 | def test_user_criar_username_repetido(db_session):
393 |     from app.services.user_service import UserService
394 |     svc = UserService(db_session)
395 |     u1 = svc.criar(nome="User1", username="repetido", email="u1@t.com", senha="123", perfil="tecnico")
396 |     u2 = svc.criar(nome="User2", username="repetido", email="u2@t.com", senha="456", perfil="admin")
397 |     assert u1.id != u2.id
398 |
399 |
400 | def test_user_criar_email_repetido(db_session):
401 |     from app.services.user_service import UserService
402 |     svc = UserService(db_session)
403 |     svc.criar(nome="User1", username="user1", email="dup@t.com", senha="123", perfil="tecnico")
404 |     with pytest.raises(Exception):
405 |         svc.criar(nome="User2", username="user2", email="dup@t.com", senha="456", perfil="admin")
406 |
407 |
408 | def test_user_criar_sem_senha(db_session):
409 |     from app.services.user_service import UserService
410 |     svc = UserService(db_session)
411 |     u = svc.criar(nome="Sem Senha", username="sem_senha", email="ss@t.com", senha="", perfil="tecnico")
412 |     assert u.id is not None
413 |
414 |
415 | def test_user_buscar_por_username_inexistente(db_session):
416 |     from app.services.user_service import UserService
417 |     svc = UserService(db_session)
418 |     assert svc.buscar_por_username_ou_email("nao_existe") is None
419 |
420 |
421 | def test_user_verificar_existente_username(db_session):
422 |     from app.services.user_service import UserService
423 |     svc = UserService(db_session)
424 |     svc.criar(nome="Existente", username="existe", email="ex@t.com", senha="123", perfil="tecnico")
425 |     resultado = svc.verificar_existente(email="ex@t.com", username="existe")
426 |     assert resultado is not None
427 |     assert resultado.username == "existe"
428 |
429 |
430 | def test_user_verificar_existente_nonexistent(db_session):
431 |     from app.services.user_service import UserService
432 |     svc = UserService(db_session)
433 |     resultado = svc.verificar_existente(email="nao@t.com", username="nao_existe")
434 |     assert resultado is None
435 |
436 |
437 | # ████████████████████████████████████████████████████████████████████████████
438 | # Edge cases - PartService
439 | # ████████████████████████████████████████████████████████████████████████████
440 |
441 | def test_part_criar_com_estoque_negativo(db_session):
442 |     from app.services.part_service import PartService
443 |     svc = PartService(db_session)

```


[illegible]

```

507 | # Edge cases - ActivityService
508 | # ████████████████████████████████████████████████████████████████████████████
509 |
510 | def test_activity_listar_por_impressora_inexistente(db_session):
511 |     from app.services.activity_service import ActivityService
512 |     svc = ActivityService(db_session)
513 |     assert svc.listar_por_impressora(99999) == []
514 |
515 |
516 | def test_activity_criar_com_kind_vazio(db_session):
517 |     from app.services.activity_service import ActivityService
518 |     from app.services.printer_service import PrinterService
519 |     p = PrinterService(db_session).criar(patrimonio="ACTKIND", modelo="HP")
520 |     svc = ActivityService(db_session)
521 |     a = svc.criar(printer_id=p.id, kind="", notes="")
522 |     assert a.id is not None
523 |
524 |
525 | def test_activity_excluir_e_restaurar(db_session):
526 |     from app.services.activity_service import ActivityService
527 |     from app.services.printer_service import PrinterService
528 |     p = PrinterService(db_session).criar(patrimonio="ACTRST", modelo="HP")
529 |     svc = ActivityService(db_session)
530 |     a = svc.criar(printer_id=p.id, kind="MANUTENCAO")
531 |     svc.excluir(a)
532 |     assert svc.buscar_por_id(a.id) is None
533 |     svc.restaurar(a)
534 |     assert svc.buscar_por_id(a.id) is not None
535 |
536 |
537 | def test_activity_contadores_ignoram_soft_deleted(db_session):
538 |     from app.services.activity_service import ActivityService
539 |     from app.services.printer_service import PrinterService
540 |     p = PrinterService(db_session).criar(patrimonio="CTSD01", modelo="HP")
541 |     svc = ActivityService(db_session)
542 |     a1 = svc.criar(printer_id=p.id, kind="MANUTENCAO", status_atividade="Aberta")
543 |     a2 = svc.criar(printer_id=p.id, kind="MANUTENCAO", status_atividade="Aberta")
544 |     svc.excluir(a1)
545 |     assert svc.contar_por_status("Aberta") == 1
546 |     if a2.tecnico_id:
547 |         assert svc.contar_por_tecnico(a2.tecnico_id) == 1
548 |
549 |
550 | def test_part_retirar_estoque_por_nome(db_session):
551 |     from app.services.part_service import PartService
552 |     svc = PartService(db_session)
553 |     p = svc.criar(nome="Tonner HP", codigo="T001", quantidade=5)
554 |     assert p.quantidade_estoque == 5
555 |     svc.retirar_estoque_por_nome("Tonner HP, Fusor")
556 |     assert svc.buscar_por_nome("Tonner HP").quantidade_estoque == 4
557 |
558 |
559 | # ████████████████████████████████████████████████████████████████████████████
560 | # Edge cases - AlertService
561 | # ████████████████████████████████████████████████████████████████████████████
562 |
563 | def test_alert_criar_semTitulo(db_session):
564 |     from app.services.alert_service import AlertService
565 |     from app.services.printer_service import PrinterService
566 |     p = PrinterService(db_session).criar(patrimonio="ALTTL", modelo="HP")
567 |     svc = AlertService(db_session)
568 |     a = svc.criar(printer_id=p.id, tipo="info", titulo="")
569 |     assert a.id is not None

```

```

570 |
571 |
572 | def test_alert_criar_sem_printer(db_session):
573 |     from app.services.alert_service import AlertService
574 |     svc = AlertService(db_session)
575 |     a = svc.criar(printer_id=None, tipo="info", titulo="Sem impressora")
576 |     assert a.id is not None
577 |     assert a.printer_id is None
578 |
579 |
580 | def test_alert_resolver_ja_resolvido(db_session):
581 |     from app.services.alert_service import AlertService
582 |     from app.services.printer_service import PrinterService
583 |     p = PrinterService(db_session).criar(patrimonio="ALRST", modelo="HP")
584 |     svc = AlertService(db_session)
585 |     a = svc.criar(printer_id=p.id, tipo="info", titulo="Ja resolvido")
586 |     svc.resolver(a)
587 |     svc.resolver(a)
588 |     assert a.resolvido is True
589 |     assert a.resolvido_em is not None
590 |
591 |
592 | # ████████████████████████████████████████████████████████████████████████████
593 | # Edge cases - AuditService
594 | # ████████████████████████████████████████████████████████████████████████████
595 |
596 | def test_audit_log_sem_usuario(db_session):
597 |     from app.services.audit_service import AuditService
598 |     svc = AuditService(db_session)
599 |     log = svc.log(user_id=None, acao="TESTE", tabela_alvo="printers")
600 |     assert log.id is not None
601 |
602 |
603 | def test_audit_listar_por_usuario_sem_logs(db_session):
604 |     from app.services.audit_service import AuditService
605 |     svc = AuditService(db_session)
606 |     assert svc.listar_por_usuario(99999) == []
607 |
608 |
609 | def test_audit_listar_por_tabela_sem_logs(db_session):
610 |     from app.services.audit_service import AuditService
611 |     svc = AuditService(db_session)
612 |     assert svc.listar_por_tabela("inexistente") == []
613 |
614 |
615 | # ████████████████████████████████████████████████████████████████████████████
616 | # Edge cases - DashboardService
617 | # ████████████████████████████████████████████████████████████████████████████
618 |
619 | def test_dashboard_resumo_sem_impressoras(db_session):
620 |     from app.services.dashboard_service import DashboardService
621 |     svc = DashboardService(db_session)
622 |     r = svc.resumo()
623 |     assert r["total_impressoras"] == 0
624 |     assert r["em_manutencao"] == 0
625 |     assert r["operacionais"] == 0
626 |
627 |
628 | def test_dashboard_grafico_pizza_sem_dados(db_session):
629 |     from app.services.dashboard_service import DashboardService
630 |     svc = DashboardService(db_session)
631 |     labels, valores = svc.dados_grafico_pizza()
632 |     assert isinstance(labels, list)

```

```

633 |         assert isinstance(valores, list)
634 |
635 |
636 | def test_dashboard_grafico_atividades_sem_dados(db_session):
637 |     from app.services.dashboard_service import DashboardService
638 |     svc = DashboardService(db_session)
639 |     meses, valores = svc.dados_grafico_atividades()
640 |     assert isinstance(meses, list)
641 |     assert isinstance(valores, list)
642 |
643 |
644 | def test_dashboard_grafico_alertas_sem_dados(db_session):
645 |     from app.services.dashboard_service import DashboardService
646 |     svc = DashboardService(db_session)
647 |     dias, criados = svc.dados_grafico_alertas()
648 |     assert isinstance(dias, list)
649 |     assert isinstance(criados, list)

```

=== ARQUIVO: .\tests\test_utils.py ===

[illegible]

```
042 |  
043 | def test_truncar_none():  
044 |     assert truncar(None, 10) == ""  
045 |  
046 |  
047 | def test_truncar_whitespace():  
048 |     assert truncar(" ", 10) == ""  
049 |  
050 |  
051 | def test_truncar_shorter_than_max():  
052 |     assert truncar("abc", 10) == "abc"  
053 |  
054 |  
055 | def test_truncar_exact_match():  
056 |     assert truncar("12345", 5) == "12345"  
057 |  
058 |  
059 | def test_truncar_longer():  
060 |     assert truncar("abcdefghij", 5) == "abcde"  
061 |  
062 |  
063 | # ■■ limpar_local ████████████████████████████████████████████████████████████  
064 |  
065 | def test_limpar_local_none():  
066 |     assert limpar_local(None) == ""  
067 |  
068 |  
069 | def test_limpar_local_empty():  
070 |     assert limpar_local("") == ""  
071 |  
072 |  
073 | def test_limpar_local_no_prefix():  
074 |     assert limpar_local("Matriz") == "Matriz"  
075 |  
076 |  
077 | def test_limpar_local_com_prefixo_predio():  
078 |     assert limpar_local("\U0001f3e2 Filial A") == "Filial A"  
079 |  
080 |  
081 | def test_limpar_local_com_prefixo_pino():  
082 |     assert limpar_local("\U0001f4cd Sala 3") == "Sala 3"  
083 |  
084 |  
085 | def test_limpar_local_apenas_prefixo():  
086 |     assert limpar_local("\U0001f3e2 ") == ""  
087 |  
088 |  
089 | # ■■ formatar_data ████████████████████████████████████████████████████████████  
090 |  
091 | def test_formatar_data_none():  
092 |     assert formatar_data(None) == "-."  
093 |  
094 |  
095 | def test_formatar_data_valida():  
096 |     d = datetime(2025, 12, 25)  
097 |     assert formatar_data(d) == "25/12/2025"  
098 |  
099 |  
100 | def test_formatar_data_formato_personalizado():  
101 |     d = datetime(2025, 1, 1)  
102 |     assert formatar_data(d, "%Y-%m-%d") == "2025-01-01"  
103 |  
104 |
```

[illegible]

[illegible]

```
231 |  
232 |  
233 | def test_merge_modelos_ambos():  
234 |     assert merge_modelos("HP P1102", "M1132") == "HP P1102/M1132"  
235 |  
236 |  
237 | def test_merge_modelos_duplicado():  
238 |     assert merge_modelos("HP P1102/M1132", "M1132/HP 2035") == "HP P1102/M1132/HP 2035"  
239 |  
240 |  
241 | def test_merge_modelos_existente_vazio():  
242 |     assert merge_modelos("", "HP P1102") == "HP P1102"  
243 |  
244 |  
245 | def test_merge_modelos_novos_vazio():  
246 |     assert merge_modelos("HP P1102", "") == "HP P1102"
```

=== ARQUIVO: .\tests__init__.py ===